



福昕保护大师

全新智能防护·保护无处不在

<http://bh.pdf365.cn/>

PDF阅读器版本过低将导致无法正常浏览，请下载最新[福昕PDF阅读器](#)
或adobePDF阅读器



未经作者允许禁止发布
封面自主设计，禁止盗用

Windows内核安全 编程技术实践 (第一版)

—— 探索 Anti Rootkit 反内核工具核心原理与技术实现细节

著作权 王瑞 (lyshark.com)



◆方法独特，提炼核心技术

◆以实战角度出发，学以致用

◆图文并茂，涵盖ARK核心实现细节

前言



本书作者：王瑞 (LyShark)

作者邮箱：me@lyshark.com

作者博客：<https://lyshark.cnblogs.com>

关于作者

野路子出道，自幼热爱计算机技术，尤其喜爱二进制安全方向，并长期致力于信息安全技术研究，自学钻研计算机技术十余年，期间没有任何人给予过任何帮助，遇到问题只能自己想办法，人最困难的时候并不是在学校求学的时光，而是在出现问题时，回头却发现身后空无一人，LyShark知道初学者的困境，并能够感同身受，发布本书只是为了无私帮助更多人，所谓前人栽树后人乘凉，知识需要积累与总结，希望你也能够将所学所想，继续传递下去。

书籍内容简介

这是一本 windows 10 x64 内核安全开发系列丛书，暂且就叫它《windows 内核安全编程技术实践》吧，本书是 LyShark 多年的技术积累编写而成，不同于市面上的多数内核开发系列丛书，本书是以底层安全角度为切入点忽略了驱动开发中项目实践部分，LyShark 发现多数丛书都会携带太多的技术概念，这些概念并不利于技术实践，本书将忽略太多没有意义的专业术语，所有文章均以实战角度出发，由简入深递进式教学，通过学习本书你可掌握反内核工具是如何实现的，这些技术细节相信市面上你绝对学不到，或者找到的都是过时的，LyShark 追求高质量文章，保证每一篇文章都是可被直接应用。

书籍撰写初衷

某一天，笔者想要实现一款 ARK 反内核工具，在找资料时发现市面上多数 ARK 工具都经过了 VMP 高强度加密，内核代码尤为宝贵这一点可以被理解，多数有源码的项目也都是过时的无法正常使用，故想要将 ARK 反内核工具功能在最新版本的 windows 10 x64 系统上面实现，既可以整理归纳自己的知识体系，也可以帮助更多底层爱好者学习内核开发技术，让更多安全爱好者从中受益。

警告

本书内容首发于博客园，根据《中华人民共和国著作权法》相关规定本人享有著作权，本书的发放仅用于技术交流学习，本书内容均为免费开放，禁止第三方倒卖，如果您在第三方购买到本书请与作者联系，作者 (LyShark) 将追究倒卖者法律责任，纯净的技术交流需要你我共同来维护，感谢您阅读并支持作者的创作。

书籍目录

- 第一章：环境配置篇
 - 1.1 WDK8.1 驱动开发环境配置
 - 1.2 WinDBG 配置内核双机调试
 - 1.3 内核测试模式过DSE签名
- 第二章：基础知识篇
 - 2.1 内核中的链表与结构体
 - 2.2 内核中的自旋锁结构
 - 2.3 内核字符串转换方法
 - 2.4 内核字符串拷贝与比较
- 第三章：驱动通信篇
 - 3.1 驱动与应用的简单通信
 - 3.2 应用DeviceIoControl开发模板
 - 3.3 通过SystemBuf与内核层通信
 - 3.4 通过ReadFile与内核层通信
 - 3.5 通过PIPE管道与内核层通信
 - 3.6 通过Async反向与内核通信
- 第四章：驱动读写篇
 - 4.1 内核CR3切换读写内存
 - 4.2 内核MDL读写进程内存
 - 4.3 通过内存拷贝读写内存
 - 4.4 内核R3与R0内存映射拷贝
- 第五章：内核SSDT解析篇
 - 5.1 内核枚举SSDT表基址
 - 5.2 枚举ShadowSSDT表基址
- 第六章：内核进程线程篇
 - 6.1 内核中枚举进程与模块
 - 6.2 监控进程与线程对象操作
 - 6.3 内核监控进程与线程创建
 - 6.4 内核DKOM实现进程隐藏
 - 6.5 内核中实现Dump进程转储
 - 6.6 内核遍历进程VAD结构体
 - 6.7 运用VAD隐藏R3内存思路
 - 6.8 内核摘链DKOM进程隐藏
 - 6.9 内核无痕隐藏自身分析
 - 6.10 内核强制结束进程运行
- 第七章：内核模块篇
 - 7.1 内核判断驱动加载状态
 - 7.2 内核取ntoskrnl模块基址
 - 7.3 内核取应用层模块基址
 - 7.4 内核通过PEB得到进程参数
 - 7.5 断链隐藏驱动程序自身

- 7.6 内核特征码搜索函数封装
- 7.7 内核LDE64引擎计算汇编长度
- 7.8 内核层InlineHook挂钩函数
- 第八章：内核枚举篇
 - 8.1 内核枚举IoTimer定时器
 - 8.2 内核枚举DpcTimer定时器
 - 8.3 内核枚举PspCidTable句柄表
 - 8.4 内核枚举Minifilter微过滤驱动
 - 8.5 内核特征码扫描PE代码段
 - 8.6 内核枚举LoadImage映像回调
 - 8.7 内核枚举Registry注册表回调
 - 8.8 内核枚举进程与线程ObCall回调
- 第九章：内核监控篇
 - 9.1 内核监控进程与线程回调
 - 9.2 内核注册并监控对象回调
 - 9.3 内核监视LoadImage映像回调
 - 9.4 内核运用LoadImage屏蔽驱动
 - 9.5 内核监控Register注册表回调
 - 9.6 内核监控FileObject文件回调
- 第十章：其他篇
 - 10.1 内核封装WSK网络通信接口
 - 10.2 内核封装TDI网络通信接口

第一章：环境配置篇

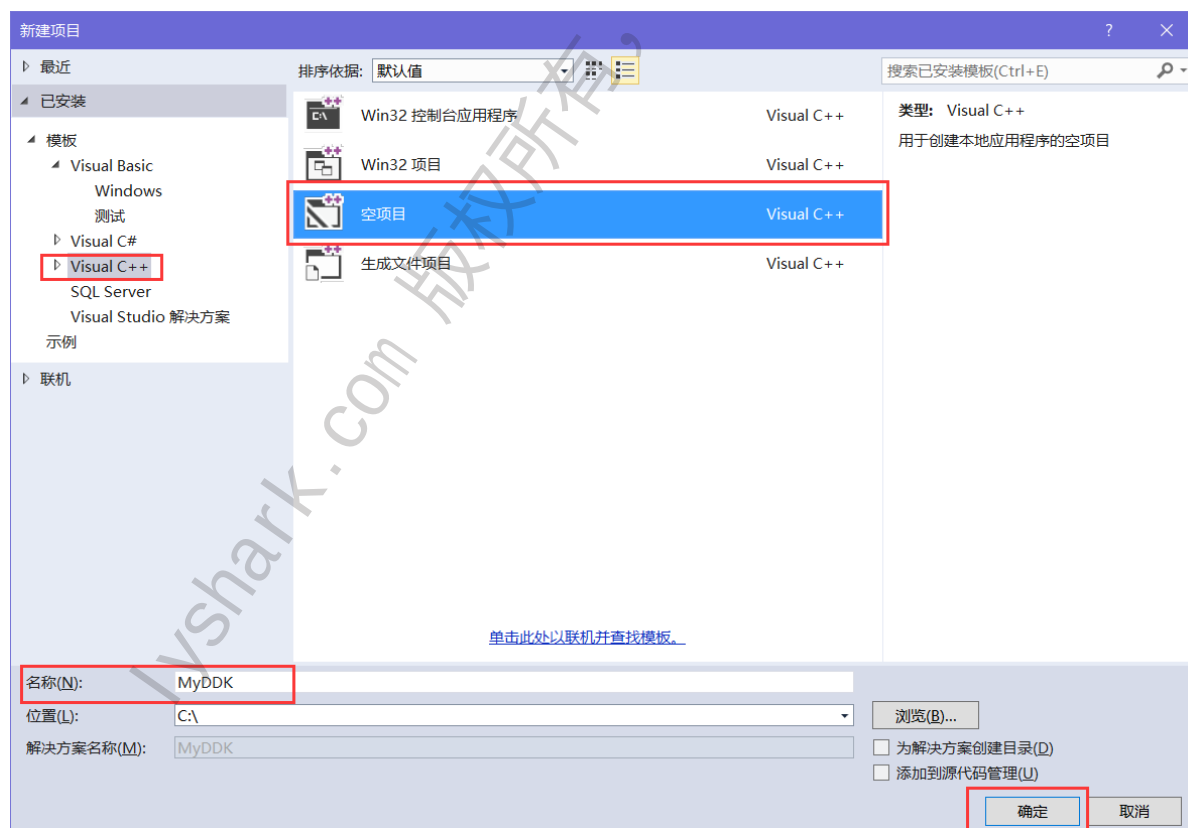
1.1 WDK8.1 驱动开发环境配置

Windows Driver Kit 是一种完全集成的驱动程序开发工具包，它包含 WinDDK 用于测试 Windows 驱动器的可靠性和稳定性，本次实验使用的是 WDK8.1 驱动开发工具包，该工具包支持 Windows 7到 Windows 10 系统的驱动开发。

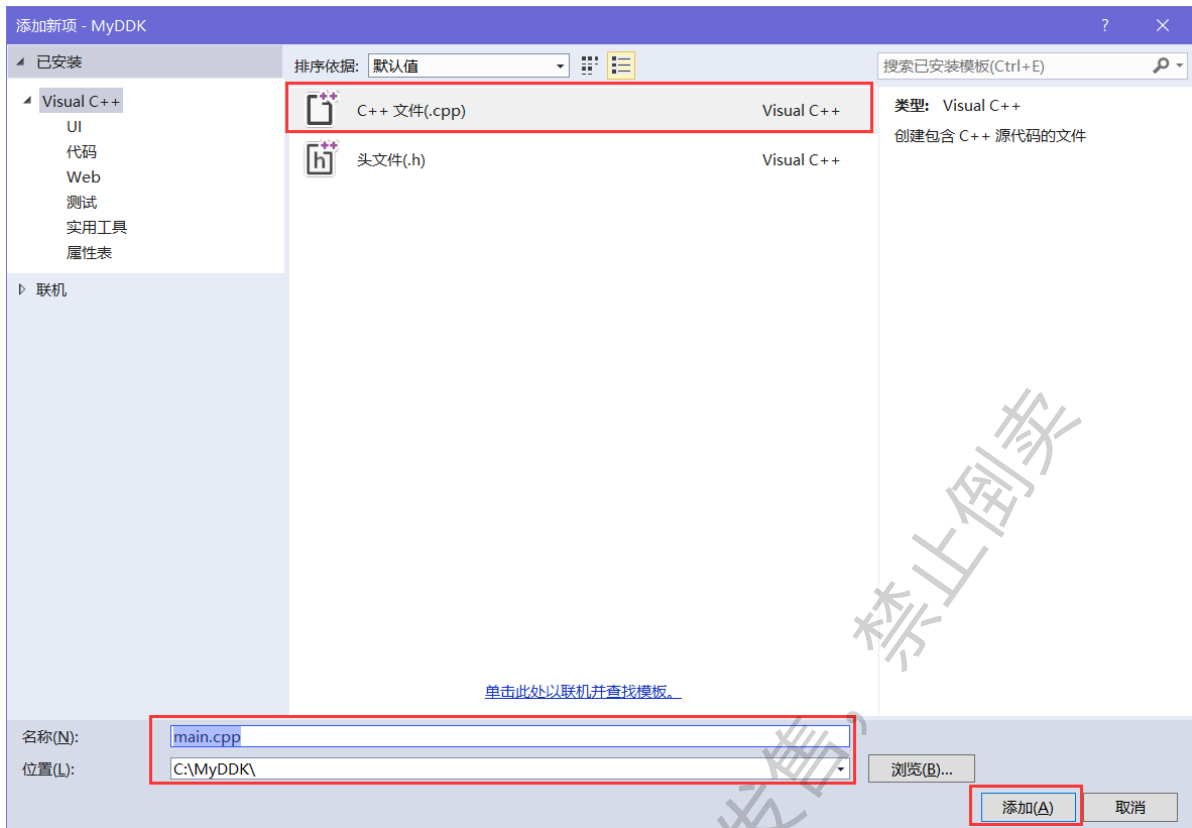
- 驱动WDK工具包推荐: Windows Driver Kit(WDK) v8.1 离线安装包
- 配置好的案例下载: <https://cdn.lyshark.com/code/WinDDK.zip>

首先你需要先安装好 Visual Studio 2013 的开发环境，然后再安装 Windows Driver Kit 8.1 的驱动开发工具包，这个工具包安装好以后1.5G左右，不过我已经把这个安装包中的关键库文件提取出来了，提取出的文件只有 80MB，直接将提取好的 winDDK.zip 解压缩到指定文件中，并配置环境即可使用，省去了安装WDK的麻烦。

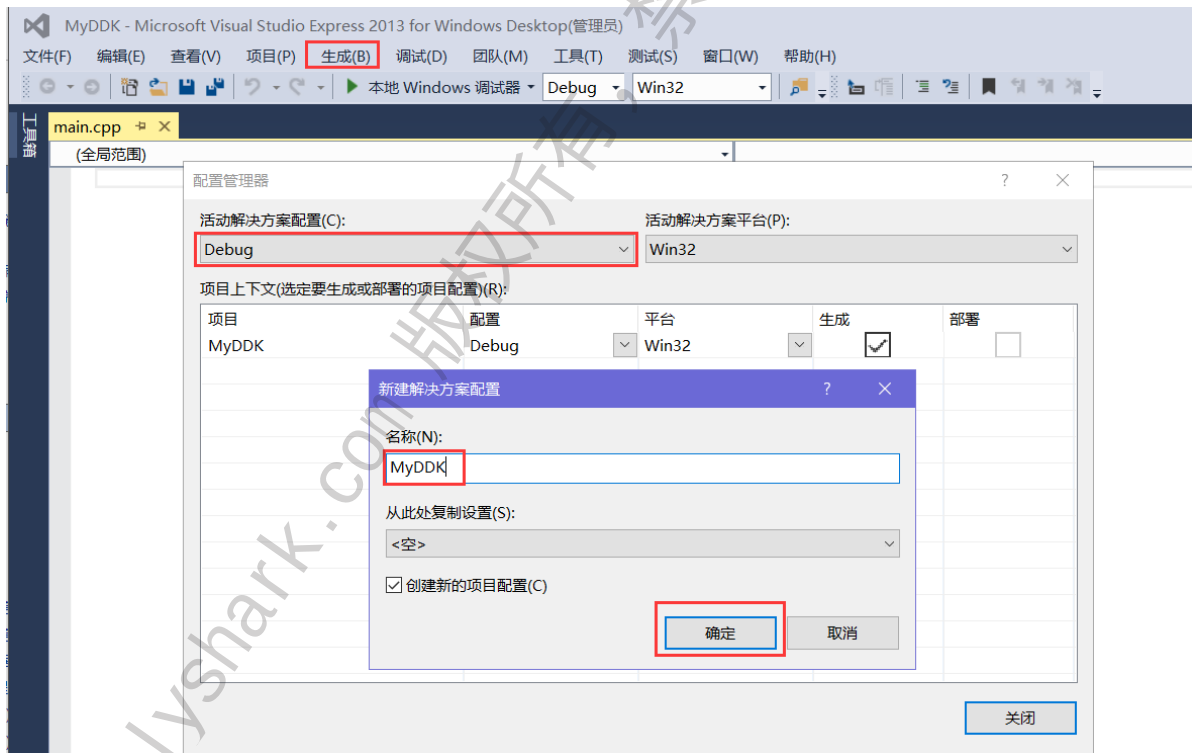
1.这里直接把我提取的 winDDK.zip 文件解压缩到C盘根目录下，然后打开 Visual Studio 开发工具，按下 Ctrl+Shift+N 新建空项目并输入项目名称为 MyDDK 即可。



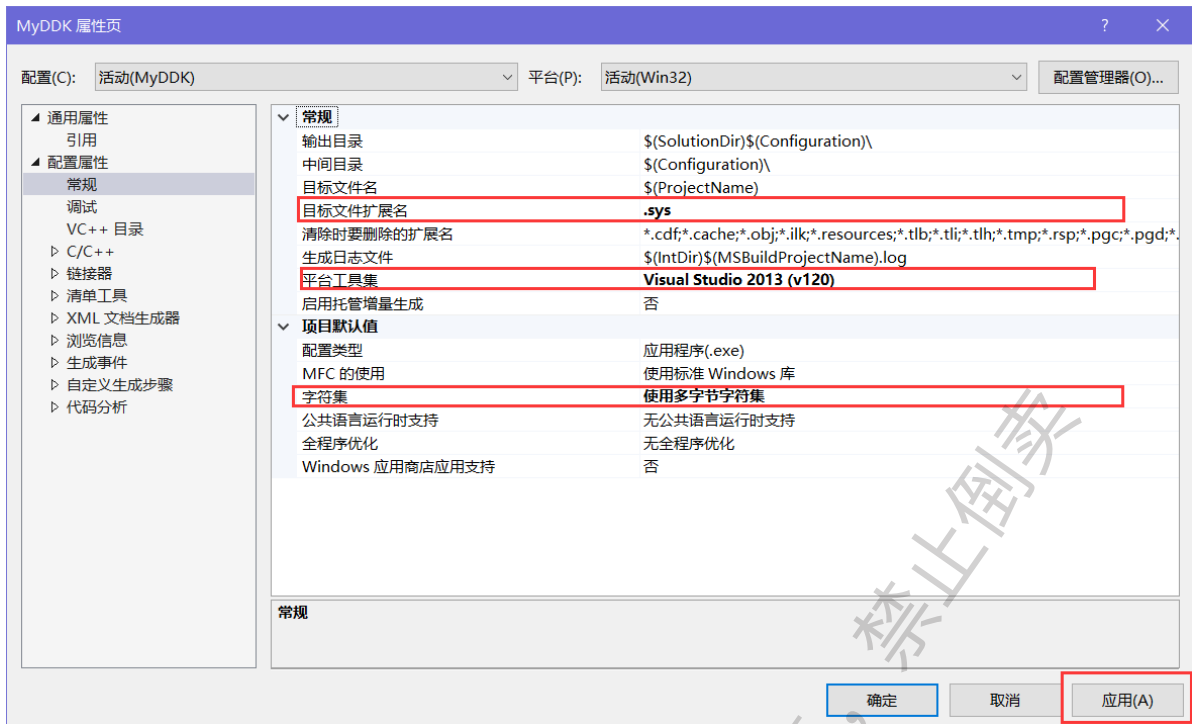
2.依次选择解决方案视图 -> 源文件 -> 添加新建项，或者直接按下 Ctrl + Shift + A 快捷打开菜单，并创建 main.cpp 文件。



3.接着我们需要修改一下配置管理器，添加自定义配置管理，选择生成->配置管理器->新建，此处我们命名为 MyDDK 即可。



4.接着修改一下配置属性中的常规属性，点击菜单栏中的调试，选择 MyDDK属性->配置->常规->修改以下几处。

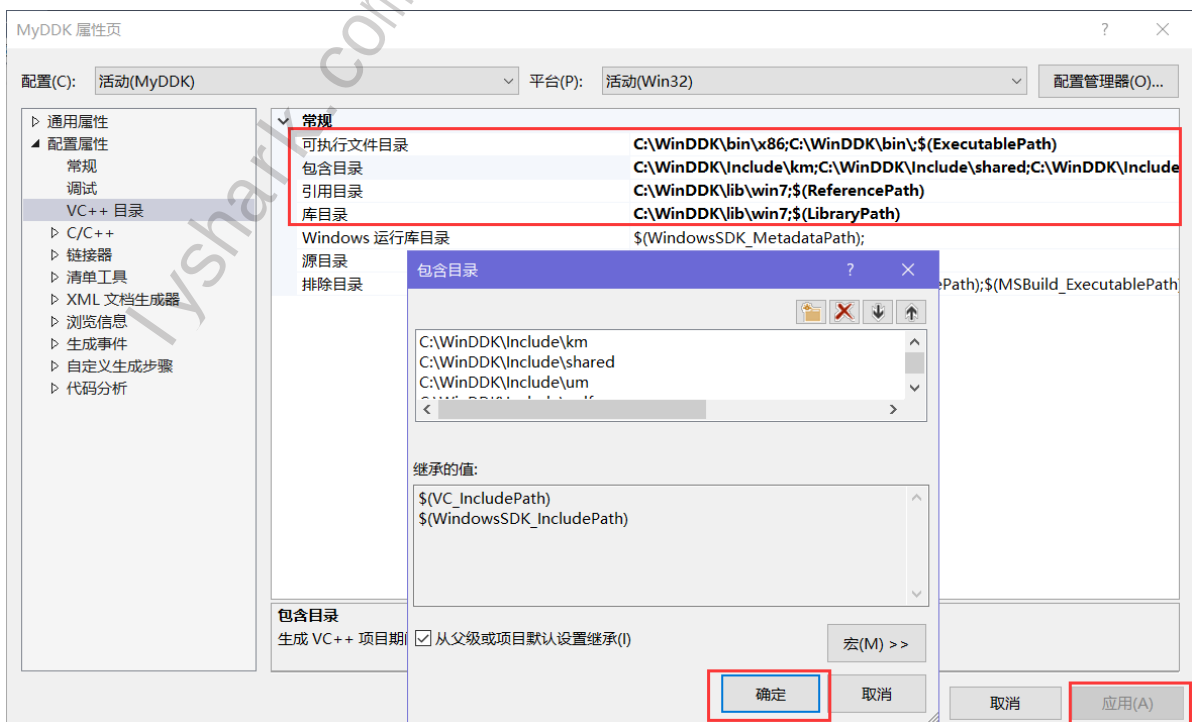


5.配置可执行文件路径与导入库路径，这里我们选择 配置属性 -> VC++目录 依次将如下信息填入配置项，如果需要编译x64位驱动只需要将x86改为x64即可，此处以x86为例。

可执行文件：
C:\winDDK\bin
C:\winDDK\bin\x86

包含目录：
C:\winDDK\Include\km
C:\winDDK\Include\um
C:\winDDK\Include\winrt
C:\winDDK\Include\shared

引用目录+库目录
C:\winDDK\Lib\win7\km\x86



6.配置 C/C++ 优化选项 配置属性 -> C/C++ -> 所有选项 -> 依次修改下方几个关键处，如果你需要编译X64 驱动需将调用约定改为 `__fastcall (/Gr)` 然后将预处理器定义中的 `_X86_` 改为 `_AMD64_` 即可编译64位 驱动了，此处以X86配置为例。

安全检查：禁用安全检查 (`/GS-`)

将警告视为错误：是 (`/WX`)

警告等级：等级 3 (`/W3`)

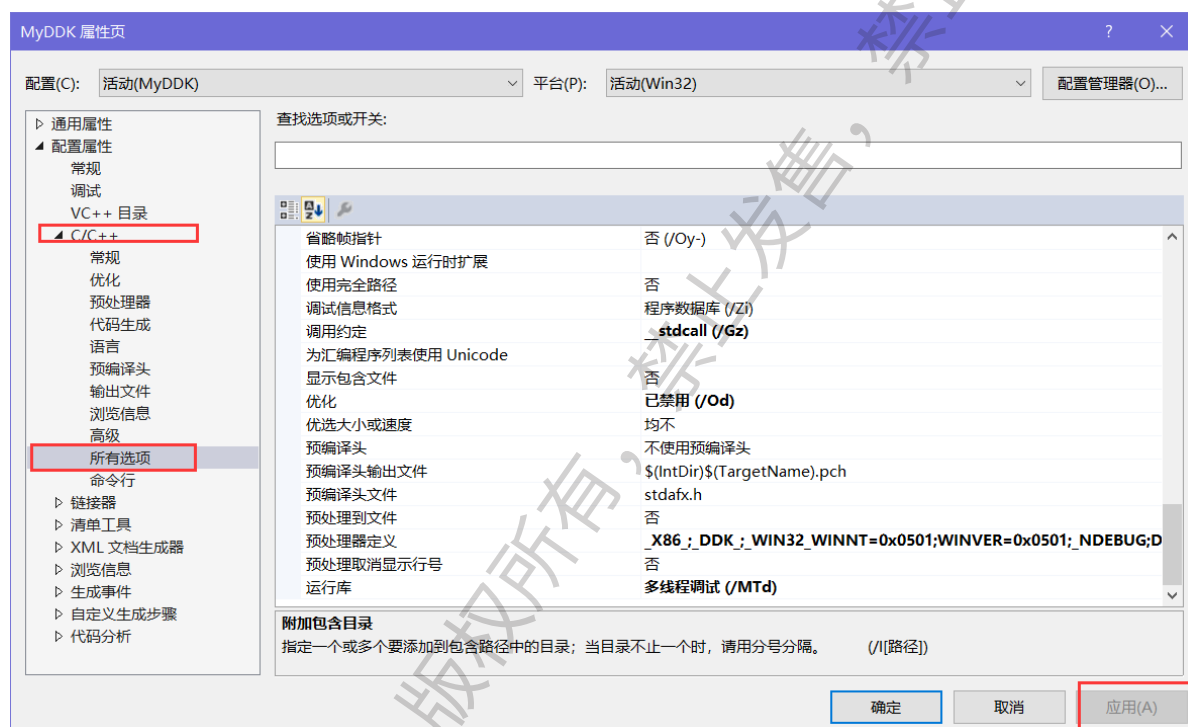
启用C++异常：否

调用约定： `__stdcall (/Gz)`

优化：已禁用 (`/Od`)

运行库：多线程调试 (`/MTd`)

预处理器定义： `_X86_;_DDK_;WIN32_WINNT=0x0501;WINVER=0x0501;_NDEBUG;DBG=0;%`
(PreprocessorDefinitions)



7.接着需要 配置连接器 选项，选择连接器 -> 所有选项 -> 依次修改下方几个关键处。

附加选项： `/IGNORE:4078 /safeseh:no`

附加依赖项： `ntoskrnl.lib;ndis.lib;Hal.lib;wdm.lib;wdmsec.lib;wmilib.lib`

固定基址：此处需要为空

忽略所有默认库：是 (`/NODEFAULTLIB`)

启用增量链接：否 (`/INCREMENTAL:NO`)

驱动程序：驱动程序 (`/Driver`)

入口点： `DriverEntry`

生成清单：否 (`/MANIFEST:NO`)

生成调试信息：是 (`/DEBUG`)

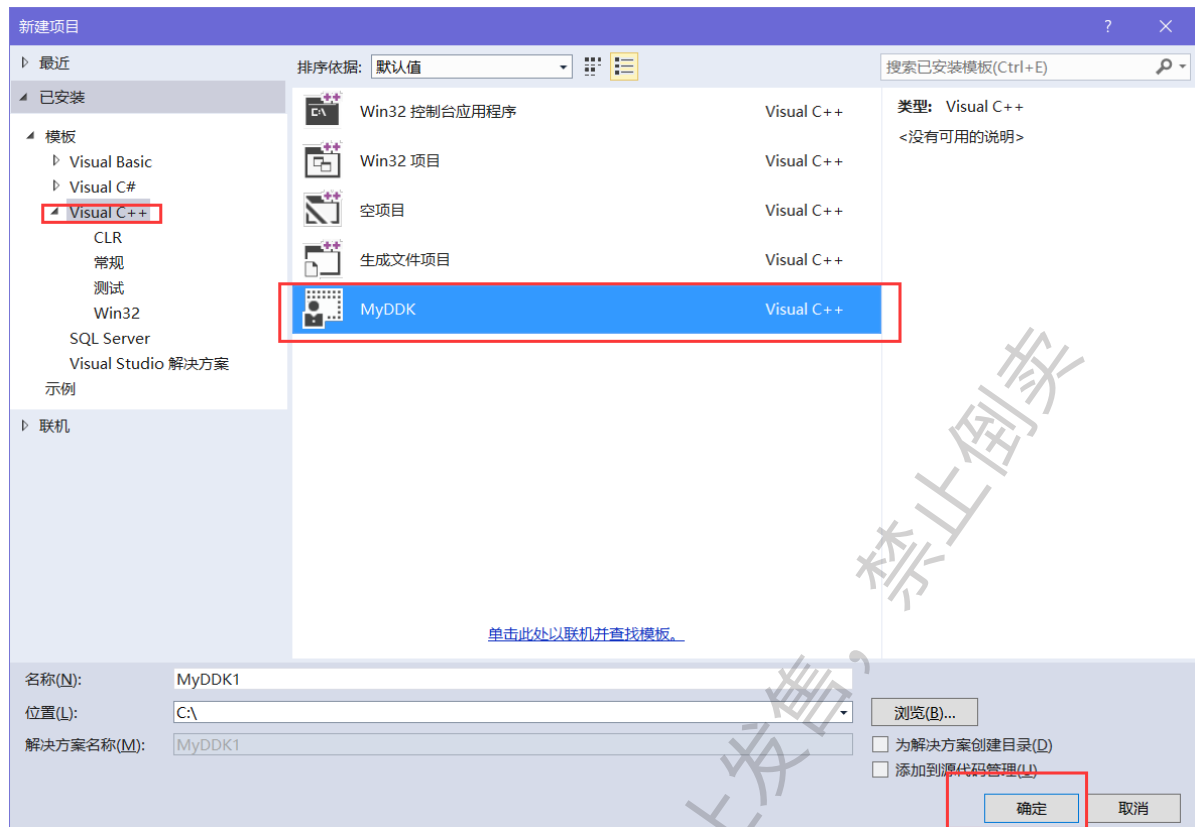
生成映射文件：是 (`/MAP`)

数据执行保护：是 (`/NXCOMPAT`)

随机基址：此处需要清空

子系统：本机 (`/SUBSYSTEM:NATIVE`)

此时关闭VS工具，再次打开，就能直接使用我们的模板来开发驱动了，不需要每次都配置。



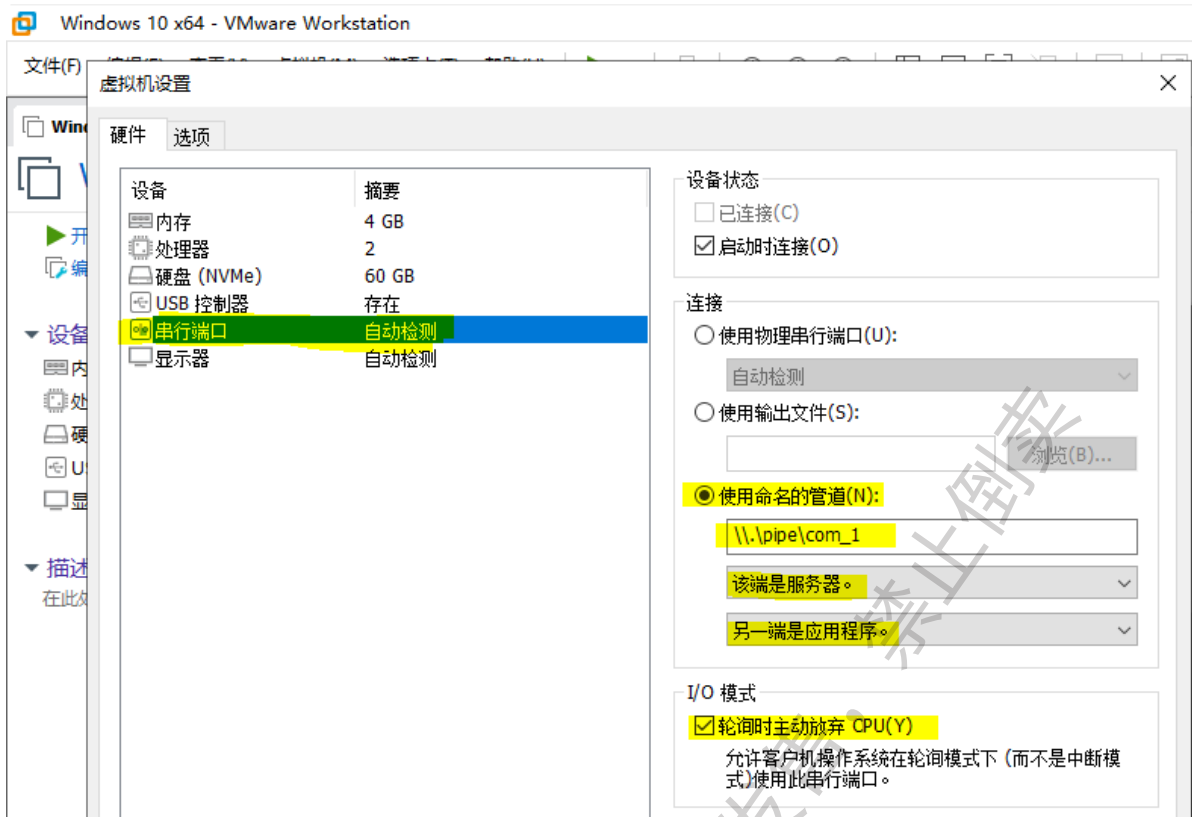
1.2 WinDBG 配置内核双机调试

WinDBG 是在 windows 平台下，强大的用户态和内核态调试工具，相比较于 Visual Studio 它是一个轻量级的调试工具，所谓轻量级指的是它的安装文件大小较小，但是其调试功能却比 VS 更为强大，WinDBG 由于是微软的产品所以能够调试 windows 系统的内核，另外一个用途是用来分析 dump 数据，本笔记用于记录如何开启 windows 系统内核调试功能，并使用 WinDBG 调试驱动。

1. 首先需要安装 vmware 虚拟机，并自行安装好 windows 10 系统，虚拟机关闭状态下添加一个管道虚拟串口，此处需要删除打印机，否则串口之间冲突。

操作步骤：编辑虚拟机设置 -> 添加 -> 串行端口 -> 完成

参数配置：使用命名管道 -> \\.\pipe\com_1 -> 该端是服务器，另一端是应用程序 -> 轮询时主动放弃 CPU -> 确定



2. 开启虚拟机中的 Windows 系统，然后以管理员身份运行CMD命令行，输入 `bcdedit` 命令，可以查看到系统的当前启动项，如果是新的系统，则只有 `{current}` 启动项以及一个 `{bootmgr}` 项。

```
管理员: cmd
C:\Windows\system32>
C:\Windows\system32>bcdedit

Windows 启动管理器
-----
标识符                {bootmgr}
device                partition=\Device\HarddiskVolume2
path                  \EFI\Microsoft\Boot\bootmgfw.efi
description            Windows Boot Manager
locale                zh-CN
inherit                {globalsettings}
default                {current}
resumeobject           {bdb0b3b2-3f21-11ed-9931-d46011246f28}
displayorder           {current}
toolsdisplayorder      {memdiag}
timeout                30

Windows 启动加载器
-----
标识符                {current}
device                partition=C:
path                  \Windows\system32\winload.efi
description            Windows 10
locale                zh-CN
inherit                {bootloadersettings}
recoverysequence       {bdb0b3b4-3f21-11ed-9931-d46011246f28}
displaymessageoverride Recovery
```

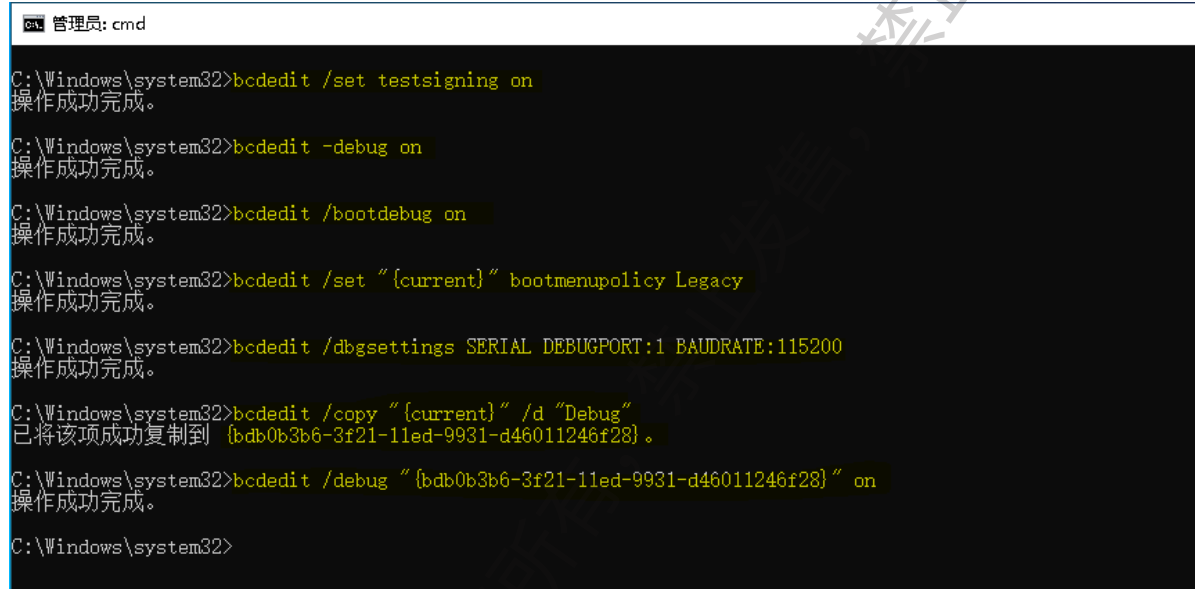
连续执行下方的三条命令，依次建立启动项，并激活调试模式。

```

C:\LyShark > bcdedit /set testsigning on
C:\LyShark > bcdedit -debug on
C:\LyShark > bcdedit /bootdebug on
C:\LyShark > bcdedit /set "{current}" bootmenupolicy Legacy          // 修改启动方式为Legacy
C:\LyShark > bcdedit /dbgsettings SERIAL DEBUGPORT:1 BAUDRATE:115200 // 设置串口1为调试端口波特率为115200
C:\LyShark > bcdedit /copy "{current}" /d "Debug"                  // 将当前配置复制到Debug启动配置
C:\LyShark > bcdedit /debug "{<新建的启动配置的标识符>}" on        // 打开调试开关

```

一气呵成，但需要注意 {<新建的启动配置的标识符>} 需替换成 {bdb0b3b6-3f21-11ed-9931-d46011246f28} 标志，如下所示。

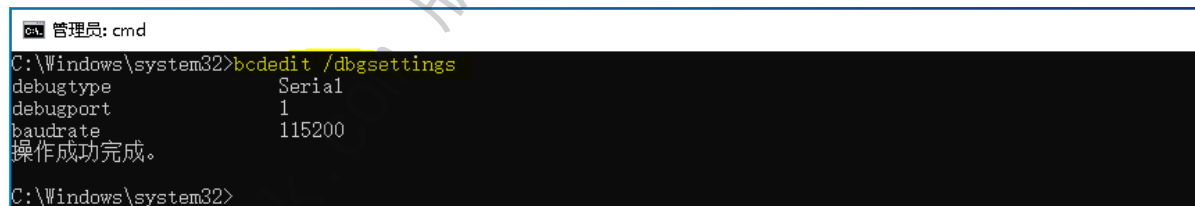


```

管理员: cmd
C:\Windows\system32>bcdedit /set testsigning on
操作成功完成。
C:\Windows\system32>bcdedit -debug on
操作成功完成。
C:\Windows\system32>bcdedit /bootdebug on
操作成功完成。
C:\Windows\system32>bcdedit /set "{current}" bootmenupolicy Legacy
操作成功完成。
C:\Windows\system32>bcdedit /dbgsettings SERIAL DEBUGPORT:1 BAUDRATE:115200
操作成功完成。
C:\Windows\system32>bcdedit /copy "{current}" /d "Debug"
已将该项成功复制到 {bdb0b3b6-3f21-11ed-9931-d46011246f28}。
C:\Windows\system32>bcdedit /debug "{bdb0b3b6-3f21-11ed-9931-d46011246f28}" on
操作成功完成。
C:\Windows\system32>

```

3.最后查看一下当前调试配置选项，执行命令 `bcdedit /dbgsettings`，显示出使用的第一个串口，波特率为 115200bps，保持默认不需要修改。



```

管理员: cmd
C:\Windows\system32>bcdedit /dbgsettings
debugtype          Serial
debugport          1
baudrate           115200
操作成功完成。
C:\Windows\system32>

```

4.配置完成后，重新启动系统，在开机的时候选择 Windows10 [启用调试程序] 则系统会黑屏，说明已经正常进入调试模式了。



Windows 启动管理器

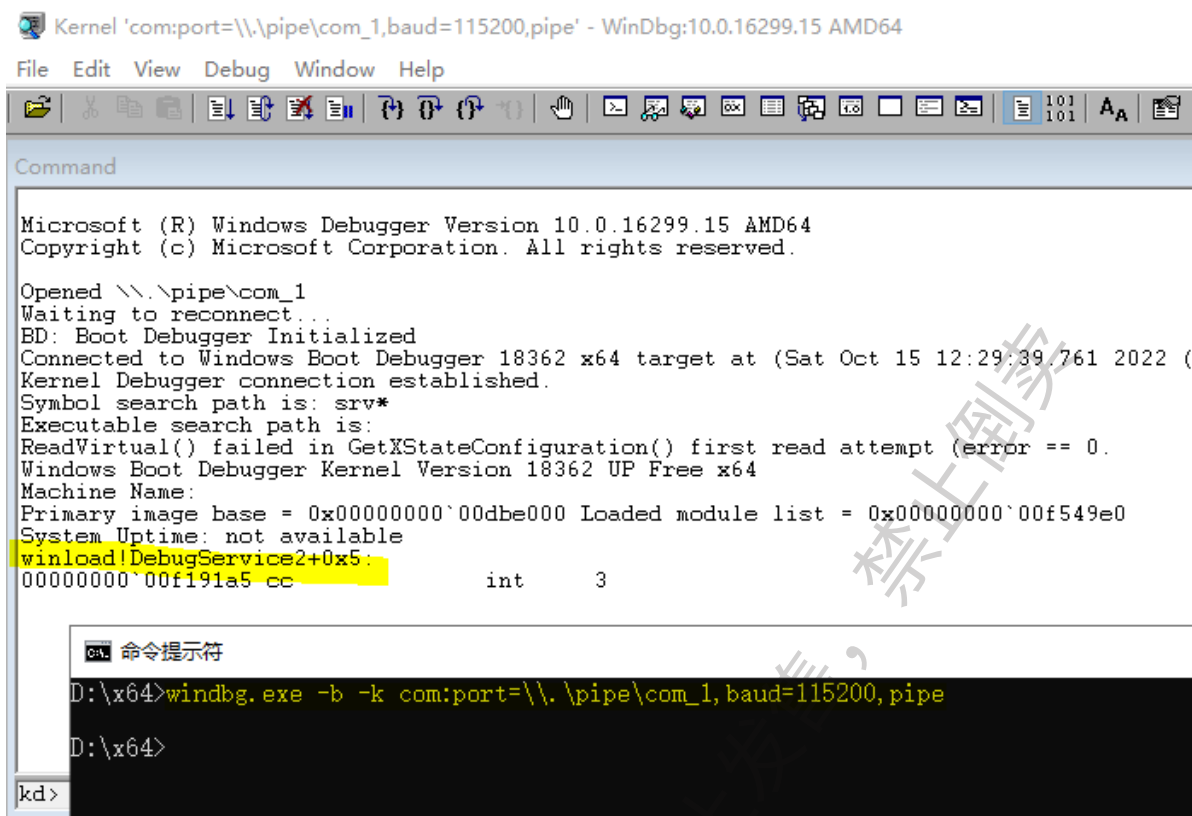
选择要启动的操作系统，或按 Tab 选择工具：
(使用箭头键突出显示你的选择，然后按 Enter。)

Windows 10 [启用调试程序] >

Debug [启用调试程序]

5.回到物理机上面，我们在命令行中切换到 WinDBG 的根目录下，并执行以下命令，即可连接虚拟机串口进行调试了。

- 执行命令 `windbg.exe -b -k com:port=\\.\pipe\com_1,baud=115200,pipe` 如下图



6.至此我们还需要加载符号，在命令行下依次执行以下命令，配置好符号加载并启动系统。

```
kd> .sympath SRV*c:\mySymbols*http://msdl.microsoft.com/download/symbols
kd> .reload
kd> g
kd> g
kd> ed nt!Kd_SXS_Mask 0
kd> ed nt!Kd_FUSION_Mask 0
kd> u KiSystemServiceUser
```

这样即可完成配置操作。

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```
Deferred SRV*c:\mySymbols*http://msdl.microsoft.com/download/symbols
Symbol search path is: SRV*c:\mySymbols*http://msdl.microsoft.com/download/symbols
Executable search path is:
Windows 10 Kernel Version 18362 MP (1 procs) Free x64
Built by: 18362.1.amd64fre.19h1_release.190318-1202
Machine Name:
Kernel base = 0xfffff807`23000000 PsLoadedModuleList = 0xfffff807`23448150
System Uptime: 0 days 0:00:00.000
nt!DebugService2+0x5:
fffff807`231c93c5 cc int 3
kd> ed nt!Kd_SXS_Mask 0
kd> ed nt!Kd_FUSION_Mask 0
kd> u KiSystemServiceUser
nt!KiSystemServiceUser:
fffff807`231d2a82 c645ab02 mov byte ptr [rbp-55h],2
fffff807`231d2a86 65488b1c2588010000 mov rbx,qword ptr gs:[188h]
fffff807`231d2a8f 0f0d8b9000000000 prefetchw [rbx+90h]
fffff807`231d2a96 0fae5dac stmxcsr dword ptr [rbp-54h]
fffff807`231d2a9a 650fae142580010000 ldmxcsr dword ptr gs:[180h]
fffff807`231d2aa3 807b0300 cmp byte ptr [rbx+3],0
fffff807`231d2aa7 66c78580000000000000 mov word ptr [rbp+80h],0
fffff807`231d2ab0 0f84a8000000 je nt!KiSystemServiceUser+0xdc (fffff807`231d2b0)
kd>
```

Ln 0, Col 0 Sys 0:KdSrv:S Proc 000:0

7.最后我们配置测试一下调试功能，首先编写以下代码，代码中使用 DbgBreakPoint() 设置断点，将会在入口处中断。

```
#include <ntifs.h>

NTSTATUS DriverDefaultHandle(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS status = STATUS_SUCCESS;
    pIrp->IoStatus.Status = status;
    pIrp->IoStatus.Information = 0;
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);

    return status;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载\n");
}

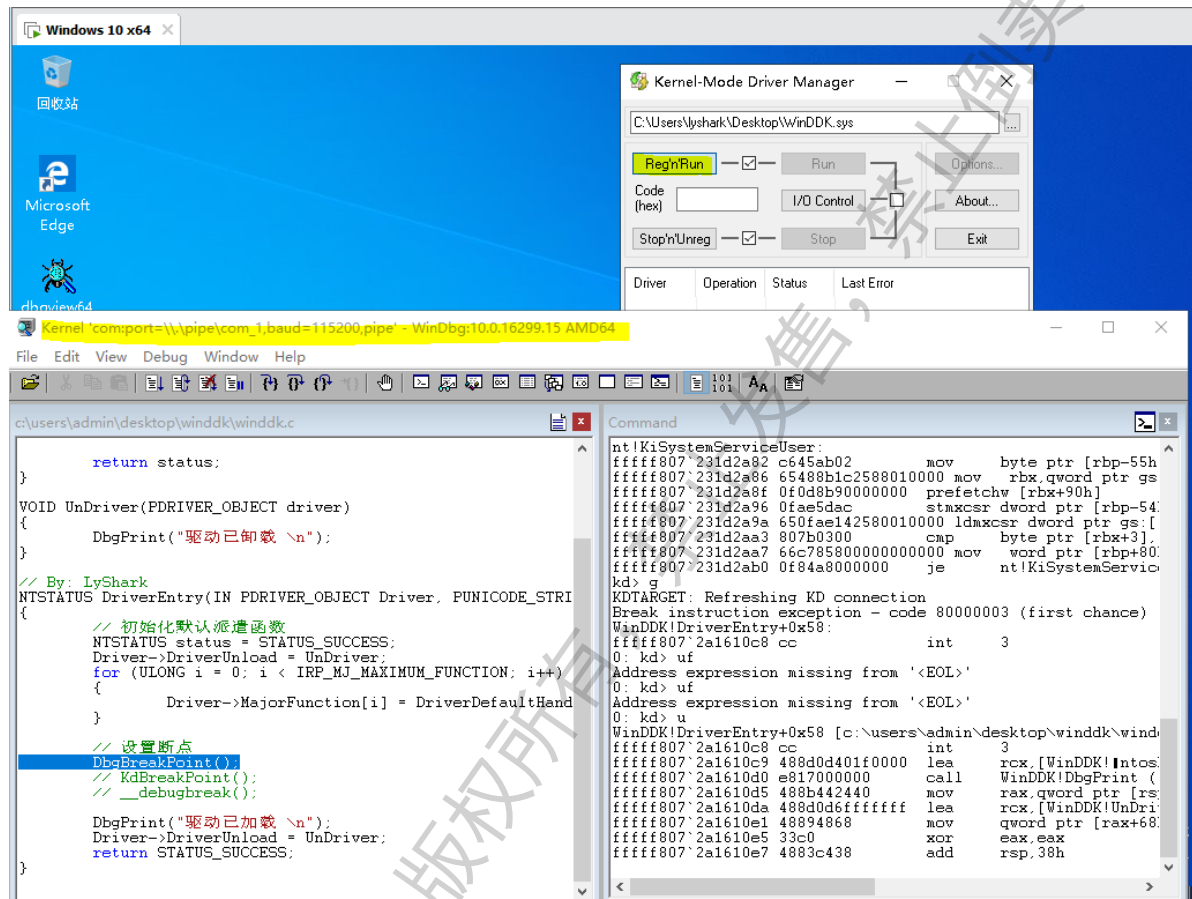
// By: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    // 初始化默认派遣函数
    NTSTATUS status = STATUS_SUCCESS;
    Driver->DriverUnload = UnDriver;
    for (ULONG i = 0; i < IRP_MJ_MAXIMUM_FUNCTION; i++)
    {
        Driver->MajorFunction[i] = DriverDefaultHandle;
    }

    // 设置断点
    DbgBreakPoint();
}
```

```
// KdBreakPoint();
// __debugbreak();

DbgPrint("驱动已加载 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}
```

当 Windows 系统加载完成以后，拖入我们的驱动文件 WinDDK.sys，并通过驱动加载工具加载运行，此时 Windows 系统会卡死，回到 WinDBG 中发现已经可以进行调试了。



1.3 内核测试模式过DSE签名

微软在 x64 系统中推出了 DSE 保护机制，DSE 全称 (Driver Signature Enforcement)，该保护机制的核心就是任何驱动程序或者是第三方驱动如果想要在正常模式下被加载则必须要经过微软的认证，当驱动程序被加载到内存时会验证签名的正确性，如果签名不正常则系统会拒绝运行驱动，这种机制也被称为驱动强制签名，该机制的作用是保护系统免受恶意软件的破坏，是提高系统安全性的一种手段。

该验证机制即便是在调试模式也需要强制签名，对于一名 驱动开发者 来说是很麻烦的一件事情，而签名的验证则是在加载时验证驱动入口 _KLDL_DATA_TABLE_ENTRY 里面的 Flags 标志，如果此标志被 pLdrData->Flags | 0x20 置位，则在调试模式下就不会在验证签名了，省去了重复签名的麻烦。

代码的实现非常容易，如下所示：

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
```

```

// 绕过签名检查
BOOLEAN BypassCheckSign(PDRIVER_OBJECT pDriverObject)
{
#ifdef _WIN64
    typedef struct _KLDR_DATA_TABLE_ENTRY
    {
        LIST_ENTRY listEntry;
        ULONG64 __Undefined1;
        ULONG64 __Undefined2;
        ULONG64 __Undefined3;
        ULONG64 NonPagedDebugInfo;
        ULONG64DllBase;
        ULONG64 EntryPoint;
        ULONG SizeOfImage;
        UNICODE_STRING path;
        UNICODE_STRING name;
        ULONG Flags;
        USHORT LoadCount;
        USHORT __Undefined5;
        ULONG64 __Undefined6;
        ULONG CheckSum;
        ULONG __padding1;
        ULONG TimeDateStamp;
        ULONG __padding2;
    } KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#else
    typedef struct _KLDR_DATA_TABLE_ENTRY
    {
        LIST_ENTRY listEntry;
        ULONG unknown1;
        ULONG unknown2;
        ULONG unknown3;
        ULONG unknown4;
        ULONG unknown5;
        ULONG unknown6;
        ULONG unknown7;
        UNICODE_STRING path;
        UNICODE_STRING name;
        ULONG Flags;
    } KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#endif

    PKLDR_DATA_TABLE_ENTRY pLdrData = (PKLDR_DATA_TABLE_ENTRY)pDriverObject->DriverSection;
    pLdrData->Flags = pLdrData->Flags | 0x20;

    return TRUE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
}

```



```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;

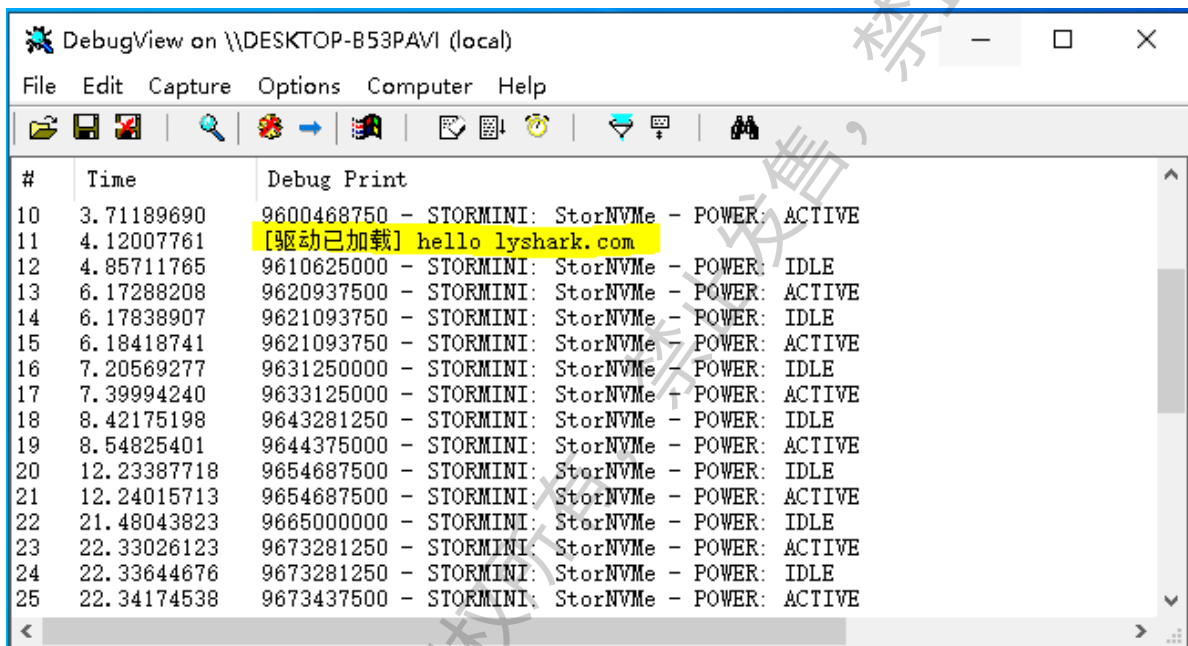
    // 绕过签名检查
    // LINKER_FLAGS=/INTEGRITYCHECK
    BypassCheckSign(Driver);

    DbgPrint("[驱动已加载] hello lyshark.com \n");

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

将程序拖入到虚拟机，直接运行即可加载，无需再继续签名：



当然这种方式只能在测试模式下使用，在正常模式也是无效的，只是为了方便测试驱动。

第二章：基础知识篇

2.1 内核中的链表与结构体

Windows内核中是无法使用 vector 容器等数据结构的，当我们需要保存一个结构体数组时，就需要使用内核中提供的专用链表结构 LIST_ENTRY 通过一些列链表操作函数对结构体进行装入弹出等操作，如下代码是本人总结的内核中使用链表存储多个结构体的通用案例。

首先实现一个枚举用户进程功能，将枚举到的进程存储到链表结构体内。

```

#include <ntifs.h>
#include <windef.h>

extern PVOID PsGetProcessPeb(_In_ PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
extern NTKERNELAPI PVOID PsGetProcessWow64Process(_In_ PEPROCESS Process);
extern NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process);

```

```

extern NTKERNELAPI HANDLE PsGetProcessInheritedFromUniqueProcessId(IN PEPROCESS
Process);

typedef struct
{
    DWORD Pid;
    UCHAR ProcessName[2048];
    DWORD Handle;
    LIST_ENTRY ListEntry;
}ProcessList;

// 根据进程ID返回进程EPROCESS结构体失败返回NULL
PEPROCESS LookupProcess(HANDLE Pid)
{
    PEPROCESS eprocess = NULL;
    NTSTATUS Status = STATUS_UNSUCCESSFUL;
    Status = PsLookupProcessByProcessId(Pid, &eprocess);
    if (NT_SUCCESS(Status))
    {
        return eprocess;
    }
    return NULL;
}

// 内核链表操作
// By: LyShark
BOOLEAN GetAllProcess()
{
    PEPROCESS eproc = NULL;
    LIST_ENTRY linkListHead;

    // 初始化链表头部
    InitializeListHead(&linkListHead);
    ProcessList *pData = NULL;

    for (int temp = 0; temp < 100000; temp += 4)
    {
        eproc = LookupProcess((HANDLE)temp);
        if (eproc != NULL)
        {
            STRING nowProcessnameString = { 0 };
            RtlInitString(&nowProcessnameString,
PsGetProcessImageFileName(eproc));

            // DbgPrint("进程名: %s --> 进程PID = %d --> 父进程PPID = %d\r\n",
            // PsGetProcessImageFileName(eproc), PsGetProcessId(eproc),
            PsGetProcessInheritedFromUniqueProcessId(eproc));

            // 分配内核堆空间
            pData = (ProcessList *)ExAllocatePool(PagedPool,
sizeof(ProcessList));
            RtlZeroMemory(pData, sizeof(ProcessList));

            // 设置变量
            pData->Pid = (DWORD)PsGetProcessId(eproc);

```

```

        RtlCopyMemory(pData->ProcessName, PsGetProcessImageFileName(eproc),
strlen(PsGetProcessImageFileName(eproc)) * 2);
        pData->Handle =
(DWORD)PsGetProcessInheritedFromUniqueProcessId(eproc);

        // 插入元素到
        InsertTailList(&linkListHead, &pData->ListEntry);
        ObDereferenceObject(eproc);
    }
}

// 输出链表内的数据
while (!IsListEmpty(&linkListHead))
{
    LIST_ENTRY *pEntry = RemoveHeadList(&linkListHead);
    pData = CONTAINING_RECORD(pEntry, ProcessList, ListEntry);

    DbgPrint("%d \n", pData->Pid);
    DbgPrint("%s \n", pData->ProcessName);
    DbgPrint("%d \n", pData->Handle);
    ExFreePool(pData);
}
return TRUE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

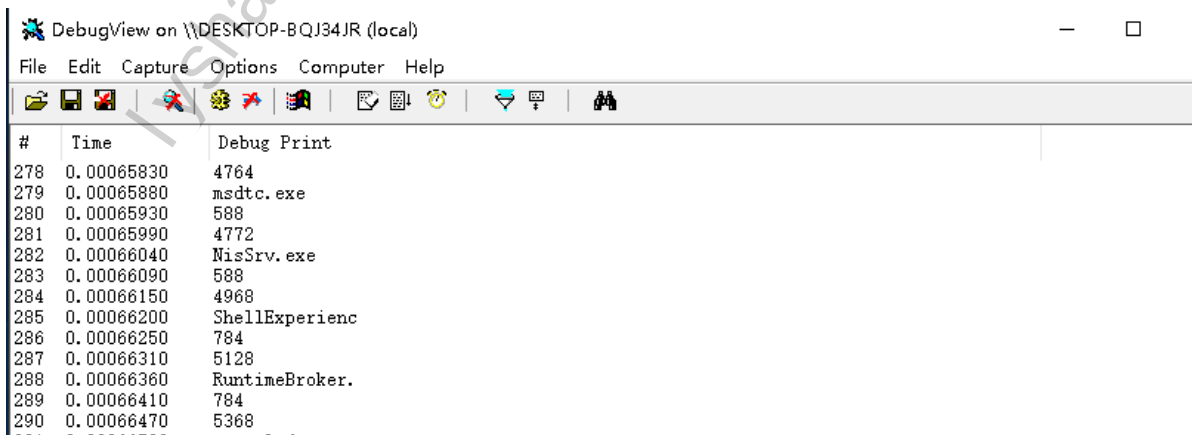
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    GetAllProcess();

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行后将可以在DbgView中看到输出的进程信息：



#	Time	Debug Print
278	0.00065830	4764
279	0.00065880	msdtc.exe
280	0.00065930	588
281	0.00065990	4772
282	0.00066040	NisSrv.exe
283	0.00066090	588
284	0.00066150	4968
285	0.00066200	ShellExperienc
286	0.00066250	784
287	0.00066310	5128
288	0.00066360	RuntimeBroker.
289	0.00066410	784
290	0.00066470	5368

如果需要返回一个结构体，则可以这样来写代码。

```

#include <ntifs.h>
#include <windef.h>

typedef struct
{
    int count;
    char username[256];
    char password[256];
}MyData;

// 模拟返回一个结构
BOOLEAN GetProcess(PVOID OutPut)
{
    RtlZeroMemory(OutPut, sizeof(MyData));
    MyData *data = OutPut;

    data->count = 100;
    RtlCopyMemory(data->username, "lyshark.com", sizeof("lyshark.com"));
    RtlCopyMemory(data->password, "https://www.cnblogs.com/lyshark",
sizeof("https://www.cnblogs.com/lyshark"));
    return TRUE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");
    PVOID Ptr = (PVOID)ExAllocatePool(NonPagedPool, sizeof(MyData));

    GetProcess(Ptr);

    MyData *data = (MyData *)Ptr;

    DbgPrint("count = %d \n", data->count);
    DbgPrint("username = %s \n", data->username);
    DbgPrint("password = %s \n", data->password);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

输出效果如下:

DebugView on \\DESKTOP-BQJ34JR (local)		
#	Time	Debug Print
431	1572.95410156	25633437500 - STORMINI: StorNVMe - POWER: ACTIVE
432	1572.95483398	25633437500 - STORMINI: StorNVMe - POWER: IDLE
433	1572.95495605	25633437500 - STORMINI: StorNVMe - POWER: ACTIVE
434	1573.96203613	25643593750 - STORMINI: StorNVMe - POWER: IDLE
435	1574.27685547	25646718750 - STORMINI: StorNVMe - POWER: ACTIVE
436	1575.28869629	25656718750 - STORMINI: StorNVMe - POWER: IDLE
437	1575.50207520	25658906250 - STORMINI: StorNVMe - POWER: ACTIVE
438	1575.50488281	hello lyshark.com
439	1575.50488281	count = 100
440	1575.50488281	username = lyshark.com
441	1575.50488281	password = https://www.cnblogs.com/lyshark
442	1576.09167480	Uninstall Driver Is OK
443	1576.50195313	25668906250 - STORMINI: StorNVMe - POWER: IDLE

2.2 内核中的自旋锁结构

提到自旋锁那就必须要说链表，在上一篇《驱动开发：内核中的链表与结构体》文章中简单实用链表结构来存储进程信息列表，相信读者应该已经理解了内核链表的基本使用，本篇文章将讲解自旋锁的简单应用，自旋锁是为了解决内核链表读写时存在线程同步问题，解决多线程同步问题必须要用锁，通常使用自旋锁，自旋锁是内核中提供了一种高IRQL锁，用同步以及独占的方式访问某个资源。

首先以简单的链表为案例，链表主要分为单向链表与双向链表，单向链表的链表节点中只有一个链表指针，其指向后一个链表元素，而双向链表节点中有两个链表节点指针，其中Blink指向前一个链表节点Flink指向后一个节点，以双向链表为例。

```
#include <ntifs.h>
#include <ntstrsafe.h>

/*
// 链表节点指针
typedef struct _LIST_ENTRY
{
    struct _LIST_ENTRY *Flink;    // 当前节点的后一个节点
    struct _LIST_ENTRY *Blink;   // 当前节点的前一个节点
}LIST_ENTRY, *PLIST_ENTRY;
*/

typedef struct _MyStruct
{
    ULONG x;
    ULONG y;
    LIST_ENTRY lpListEntry;
}MyStruct, *pMyStruct;

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// By: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("By:LyShark \n");
    DbgPrint("Email:me@lyshark.com \n");
    // 初始化头节点
    LIST_ENTRY ListHeader = { 0 };
    InitializeListHead(&ListHeader);
```

```

// 定义链表元素
MyStruct testA = { 0 };
MyStruct testB = { 0 };
MyStruct testC = { 0 };

testA.x = 100;
testA.y = 200;

testB.x = 1000;
testB.y = 2000;

testC.x = 10000;
testC.y = 20000;

// 分别插入节点到头部和尾部
InsertHeadList(&ListHeader, &testA.lplListEntry);
InsertTailList(&ListHeader, &testB.lplListEntry);
InsertTailList(&ListHeader, &testC.lplListEntry);

// 节点不为空 则 移除一个节点
if (IsListEmpty(&ListHeader) == FALSE)
{
    RemoveEntryList(&testA.lplListEntry);
}

// 输出链表数据
PLIST_ENTRY pListEntry = NULL;
pListEntry = ListHeader.Flink;

while (pListEntry != &ListHeader)
{
    // 计算出成员距离结构体顶部内存距离
    pMyStruct ptr = CONTAINING_RECORD(pListEntry, MyStruct, lplListEntry);
    DbgPrint("节点元素X = %d 节点元素Y = %d \n", ptr->x, ptr->y);

    // 得到下一个元素地址
    pListEntry = pListEntry->Flink;
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

链表输出效果如下：

DebugView on \\DESKTOP-BQJ34JR (local)		
File Edit Capture Options Computer Help		
#	Time	Debug Print
1	0.00000000	By: LyShark
2	0.00000130	Email: me@lyshark.com
3	0.00000250	节点元素X = 1000 节点元素Y = 2000
4	0.00000350	节点元素X = 10000 节点元素Y = 20000
5	0.12096060	2704062500 - STORMINI: StorNVMe - POWER: IDLE
6	0.43899781	2707187500 - STORMINI: StorNVMe - POWER: ACTIVE
7	1.02144492	驱动卸载成功
8	1.45991313	2717343750 - STORMINI: StorNVMe - POWER: IDLE
9	3.60306191	2738906250 - STORMINI: StorNVMe - POWER: ACTIVE
10	3.60958052	2738906250 - STORMINI: StorNVMe - POWER: IDLE
11	3.61152840	2738906250 - STORMINI: StorNVMe - POWER: ACTIVE
12	4.62320614	2749062500 - STORMINI: StorNVMe - POWER: IDLE
13	4.96565485	2752500000 - STORMINI: StorNVMe - POWER: ACTIVE

如上所述，内核链表读写时存在线程同步问题，解决多线程同步问题必须要用锁，通常使用自旋锁，自旋锁是内核中提供了一种高IRQL锁，用同步以及独占的方式访问某个资源。

```
#include <ntifs.h>
#include <ntstrsafe.h>

/*
// 链表节点指针
typedef struct _LIST_ENTRY
{
    struct _LIST_ENTRY *Flink;    // 当前节点的后一个节点
    struct _LIST_ENTRY *Blink;    // 当前节点的前一个节点
}LIST_ENTRY, *PLIST_ENTRY;
*/

typedef struct _MyStruct
{
    ULONG x;
    ULONG y;
    LIST_ENTRY lpListEntry;
}MyStruct, *pMyStruct;

// 定义全局链表和全局锁
LIST_ENTRY my_list_header;
KSPIN_LOCK my_list_lock;

// 初始化
void Init()
{
    InitializeListHead(&my_list_header);
    KeInitializeSpinLock(&my_list_lock);
}

// 函数内使用锁
void function_ins()
{
    KIRQL Irql;

    // 加锁
    KeAcquireSpinLock(&my_list_lock, &Irql);

    DbgPrint("锁内部执行 \n");
}
```

```

// 释放锁
KeReleaseSpinLock(&my_list_lock, Irql);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// By: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("By:LyShark \n");
    DbgPrint("Email:me@lyshark.com \n");

    // 初始化链表
    Init();

    // 分配链表空间
    pMyStruct testA = (pMyStruct)ExAllocatePool(NonPagedPoolExecute,
    sizeof(pMyStruct));
    pMyStruct testB = (pMyStruct)ExAllocatePool(NonPagedPoolExecute,
    sizeof(pMyStruct));

    // 赋值
    testA->x = 100;
    testA->y = 200;

    testB->x = 1000;
    testB->y = 2000;

    // 向全局链表中插入数据
    if (NULL != testA && NULL != testB)
    {
        ExInterlockedInsertHeadList(&my_list_header, (PLIST_ENTRY)&testA-
        >lpListEntry, &my_list_lock);
        ExInterlockedInsertTailList(&my_list_header, (PLIST_ENTRY)&testB-
        >lpListEntry, &my_list_lock);
    }

    function_ins();

    // 移除节点A并放入到remove_entry中
    PLIST_ENTRY remove_entry = ExInterlockedRemoveHeadList(&testA->lpListEntry,
    &my_list_lock);

    // 输出链表数据
    while (remove_entry != &my_list_header)
    {
        // 计算出成员距离结构体顶部内存距离
        pMyStruct ptr = CONTAINING_RECORD(remove_entry, MyStruct, lpListEntry);
        DbgPrint("节点元素X = %d 节点元素Y = %d \n", ptr->x, ptr->y);

        // 得到下一个元素地址
        remove_entry = remove_entry->Flink;
    }
}

```



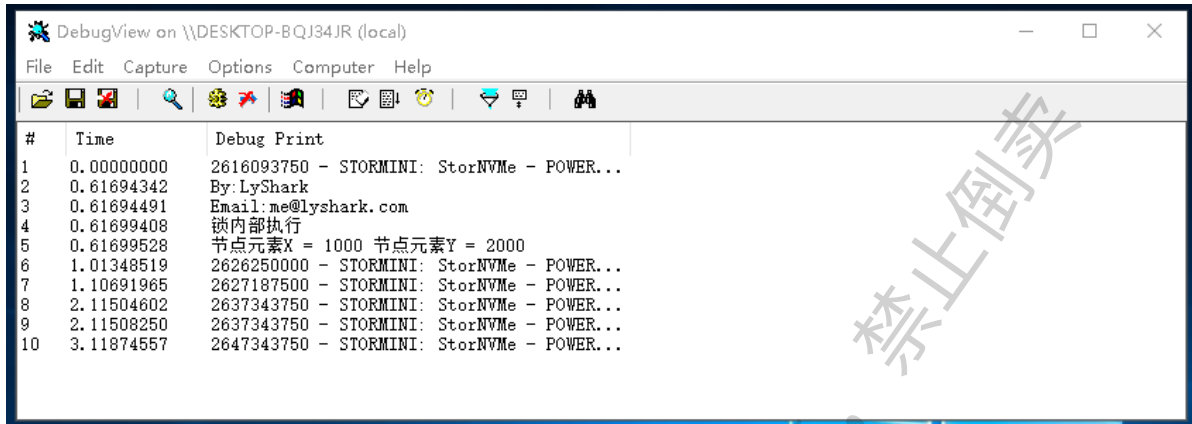
```

}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

加锁后执行效果如下：



2.3 内核字符串转换方法

在内核编程中字符串有两种格式 ANSI_STRING 与 UNICODE_STRING，这两种格式是微软推出的安全版本的字符串结构体，也是微软推荐使用的格式，通常情况下 ANSI_STRING 代表的类型是 char * 也就是 ANSI 多字节模式的字符串，而 UNICODE_STRING 则代表的是 wchar* 也就是 UNICODE 类型的字符，如下文章将介绍这两种字符格式在内核中是如何转换的。

在内核开发模式下 初始化字符串 也需要调用专用的初始化函数，如下分别初始化 ANSI 和 UNICODE 字符串，我们来看看代码是如何实现的。

```

#include <ntifs.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    // 定义内核字符串
    ANSI_STRING ansi;
    UNICODE_STRING unicode;
    UNICODE_STRING str;

    // 定义普通字符串
    char * char_string = "hello lyshark";
    wchar_t *wchar_string = (WCHAR*)"hello lyshark";

    // 初始化字符串的多种方式
    RtlInitAnsiString(&ansi, char_string);
    RtlInitUnicodeString(&unicode, wchar_string);
    RtlUnicodeStringInit(&str, L"hello lyshark");

    // 改变原始字符串（乱码位置，此处仅用于演示赋值方式）

```

```

char_string[0] = (CHAR)"A";           // char类型每个占用1字节
char_string[1] = (CHAR)"B";

wchar_string[0] = (WCHAR)"A";        // wchar类型每个占用2字节
wchar_string[2] = (WCHAR)"B";

// 输出字符串 %Z
DbgPrint("输出ANSI: %Z \n", &ansi);
DbgPrint("输出WCHAR: %Z \n", &unicode);
DbgPrint("输出字符串: %wZ \n", &str);

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
56	31.80723572	666406250 - STORMINI: StorNVMe - POWER: IDLE
57	31.80726433	666406250 - STORMINI: StorNVMe - POWER: ACTIVE
58	32.80714798	676406250 - STORMINI: StorNVMe - POWER: IDLE
59	32.80838394	676406250 - STORMINI: StorNVMe - POWER: ACTIVE
60	33.82435608	686562500 - STORMINI: StorNVMe - POWER: IDLE
61	34.51197052	693437500 - STORMINI: StorNVMe - POWER: ACTIVE
62	34.51274872	693437500 - STORMINI: StorNVMe - POWER: IDLE
63	34.51290894	693437500 - STORMINI: StorNVMe - POWER: ACTIVE
64	40.36994171	752031250 - STORMINI: StorNVMe - POWER: IDLE
65	40.37130737	752031250 - STORMINI: StorNVMe - POWER: ACTIVE
66	40.55623627	输出ANSI: d磗lo lyshark
67	40.55624008	输出WCHAR: ?11?lyshark输出字符串: hello lyshark
68	40.55624390	驱动加载成功
69	41.33798981	驱动卸载成功
70	41.37001038	762031250 - STORMINI: StorNVMe - POWER: IDLE
71	41.85396957	766875000 - STORMINI: StorNVMe - POWER: ACTIVE
72	47.71379852	825468750 - STORMINI: StorNVMe - POWER: ACTIVE

内核中还可实现 字符串与整数 之间的灵活转换，内核中提供了 `RtlUnicodeStringToInteger` 这个函数来实现 字符串转整数，与之对应的 `RtlIntegerToUnicodeString` 则是将 整数转为字符串 这两个内核函数也是非常常用的。

```

#include <ntifs.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// Power: lyshark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS flag;
    ULONG number;

    DbgPrint("hello lyshark \n");
}

```

```

UNICODE_STRING uncode_buffer_source = { 0 };
UNICODE_STRING uncode_buffer_target = { 0 };

// 字符串转为数字
// By: LyShark
RtlInitUnicodeString(&uncode_buffer_source, L"100");
flag = RtlUnicodeStringToInteger(&uncode_buffer_source, 10, &number);

if (NT_SUCCESS(flag))
{
    DbgPrint("字符串 -> 数字: %d \n", number);
}

// 数字转为字符串
uncode_buffer_target.Buffer = (PWSTR)ExAllocatePool(PagedPool, 1024);
uncode_buffer_target.MaximumLength = 1024;

flag = RtlIntegerToUnicodeString(number, 10, &uncode_buffer_target);

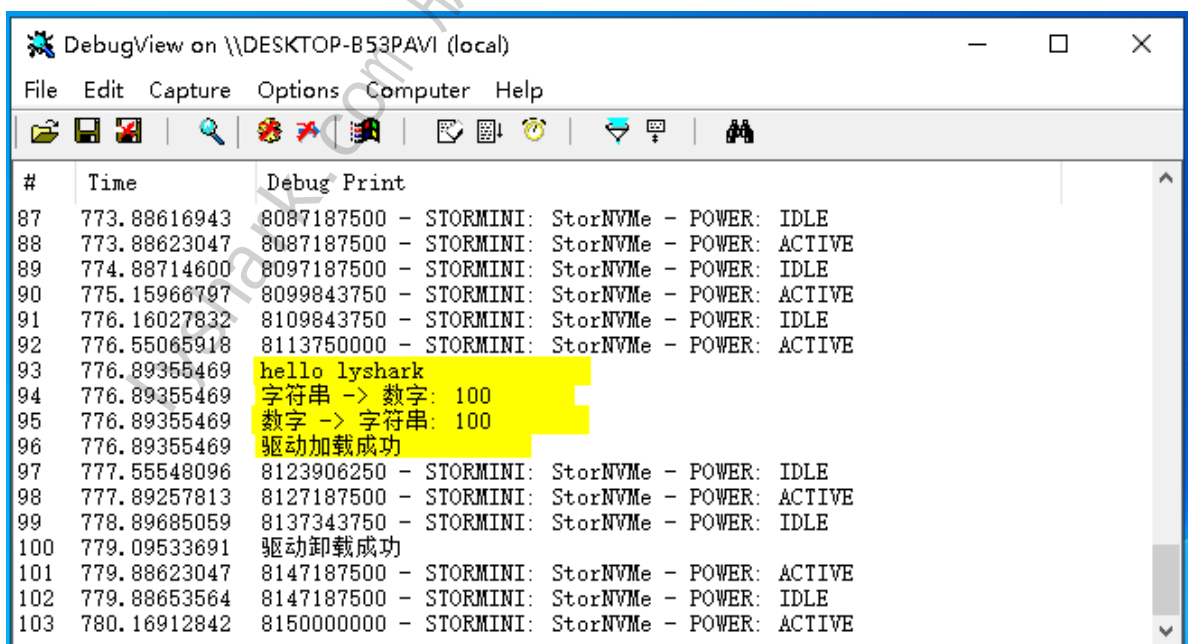
if (NT_SUCCESS(flag))
{
    DbgPrint("数字 -> 字符串: %wZ \n", &uncode_buffer_target);
}

// 释放堆空间
RtlFreeUnicodeString(&uncode_buffer_target);

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:



#	Time	Debug Print
87	773.88616943	8087187500 - STORMINI: StorNVMe - POWER: IDLE
88	773.88623047	8087187500 - STORMINI: StorNVMe - POWER: ACTIVE
89	774.88714600	8097187500 - STORMINI: StorNVMe - POWER: IDLE
90	775.15966797	8099843750 - STORMINI: StorNVMe - POWER: ACTIVE
91	776.16027832	8109843750 - STORMINI: StorNVMe - POWER: IDLE
92	776.55065918	8113750000 - STORMINI: StorNVMe - POWER: ACTIVE
93	776.89355469	hello lyshark
94	776.89355469	字符串 -> 数字: 100
95	776.89355469	数字 -> 字符串: 100
96	776.89355469	驱动加载成功
97	777.55548096	8123906250 - STORMINI: StorNVMe - POWER: IDLE
98	777.89257813	8127187500 - STORMINI: StorNVMe - POWER: ACTIVE
99	778.89685059	8137343750 - STORMINI: StorNVMe - POWER: IDLE
100	779.09533691	驱动卸载成功
101	779.88623047	8147187500 - STORMINI: StorNVMe - POWER: ACTIVE
102	779.88653564	8147187500 - STORMINI: StorNVMe - POWER: IDLE
103	780.16912842	8150000000 - STORMINI: StorNVMe - POWER: ACTIVE

继续看另一种转换模式，将 UNICODE_STRING 结构转换成 ANSI_STRING 结构，代码中调用了 `RtlUnicodeStringToAnsiString` 内核函数，该函数也是微软提供的。

```

#include <ntifs.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// Power: lyshark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING uncode_buffer_source = { 0 };
    ANSI_STRING ansi_buffer_target = { 0 };

    // 初始化 UNICODE 字符串
    RtlInitUnicodeString(&uncode_buffer_source, L"hello lyshark");

    // 转换函数
    NTSTATUS flag = RtlUnicodeStringToAnsiString(&ansi_buffer_target,
&uncode_buffer_source, TRUE);

    if (NT_SUCCESS(flag))
    {
        DbgPrint("ANSI: %Z \n", &ansi_buffer_target);
    }

    // 销毁ANSI字符串
    RtlFreeAnsiString(&ansi_buffer_target);

    DbgPrint("驱动加载成功 \n");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
108	1034.17053223	10690000000 - STORMINI: StorNVMe - POWER: ACTIVE
109	1035.17138672	10700000000 - STORMINI: StorNVMe - POWER: IDLE
110	1035.75878906	10705937500 - STORMINI: StorNVMe - POWER: ACTIVE
111	1035.75964355	10705937500 - STORMINI: StorNVMe - POWER: IDLE
112	1035.75964355	10705937500 - STORMINI: StorNVMe - POWER: ACTIVE
113	1036.76379395	10715937500 - STORMINI: StorNVMe - POWER: IDLE
114	1036.83557129	10716718750 - STORMINI: StorNVMe - POWER: ACTIVE
115	1036.83691406	hello lyshark
116	1036.83691406	ANSI: hello lyshark
117	1036.83691406	驱动加载成功
118	1037.83874512	10726718750 - STORMINI: StorNVMe - POWER: IDLE
119	1038.18737793	10730156250 - STORMINI: StorNVMe - POWER: ACTIVE
120	1039.19921875	10740312500 - STORMINI: StorNVMe - POWER: IDLE
121	1040.92138672	10757500000 - STORMINI: StorNVMe - POWER: ACTIVE
122	1040.92163086	10757500000 - STORMINI: StorNVMe - POWER: IDLE
123	1040.92565918	驱动卸载成功
124	1041.21044922	10760468750 - STORMINI: StorNVMe - POWER: ACTIVE

如果将上述过程反过来，将 ANSI_STRING 转换为 UNICODE_STRING 结构，则需要调用 RtlAnsiStringToUnicodeString 这个内核专用函数实现。

```
#include <ntifs.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// Power: lyshark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING unicode_buffer_source = { 0 };
    ANSI_STRING ansi_buffer_target = { 0 };

    // 初始化字符串
    RtlInitString(&ansi_buffer_target, "hello lyshark");

    // 转换函数
    NTSTATUS flag = RtlAnsiStringToUnicodeString(&unicode_buffer_source,
    &ansi_buffer_target, TRUE);
    if (NT_SUCCESS(flag))
    {
        DbgPrint("UNICODE: %wZ \n", &unicode_buffer_source);
    }

    // 销毁UNICODE字符串
    RtlFreeUnicodeString(&unicode_buffer_source);

    DbgPrint("驱动加载成功 \n");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}
```

代码输出效果：

DebugView on \\DESKTOP-B53PAVI (local)		
File Edit Capture Options Computer Help		
#	Time	Debug Print
122	1040.92163086	10757500000 - STORMINI: StorNVMe - POWER: IDLE
123	1040.92565918	驱动卸载成功
124	1041.21044922	10760468750 - STORMINI: StorNVMe - POWER: ACTIVE
125	1183.64953613	12184843750 - STORMINI: StorNVMe - POWER: IDLE
126	1184.31970215	12191562500 - STORMINI: StorNVMe - POWER: ACTIVE
127	1184.32006836	12191562500 - STORMINI: StorNVMe - POWER: IDLE
128	1184.32043457	12191562500 - STORMINI: StorNVMe - POWER: ACTIVE
129	1184.36450195	hello lyshark
130	1184.36450195	UNICODE: hello lyshark
131	1184.36450195	驱动加载成功
132	1185.32958984	12201562500 - STORMINI: StorNVMe - POWER: IDLE
133	1185.35583496	12201875000 - STORMINI: StorNVMe - POWER: ACTIVE
134	1185.49902344	驱动卸载成功
135	1186.35632324	12211875000 - STORMINI: StorNVMe - POWER: IDLE
136	1187.65563965	12224843750 - STORMINI: StorNVMe - POWER: ACTIVE
137	1187.65625000	12224843750 - STORMINI: StorNVMe - POWER: IDLE
138	1187.65637207	12224843750 - STORMINI: StorNVMe - POWER: ACTIVE

如上代码是内核通用结构体之间的转换类型，又是还需要将各类结构体转为普通的字符类型，例如下方的两个案例：

例如将 UNICODE_STRING 转为 CHAR* 类型。

```
#define _CRT_SECURE_NO_WARNINGS
#include <ntifs.h>
#include <windef.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// powerBY: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING uncode_buffer_source = { 0 };
    ANSI_STRING ansi_buffer_target = { 0 };
    char szBuf[1024] = { 0 };

    // 初始化 UNICODE 字符串
    RtlInitUnicodeString(&uncode_buffer_source, L"hello lyshark");

    // 转换函数
    NTSTATUS flag = RtlUnicodeStringToAnsiString(&ansi_buffer_target,
        &uncode_buffer_source, TRUE);

    if (NT_SUCCESS(flag))
    {
        strcpy(szBuf, ansi_buffer_target.Buffer);
        DbgPrint("输出char*字符串: %s \n", szBuf);
    }
}
```

```

// 销毁ANSI字符串
RtlFreeAnsiString(&ansi_buffer_target);

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
137	1187.65625000	12224843750 - STORMINI: StorNVMe - POWER: IDLE
138	1187.65637207	12224843750 - STORMINI: StorNVMe - POWER: ACTIVE
139	1472.04418945	15068750000 - STORMINI: StorNVMe - POWER: ACTIVE
140	1472.04443359	15068750000 - STORMINI: StorNVMe - POWER: IDLE
141	1472.04455566	15068750000 - STORMINI: StorNVMe - POWER: ACTIVE
142	1472.04675293	hello lyshark
143	1472.04675293	输出char*字符串: hello lyshark
144	1472.04675293	驱动加载成功
145	1473.04760742	15078750000 - STORMINI: StorNVMe - POWER: IDLE
146	1473.38745117	15082187500 - STORMINI: StorNVMe - POWER: ACTIVE
147	1474.39929199	15092343750 - STORMINI: StorNVMe - POWER: IDLE
148	1474.50231934	15093281250 - STORMINI: StorNVMe - POWER: ACTIVE
149	1475.50756836	15103437500 - STORMINI: StorNVMe - POWER: IDLE
150	1477.35461426	驱动卸载成功
151	1477.47595215	15123125000 - STORMINI: StorNVMe - POWER: ACTIVE
152	1477.47631836	15123125000 - STORMINI: StorNVMe - POWER: IDLE
153	1477.47680664	15123125000 - STORMINI: StorNVMe - POWER: ACTIVE

如果反过来, 将 `CHAR*` 类型转为 `UNICODE_STRING` 结构呢, 可以进行中转最终转为 `UNICODE_STRING` 结构体。

```

#define _CRT_SECURE_NO_WARNINGS
#include <ntifs.h>
#include <windef.h>
#include <ntstrsafe.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

// powerBY: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING unicode_buffer_source = { 0 };
    ANSI_STRING ansi_buffer_target = { 0 };

    // 设置CHAR*
    char szBuf[1024] = { 0 };
    strcpy(szBuf, "hello lyshark");

    // 初始化ANSI字符串
    RtlInitString(&ansi_buffer_target, szBuf);
}

```



```

// 转换函数
NTSTATUS flag = RtlAnsiStringToUnicodeString(&unicode_buffer_source,
&ansi_buffer_target, TRUE);
if (NT_SUCCESS(flag))
{
    DbgPrint("UNICODE: %wZ \n", &unicode_buffer_source);
}

// 销毁UNICODE字符串
RtlFreeUnicodeString(&unicode_buffer_source);

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
160	1600.48962402	16353281250 - STORMINI: StorNVMe - POWER: IDLE
161	1600.84045410	16356718750 - STORMINI: StorNVMe - POWER: ACTIVE
162	1601.84057617	16366718750 - STORMINI: StorNVMe - POWER: IDLE
163	1601.94750977	驱动卸载成功
164	1602.57849121	16374062500 - STORMINI: StorNVMe - POWER: ACTIVE
165	1602.57934570	16374062500 - STORMINI: StorNVMe - POWER: IDLE
166	1604.77258301	hello lyshark
167	1604.77258301	UNICODE: hello lyshark
168	1604.77258301	驱动加载成功
169	1604.78942871	16396250000 - STORMINI: StorNVMe - POWER: ACTIVE
170	1604.78967285	16396250000 - STORMINI: StorNVMe - POWER: IDLE
171	1604.78979492	16396250000 - STORMINI: StorNVMe - POWER: ACTIVE
172	1605.79370117	16406250000 - STORMINI: StorNVMe - POWER: IDLE
173	1606.07885742	16409062500 - STORMINI: StorNVMe - POWER: ACTIVE
174	1607.05871582	驱动卸载成功
175	1607.08361816	16419218750 - STORMINI: StorNVMe - POWER: IDLE
176	1607.58032227	16424062500 - STORMINI: StorNVMe - POWER: ACTIVE

2.4 内核字符串拷贝与比较

在上一篇文章《驱动开发：内核字符串转换方法》中简单介绍了内核是如何使用字符串以及字符串之间的转换方法，本章将继续探索字符串的拷贝与比较，与应用层不同内核字符串拷贝与比较也需要使用内核专用的API函数，字符串的拷贝往往伴随有内核内存分配，我们将首先简单介绍内核如何分配堆空间，然后再以此为契机简介字符串的拷贝与比较。

首先内核中的堆栈分配可以使用 `ExAllocatePool()` 这个内核函数实现，此外还可以使用 `ExAllocatePoolWithTag()` 函数，两者的区别是，第一个函数可以直接分配内存，第二个函数在分配时需要指定一个标签，此外内核属性常用的有两种 `NonPagedPool` 用于分配非分页内存，而 `PagePool` 则用于分配分页内存，在开发中推荐使用非分页内存，因为分页内存数量有限。

内存分配使用 `ExAllocatePool` 函数，内存拷贝可使用 `RtlCopyMemory` 函数，需要注意该函数其实是对 `Memcpy` 函数的包装。

```

#include <ntifs.h>

VOID UnDriver(PDRIVER_OBJECT driver)

```



```

{
    DbgPrint("驱动已卸载 \n");
}

// PowerBy: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    UNICODE_STRING unicode_buffer = { 0 };

    DbgPrint("hello lyshark \n");

    wchar_t * wchar_string = L"hello lyshark";

    // 设置最大长度
    unicode_buffer.MaximumLength = 1024;

    // 分配内存空间
    unicode_buffer.Buffer = (PWSTR)ExAllocatePool(PagedPool, 1024);

    // 设置字符串长度 因为是宽字符，所以是字符长度的 2 倍
    unicode_buffer.Length = wcslen(wchar_string) * 2;

    // 保证缓冲区足够大，否则程序终止
    ASSERT(unicode_buffer.MaximumLength >= unicode_buffer.Length);

    // 将 wchar_string 中的字符串拷贝到 unicode_buffer.Buffer
    RtlCopyMemory(unicode_buffer.Buffer, wchar_string, unicode_buffer.Length);

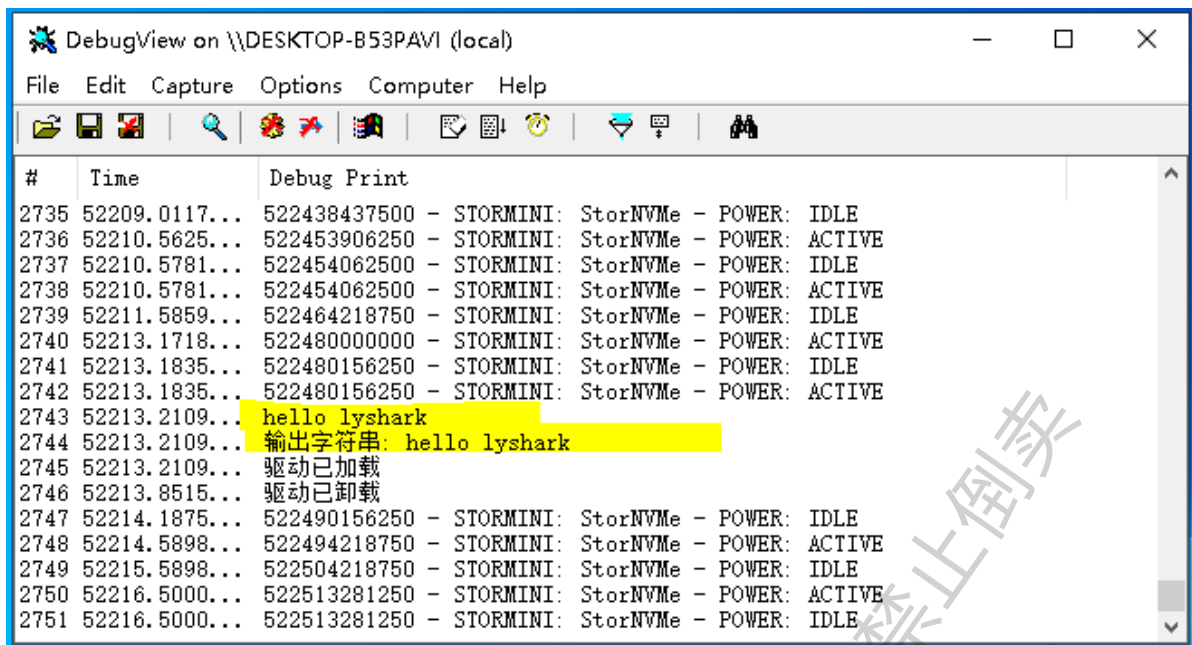
    // 设置字符串长度 并输出
    unicode_buffer.Length = wcslen(wchar_string) * 2;
    DbgPrint("输出字符串: %wZ \n", unicode_buffer);

    // 释放堆空间
    ExFreePool(unicode_buffer.Buffer);
    unicode_buffer.Buffer = NULL;
    unicode_buffer.Length = unicode_buffer.MaximumLength = 0;

    DbgPrint("驱动已加载 \n");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

代码输出效果：



实现 空间分配，字符串结构 `UNICODE_STRING` 可以定义数组，空间的分配也可以循环进行，例如我们分配十个字符串结构，并输出结构内的参数。

```
#include <ntifs.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

// PowerBy: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    UNICODE_STRING unicode_buffer[10] = { 0 };
    wchar_t * wchar_string = L"hello lyshark";

    DbgPrint("hello lyshark \n");

    int size = sizeof(unicode_buffer) / sizeof(unicode_buffer[0]);
    DbgPrint("数组长度: %d \n", size);

    for (int x = 0; x < size; x++)
    {
        // 分配空间
        unicode_buffer[x].Buffer = (PWSTR)ExAllocatePool(PagedPool, 1024);

        // 设置长度
        unicode_buffer[x].MaximumLength = 1024;
        unicode_buffer[x].Length = wcslen(wchar_string) * sizeof(WCHAR);
        ASSERT(unicode_buffer[x].MaximumLength >= unicode_buffer[x].Length);

        // 拷贝字符串并输出
        RtlCopyMemory(unicode_buffer[x].Buffer, wchar_string,
            unicode_buffer[x].Length);
        unicode_buffer[x].Length = wcslen(wchar_string) * sizeof(WCHAR);
        DbgPrint("循环: %d 输出字符串: %wZ \n", x, unicode_buffer[x]);
    }
}
```

```

// 释放内存
ExFreePool(unicode_buffer[x].Buffer);
unicode_buffer[x].Buffer = NULL;
unicode_buffer[x].Length = unicode_buffer[x].MaximumLength = 0;
}

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
2755	52430.2695...	524650937500 - STORMINI: StorNVMe - POWER: ACTIVE
2756	52430.3007...	hello lyshark
2757	52430.3007...	数组长度: 10
2758	52430.3007...	循环: 0 输出字符串: hello lyshark
2759	52430.3007...	循环: 1 输出字符串: hello lyshark
2760	52430.3007...	循环: 2 输出字符串: hello lyshark
2761	52430.3007...	循环: 3 输出字符串: hello lyshark
2762	52430.3007...	循环: 4 输出字符串: hello lyshark
2763	52430.3007...	循环: 5 输出字符串: hello lyshark
2764	52430.3007...	循环: 6 输出字符串: hello lyshark
2765	52430.3007...	循环: 7 输出字符串: hello lyshark
2766	52430.3007...	循环: 8 输出字符串: hello lyshark
2767	52430.3007...	循环: 9 输出字符串: hello lyshark
2768	52430.3007...	驱动加载成功
2769	52431.2500...	驱动已卸载
2770	52431.2734...	524661093750 - STORMINI: StorNVMe - POWER: IDLE
2771	52431.6250...	524664531250 - STORMINI: StorNVMe - POWER: ACTIVE

实现字符串拷贝，此处可以直接使用 `RtlCopyMemory` 函数直接对内存操作，也可以调用内核提供的 `RtlCopyUnicodeString` 函数来实现，具体代码如下。

```

#include <ntifs.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

// PowerBy: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING unicode_buffer_source = { 0 };
    UNICODE_STRING unicode_buffer_target = { 0 };

    // 该函数可用于初始化字符串
    RtlInitUnicodeString(&unicode_buffer_source, L"hello lyshark");

    // 初始化target字符串,分配空间
    unicode_buffer_target.Buffer = (PWSTR)ExAllocatePool(PagedPool, 1024);
    unicode_buffer_target.MaximumLength = 1024;
}

```

```

// 将source中的内容拷贝到target中
RtlCopyUnicodeString(&unicode_buffer_target, &unicode_buffer_source);

// 输出结果
DbgPrint("source = %wZ \n", &unicode_buffer_source);
DbgPrint("target = %wZ \n", &unicode_buffer_target);

// 释放空间 source 无需销毁
// 如果强制释放掉source则会导致系统蓝屏,因为source是在栈上的
RtlFreeUnicodeString(&unicode_buffer_target);

DbgPrint("驱动加载成功 \n");

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
2807	52642.1445...	526769687500 - STORMINI: StorNVMe - POWER: IDLE
2808	52642.1835...	526770156250 - STORMINI: StorNVMe - POWER: ACTIVE
2809	61509.7773...	615445937500 - STORMINI: StorNVMe - POWER: IDLE
2810	61510.1289...	615449531250 - STORMINI: StorNVMe - POWER: ACTIVE
2811	61511.1328...	615459531250 - STORMINI: StorNVMe - POWER: IDLE
2812	61511.5000...	615463281250 - STORMINI: StorNVMe - POWER: ACTIVE
2813	61512.5156...	615473437500 - STORMINI: StorNVMe - POWER: IDLE
2814	61512.8632...	615476875000 - STORMINI: StorNVMe - POWER: ACTIVE
2815	61512.9414...	hello lyshark
2816	61512.9414...	source = hello lyshark
2817	61512.9414...	target = hello lyshark
2818	61512.9414...	驱动加载成功
2819	61513.8671...	615486875000 - STORMINI: StorNVMe - POWER: IDLE
2820	61513.9296...	615487500000 - STORMINI: StorNVMe - POWER: ACTIVE
2821	61514.0625...	驱动已卸载
2822	61514.9414...	615497656250 - STORMINI: StorNVMe - POWER: IDLE
2823	61514.9414...	615497656250 - STORMINI: StorNVMe - POWER: ACTIVE

实现字符串比较, 如果需要比较两个 UNICODE_STRING 字符串结构体是否相等, 那么可以使用 RtlEqualUnicodeString 这个内核函数实现, 该函数第三个参数是返回值类型, 如果是TRUE则默认返回真, 否则返回假, 具体代码如下。

```

#include <ntifs.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

// PowerBy: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING unicode_buffer_source = { 0 };
}

```

```

UNICODE_STRING unicode_buffer_target = { 0 };

// 该函数可用于初始化字符串
RtlInitUnicodeString(&unicode_buffer_source, L"hello lyshark");
RtlInitUnicodeString(&unicode_buffer_target, L"hello lyshark");

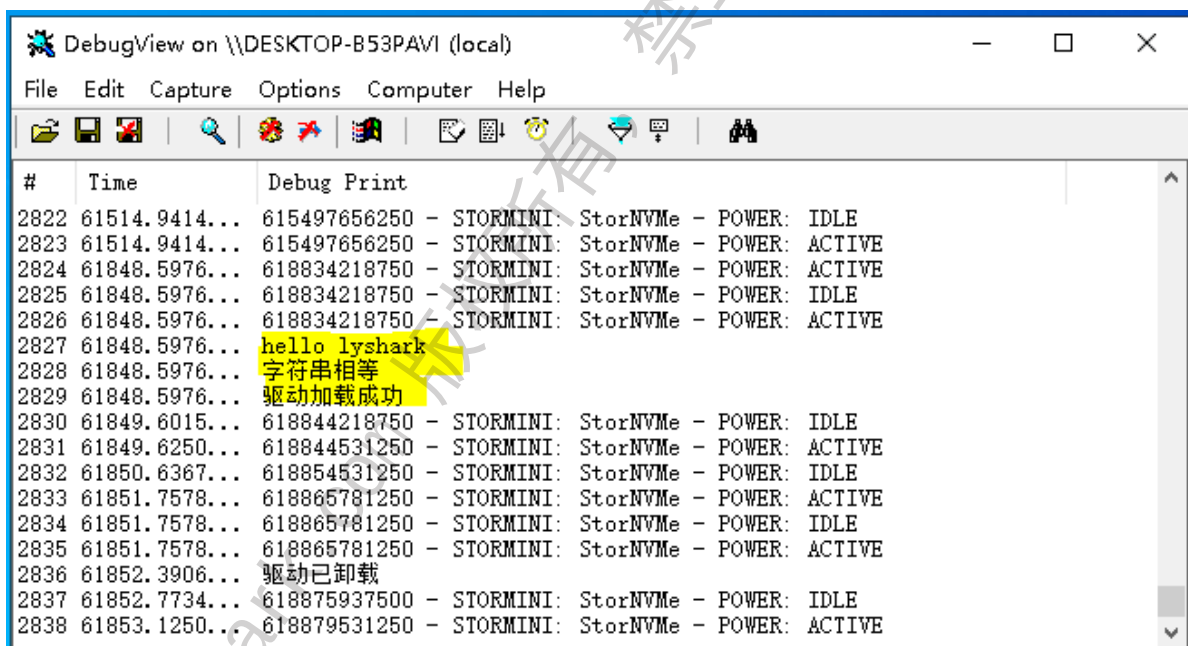
// 比较字符串是否相等
if (RtlEqualUnicodeString(&unicode_buffer_source, &unicode_buffer_target,
TRUE))
{
    DbgPrint("字符串相等 \n");
}
else
{
    DbgPrint("字符串不相等 \n");
}

DbgPrint("驱动加载成功 \n");

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码输出效果：



#	Time	Debug Print
2822	61514.9414...	615497656250 - STORMINI: StorNVMe - POWER: IDLE
2823	61514.9414...	615497656250 - STORMINI: StorNVMe - POWER: ACTIVE
2824	61848.5976...	618834218750 - STORMINI: StorNVMe - POWER: ACTIVE
2825	61848.5976...	618834218750 - STORMINI: StorNVMe - POWER: IDLE
2826	61848.5976...	618834218750 - STORMINI: StorNVMe - POWER: ACTIVE
2827	61848.5976...	hello lyshark
2828	61848.5976...	字符串相等
2829	61848.5976...	驱动加载成功
2830	61849.6015...	618844218750 - STORMINI: StorNVMe - POWER: IDLE
2831	61849.6250...	618844531250 - STORMINI: StorNVMe - POWER: ACTIVE
2832	61850.6367...	618854531250 - STORMINI: StorNVMe - POWER: IDLE
2833	61851.7578...	618865781250 - STORMINI: StorNVMe - POWER: ACTIVE
2834	61851.7578...	618865781250 - STORMINI: StorNVMe - POWER: IDLE
2835	61851.7578...	618865781250 - STORMINI: StorNVMe - POWER: ACTIVE
2836	61852.3906...	驱动已卸载
2837	61852.7734...	618875937500 - STORMINI: StorNVMe - POWER: IDLE
2838	61853.1250...	618879531250 - STORMINI: StorNVMe - POWER: ACTIVE

有时在字符串比较时需要统一字符串格式，例如全部变大写以后在做比较等，此时可以使用 `RtlUppcaseUnicodeString` 函数将小写字符串为大写，然后在做比较，代码如下。

```

#include <ntifs.h>

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

// PowerBy: LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)

```

```

{
    DbgPrint("hello lyshark \n");

    UNICODE_STRING uncode_buffer_source = { 0 };
    UNICODE_STRING uncode_buffer_target = { 0 };

    // 该函数可用于初始化字符串
    RtlInitUnicodeString(&uncode_buffer_source, L"hello lyshark");
    RtlInitUnicodeString(&uncode_buffer_target, L"HELLO LYSHARK");

    // 字符串小写变大写
    RtlUppcaseUnicodeString(&uncode_buffer_target, &uncode_buffer_source, TRUE);
    DbgPrint("小写输出: %wZ \n", &uncode_buffer_source);
    DbgPrint("变大写输出: %wZ \n", &uncode_buffer_target);

    // 销毁字符串
    RtlFreeUnicodeString(&uncode_buffer_target);

    DbgPrint("驱动加载成功 \n");

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

代码输出效果:

#	Time	Debug Print
2836	61852.3906...	驱动已卸载
2837	61852.7734...	618875937500 - STORMINI: StorNVMe - POWER: IDLE
2838	61853.1250...	618879531250 - STORMINI: StorNVMe - POWER: ACTIVE
2839	62121.0039...	621558281250 - STORMINI: StorNVMe - POWER: ACTIVE
2840	62121.0039...	621558281250 - STORMINI: StorNVMe - POWER: IDLE
2841	62121.0039...	621558281250 - STORMINI: StorNVMe - POWER: ACTIVE
2842	62121.1484...	hello lyshark
2843	62121.1484...	小写输出: hello lyshark
2844	62121.1484...	变大写输出: HELLO LYSHARK
2845	62121.1484...	驱动加载成功
2846	62122.0078...	621568281250 - STORMINI: StorNVMe - POWER: IDLE
2847	62122.1367...	621569687500 - STORMINI: StorNVMe - POWER: ACTIVE
2848	62123.1523...	621579843750 - STORMINI: StorNVMe - POWER: IDLE
2849	62125.6718...	驱动已卸载
2850	62126.0039...	621608281250 - STORMINI: StorNVMe - POWER: ACTIVE
2851	62126.0039...	621608281250 - STORMINI: StorNVMe - POWER: IDLE
2852	62126.0039...	621608281250 - STORMINI: StorNVMe - POWER: ACTIVE

第三章：驱动通信篇

3.1 驱动与应用的简单通信

驱动程序与应用程序的通信离不开派遣函数，派遣函数是Windows驱动编程中的重要概念，一般情况下驱动程序负责处理I/O特权请求，而大部分IO的处理请求是在派遣函数中处理的，当用户请求数据时，操作系统会提前处理好请求，并将其派遣到指定的内核函数中执行，接下来将详细说明派遣函数的使用并通过派遣函数读取Shadow SSDT中的内容。

先来简单介绍一下 IRP(I/O Request Package) 输入输出请求包, 该请求包在Windows内核中是一个非常重要的数据结构, 当我们的上层应用与底层的驱动程序通信时, 应用程序就会发出I/O请求, 操作系统将该请求转化为相应的IRP数据, 然后会根据不同的请求数据将请求派遣到相应的驱动函数中执行, 这一点有点类似于Windows的消息机制。

简单的驱动通信: 注册两个派遣函数, 当设备创建的时候触发, 以及关闭时触发。

```
#include <ntddk.h>

VOID UnDriver(PDRIVER_OBJECT pDriver)
{
    PDEVICE_OBJECT pDev;          // 用来取得要删除设备对象
    UNICODE_STRING SymLinkName;    // 局部变量symLinkName

    pDev = pDriver->DeviceObject;
    IoDeleteDevice(pDev);          // 调用
    IoDeleteDevice用于删除设备
    RtlInitUnicodeString(&SymLinkName, L"\\??\\My_Driver"); // 初始化字符串将
    symLinkName定义成需要删除的符号链接名称
    IoDeleteSymbolicLink(&SymLinkName); // 调用
    IoDeleteSymbolicLink删除符号链接
    DbgPrint("删除设备与符号链接成功...");
}

NTSTATUS DispatchCreate(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CREATE 成功执行 !\\n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}

NTSTATUS DispatchClose(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CLOSE 成功执行 !\\n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}

NTSTATUS CreateDriverObject(IN PDRIVER_OBJECT pDriver)
{
    NTSTATUS Status;
    PDEVICE_OBJECT pDevObj;
    UNICODE_STRING DriverName;
    UNICODE_STRING SymLinkName;

    RtlInitUnicodeString(&DriverName, L"\\Device\\My_Device");
    Status = IoCreateDevice(pDriver, 0, &DriverName, FILE_DEVICE_UNKNOWN, 0,
    TRUE, &pDevObj);
    DbgPrint("命令 IoCreateDevice 状态: %d", Status);

    // DO_BUFFERED_IO 设置读写方式 Flags的三个不同的值分别为: DO_BUFFERED_IO、
    DO_DIRECT_IO和0
}
```



```

    pDevObj->Flags |= DO_BUFFERED_IO;
    RtlInitUnicodeString(&SymLinkName, L"\\??\\My_Device");
    Status = IoCreateSymbolicLink(&SymLinkName, &DriverName);
    DbgPrint("当前命令IoCreateSymbolicLink状态: %d", Status);
    return STATUS_SUCCESS;
}

NTSTATUS DriverEntry(PDRIVER_OBJECT pDriver, PUNICODE_STRING RegistryPath)
{
    CreateDriverObject(pDriver);           // 调用创建设备子过程
    // 注册两个派遣函数, 分别对应创建与关闭, 派遣函数名可自定义
    pDriver->MajorFunction[IRP_MJ_CREATE] = DispatchCreate;    // 创建成功派遣函数
    pDriver->MajorFunction[IRP_MJ_CLOSE] = DispatchClose;      // 关闭派遣函数

    DbgPrint("驱动加载完成...");
    pDriver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

客户端代码

```

#include <windows.h>
#include <stdio.h>
#include <winioctl.h>

int main()
{
    HANDLE hDevice = CreateFile(L"\\\\.\\My_Device", GENERIC_READ |
    GENERIC_WRITE, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hDevice == INVALID_HANDLE_VALUE) //判断hDevice返回值是否为空
    {
        printf("获取驱动句柄失败! 错误: %d\\n", GetLastError());
        getchar();
    }

    getchar();
    CloseHandle(hDevice);
    return 0;
}

```

读取驱动中的数据: 实现读取内核缓冲区中的数据, 并打印出来。

```

#include <ntddk.h>

VOID UnDriver(PDRIVER_OBJECT pDriver)
{
    PDEVICE_OBJECT pDev;           // 用来取得要删除设备对象
    UNICODE_STRING SymLinkName;    // 局部变量symLinkName
    pDev = pDriver->DeviceObject;
    IoDeleteDevice(pDev);           // 调用
    // IoDeleteDevice用于删除设备
    RtlInitUnicodeString(&SymLinkName, L"\\??\\My_Driver"); // 初始化字符串将
    // symLinkName定义成需要删除的符号链接名称
}

```



```

        IoDeleteSymbolicLink(&SymLinkName); // 调用
IoDeleteSymbolicLink删除符号链接
        DbgPrint("删除设备与符号链接成功...");
    }
NTSTATUS DispatchCreate(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CREATE 成功执行 !\n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}
NTSTATUS DispatchClose(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CLOSE 成功执行 !\n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}

NTSTATUS DispatchRead(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS Status = STATUS_SUCCESS;
    PIO_STACK_LOCATION Stack = IoGetCurrentIrpStackLocation(pIrp);
    ULONG uIReadLength = Stack->Parameters.Read.Length;
    pIrp->IoStatus.Status = Status;
    pIrp->IoStatus.Information = uIReadLength;
    DbgPrint("应用要读取的长度: %d\n", uIReadLength);

    // 将内核中的缓冲区全部填充为0x68 方便演示读取的效果
    memset(pIrp->AssociatedIrp.SystemBuffer, 0x68, uIReadLength);
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);
    return Status;
}

NTSTATUS CreateDriverObject(IN PDRIVER_OBJECT pDriver)
{
    NTSTATUS Status;
    PDEVICE_OBJECT pDevObj;
    UNICODE_STRING DriverName;
    UNICODE_STRING SymLinkName;

    RtlInitUnicodeString(&DriverName, L"\\Device\\My_Device");
    Status = IoCreateDevice(pDriver, 0, &DriverName, FILE_DEVICE_UNKNOWN, 0,
TRUE, &pDevObj);
    DbgPrint("命令 IoCreateDevice 状态: %d", Status);
    pDevObj->Flags |= DO_BUFFERED_IO;
    RtlInitUnicodeString(&SymLinkName, L"\\??\\My_Device");
    Status = IoCreateSymbolicLink(&SymLinkName, &DriverName);
    DbgPrint("当前命令IoCreateSymbolicLink状态: %d", Status);
    return STATUS_SUCCESS;
}

NTSTATUS DriverEntry(PDRIVER_OBJECT pDriver, PUNICODE_STRING RegistryPath)
{
    CreateDriverObject(pDriver); // 调用创建设备

```

```

pDriver->MajorFunction[IRP_MJ_CREATE] = DispatchCreate;    // 创建成功派遣函数
pDriver->MajorFunction[IRP_MJ_CLOSE] = DispatchClose;      // 关闭派遣函数
pDriver->MajorFunction[IRP_MJ_READ] = DispatchRead;

DbgPrint("驱动加载完成...");
pDriver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

客户端代码

```

#include <windows.h>
#include <stdio.h>
#include <winioctl.h>

int main()
{
    HANDLE hDevice = CreateFile(L"\\\\.\\My_Device", GENERIC_READ |
GENERIC_WRITE, 0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hDevice == INVALID_HANDLE_VALUE)
    {
        printf("获取驱动句柄失败: %d\\n", GetLastError());
        getchar();
    }

    UCHAR buffer[10];
    ULONG ulRead;

    ReadFile(hDevice, buffer, 10, &ulRead, 0);
    for (int i = 0; i < (int)ulRead; i++)
    {
        printf("%02x", buffer[i]);
    }
    getchar();
    CloseHandle(hDevice);
    return 0;
}

```

3.2 应用DeviceIoControl开发模板

内核中执行代码后需要将结果动态显示给应用层的用户，DeviceIoControl 是直接发送控制代码到指定的设备驱动程序，使相应的移动设备以执行相应的操作的函数，如下代码是一个经典的驱动开发模板框架，在开发经典驱动时会用到的一个通用案例。

驱动程序开发通用模板代码如下：

```

#include <ntifs.h>
#include <windef.h>

// 控制器
#define IOCTL_IO_LyShark
CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)

// 卸载驱动执行

```

```

VOID UnDriver(PDRIVER_OBJECT pDriver)
{
    PDEVICE_OBJECT pDev; // 用来取得要删除设备对象
    UNICODE_STRING SymLinkName; // 局部变量
    symLinkName
    pDev = pDriver->DeviceObject;
    IoDeleteDevice(pDev); // 调用
    IoDeleteDevice用于删除设备
    RtlInitUnicodeString(&SymLinkName, L"\\??\\LySharkDriver"); // 初始化字符串
    将symLinkName定义成需要删除的符号链接名称
    IoDeleteSymbolicLink(&SymLinkName); // 调用
    IoDeleteSymbolicLink删除符号链接
    DbgPrint("驱动卸载完毕...");
}

// 创建设备连接
NTSTATUS CreateDriverObject(IN PDRIVER_OBJECT pDriver)
{
    NTSTATUS Status;
    PDEVICE_OBJECT pDevObj;
    UNICODE_STRING DriverName;
    UNICODE_STRING SymLinkName;

    // 创建设备名称字符串
    RtlInitUnicodeString(&DriverName, L"\\Device\\LySharkDriver");
    Status = IoCreateDevice(pDriver, 0, &DriverName, FILE_DEVICE_UNKNOWN, 0,
    TRUE, &pDevObj);

    // 指定通信方式为缓冲区
    pDevObj->Flags |= DO_BUFFERED_IO;

    // 创建符号链接
    RtlInitUnicodeString(&SymLinkName, L"\\??\\LySharkDriver");
    Status = IoCreateSymbolicLink(&SymLinkName, &DriverName);
    return STATUS_SUCCESS;
}

// 创建回调函数
NTSTATUS DispatchCreate(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CREATE 执行 \n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}

// 关闭回调函数
NTSTATUS DispatchClose(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
    DbgPrint("派遣函数 IRP_MJ_CLOSE 执行 \n");
    IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
    return STATUS_SUCCESS; // 返回成功
}

```

```

// 主控制器,用于判断R3发送的控制信号
// lyshark.com
NTSTATUS DispatchIoctl(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS status = STATUS_INVALID_DEVICE_REQUEST;
    PIO_STACK_LOCATION pIrpStack;
    ULONG uIoControlCode;
    PVOID pIoBuffer;
    ULONG uInSize;
    ULONG uOutSize;

    // 获得IRP里的关键数据
    pIrpStack = IoGetCurrentIrpStackLocation(pIrp);

    // 获取控制码
    uIoControlCode = pIrpStack->Parameters.DeviceIoControl.IoControlCode;

    // 输入和输出的缓冲区 (DeviceIoControl的InBuffer和OutBuffer都是它)
    pIoBuffer = pIrp->AssociatedIrp.SystemBuffer;

    // EXE发送传入数据的BUFFER长度 (DeviceIoControl的nInBufferSize)
    uInSize = pIrpStack->Parameters.DeviceIoControl.InputBufferLength;

    // EXE接收传出数据的BUFFER长度 (DeviceIoControl的nOutBufferSize)
    uOutSize = pIrpStack->Parameters.DeviceIoControl.OutputBufferLength;

    // 对不同控制信号的处理流程
    switch (uIoControlCode)
    {
        // 接收或发送
        case IOCTL_IO_LyShark:
        {
            DWORD dw = 0;

            // 得到输入参数
            memcpy(&dw, pIoBuffer, sizeof(DWORD));

            DbgPrint("[+] hello lyshark \n");

            // 对输入参数进行处理
            dw++;

            // 设置输出参数
            memcpy(pIoBuffer, &dw, sizeof(DWORD));

            // 返回通信状态
            status = STATUS_SUCCESS;
            break;
        }

        // 其他控制信号的处理流程
        // ...

    }

    pIrp->IoStatus.Status = status;
    pIrp->IoStatus.Information = uOutSize;
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);
    return status;
}

```

```

}

// 设定DeviceIoControl的*lpBytesReturned的值（如果通信失败则返回0长度）
if (status == STATUS_SUCCESS)
    pIrp->IoStatus.Information = uOutSize;
else
    pIrp->IoStatus.Information = 0;

// 设定DeviceIoControl的返回值是成功还是失败
pIrp->IoStatus.Status = status;
IoCompleteRequest(pIrp, IO_NO_INCREMENT);
return status;
}

// 入口函数
NTSTATUS DriverEntry(PDRIVER_OBJECT pDriver, PUNICODE_STRING RegistryPath)
{
    // 调用创建设备
    CreateDriverObject(pDriver);

    pDriver->DriverUnload = UnDriver; // 卸载函数
    pDriver->MajorFunction[IRP_MJ_CREATE] = DispatchCreate; // 创建派遣函数
    pDriver->MajorFunction[IRP_MJ_CLOSE] = DispatchClose; // 关闭派遣函数
    pDriver->MajorFunction[IRP_MJ_DEVICE_CONTROL] = DispatchIoctl; // 分发函数

    DbgPrint("By:LyShark ...");

    return STATUS_SUCCESS;
}

```

应用层通用测试模板代码如下:

```

#include <iostream>
#include <windows.h>
#include <winioctl.h>

#define IOCTL_IO_LyShark
CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)

int main(int argc, char *argv[])
{
    HANDLE hDevice = CreateFileA("\\\\.\\LySharkDriver", GENERIC_READ |
GENERIC_WRITE, 0,
    NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hDevice == INVALID_HANDLE_VALUE)
    {
        CloseHandle(hDevice);
        return 0;
    }

    // 发送控制信号

    // input = 发送数据 output = 接受数据 ref_len = 数据长度
    DWORD input = 100, output = 0, ref_len = 0;

```

```

DeviceIoControl(hDevice, IOCTL_IO_LyShark, &input, sizeof(input), &output,
sizeof(output), &ref_len, 0);

printf("输出: %d \n", output);

system("pause");
CloseHandle(hDevice);
return 0;
}

```

输出效果如下:

3.3 通过SystemBuf与内核层通信

内核层与应用层之间的数据交互是必不可少的部分，只有内核中的参数可以传递给用户数据才有意义，一般驱动多数情况下会使用 SystemBuf 缓冲区进行通信，也可以直接使用网络套接字实现通信，如下将简单介绍通过SystemBuf实现的内核层与应用层通信机制。

内核与应用层传递结构体，实现应用层用户传入一个结构体到内核，内核处理后返回一段字符串。

内核代码如下，代码已经备注。

```

#include <ntifs.h>
#include <windef.h>

#define My_Code
CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)

// 通信结构体
typedef struct Hread
{
    ULONG Flage;
    ULONG Addr;
    ULONG writeBufferAddr;
    ULONG Size;
    ULONG Pid;
}_Hread, *PtrHread;

typedef struct _DEVICE_EXTENSION
{

```

```

        UNICODE_STRING SymLinkName;
    } DEVICE_EXTENSION, *PDEVICE_EXTENSION;

// 驱动关闭提示
VOID DriverUnload(PDRIVER_OBJECT pDriverObject)
{
    PDEVICE_OBJECT pDevObj;
    pDevObj = pDriverObject->DeviceObject;
    PDEVICE_EXTENSION pDevExt = (PDEVICE_EXTENSION)pDevObj->DeviceExtension;
    UNICODE_STRING pLinkName = pDevExt->SymLinkName;

    IoDeleteSymbolicLink(&pLinkName);
    IoDeleteDevice(pDevObj);
}

// 默认派遣
NTSTATUS DefDispatchRoutine(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS status = STATUS_SUCCESS;
    pIrp->IoStatus.Status = status;
    pIrp->IoStatus.Information = 0;
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);
    return status;
}

// 主派遣函数
NTSTATUS IoctlDispatchRoutine(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS Status = STATUS_UNSUCCESSFUL;
    ULONG_PTR Informaiton = 0;
    PVOID InputData = NULL;
    ULONG InputDataLength = 0;
    PVOID OutputData = NULL;
    ULONG OutputDataLength = 0;
    PIO_STACK_LOCATION IoStackLocation = IoGetCurrentIrpStackLocation(pIrp);

    // Irp堆栈
    InputData = pIrp->AssociatedIrp.SystemBuffer;
    // 输入堆栈
    OutputData = pIrp->AssociatedIrp.SystemBuffer;
    // 输出堆栈
    InputDataLength = IoStackLocation->Parameters.DeviceIoControl.InputBufferLength; // 输入数据大小
    OutputDataLength = IoStackLocation->Parameters.DeviceIoControl.OutputBufferLength; // 输出数据大小
    ULONG Code = IoStackLocation->Parameters.DeviceIoControl.IoControlCode;
    // 控制码

    switch (Code)
    {
        case My_Code:
        {
            PtrHread PtrBuff = (PtrHread)InputData;
            ULONG RetFlage = PtrBuff->Flage;
            ULONG RetAddr = PtrBuff->Addr;
            ULONG RetBufferAddr = PtrBuff->WriteBufferAddr;
        }
    }
}

```

```

        ULONG Size = PtrBuff->Size;
        ULONG Pid = PtrBuff->Pid;

        DbgPrint("读取文件标志: %d", RetFlage);
        DbgPrint("读取写入地址: %x", RetAddr);
        DbgPrint("读取缓冲区大小: %d", RetBufferAddr);
        DbgPrint("读取当前大小: %d", Size);
        DbgPrint("要操作进程PID: %d", Pid);

        // 通过内存返回数据.
        char *retBuffer = "hello lyshark";
        memcpy(OutputData, retBuffer, strlen(retBuffer));
        Informaiton = strlen(retBuffer) + 1;
        Status = STATUS_SUCCESS;

        // 通过内存返回数据,另一种通信方式.
        /*
        PVOID addr = (PVOID)"ok";
        RtlCopyMemory(OutputData, addr, 4);
        Informaiton = 4;
        Status = STATUS_SUCCESS;
        */
        break;
    }
}

pIrp->IoStatus.Status = Status; // 设置IRP完成状态, 会设置用户
模式下的GetLastError
pIrp->IoStatus.Information = Informaiton; // 设置操作的字节
IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 完成IRP, 不增加优先级
return Status;
}

// 驱动入口
NTSTATUS DriverEntry(PDRIVER_OBJECT pDriverObject, PUNICODE_STRING
pRegistryPath)
{
    pDriverObject->DriverUnload = DriverUnload; // 注册驱
    动卸载函数
    pDriverObject->MajorFunction[IRP_MJ_CREATE] = DefDispatchRoutine; // 注册派
    遣函数
    pDriverObject->MajorFunction[IRP_MJ_CLOSE] = DefDispatchRoutine;
    pDriverObject->MajorFunction[IRP_MJ_WRITE] = DefDispatchRoutine;
    pDriverObject->MajorFunction[IRP_MJ_READ] = DefDispatchRoutine;
    pDriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = IoctlDispatchRoutine;

    NTSTATUS status;
    PDEVICE_OBJECT pDevObj;
    PDEVICE_EXTENSION pDevExt;

    // 创建设备名称的字符串
    UNICODE_STRING devName;
    RtlInitUnicodeString(&devName, L"\\Device\\MyDevice");

    // 创建设备

```



```

        status = IoCreateDevice(pDriverObject, sizeof(DEVICE_EXTENSION), &devName,
FILE_DEVICE_UNKNOWN, 0, TRUE, &pDevObj);
        pDevObj->Flags |= DO_BUFFERED_IO; // 将设备设置为缓冲I/O设备

        pDevExt = (PDEVICE_EXTENSION)pDevObj->DeviceExtension; // 得到设备扩展

        // 创建符号链接
        UNICODE_STRING symLinkName;
        RtlInitUnicodeString(&symLinkName, L"\\??\\MyDevice");
        pDevExt->SymLinkName = symLinkName;
        status = IoCreateSymbolicLink(&symLinkName, &devName);
        return STATUS_SUCCESS;
    }

```

客户端代码中只需要通过 `DeviceIoControl()` 发送控制信号即可，需要注意驱动需要安装并运行起来，否则无法获取到数据。

```

#include <windows.h>
#include <iostream>

// 自定义的控制信号
#define My_Code
CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED, FILE_ANY_ACCESS)

// 通信结构体
typedef struct Hread
{
    ULONG Flage;
    ULONG Addr;
    ULONG WriteBufferAddr;
    ULONG Size;
    ULONG Pid;
}_Hread, *PtrHread;

int main(int argc, char* argv[])
{
    // 创建
    HANDLE handle = CreateFileA("\\\\.\\MyDevice", GENERIC_READ | GENERIC_WRITE,
0, NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);

    unsigned char RetBufferData[20] = { 0 };
    DWORD ReturnLength = 4;
    _Hread buf;

    buf.Flage = 2;
    buf.Addr = 0x401234;
    buf.WriteBufferAddr = 1024;
    buf.Size = 100;
    buf.Pid = 2566;

    DeviceIoControl(handle, My_Code, &buf, 20, (LPVOID)RetBufferData, 4,
&ReturnLength, 0);

    for (size_t i = 0; i < 20; i++)
    {

```

```

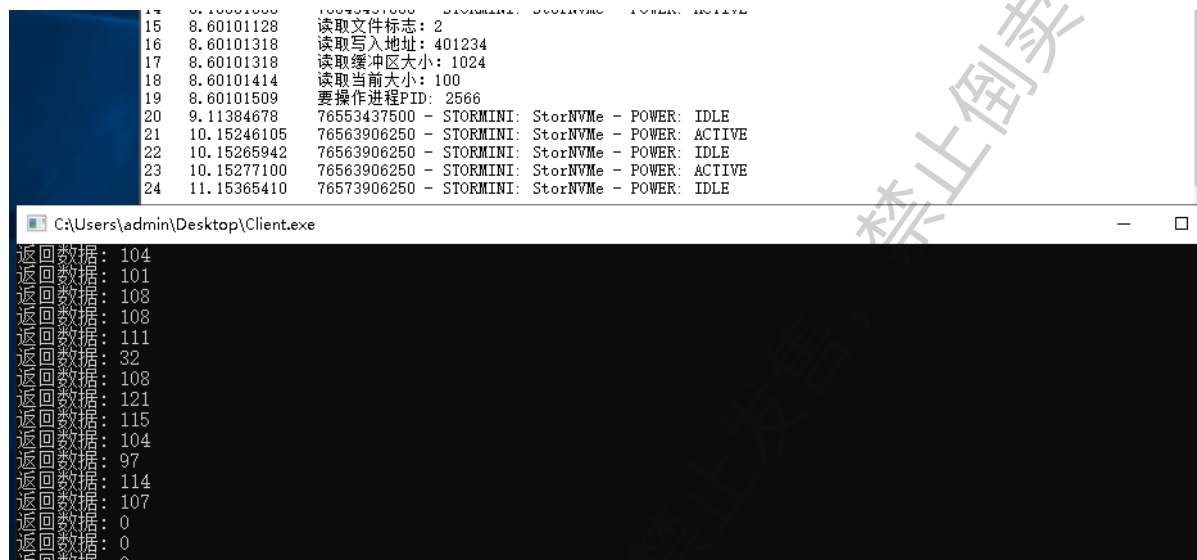
    printf("返回数据: %d \n", RetBufferData[i]);
}

CloseHandle(handle);

getchar();
return 0;
}

```

运行这段代码我们看下返回效果:



```

15  8.60101128  读取文件标志: 2
16  8.60101318  读取写入地址: 401234
17  8.60101318  读取缓冲区大小: 1024
18  8.60101414  读取当前大小: 100
19  8.60101509  要操作进程PID: 2566
20  9.11384678  76553437500 - STORMINI: StorNVMe - POWER: IDLE
21  10.15246105 76563906250 - STORMINI: StorNVMe - POWER: ACTIVE
22  10.15265942 76563906250 - STORMINI: StorNVMe - POWER: IDLE
23  10.15277100 76563906250 - STORMINI: StorNVMe - POWER: ACTIVE
24  11.15365410 76573906250 - STORMINI: StorNVMe - POWER: IDLE

C:\Users\admin\Desktop\Client.exe

返回数据: 104
返回数据: 101
返回数据: 108
返回数据: 108
返回数据: 111
返回数据: 32
返回数据: 108
返回数据: 121
返回数据: 115
返回数据: 104
返回数据: 97
返回数据: 114
返回数据: 107
返回数据: 0
返回数据: 0
返回数据: 0

```

3.4 通过ReadFile与内核层通信

驱动与应用程序的通信是非常有必要的, 内核中执行代码后需要将其动态显示给应用层, 但驱动程序与应用层毕竟不在一个地址空间内, 为了实现内核与应用层数据交互则必须有通信的方法, 微软为我们提供了三种通信方式, 如下先来介绍通过 ReadFile 系列函数实现的通信模式。

长话短说, 不说没用的概念, 首先系统中支持的通信模式可以总结为三种。

- 缓冲区方式读写(DO_BUFFERED_IO)
- 直接方式读写(DO_DIRECT_IO)
- 其他方式读写

而通过 ReadFile, WriteFile 系列函数实现的通信机制则属于 缓冲区通信 模式, 在该模式下操作系统会将应用层中的数据复制到内核中, 此时应用层调用 ReadFile, WriteFile 函数进行读写时, 在驱动内会自动触发 IRP_MJ_READ 与 IRP_MJ_WRITE 这两个派遣函数, 在派遣函数内则可以对收到的数据进行各类处理。

首先需要实现初始化各类派遣函数这么一个案例, 如下代码则是通用的一种初始化派遣函数的基本框架, 分别处理了 IRP_MJ_CREATE 创建派遣, 以及 IRP_MJ_CLOSE 关闭的派遣, 此外函数 DriverDefaultHandle 的作用时初始化其他派遣用的, 也就是将除去 CREATE/CLOSE 这两个派遣之外, 其他的全部赋值成初始值的意思, 当然不增加此段代码也是无妨, 并不影响代码的实际执行。

```

#include <ntifs.h>

// 卸载驱动执行
VOID UnDriver(PDRIVER_OBJECT pDriver)
{

```

```

        PDEVICE_OBJECT pDev; // 用来取得要删除设备对象
        UNICODE_STRING SymLinkName; // 局部变量
    symLinkName
        pDev = pDriver->DeviceObject;
        IoDeleteDevice(pDev); // 调用
    IoDeleteDevice用于删除设备
        RtlInitUnicodeString(&SymLinkName, L"\\??\\LySharkDriver"); // 初始化字符串
    将symLinkName定义成需要删除的符号链接名称
        IoDeleteSymbolicLink(&SymLinkName); // 调用
    IoDeleteSymbolicLink删除符号链接
        DbgPrint("驱动卸载完毕...");
    }

    // 创建设备连接
    // LyShark.com
    NTSTATUS CreateDriverObject(IN PDRIVER_OBJECT pDriver)
    {
        NTSTATUS Status;
        PDEVICE_OBJECT pDevObj;
        UNICODE_STRING DriverName;
        UNICODE_STRING SymLinkName;

        // 创建设备名称字符串
        RtlInitUnicodeString(&DriverName, L"\\Device\\LySharkDriver");
        Status = IoCreateDevice(pDriver, 0, &DriverName, FILE_DEVICE_UNKNOWN, 0,
        TRUE, &pDevObj);

        // 指定通信方式为缓冲区
        pDevObj->Flags |= DO_BUFFERED_IO;

        // 创建符号链接
        RtlInitUnicodeString(&SymLinkName, L"\\??\\LySharkDriver");
        Status = IoCreateSymbolicLink(&SymLinkName, &DriverName);
        return STATUS_SUCCESS;
    }

    // 创建回调函数
    NTSTATUS DispatchCreate(PDEVICE_OBJECT pDevObj, PIRP pIrp)
    {
        pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
        DbgPrint("派遣函数 IRP_MJ_CREATE 执行 \\n");
        IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
        return STATUS_SUCCESS; // 返回成功
    }

    // 关闭回调函数
    NTSTATUS DispatchClose(PDEVICE_OBJECT pDevObj, PIRP pIrp)
    {
        pIrp->IoStatus.Status = STATUS_SUCCESS; // 返回成功
        DbgPrint("派遣函数 IRP_MJ_CLOSE 执行 \\n");
        IoCompleteRequest(pIrp, IO_NO_INCREMENT); // 指示完成此IRP
        return STATUS_SUCCESS; // 返回成功
    }
}

```

```

// 默认派遣函数
NTSTATUS DriverDefaultHandle(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS status = STATUS_SUCCESS;
    pIrp->IoStatus.Status = status;
    pIrp->IoStatus.Information = 0;
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);

    return status;
}

// 入口函数
// By: LyShark
NTSTATUS DriverEntry(PDRIVER_OBJECT pDriver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 调用创建设备
    CreateDriverObject(pDriver);

    pDriver->DriverUnload = UnDriver; // 卸载函数
    pDriver->MajorFunction[IRP_MJ_CREATE] = DispatchCreate; // 创建派遣函数
    pDriver->MajorFunction[IRP_MJ_CLOSE] = DispatchClose; // 关闭派遣函数

    // 初始化其他派遣
    for (ULONG i = 0; i < IRP_MJ_MAXIMUM_FUNCTION; i++)
    {
        DbgPrint("初始化派遣: %d \n", i);
        pDriver->MajorFunction[i] = DriverDefaultHandle;
    }

    DbgPrint("驱动加载完成...");

    return STATUS_SUCCESS;
}

```

代码运行效果如下:

DebugView on \\DESKTOP-B53PAV1 (local)

#	Time	Debug Print
2999	156186.390...	1562212187500 - STORMINI: StorNVMe - POWER: ACTIVE
3000	156186.390...	hello lyshark
3001	156186.390...	初始化派遣: 0
3002	156186.390...	初始化派遣: 1
3003	156186.390...	初始化派遣: 2
3004	156186.390...	初始化派遣: 3
3005	156186.390...	初始化派遣: 4
3006	156186.390...	初始化派遣: 5
3007	156186.390...	初始化派遣: 6
3008	156186.390...	初始化派遣: 7
3009	156186.390...	初始化派遣: 8
3010	156186.390...	初始化派遣: 9
3011	156186.390...	初始化派遣: 10
3012	156186.390...	初始化派遣: 11
3013	156186.390...	初始化派遣: 12
3014	156186.390...	初始化派遣: 13
3015	156186.390...	初始化派遣: 14

通用框架有了，接下来就是让该驱动支持使用 ReadWrite 的方式实现通信，首先我们需要在 DriverEntry 处增加两个派遣处理函数的初始化。

```
// 入口函数
// By: LyShark
NTSTATUS DriverEntry(PDRIVER_OBJECT pDriver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 调用创建设备
    CreateDriverObject(pDriver);

    // 初始化其他派遣
    for (ULONG i = 0; i < IRP_MJ_MAXIMUM_FUNCTION; i++)
    {
        DbgPrint("初始化派遣: %d \n", i);
        pDriver->MajorFunction[i] = DriverDefaultHandle;
    }

    pDriver->DriverUnload = UnDriver; // 卸载函数
    pDriver->MajorFunction[IRP_MJ_CREATE] = DispatchCreate; // 创建派遣函数
    pDriver->MajorFunction[IRP_MJ_CLOSE] = DispatchClose; // 关闭派遣函数

    // 增加派遣处理
    pDriver->MajorFunction[IRP_MJ_READ] = DispatchRead; // 读取派遣函数
    pDriver->MajorFunction[IRP_MJ_WRITE] = DispatchWrite; // 写入派遣函数

    DbgPrint("驱动加载完成...");

    return STATUS_SUCCESS;
}
```

接着，我们需要分别实现这两个派遣处理函数，如下 DispatchRead 负责读取时触发，与之对应 DispatchWrite 负责写入触发。

- 引言：
- 对于读取请求 I/O 管理器 分配一个与用户模式的缓冲区大小相同的系统缓冲区 SystemBuffer，当完成请求时 I/O 管理器将驱动程序已经提供的数据从系统缓冲区复制到用户缓冲区。
- 对于写入请求，会分配一个系统缓冲区并将 SystemBuffer 设置为地址，用户缓冲区的内容会被复制到系统缓冲区，但是不设置 UserBuffer 缓冲。

通过 IoGetCurrentIrpStackLocation(pIrp) 接收读写请求长度，偏移等基本参数，AssociatedIrp.SystemBuffer 则是读写缓冲区，IoStatus.Information 是输出缓冲字节数，Parameters.Read.Length 是读取写入的字节数。

```
// 读取回调函数
NTSTATUS DispatchRead(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS Status = STATUS_SUCCESS;
    PIO_STACK_LOCATION Stack = IoGetCurrentIrpStackLocation(pIrp);
    ULONG ulReadLength = Stack->Parameters.Read.Length;

    char szBuf[128] = "hello lyshark";
```

```

pIrp->IoStatus.Status = Status;
pIrp->IoStatus.Information = ulReadLength;
DbgPrint("读取长度: %d \n", ulReadLength);

// 取出字符串前5个字节返回给R3层
memcpy(pIrp->AssociatedIrp.SystemBuffer, szBuf, ulReadLength);

IoCompleteRequest(pIrp, IO_NO_INCREMENT);
return Status;
}

// 接收传入回调函数
// By: LyShark
NTSTATUS DispatchWrite(struct _DEVICE_OBJECT *DeviceObject, struct _IRP *Irp)
{
    NTSTATUS Status = STATUS_SUCCESS;
    PIO_STACK_LOCATION Stack = IoGetCurrentIrpStackLocation(Irp);
    ULONG ulWriteLength = Stack->Parameters.Write.Length;
    PVOID ulWriteData = Irp->AssociatedIrp.SystemBuffer;

    // 输出传入字符串
    DbgPrint("传入长度: %d 传入数据: %s \n", ulWriteLength, ulWriteData);

    IoCompleteRequest(Irp, IO_NO_INCREMENT);
    return Status;
}

```

如上部分都是在讲解驱动层面的读写派遣，应用层还没有介绍，在应用层我们只需要调用 `ReadFile` 函数当调用该函数时驱动中会使用 `DispatchRead` 派遣例程来处理这个请求，同理调用 `WriteFile` 函数则触发的是 `DispatchWrite` 派遣例程。

我们首先从内核中读出前五个字节并放入缓冲区内，输出该缓冲区内的数据，然后在调用写入，将 `hello lyshark` 写回到内核里里面，这段代码可以这样来写。

```

#include <iostream>
#include <windows.h>
#include <winioctl.h>

int main(int argc, char *argv[])
{
    HANDLE hDevice = CreateFileA("\\\\.\\LySharkDriver", GENERIC_READ |
    GENERIC_WRITE, 0,
    NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
    if (hDevice == INVALID_HANDLE_VALUE)
    {
        CloseHandle(hDevice);
        return 0;
    }

    // 从内核读取数据到本地
    char buffer[128] = { 0 };
    ULONG length;

    // 读入到buffer长度为5
    // By: lyshark.com

```

```

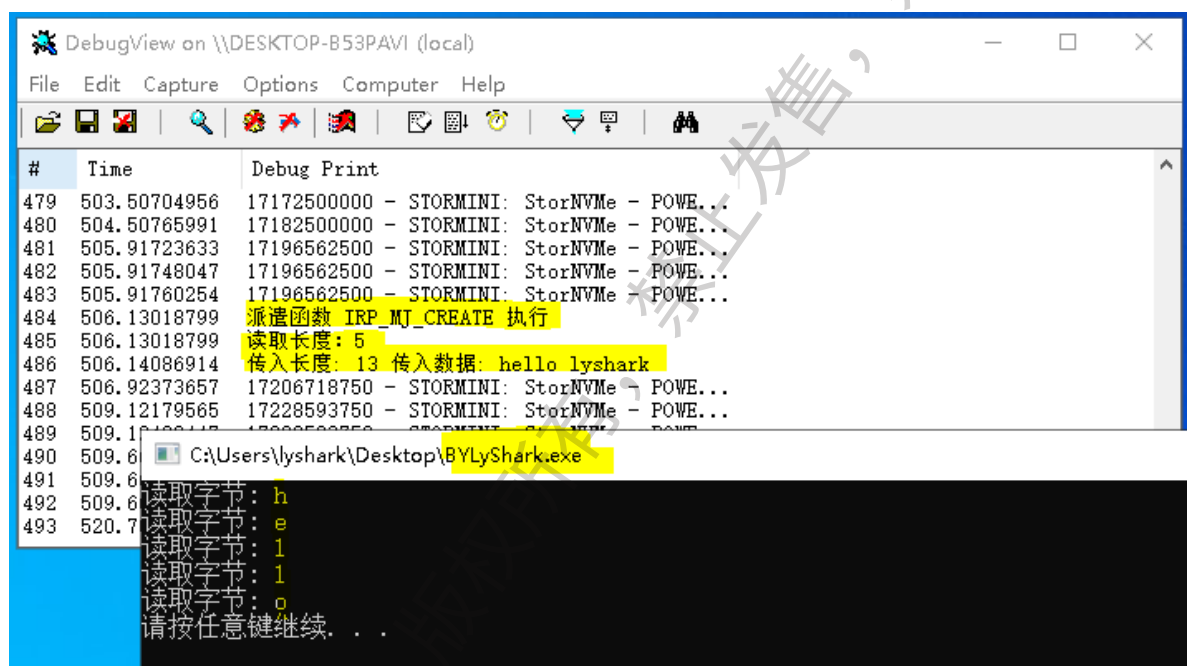
ReadFile(hDevice, buffer, 5, &length, 0);
for (int i = 0; i < (int)length; i++)
{
    printf("读取字节: %c", buffer[i]);
}

// 写入数据到内核
char write_buffer[128] = "hello lyshark";
ULONG write_length;
WriteFile(hDevice, write_buffer, strlen(write_buffer), &write_length, 0);

system("pause");
CloseHandle(hDevice);
return 0;
}

```

使用驱动工具安装我们的驱动，然后运行该应用层程序，实现通信，效果如下所示：



3.5 通过PIPE管道与内核层通信

在本人前一篇博文《驱动开发：通过ReadFile与内核层通信》详细介绍了如何使用应用层 ReadFile 系列函数实现内核通信，本篇将继续延伸这个知识点，介绍利用 PIPE 命名管道实现应用层与内核层之间的多次通信方法。

- 什么是PIPE管道？

在Windows编程中，数据重定向需要用到管道PIPE，管道是一种用于在进程间共享数据的机制，通常由两端组成，数据从一端流入则必须从另一端流出，也就是一读一写，利用这种机制即可实现进程间直接通信。管道的本质其实是一段共享内存区域，多数情况下管道是用于应用层之间的数据交换的，其实驱动中依然可以使用命名管道实现应用层与内核层的直接通信。

那么如何在内核中创建一个管道？请看以下代码片段，以及MSDN针对函数的解析。

- InitializeObjectAttributes
 - 初始化一个 OBJECT_ATTRIBUTES 结构，它设置将被打开的对象句柄的属性。然后调用方可以将一个指向该结构的指针传递给实际打开句柄的例程。
- ZwCreateFile

- 该函数的作用是创建或打开一个已经存在的文件，在这里其实是打开 `objAttr` 这个文件。
- `KeInitializeEvent`
 - 将事件对象初始化为同步 (单个服务) 或通知类型事件，并将其设置为已发出信号或未发出信号的状态。

```
HANDLE g_hClient;
IO_STATUS_BLOCK g_ioStatusBlock;
KEVENT g_event;

VOID NdisMSleep(IN ULONG MicrosecondsToSleep);

// 初始化管道
void init()
{
    UNICODE_STRING uniName;
    OBJECT_ATTRIBUTES objAttr;

    RtlInitUnicodeString(&uniName, L"\\DosDevices\\Pipe\\LySharkPipeConn");
    InitializeObjectAttributes(&objAttr, &uniName, OBJ_CASE_INSENSITIVE |
OBJ_KERNEL_HANDLE, NULL, NULL);

    ZwCreateFile(&g_hClient, GENERIC_READ | GENERIC_WRITE, &objAttr,
&g_ioStatusBlock, NULL, FILE_ATTRIBUTE_NORMAL, 0, FILE_OPEN,
FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0);
    if (!g_hClient)
    {
        return;
    }
    KeInitializeEvent(&g_event, SynchronizationEvent, TRUE);
}
```

原理就是打开 `\\DosDevices\\Pipe\\LySharkPipeConn` 文件，然后将事件对象初始化为同步状态。

接下来就是如何将数据发送给应用层的问题，发送问题可以调用 `zwwriteFile` 这个内核函数，如下我们实现的效果是将一个 `char` 类型的字符串传输给应用层。

```
// 将数据传到R3应用层
// LyShark
VOID ReportToR3(char* m_parameter, int lent)
{
    if (!NT_SUCCESS(ZwwriteFile(g_hClient, NULL, NULL, NULL, &g_ioStatusBlock,
(void*)m_parameter, lent, NULL, NULL)))
    {
        DbgPrint("写出错误");
    }
}
```

内核层的核心代码就是如上这些，将这些整合在一起完整代码如下所示：

```
#include <ntifs.h>
#include <ndis.h>
#include <stdio.h>
```



```

HANDLE g_hClient;
IO_STATUS_BLOCK g_ioStatusBlock;
KEVENT g_event;

VOID NdisMSleep(IN ULONG MicrosecondsToSleep);

// 初始化管道
void init()
{
    UNICODE_STRING uniName;
    OBJECT_ATTRIBUTES objAttr;

    RtlInitUnicodeString(&uniName, L"\\DosDevices\\Pipe\\LySharkPipeConn");
    InitializeObjectAttributes(&objAttr, &uniName, OBJ_CASE_INSENSITIVE |
OBJ_KERNEL_HANDLE, NULL, NULL);

    ZwCreateFile(&g_hClient, GENERIC_READ | GENERIC_WRITE, &objAttr,
&g_ioStatusBlock, NULL, FILE_ATTRIBUTE_NORMAL, 0, FILE_OPEN,
FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0);
    if (!g_hClient)
    {
        return;
    }
    KeInitializeEvent(&g_event, SynchronizationEvent, TRUE);
}

// 将数据传到R3应用层
// LyShark
VOID ReportToR3(char* m_parameter, int lent)
{
    if (!NT_SUCCESS(ZwWriteFile(g_hClient, NULL, NULL, NULL, &g_ioStatusBlock,
(void*)m_parameter, lent, NULL, NULL)))
    {
        DbgPrint("写出错误");
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    init();

    // 延时3秒
    NdisMSleep(3000000);

    DbgPrint("hello lyshark \n");
    for (int x = 0; x < 10; x++)
    {
        // 分配空间
        char *report = (char*)ExAllocatePoolWithTag(NonPagedPool, 4096, 'lysh');
        if (report)

```

```

    {
        RtlZeroMemory(report, 4096);

        RtlCopyMemory(report, "hello lyshark", 13);

        // 发送到应用层
        ReportToR3(report, 4096);
        ExFreePool(report);
    }
}

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

内核中创建了命名管道，客户端就需要创建一个相同名称的管道，并通过 ReadFile 函数读取管道中的数据，应用层核心代码如下所示：

```

#include <iostream>
#include <windows.h>

int main(int argc, char *argv[])
{
    HANDLE hPipe = CreateNamedPipe(TEXT("\\\\.\\Pipe\\LySharkPipeConn"),
    PIPE_ACCESS_DUPLEX | FILE_FLAG_OVERLAPPED, PIPE_TYPE_MESSAGE |
    PIPE_READMODE_MESSAGE | PIPE_WAIT, PIPE_UNLIMITED_INSTANCES, 0, 0,
    NMPWAIT_WAIT_FOREVER, NULL);
    if (INVALID_HANDLE_VALUE == hPipe)
    {
        return false;
    }

    const int size = 1024 * 10;
    char buf[size];
    DWORD rlen = 0;
    while (true)
    {
        //if (ConnectNamedPipe(hPipe, NULL) != NULL)
        // PowerBy: LyShark.com
        if (1)
        {
            if (ReadFile(hPipe, buf, size, &rlen, NULL) == FALSE)
            {
                continue;
            }
            else
            {
                //接收信息
                char* buffer_tmp = (char*)&buf;

                // 拷贝前半部分,不包括 buffer_data
                char* buffer = (char*)malloc(size);
                memcpy(buffer, buffer_tmp, size);
            }
        }
    }
}

```

```

        printf("内核层数据: %s \n", buffer);

        free(buffer_tmp);
        free(buffer);
    }
}

system("pause");
return 0;
}

```

至此将驱动签名后运行，并迅速打开应用层程序等待同步发送事件，即可得到如下返回结果。

此处有必要解释一下为什么会写出错误，很简单这段代码并没有控制何时触发事件，导致两边不同步，因为只是一个案例用于演示管道的应用方法，所以大家不要太较真，如果不想出错误这段代码还有很多需要改进的地方。

The screenshot shows the DebugView application window. The 'Debug Print' pane displays a list of events with columns for line number, time, and the debug print message. The messages show a sequence of 'hello lyshark' and system status updates from '3635468750 - STORMINI: StorNVMe - POWER: IDLE' and '3645468750 - STORMINI: StorNVMe - POWER: ACTIVE'. A blue highlight is on line 138, which contains a repeated error message: '写出错误写出错误写出错误写出错误写出错误写出错误写出...'. Below the DebugView window, a console window titled 'C:\Users\lyshark\Desktop\ConsoleApplication1.exe' shows the output '内核层数据: hello lyshark'.

管道不仅可以传输字符串完全可以传输结构体数据，如下我们定义一个 Networkreport 结构体，并通过管道的方式多次传输给应用层，这部分传输模式适合用于驱动中一次性突出多个结构体，例如进程列表的输出，ARK工具中的驱动列表输出等功能的实现。

驱动层完整代码

```

#include <ntifs.h>
#include <ndis.h>
#include <stdio.h>

HANDLE g_hClient;
IO_STATUS_BLOCK g_ioStatusBlock;
KEVENT g_event;

typedef struct
{
    int type;
    unsigned long address;
    unsigned long buffer_data_len;
    char buffer_data[0];
}Networkreport;

```

```

VOID NdisMSleep(IN ULONG MicrosecondsToSleep);

// 初始化管道
void init()
{
    UNICODE_STRING uniName;
    OBJECT_ATTRIBUTES objAttr;

    RtlInitUnicodeString(&uniName, L"\\DosDevices\\Pipe\\LySharkPipeConn");
    InitializeObjectAttributes(&objAttr, &uniName, OBJ_CASE_INSENSITIVE |
OBJ_KERNEL_HANDLE, NULL, NULL);

    ZwCreateFile(&g_hClient, GENERIC_READ | GENERIC_WRITE, &objAttr,
&g_ioStatusBlock, NULL, FILE_ATTRIBUTE_NORMAL, 0, FILE_OPEN,
FILE_SYNCHRONOUS_IO_NONALERT, NULL, 0);
    if (!g_hClient)
    {
        return;
    }
    KeInitializeEvent(&g_event, SynchronizationEvent, TRUE);
}

// 将数据传到R3应用层
// PowerBy: LyShark.com
VOID ReportToR3(Networkreport* m_parameter, int lent)
{
    if (!NT_SUCCESS(ZwwriteFile(g_hClient, NULL, NULL, NULL, &g_ioStatusBlock,
(void*)m_parameter, lent, NULL, NULL)))
    {
        DbgPrint("写出错误");
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    init();

    // 延时3秒
    NdisMSleep(3000000);
    DbgPrint("hello lyshark \n");

    for (int x = 0; x < 10; x++)
    {
        // 分配空间
        Networkreport *report = (Networkreport*)ExAllocatePoolWithTag(NonPagedPool,
4096, 'lysh');
        if (report)
        {
            RtlZeroMemory(report, 4096);
        }
    }
}

```

```

report->type = x;
report->address = 401000 + x;
report->buffer_data_len = 13;

// 定位到结构体最后一个元素上
unsigned char * tmp = (unsigned char *)report + sizeof(Networkreport);
memcpy(tmp, "hello lyshark", 13);

// 发送到应用层
ReportToR3(report, 4096);
ExFreePool(report);
}
}

DbgPrint("驱动加载成功 \n");
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

应用层完整代码

```

#include <iostream>
#include <windows.h>

typedef struct
{
    int type;
    unsigned long address;
    unsigned long buffer_data_len;
    char *buffer_data;
}Networkreport;

int main(int argc, char *argv[])
{
    HANDLE hPipe = CreateNamedPipe(TEXT("\\\\.\\Pipe\\LySharkPipeConn"),
    PIPE_ACCESS_DUPLEX | FILE_FLAG_OVERLAPPED, PIPE_TYPE_MESSAGE |
    PIPE_READMODE_MESSAGE | PIPE_WAIT, PIPE_UNLIMITED_INSTANCES, 0, 0,
    NMPWAIT_WAIT_FOREVER, NULL);
    if (INVALID_HANDLE_VALUE == hPipe)
    {
        return false;
    }

    const int size = 1024 * 10;
    char buf[size];
    DWORD rlen = 0;
    while (true)
    {
        //if (ConnectNamedPipe(hPipe, NULL) != NULL)
        if (1 == 1)
        {
            if (ReadFile(hPipe, buf, size, &rlen, NULL) == FALSE)
            {
                continue;
            }
        }
    }
}

```

```

else
{
    //接收信息
    Networkreport* buffer_tmp = (Networkreport*)&buf;
    SIZE_T buffer_len = sizeof(Networkreport) + buffer_tmp-
>buffer_data_len;

    // 拷贝前半部分,不包括 buffer_data
    Networkreport* buffer = (Networkreport*)malloc(buffer_len);
    memcpy(buffer, buffer_tmp, buffer_len);

    // 对后半部 分配空间
    // By: LyShark
    char* data = (char*)malloc(buffer->buffer_data_len);
    unsigned char* tmp = (unsigned char *)buffer +
sizeof(Networkreport) - 4;
    memcpy(data, tmp, buffer->buffer_data_len);

    printf("输出数据: %s \n", data);
    printf("地址: %d \n", buffer_tmp->address);
    printf("长度: %d \n", buffer_tmp->type);
    printf("输出长度: %d \n", buffer_tmp->buffer_data_len);

    free(data);
    free(buffer);
}
}
}
system("pause");
return 0;
}

```

结构体一次性输出多个，效果如下所示：

#	Time	Debug Print
217	880.31048584	hello lyshark
218	880.31652832	驱动加载成功
219	880.7702	命令提示符 - lyshark.exe
220	881.4118	
221	881.4121	
222	881.4121	C:\Users\lyshark\Desktop>
223	882.4194	
224	882.6598	
225	883.6674	C:\Users\lyshark\Desktop>lyshark.exe
226	892.4405	输出数据: hello lyshark
227	892.6712	地址: 401000
228	893.6800	长度: 0
229	893.6801	输出长度: 13
1	929.2788	输出数据: hello lyshark
2	930.2807	地址: 401001
		长度: 1
		输出长度: 13
		输出数据: hello lyshark
		地址: 401002
		长度: 2
		输出长度: 13

3.6 通过Async反向与内核通信

在前几篇文章中给大家具体解释了驱动与应用层之间正向通信的一些经典案例，本章将继续学习驱动通信，不过这次我们学习的是通过运用 Async 异步模式实现的反向通信，反向通信机制在开发中时常被用到，例如一个杀毒软件如果监控到有异常进程运行或有异常注册表被改写后，该驱动需要主动的通知应用层进程让其知道，这就需要用到驱动反向通信的相关知识点，如下将循序渐进的实现一个反向通信案例。

在开始学习Async反向通信之前先来研究一个Sync正向通信案例，不论是正向反向通信其在通信模式上与《驱动开发：通过ReadFile与内核层通信》所介绍的通信模式基本一致，都是通过 ReadFile 触发驱动中的 IRP_MJ_READ 读取派遣，唯一的区别是在传输数据时使用了 MmGetSystemAddressForMdl 方式，它将给定 MDL 描述的物理页面映射到系统空间，并调用 RtlCopyMemory() 将全局字符串复制到这个空间内，这样客户端就可以循环读取内核传出的数据。

我们来看驱动端代码是如何实现的这个功能，代码并没有什么特殊的无法理解的点，只是需要注意我们在驱动入口调用 IoCreateDevice() 时传入了第二个参数 FILE_DEVICE_EXTENSION，该参数的作用是，创建设备时，指定设备扩展内存的大小，传一个值进去，就会给设备分配一块非页面内存。

```
#include <ntddk.h>
#include <stdio.h>

// 保存一段非分页内存,用于给全局变量使用
#define FILE_DEVICE_EXTENSION 4096

// 定义全局字符串
static int global_count = 0;
static char global_char[5][128] = { 0 };

// 驱动绑定默认派遣函数
NTSTATUS _DefaultDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    _pIrp->IoStatus.Status = STATUS_NOT_SUPPORTED;
    _pIrp->IoStatus.Information = 0;
    IoCompleteRequest(_pIrp, IO_NO_INCREMENT);
    return _pIrp->IoStatus.Status;
}

// 驱动创建后触发
NTSTATUS _SyncCreateCloseDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    _pIrp->IoStatus.Status = STATUS_SUCCESS;
    _pIrp->IoStatus.Information = 0;
    IoCompleteRequest(_pIrp, IO_NO_INCREMENT);
    return _pIrp->IoStatus.Status;
}

// 应用层读数据后触发
NTSTATUS _SyncReadDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    NTSTATUS status = STATUS_SUCCESS;
    PIO_STACK_LOCATION pIrpStack = IoGetCurrentIrpStackLocation(_pIrp);
    PVOID pBuffer = NULL;
    ULONG uBufferLen = 0;
```

```

do
{
    // 读写请求使用的是直接I/O方式
    pBuffer = MmGetSystemAddressForMdl(_pIrp->MdlAddress);
    if (pBuffer == NULL)
    {
        status = STATUS_UNSUCCESSFUL;
        break;
    }
    uBufferLen = pIrpStack->Parameters.Read.Length;
    DbgPrint("读字节长度: %d \n", uBufferLen);

    // 最大支持20字节读请求
    uBufferLen = uBufferLen >= 20 ? 20 : uBufferLen;

    // 输出五次字符串
    if (global_count < 5)
    {
        RtlCopyMemory(pBuffer, global_char[global_count], uBufferLen);
        global_count = global_count + 1;
    }

} while (FALSE);

// 填写返回状态及返回大小
_pIrp->IoStatus.Status = status;
_pIrp->IoStatus.Information = uBufferLen;

// 完成IRP
IoCompleteRequest(_pIrp, IO_NO_INCREMENT);
return status;
}

// 卸载驱动
VOID _UnloadDispatch(PDRIVER_OBJECT _pDriverObject)
{
    // 删除创建的设备
    UNICODE_STRING win32DeviceName;
    RtlInitUnicodeString(&win32DeviceName, L"\\DosDevices\\LySharkSync");
    IoDeleteDevice(_pDriverObject->DeviceObject);
}

// 驱动入口
NTSTATUS DriverEntry(PDRIVER_OBJECT _pDriverObject, PUNICODE_STRING
_pRegistryPath)
{
    UNICODE_STRING DeviceName, win32DeivceName;
    PDEVICE_OBJECT pDeviceObject = NULL;
    NTSTATUS status;
    HANDLE hThread;
    OBJECT_ATTRIBUTES ObjectAttributes;

    // 设置符号名
    RtlInitUnicodeString(&DeviceName, L"\\Device\\LySharkSync");
    RtlInitUnicodeString(&win32DeivceName, L"\\DosDevices\\LySharkSync");

```



```

// 循环初始化IRP函数
for (ULONG i = 0; i <= IRP_MJ_MAXIMUM_FUNCTION; i++)
{
    _pDriverObject->MajorFunction[i] = _DefaultDispatch;
}

// 再次覆盖派遣函数
_pDriverObject->MajorFunction[IRP_MJ_CREATE] = _SyncCreateCloseDispatch;
_pDriverObject->MajorFunction[IRP_MJ_CLOSE] = _SyncCreateCloseDispatch;
_pDriverObject->MajorFunction[IRP_MJ_READ] = _SyncReadDispatch;
_pDriverObject->DriverUnload = _UnloadDispatch;

// 分配一个自定义扩展 大小为sizeof (DEVEXT)
// By: LyShark.com
status = IoCreateDevice(_pDriverObject, sizeof(FILE_DEVICE_EXTENSION),
&DeviceName, FILE_DEVICE_UNKNOWN, 0, FALSE, &pDeviceObject);
if (!NT_SUCCESS(status))
    return status;

if (!pDeviceObject)
    return STATUS_UNEXPECTED_IO_ERROR;

// 为全局变量赋值
strcpy(global_char[0], "hi, lyshark A");
strcpy(global_char[1], "hi, lyshark B");
strcpy(global_char[2], "hi, lyshark C");
strcpy(global_char[3], "hi, lyshark D");
strcpy(global_char[4], "hi, lyshark E");

// 指定读写方式为 直接I/O MDL模式
pDeviceObject->Flags |= DO_DIRECT_IO;

// 数据传输时地址校验大小
pDeviceObject->AlignmentRequirement = FILE_WORD_ALIGNMENT;
status = IoCreateSymbolicLink(&win32DeviceName, &DeviceName);

pDeviceObject->Flags &= ~DO_DEVICE_INITIALIZING;
return STATUS_SUCCESS;
}

```

对于应用层来说并没有什么特别的，同样调用 `ReadFile` 读取内核中的参数，同样for循环读取五次，代码如下：

```

#include <stdio.h>
#include <windows.h>

int main(int argc, char *argv[])
{
    HANDLE hFile;
    char Buffer[10] = { 0 };
    DWORD dwRet = 0;
    BOOL bRet;

```

```

hFile = CreateFileA("\\\\.\\LySharkSync", GENERIC_READ, FILE_SHARE_READ,
NULL, OPEN_EXISTING, FILE_ATTRIBUTE_NORMAL, NULL);
if (hFile == INVALID_HANDLE_VALUE)
    return 0;

for (int x = 0; x < 5; x++)
{
    bRet = ReadFile(hFile, Buffer, 20, &dwRet, NULL);
    if (!bRet)
    {
        CloseHandle(hFile);
        return 0;
    }
    printf("读入数据: %s -> 读取长度: %d \n", Buffer, dwRet);
}
return 0;
}

```

这段代码运行效果如下:

The screenshot shows a DebugView window titled 'DebugView on \\DESKTOP-B53PAVI (local)'. The main pane displays a list of system events. Two events from 'STORNTFS: StorNVMe' are highlighted: one with 'POWER: ACTIVE' and another with 'POWER: IDLE'. Below the logs, a command prompt window is open, showing the command 'C:\Users\lyshark\Desktop>lyshark.exe' and its output, which consists of five lines of '读入数据: -> 读取长度: 20'.

与同步模式不同, 异步模式 虽然同样使用 `ReadFile` 实现通信, 但在通信中引入了 `Event` 事件通知机制, 这也是异步与同步最大的区别所在, 用户层可以分别创建多个 `Event` 事件, 等待内核依次做出相应并最终一并返回。

首先驱动内定义了 `_DeviceExtension` 自定义接口, 该接口用于保存此次事件所对应的 `Irp` 以及其所对应的 `DPC` 时间等。

异步分发函数 `_AsyncReadDispatch` 同样是被 `IRP_MJ_READ` 派遣函数触发的, 触发后其内部会首先 `IoGetCurrentIrpStackLocation` 得到当前 `IRP` 的堆栈信息, 然后设置 `IoMarkIrpPending()` 并最终将该 `IRP` 通过 `InsertTailList()` 插入到 `IRP` 链表内等待被处理。

- `IoMarkIrpPending`
 - 用于标记指定的 `IRP`, 标志着某个驱动的分发例程 (分发函数) 因需要被其他的驱动程序进一步处理最终返回 `STATUS_PENDING` 状态。

函数 `_CustomDpc` 则是定时器内部要执行的具体操作, 在 `DriverEntry` 驱动入口处做了如下初始化, 初始化了链表, 并初始化了一个定时器, 最后启动这个定时器每隔1秒都会执行一次 `_CustomDpc` 如果我们的 `IRP` 链表内 `IsListEmpty()` 检测 存在数据, 则会主动拷贝内存 `RtlCopyMemory` 并推送到应用层。

```

// 初始化IRP链表
InitializeListHead(&pDevExt->IrplList);
// 初始化定时器
KeInitializeTimer(&(pDevExt->timer));
// 初始化DPC pDevExt是传给_CustomDpc函数的参数
KeInitializeDpc(&pDevExt->dpc, (PKDEFERRED_ROUTINE)_CustomDpc, pDevExt);

// 设置定时时间位1s
pDevExt->liDueTime = RtlConvertLongToLargeInteger(-10000000);
// 启动定时器
KeSetTimer(&pDevExt->timer, pDevExt->liDueTime, &pDevExt->dpc);

```

驱动层完成代码如下所示:

```

#include <ntddk.h>

// 自定义接口扩展
typedef struct _DeviceExtension
{
    LIST_ENTRY IrplList;
    KTIMER timer;
    LARGE_INTEGER liDueTime;
    KDPC dpc;
} DEV_EXT, *PDEV_EXT;

// 默认派遣函数
NTSTATUS _DefaultDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    _pIrp->IoStatus.Status = STATUS_NOT_SUPPORTED;
    _pIrp->IoStatus.Information = 0;
    IoCompleteRequest(_pIrp, IO_NO_INCREMENT);
    return _pIrp->IoStatus.Status;
}

// 创建派遣函数
NTSTATUS _AsyncCreateCloseDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    _pIrp->IoStatus.Status = STATUS_SUCCESS;
    _pIrp->IoStatus.Information = 0;
    IoCompleteRequest(_pIrp, IO_NO_INCREMENT);
    return _pIrp->IoStatus.Status;
}

// 读取派遣函数
NTSTATUS _AsyncReadDispatch(PDEVICE_OBJECT _pDeviceObject, PIRP _pIrp)
{
    NTSTATUS status;
    PIO_STACK_LOCATION pIrpStack = IoGetCurrentIrpStackLocation(_pIrp);
    PDEV_EXT pDevExt = (PDEV_EXT)_pDeviceObject->DeviceExtension;

    IoMarkIrpPending(_pIrp);

    // 将IRP插入自定义链表中插入的是ListEntry
    InsertTailList(&pDevExt->IrplList, &_pIrp->Tail.Overlay.ListEntry);
}

```

```

// 返回pending 主要返回给I/O管理器的值必须和IRP的Pending标志位一致
// By: LyShark.com
// 即调用iomarkirpending和返回值要一致
return STATUS_PENDING;
}

// DPC线程
VOID _CustomDpc(PKDPC Dpc, PVOID DeferredContext, PVOID SystemArgument1, PVOID
SystemArgument2)
{
    PIRP pIrp;
    PDEV_EXT pDevExt = (PDEV_EXT)DeferredContext;
    PVOID pBuffer = NULL;
    ULONG uBufferLen = 0;
    PIO_STACK_LOCATION pIrpStack = NULL;

    do
    {
        if (!pDevExt)
        {
            break;
        }

        // 检查尾端IRP链表是否为空 为空则跳出
        if (IsListEmpty(&pDevExt->IrpList))
        {
            break;
        }

        // 从IRP链表中取出一个IRP并完成该IRP 取出的是ListEntry的地址
        PLIST_ENTRY pListEntry = (PLIST_ENTRY)RemoveHeadList(&pDevExt->IrpList);
        if (!pListEntry)
            break;

        pIrp = (PIRP)CONTAINING_RECORD(pListEntry, IRP, Tail.Overlay.ListEntry);
        pIrpStack = IoGetCurrentIrpStackLocation(pIrp);

        DbgPrint("当前DPC Irp: 0x%x\n", pIrp);

        // 驱动程序的读写方式位直接I/O
        pBuffer = MmGetSystemAddressForMdl(pIrp->MdlAddress);
        if (pBuffer == NULL)
        {
            pIrp->IoStatus.Status = STATUS_UNSUCCESSFUL;
            pIrp->IoStatus.Information = 0;
            IoCompleteRequest(pIrp, IO_NO_INCREMENT);

            break;
        }
        uBufferLen = pIrpStack->Parameters.Read.Length;
        DbgPrint("读取DPC长度: %d\n", uBufferLen);

        // 支持5字节以下的读请求
        uBufferLen = uBufferLen > 13 ? 13 : uBufferLen;
    }
}

```

```

// 复制请求内容
RtlCopyMemory(pBuffer, "hello lyshark", uBufferLen);

pIrp->IoStatus.Status = STATUS_SUCCESS;
pIrp->IoStatus.Information = uBufferLen;

// 完成该IRP
IoCompleteRequest(pIrp, IO_NO_INCREMENT);
} while (FALSE);

// 重新设置定时器
KeSetTimer(&pDevExt->timer, pDevExt->liDueTime, &pDevExt->dpc);
}

// 卸载驱动
VOID _UnloadDispatch(PDRIVER_OBJECT _pDriverObject)
{
    UNICODE_STRING Win32DeviceName;
    PDEV_EXT pDevExt = (PDEV_EXT)_pDriverObject->DeviceObject->DeviceExtension;

    RtlInitUnicodeString(&Win32DeviceName, L"\\DosDevices\\LySharkAsync");

    // 删除定时器
    // LyShark
    KeCancelTimer(&pDevExt->timer);
    // 删除创建的设备
    IoDeleteDevice(_pDriverObject->DeviceObject);
}

// 驱动入口
NTSTATUS DriverEntry(PDRIVER_OBJECT _pDriverObject, PUNICODE_STRING
_pRegistryPath)
{
    UNICODE_STRING DeviceName, Win32DeviceName;
    PDEVICE_OBJECT pDeviceObject = NULL;
    NTSTATUS status;
    PDEV_EXT pDevExt = NULL;
    HANDLE hThread;
    OBJECT_ATTRIBUTES ObjectAttributes;
    CLIENT_ID CID;

    RtlInitUnicodeString(&DeviceName, L"\\Device\\LySharkAsync");
    RtlInitUnicodeString(&Win32DeviceName, L"\\DosDevices\\LySharkAsync");

    for (ULONG i = 0; i <= IRP_MJ_MAXIMUM_FUNCTION; i++)
    {
        _pDriverObject->MajorFunction[i] = _DefaultDispatch;
    }

    _pDriverObject->MajorFunction[IRP_MJ_CREATE] = _AsyncCreateCloseDispatch;
    _pDriverObject->MajorFunction[IRP_MJ_CLOSE] = _AsyncCreateCloseDispatch;
    _pDriverObject->MajorFunction[IRP_MJ_READ] = _AsyncReadDispatch;
    _pDriverObject->DriverUnload = _UnloadDispatch;
}

```

```

// 分配自定义扩展
status = IoCreateDevice(_pDriverObject, sizeof(DEV_EXT), &DeviceName,
FILE_DEVICE_UNKNOWN, 0, FALSE, &pDeviceObject);
if (!NT_SUCCESS(status))
    return status;
if (!pDeviceObject)
    return STATUS_UNEXPECTED_IO_ERROR;

pDeviceObject->Flags |= DO_DIRECT_IO;
pDeviceObject->AlignmentRequirement = FILE_WORD_ALIGNMENT;
status = IoCreateSymbolicLink(&win32DeviceName, &DeviceName);

pDeviceObject->Flags &= ~DO_DEVICE_INITIALIZING;
pDevExt = (PDEV_EXT)pDeviceObject->DeviceExtension;

// 初始化IRP链表
InitializeListHead(&pDevExt->IrpList);
// 初始化定时器
KeInitializeTimer(&(pDevExt->timer));
// 初始化DPC pDevExt是传给_CustomDpc函数的参数
KeInitializeDpc(&pDevExt->dpc, (PKDEFERRED_ROUTINE)_CustomDpc, pDevExt);

// 设置定时时间位1s
pDevExt->liDueTime = RtlConvertLongToLargeInteger(-10000000);
// 启动定时器
KeSetTimer(&pDevExt->timer, pDevExt->liDueTime, &pDevExt->dpc);

return STATUS_SUCCESS;
}

```

驱动层说完了，接下来是应用层，对于应用层来说，需要使用 `CreateEvent` 打开通知事件，或者叫做事件对象，然后当有通知时，则直接使用 `ReadFile` 读取对应的缓冲区，当所有读取全部结束 `WaitForMultipleObjects` 等待结束即输出结果。

```

#include <stdio.h>
#include <windows.h>

int main(int argc, char *argv[])
{
    HANDLE hFile;
    char Buffer[3][32] = { 0 };
    DWORD dwRet[3] = { 0 };
    OVERLAPPED ol[3] = { 0 };
    HANDLE hEvent[3] = { 0 };

    // By:LyShark
    hFile = CreateFileA("\\\\.\\LySharkAsync", GENERIC_READ, FILE_SHARE_READ,
NULL, OPEN_EXISTING,
FILE_ATTRIBUTE_NORMAL | FILE_FLAG_OVERLAPPED,
NULL);
    if (INVALID_HANDLE_VALUE == hFile)
        return 0;

    // event用来通知请求完成
    hEvent[0] = CreateEvent(NULL, TRUE, FALSE, NULL);
}

```

```

ol[0].hEvent = hEvent[0];

hEvent[1] = CreateEvent(NULL, TRUE, FALSE, NULL);
ol[1].hEvent = hEvent[1];

hEvent[2] = CreateEvent(NULL, TRUE, FALSE, NULL);
ol[2].hEvent = hEvent[2];

// 读取事件内容到缓存
ReadFile(hFile, Buffer[0], 13, &dwRet[0], &ol[0]);
ReadFile(hFile, Buffer[1], 13, &dwRet[1], &ol[1]);
ReadFile(hFile, Buffer[2], 13, &dwRet[2], &ol[2]);

// 等待三个事件执行完毕
WaitForMultipleObjects(3, hEvent, TRUE, INFINITE);

// 输出结果
printf("缓存LyShark A: %s \n", Buffer[0]);
printf("缓存LyShark B: %s \n", Buffer[1]);
printf("缓存LyShark C: %s \n", Buffer[2]);

CloseHandle(hFile);
return 0;
}

```

这段代码最终运行效果如下：

DebugView on \\DESKTOP-B53PAVI (local)

#	Time	Debug Print
1932	3073.91040039	39741718750 - STORMINI: StorNVMe - POWER: IDLE
1933	3073.91040039	39741718750 - STORMINI: StorNVMe - POWER: ACTIVE
1934	3074.30590820	当前DPC Irp: 0x492e6cd0
1935	3074.30615234	读取DPC长度: 13
1936	3074.91308594	39751718750 - STORMINI: StorNVMe - POWER: IDLE
1937	3075.19018555	39754531250 - STORMINI: StorNVMe - POWER: ACTIVE
1938	3075.30615234	当前DPC Irp: 0x492c9e10
1939	3075.30615234	读取DPC长度: 13
1940	3076.20117188	39764687500 - STORMINI: StorNVMe - POWER: IDLE
1941	3076.30737305	当前DPC Irp: 0x48f3fcd0
1942	3076.30737305	读取DPC长度: 13
1943	3080.1970214	命令提示符
1944	3080.1982421	
1945	3080.1984863	C:\Users\lyshark\Desktop>lyshark.exe
1946	3081.1997070	缓存LyShark A: hello lyshark
		缓存LyShark B: hello lyshark
		缓存LyShark C: hello lyshark

第四章：驱动读写篇

4.1 内核CR3切换读写内存

首先CR3是什么，CR3是一个寄存器，该寄存器内保存有页目录表物理地址(PDBR地址)，其实CR3内部存放的就是页目录表的内存基地址，运用CR3切换可实现对特定进程内存地址的强制读写操作，此类读写属于有痕读写，多数驱动保护都会将这个地址改为无效，此时CR3读写就失效了，当然如果能找到CR3的正确地址，此方式也是靠谱的一种读写机制。

在读写进程之前需要先找到进程的 `PEPROCESS` 结构，查找结构的方法也很简单，依次遍历进程并对比进程名称即可得到。

```

#include <ntifs.h>
#include <windef.h>
#include <intrin.h>

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

// 定义全局EProcess结构
PEPROCESS Global_Peprocess = NULL;

// 根据进程名获得EPROCESS结构
NTSTATUS GetProcessObjectByName(char *name)
{
    NTSTATUS Status = STATUS_UNSUCCESSFUL;
    SIZE_T i;

    __try
    {
        for (i = 100; i < 20000; i += 4)
        {
            NTSTATUS st;
            PEPROCESS ep;
            st = PsLookupProcessByProcessId((HANDLE)i, &ep);
            if (NT_SUCCESS(st))
            {
                char *pn = PsGetProcessImageFileName(ep);
                if (_stricmp(pn, name) == 0)
                {
                    Global_Peprocess = ep;
                }
            }
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        return Status;
    }
    return Status;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    NTSTATUS nt = GetProcessObjectByName("Tutorial-i386.exe");

    if (NT_SUCCESS(nt))
    {
        DbgPrint("[+] eprocess = %x \n", Global_Peprocess);
    }
}

```



```

}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

以打开 Tutorial-i386.exe 为例，打开后即可返回他的 Process，当然也可以直接传入进程PID同样可以得到进程 Process 结构地址。

```

// 根据PID打开进程
PEPROCESS Peprocess = NULL;
DWORD PID = 6672;
NTSTATUS nt = PsLookupProcessByProcessId((HANDLE)PID, &Peprocess);

```

通过CR3读取内存实现代码如下，我们读取 Tutorial-i386.exe 里面的 0x0009EDC8 这段内存，读出长度是4字节，代码如下。

```

#include <ntifs.h>
#include <windef.h>
#include <intrin.h>

#define DIRECTORY_TABLE_BASE 0x028

#pragma intrinsic(_disable)
#pragma intrinsic(_enable)

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

// 关闭写保护
KIRQL Open()
{
    KIRQL irq1 = KeRaiseIrqlToDpcLevel();
    UINT64 cr0 = __readcr0();
    cr0 &= 0xffffffffffff;
    __writecr0(cr0);
    _disable();
    return irq1;
}

// 开启写保护
void Close(KIRQL irq1)
{
    UINT64 cr0 = __readcr0();
    cr0 |= 0x10000;
    _enable();
    __writecr0(cr0);
    KeLowerIrql(irq1);
}

// 检查内存
ULONG64 CheckAddressVal(PVOID p)
{

```

```

    if (MmIsAddressValid(p) == FALSE)
        return 0;
    return *(PULONG64)p;
}

// CR3 寄存器读内存
BOOLEAN CR3_ReadProcessMemory(IN PEPROCESS Process, IN PVOID Address, IN UINT32
Length, OUT PVOID Buffer)
{
    ULONG64 pDTB = 0, OldCr3 = 0, vAddr = 0;
    pDTB = CheckAddressVal((UCHAR*)Process + DIRECTORY_TABLE_BASE);
    if (pDTB == 0)
    {
        return FALSE;
    }

    _disable();
    OldCr3 = __readcr3();
    __writecr3(pDTB);
    _enable();

    if (MmIsAddressValid(Address))
    {
        RtlCopyMemory(Buffer, Address, Length);
        DbgPrint("读入数据: %ld", *(PDWORD)Buffer);
        return TRUE;
    }

    _disable();
    __writecr3(OldCr3);
    _enable();
    return FALSE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 根据PID打开进程
    PEPROCESS Pprocess = NULL;
    DWORD PID = 6672;
    NTSTATUS nt = PsLookupProcessByProcessId((HANDLE)PID, &Pprocess);

    DWORD buffer = 0;

    BOOLEAN b1 = CR3_ReadProcessMemory(Pprocess, (PVOID)0x0009EDC8, 4,
&buffer);

    DbgPrint("readbuf = %x \n", buffer);
    DbgPrint("readbuf = %d \n", buffer);
}

```

```

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

读出后输出效果如下:

The screenshot shows a debugger window with a list of kernel memory addresses and their values. A red box highlights the following text:

```

3 1.26139104 13960468750 - STORMINI: StorNVMe - POWER: IDLE
4 1.26155710 13960468750 - STORMINI: StorNVMe - POWER: ACTIVE
5 2.22152472 hello lyshark
6 2.22152829 读入数据: 100
7 2.22152925 readbuf = 64
8 2.22152996 readbuf = 100
9 2.26937366 13970468750 - STORMINI: StorNVMe - POWER: IDLE
10 3.19279313 Uninstall Driver Is OK
11 3.90892839 13986875000 - STORMINI: StorNVMe - POWER: ACTIVE
12 3.90945435 13986875000 - STORMINI: StorNVMe - POWER: IDLE
13 4.03330183 13988125000 - STORMINI: StorNVMe - POWER: ACTIVE
14 5.03310061 13998125000 - STORMINI: StorNVMe - POWER: IDLE
15 5.37245941 14001562500 - STORMINI: StorNVMe - POWER: ACTIVE

```

Below the memory list, there is a table with the following columns: 激活 (Activate), 描述 (Description), 地址 (Address), 类型 (Type), and 数值 (Value). The table contains one row with the following data:

激活	描述	地址	类型	数值
☒	无描述	0009EDC8	4 字节	100

To the right of the table, there is a window titled "Kernel-Mode Driver Manager" with the following controls:

- Path: C:\Users\admin\Desktop\WinDDK.sys
- Buttons: BeginRun, Run, I/O Control, Stop'nUnreg, Stop
- Code (hex): [Empty field]

写出内存与读取基本一致, 代码如下。

```

#include <ntifs.h>
#include <windef.h>
#include <intrin.h>

#define DIRECTORY_TABLE_BASE 0x028

#pragma intrinsic(_disable)
#pragma intrinsic(_enable)

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

// 关闭写保护
KIRQL Open()
{
    KIRQL irq1 = KeRaiseIrqlToDpcLevel();
    UINT64 cr0 = __readcr0();
    cr0 &= 0xfffffffffffff;
    __writecr0(cr0);
    _disable();
    return irq1;
}

// 开启写保护
void Close(KIRQL irq1)
{
    UINT64 cr0 = __readcr0();
    cr0 |= 0x10000;
    _enable();
    __writecr0(cr0);
    KeLowerIrql(irq1);
}

```

```

// 检查内存
ULONG64 CheckAddressVal(PVOID p)
{
    if (MmIsAddressValid(p) == FALSE)
        return 0;
    return *(PULONG64)p;
}

// CR3 寄存器写内存
BOOLEAN CR3_writeProcessMemory(IN PEPROCESS Process, IN PVOID Address, IN UINT32
Length, IN PVOID Buffer)
{
    ULONG64 pDTB = 0, oldCr3 = 0, vAddr = 0;

    // 检查内存
    pDTB = CheckAddressVal((UCHAR*)Process + DIRECTORY_TABLE_BASE);
    if (pDTB == 0)
    {
        return FALSE;
    }

    _disable();

    // 读取CR3
    oldCr3 = __readcr3();

    // 写CR3
    __writecr3(pDTB);
    _enable();

    // 验证并拷贝内存
    if (MmIsAddressValid(Address))
    {
        RtlCopyMemory(Address, Buffer, Length);
        return TRUE;
    }
    _disable();

    // 恢复CR3
    __writecr3(oldCr3);
    _enable();
    return FALSE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 根据PID打开进程

```

```

PEPROCESS Pprocess = NULL;
DWORD PID = 6672;
NTSTATUS nt = PsLookupProcessByProcessId((HANDLE)PID, &Pprocess);

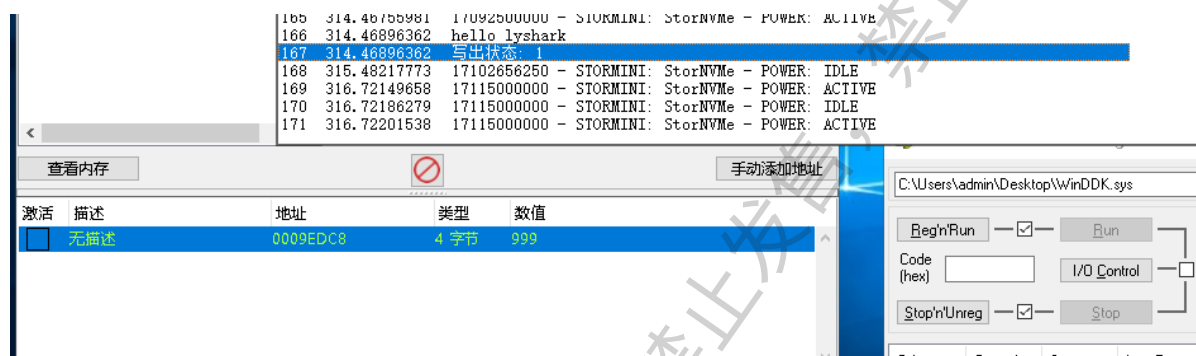
DWORD buffer = 999;

BOOLEAN b1 = CR3_WriteProcessMemory(Pprocess, (PVOID)0x0009EDC8, 4,
&buffer);
DbgPrint("写出状态: %d \n", b1);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

写出后效果如下:



4.2 内核MDL读写进程内存

MDL内存读写是最常用的一种读写模式，通常需要附加到指定进程空间内然后调用内存拷贝得到对端内存中的数据，在调用结束后再将其空间释放掉，通过这种方式实现内存读写操作，此种模式的读写操作也是最推荐使用的相比于CR3切换来说，此方式更稳定并不会受寄存器的影响。

MDL读取内存步骤

- 1.调用PsLookupProcessByProcessId得到进程Process结构
- 2.调用KeStackAttachProcess附加到对端进程内
- 3.调用ProbeForRead检查内存是否可读写
- 4.拷贝内存空间中的数据到自己的缓冲区内
- 5.调用KeUnstackDetachProcess接触绑定
- 6.调用ObDereferenceObject使对象引用数减1

代码总结起来应该是如下样子，用户传入一个结构体，输出对应长度的字节数据：

```

#include <ntifs.h>
#include <windef.h>

typedef struct
{
    DWORD pid;                // 要读写的进程ID
    DWORD64 address;          // 要读写的地址
    DWORD size;                // 读写长度
    BYTE* data;                // 要读写的数据
}ReadMemoryStruct;

// MDL读内存

```

```

BOOL MDLReadMemory(ReadMemoryStruct* data)
{
    BOOL bRet = TRUE;
    PEPROCESS process = NULL;

    PsLookupProcessByProcessId(data->pid, &process);

    if (process == NULL)
    {
        return FALSE;
    }

    BYTE* GetData;
    __try
    {
        GetData = ExAllocatePool(PagedPool, data->size);
    }
    __except (1)
    {
        return FALSE;
    }

    KAPC_STATE stack = { 0 };
    KeStackAttachProcess(process, &stack);

    __try
    {
        ProbeForRead(data->address, data->size, 1);
        RtlCopyMemory(GetData, data->address, data->size);
    }
    __except (1)
    {
        bRet = FALSE;
    }

    ObDereferenceObject(process);
    KeUnstackDetachProcess(&stack);
    RtlCopyMemory(data->data, GetData, data->size);
    ExFreePool(GetData);
    return bRet;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    ReadMemoryStruct ptr;

    ptr.pid = 6672;
    ptr.address = 0x402c00;
}

```



```

    DWORD pid;                // 要读写的进程ID
    DWORD64 address;          // 要读写的地址
    DWORD size;               // 读写长度
    BYTE* data;               // 要读写的数据
}ReadMemoryStruct;

// MDL写内存
BOOL MDLWriteMemory(ReadMemoryStruct* data)
{
    BOOL bRet = TRUE;
    PEPROCESS process = NULL;

    PsLookupProcessByProcessId(data->pid, &process);
    if (process == NULL)
    {
        return FALSE;
    }

    BYTE* GetData;
    __try
    {
        GetData = ExAllocatePool(PagedPool, data->size);
    }
    __except (1)
    {
        return FALSE;
    }

    for (int i = 0; i < data->size; i++)
    {
        GetData[i] = data->data[i];
    }

    KAPC_STATE stack = { 0 };
    KeStackAttachProcess(process, &stack);

    PMDL mdl = IoAllocateMdl(data->address, data->size, 0, 0, NULL);
    if (mdl == NULL)
    {
        return FALSE;
    }

    MmBuildMdlForNonPagedPool(mdl);

    BYTE* ChangeData = NULL;

    __try
    {
        ChangeData = MmMapLockedPages(mdl, KernelMode);
        RtlCopyMemory(ChangeData, GetData, data->size);
    }
    __except (1)
    {
        bRet = FALSE;
        goto END;
    }
}

```



```

    }

END:
    IoFreeMdl(mdl);
    ExFreePool(GetData);
    KeUnstackDetachProcess(&stack);
    ObDereferenceObject(process);

    return bRet;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    ReadMemoryStruct ptr;

    ptr.pid = 6672;
    ptr.address = 0x402c00;
    ptr.size = 5;

    // 需要写入的数据
    ptr.data = ExAllocatePool(PagedPool, ptr.size);

    // 循环设置
    for (size_t i = 0; i < 5; i++)
    {
        ptr.data[i] = 0x90;
    }

    // 写内存
    MDLWriteMemory(&ptr);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

写出效果如下：

```
保护:可执行/只读 AllocationBase=00400000 基址=00402000 长度=17C000 模块=Tutorial-i386.ex^
地址 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 0123456789ABCDEF
00402C00 90 90 90 90 90 00 00 B9 00 B0 65 00 BA 04 B0 65 .. e. . e
00402C10 00 B8 50 E0 57 00 E8 45 FF FF FF E8 50 FF FF FF . P W. E P
00402C20 B8 10 E0 57 00 E8 B6 0A 01 00 C3 00 00 00 00 00 . W. ... ....
00402C30 E8 FB C2 00 00 A1 50 F1 57 00 8B 15 50 F1 57 00 .. P W. .P W.
00402C40 8B 15 50 F1 57 00 8B 15 50 F1 57 00 .. P W. .P W.
00402C50 65 48 48 48 48 48 48 48 48 48 48 48 48 48 48 48
00402C60 00 File Edit Capture Options Computer Help
00402C70 E8 48 48 48 48 48 48 48 48 48 48 48 48 48 48
00402C80 D0 48 48 48 48 48 48 48 48 48 48 48 48 48 48
00402C90 C5 48 48 48 48 48 48 48 48 48 48 48 48 48 48
00402CA0 64 # Time Debug Print
00402CB0 90 155 55.39398575 ba
00402CC0 91 156 55.39398575 30
00402CD0 F7 157 55.39398575 f7
00402CE0 62 158 55.39398575 5d
00402CF0 62 159 55.39398575 0
00402D00 62 160 55.39398575 e8
00402D10 00 161 55.39398575 cf
```

4.3 通过内存拷贝读写内存

内核中读写内存的方式有很多，典型的读写方式有CR3读写，MDL读写，以及今天要给大家分享的内存拷贝实现读写，拷贝读写的使用 MmCopyVirtualMemory 这个内核API函数实现，通过调用该函数即可很容易的实现内存的拷贝读写。

封装 KeReadProcessMemory() 内存读取。

```
#include <ntifs.h>
#include <windef.h>
#include <stdlib.h>

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);
NTSTATUS NTAPI MmCopyVirtualMemory(PEPROCESS SourceProcess, PVOID SourceAddress,
PEPROCESS TargetProcess, PVOID TargetAddress, SIZE_T BufferSize, KPROCESSOR_MODE
PreviousMode, PSIZE_T ReturnSize);

// 定义全局EProcess结构
PEPROCESS Global_Peprocess = NULL;

// 普通Ke内存读取
NTSTATUS KeReadProcessMemory(PVOID SourceAddress, PVOID TargetAddress, SIZE_T
Size)
{
    __try
    {
        PEPROCESS TargetProcess = PsGetCurrentProcess();
        SIZE_T Result;
        if (NT_SUCCESS(MmCopyVirtualMemory(Global_Peprocess, SourceAddress,
        TargetProcess, TargetAddress, Size, KernelMode, &Result)))
            return STATUS_SUCCESS;
        else
            return STATUS_ACCESS_DENIED;
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        return STATUS_ACCESS_DENIED;
    }
    return STATUS_ACCESS_DENIED;
}
```

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("Uninstall Driver Is OK \n");
}

// By:lyshark.cnblogs.com
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 根据PID打开进程
    DWORD PID = 6672;
    NTSTATUS nt = PsLookupProcessByProcessId((HANDLE)PID, &Global_Peprocess);

    DWORD ref_value = 0;

    // 将地址处读取4字节到ref_value中
    NTSTATUS read_nt = KeReadProcessMemory((PVOID)0x0009EDC8, &ref_value, 4);

    DbgPrint("读出数据: %d \n", ref_value);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

读取效果如下:

The screenshot shows a debugger window with a list of loaded modules on the left. The address 0009EDC8 is highlighted in the memory dump, and the value 999 is shown. The debugger interface includes a list of loaded modules, a memory dump, and a control panel with buttons like 'Begin/Run', 'Run', 'Stop', and 'End'.

封装 KeWriteProcessMemory() 内存读取。

```

#include <ntifs.h>
#include <windef.h>
#include <stdlib.h>

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PPEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);
NTSTATUS NtAPI MmCopyVirtualMemory(PEPROCESS SourceProcess, PVOID SourceAddress,
PEPROCESS TargetProcess, PVOID TargetAddress, SIZE_T BufferSize, KPROCESSOR_MODE
PreviousMode, PSIZE_T ReturnSize);

// 定义全局EProcess结构
PEPROCESS Global_Peprocess = NULL;

```

```
// 普通ke内存写入
NTSTATUS KwriteProcessMemory(PVOID SourceAddress, PVOID TargetAddress, SIZE_T
Size)
{
    PEPROCESS SourceProcess = PsGetCurrentProcess();
    PEPROCESS TargetProcess = Global_Peprocess;
    SIZE_T Result;

    if (NT_SUCCESS(MmCopyVirtualMemory(SourceProcess, SourceAddress,
TargetProcess, TargetAddress, Size, KernelMode, &Result)))
        return STATUS_SUCCESS;
    else
        return STATUS_ACCESS_DENIED;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("Uninstall Driver Is OK \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    // 根据PID打开进程
    DWORD PID = 6672;
    NTSTATUS nt = PsLookupProcessByProcessId((HANDLE)PID, &Global_Peprocess);

    DWORD ref_value = 10;

    // 将地址处写出4字节
    NTSTATUS read_nt = KwriteProcessMemory((PVOID)0x0009EDC8, &ref_value, 4);

    DbgPrint("写入数据: %d \n", ref_value);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}
```

写出内存效果:

Address	Type	Value
0009EDC8	4 字节	10

Driver	Operation	Status	Last Error
WinDDK	Stop	Fail	指定的服务未安装。
WinDDK	Unregister	Fail	指定的服务未安装。

4.4 内核R3与R0内存映射拷贝

在上一篇博文《驱动开发：内核通过PEB得到进程参数》中我们通过使用 `KeStackAttachProcess` 附加进程的方式得到了该进程的PEB结构信息，本篇文章同样需要使用进程附加功能，但这次我们将实现一个更加有趣的功能，在某些情况下应用层与内核层需要共享一片内存区域通过这片区域可打通内核与应用层的隔离，此类功能的实现依附于MDL内存映射机制实现。

应用层(R3)数据映射到内核层(R0)

先来实现将R3内存数据拷贝到R0中，功能实现所调用的API如下：

- `IoAllocateMdl` 该函数用于创建 MDL (类似初始化)
- `MmProbeAndLockPages` 用于锁定创建的地址其中 `UserMode` 代表用户层, `IoReadAccess` 以读取的方式锁定
- `MmGetSystemAddressForMdlSafe` 用于从 MDL 中得到映射内存地址
- `RtlCopyMemory` 用于内存拷贝,将 `DstAddr` 应用层中的数据拷贝到 `pMappedSrc` 中
- `MmUnlockPages` 拷贝结束后解锁 `pSrcMdl`
- `IoFreeMdl` 释放 MDL

内存拷贝 `SafeCopyMemory_R3_to_R0` 函数封装代码如下：

```
#include <ntifs.h>
#include <windef.h>

// 分配内存
void* RtlAllocateMemory(BOOLEAN InZeroMemory, SIZE_T InSize)
{
    void* Result = ExAllocatePoolWithTag(NonPagedPool, InSize, 'lysh');
    if (InZeroMemory && (Result != NULL))
        RtlZeroMemory(Result, InSize);
    return Result;
}

// 释放内存
void RtlFreeMemory(void* InPointer)
{
    ExFreePool(InPointer);
}

/*
将应用层中的内存复制到内核变量中

SrcAddr  r3地址要复制
DstAddr  R0申请的地址
Size     拷贝长度
*/
NTSTATUS SafeCopyMemory_R3_to_R0(ULONG_PTR SrcAddr, ULONG_PTR DstAddr, ULONG
Size)
{
    NTSTATUS status = STATUS_UNSUCCESSFUL;
    ULONG nRemainSize = PAGE_SIZE - (SrcAddr & 0xFFF);
    ULONG nCopyedSize = 0;

    if (!SrcAddr || !DstAddr || !Size)
    {
```

```

        return status;
    }

    while (nCopiedSize < Size)
    {
        PMDL pSrcMdl = NULL;
        PVOID pMappedSrc = NULL;

        if (Size - nCopiedSize < nRemainSize)
        {
            nRemainSize = Size - nCopiedSize;
        }

        // 创建MDL
        pSrcMdl = IoAllocateMdl((PVOID)(SrcAddr & 0xFFFFFFFFFFFF000),
        PAGE_SIZE, FALSE, FALSE, NULL);
        if (pSrcMdl)
        {
            __try
            {
                // 锁定内存页面(UserMode代表应用层)
                MmProbeAndLockPages(pSrcMdl, UserMode, IoReadAccess);

                // 从MDL中得到映射内存地址
                pMappedSrc = MmGetSystemAddressForMdlSafe(pSrcMdl,
                NormalPagePriority);
            }
            __except (EXCEPTION_EXECUTE_HANDLER)
            {
            }
        }

        if (pMappedSrc)
        {
            __try
            {
                // 将MDL中的映射拷贝到pMappedSrc内存中
                RtlCopyMemory((PVOID)DstAddr, (PVOID)((ULONG_PTR)pMappedSrc +
                (SrcAddr & 0xFFF)), nRemainSize);
            }
            __except (1)
            {
                // 拷贝内存异常
            }

            // 释放锁
            MmUnlockPages(pSrcMdl);
        }

        if (pSrcMdl)
        {
            // 释放MDL
            IoFreeMdl(pSrcMdl);
        }
    }
}

```

```

        if (nCopiedSize)
        {
            nRemainSize = PAGE_SIZE;
        }

        nCopiedSize += nRemainSize;
        SrcAddr += nRemainSize;
        DstAddr += nRemainSize;
    }

    status = STATUS_SUCCESS;
    return status;
}

```

调用该函数实现拷贝，如下代码中首先 `PsLookupProcessByProcessId` 得到进程 `EPROCESS` 结构，并 `KeStackAttachProcess` 附加进程，声明 `pTempBuffer` 指针用于存储 `RtlAllocateMemory` 开辟的内存空间，`nSize` 则代表读取应用层进程数据长度，`ModuleBase` 则是读入进程基址，调用 `SafeCopyMemory_R3_to_R0` 即可将应用层数据拷贝到内核空间，并最终 `BYTE* data` 转为 `BYTE` 字节的方式输出。

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

// lyshark.com
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    NTSTATUS status = STATUS_UNSUCCESSFUL;
    PEPROCESS eproc = NULL;
    KAPC_STATE kpc = { 0 };

    __try
    {
        // HANDLE 进程PID
        status = PsLookupProcessByProcessId((HANDLE)4556, &eproc);

        if (NT_SUCCESS(status))
        {
            // 附加进程
            KeStackAttachProcess(eproc, &kpc);

            // -----

            // 开始映射
            // -----

            // 将用户空间内存映射到内核空间
            PVOID pTempBuffer = NULL;
            ULONG nSize = 0x1024;
            ULONG_PTR ModuleBase = 0x0000000140001000;

```

```

// 分配内存
pTempBuffer = RtlAllocateMemory(TRUE, nSize);
if (pTempBuffer)
{
    // 拷贝数据到R0
    status = SafeCopyMemory_R3_to_R0(ModuleBase,
    (ULONG_PTR)pTempBuffer, nSize);
    if (NT_SUCCESS(status))
    {
        DbgPrint("[*] 拷贝应用层数据到内核里 \n");
    }

    // 转成BYTE方便读取
    BYTE* data = pTempBuffer;

    for (size_t i = 0; i < 10; i++)
    {
        DbgPrint("%02X \n", data[i]);
    }
}

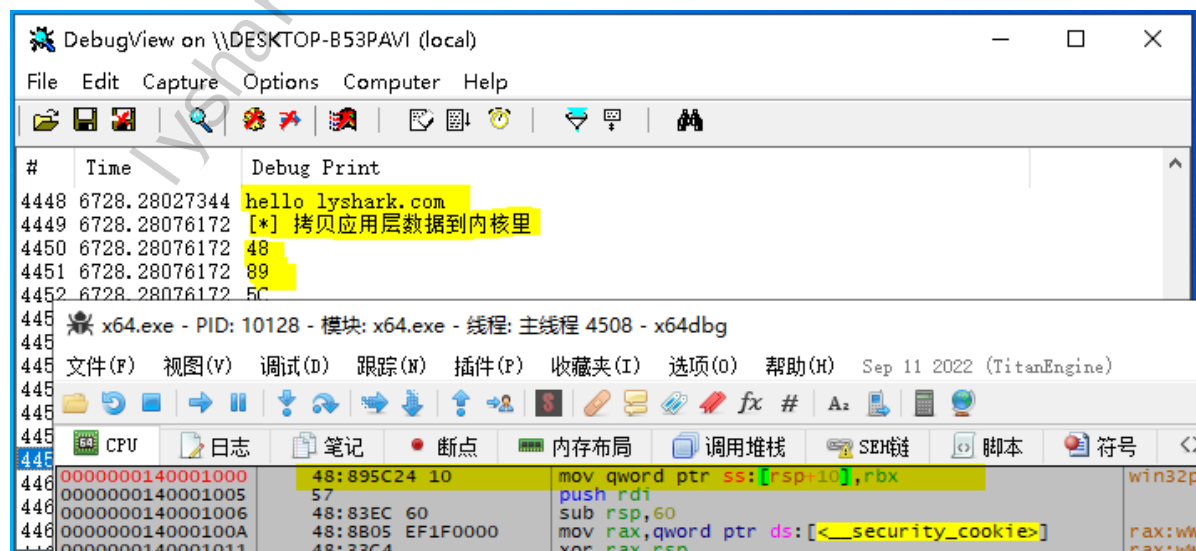
// 释放空间
RtlFreeMemory(pTempBuffer);

// 脱离进程
KeUnstackDetachProcess(&kpc);
}
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码运行后即可将进程中 0x0000000140001000 处的数据读入内核空间并输出：



内核层(R0)数据映射到应用层(R3)

与上方功能实现相反 `SafeCopyMemory_R0_to_R3` 函数则用于将一个内核层中的缓冲区写出到应用层中，写出过程：

- `IoAllocateMdl` 分别调用MDL分配,源地址 `SrcAddr` 目标地址 `DstAddr` 均创建
- `MmBuildMdlForNonPagedPool` 该 MDL 指定非分页虚拟内存缓冲区，并对其进行更新以描述基础物理页
- `MmGetSystemAddressForMdlSafe` 调用两次得到源地址，分别获取 `pSrcMdl`, `pDstMdl` 两个MDL的
- `MmProbeAndLockPages` 以写入方式锁定用户层中 `pDstMdl` 的地址

内存拷贝 `SafeCopyMemory_R0_to_R3` 函数封装代码如下：

```
// 分配内存
void* RtlAllocateMemory(BOOLEAN InZeroMemory, SIZE_T InSize)
{
    void* Result = ExAllocatePoolWithTag(NonPagedPool, InSize, 'lysh');
    if (InZeroMemory && (Result != NULL))
        RtlZeroMemory(Result, InSize);
    return Result;
}

// 释放内存
void RtlFreeMemory(void* InPointer)
{
    ExFreePool(InPointer);
}

/*
将内存中的数据复制到R3中

SrcAddr  R0要复制的地址
DstAddr  返回R3的地址
Size     拷贝长度
*/
NTSTATUS SafeCopyMemory_R0_to_R3(PVOID SrcAddr, PVOID DstAddr, ULONG Size)
{
    PMDL pSrcMdl = NULL, pDstMdl = NULL;
    PCHAR pSrcAddress = NULL, pDstAddress = NULL;
    NTSTATUS st = STATUS_UNSUCCESSFUL;

    // 分配MDL 源地址
    pSrcMdl = IoAllocateMdl(SrcAddr, Size, FALSE, FALSE, NULL);
    if (!pSrcMdl)
    {
        return st;
    }

    // 该 MDL 指定非分页虚拟内存缓冲区，并对其进行更新以描述基础物理页。
    MmBuildMdlForNonPagedPool(pSrcMdl);

    // 获取源地址MDL地址
    pSrcAddress = MmGetSystemAddressForMdlSafe(pSrcMdl, NormalPagePriority);
```

```

if (!pSrcAddress)
{
    IoFreeMdl(pSrcMdl);
    return st;
}

// 分配MDL 目标地址
pDstMdl = IoAllocateMdl(DstAddr, Size, FALSE, FALSE, NULL);
if (!pDstMdl)
{
    IoFreeMdl(pSrcMdl);
    return st;
}

__try
{
    // 以写入的方式锁定目标MDL
    MmProbeAndLockPages(pDstMdl, UserMode, IoWriteAccess);

    // 获取目标地址MDL地址
    pDstAddress = MmGetSystemAddressForMdlSafe(pDstMdl, NormalPagePriority);
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
}

if (pDstAddress)
{
    __try
    {
        // 将源地址拷贝到目标地址
        RtlCopyMemory(pDstAddress, pSrcAddress, Size);
    }
    __except (1)
    {
        // 拷贝内存异常
    }
    MmUnlockPages(pDstMdl);
    st = STATUS_SUCCESS;
}

IoFreeMdl(pDstMdl);
IoFreeMdl(pSrcMdl);

return st;
}

```

调用该函数实现拷贝，此处除去附加进程以外，在拷贝之前调用了 `zwAllocateVirtualMemory` 将内存属性设置为 `PAGE_EXECUTE_READWRITE` 可读可写可执行状态，然后在向该内存中写出 `pTempBuffer` 变量中的内容，此变量中的数据是 0x90 填充的区域。

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

```

```

}

// lyshark.com
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    NTSTATUS status = STATUS_UNSUCCESSFUL;
    PEPROCESS eproc = NULL;
    KAPC_STATE kpc = { 0 };

    __try
    {
        // HANDLE 进程PID
        status = PsLookupProcessByProcessId((HANDLE)4556, &eproc);

        if (NT_SUCCESS(status))
        {
            // 附加进程
            KeStackAttachProcess(eproc, &kpc);

            // -----

            // 开始映射
            // -----

            // 将用户空间内存映射到内核空间
            PVOID pTempBuffer = NULL;
            ULONG nSize = 0x1024;
            PVOID ModuleBase = 0x0000000140001000;

            // 分配内存
            pTempBuffer = RtlAllocateMemory(TRUE, nSize);
            if (pTempBuffer)
            {
                memset(pTempBuffer, 0x90, nSize);

                // 设置内存属性 PAGE_EXECUTE_READWRITE
                ZwAllocateVirtualMemory(NtCurrentProcess(), &ModuleBase, 0,
                &nSize, MEM_RESERVE, PAGE_EXECUTE_READWRITE);
                ZwAllocateVirtualMemory(NtCurrentProcess(), &ModuleBase, 0,
                &nSize, MEM_COMMIT, PAGE_EXECUTE_READWRITE);

                // 将数据拷贝到R3中
                status = SafeCopyMemory_R0_to_R3(pTempBuffer, &ModuleBase,
                nSize);

                if (NT_SUCCESS(status))
                {
                    DbgPrint("[*] 拷贝内核数据到应用层 \n");
                }
            }

            // 释放空间
            RtlFreeMemory(pTempBuffer);
        }
    }
}

```

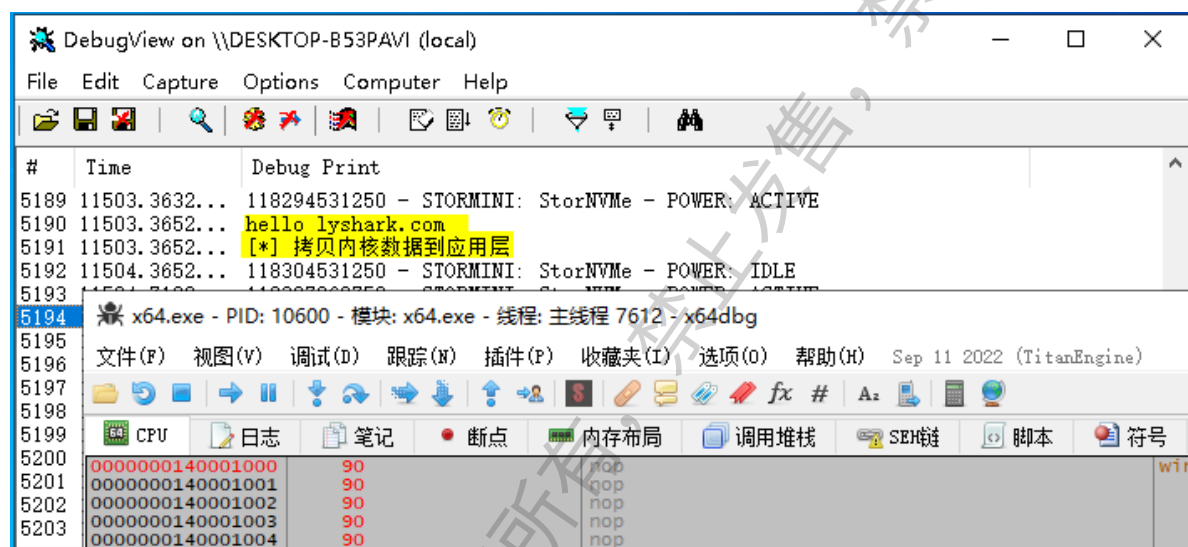
```

        // 脱离进程
        KeUnstackDetachProcess(&kpc);
    }
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

拷贝成功后，应用层进程内将会被填充为Nop指令。



第五章：内核SSDT解析篇

5.1 内核枚举SSDT表基址

三年前面朝黄土背朝天的我，写了一篇如何在 windows 7 系统下枚举内核 SSDT 表的文章《驱动开发：内核读取SSDT表基址》三年过去了我还是个单身狗，开个玩笑，微软的 windows 10 系统已经覆盖了大多数个人PC终端，以前的方法也该进行迭代更新了，或许在网上你能够找到类似的文章，但我可以百分百肯定都不能用，今天 Lyshark 将带大家一起分析 win10 x64 最新系统 SSDT 表的枚举实现。

看一款闭源ARK工具的枚举效果:

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
SSDT	Shadow SSDT	内核钩子	系统中断表	Object钩子								

索引	函数名	原始函数地址	当前函数地址	当前函数地址所在模块
0	NtAccessCheck	0xFFFFF8036EB12340	0xFFFFF8036EB12340	C:\Windows\system32\ntoskrnl.exe
1	NtWorkerFactoryWorkerReady	0xFFFFF8036EB1C3D0	0xFFFFF8036EB1C3D0	C:\Windows\system32\ntoskrnl.exe
2	NtAcceptConnectPort	0xFFFFF8036F0DE450	0xFFFFF8036F0DE450	C:\Windows\system32\ntoskrnl.exe
3	NtMapUserPhysicalPagesScatter	0xFFFFF8036F2992F0	0xFFFFF8036F2992F0	C:\Windows\system32\ntoskrnl.exe
4	NtWaitForSingleObject	0xFFFFF8036EFF3B50	0xFFFFF8036EFF3B50	C:\Windows\system32\ntoskrnl.exe
5	NtCallbackReturn	0xFFFFF8036EBC4E10	0xFFFFF8036EBC4E10	C:\Windows\system32\ntoskrnl.exe
6	NtReadFile	0xFFFFF8036EFE5220	0xFFFFF8036EFE5220	C:\Windows\system32\ntoskrnl.exe
7	NtDeviceIoControlFile	0xFFFFF8036EFE8770	0xFFFFF8036EFE8770	C:\Windows\system32\ntoskrnl.exe

直接步入正题，首先 SSDT 表中文为系统服务描述符表，SSDT表的作用是把应用层与内核层联系起来起到桥梁的作用，枚举 SSDT表也是反内核工具最基本的功能，通常在 64 位系统中要想找到 SSDT 表，需要先找到 `KeServiceDescriptorTable` 这个函数，由于该函数没有被导出，所以只能动态的查找它的地址，庆幸的是我们可以通过查找 `msr(c0000082)` 这个特殊的寄存器来替代查找 `KeServiceDescriptorTable` 这一步，在新版系统中查找 SSDT 可以归纳为如下这几个步骤。

- `rdmsr c0000082 -> KiSystemCall64Shadow -> KiSystemServiceUser -> SSDT`

首先第一步通过 `rdmsr c0000082` MSR 寄存器得到 `KiSystemCall64Shadow` 的函数地址，计算 `KiSystemCall64Shadow` 与 `KiSystemServiceUser` 偏移量，如下图所示。

- 得到相对偏移 `6ed53180(KiSystemCall64Shadow) - 6ebd2a82(KiSystemServiceUser) = 1806FE`
- 也就是说 `6ed53180(rdmsr) - 1806FE = KiSystemServiceUser`

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help

Command
0: kd> .printf "hello lyshark.com \n\n"
hello lyshark.com

0: kd> rdmsr c0000082
msr[c0000082] = fffff803`6ed53180
0: kd> u KiSystemCall64Shadow
nt!KiSystemCall64Shadow:
fffff803`6ed53180 0f01f8 swapgs
fffff803`6ed53183 654889242510700000 mov     qword ptr gs:[7010h],rsp
fffff803`6ed5318c 65488b242500700000 mov     rsp,qword ptr gs:[7000h]
fffff803`6ed53195 650fba24251870000001 bt     dword ptr gs:[7018h],1
fffff803`6ed5319f 7203 jb     nt!KiSystemCall64Shadow+0x24 (fffff803`6ed531a4)
fffff803`6ed531a1 0f22dc mov     cr3,rsp
fffff803`6ed531a4 65488b242508700000 mov     rsp,qword ptr gs:[7008h]
fffff803`6ed531ad 6a2b push   2Bh
0: kd> u KiSystemServiceUser
nt!KiSystemServiceUser:
fffff803`6ebd2a82 c645ab02 mov     byte ptr [rbp-55h],2
fffff803`6ebd2a86 65488b1c2588010000 mov     rbx,qword ptr gs:[188h]
fffff803`6ebd2a8f 0f0d8b9000000000 prefetchw [rbx+90h]
fffff803`6ebd2a96 0fae5dac stmxcsr dword ptr [rbp-54h]
fffff803`6ebd2a9a 650fae142580010000 ldmxcsr dword ptr gs:[180h]
fffff803`6ebd2aa3 807b0300 cmp     byte ptr [rbx+3],0
fffff803`6ebd2aa7 66c785800000000000 mov     word ptr [rbp+80h],0
fffff803`6ebd2ab0 0f84a800000000 je     nt!KiSystemServiceUser+0xdc (fffff803`6ebd2b5e)
```

如上当我们找到了 `KiSystemServiceUser` 的地址以后，在 `KiSystemServiceUser` 向下搜索可找到 `KiSystemServiceRepeat` 里面就是我们要找的 SSDT 表基址。

其中 `fffff8036ef8c880` 则是 SSDT 表的基址，紧随其后的 `fffff8036ef74a80` 则是 SSSDT 表的基址。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help

Command
nt!KiSystemServiceRepeat:
fffff803`6ebd2b94 4c8d15e59c3b00 lea     r10,[nt!KeServiceDescriptorTable (fffff803`6ef8c880)]
fffff803`6ebd2b9b 4c8d1d0e3a00 lea     r11,[nt!KeServiceDescriptorTableShadow (fffff803`6ef74a80)]
fffff803`6ebd2ba2 f743788000000000 test    dword ptr [rbx+78h],80h
fffff803`6ebd2ba9 7413 je     nt!KiSystemServiceRepeat+0x2a (fffff803`6ebd2bbe) Branch

nt!KiSystemServiceRepeat+0x17:
fffff803`6ebd2bab f7437800002000 test    dword ptr [rbx+78h],200000h
fffff803`6ebd2bb2 7407 je     nt!KiSystemServiceRepeat+0x27 (fffff803`6ebd2bbb) Branch

nt!KiSystemServiceRepeat+0x20:
fffff803`6ebd2bb4 4c8d1d051f3a00 lea     r11,[nt!KeServiceDescriptorTableFilter (fffff803`6ef74ac0)]

nt!KiSystemServiceRepeat+0x27:
fffff803`6ebd2bbb 4d8bd3 mov     r10,r11
```

那么如果将这个过程通过代码的方式来实现，我们还需要使用《驱动开发：内核枚举 IoTimer 定时器》中所使用的特征码定位技术，如下我们查找这段特征。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#pragma intrinsic(__readmsr)

ULONGLONG ssdt_address = 0;

// 获取 KeServiceDescriptorTable 首地址
ULONGLONG GetLySharkCOMKeServiceDescriptorTable()
{
    // 设置起始位置
    PCHAR StartSearchAddress = (PCHAR)__readmsr(0xc0000082) - 0x1806FE;

    // 设置结束位置
    PCHAR EndSearchAddress = StartSearchAddress + 0x100000;
    DbgPrint("[LyShark Search] 扫描起始地址: %p --> 扫描结束地址: %p \n",
    StartSearchAddress, EndSearchAddress);

    PCHAR ByteCode = NULL;

    UCHAR OpCodeA = 0, OpCodeB = 0, OpCodeC = 0;
    ULONGLONG addr = 0;
    ULONG templong = 0;

    for (ByteCode = StartSearchAddress; ByteCode < EndSearchAddress; ByteCode++)
    {
        // 使用MmIsAddressValid()函数检查地址是否有页面错误
        if (MmIsAddressValid(ByteCode) && MmIsAddressValid(ByteCode + 1) &&
        MmIsAddressValid(ByteCode + 2))
        {
            OpCodeA = *ByteCode;
            OpCodeB = *(ByteCode + 1);
            OpCodeC = *(ByteCode + 2);

            // 对比特征值 寻找 nt!KeServiceDescriptorTable 函数地址
            /*
            nt!KiSystemServiceRepeat:
            fffff803`6ebd2b94 4c8d15e59c3b00 lea     r10,
[nt!KeServiceDescriptorTable (fffff803`6ef8c880)]
            fffff803`6ebd2b9b 4c8d1dde1e3a00 lea     r11,
[nt!KeServiceDescriptorTableshadow (fffff803`6ef74a80)]
            fffff803`6ebd2ba2 f7437880000000 test    dword ptr [rbx+78h],80h
            fffff803`6ebd2ba9 7413                      je
nt!KiSystemServiceRepeat+0x2a (fffff803`6ebd2bbe) Branch
            */
            if (OpCodeA == 0x4c && OpCodeB == 0x8d && OpCodeC == 0x15)
            {
                // 获取高位地址fffff802
                memcpy(&templong, ByteCode + 3, 4);

                // 与低位64da4880地址相加得到完整地址
                addr = (ULONGLONG)templong + (ULONGLONG)ByteCode + 7;
            }
        }
    }
}

```

```

        return addr;
    }
}
}
return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com");

    ssdt_address = GetLySharkCOMKeServiceDescriptorTable();
    DbgPrint("[LyShark] SSDT = %p \n", ssdt_address);

    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

如上代码中所提及的步骤我想不需要再做解释了，这段代码运行后即可输出SSDT表的基址。

#	Time	Debug Print
24	18.90486145	2925468750 - STORMINI: StorNVMe - POWER: ACTIVE
25	19.13070488	hello lyshark.com
26	19.13571739	[LyShark Search] 扫描起始地址: FFFFF8036EBD2A82 --> 扫描结束地址: FFFFF8036EBD2A82
27	19.14109993	[LyShark] SSDT = FFFFF8036EF8C880
28	20.04025650	2935468750 - STORMINI: StorNVMe - POWER: IDLE
29	20.15188217	2936562500 - STORMINI: StorNVMe - POWER: ACTIVE
30	21.17152405	2946718750 - STORMINI: StorNVMe - POWER: IDLE
31	24.79397392	2982812500 - STORMINI: StorNVMe - POWER: ACTIVE
32	24.80368996	2982812500 - STORMINI: StorNVMe - POWER: IDLE
33	25.69594955	2991718750 - STORMINI: StorNVMe - POWER: ACTIVE
34	25.70223618	2991718750 - STORMINI: StorNVMe - POWER: IDLE
35	25.71316338	2991718750 - STORMINI: StorNVMe - POWER: ACTIVE
36	26.72927666	3001875000 - STORMINI: StorNVMe - POWER: IDLE
37	29.84280968	3032968750 - STORMINI: StorNVMe - POWER: ACTIVE
38	29.85138512	3032968750 - STORMINI: StorNVMe - POWER: IDLE
39	29.85584068	3032968750 - STORMINI: StorNVMe - POWER: ACTIVE
40	30.86251831	3042968750 - STORMINI: StorNVMe - POWER: IDLE

如上通过调用 `GetLySharkCOMKeServiceDescriptorTable()` 得到 SSDT 地址以后我们就需要对该地址进行解密操作。

得到 `ServiceTableBase` 的地址后，就能得到每个服务函数的地址。但这个表存放的并不是 SSDT 函数的完整地址，而是其相对于 `ServiceTableBase[Index]>>4` 的数据，每个数据占四个字节，所以计算指定 Index 函数完整地址的公式是；

- 在x86平台上: $\text{FuncAddress} = \text{KeServiceDescriptorTable} + 4 * \text{Index}$
- 在x64平台上: $\text{FuncAddress} = [\text{KeServiceDescriptorTable} + 4 * \text{Index}] >> 4 + \text{KeServiceDescriptorTable}$

如下汇编代码就是一段解密代码，代码中 `rcx` 寄存器传入SSDT的下标，而 `rdx` 寄存器则是传入SSDT表基址。

```
48:8BC1          | mov rax,rcx          |
rcx=index
4C:8D12          | lea r10,qword ptr ds:[rdx]          | rdx=ssdt
8BF8           | mov edi,eax          |
C1EF 07         | shr edi,7            |
83E7 20         | and edi,20           |
4E:8B1417       | mov r10,qword ptr ds:[rdi+r10]      |
4D:631C82       | movsxd r11,dword ptr ds:[r10+rax*4] |
49:8BC3         | mov rax,r11          |
49:C1FB 04       | sar r11,4            |
4D:03D3         | add r10,r11          |
49:8BC2         | mov rax,r10          |
C3             | ret                  |
```

有了解密公式以后代码的编写就变得很容易，如下是读取SSDT的完整代码。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#pragma intrinsic(__readmsr)

typedef struct _SYSTEM_SERVICE_TABLE
{
    PVOID      ServiceTableBase;
    PVOID      ServiceCounterTableBase;
    ULONGLONG  NumberOfServices;
    PVOID      ParamTableBase;
} SYSTEM_SERVICE_TABLE, *PSYSTEM_SERVICE_TABLE;

ULONGLONG ssdt_base_aadress;
PSYSTEM_SERVICE_TABLE KeServiceDescriptorTable;

typedef UINT64(__fastcall *SCFN)(UINT64, UINT64);
SCFN scfn;

// 解密算法
VOID DecodeSSDT()
{
    UCHAR strShellCode[36] =
    "\x48\x8B\xC1\x4C\x8D\x12\x8B\xF8\xC1\xEF\x07\x83\xE7\x20\x4E\x8B\x14\x17\x4D\x63\x1C\x82\x49\x8B\xC3\x49\xC1\xFB\x04\x4D\x03\xD3\x49\x8B\xC2\xC3";
    /*
    48:8BC1          | mov rax,rcx          |
rcx=index
    4C:8D12          | lea r10,qword ptr ds:[rdx]          |
rdx=ssdt
    8BF8           | mov edi,eax          |
    C1EF 07         | shr edi,7            |
```



```

83E7 20          | and edi,20          |
4E:8B1417        | mov r10,qword ptr ds:[rdi+r10] |
4D:631C82        | movsxd r11,dword ptr ds:[r10+rax*4] |
49:8BC3          | mov rax,r11         |
49:C1FB 04       | sar r11,4           |
4D:03D3          | add r10,r11         |
49:8BC2          | mov rax,r10         |
C3              | ret                 |
*/
scfn = ExAllocatePool(NonPagedPool, 36);
memcpy(scfn, strShellCode, 36);
}

// 获取 KeServiceDescriptorTable 首地址
ULONGLONG GetKeServiceDescriptorTable()
{
    // 设置起始位置
    PCHAR StartSearchAddress = (PCHAR)__readmsr(0xC0000082) - 0x1806FE;

    // 设置结束位置
    PCHAR EndSearchAddress = StartSearchAddress + 0x8192;
    DbgPrint("扫描起始地址: %p --> 扫描结束地址: %p \n", StartSearchAddress,
EndSearchAddress);

    PCHAR ByteCode = NULL;

    UCHAR OpCodeA = 0, OpCodeB = 0, OpCodeC = 0;
    ULONGLONG addr = 0;
    ULONG templong = 0;

    for (ByteCode = StartSearchAddress; ByteCode < EndSearchAddress; ByteCode++)
    {
        // 使用MmIsAddressValid()函数检查地址是否有页面错误
        if (MmIsAddressValid(ByteCode) && MmIsAddressValid(ByteCode + 1) &&
MmIsAddressValid(ByteCode + 2))
        {
            OpCodeA = *ByteCode;
            OpCodeB = *(ByteCode + 1);
            OpCodeC = *(ByteCode + 2);

            // 对比特征值 寻找 nt!KeServiceDescriptorTable 函数地址
            // LyShark.com
            // 4c 8d 15 e5 9e 3b 00 lea r10,[nt!KeServiceDescriptorTable
(fffff802`64da4880)]
            // 4c 8d 1d de 20 3a 00 lea r11,[nt!KeServiceDescriptorTableShadow
(fffff802`64d8ca80)]
            if (OpCodeA == 0x4c && OpCodeB == 0x8d && OpCodeC == 0x15)
            {
                // 获取高位地址fffff802
                memcpy(&templong, ByteCode + 3, 4);

                // 与低位64da4880地址相加得到完整地址
                addr = (ULONGLONG)templong + (ULONGLONG)ByteCode + 7;
                return addr;
            }
        }
    }
}

```

```

    }
}
return 0;
}

// 得到函数相对偏移地址
ULONG GetOffsetAddress(ULONGLONG FuncAddr)
{
    ULONG dwtmp = 0;
    PULONG ServiceTableBase = NULL;
    if (KeServiceDescriptorTable == NULL)
    {
        KeServiceDescriptorTable =
        (PSYSTEM_SERVICE_TABLE)GetKeServiceDescriptorTable();
    }
    ServiceTableBase = (PULONG)KeServiceDescriptorTable->ServiceTableBase;
    dwtmp = (ULONG)(FuncAddr - (ULONGLONG)ServiceTableBase);
    return dwtmp << 4;
}

// 根据序号得到函数地址
ULONGLONG GetSSDTFunctionAddress(ULONGLONG NtApiIndex)
{
    ULONGLONG ret = 0;
    if (ssdt_base_address == 0)
    {
        // 得到ssdt基地址
        ssdt_base_address = GetKeServiceDescriptorTable();
    }
    if (scfn == NULL)
    {
        DecodeSSDT();
    }
    ret = scfn(NtApiIndex, ssdt_base_address);
    return ret;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONGLONG ssdt_address = GetKeServiceDescriptorTable();
    DbgPrint("SSDT基地址 = %p \n", ssdt_address);

    // 根据序号得到函数地址
    ULONGLONG address = GetSSDTFunctionAddress(51);
    DbgPrint("[LyShark] NtOpenFile地址 = %p \n", address);

    // 得到相对SSDT的偏移量
    DbgPrint("函数相对偏移地址 = %p \n", GetOffsetAddress(address));
}

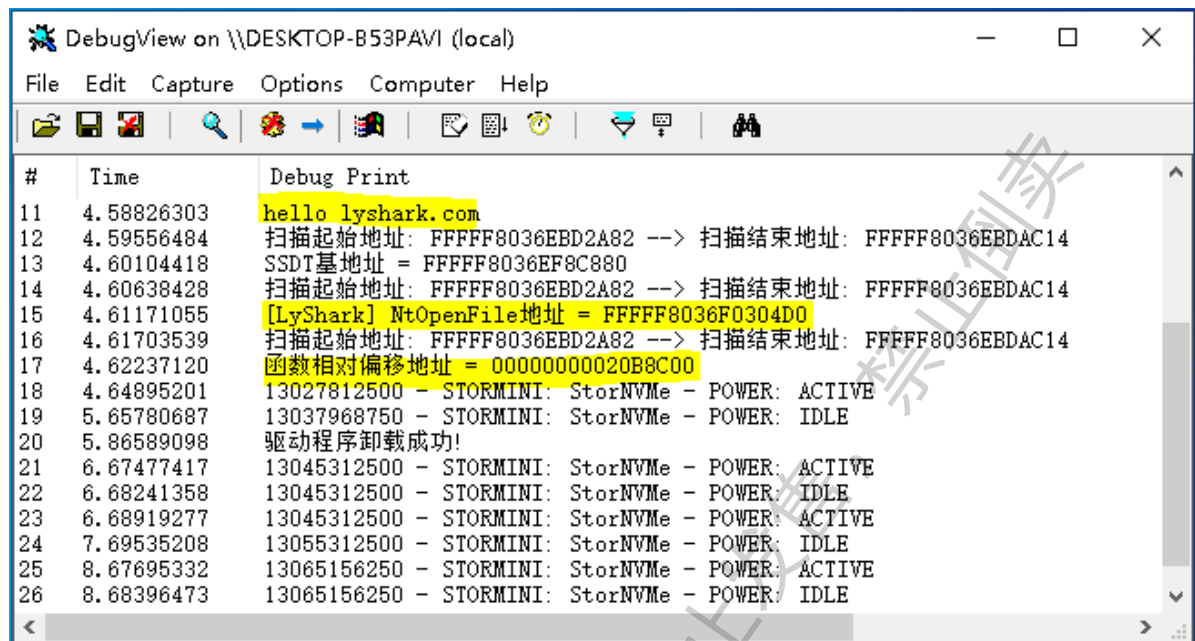
```

```

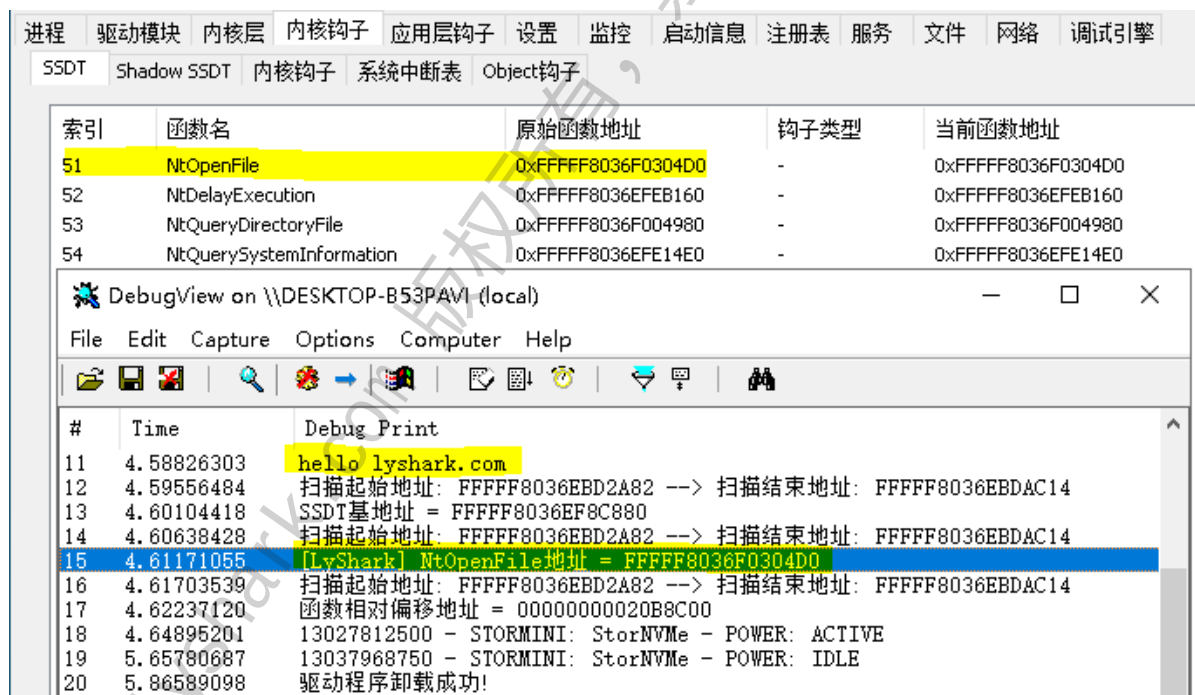
DriverObject->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

运行后即可得到 SSDT 下标为 51 的函数也就是得到 NtOpenFile 的绝对地址和相对地址。



你也可以打开ARK工具，对比一下是否一致，如下图所示，Lyshark 的代码是没有任何问题的。



5.2 枚举ShadowSSDT表基址

在笔者上一篇文章《驱动开发：win10枚举完整SSDT地址表》实现了针对 SSDT 表的枚举功能，本章继续实现对 SSSDT 表的枚举，ShadowSSDT 中文名 影子系统服务描述表，SSSDT其主要的的作用是管理系统中的图形化界面，其 win32 子系统的内核实现是 win32k.sys 驱动，属于 GUI 线程的一部分，其自身没有导出表，枚举 SSSDT 表其与 SSDT 原理基本一致。

如下是闭源ARK工具的枚举效果：

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
SSDT	Shadow SSDT	内核钩子	系统中断表	Object钩子								

索引	函数名	原始函数地址	钩子类型	当前函数地址
0	NtUserGetOwnerTransformedMoni...	0xFFFFFAD520FE2282	-	0xFFFFFAD520FE2282
1	NtUserYieldTask	0xFFFFFAD520FE2294	-	0xFFFFFAD520FE2294
2	NtUserSetSensorPresence	0xFFFFFAD520FE22A6	-	0xFFFFFAD520FE22A6
3	NtUserGetThreadState	0xFFFFFAD520FE22B8	-	0xFFFFFAD520FE22B8
4	NtUserPeekMessage	0xFFFFFAD520FE22CA	-	0xFFFFFAD520FE22CA
5	NtUserCallOneParam	0xFFFFFAD520FE22DC	-	0xFFFFFAD520FE22DC
6	NtUserGetKeyState	0xFFFFFAD520FE22EE	-	0xFFFFFAD520FE22EE
7	NtUserInvalidateRect	0xFFFFFAD520FE2300	-	0xFFFFFAD520FE2300

首先需要找到 SSDT 表的位置，通过《驱动开发：win10内核枚举SSDT表基址》文章中的分析可知，SSSDT就在SSDT的下面，只需要枚举 4c8d1dde1e3a00 特征即可，如果你找不到上一篇具体分析流程了，那么多半你是看到了转载文章。

```

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
0: kd> u KiSystemServiceRepeat
nt!KiSystemServiceRepeat:
fffff802`7c1d2b94 4c8d15e59c3b00 lea     r10,[nt!KeServiceDescriptorTable (fffff802`7c58c880)]
fffff802`7c1d2b9b 4c8d1dde1e3a00 lea     r11,[nt!KeServiceDescriptorTableShadow (fffff802`7c574a80)]
fffff802`7c1d2ba2 f7437880000000 test    dword ptr [rbx+78h],80h
fffff802`7c1d2ba9 7413                      je      nt!KiSystemServiceRepeat+0x2a (fffff802`7c1d2bbe)
fffff802`7c1d2bab f7437800002000 test    dword ptr [rbx+78h],200000h
fffff802`7c1d2bb2 7407                      je      nt!KiSystemServiceRepeat+0x27 (fffff802`7c1d2bbb)
fffff802`7c1d2bb4 4c8d1d051f3a00 lea     r11,[nt!KeServiceDescriptorTableFilter (fffff802`7c574ac0)]
fffff802`7c1d2bbb 4d8bd3                    mov     r10,r11

```

先实现第一个功能，得到 SSDT 表的基地址以及 SSDT 函数个数，完整代码如下所示。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#pragma intrinsic(__readmsr)

typedef struct _SYSTEM_SERVICE_TABLE
{
    PVOID          ServiceTableBase;
    PVOID          ServiceCounterTableBase;
    ULONGLONG      NumberOfServices;
    PVOID          ParamTableBase;
} SYSTEM_SERVICE_TABLE, *PSYSTEM_SERVICE_TABLE;

PSYSTEM_SERVICE_TABLE KeServiceDescriptorTableShadow = 0;
ULONG64 ul64W32pServiceTable = 0;

// 获取 KeServiceDescriptorTableShadow 首地址
ULONGLONG GetKeServiceDescriptorTableShadow()
{
    // 设置起始位置
    PUCHAR StartSearchAddress = (PUCHAR)__readmsr(0xc0000082) - 0x1808FE;

    // 设置结束位置
    PUCHAR EndSearchAddress = StartSearchAddress + 0x8192;
}

```

```

// DbgPrint("扫描起始地址: %p --> 扫描结束地址: %p \n", StartSearchAddress,
EndSearchAddress);

PUCHAR ByteCode = NULL;

UCHAR OpCodeA = 0, OpCodeB = 0, OpCodeC = 0;
ULONGLONG addr = 0;
ULONG templong = 0;

for (ByteCode = StartSearchAddress; ByteCode < EndSearchAddress; ByteCode++)
{
    // 使用MmIsAddressValid()函数检查地址是否有页面错误
    if (MmIsAddressValid(ByteCode) && MmIsAddressValid(ByteCode + 1) &&
MmIsAddressValid(ByteCode + 2))
    {
        OpCodeA = *ByteCode;
        OpCodeB = *(ByteCode + 1);
        OpCodeC = *(ByteCode + 2);

        // 对比特征值 寻找 nt!KeServiceDescriptorTable 函数地址
        /*
lyshark.com kd> u KiSystemServiceRepeat
nt!KiSystemServiceRepeat:
fffff802`7c1d2b94 4c8d15e59c3b00 lea     r10,
[nt!KeServiceDescriptorTable (fffff802`7c58c880)]
fffff802`7c1d2b9b 4c8d1dde1e3a00 lea     r11,
[nt!KeServiceDescriptorTableshadow (fffff802`7c574a80)]
fffff802`7c1d2ba2 f7437880000000 test    dword ptr
[rbx+78h],80h
fffff802`7c1d2ba9 7413                     je
nt!KiSystemServiceRepeat+0x2a (fffff802`7c1d2bbe)
fffff802`7c1d2bab f7437800002000 test    dword ptr
[rbx+78h],200000h
fffff802`7c1d2bb2 7407                     je
nt!KiSystemServiceRepeat+0x27 (fffff802`7c1d2bbb)
fffff802`7c1d2bb4 4c8d1d051f3a00 lea     r11,
[nt!KeServiceDescriptorTableFilter (fffff802`7c574ac0)]
fffff802`7c1d2bbb 4d8bd3                   mov     r10,r11
*/
        if (OpCodeA == 0x4c && OpCodeB == 0x8d && OpCodeC == 0x1d)
        {
            // 获取高位地址fffff802
            memcpy(&templong, ByteCode + 3, 4);

            // 与低位64da4880地址相加得到完整地址
            addr = (ULONGLONG)templong + (ULONGLONG)ByteCode + 7;
            return addr;
        }
    }
}

return 0;
}

// 得到SSSDT个数
ULONGLONG GetSSSDTCount()

```

```

{
    PSYSTEM_SERVICE_TABLE pwin32k;
    ULONGLONG w32pServiceTable;

    pwin32k = (PSYSTEM_SERVICE_TABLE)((ULONG64)KeServiceDescriptorTableShadow +
sizeof(SYSTEM_SERVICE_TABLE));
    w32pServiceTable = (ULONGLONG)(pwin32k->ServiceTableBase);
    // DbgPrint("Count => %d \n", pwin32k->NumberOfServices);

    return pwin32k->NumberOfServices;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    KeServiceDescriptorTableShadow =
(PSYSTEM_SERVICE_TABLE)GetKeServiceDescriptorTableShadow();

    DbgPrint("[LyShark] SSSDT基地址 = 0x%p \n", KeServiceDescriptorTableShadow);

    ULONGLONG count = GetSSSDTCount();

    DbgPrint("[LyShark] SSSDT个数 = %d \n", count);

    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

这段代码运行后即可得到 SSSDT 表基地址，以及该表中函数个数。

#	Time	Debug Print
1	0.00000000	8040468750 - STORMINI: StorNVMe - POWER: IDLE
2	0.23547520	8042812500 - STORMINI: StorNVMe - POWER: ACTIVE
3	0.39969549	hello lyshark.com
4	0.40356329	[LyShark] SSSDT基地址 = 0xFFFFF8027C574A80
5	0.40890101	[LyShark] SSSDT个数 = 1258
6	1.17934072	驱动程序卸载成功!
7	1.66242564	8052812500 - STORMINI: StorNVMe - POWER: IDLE
8	1.77664435	8053593750 - STORMINI: StorNVMe - POWER: ACTIVE

在此基础之上增加枚举计算过程即可，完整源代码如下所示。

SSSDT 函数起始index是 0x1000，但 w32pServiceTable 是从基址开始记录的，这个误差则需要 (index-0x1000) 来得到，至于 +4 则是下一个元素与上一个元素的偏移。

计算公式：

- $W32pServiceTable + 4 * (index - 0x1000)$

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#pragma intrinsic(__readmsr)

typedef struct _SYSTEM_SERVICE_TABLE
{
    PVOID          ServiceTableBase;
    PVOID          ServiceCounterTableBase;
    ULONGLONG      NumberOfServices;
    PVOID          ParamTableBase;
} SYSTEM_SERVICE_TABLE, *PSYSTEM_SERVICE_TABLE;

PSYSTEM_SERVICE_TABLE KeServiceDescriptorTableShadow = 0;
ULONG64 u164W32pServiceTable = 0;

// 获取 KeServiceDescriptorTableShadow 首地址
ULONGLONG GetKeServiceDescriptorTableShadow()
{
    // 设置起始位置
    PCHAR StartSearchAddress = (PCHAR)__readmsr(0xC0000082) - 0x1808FE;

    // 设置结束位置
    PCHAR EndSearchAddress = StartSearchAddress + 0x8192;
    // DbgPrint("扫描起始地址: %p --> 扫描结束地址: %p \n", StartSearchAddress,
    EndSearchAddress);

    PCHAR ByteCode = NULL;

    UCHAR OpCodeA = 0, OpCodeB = 0, OpCodeC = 0;
    ULONGLONG addr = 0;
    ULONG tempLong = 0;

    for (ByteCode = StartSearchAddress; ByteCode < EndSearchAddress; ByteCode++)
    {
        // 使用MmIsAddressValid()函数检查地址是否有页面错误
        if (MmIsAddressValid(ByteCode) && MmIsAddressValid(ByteCode + 1) &&
            MmIsAddressValid(ByteCode + 2))
        {
            OpCodeA = *ByteCode;
            OpCodeB = *(ByteCode + 1);
            OpCodeC = *(ByteCode + 2);

            // 对比特征值 寻找 nt!KeServiceDescriptorTable 函数地址
            /*
            lyshark.com kd> u KiSystemServiceRepeat
```

```

        nt!KiSystemServiceRepeat:
            ffffffff802`7c1d2b94 4c8d15e59c3b00 lea     r10,
[nt!KeServiceDescriptorTable (fffff802`7c58c880)]
            ffffffff802`7c1d2b9b 4c8d1dde1e3a00 lea     r11,
[nt!KeServiceDescriptorTableShadow (fffff802`7c574a80)]
            ffffffff802`7c1d2ba2 f7437880000000 test    dword ptr [rbx+78h],80h
            ffffffff802`7c1d2ba9 7413                      je
nt!KiSystemServiceRepeat+0x2a (fffff802`7c1d2bbe)
            ffffffff802`7c1d2bab f7437800002000 test    dword ptr
[rbx+78h],200000h
            ffffffff802`7c1d2bb2 7407                      je
nt!KiSystemServiceRepeat+0x27 (fffff802`7c1d2bbb)
            ffffffff802`7c1d2bb4 4c8d1d051f3a00 lea     r11,
[nt!KeServiceDescriptorTableFilter (fffff802`7c574ac0)]
            ffffffff802`7c1d2bbb 4d8bd3                    mov     r10,r11
        */
        if (OpCodeA == 0x4c && OpCodeB == 0x8d && OpCodeC == 0x1d)
        {
            // 获取高位地址fffff802
            memcpy(&templong, ByteCode + 3, 4);

            // 与低位64da4880地址相加得到完整地址
            addr = (ULONGLONG)templong + (ULONGLONG)ByteCode + 7;
            return addr;
        }
    }
}
return 0;
}

// 得到SSSDT个数
ULONGLONG GetSSSDTCount()
{
    PSYSTEM_SERVICE_TABLE pwin32k;
    ULONGLONG w32pServiceTable;

    pwin32k = (PSYSTEM_SERVICE_TABLE)((ULONG64)KeServiceDescriptorTableShadow +
sizeof(SYSTEM_SERVICE_TABLE));
    w32pServiceTable = (ULONGLONG)(pwin32k->ServiceTableBase);
    // DbgPrint("Count => %d \n", pwin32k->NumberOfServices);

    return pwin32k->NumberOfServices;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    KeServiceDescriptorTableShadow =
(PSYSTEM_SERVICE_TABLE)GetKeServiceDescriptorTableShadow();

```



```

DbgPrint("[LyShark] SSSDT基地址 = 0x%p \n", KeServiceDescriptorTableShadow);

ULONGLONG count = GetSSSDTCount();

DbgPrint("[LyShark] SSSDT个数 = %d \n", count);

// 循环枚举SSSDT
for (size_t Index = 0; Index < count; Index++)
{

    PSYSTEM_SERVICE_TABLE pwin32k;
    ULONGLONG w32pServiceTable;

    pwin32k = (PSYSTEM_SERVICE_TABLE)
((ULONG64)KeServiceDescriptorTableShadow + sizeof(SYSTEM_SERVICE_TABLE));
    w32pServiceTable = (ULONGLONG)(pwin32k->ServiceTableBase);

    // 获取SSSDT地址
    //ln win32k!w32pServiceTable+((poi(win32k!w32pServiceTable+4*(1-
1000))&0x00000000`ffffffff)>>4)-10000000
    //u win32k!w32pServiceTable+((poi(win32k!w32pServiceTable+4*(Index-
0x1000))&0x00000000`ffffffff)>>4)-0x10000000

    //u poi(win32k!w32pServiceTable+4*(1-0x1000))
    //u poi(win32k!w32pServiceTable+4*(1-0x1000))&0x00000000`ffffffff
    //u (poi(win32k!w32pServiceTable+4*(1-0x1000))&0x00000000`ffffffff)>>4

    //u win32k!w32pServiceTable+((poi(win32k!w32pServiceTable+4*(1-
0x1000))&0x00000000`ffffffff)>>4)-0x10000000

    ULONGLONG qword_temp = 0;
    LONG dw = 0;

    // SSSDT 下标从1000开始，而w32pServiceTable是从0开始
    // + 4 则是每次向下4字节就是下一个地址
    qword_temp = w32pServiceTable + 4 * (Index - 0x1000);

    dw = *(PLONG)qword_temp;
    // dw = qword_temp & 0x00000000ffffffff;
    dw = dw >> 4;
    qword_temp = w32pServiceTable + (LONG64)dw;

    DbgPrint("[LyShark] ID: %d | SSSDT: 0x%p \n", Index, qword_temp);
}

DriverObject->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

枚举效果如下所示（存在问题）：

#	Time	Debug Print
4	0.00000000	hello lyshark.com
5	0.00482930	[LyShark] SSSDT地址 = 0xFFFFF8027C574A80
6	0.01016300	[LyShark] SSSDT个数 = 1258
7	0.01551020	[LyShark] ID: 0 SSSDT: 0xFFFFFAD523150800
8	0.02089910	[LyShark] ID: 1 SSSDT: 0xFFFFFAD523150800
9	0.02627180	[LyShark] ID: 2 SSSDT: 0xFFFFFAD523150800
10	0.03169280	[LyShark] ID: 3 SSSDT: 0xFFFFFAD523150800
11	0.03702110	[LyShark] ID: 4 SSSDT: 0xFFFFFAD523150801
12	0.04239580	[LyShark] ID: 5 SSSDT: 0xFFFFFAD523150801
13	0.04776480	[LyShark] ID: 6 SSSDT: 0xFFFFFAD523150801
14	0.05314800	[LyShark] ID: 7 SSSDT: 0xFFFFFAD523150801
15	0.05852630	[LyShark] ID: 8 SSSDT: 0xFFFFFAD523150802
16	0.06389920	[LyShark] ID: 9 SSSDT: 0xFFFFFAD523150802
17	0.06925580	[LyShark] ID: 10 SSSDT: 0xFFFFFAD523150802
18	0.07465830	[LyShark] ID: 11 SSSDT: 0xFFFFFAD523150802
19	0.07998320	[LyShark] ID: 12 SSSDT: 0xFFFFFAD523150803

注这一步必须要在GUI线程中执行，否则会异常，建议将枚举过程写成DLL文件，注入到 explorer.exe 进程内执行。

第六章：内核进程线程篇

6.1 内核中枚举进程与模块

内核枚举进程使用 `PspCidTable` 这个未公开的函数，它能最大的好处是能得到进程的 `EPROCESS` 地址，由于是未公开的函数，所以我们需要变通的调用这个函数，通过 `PsLookupProcessByProcessId` 函数查到进程的 `EPROCESS`，如果 `PsLookupProcessByProcessId` 返回失败，则证明此进程不存在，如果返回成功则把 `EPROCESS`、`PID`、`PPID`、进程名等通过 `DbgPrint` 打印到屏幕上。

内核枚举进程： 进程就是活动起来的程序，每一个进程在内核里，都有一个名为 `EPROCESS` 的结构记录它的详细信息，其中就包括进程名，`PID`，`PPID`，进程路径等，通常在应用层枚举进程只列出所有进程的编号即可，不过在内核层需要把它的 `EPROCESS` 地址给列举出来。

内核枚举进程使用 `PspCidTable` 这个未公开的函数，它能最大的好处是能得到进程的 `EPROCESS` 地址，由于是未公开的函数，所以我们需要变通的调用这个函数，通过 `PsLookupProcessByProcessId` 函数查到进程的 `EPROCESS`，如果 `PsLookupProcessByProcessId` 返回失败，则证明此进程不存在，如果返回成功则把 `EPROCESS`、`PID`、`PPID`、进程名等通过 `DbgPrint` 打印到屏幕上。

```
#include <ntifs.h>

NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process); //未公开的进行
导出即可
NTKERNELAPI HANDLE PsGetProcessInheritedFromUniqueProcessId(IN PEPROCESS
Process); //未公开进行导出

// 根据进程ID返回进程EPROCESS结构体,失败返回NULL
PEPROCESS LookupProcess(HANDLE Pid)
{
    PEPROCESS eprocess = NULL;
    NTSTATUS Status = STATUS_UNSUCCESSFUL;
    Status = PsLookupProcessByProcessId(Pid, &eprocess);
    if (NT_SUCCESS(Status))
        return eprocess;
}
```

```

        return NULL;
    }

VOID EnumProcess()
{
    PEPROCESS eproc = NULL;
    for (int temp = 0; temp < 100000; temp += 4)
    {
        eproc = LookupProcess((HANDLE)temp);
        if (eproc != NULL)
        {
            DbgPrint("进程名: %s --> 进程PID = %d --> 父进程PPID = %d\r\n", PsGetProcessImageFileName(eproc), PsGetProcessId(eproc), PsGetProcessInheritedFromUniqueProcessId(eproc));
            ObDereferenceObject(eproc);
        }
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    EnumProcess();
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

输出效果如下所示:

#	Time	Debug Print
24	0.00008860	进程名: MemCompression --> 进程PID = 1168 --> 父进程PPID = 4
25	0.00009630	进程名: svchost.exe --> 进程PID = 1324 --> 父进程PPID = 576
26	0.00009850	进程名: svchost.exe --> 进程PID = 1340 --> 父进程PPID = 576
27	0.00010260	进程名: svchost.exe --> 进程PID = 1416 --> 父进程PPID = 576
28	0.00010520	进程名: WmiPrvSE.exe --> 进程PID = 1448 --> 父进程PPID = 692
29	0.00010980	进程名: svchost.exe --> 进程PID = 1524 --> 父进程PPID = 576
30	0.00011990	进程名: VGAuthService. --> 进程PID = 1756 --> 父进程PPID = 576
31	0.00012200	进程名: vmtoolsd.exe --> 进程PID = 1764 --> 父进程PPID = 576
32	0.00013480	进程名: msdtc.exe --> 进程PID = 2132 --> 父进程PPID = 576
33	0.00014060	进程名: ApplicationFra --> 进程PID = 2232 --> 父进程PPID = 692
34	0.00014270	进程名: SystemSettings --> 进程PID = 2244 --> 父进程PPID = 692
35	0.00014500	进程名: svchost.exe --> 进程PID = 2268 --> 父进程PPID = 576
36	0.00014850	进程名: sihost.exe --> 进程PID = 2312 --> 父进程PPID = 1004
37	0.00015080	进程名: svchost.exe --> 进程PID = 2328 --> 父进程PPID = 576
38	0.00015550	进程名: taskhostw.exe --> 进程PID = 2400 --> 父进程PPID = 1004
39	0.00015880	进程名: smss.exe --> 进程PID = 2404 --> 父进程PPID = 4

内核枚举线程: 内核线程的枚举与进程相似，线程中也存在一个ETHREAD结构，但在枚举线程之前需要先枚举到指定进程的eprocess结构，然后根据eprocess结构对指定线程进行枚举。

```

#include <ntddk.h>
#include <windef.h>

// 声明API
NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE Id, PEPROCESS *Process);
NTKERNELAPI NTSTATUS PsLookupThreadByThreadId(HANDLE Id, PETHREAD *Thread);
NTKERNELAPI PEPROCESS IoThreadToProcess(PETHREAD Thread);

```

```

//根据进程ID返回进程EPROCESS，失败返回NULL
PEPROCESS LookupProcess(HANDLE Pid)
{
    PEPROCESS eprocess = NULL;
    if (NT_SUCCESS(PsLookupProcessByProcessId(Pid, &eprocess)))
        return eprocess;
    else
        return NULL;
}

//根据线程ID返回线程ETHREAD，失败返回NULL
PETHREAD LookupThread(HANDLE Tid)
{
    PETHREAD ethread;
    if (NT_SUCCESS(PsLookupThreadByThreadId(Tid, &ethread)))
        return ethread;
    else
        return NULL;
}

//枚举指定进程中的线程
VOID EnumThread(PEPROCESS Process)
{
    ULONG i = 0, c = 0;
    PETHREAD ethrd = NULL;
    PEPROCESS eproc = NULL;
    for (i = 4; i < 262144; i = i + 4) // 一般来说没有超过100000的PID和TID
    {
        ethrd = LookupThread((HANDLE)i);
        if (ethrd != NULL)
        {
            //获得线程所属进程
            eproc = IoThreadToProcess(ethrd);
            if (eproc == Process)
            {
                //打印出ETHREAD和TID
                DbgPrint("线程: ETHREAD=%p TID=%ld\n", ethrd,
                (ULONG)PsGetThreadId(ethrd));
            }
            ObDereferenceObject(ethrd);
        }
    }
}

// 通过枚举的方式定位到指定的进程，这里传递一个进程名称
VOID MyEnumThread(char *ProcessName)
{
    ULONG i = 0;
    PEPROCESS eproc = NULL;
    for (i = 4; i < 100000000; i = i + 4)
    {
        eproc = LookupProcess((HANDLE)i);
        if (eproc != NULL)
        {

```

```

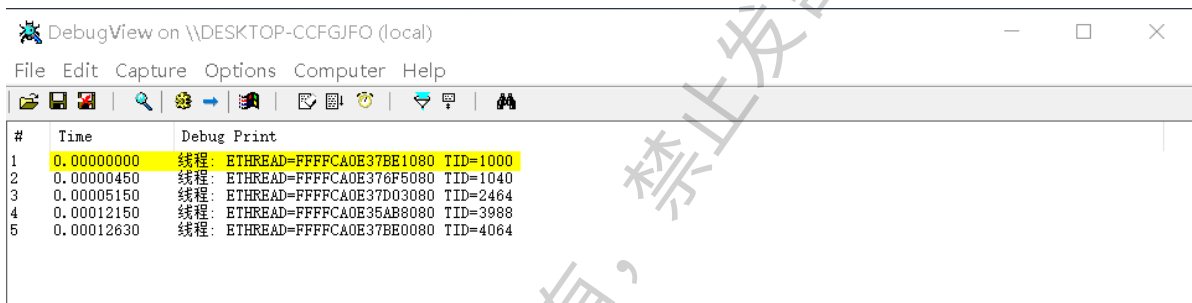
        ObDereferenceObject(eproc);
        if (strstr(PsGetProcessImageFileName(eproc), ProcessName) != NULL)
        {
            EnumThread(eproc); // 相等则说明是我们想要的进程，直接枚举其中的线程
        }
    }
}

VOID DriverUnload(IN PDRIVER_OBJECT DriverObject){}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING
RegistryPath)
{
    MyEnumThread("calc.exe");
    DriverObject->DriverUnload = DriverUnload;
    return STATUS_SUCCESS;
}

```

输出效果如下所示：



内核枚举进程模块：枚举进程中的所有模块信息，DLL模块记录在 PEB 的 LDR 链表里，LDR 是一个双向链表，枚举链表即可，相应的卸载可使用 `MmUnmapViewOfSection` 函数，分别传入进程的 `EPROCESS`，DLL 模块基址即可。

```

#include <ntddk.h>
#include <windef.h>

//声明结构体
typedef struct _KAPC_STATE
{
    LIST_ENTRY ApcListHead[2];
    PKPROCESS Process;
    UCHAR KernelApcInProgress;
    UCHAR KernelApcPending;
    UCHAR UserApcPending;
} KAPC_STATE, *PKAPC_STATE;

typedef struct _LDR_DATA_TABLE_ENTRY
{
    LIST_ENTRY64 InLoadOrderLinks;
    LIST_ENTRY64 InMemoryOrderLinks;
    LIST_ENTRY64 InInitializationOrderLinks;
    PVOID DllBase;
    PVOID EntryPoint;
    ULONG SizeOfImage;
}

```

```

        UNICODE_STRING  FullDllName;
        UNICODE_STRING  BaseDllName;
        ULONG           Flags;
        USHORT          LoadCount;
        USHORT          TlsIndex;
        PVOID           SectionPointer;
        ULONG           CheckSum;
        PVOID           LoadedImports;
        PVOID           EntryPointActivationContext;
        PVOID           PatchInformation;
        LIST_ENTRY64     ForwarderLinks;
        LIST_ENTRY64     ServiceTagLinks;
        LIST_ENTRY64     StaticLinks;
        PVOID           ContextInformation;
        ULONG64          OriginalBase;
        LARGE_INTEGER     LoadTime;
    } LDR_DATA_TABLE_ENTRY, *PLDR_DATA_TABLE_ENTRY;

```

```

ULONG64 LdrInPebOffset = 0x018;    //peb.ldr
ULONG64 ModListInPebOffset = 0x010; //peb.ldr.InLoadOrderModuleList

```

//声明API

```

NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process);
NTKERNELAPI PPEB PsGetProcessPeb(PEPROCESS Process);
NTKERNELAPI HANDLE PsGetProcessInheritedFromUniqueProcessId(IN PEPROCESS
Process);

```

//根据进程ID返回进程EPROCESS，失败返回NULL

```

PEPROCESS LookupProcess(HANDLE Pid)
{
    PEPROCESS eprocess = NULL;
    if (NT_SUCCESS(PsLookupProcessByProcessId(Pid, &eprocess)))
        return eprocess;
    else
        return NULL;
}

```

//枚举指定进程的模块

```

VOID EnumModule(PEPROCESS Process)
{
    SIZE_T Peb = 0;
    SIZE_T Ldr = 0;
    PLIST_ENTRY ModListHead = 0;
    PLIST_ENTRY Module = 0;
    ANSI_STRING AnsiString;
    KAPC_STATE ks;
    //EPROCESS地址无效则退出
    if (!MmIsValidAddress(Process))
        return;
    //获取PEB地址
    Peb = (SIZE_T)PsGetProcessPeb(Process);
    //PEB地址无效则退出
    if (!Peb)
        return;
}

```

```

//依附进程
KeStackAttachProcess(Process, &ks);
__try
{
    //获得LDR地址
    Ldr = Peb + (SIZE_T)LdrInPebOffset;
    //测试是否可读,不可读则抛出异常退出
    ProbeForRead((CONST PVOID)Ldr, 8, 8);
    //获得链表头
    ModListHead = (PLIST_ENTRY)(*(PULONG64)Ldr + ModListInPebOffset);
    //再次测试可读性
    ProbeForRead((CONST PVOID)ModListHead, 8, 8);
    //获得第一个模块的信息
    Module = ModListHead->Flink;
    while (ModListHead != Module)
    {
        //打印信息: 基址、大小、DLL路径
        DbgPrint("模块基址=%p 大小=%ld 路径=%wZ\n", (PVOID)
            (((PLDR_DATA_TABLE_ENTRY)Module)->DllBase),
            (ULONG)(((PLDR_DATA_TABLE_ENTRY)Module)->SizeOfImage), &
            (((PLDR_DATA_TABLE_ENTRY)Module)->FullDllName));
        Module = Module->Flink;
        //测试下一个模块信息的可读性
        ProbeForRead((CONST PVOID)Module, 80, 8);
    }
}
__except (EXCEPTION_EXECUTE_HANDLER){;}
//取消依附进程
KeUnstackDetachProcess(&ks);
}

// 通过枚举的方式定位到指定的进程,这里传递一个进程名称
VOID MyEnumModule(char *ProcessName)
{
    ULONG i = 0;
    PEPROCESS eproc = NULL;
    for (i = 4; i < 100000000; i = i + 4)
    {
        eproc = LookupProcess((HANDLE)i);
        if (eproc != NULL)
        {
            ObpReferenceObject(eproc);
            if (strstr(PsGetProcessImageFileName(eproc), ProcessName) != NULL)
            {
                EnumModule(eproc); // 相等则说明是我们想要的进程,直接枚举其中的线程
            }
        }
    }
}

VOID DriverUnload(IN PDRIVER_OBJECT DriverObject){}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING
RegistryPath)
{

```

```

MyEnumModule("calc.exe");
DriverObject->DriverUnload = DriverUnload;
return STATUS_SUCCESS;
}

```

输出效果如下所示：

#	Time	Debug Print
1	0.00000000	[2084] Suspending
2	7.90246582	模块基址=00007FF752E80000 大小=720896 路径=C:\Windows\System32\win32calc.exe
3	7.90247059	模块基址=00007FFCE0950000 大小=2019328 路径=C:\Windows\SYSTEM32\ntdll.dll
4	7.90247297	模块基址=00007FFCEB700000 大小=733184 路径=C:\Windows\System32\KERNEL32.DLL
5	7.90247583	模块基址=00007FFCDD7F0000 大小=2699264 路径=C:\Windows\System32\KERNELBASE.dll
6	7.90247917	模块基址=00007FFCDD7F0000 大小=21950464 路径=C:\Windows\System32\SHELL32.dll
7	7.90248156	模块基址=00007FFCDEE00000 大小=647168 路径=C:\Windows\System32\msvcrt.dll
8	7.90248394	模块基址=00007FFCDD7A0000 大小=303104 路径=C:\Windows\System32\cfgmgr32.dll
9	7.90248632	模块基址=00007FFCDDCE0000 大小=1032192 路径=C:\Windows\System32\ucrtbase.dll
10	7.90248966	模块基址=00007FFCE08A0000 大小=688128 路径=C:\Windows\System32\shcore.dll
11	7.90249205	模块基址=00007FFCDEEA0000 大小=1187840 路径=C:\Windows\System32\RPCRT4.dll
12	7.90249443	模块基址=00007FFCDD7A0000 大小=303104 路径=C:\Windows\System32\cfgmgr32.dll

内核枚举加载SYS文件： 内核中的SYS文件也是通过双向链表的方式相连接的，我们可以通过遍历 `LDR_DATA_TABLE_ENTRY` 结构(遍历自身DriverSection成员)，就能够得到全部的模块信息。

```

#include <ntddk.h>
#include <wdm.h>

typedef struct _LDR_DATA_TABLE_ENTRY {
    LIST_ENTRY InLoadOrderLinks;
    LIST_ENTRY InMemoryOrderLinks;
    LIST_ENTRY InInitializationOrderLinks;
    PVOID DllBase;
    PVOID EntryPoint;
    ULONG SizeOfImages;
    UNICODE_STRING FullDllName;
    UNICODE_STRING BaseDllName;
    ULONG Flags;
    USHORT LoadCount;
    USHORT TlsIndex;
    union {
        LIST_ENTRY HashLinks;
        struct {
            PVOID SectionPointer;
            ULONG CheckSum;
        };
    };
    union {
        struct {
            ULONG TimeDateStamp;
        };
        struct {
            PVOID LoadedImports;
        };
    };
};

LDR_DATA_TABLE_ENTRY, *PLDR_DATA_TABLE_ENTRY;

VOID DriverUnload(IN PDRIVER_OBJECT DriverObject){}

```



```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, PUNICODE_STRING
RegistryPath)
{
    ULONG count = 0;
    NTSTATUS Status;
    DriverObject->DriverUnload = DriverUnload;

    PLDR_DATA_TABLE_ENTRY pLdr = NULL;
    PLIST_ENTRY pListEntry = NULL;
    PLDR_DATA_TABLE_ENTRY pModule = NULL;
    PLIST_ENTRY pCurrentListEntry = NULL;

    pLdr = (PLDR_DATA_TABLE_ENTRY)DriverObject->DriverSection;
    pListEntry = pLdr->InLoadOrderLinks.Flink;
    pCurrentListEntry = pListEntry->Flink;

    while (pCurrentListEntry != pListEntry)
    {
        pModule = CONTAINING_RECORD(pCurrentListEntry, LDR_DATA_TABLE_ENTRY,
InLoadOrderLinks);
        if (pModule->BaseDllName.Buffer != 0)
        {
            DbgPrint("基址: %p ---> 偏移: %p ---> 结束地址: %p---> 模块名: %wZ \r\n",
pModule->DllBase, pModule->SizeOfImages - (LONGLONG)pModule->DllBase,
(LONGLONG)pModule->DllBase + pModule->SizeOfImages, pModule-
>BaseDllName);
        }
        pCurrentListEntry = pCurrentListEntry->Flink;
    }
    DriverObject->DriverUnload = DriverUnload;
    return STATUS_SUCCESS;
}

```

输出效果如下所示:

DebugView on \\DESKTOP-CCFGJFO (local)

#	Time	Debug Print
110	0.00032100	基址: FFFFF802C6F60000 ---> 偏移: 000007FD39116000 ---> 结束地址: FFFFF802C6F60000 ---> 模块名: ks.sys
111	0.00032400	基址: FFFFF802C6F60000 ---> 偏移: 000007FD3901F000 ---> 结束地址: FFFFF802C6F60000 ---> 模块名: dump_storport.sys
112	0.00032710	基址: FFFFF802C7020000 ---> 偏移: 000007FD38FFF000 ---> 结束地址: FFFFF802C703F000 ---> 模块名: dump_LSI_SAS.sys
113	0.00033020	基址: FFFFF802C7060000 ---> 偏移: 000007FD38FBD000 ---> 结束地址: FFFFF802C707D000 ---> 模块名: dump_dumpfive.sys
114	0.00033320	基址: FFFFF802C766B0000 ---> 偏移: 00003AD8994DB000 ---> 结束地址: FFFFF802C766C3B000 ---> 模块名: win32k.sys
115	0.00033650	基址: FFFFF802C766B0000 ---> 偏移: 00003AD899B92000 ---> 结束地址: FFFFF802C766B92000 ---> 模块名: win32kfull.sys
116	0.00033950	基址: FFFFF802C7420000 ---> 偏移: 000007FD38BF3000 ---> 结束地址: FFFFF802C7433000 ---> 模块名: HIDPARSE.SYS
117	0.00034270	基址: FFFFF802C7670F0000 ---> 偏移: 00003AD89917D000 ---> 结束地址: FFFFF802C76735D000 ---> 模块名: win32kbase.sys
118	0.00034560	基址: FFFFF802C76B0000 ---> 偏移: 000007FD389C0000 ---> 结束地址: FFFFF802C7720000 ---> 模块名: dxgmmnl.sys
119	0.00034860	基址: FFFFF802C7730000 ---> 偏移: 000007FD388E6000 ---> 结束地址: FFFFF802C7746000 ---> 模块名: monitor.sys

6.2 监控进程与线程对象操作

监控进程对象和线程对象操作, 可以使用 `ObRegisterCallbacks` 这个内核回调函数, 通过回调我们可以实现保护 `calc.exe` 进程不被关闭, 具体操作从 `operationInformation->Object` 获得进程或线程的对象, 然后再回调中判断是否是计算器, 如果是就直接去掉 `TERMINATE_PROCESS` 或 `TERMINATE_THREAD` 权限即可。

监控进程对象

附上进程监控回调的写法:

```

#include <ntddk.h>
#include <ntstrsafe.h>

PVOID GLOBLE_Object_Handle;

OB_PREOP_CALLBACK_STATUS MyObjectCallback(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION OperationInformation)
{
    DbgPrint("执行了我们的回调函数...");
    return STATUS_SUCCESS;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    ObUnregisterCallbacks(GLOBLE_Object_Handle);
    DbgPrint("回调卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    OB_OPERATION_REGISTRATION Base;
    OB_CALLBACK_REGISTRATION CallbackReg;

    CallbackReg.RegistrationContext = NULL;

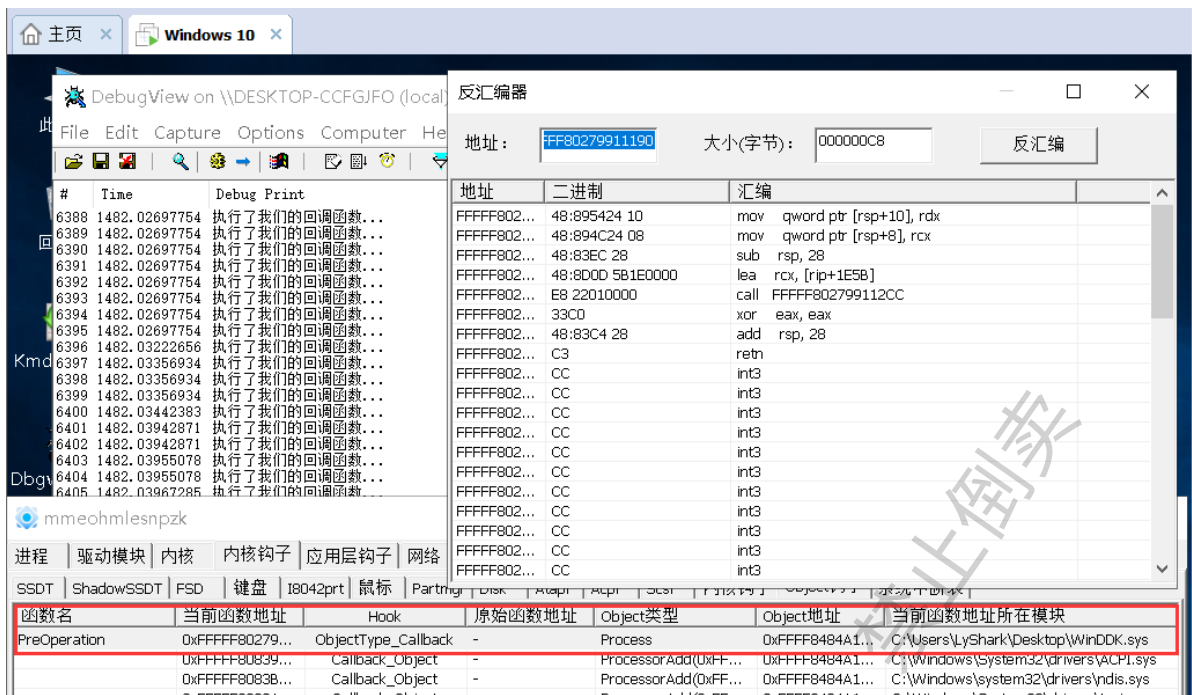
    CallbackReg.Version = OB_FLT_REGISTRATION_VERSION;
    CallbackReg.OperationRegistration = &Base;
    CallbackReg.OperationRegistrationCount = 1;
    RtlUnicodeStringInit(&CallbackReg.Altitude, L"600000");
    Base.ObjectType = PsProcessType;

    Base.Operations = OB_OPERATION_HANDLE_CREATE;
    Base.PreOperation = MyObjectCallback;
    Base.PostOperation = NULL;

    if (ObRegisterCallbacks(&CallbackReg, &GLOBLE_Object_Handle)) // 注册回调
        DbgPrint("回调注册成功...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

上方代码运行后，我们可以打开Xuetr扫描一下内核Object钩子，可以看到已经成功挂钩了。



检测计算器进程的关闭状态，代码如下：

```
#include <ntddk.h>
#include <wdm.h>
#include <ntstrsafe.h>
#define PROCESS_TERMINATE 1

PVOID Gbgle_Object_Handle;
NTKERNELAPI UCHAR * PsGetProcessImageFileName(__in PEPROCESS Process);

char* GetProcessImageNameByProcessID(ULONG ulProcessID)
{
    NTSTATUS Status;
    PEPROCESS EProcess = NULL;
    Status = PsLookupProcessByProcessId((HANDLE)ulProcessID, &EProcess);
    if (!NT_SUCCESS(Status))
        return FALSE;
    ObDereferenceObject(EProcess);
    return (char*)PsGetProcessImageFileName(EProcess);
}

OB_PREOP_CALLBACK_STATUS MyobjectCallBack(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION Operation)
{
    char ProcName[256] = { 0 };
    HANDLE pid = PsGetProcessId((PEPROCESS)Operation->Object); // 取出当前调用函数的PID
    strcpy(ProcName, GetProcessImageNameByProcessID((ULONG)pid)); // 通过PID取出进程名,然后直接拷贝内存
    //DbgPrint("当前进程的名字是: %s", ProcName);

    if (strstr(ProcName, "win32calc.exe"))
    {
        if (Operation->Operation == OB_OPERATION_HANDLE_CREATE)
        {

```

```

        if ((Operation->Parameters-
>CreateHandleInformation.OriginalDesiredAccess & PROCESS_TERMINATE) ==
PROCESS_TERMINATE)
        {
            DbgPrint("你想结束进程?");
            // 如果是计算器，则去掉它的结束权限，在win10上无效
            Operation->Parameters->CreateHandleInformation.DesiredAccess =
~THREAD_TERMINATE;
            return STATUS_UNSUCCESSFUL;
        }
    }
    return STATUS_SUCCESS;
}

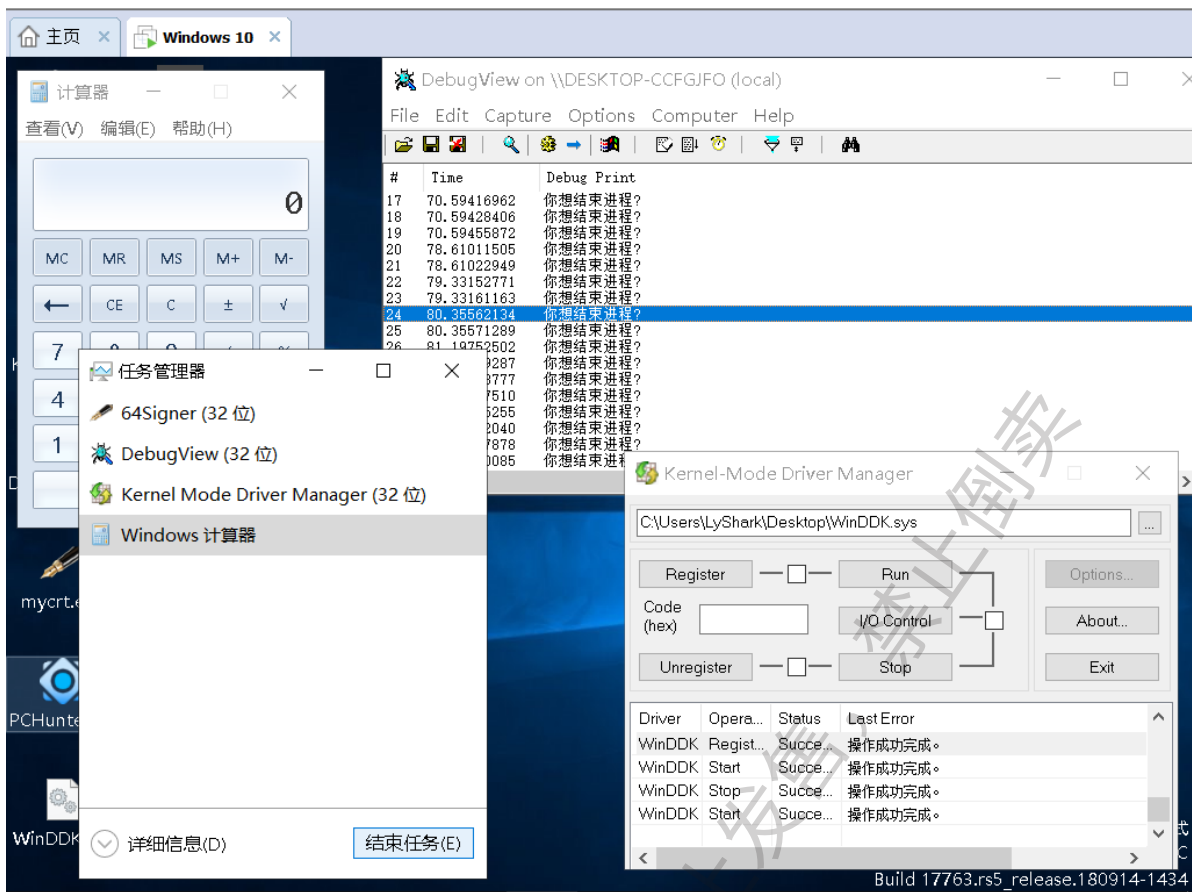
VOID UnDriver(PDRIVER_OBJECT driver)
{
    ObUnRegisterCallbacks(Globle_Object_Handle);
    DbgPrint("回调卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS obst = 0;
    OB_CALLBACK_REGISTRATION obReg;
    OB_OPERATION_REGISTRATION opReg;

    memset(&obReg, 0, sizeof(obReg));
    obReg.Version = ObGetFilterVersion();
    obReg.OperationRegistrationCount = 1;
    obReg.RegistrationContext = NULL;
    RtlInitUnicodeString(&obReg.Altitude, L"321125");
    obReg.OperationRegistration = &opReg;
    memset(&opReg, 0, sizeof(opReg));
    opReg.ObjectType = PsProcessType;
    opReg.Operations = OB_OPERATION_HANDLE_CREATE |
OB_OPERATION_HANDLE_DUPLICATE;
    opReg.PreOperation = (POB_PRE_OPERATION_CALLBACK)&MyObjectCallBack;
    obst = ObRegisterCallbacks(&obReg, &Globle_Object_Handle);
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

首先运行计算器，然后启动驱动保护，此时我们在任务管理器中就无法结束计算器进程了。



监控进程中模块加载

系统中的模块加载包括用户层模块DLL和内核模块SYS的加载，在 Windows X64 环境下我们可以调用 `PsSetLoadImageNotifyRoutine` 内核函数来设置一个映像加载通告例程，当有驱动或者DLL被加载时，回调函数就会被调用从而执行我们自己的回调例程。

```
#include <ntddk.h>
#include <ntimage.h>

PVOID GetDriverEntryByImageBase(PVOID ImageBase)
{
    PIMAGE_DOS_HEADER pDOSHeader;
    PIMAGE_NT_HEADERS64 pNTHHeader;
    PVOID pEntryPoint;
    pDOSHeader = (PIMAGE_DOS_HEADER)ImageBase;
    pNTHHeader = (PIMAGE_NT_HEADERS64)((ULONG64)ImageBase + pDOSHeader->e_lfanew);
    pEntryPoint = (PVOID)((ULONG64)ImageBase + pNTHHeader->OptionalHeader.AddressOfEntryPoint);
    return pEntryPoint;
}

VOID MyLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ProcessId, PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;
    if (FullImageName != NULL && MmIsAddressValid(FullImageName)) // MmIsAddress
    验证地址可用性
    {
        if (ProcessId == 0)
        {

```

```

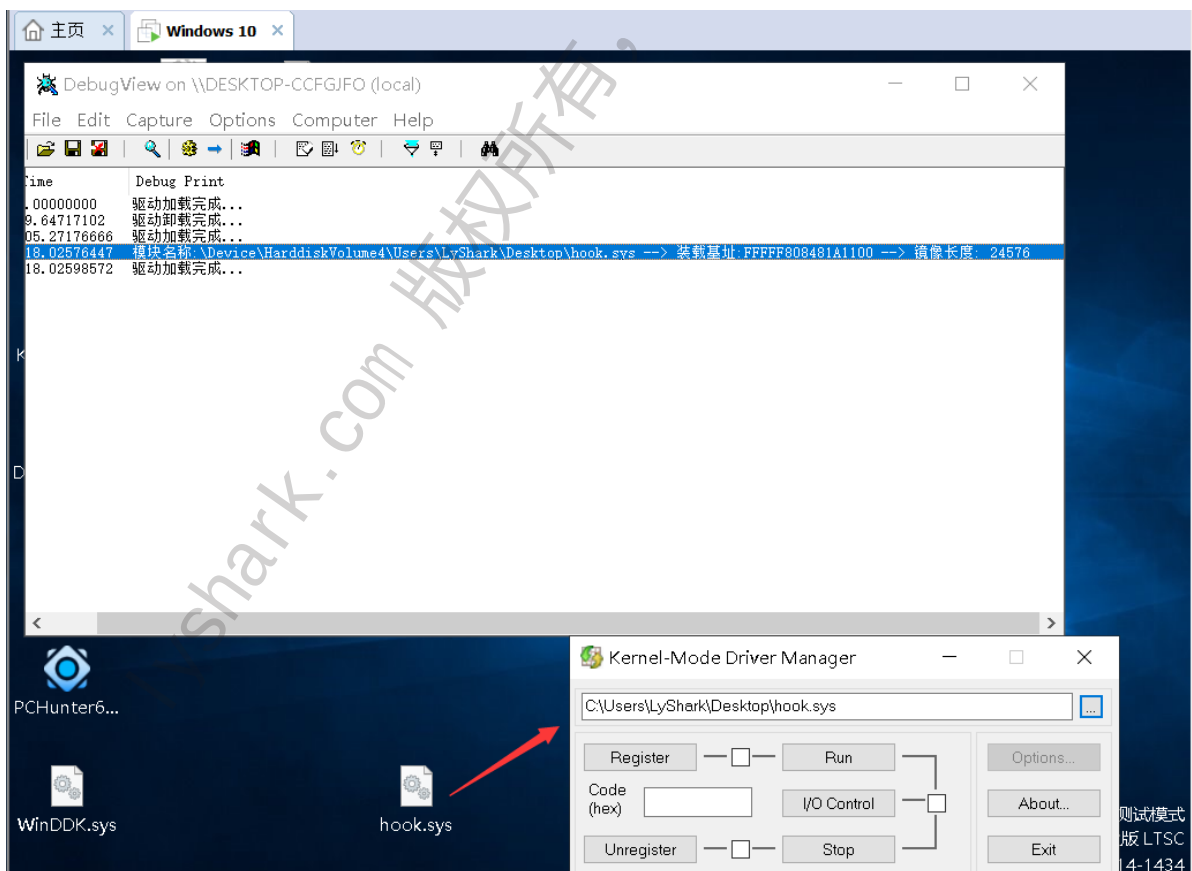
        pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
        DbgPrint("模块名称:%wZ --> 装载基址:%p --> 镜像长度: %d", FullImageName,
        pDrvEntry, ImageInfo->ImageSize);
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLoadImageNotifyRoutine);
    DbgPrint("驱动卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    PsSetLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLoadImageNotifyRoutine);
    DbgPrint("驱动加载完成...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

输出效果如下所示:



接着我们给上方的代码加上判断功能，只需在上方代码的基础上小改一下即可，需要注意回调函数中的第二个参数，如果返回值为零则表示加载SYS，如果返回非零则表示加载DLL

```

VOID UnicodeToChar(PUNICODE_STRING dst, char *src)

```

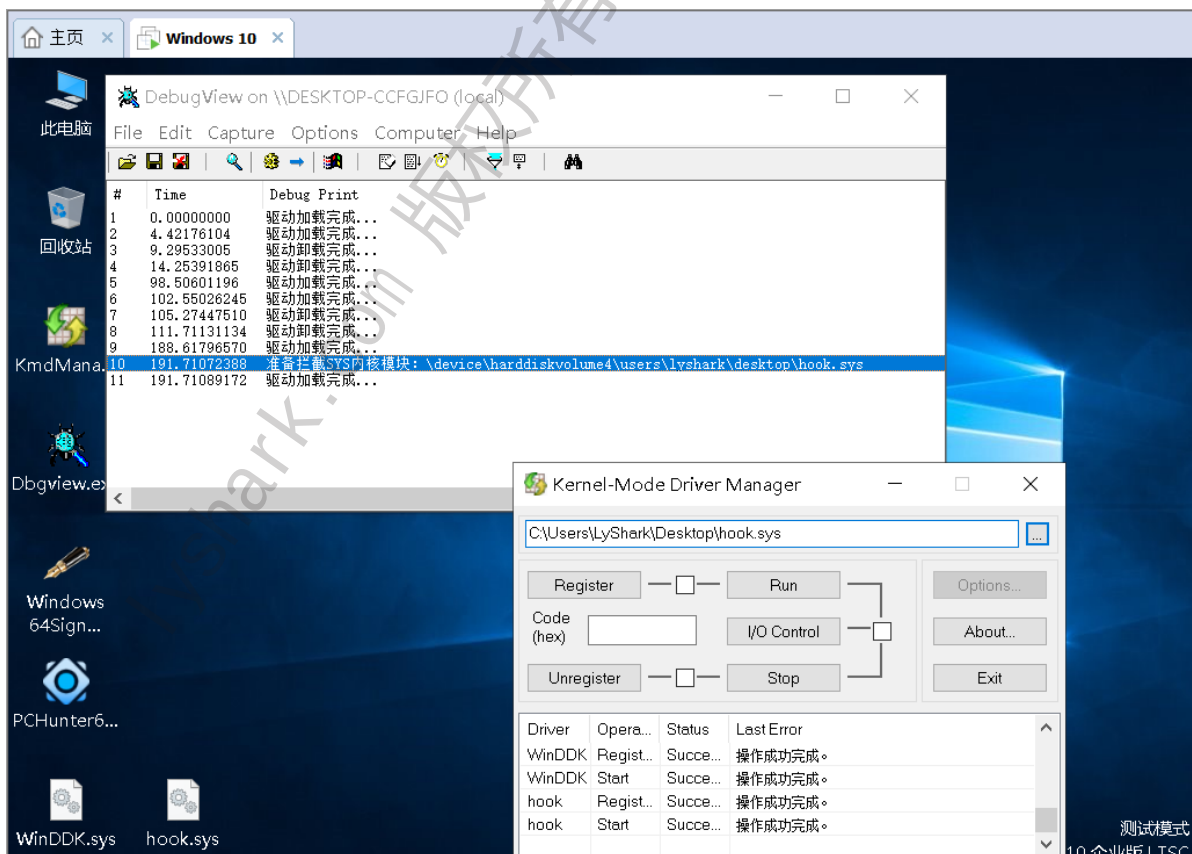
```

{
    ANSI_STRING string;
    RtlUnicodeStringToAnsiString(&string, dst, TRUE);
    strcpy(src, string.Buffer);
    RtlFreeAnsiString(&string);
}

VOID MyLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE
ModuleStyle, PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;
    char szFullImageName[256] = { 0 };
    if (FullImageName != NULL && MmIsAddressValid(FullImageName)) // MmIsAddress
验证地址可用性
    {
        if (ModuleStyle == 0) // ModuleStyle为零表示加载sys非零表示加载DLL
        {
            pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
            UnicodeToChar(FullImageName, szFullImageName);
            if (strstr(_strlwr(szFullImageName), "hook.sys"))
            {
                DbgPrint("准备拦截SYS内核模块: %s", _strlwr(szFullImageName));
            }
        }
    }
}
}

```

输出效果如下所示：



上方代码就可以判断加载的模块并作出处理动作了，但是我们仍然无法判断到底是那个进程加载的 hook.sys 驱动，因为回调函数很底层，到了一定的深度之后就无法判断到底是谁主动引发的行为了，一切都是系统的行为。

判断了是驱动后，接着我们就要实现屏蔽驱动，通过 ImageInfo->ImageBase 来获取被加载驱动程序 hook.sys 的映像基址，然后找到NT头的OptionalHeader节点，该节点里面就是被加载驱动入口的地址，通过汇编在驱动头部写入ret返回指令，即可实现屏蔽加载特定驱动文件。

```
#include <ntddk.h>
#include <intrin.h>
#include <ntimage.h>

PVOID GetDriverEntryByImageBase(PVOID ImageBase)
{
    PIMAGE_DOS_HEADER pDOSHeader;
    PIMAGE_NT_HEADERS64 pNTHHeader;
    PVOID pEntryPoint;
    pDOSHeader = (PIMAGE_DOS_HEADER)ImageBase;
    pNTHHeader = (PIMAGE_NT_HEADERS64)((ULONG64)ImageBase + pDOSHeader->e_lfanew);
    pEntryPoint = (PVOID)((ULONG64)ImageBase + pNTHHeader->OptionalHeader.AddressOfEntryPoint);
    return pEntryPoint;
}

VOID UnicodeToChar(PUNICODE_STRING dst, char *src)
{
    ANSI_STRING string;
    RtlUnicodeStringToAnsiString(&string, dst, TRUE);
    strcpy(src, string.Buffer);
    RtlFreeAnsiString(&string);
}

// 使用开关写保护需要在 C/C++ 优化中启用内部函数
KIRQL WPOFFx64() // 关闭写保护
{
    KIRQL irq1 = KeRaiseIrqlToDpcLevel();
    UINT64 cr0 = __readcr0();
    cr0 &= 0xffffffffffffe000;
    _disable();
    __writecr0(cr0);
    return irq1;
}

void WPONx64(KIRQL irq1) // 开启写保护
{
    UINT64 cr0 = __readcr0();
    cr0 |= 0x10000;
    _enable();
    __writecr0(cr0);
    KeLowerIrql(irq1);
}

BOOLEAN DenyLoadDriver(PVOID DriverEntry)
{
    UCHAR fuck[] = "\xB8\x22\x00\x00\xC0\xC3";
    KIRQL kirql;
    /* 在模块开头写入以下汇编指令
    Mov eax,c0000022h
    ret
    */
    if (DriverEntry == NULL) return FALSE;
```



```

        kirql = WPOFFx64();
        memcpy(DriverEntry, fuck, sizeof(fuck) / sizeof(fuck[0]));
        WPONx64(kirql);
        return TRUE;
    }

VOID MyLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ModuleStyle,
PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;
    char szFullImageName[256] = { 0 };
    if (FullImageName != NULL && MmIsAddressValid(FullImageName)) // MmIsAddress
验证地址可用性
    {
        if (ModuleStyle == 0) // ModuleStyle为零表示加载sys非零表示加载DLL
        {
            pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
            UnicodeToChar(FullImageName, szFullImageName);
            if (strstr(_strlwr(szFullImageName), "hook.sys"))
            {
                DbgPrint("拦截SYS内核模块: %s", szFullImageName);
                DenyLoadDriver(pDrvEntry);
            }
        }
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLoadImageNotifyRoutine);
    DbgPrint("驱动卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    PsSetLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLoadImageNotifyRoutine);
    DbgPrint("驱动加载完成...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

屏蔽DLL加载，只需要在上面的代码上稍微修改一下就好，这里提供到另一种写法。

```

char *UnicodeToLongString(PUNICODE_STRING uString)
{
    ANSI_STRING asStr;
    char *Buffer = NULL;;
    RtlUnicodeStringToAnsiString(&asStr, uString, TRUE);
    Buffer = ExAllocatePoolWithTag(NonPagedPool, uString->MaximumLength *
sizeof(wchar_t), 0);
    if (Buffer == NULL)
        return NULL;
    RtlCopyMemory(Buffer, asStr.Buffer, asStr.Length);
}

```

```

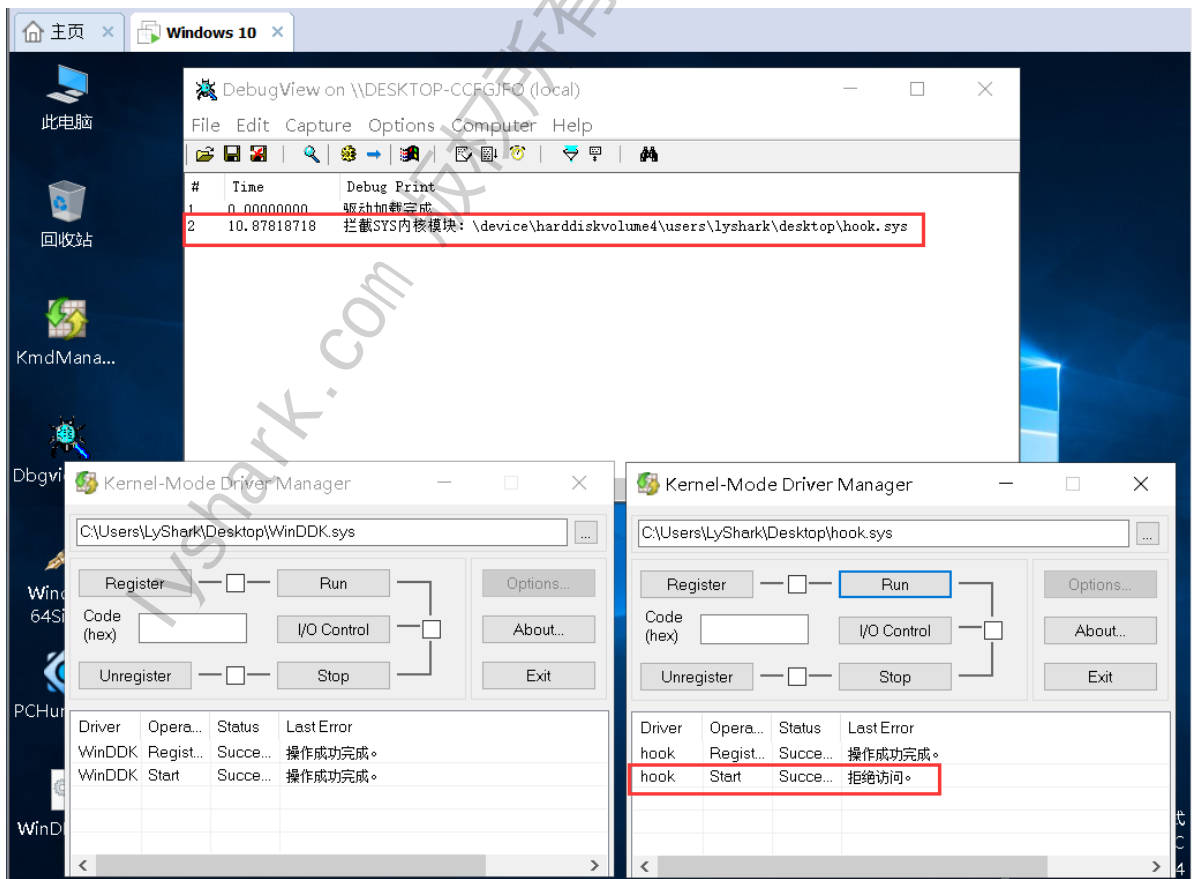
return Buffer;
}

VOID MyLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ModuleStyle,
PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;
    char *PareString = NULL;

    if (MmIsAddressValid(FullImageName))
    {
        if (ModuleStyle != 0) // 非零则监控DLL加载
        {
            PareString = UnicodeToLongString(FullImageName);
            if (PareString != NULL)
            {
                if (strstr(PareString, "hook.dll"))
                {
                    pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
                    if (pDrvEntry != NULL)
                        DenyLoadDriver(pDrvEntry);
                }
            }
        }
    }
}

```

我们以屏蔽SYS内核模块为例，当驱动文件 winDDK.sys 被加载后，尝试加载 hook.sys 会提示拒绝访问，说明我们的驱动保护生效了。



关键的内核进程操作已经分享完了，杀软的主动防御系统，游戏的保护系统等都会用到这些东西。

进程	驱动模块	内核	内核钩子	应用层钩子	网络	注册表	文件	启动信息	系统杂项	电脑体检	配置	关于	
SSDT	ShadowSSDT	FSD	键盘	I8042prt	鼠标	Partmgr	Disk	Atapi	Acpi	Scsi	内核钩子	Object钩子	系统中断表
函数名	当前函数地址	Hook	Object类型	Object地址	当前函数地址所在模块								
DeleteProcedure	0xFFFFF807CB...	-	TmRm	0xFFFFF838ECE...	C:\Windows\System32\drivers\tm.sys								
SeDefaultObjectMeth...	0xFFFFF80574...	-	TmRm	0xFFFFF838ECE...	C:\Windows\system32\ntoskrnl.exe								
PreOperation	0xFFFFF80589...	ObjectType_Ca...	Process	0xFFFFF838ECE...	C:\Windows\system32\drivers\QQProtectX64.sys								
PostOperation	0xFFFFF80589...	ObjectType_Ca...	Process	0xFFFFF838ECE...	C:\Program Files (x86)\Tencent\QQPCMgr\13.3.20238.213\QQS								
PreOperation	0xFFFFF80589...	ObjectType_Ca...	Process	0xFFFFF838ECE...	C:\Program Files (x86)\Tencent\QQPCMgr\13.3.20238.213\QQS								
PreOperation	0xFFFFF80589...	ObjectType_Ca...	Thread	0xFFFFF838ECE...	C:\Windows\system32\drivers\QQProtectX64.sys								
PostOperation	0xFFFFF80589...	ObjectType_Ca...	Thread	0xFFFFF838ECE...	C:\Program Files (x86)\Tencent\QQPCMgr\13.3.20238.213\QQS								
PreOperation	0xFFFFF80589...	ObjectType_Ca...	Thread	0xFFFFF838ECE...	C:\Program Files (x86)\Tencent\QQPCMgr\13.3.20238.213\QQS								
GetCellRoutine	0xFFFFF80574...	-	HHIVE	0xFFFFF9F8ACF...	C:\Windows\system32\ntoskrnl.exe								
ReleaseCellRoutine	0xFFFFF80574...	-	HHIVE	0xFFFFF9F8ACF...	C:\Windows\system32\ntoskrnl.exe								

6.3 内核监控进程与线程创建

监控进程的启动与退出可以使用 `PsSetCreateProcessNotifyRoutineEx` 来创建回调，当新进程产生时，回调函数会被率先执行，然后执行我们自己的 `MyCreateProcessNotifyEx` 函数，并在内部进行打印输出。

```
#include <ntddk.h>

NTKERNELAPI PCHAR PsGetProcessImageFileName(PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);

PCHAR GetProcessNameByProcessId(HANDLE ProcessId)
{
    NTSTATUS st = STATUS_UNSUCCESSFUL;
    PEPROCESS ProcessObj = NULL;
    PCHAR string = NULL;
    st = PsLookupProcessByProcessId(ProcessId, &ProcessObj);
    if (NT_SUCCESS(st))
    {
        string = PsGetProcessImageFileName(ProcessObj);
        ObfDereferenceObject(ProcessObj);
    }
    return string;
}

VOID MyCreateProcessNotifyEx(PEPROCESS Process, HANDLE ProcessId,
PPS_CREATE_NOTIFY_INFO CreateInfo)
{
    char ProcName[16] = { 0 };
    if (CreateInfo != NULL)
    {
        strcpy(ProcName, PsGetProcessImageFileName(Process));
        DbgPrint("父进程ID: %ld ---->父进程名: %s ---->进程名: %s---->进程路径: %wZ",
CreateInfo->ParentProcessId,
        GetProcessNameByProcessId(CreateInfo->ParentProcessId),
        PsGetProcessImageFileName(Process), CreateInfo->ImageFileName);
    }
    else
    {
        strcpy(ProcName, PsGetProcessImageFileName(Process));
        DbgPrint("进程[ %s ] 离开了，程序被关闭了", ProcName);
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
```

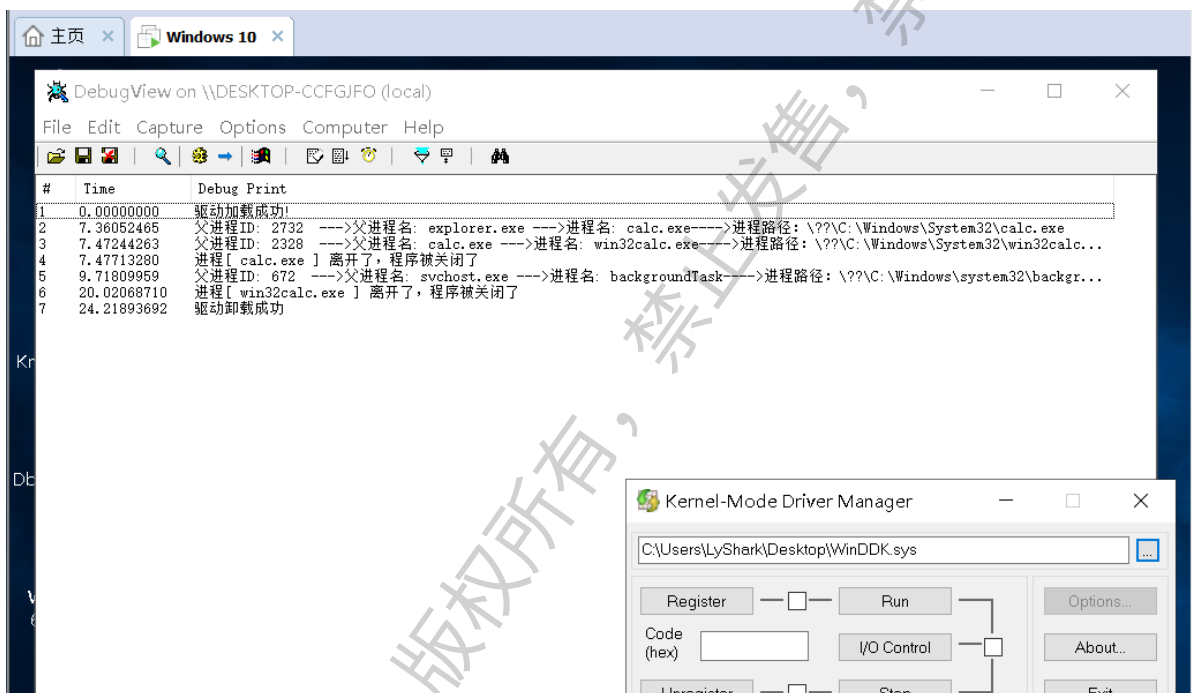
```

{

PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)MyCreateProcessNotifyEx, TRUE);
}
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;
    status =
PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)MyCreateProcessNotifyEx, FALSE);
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

输出效果如下所示：



在上方代码基础上进行一定的改进，思路：通过 `PsGetProcessImageFileName` 即将PID转换为进程名，然后通过 `_stricmp` 对比，如果发现是 `calc.exe` 进程则拒绝执行，禁止特定服务的运行，实现代码如下：

```

#include <ntddk.h>

NTKERNELAPI PCHAR PsGetProcessImageFileName(PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS *Process);

PCHAR GetProcessNameByProcessId(HANDLE ProcessId)
{
    NTSTATUS st = STATUS_UNSUCCESSFUL;
    PEPROCESS ProcessObj = NULL;
    PCHAR string = NULL;
    st = PsLookupProcessByProcessId(ProcessId, &ProcessObj);
    if (NT_SUCCESS(st))
    {
        string = PsGetProcessImageFileName(ProcessObj);
    }
}

```

```

        ObfDereferenceObject(ProcessObj);
    }
    return string;
}

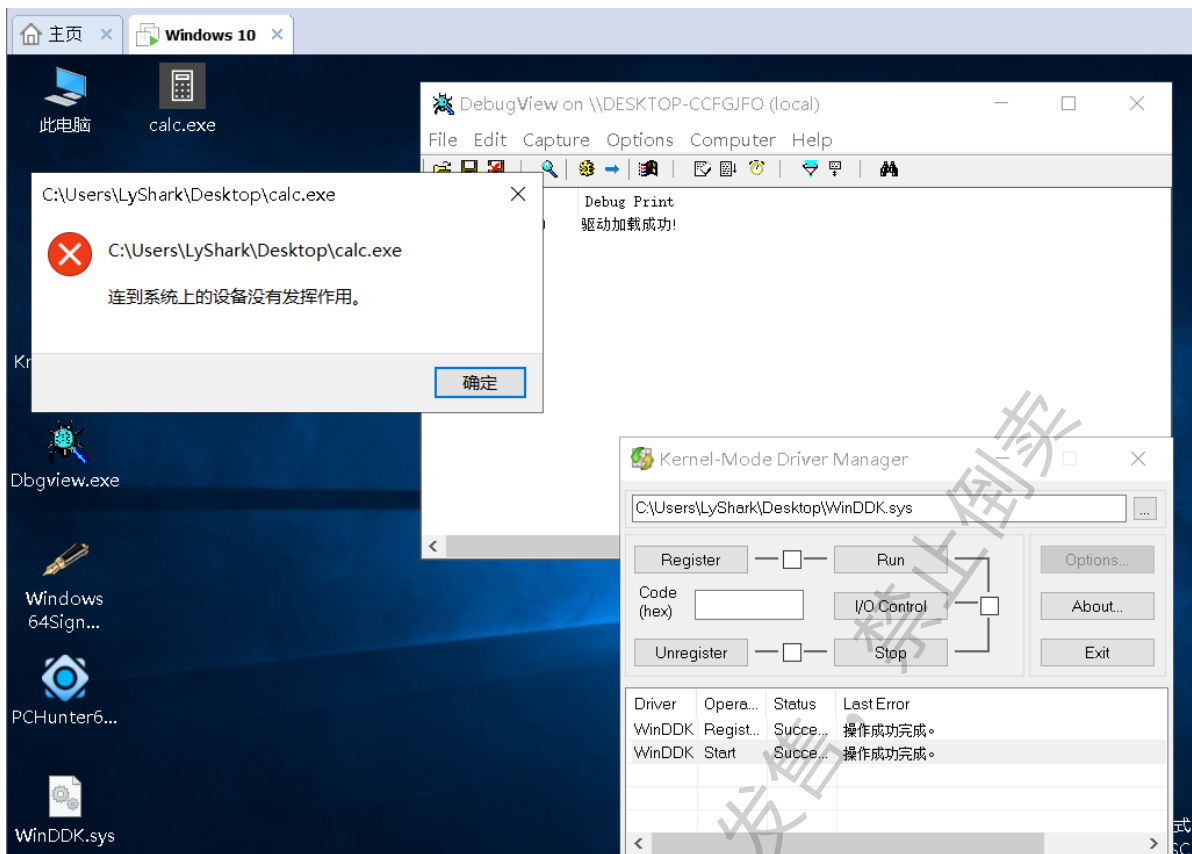
VOID MyCreateProcessNotifyEx(PEPROCESS Process, HANDLE ProcessId,
PPS_CREATE_NOTIFY_INFO CreateInfo)
{
    char ProcName[16] = { 0 };
    if (CreateInfo != NULL)
    {
        strcpy(ProcName, PsGetProcessImageFileName(Process));
        if (!_stricmp(ProcName, "calc.exe"))
        {
            CreateInfo->CreationStatus = STATUS_UNSUCCESSFUL;
        }
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)MyCreateProcessNotifyEx, TRUE);
    DbgPrint(("驱动卸载成功"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;
    status =
    PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)MyCreateProcessNotifyEx, FALSE);
    Driver->DriverUnload = UnDriver;
    DbgPrint(("驱动加载成功!"));
    return STATUS_SUCCESS;
}

```

将上方代码编译，当我们加载驱动程序以后，再次打开 C:\windows\System32\calc.exe 计算器进程则提示无法打开，我们的驱动已经成功的拦截了本次的请求。



而检测线程操作与检测进程差不多，检测线程需要调用 `PsSetCreateThreadNotifyRoutine` 创建回调函数，然后就可以检测线程的创建了，具体代码如下：

```
#include <ntddk.h>

NTKERNELAPI PCHAR PsGetProcessImageFileName(PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);

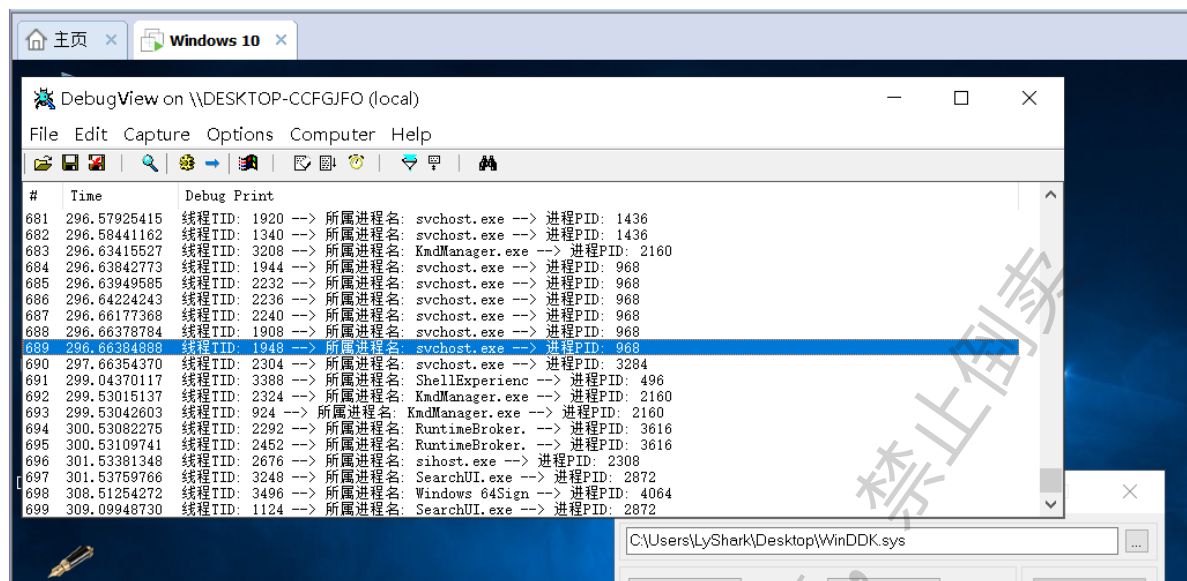
VOID MyCreateThreadNotify(HANDLE ProcessId, HANDLE ThreadId, BOOLEAN Create)
{
    PEPROCESS eprocess = NULL;
    PsLookupProcessByProcessId(ProcessId, &eprocess); // 通过此函数
    拿到程序的EPROCESS结构
    if (Create)
        DbgPrint("线程TID: %ld --> 所属进程名: %s --> 进程PID: %ld \n", ThreadId,
        PsGetProcessImageFileName(eprocess), PsGetProcessId(eprocess));
    else
        DbgPrint("%s 线程已退出...", ThreadId);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveCreateThreadNotifyRoutine(MyCreateThreadNotify);
    DbgPrint(("驱动卸载成功"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;
    status = PsSetCreateThreadNotifyRoutine(MyCreateThreadNotify);
    DbgPrint("PsSetCreateThreadNotifyRoutine: %x", status);
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}
```

```
}
```

输出效果如下所示：



6.4 内核DKOM实现进程隐藏

DKOM 就是直接内核对象操作技术，我们所有的操作都会被系统记录在内存中，而驱动进程隐藏的做旧就是操作进程的EPROCESS结构与线程的ETHREAD结构、链表，要实现进程的隐藏我们只需要将某个进程中的信息，在系统EPROCESS链表中摘除即可实现进程隐藏。

DKOM 隐藏进程的本质是操作EPROCESS结构体，EPROCESS结构体中包含了系统中的所有进程相关信息，还有很多指向其他结构的指针，首先我们可以通过WinDBG在内核调试模式下输入 `dt_eprocess` 即可查看到当前的EPROCESS结构体的偏移信息，结构较多，但常用的就下面这几个。

```
kd> dt_eprocess
```

```
nt!_EPROCESS
```

```
+0x000 Pcb : _KPROCESS
+0x160 ProcessLock : _EX_PUSH_LOCK
+0x168 CreateTime : _LARGE_INTEGER // 创建时间
+0x170 ExitTime : _LARGE_INTEGER // 退出时间
+0x180 UniqueProcessId : Ptr64 Void // 进程的PID
+0x188 ActiveProcessLinks : _LIST_ENTRY // 活动进程链表
+0x200 ObjectTable : Ptr64 _HANDLE_TABLE // 指向句柄表的指针
+0x2d8 Session : Ptr64 Void // 会话列表
+0x2e0 ImageFileName : [15] Uchar // 进程的名称
+0x308 ThreadListHead : _LIST_ENTRY // 进程中的线程链表结构
```

```
构
```

```
+0x320 Wow64Process : Ptr64 Void // 32位进程链表
+0x328 ActiveThreads : Uint4B // 活动的线程
+0x32c ImagePathHash : Uint4B // 镜像路径的
```

```
Hash值
```

```
+0x338 Peb : Ptr64 _PEB // 指向
```

```
PEB结构的指针
```

```
+0x440 Flags : Uint4B // 进程
```

```
标志
```


要实现进程的隐藏我们需要关注结构中的 `ActiveProcessLinks` 该指针把每个进程的EPROCESS结构体连接成了双向链表，我们可以使用 `ZwQuerySystemInformation` 这个函数来遍历出所有的进程信息，要实现进程的隐藏，只需要将某个进程的EPROCESS从结构体中摘除，那么通过 `ZwQuerySystemInformation` 函数就无法遍历出被摘链的进程了，从而实现了进程的隐藏。

在实现进程隐藏之前，我们需要通过代码的方式获取到当前系统中所有进程的EPROCESS信息，我们可以通过 `PsLookupProcessByProcessId` 函数获取到指定进程的ID，然后通过 `PsGetProcessImageFileName` 函数取出结构名称，并通过 `_stricmp` 判断是否是我们想要隐藏的程

```
#include <ntifs.h>
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

PEPROCESS GetProcessObjectByName(char *name)
{
    SIZE_T temp;
    for (temp = 100; temp < 10000; temp += 4)
    {
        NTSTATUS status;
        PEPROCESS ep;
        status = PsLookupProcessByProcessId((HANDLE)temp, &ep);
        if (NT_SUCCESS(status))
        {
            char *pn = PsGetProcessImageFileName(ep);
            if (_stricmp(pn, name) == 0)
                return ep;
        }
    }
    return NULL;
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    PEPROCESS PROC = NULL;
    PROC = GetProcessObjectByName("calc.exe");
    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}
```

然后得到句柄以后直接摘除进程的结构即可实现隐藏，这种摘除方式比较草率，如果关闭驱动后没有手工还原的话可能会导致蓝屏，该方法只用于在Win7上使用,Win10没试过。

```
#include <ntifs.h>
#define PROCESS_ACTIVE_PROCESS_LINKS_OFFSET 0x188

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
```



```

NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

PEPROCESS GetProcessObjectByName(char *name)
{
    SIZE_T temp;
    for (temp = 100; temp < 10000; temp += 4)
    {
        NTSTATUS status;
        PEPROCESS ep;
        status = PsLookupProcessByProcessId((HANDLE)temp, &ep);
        if (NT_SUCCESS(status))
        {
            char *pn = PsGetProcessImageFileName(ep);
            if (_stricmp(pn, name) == 0)
                return ep;
        }
    }
    return NULL;
}

VOID RemoveListEntry(PLIST_ENTRY ListEntry)
{
    KIRQL OldIrql;
    OldIrql = KeRaiseIrqlToDpcLevel();
    if (ListEntry->Flink != ListEntry && ListEntry->Blink != ListEntry
        && ListEntry->Blink->Flink == ListEntry && ListEntry->Flink->Blink == ListEntry)
    {
        ListEntry->Flink->Blink = ListEntry->Blink;
        ListEntry->Blink->Flink = ListEntry->Flink;
        ListEntry->Flink = ListEntry;
        ListEntry->Blink = ListEntry;
    }
    KeLowerIrql(OldIrql);
}

// 隐藏指定进程(会蓝屏)
BOOLEAN HideProcessB(PUCHAR pszHideProcessName)
{
    PEPROCESS pFirstEProcess = NULL, pEProcess = NULL;
    ULONG ulOffset = 0;
    HANDLE hProcessId = NULL;
    PUCHAR pszProcessName = NULL;

    // 获取相应偏移大小
    ulOffset = PROCESS_ACTIVE_PROCESS_LINKS_OFFSET;

    if (0 == ulOffset)
    {
        return FALSE;
    }
}

```

```

// 获取当前进程结构对象
pFirstEProcess = PsGetCurrentProcess();
pEProcess = pFirstEProcess;

// 开始遍历枚举进程
do
{
    // 从 EPROCESS 获取进程 PID
    hProcessId = PsGetProcessId(pEProcess);

    // 从 EPROCESS 获取进程名称
    pszProcessName = PsGetProcessImageFileName(pEProcess);

    // 隐藏指定进程
    if (0 == _stricmp(pszProcessName, pszHideProcessName))
    {
        // 摘链
        RemoveEntryList((PLIST_ENTRY)((PUCHAR)pEProcess + ulOffset));
        break;
    }

    // 根据偏移计算下一个进程的 EPROCESS
    pEProcess = (PEPROCESS)((PUCHAR)((PLIST_ENTRY)((PUCHAR)pEProcess + ulOffset))->Flink) - ulOffset);
} while (pFirstEProcess != pEProcess);
return TRUE;
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    PEPROCESS PRoc = NULL;
    PRoc = GetProcessObjectByName("calc.exe");

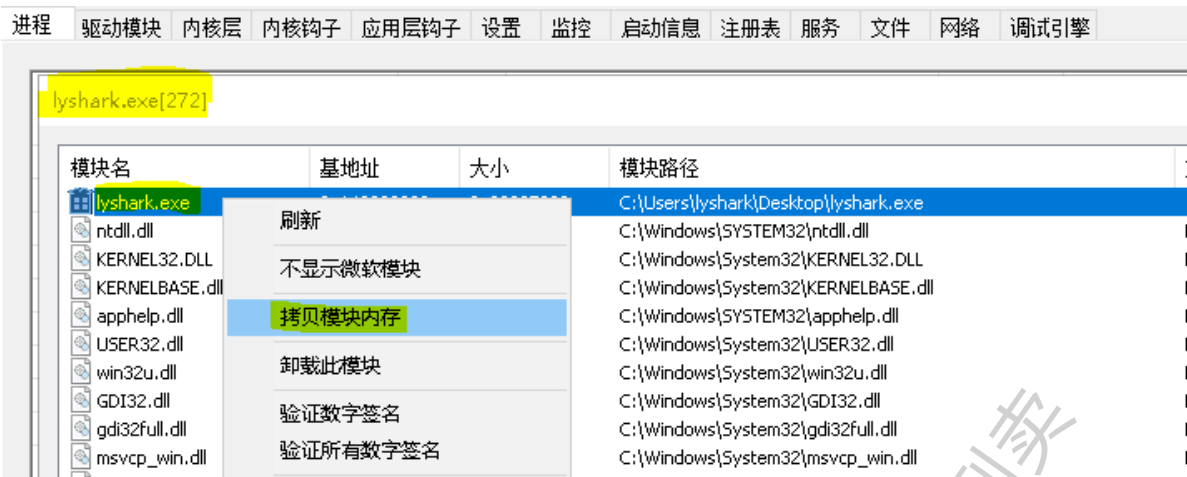
    // 摘除结构中的calc.exe 实现驱动隐藏计算器
    RemoveListEntry((PLIST_ENTRY)((ULONG64)PRoc + PROCESS_ACTIVE_PROCESS_LINKS_OFFSET));
    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

没什么难度，就是摘链，而这种断链隐藏，不仅操作不好会蓝屏且很容易被发现，只是能在任务管理器看不到而已。

6.5 内核中实现Dump进程转储

多数ARK反内核工具中都存在驱动级别的内存转存功能，该功能可以将应用层中运行进程的内存镜像转存到特定目录下，内存转存功能在应对加壳程序的分析尤为重要，当进程在内存中解码后，我们可以很容易的将内存镜像导出，从而更好的对样本进行分析，当然某些加密壳可能无效但绝大多数情况下是可以被转存的。



在上一篇文章《驱动开发：内核R3与R0内存映射拷贝》介绍了一种方式 `SafeCopyMemory_R3_to_R0` 可以将应用层进程的内存空间映射到内核中，要实现内存转储功能我们还是需要使用这个映射函数，只是需要在此函数上增加一些功能而已。

在实现转存之前，需要得到两个东西，进程内 模块基地址 以及 模块长度 这两个参数是必不可少的，至于内核中如何得到指定进程的模块数据，在很早之前的文章《驱动开发：内核中枚举进程线程与模块》中有详细的参考方法，这里就在此基础上实现一个简单的进程模块遍历功能。

如下代码中使用的就是 枚举 进程 PEB 结构得到更多参数的具体实现，如果不懂得可以研读《驱动开发：内核通过PEB得到进程参数》这篇文章此处不再赘述。

```
#include <ntddk.h>
#include <windef.h>

// 声明结构体
typedef struct _KAPC_STATE
{
    LIST_ENTRY ApcListHead[2];
    PKPROCESS Process;
    UCHAR KernelApcInProgress;
    UCHAR KernelApcPending;
    UCHAR UserApcPending;
} KAPC_STATE, *PKAPC_STATE;

typedef struct _LDR_DATA_TABLE_ENTRY
{
    LIST_ENTRY64 InLoadOrderLinks;
    LIST_ENTRY64 InMemoryOrderLinks;
    LIST_ENTRY64 InInitializationOrderLinks;
    PVOID DllBase;
    PVOID EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING FullDllName;
    UNICODE_STRING BaseDllName;
    ULONG Flags;
    USHORT LoadCount;
    USHORT TlsIndex;
    PVOID SectionPointer;
    ULONG CheckSum;
    PVOID LoadedImports;
    PVOID EntryPointActivationContext;
    PVOID PatchInformation;
}
```

```

LIST_ENTRY64 ForwarderLinks;
LIST_ENTRY64 ServiceTagLinks;
LIST_ENTRY64 StaticLinks;
PVOID ContextInformation;
ULONG64 OriginalBase;
LARGE_INTEGER LoadTime;
} LDR_DATA_TABLE_ENTRY, *PLDR_DATA_TABLE_ENTRY;

// 偏移地址
ULONG64 LdrInPebOffset = 0x018; //peb.ldr
ULONG64 ModListInPebOffset = 0x010; //peb.ldr.InLoadOrderModuleList

// 声明API
NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process);
NTKERNELAPI PPEB PsGetProcessPeb(PEPROCESS Process);
NTKERNELAPI HANDLE PsGetProcessInheritedFromUniqueProcessId(IN PEPROCESS Process);

// 根据进程ID返回进程EPROCESS，失败返回NULL
PEPROCESS LookupProcess(HANDLE Pid)
{
    PEPROCESS eprocess = NULL;
    if (NT_SUCCESS(PsLookupProcessByProcessId(Pid, &eprocess)))
        return eprocess;
    else
        return NULL;
}

// 枚举指定进程的模块
// By: LyShark.com
VOID EnumModule(PEPROCESS Process)
{
    SIZE_T Peb = 0;
    SIZE_T Ldr = 0;
    PLIST_ENTRY ModListHead = 0;
    PLIST_ENTRY Module = 0;
    ANSI_STRING AnsiString;
    KAPC_STATE ks;

    // EPROCESS地址无效则退出
    if (!MmIsAddressValid(Process))
        return;

    // 获取PEB地址
    Peb = (SIZE_T)PsGetProcessPeb(Process);

    // PEB地址无效则退出
    if (!Peb)
        return;

    // 依附进程
    KeStackAttachProcess(Process, &ks);
    __try
    {
        // 获得LDR地址

```

```

Ldr = Peb + (SIZE_T)LdrInPebOffset;
// 测试是否可读，不可读则抛出异常退出
ProbeForRead((CONST PVOID)Ldr, 8, 8);
// 获得链表头
ModListHead = (PLIST_ENTRY)((PULONG64)Ldr + ModListInPebOffset);
// 再次测试可读性
ProbeForRead((CONST PVOID)ModListHead, 8, 8);
// 获得第一个模块的信息
Module = ModListHead->Flink;

while (ModListHead != Module)
{
    //打印信息：基址、大小、DLL路径
    DbgPrint("模块基址 = %p | 大小 = %ld | 模块名 = %wz | 完整路径= %wz \n",
        (PVOID)((PLDR_DATA_TABLE_ENTRY)Module->DllBase),
        (ULONG)((PLDR_DATA_TABLE_ENTRY)Module->SizeOfImage),
        &((PLDR_DATA_TABLE_ENTRY)Module->BaseDllName),
        &((PLDR_DATA_TABLE_ENTRY)Module->FullDllName)
    );
    Module = Module->Flink;

    // 测试下一个模块信息的可读性
    ProbeForRead((CONST PVOID)Module, 80, 8);
}
}
__except (EXCEPTION_EXECUTE_HANDLER){ ; }

// 取消依附进程
KeUnstackDetachProcess(&ks);
}

VOID DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING
RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONG i = 0;
    PEPROCESS eproc = NULL;
    for (i = 4; i < 100000000; i = i + 4)
    {
        eproc = LookupProcess((HANDLE)i);
        if (eproc != NULL)
        {
            ObDereferenceObject(eproc);
            if (strstr(PsGetProcessImageFileName(eproc), "lyshark.exe") != NULL)
            {
                EnumModule(eproc);
            }
        }
    }
}

```

```

DriverObject->DriverUnload = DriverUnload;
return STATUS_SUCCESS;
}

```

如上我们指定获取应用层 lyshark.exe 进程的模块信息，并可得到以下输出效果：

#	Time	Debug Print
1	0.00000000	81576562500 - STORMINI: StorNVMe - POWER: IDLE
2	0.30965930	hello lyshark.com
3	0.30967739	模块基址 = 0000000014000000 大小 = 28672 模块名 = lyshark.exe 完整路径= C:\Us
4	0.30968061	模块基址 = 00007FFADCE20000 大小 = 2031616 模块名 = ntdll.dll 完整路径= C:\Win
5	0.30968341	模块基址 = 00007FFADCB00000 大小 = 729088 模块名 = KERNEL32.DLL 完整路径= C:\W
6	0.30968660	模块基址 = 00007FFADA240000 大小 = 2764800 模块名 = KERNELBASE.dll 完整路径= C
7	0.30968931	模块基址 = 00007FFAD73C0000 大小 = 585728 模块名 = apphelp.dll 完整路径= C:\W
8	0.30969220	模块基址 = 00007FFADB970000 大小 = 1654784 模块名 = USER32.dll 完整路径= C:\W
9	0.30969521	模块基址 = 00007FFADAEA0000 大小 = 135168 模块名 = win32u.dll 完整路径= C:\Win
10	0.30969751	模块基址 = 00007FFADBE80000 大小 = 155648 模块名 = GDI32.dll 完整路径= C:\Win
11	0.30970019	模块基址 = 00007FFAD9F50000 大小 = 1654784 模块名 = gdi32full.dll 完整路径= C
12	0.30970320	模块基址 = 00007FFADA1A0000 大小 = 647168 模块名 = msvcrt.dll 完整路径= C:\
13	0.30970591	模块基址 = 00007FFAD9DB0000 大小 = 1024000 模块名 = ucrtbase.dll 完整路径= C:
14	0.30970901	模块基址 = 00007FFABF770000 大小 = 978944 模块名 = MSVCRT120.dll 完整路径= C:\
15	0.30971181	模块基址 = 00007FFADCB00000 大小 = 188416 模块名 = IMM32.DLL 完整路径= C:\Win
16	0.30971441	模块基址 = 00007FFAD7AE0000 大小 = 626688 模块名 = uxtheme.dll 完整路径= C:\W
17	0.30971739	模块基址 = 00007FFADAED0000 大小 = 647168 模块名 = msvcrt.dll 完整路径= C:\Win

上篇文章中的代码就不再啰嗦了，这里只给出内存转存的核心代码，如下代码：

- RtlInitUnicodeString 用于初始化转存后的名字字符串
- ZwCreateFile 内核中创建文件到应用层
- ZwWriteFile 将文件写出到文件
- ZwClose 最后是关闭文件并释放堆空间

很简单只是利用了 SafeCopyMemory_R3_to_R0 将进程内存读取到缓冲区内，并将缓冲区写出到C盘目录下。

```

// 进程内存拷贝函数
// By: LyShark.com
NTSTATUS ProcessDumps(PEPROCESS pEprocess, ULONG_PTR nBase, ULONG nSize)
{
    BOOLEAN bAttach = FALSE;
    KAPC_STATE ks = { 0 };
    PVOID pBuffer = NULL;
    NTSTATUS status = STATUS_UNSUCCESSFUL;

    if (nSize == 0 || pEprocess == NULL)
    {
        return status;
    }

    pBuffer = ExAllocatePoolWithTag(PagedPool, nSize, 'lysh');
    if (!pBuffer)
    {
        return status;
    }

    memset(pBuffer, 0, nSize);
}

```

```

if (pEprocess != IoGetCurrentProcess())
{
    KeStackAttachProcess(pEprocess, &ks);
    bAttach = TRUE;
}

status = SafeCopyMemory_R3_to_R0(nBase, (ULONG_PTR)pBuffer, nSize);

if (bAttach)
{
    KeUnstackDetachProcess(&ks);
    bAttach = FALSE;
}

OBJECT_ATTRIBUTES object;
IO_STATUS_BLOCK io;
HANDLE hFile;
UNICODE_STRING log;

// 导出文件名称
RtlInitUnicodeString(&log, L"\\??\\C:\\\\lyshark_dumps.exe");
InitializeObjectAttributes(&object, &log, OBJ_CASE_INSENSITIVE, NULL, NULL);

status = ZwCreateFile(&hFile,
    GENERIC_WRITE,
    &object,
    &io,
    NULL,
    FILE_ATTRIBUTE_NORMAL,
    FILE_SHARE_WRITE,
    FILE_OPEN_IF,
    FILE_SYNCHRONOUS_IO_NONALERT,
    NULL,
    0);

if (!NT_SUCCESS(status))
{
    DbgPrint("打开文件错误 \n");
    return STATUS_SUCCESS;
}

ZwWriteFile(hFile, NULL, NULL, NULL, &io, pBuffer, nSize, NULL, NULL);
DbgPrint("写出字节数: %d \n", io.Information);
DbgPrint("[*] LyShark.exe 已转存");
ZwClose(hFile);

if (pBuffer)
{
    ExFreePoolWithTag(pBuffer, 'lysh');
    pBuffer = NULL;
}

return status;
}

```

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

// lyshark.com
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

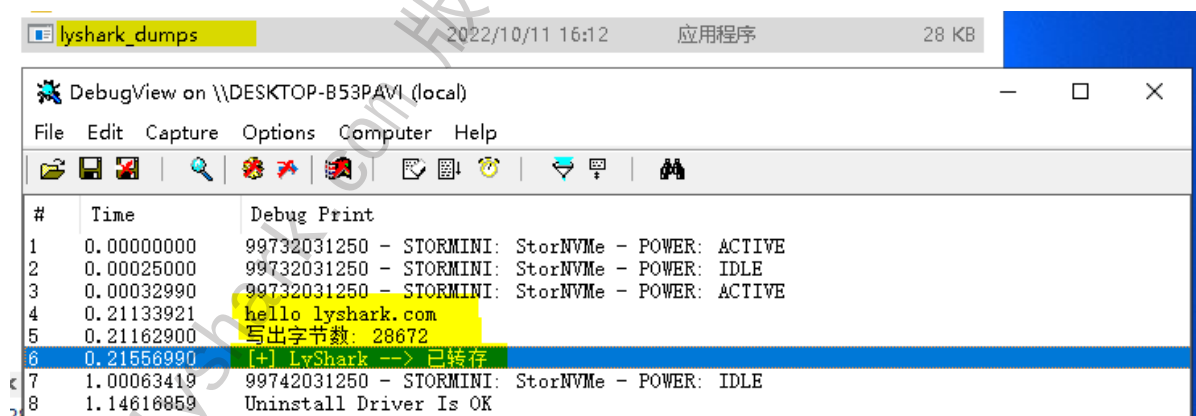
    NTSTATUS ntStatus;
    PEPROCESS pCurProcess = NULL;

    __try
    {
        ntStatus = PsLookupProcessByProcessId((HANDLE)272, &pCurProcess);
        if (NT_SUCCESS(ntStatus))
        {
            // 设置基地址以及长度
            ntStatus = ProcessDumps(pCurProcess, 0x140000000, 1024);
            ObDereferenceObject(pCurProcess);
        }
    }
    __except (1)
    {
        ntStatus = GetExceptionCode();
    }

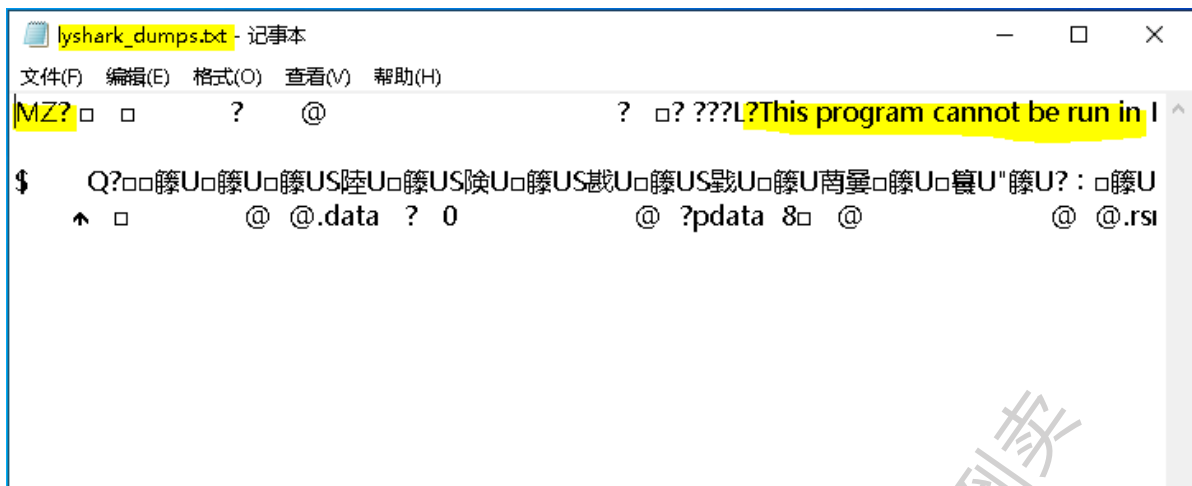
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

转存后效果如下所示：



至于导出的进程无法运行只是没有修复而已(后期会讲)，可以打开看看是没错的。



6.6 内核遍历进程VAD结构体

在上一篇文章《驱动开发：内核中实现Dump进程转储》中我们实现了ARK工具的转存功能，本篇文章继续以内存为出发点介绍 VAD 结构，该结构的全称是 Virtual Address Descriptor 即 虚拟地址描述符，VAD 是一个 AVL 自平衡二叉树，树的每一个节点代表一段虚拟地址空间。程序中的代码段，数据段，堆段都会各种占用一个或多个 VAD 节点，由一个 MMVAD 结构完整描述。

VAD结构的遍历效果如下：

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
Dialog												
地址	大小	Protect	State	Type	模块							
0x0	0x00010000	No Access	Free									
0x10000	0x00010000	ReadWrite	Commit	Map								
0x20000	0x00002000	ReadWrite	Commit	Private								
0x22000	0x0000B000		Reserve	Private								
0x2D000	0x00003000	No Access	Free									
0x30000	0x0001B000	Read	Commit	Map								
0x4B000	0x00005000	No Access	Free									
0x50000	0x000F7000		Reserve	Private								
0x147000	0x00003000	ReadWrite & Guard	Commit	Private								
0x14A000	0x00006000	ReadWrite	Commit	Private								

那么这个结构在哪？每一个进程都有自己单独的 VAD 结构树，这个结构通常在 EPROCESS 结构里面里面，在内核调试模式下使用 `dt _EPROCESS` 可得到如下信息。

```
lyshark.com 1: kd> dt _EPROCESS
ntdll!_EPROCESS
+0x500 Vm : _MMSUPPORT_FULL
+0x640 MmProcessLinks : _LIST_ENTRY
+0x650 ModifiedPageCount : Uint4B
+0x654 ExitStatus : Int4B
+0x658 VadRoot : _RTL_AVL_TREE
+0x660 VadHint : Ptr64 Void
+0x668 VadCount : Uint8B
+0x670 VadPhysicalPages : Uint8B
+0x678 VadPhysicalPagesLimit : Uint8B
```

可以看到在本系统中VAD的偏移是 +0x658 紧跟其后的还有 vadCount 的计数等。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
+0x654 ExitStatus      : Int4B
+0x658 VadRoot         : RTL_AVL_TREE
+0x660 VadHint         : Ptr64 Void
+0x668 VadCount        : Uint8B
+0x670 VadPhysicalPages : Uint8B
+0x678 VadPhysicalPagesLimit : Uint8B
+0x680 AlpcContext     : _ALPC_PROCESS_CONTEXT
+0x6a0 TimerResolutionLink : _LIST_ENTRY
+0x6b0 TimerResolutionStackRecord : Ptr64 _PO_DIAG_STACK_RECORD
+0x6b8 RequestedTimerResolution : Uint4B
+0x6bc SmallestTimerResolution : Uint4B
+0x6c0 ExitTime        : _LARGE_INTEGER
+0x6c8 InvertedFunctionTable : Ptr64 _INVERTED_FUNCTION_TABLE
+0x6d0 InvertedFunctionTableLock : _EX_PUSH_LOCK
+0x6d8 ActiveThreadsHighWatermark : Uint4B
```

VAD结构是如何被添加的？通常情况下系统调用 `VirtualAllocate` 等申请一段堆内存时，则会在VAD树上增加一个结点 `_MMVAD` 结构体，需要说明的是栈并不受VAD的管理。由系统直接分配空间，并把地址记录在了TEB中。

```
lyshark.com 0: kd> dt _MMVAD
nt!_MMVAD
+0x000 Core          : _MMVAD_SHORT
+0x040 u2            : <anonymous-tag>
+0x048 Subsection    : Ptr64 _SUBSECTION
+0x050 FirstPrototypePte : Ptr64 _MMPTE
+0x058 LastContiguousPte : Ptr64 _MMPTE
+0x060 ViewLinks     : _LIST_ENTRY
+0x070 VadsProcess   : Ptr64 _EPROCESS
+0x078 u4            : <anonymous-tag>
+0x080 FileObject    : Ptr64 _FILE_OBJECT
```

结构体 `MMVAD` 则是每一个 VAD 内存块的属性，这个内存结构定义在WinDBG中可看到。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
1: kd> dt _MMVAD
nt!_MMVAD
+0x000 Core          : _MMVAD_SHORT
+0x040 u2            : <anonymous-tag>
+0x048 Subsection    : Ptr64 _SUBSECTION
+0x050 FirstPrototypePte : Ptr64 _MMPTE
+0x058 LastContiguousPte : Ptr64 _MMPTE
+0x060 ViewLinks     : _LIST_ENTRY
+0x070 VadsProcess   : Ptr64 _EPROCESS
+0x078 u4            : <anonymous-tag>
+0x080 FileObject    : Ptr64 _FILE_OBJECT
```

如上在 `EPROCESS` 结构中可以找到VAD结构的相对偏移 `+0x658` 以及进程VAD计数偏移 `+0x668`，我们首先通过 `!process 0 0` 指令得到当前所有进程的 `EPROCESS` 结构，并选中进程。

```
lyshark.com 0: kd> !process 0 0
PROCESS fffffe28fbb0860c0
  SessionId: 1 Cid: 11a8 Peb: 0035c000 ParentCid: 11c8
  DirBase: 309f3002 ObjectTable: fffffac87ba3da580 HandleCount: 145.
  Image: x64.exe
```

此处的 fffffe28fbb0860c0 正是我们所需要的 EPROCESS 结构。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help

Command
DirBase: 0833b002 ObjectTable: fffffac87bca848c0 HandleCount: 221.
Image: svchost.exe

PROCESS fffffe28fbb0860c0
SessionId: 1 Cid: 11a8 Peb: 0035c000 ParentCid: 11c8
DirBase: 309f3002 ObjectTable: fffffac87ba3da580 HandleCount: 145.
Image: x64.exe

PROCESS fffffe28fbc239080
SessionId: 0 Cid: 0518 Peb: db0a1d5000 ParentCid: 0274
DirBase: 905b8002 ObjectTable: fffffac87b89bd4c0 HandleCount: 120.
Image: svchost.exe
```

当需要得到该进程的VAD结构时，只需要使用 !vad fffffe28fbb0860c0 + 0x658 来显示该进程的VAD树。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help

Command
1: kd> !vad fffffe28fbb0860c0 + 0x658
VAD      Level      Start      End Commit
fffffe28fbcdbbdc0 5      10      1f      0 Mapped      READWRITE      Pagefile
fffffe28fbad23b90 6      20      2c      2 Private     READWRITE      Pagefile
fffffe28fbcdbbbe0 4      30      4a      0 Mapped      READONLY      Pagefile
fffffe28fbb64da00 5      50      14f     9 Private     READWRITE      Pagefile
fffffe28fbcdbbbc80 3      150     153     0 Mapped      READONLY      Pagefile
fffffe28fbcdbaba0 6      160     160     0 Mapped      READONLY      Pagefile
fffffe28fbb64df50 5      170     171     2 Private     READWRITE      Pagefile
fffffe28fbcdbbc7c0 6      180     187     0 Mapped      READONLY      Pagefile
fffffe28fbb7200b0 4      190     190     1 Private     READWRITE      Pagefile
fffffe28fbb71fc00 6      1a0     1a0     1 Private     READWRITE      Pagefile
fffffe28fbb71fa20 5      1b0     1bc     1 Private     READWRITE      Pagefile
fffffe28fbcdbcd60 7      1c0     1c3     0 Mapped      READONLY      Pagefile
fffffe28fbb71fd90 6      1d0     1dc     1 Private     READWRITE      Pagefile
fffffe28fbcdbc1040 7      1e0     1e4     0 Mapped      READWRITE      Pagefile
fffffe28fbb64de10 2      200     3ff     3 Private     READWRITE      Pagefile
fffffe28fbb720150 6      450     45f     7 Private     READWRITE      Pagefile
```

至于获取VAD有多少条，则可以直接使用 !vad fffffe28fbb0860c0 + 0x668 来获取到。

```
Total VADs: 66, average level: 6, maximum depth: 7
Total private commit: 0x13a pages (1256 KB)
Total shared commit: 0x4b6 pages (4824 KB)
1: kd> !vad fffffe28fbb0860c0 + 0x668
VAD      Level      Start      End Commit
Unable to get LeftChild of nt!_MMVAD_SHORT at 0000000000000042

1: kd>
```

既然手动可以遍历出来，那么自动化也并不难，首先定义头文件 vad.h 同样这是微软定义，如果想要的到最新的，自己下载WinDBG调试内核输入命令。

```
#pragma once
#include <ntifs.h>

typedef struct _MM_GRAPHICS_VAD_FLAGS // 15 elements, 0x4 bytes (sizeof)
{
    /*0x000*/ ULONG32 Lock : 1; // 0 BitPosition
```

```

/*0x000*/    ULONG32    LockContended : 1;           // 1 BitPosition

/*0x000*/    ULONG32    DeleteInProgress : 1;         // 2 BitPosition

/*0x000*/    ULONG32    NoChange : 1;                 // 3 BitPosition

/*0x000*/    ULONG32    VadType : 3;                  // 4 BitPosition

/*0x000*/    ULONG32    Protection : 5;               // 7 BitPosition

/*0x000*/    ULONG32    PreferredNode : 6;            // 12 BitPosition

/*0x000*/    ULONG32    PageSize : 2;                 // 18 BitPosition

/*0x000*/    ULONG32    PrivateMemoryAlwaysSet : 1;   // 20 BitPosition

/*0x000*/    ULONG32    Writewatch : 1;               // 21 BitPosition

/*0x000*/    ULONG32    FixedLargePageSize : 1;       // 22 BitPosition

/*0x000*/    ULONG32    ZeroFillPagesOptional : 1;    // 23 BitPosition

/*0x000*/    ULONG32    GraphicsAlwaysSet : 1;         // 24 BitPosition

/*0x000*/    ULONG32    GraphicsUseCoherentBus : 1;    // 25 BitPosition

/*0x000*/    ULONG32    GraphicsPageProtection : 3;   // 26 BitPosition

}MM_GRAPHICS_VAD_FLAGS, *PMM_GRAPHICS_VAD_FLAGS;
typedef struct _MM_PRIVATE_VAD_FLAGS // 15 elements, 0x4 bytes (sizeof)
{
/*0x000*/    ULONG32    Lock : 1;                       // 0 BitPosition

/*0x000*/    ULONG32    LockContended : 1;              // 1 BitPosition

/*0x000*/    ULONG32    DeleteInProgress : 1;            // 2 BitPosition

/*0x000*/    ULONG32    NoChange : 1;                   // 3 BitPosition

/*0x000*/    ULONG32    VadType : 3;                    // 4 BitPosition

/*0x000*/    ULONG32    Protection : 5;                 // 7 BitPosition

/*0x000*/    ULONG32    PreferredNode : 6;              // 12 BitPosition

/*0x000*/    ULONG32    PageSize : 2;                   // 18 BitPosition

/*0x000*/    ULONG32    PrivateMemoryAlwaysSet : 1;     // 20 BitPosition

/*0x000*/    ULONG32    Writewatch : 1;                 // 21 BitPosition

/*0x000*/    ULONG32    FixedLargePageSize : 1;         // 22 BitPosition

/*0x000*/    ULONG32    ZeroFillPagesOptional : 1;      // 23 BitPosition

```

```

/*0x000*/    ULONG32    Graphics : 1;           // 24 BitPosition

/*0x000*/    ULONG32    Enclave : 1;           // 25 BitPosition

/*0x000*/    ULONG32    ShadowStack : 1;       // 26 BitPosition

}MM_PRIVATE_VAD_FLAGS, *PMM_PRIVATE_VAD_FLAGS;

typedef struct _MMVAD_FLAGS                // 9 elements, 0x4 bytes (sizeof)
{
    /*0x000*/    ULONG32    Lock : 1;           // 0 BitPosition

    /*0x000*/    ULONG32    LockContended : 1;  // 1 BitPosition

    /*0x000*/    ULONG32    DeleteInProgress : 1; // 2 BitPosition

    /*0x000*/    ULONG32    NoChange : 1;       // 3 BitPosition

    /*0x000*/    ULONG32    VadType : 3;        // 4 BitPosition

    /*0x000*/    ULONG32    Protection : 5;     // 7 BitPosition

    /*0x000*/    ULONG32    PreferredNode : 6;   // 12 BitPosition

    /*0x000*/    ULONG32    PageSize : 2;       // 18 BitPosition

    /*0x000*/    ULONG32    PrivateMemory : 1;   // 20 BitPosition

}MMVAD_FLAGS, *PMMVAD_FLAGS;

typedef struct _MM_SHARED_VAD_FLAGS        // 11 elements, 0x4 bytes
(sizeof)
{
    /*0x000*/    ULONG32    Lock : 1;           // 0 BitPosition

    /*0x000*/    ULONG32    LockContended : 1;  // 1 BitPosition

    /*0x000*/    ULONG32    DeleteInProgress : 1; // 2 BitPosition

    /*0x000*/    ULONG32    NoChange : 1;       // 3 BitPosition

    /*0x000*/    ULONG32    VadType : 3;        // 4 BitPosition

    /*0x000*/    ULONG32    Protection : 5;     // 7 BitPosition

    /*0x000*/    ULONG32    PreferredNode : 6;   // 12 BitPosition

    /*0x000*/    ULONG32    PageSize : 2;       // 18 BitPosition

    /*0x000*/    ULONG32    PrivateMemoryAlwaysClear : 1; // 20 BitPosition

    /*0x000*/    ULONG32    PrivateFixup : 1;    // 21 BitPosition

```

```

/*0x000*/    ULONG32    HotPatchAllowed : 1;           // 22 BitPosition

}MM_SHARED_VAD_FLAGS, *PMM_SHARED_VAD_FLAGS;

typedef struct _MMVAD_FLAGS2                                // 7 elements, 0x4 bytes (sizeof)
{
    /*0x000*/    ULONG32    FileOffset : 24;           // 0 BitPosition

    /*0x000*/    ULONG32    Large : 1;                // 24 BitPosition

    /*0x000*/    ULONG32    TrimBehind : 1;           // 25 BitPosition

    /*0x000*/    ULONG32    Inherit : 1;              // 26 BitPosition

    /*0x000*/    ULONG32    NoValidationNeeded : 1;   // 27 BitPosition

    /*0x000*/    ULONG32    PrivateDemandZero : 1;   // 28 BitPosition

    /*0x000*/    ULONG32    Spare : 3;                // 29 BitPosition

}MMVAD_FLAGS2, *PMMVAD_FLAGS2;

typedef struct _MMVAD_SHORT
{
    RTL_BALANCED_NODE VadNode;
    UINT32 StartingVpn;                                /*0x18*/
    UINT32 EndingVpn;                                  /*0x01C*/
    UCHAR StartingVpnHigh;
    UCHAR EndingVpnHigh;
    UCHAR CommitChargeHigh;
    UCHAR SpareNT64VadUChar;
    INT32 ReferenceCount;
    EX_PUSH_LOCK PushLock;                            /*0x028*/
    struct
    {
        union
        {
            ULONG_PTR flag;
            MM_PRIVATE_VAD_FLAGS PrivateVadFlags;
        } /*0x030*/
        MMVAD_FLAGS VadFlags;
        MM_GRAPHICS_VAD_FLAGS GraphicsVadFlags;
        MM_SHARED_VAD_FLAGS SharedVadFlags;
    } Flags;
    }u1;

    PVOID EventList;                                  /*0x038*/

}MMVAD_SHORT, *PMMVAD_SHORT;

typedef struct _MMADDRESS_NODE
{
    ULONG64 u1;
    struct _MMADDRESS_NODE* LeftChild;

```

```

    struct _MMADDRESS_NODE* RightChild;
    ULONG64 StartingVpn;
    ULONG64 EndingVpn;
}MMADDRESS_NODE, *PMMADDRESS_NODE;

typedef struct _MMEXTEND_INFO // 2 elements, 0x10 bytes (sizeof)
{
    /*0x000*/    UINT64    CommittedSize;
    /*0x008*/    ULONG32    ReferenceCount;
    /*0x00c*/    UINT8      _PADDING0_[0x4];
}MMEXTEND_INFO, *PMMEXTEND_INFO;
struct _SEGMENT
{
    struct _CONTROL_AREA* ControlArea;
    ULONG TotalNumberOfPtes;
    ULONG SegmentFlags;
    ULONG64 NumberOfCommittedPages;
    ULONG64 SizeOfSegment;
    union
    {
        struct _MMEXTEND_INFO* ExtendInfo;
        void* BasedAddress;
    }u;
    ULONG64 SegmentLock;
    ULONG64 u1;
    ULONG64 u2;
    PVOID* PrototypePte;
    ULONGLONG ThePtes[0x1];
};

typedef struct _EX_FAST_REF
{
    union
    {
        PVOID Object;
        ULONG_PTR RefCnt : 3;
        ULONG_PTR Value;
    };
} EX_FAST_REF, *PEX_FAST_REF;

typedef struct _CONTROL_AREA // 17 elements, 0x80 bytes
(sizeof)
{
    /*0x000*/    struct _SEGMENT* Segment;
    union // 2 elements, 0x10 bytes
    (sizeof)
    {
        /*0x008*/    struct _LIST_ENTRY ListHead; // 2
        /*0x008*/    VOID* AweContext;
    };
    /*0x018*/    UINT64    NumberOfSectionReferences;
    /*0x020*/    UINT64    NumberOfPfnReferences;
    /*0x028*/    UINT64    NumberOfMappedViews;
    /*0x030*/    UINT64    NumberOfUserReferences;

```

```

        /*0x038*/      ULONG32 u;                                // 2 elements, 0x4 bytes
(sizeof)
        /*0x03c*/      ULONG32 u1;                               // 2 elements, 0x4 bytes
(sizeof)
        /*0x040*/      struct _EX_FAST_REF FilePointer;         // 3 elements,
0x8 bytes (sizeof)
        // 4 elements, 0x8 bytes (sizeof)
}CONTROL_AREA, *PCONTROL_AREA;

typedef struct _SUBSECTION_
{
    struct _CONTROL_AREA* ControlArea;

}SUBSECTION, *PSUBSECTION;

typedef struct _MMVAD
{
    MMVAD_SHORT Core;
    union                                /*0x040*/
    {
        UINT32 LongFlags2;
        //现在用不到省略
        MMVAD_FLAGS2 VadFlags2;

    }u2;
    PSUBSECTION Subsection;              /*0x048*/
    PVOID FirstPrototypePte;             /*0x050*/
    PVOID LastContiguousPte;             /*0x058*/
    LIST_ENTRY ViewLinks;                 /*0x060*/
    PEPROCESS VadsProcess;                /*0x070*/
    PVOID u4;                             /*0x078*/
    PVOID FileObject;                    /*0x080*/
}MMVAD, *PMMVAD;

typedef struct _RTL_AVL_TREE              // 1 elements, 0x8 bytes (sizeof)
{
    /*0x000*/      struct _RTL_BALANCED_NODE* Root;
}RTL_AVL_TREE, *PRTL_AVL_TREE;

typedef struct _VAD_INFO_
{
    ULONG_PTR pVad;
    ULONG_PTR startVpn;
    ULONG_PTR endVpn;
    ULONG_PTR pFileObject;
    ULONG_PTR flags;
}VAD_INFO, *PVAD_INFO;

typedef struct _ALL_VADS_
{
    ULONG nCnt;
    VAD_INFO VadInfos[1];
}ALL_VADS, *PALL_VADS;

```



```

typedef struct _MMSECTION_FLAGS // 27 elements, 0x4 bytes
(
    sizeof
    {
        /*0x000*/    UINT32    BeingDeleted : 1; // 0
        BitPosition
        /*0x000*/    UINT32    BeingCreated : 1; // 1
        BitPosition
        /*0x000*/    UINT32    BeingPurged : 1; // 2
        BitPosition
        /*0x000*/    UINT32    NoModifiedWriting : 1; // 3
        BitPosition
        /*0x000*/    UINT32    FailAllIo : 1; // 4
        BitPosition
        /*0x000*/    UINT32    Image : 1; // 5
        BitPosition
        /*0x000*/    UINT32    Based : 1; // 6
        BitPosition
        /*0x000*/    UINT32    File : 1; // 7
        BitPosition
        /*0x000*/    UINT32    AttemptingDelete : 1; // 8
        BitPosition
        /*0x000*/    UINT32    PrefetchCreated : 1; // 9
        BitPosition
        /*0x000*/    UINT32    PhysicalMemory : 1; // 10
        BitPosition
        /*0x000*/    UINT32    ImageControlAreaOnRemovableMedia : 1; // 11
        BitPosition
        /*0x000*/    UINT32    Reserve : 1; // 12
        BitPosition
        /*0x000*/    UINT32    Commit : 1; // 13
        BitPosition
        /*0x000*/    UINT32    NoChange : 1; // 14
        BitPosition
        /*0x000*/    UINT32    WasPurged : 1; // 15
        BitPosition
        /*0x000*/    UINT32    UserReference : 1; // 16
        BitPosition
        /*0x000*/    UINT32    GlobalMemory : 1; // 17
        BitPosition
        /*0x000*/    UINT32    DeleteOnClose : 1; // 18
        BitPosition
        /*0x000*/    UINT32    FilePointerNull : 1; // 19
        BitPosition
        /*0x000*/    ULONG32    PreferredNode : 6; // 20
        BitPosition
        /*0x000*/    UINT32    GlobalOnlyPerSession : 1; // 26
        BitPosition
        /*0x000*/    UINT32    UserWritable : 1; // 27
        BitPosition
        /*0x000*/    UINT32    SystemVaAllocated : 1; // 28
        BitPosition
        /*0x000*/    UINT32    PreferredFsCompressionBoundary : 1; // 29
        BitPosition
        /*0x000*/    UINT32    UsingFileExtents : 1; // 30
        BitPosition
    }

```

```

/*0x000*/      UINT32      PageSize64K : 1;                // 31
BitPosition
}MMSECTION_FLAGS, *PMMSECTION_FLAGS;

typedef struct _SECTION                                // 9 elements, 0x40 bytes
(sizeof)
{
    /*0x000*/      struct _RTL_BALANCED_NODE SectionNode;    // 6 elements,
0x18 bytes (sizeof)
    /*0x018*/      UINT64      StartingVpn;
    /*0x020*/      UINT64      EndingVpn;
    /*0x028*/      union {
        PCONTROL_AREA   ControlArea;
        PVOID           FileObject;

        }u1;                // 4 elements, 0x8 bytes (sizeof)
    /*0x030*/      UINT64      SizeOfSection;
    /*0x038*/      union {
        ULONG32   LongFlags;
        MMSECTION_FLAGS   Flags;
        }u;                // 2 elements, 0x4 bytes (sizeof)
    struct                                // 3 elements, 0x4 bytes
(sizeof)
    {
        /*0x03C*/      ULONG32      InitialPageProtection : 12; // 0
        BitPosition
        /*0x03C*/      ULONG32      SessionId : 19;                // 12
        BitPosition
        /*0x03C*/      ULONG32      NoValidationNeeded : 1;        // 31
        BitPosition
    };
}SECTION, *PSECTION;

```

引入 `vad.h` 头文件，并写入如下代码，此处的 `eprocess_offset_VadRoot` 以及 `eprocess_offset_VadCount` 则是上方得出的相对于 `EPROCESS` 结构的偏移值，每个系统都不一样，版本不同偏移值会不同。

```

#include "vad.h"
#include <ntifs.h>

// 定义VAD相对于EProcess头部偏移值
#define eprocess_offset_VadRoot 0x658
#define eprocess_offset_VadCount 0x668

VOID EnumVad(PMMVAD Root, PALL_VADS pBuffer, ULONG nCnt)
{
    if (!Root || !pBuffer || !nCnt)
    {
        return;
    }

    __try
    {
        if (nCnt > pBuffer->nCnt)
        {

```

```

// 得到起始页与结束页
ULONG64 endptr = (ULONG64)Root->Core.EndingVpnHigh;
endptr = endptr << 32;

ULONG64 startptr = (ULONG64)Root->Core.StartingVpnHigh;
startptr = startptr << 32;

// 得到根节点
pBuffer->VadInfos[pBuffer->nCnt].pVad = (ULONG_PTR)Root;

// 起始页: startingVpn * 0x1000
pBuffer->VadInfos[pBuffer->nCnt].startVpn = (startptr | Root-
>Core.StartingVpn) << PAGE_SHIFT;

// 结束页: EndVpn * 0x1000 + 0xfff
pBuffer->VadInfos[pBuffer->nCnt].endVpn = ((endptr | Root-
>Core.EndingVpn) << PAGE_SHIFT) + 0xffff;

// VAD标志 928 = Mapped 1049088 = Private ....
pBuffer->VadInfos[pBuffer->nCnt].flags = Root->Core.u1.Flags.flag;

// 验证节点可读性
if (MmIsAddressValid(Root->Subsection) && MmIsAddressValid(Root-
>Subsection->ControlArea))
{
    if (MmIsAddressValid((PVOID)((Root->Subsection->ControlArea-
>FilePointer.Value >> 4) << 4)))
    {
        pBuffer->VadInfos[pBuffer->nCnt].pFileObject = ((Root-
>Subsection->ControlArea->FilePointer.Value >> 4) << 4);
    }
}
pBuffer->nCnt++;
}

if (MmIsAddressValid(Root->Core.VadNode.Left))
{
    // 递归枚举左子树
    EnumVad((PMMVAD)Root->Core.VadNode.Left, pBuffer, nCnt);
}

if (MmIsAddressValid(Root->Core.VadNode.Right))
{
    // 递归枚举右子树
    EnumVad((PMMVAD)Root->Core.VadNode.Right, pBuffer, nCnt);
}
}
__except (1)
{
}
}

BOOLEAN EnumProcessVad(ULONG Pid, PALL_VADS pBuffer, ULONG nCnt)
{
    PEPROCESS Pprocess = 0;

```

```

PRTL_AVL_TREE Table = NULL;
PMMVAD Root = NULL;

// 通过进程PID得到进程EProcess
if (NT_SUCCESS(PsLookupProcessByProcessId((HANDLE)Pid, &Peprocess)))
{
    // 与偏移相加得到VAD头节点
    Table = (PRTL_AVL_TREE)((UCHAR*)Peprocess + eprocess_offset_VadRoot);
    if (!MmIsValidAddress(Table) || !eprocess_offset_VadRoot)
    {
        return FALSE;
    }

    __try
    {
        // 取出头节点
        Root = (PMMVAD)Table->Root;

        if (nCnt > pBuffer->nCnt)
        {
            // 得到起始页与结束页
            ULONG64 endptr = (ULONG64)Root->Core.EndingVpnHigh;
            endptr = endptr << 32;

            ULONG64 startptr = (ULONG64)Root->Core.StartingVpnHigh;
            startptr = startptr << 32;

            pBuffer->VadInfos[pBuffer->nCnt].pVad = (ULONG_PTR)Root;

            // 起始页: startingvpn * 0x1000
            pBuffer->VadInfos[pBuffer->nCnt].startVpn = (startptr | Root->Core.StartingVpn) << PAGE_SHIFT;

            // 结束页: EndVpn * 0x1000 + 0xfff
            pBuffer->VadInfos[pBuffer->nCnt].endVpn = (endptr | Root->Core.EndingVpn) << PAGE_SHIFT;
            pBuffer->VadInfos[pBuffer->nCnt].flags = Root->Core.u1.Flags.flag;

            if (MmIsValidAddress(Root->Subsection) && MmIsValidAddress(Root->Subsection->ControlArea))
            {
                if (MmIsValidAddress((PVOID)((Root->Subsection->ControlArea->FilePointer.Value >> 4) << 4)))
                {
                    pBuffer->VadInfos[pBuffer->nCnt].pFileObject = ((Root->Subsection->ControlArea->FilePointer.Value >> 4) << 4);
                }
            }
            pBuffer->nCnt++;
        }

        // 枚举左子树
        if (Table->Root->Left)
        {

```

```

        EnumVad((MMVAD*)Table->Root->Left, pBuffer, nCnt);
    }

    // 枚举右子树
    if (Table->Root->Right)
    {
        EnumVad((MMVAD*)Table->Root->Right, pBuffer, nCnt);
    }
}
__finally
{
    ObDereferenceObject(Peprocess);
}
}
else
{
    return FALSE;
}

return TRUE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    typedef struct
    {
        ULONG nPid;
        ULONG nSize;
        PALL_VADS pBuffer;
    }VADProcess;

    __try
    {
        VADProcess vad = { 0 };

        vad.nPid = 4520;

        // 默认有1000个线程
        vad.nSize = sizeof(VAD_INFO) * 0x5000 + sizeof(ULONG);

        // 分配临时空间
        vad.pBuffer = (PALL_VADS)ExAllocatePool(PagedPool, vad.nSize);

        // 根据传入长度得到枚举数量
        ULONG nCount = (vad.nSize - sizeof(ULONG)) / sizeof(VAD_INFO);

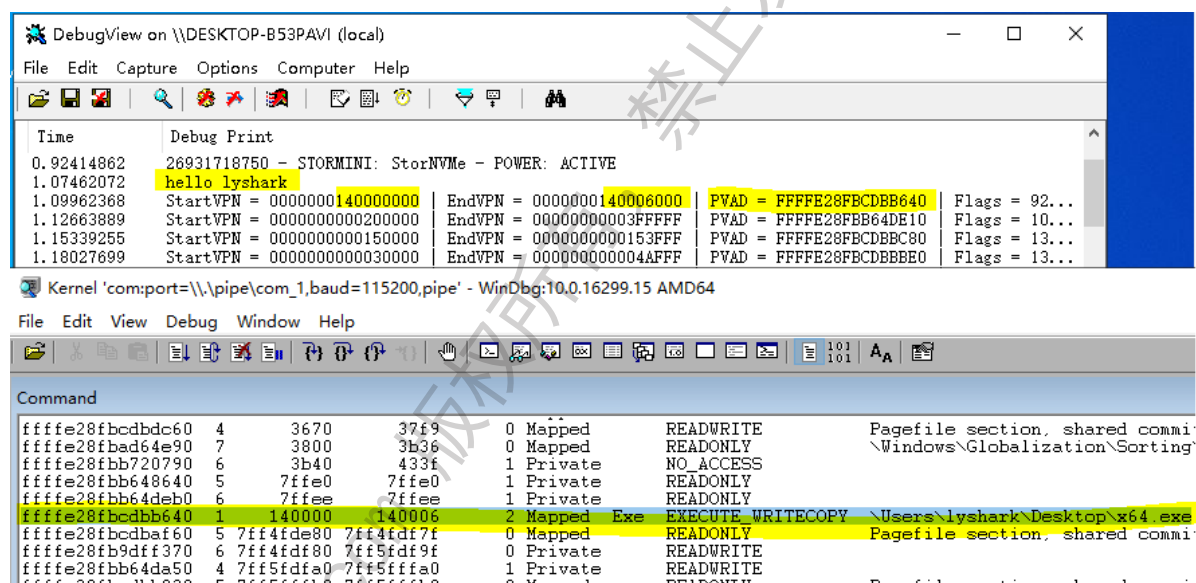
        // 枚举VAD
        EnumProcessVad(vad.nPid, vad.pBuffer, nCount);
    }
}

```

```
// 输出VAD
for (size_t i = 0; i < vad.pBuffer->nCnt; i++)
{
    DbgPrint("StartVPN = %p | ", vad.pBuffer->vadInfos[i].startVpn);
    DbgPrint("EndVPN = %p | ", vad.pBuffer->vadInfos[i].endVpn);
    DbgPrint("PVAD = %p | ", vad.pBuffer->vadInfos[i].pVad);
    DbgPrint("Flags = %d | ", vad.pBuffer->vadInfos[i].flags);
    DbgPrint("pFileObject = %p \n", vad.pBuffer->vadInfos[i].pFileObject);
}

__except (1)
{
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}
```

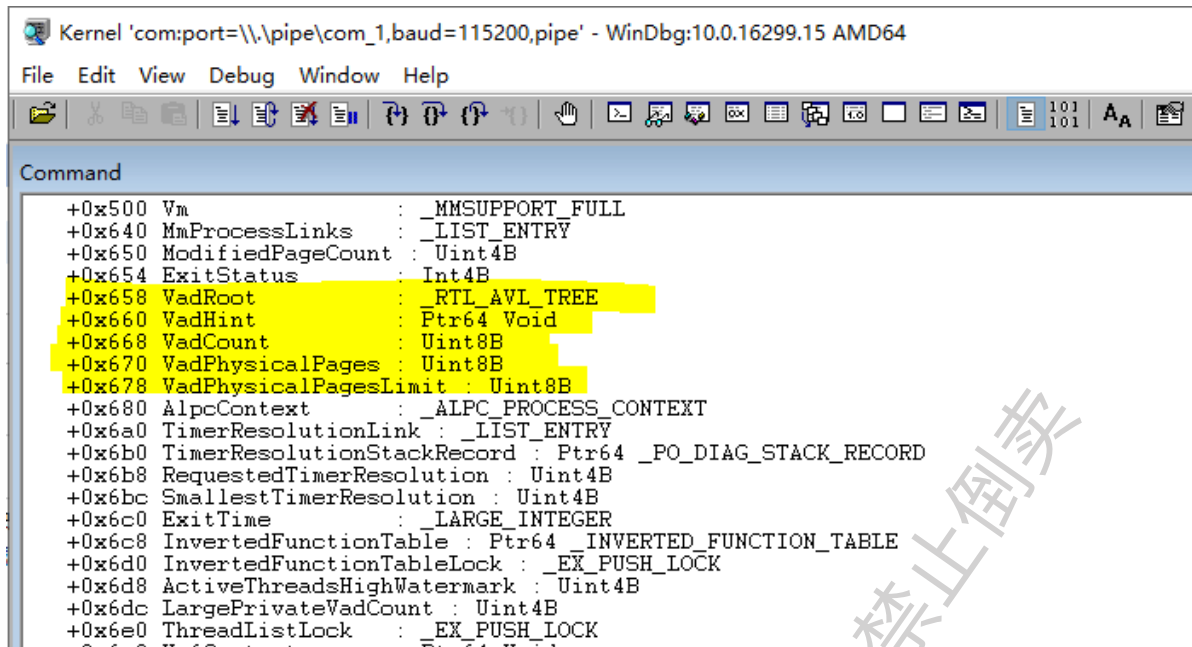
程序运行后输出效果如下:



6.7 运用VAD隐藏R3内存思路

在进程的 `_EPROCESS` 中有一个 `_RTL_AVL_TREE` 类型的 `VadRoot` 成员，它是一个存放进程内存块的二叉树结构，如果我们找到了这个二叉树中我们想要隐藏的内存，直接将这个内存在二叉树中抹去，其实是让上一个节点的 `EndingVpn` 指向下个节点的 `EndingVpn`，类似于摘链隐藏进程，就可以达到隐藏的效果。

通过 `dt _EPROCESS` 得到 `EProcess` 结构 `VadRoot` 如下:



例如当调用 `VirtualAlloc` 分配内存空间。

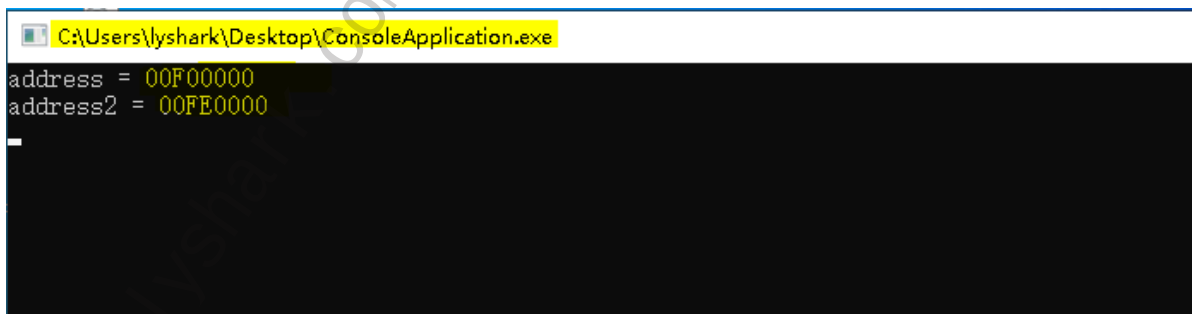
```
#include <iostream>
#include <windows.h>

int main(int argc, char *argv[])
{
    LPVOID p1 = VirtualAlloc(NULL, 0x10000, MEM_COMMIT, PAGE_READWRITE);
    LPVOID p2 = VirtualAlloc(NULL, 0x10000, MEM_COMMIT, PAGE_EXECUTE_READWRITE);

    std::cout << "address = " << p1 << std::endl;
    std::cout << "address2 = " << p2 << std::endl;

    getchar();
    return 0;
}
```

运行程序得到两个内存地址 `0xf00000` 和 `0xfe0000`



通过 `!process 0 0` 枚举所有进程，并得到我们所需进程的 `EProcess` 地址。

```

PROCESS ffffff28fbb6be080
  SessionId: 1 Cid: 0b8c Peb: d433d45000 ParentCid: 0310
  DirBase: b53ed002 ObjectTable: ffffff87bb1dfc40 HandleCount: 318.
  Image: smartscreen.exe

PROCESS ffffff28fbb451080
  SessionId: 1 Cid: 0584 Peb: 011f4000 ParentCid: 11c8
  DirBase: 2e56b002 ObjectTable: ffffff87bca84240 HandleCount: 59.
  Image: ConsoleApplication.exe

PROCESS ffffff28fbc210080
  SessionId: 0 Cid: 14d8 Peb: 1d79d86000 ParentCid: 0274
  DirBase: 91157002 ObjectTable: ffffff87bca85340 HandleCount: 258.
  Image: svchost.exe

PROCESS ffffff28fbb7e3080
  SessionId: 1 Cid: 12b4 Peb: 9c5eba8000 ParentCid: 0584
  DirBase: 2ca1b002 ObjectTable: ffffff87bca87640 HandleCount: 253.
  Image: conhost.exe

```

1: kd>

Ln 0, Col 0 Sys 0:KdSrv:S Pr

检查进程 !process ffffff28fbb451080 得到 VAD 地址 ffffff28fbc0b7e40

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```

1: kd> !process ffffff28fbb451080
PROCESS ffffff28fbb451080
  SessionId: 1 Cid: 0584 Peb: 011f4000 ParentCid: 11c8
  DirBase: 2e56b002 ObjectTable: ffffff87bca84240 HandleCount: 59.
  Image: ConsoleApplication.exe
  VadRoot: ffffff28fbc0b7e40 Vads 35 Clone 0 Private 134. Modified 8. Locked 2.
  DeviceMap ffffff87b7ef86f0
  Token ffffff87baeea5f0
  ElapsedTime 00:02:18.452
  UserTime 00:00:00.000
  KernelTime 00:00:00.000
  QuotaPoolUsage[PagedPool] 28056
  QuotaPoolUsage[NonPagedPool] 4952
  Working Set Sizes (now,min,max) (846, 50, 345) (3384KB, 200KB, 1380KB)
  PeakWorkingSetSize 838
  VirtualSize 13 Mb
  PeakVirtualSize 17 Mb
  PageFaultCount 923
  MemoryPriority BACKGROUND
  BasePriority 8
  CommitCharge 198
  Job ffffff28fbc288060

```

此处以 0xf00000 为例，这里我们看到 windbg 中的值和进程中分配的内存地址并不完全一样，这是因为 x86 cpu 默认内存页大小 4k 也就是 0x1000，所以这里还要再乘以 0x1000 才是真正的内存地址。

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```

1: kd> !vad ffffff28fbc0b7e40
VAD Level Start End Commit
fffff28fbc0b7da0 4 3b6 3b6 2 Mapped Exe EXECUTE_WRITECOPY \Users\
fffff28fbc0b8f20 3 ee0 eef 0 Mapped READWRITE Pagefil
fffff28fbc0b9cf880 4 ef0 ef7 1 Private READWRITE
fffff28fbc0b9e1f60 5 f00 f0f 16 Private READWRITE
fffff28fbc0b8c00 2 f10 f2a 0 Mapped READONLY Pagefil
fffff28fbc0b9dcf10 4 f30 f6f 11 Private READWRITE
fffff28fbc0b8ca0 3 f70 f73 0 Mapped READONLY Pagefil
fffff28fbc0b92e0 1 f80 f80 0 Mapped READONLY Pagefil
fffff28fbc0b9ddd70 5 f90 f91 2 Private READWRITE
fffff28fbc0b9e24b0 4 fe0 fef 16 Private EXECUTE_READWRITE

```

所以计算结果刚好等于 0xf00000



而隐藏进程内特定内存段核心代码在于 `p1->EndingVpn = p2->EndingVpn`; 将VAD前后节点连接。

```
PMMVAD p1 = vad_enum((PMMVAD)VadRoot, 0x3a0); // 遍历第一个结点
PMMVAD p2 = vad_enum((PMMVAD)VadRoot, 0x3b0); // 遍历找到第二个结点
if (p1 && p2)
{
    p1->EndingVpn = p2->EndingVpn; // 将第二个结点完全隐藏起来
}
```

6.8 内核摘链DKOM进程隐藏

DKOM 即直接内核对象操作，我们所有的操作都会被系统记录在内存中，而驱动进程隐藏就是操作进程的EPROCESS结构与线程的ETHREAD结构、链表，要实现进程的隐藏我们只需要将某个进程中的信息，在系统EPROCESS链表中摘除即可实现进程隐藏。

结构体中包含了系统中的所有进程相关信息，通过WinDBG在内核调试模式下输入 `dt _eprocess` 即可查看到当前的EPROCESS结构体的偏移信息。

```
1: kd> dt _EPROCESS

ntdll!_EPROCESS
+0x000 Pcb                : _KPROCESS
+0x2e0 ProcessLock        : _EX_PUSH_LOCK
+0x2e8 UniqueProcessId    : Ptr64 Void
+0x2f0 ActiveProcessLinks : _LIST_ENTRY // 活动进程链表
+0x300 RundownProtect     : _EX_RUNDOWN_REF
+0x308 Flags2             : Uint4B
```

在实现进程隐藏之前，需要通过代码的方式获取到当前系统中所有进程 EPROCESS 信息。

```
#include <ntifs.h>

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}
```

```

PEPROCESS GetProcessObjectByName(char *name)
{
    SIZE_T temp;
    for (temp = 100; temp < 10000; temp += 4)
    {
        NTSTATUS status;
        PEPROCESS ep;
        status = PsLookupProcessByProcessId((HANDLE)temp, &ep);
        if (NT_SUCCESS(status))
        {
            char *pn = PsGetProcessImageFileName(ep);
            if (_stricmp(pn, name) == 0)
                return ep;
        }
    }
    return NULL;
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    PEPROCESS Proc = NULL;
    Proc = GetProcessObjectByName("C32Asm.exe");
    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

得到句柄以后直接摘除进程的结构即可实现隐藏，该代码只找了Win10系统下的偏移地址，故只能在Win10下使用。

```

#include <ntifs.h>

#define PROCESS_ACTIVE_PROCESS_LINKS_OFFSET 0x2f0

NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI CHAR* PsGetProcessImageFileName(PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("驱动程序卸载成功! \n"));
}

PEPROCESS GetProcessObjectByName(char *name)
{
    SIZE_T temp;
    for (temp = 100; temp < 10000; temp += 4)
    {
        NTSTATUS status;
        PEPROCESS ep;
        status = PsLookupProcessByProcessId((HANDLE)temp, &ep);
        if (NT_SUCCESS(status))
        {
            char *pn = PsGetProcessImageFileName(ep);
            if (_stricmp(pn, name) == 0)
                return ep;
        }
    }
}

```

```

    }
}
return NULL;
}

// 隐藏进程
VOID HideProcess(PLIST_ENTRY ListEntry)
{
    KIRQL OldIrql;
    OldIrql = KeRaiseIrqlToDpcLevel();
    if (ListEntry->Flink != ListEntry && ListEntry->Blink != ListEntry &&
        ListEntry->Blink->Flink == ListEntry && ListEntry->Flink->Blink == ListEntry)
    {
        ListEntry->Flink->Blink = ListEntry->Blink;
        ListEntry->Blink->Flink = ListEntry->Flink;
        ListEntry->Flink = ListEntry;
        ListEntry->Blink = ListEntry;
    }
    KeLowerIrql(OldIrql);
}

NTSTATUS DriverEntry(PDRIVER_OBJECT DriverObject, PUNICODE_STRING RegistryPath)
{
    PEPROCESS Proc = NULL;
    Proc = GetProcessObjectByName("C32Asm.exe");

    // 摘除结构中的C32Asm.exe实现驱动隐藏
    HideProcess((PLIST_ENTRY)((ULONG64)Proc +
        PROCESS_ACTIVE_PROCESS_LINKS_OFFSET));

    DriverObject->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

6.9 内核无痕隐藏自身分析

在笔者前面有一篇文章《驱动开发：断链隐藏驱动程序自身》通过摘除驱动的链表实现了断链隐藏自身的目的，但此方法恢复时会触发PG会蓝屏，偶然间在网上找到了一个作者介绍的一种方法，觉得有必要详细分析一下他是如何实现的进程隐藏的，总体来说作者的思路是最终寻找到 `MiProcessLoaderEntry` 的入口地址，该函数的作用是将驱动信息加入链表和移除链表，运用这个函数即可动态处理驱动的添加和移除问题。

- `MiProcessLoaderEntry(pDriverObject->DriverSection, 1)` 添加
- `MiProcessLoaderEntry(pDriverObject->DriverSection, 0)` 移除

那么如何找到 `MiProcessLoaderEntry` 函数入口地址就是下一步的目标，寻找入口可以总结为：

1. 寻找 `MmUnloadSystemImage` 函数地址，可通过 `MmGetSystemRoutineAddress` 函数得到。
2. 在 `MmUnloadSystemImage` 里面寻找 `MiUnloadSystemImage` 函数地址。
3. 在 `MiUnloadSystemImage` 里面继续寻找 `MiProcessLoaderEntry` 即可。

搜索 `MmUnloadSystemImage` 可定位到 `call nt!MiUnloadSystemImage` 地址。

```

Command

0: kd> uf MmUnloadSystemImage
nt!MmUnloadSystemImage:
fffff801`37943500 48895c2408      mov     qword ptr [rsp+8],rbx
fffff801`37943505 57             push    rdi
fffff801`37943506 4883ec20       sub     rsp,20h
fffff801`3794350a 488bf9        mov     rdi,rcx
fffff801`3794350d e87eb3ebff     call    nt!MmAcquireLoadLock (fffff801`377fe890)
fffff801`37943512 83caff        or      edx,0FFFFFFFFh
fffff801`37943515 488bcf        mov     rcx,rdi
fffff801`37943518 488bd8        mov     rbx,rax
fffff801`3794351b e860b4ebff     call    nt!MiUnloadSystemImage (fffff801`377fe980)
fffff801`37943520 488bc8        mov     rcx,rbx
fffff801`37943523 e808b3ebff     call    nt!MmReleaseLoadLock (fffff801`377fe830)
fffff801`37943528 488b5c2430     mov     rbx,qword ptr [rsp+30h]
fffff801`3794352d 33c0          xor     eax,eax
fffff801`3794352f 4883c420     add     rsp,20h
fffff801`37943533 5f            pop     rdi
fffff801`37943534 c3            ret

```

搜索 MiUnloadSystemImage 定位到 call nt!MiProcessLoaderEntry 即得到了我们想要的。

```

Command

nt!MiUnloadSystemImage+0x399:
fffff801`377fed19 33d2          xor     edx,edx
fffff801`377fed1b 488bc8        mov     rcx,rbx
fffff801`377fed1e e84162b4ff     call    nt!MiProcessLoaderEntry (fffff801`37344f64)
fffff801`377fed23 8b05d756f7ff   mov     eax,dword ptr [nt!PerfGlobalGroupMask (fffff801`377fed29)
fffff801`377fed29 a804          test    al,4
fffff801`377fed2b 7440          je      nt!MiUnloadSystemImage+0x3ed (fffff801`377fed2e)

```

根据前面 枚举篇 系列文章，定位这段特征很容易实现，如下是一段参考代码。

```

// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntddk.h>
#include <ntstrsafe.h>

typedef NTSTATUS(__fastcall *MiProcessLoaderEntry)(PVOID pDriverSection, BOOLEAN
bLoad);

// 取出指定函数地址
PVOID GetProcAddress(WCHAR *FuncName)
{
    UNICODE_STRING u_FuncName = { 0 };
    PVOID ref = NULL;

    RtlInitUnicodeString(&u_FuncName, FuncName);
    ref = MmGetSystemRoutineAddress(&u_FuncName);

    if (ref != NULL)
    {
        return ref;
    }

    return ref;
}

```

```

// 特征定位 MiUnloadSystemImage
ULONG64 GetMiUnloadSystemImageAddress()
{
    // 在MmUnloadSystemImage函数中搜索的Code
    /*
    lyshark.com: kd> uf MmUnloadSystemImage
        ffffffff801`37943512 83caff          or          edx,0FFFFFFFFh
        ffffffff801`37943515 488bcf          mov         rcx,rdi
        ffffffff801`37943518 488bd8          mov         rbx,rax
        ffffffff801`3794351b e860b4ebff      call        nt!MiUnloadSystemImage
    (ffffffff801`377fe980)
    */
    CHAR MmUnloadSystemImage_Code[] = "\x83\xCA\xFF" // or          edx, 0FFFFFFFFh
        "\x48\x8B\xCF" // mov         rcx, rdi
        "\x48\x8B\xD8" // mov         rbx, rax
        "\xE8"; // call
    nt!MiUnloadSystemImage (ffffffff801`377fe980)

    ULONG_PTR MmUnloadSystemImageAddress = 0;
    ULONG_PTR MiUnloadSystemImageAddress = 0;
    ULONG_PTR StartAddress = 0;

    MmUnloadSystemImageAddress =
    (ULONG_PTR)GetProcAddress(L"MmUnloadSystemImage");
    if (MmUnloadSystemImageAddress == 0)
    {
        return 0;
    }

    // 在MmUnloadSystemImage中搜索特征码寻找MiUnloadSystemImage
    StartAddress = MmUnloadSystemImageAddress;
    while (StartAddress < MmUnloadSystemImageAddress + 0x500)
    {
        if (memcmp((VOID*)StartAddress, MmUnloadSystemImage_Code,
        strlen(MmUnloadSystemImage_Code)) == 0)
        {
            // 跳过call之前的指令
            StartAddress += strlen(MmUnloadSystemImage_Code);

            // 取出 MiUnloadSystemImage地址
            MiUnloadSystemImageAddress = *(LONG*)StartAddress + StartAddress +
4;
            break;
        }
        ++StartAddress;
    }

    if (MiUnloadSystemImageAddress != 0)
    {
        return MiUnloadSystemImageAddress;
    }
    return 0;
}

```

```

// 特征定位 MiProcessLoaderEntry
MiProcessLoaderEntry GetMiProcessLoaderEntry(ULONG64 StartAddress)
{
    if (StartAddress == 0)
    {
        return NULL;
    }

    while (StartAddress < StartAddress + 0x600)
    {
        // 操作数MiProcessLoaderEntry内存地址是动态变化的
        /*
        lyshark.com: kd> uf MiUnloadSystemImage
            ffffffff801`377fed19 33d2          xor     edx,edx
            ffffffff801`377fed1b 488bcb        mov     rcx,rbx
            ffffffff801`377fed1e e84162b4ff    call    nt!MiProcessLoaderEntry
(ffffffff801`37344f64)
            ffffffff801`377fed23 8b05d756f7ff mov     eax,dword ptr
[nt!PerfGlobalGroupMask (ffffffff801`37774400)]
            ffffffff801`377fed29 a804          test    al,4
            ffffffff801`377fed2b 7440          je
nt!MiUnloadSystemImage+0x3ed (ffffffff801`377fed6d) Branch
            E8 call | 8B 05 mov eax
        */

        // ffffffff801`377fed1e | ffffffff801`377fed23
        // 判断特征 0xE8(call) | 0x8B 0x05(mov eax)
        if (*(UCHAR*)StartAddress == 0xE8 && *(UCHAR*)(StartAddress + 5) ==
0x8B && *(UCHAR*)(StartAddress + 6) == 0x05)
        {
            // 跳过一个字节call的E8
            StartAddress++;

            // StartAddress + 1 + 4
            return (MiProcessLoaderEntry)(*(LONG*)StartAddress + StartAddress +
4);
        }
        ++StartAddress;
    }
    return NULL;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONG64 MiUnloadSystemImageAddress = GetMiUnloadSystemImageAddress();
    DbgPrint("MiUnloadSystemImageAddress = %p \n", MiUnloadSystemImageAddress);
}

```

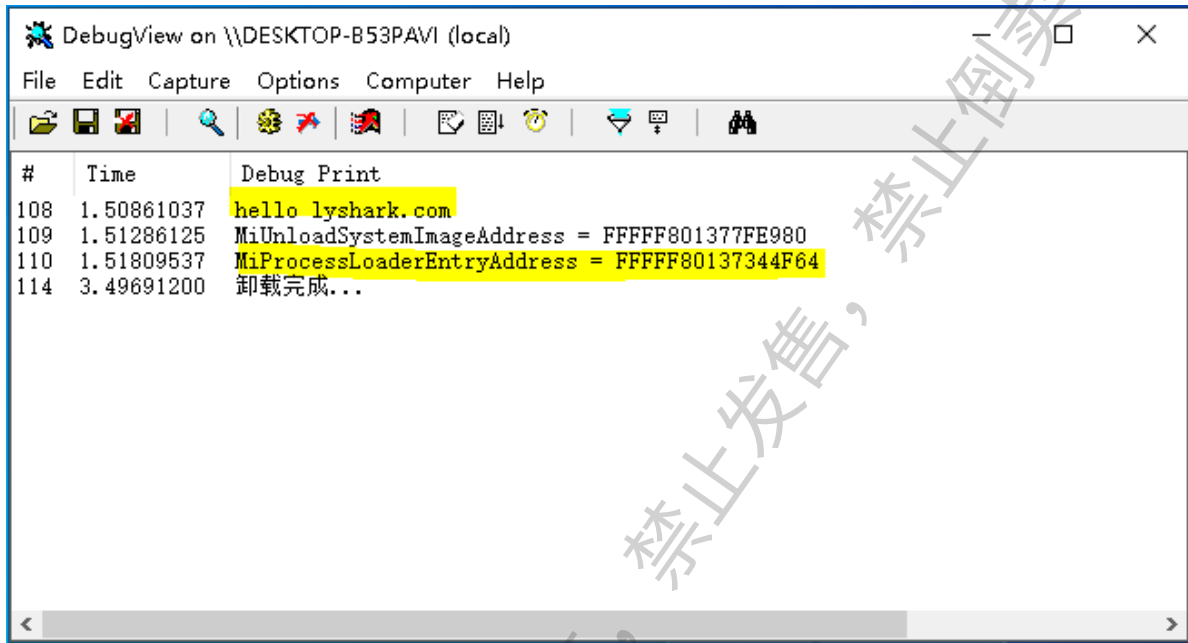
```

MiProcessLoaderEntry MiProcessLoaderEntryAddress =
GetMiProcessLoaderEntry(MiUnloadSystemImageAddress);
DbgPrint("MiProcessLoaderEntryAddress = %p \n",
(ULONG64)MiProcessLoaderEntryAddress);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

运行驱动程序，即可得到 MiProcessLoaderEntryAddress 的内存地址。



得到内存地址之后，直接破坏掉自身驱动的入口地址等，即可实现隐藏自身。

```

// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <ntstrsafe.h>

typedef NTSTATUS(*NTQUERYSYSTEMINFORMATION)(
    IN ULONG SystemInformationClass,
    OUT PVOID SystemInformation,
    IN ULONG_PTR SystemInformationLength,
    OUT PULONG_PTR ReturnLength OPTIONAL);

NTSYSAPI NTSTATUS NTAPI ObReferenceObjectByName(
    __in PUNICODE_STRING ObjectName,
    __in ULONG Attributes,
    __in_opt PACCESS_STATE AccessState,
    __in_opt ACCESS_MASK DesiredAccess,
    __in POBJECT_TYPE ObjectType,
    __in KPROCESSOR_MODE AccessMode,
    __inout_opt PVOID ParseContext,
    __out PVOID* Object
);

typedef struct _SYSTEM_MODULE_INFORMATION
{

```

```

HANDLE Section;
PVOID MappedBase;
PVOID Base;
ULONG Size;
ULONG Flags;
USHORT LoadOrderIndex;
USHORT InitOrderIndex;
USHORT LoadCount;
USHORT PathLength;
CHAR ImageName[256];
} SYSTEM_MODULE_INFORMATION, *PSYSTEM_MODULE_INFORMATION;

typedef struct _LDR_DATA_TABLE_ENTRY
{
    LIST_ENTRY InLoadOrderLinks;
    LIST_ENTRY InMemoryOrderLinks;
    LIST_ENTRY InInitializationOrderLinks;
    PVOID DllBase;
    PVOID EntryPoint;
} LDR_DATA_TABLE_ENTRY, *PLDR_DATA_TABLE_ENTRY;

extern POBJECT_TYPE *IoDriverObjectType;
typedef NTSTATUS(__fastcall *MiProcessLoaderEntry)(PVOID pDriverSection, BOOLEAN
bLoad);
ULONG64 MiUnloadSystemImageAddress = 0;

// 取出指定函数地址
PVOID GetProcAddress(WCHAR *FuncName)
{
    UNICODE_STRING u_FuncName = { 0 };
    PVOID ref = NULL;

    RtlInitUnicodeString(&u_FuncName, FuncName);
    ref = MmGetSystemRoutineAddress(&u_FuncName);

    if (ref != NULL)
    {
        return ref;
    }

    return ref;
}

// 特征定位 MiUnloadSystemImage
ULONG64 GetMiUnloadSystemImageAddress()
{
    CHAR MmUnloadSystemImage_Code[] =
        "\x83\xCA\xFF\x48\x8B\xCF\x48\x8B\xD8\xE8";

    ULONG_PTR MmUnloadSystemImageAddress = 0;
    ULONG_PTR MiUnloadSystemImageAddress = 0;
    ULONG_PTR StartAddress = 0;

    MmUnloadSystemImageAddress =
        (ULONG_PTR)GetProcAddress(L"MmUnloadSystemImage");

```



```

if (MmUnloadSystemImageAddress == 0)
{
    return 0;
}

// 在MmUnloadSystemImage中搜索特征码寻找MiUnloadSystemImage
StartAddress = MmUnloadSystemImageAddress;
while (StartAddress < MmUnloadSystemImageAddress + 0x500)
{
    if (memcmp((VOID*)StartAddress, MmUnloadSystemImage_Code,
        strlen(MmUnloadSystemImage_Code)) == 0)
    {
        StartAddress += strlen(MmUnloadSystemImage_Code);
        MiUnloadSystemImageAddress = *(LONG*)StartAddress + StartAddress + 4;
        break;
    }
    ++StartAddress;
}

if (MiUnloadSystemImageAddress != 0)
{
    return MiUnloadSystemImageAddress;
}
return 0;
}

// 特征定位 MiProcessLoaderEntry
MiProcessLoaderEntry GetMiProcessLoaderEntry(ULONG64 StartAddress)
{
    if (StartAddress == 0)
    {
        return NULL;
    }

    while (StartAddress < StartAddress + 0x600)
    {
        if (*(UCHAR*)StartAddress == 0xE8 && *(UCHAR*)(StartAddress + 5) == 0x8B &&
            *(UCHAR*)(StartAddress + 6) == 0x05)
        {
            StartAddress++;
            return (MiProcessLoaderEntry)(*(LONG*)StartAddress + StartAddress + 4);
        }
        ++StartAddress;
    }
    return NULL;
}

// 根据驱动名获取驱动对象
BOOLEAN GetDriverObjectByName(PDRIVER_OBJECT *DriverObject, WCHAR *DriverName)
{
    PDRIVER_OBJECT TempObject = NULL;
    UNICODE_STRING u_DriverName = { 0 };
    NTSTATUS Status = STATUS_UNSUCCESSFUL;

    RtlInitUnicodeString(&u_DriverName, DriverName);

```

```

        Status = ObReferenceObjectByName(&u_DriverName, OBJ_CASE_INSENSITIVE, NULL,
0, *IoDriverObjectType, KernelMode, NULL, &TempObject);
        if (!NT_SUCCESS(Status))
        {
            *DriverObject = NULL;
            return FALSE;
        }

        *DriverObject = TempObject;
        return TRUE;
    }

BOOLEAN SupportSEH(PDRIVER_OBJECT DriverObject)
{
    PDRIVER_OBJECT Object = NULL;;
    PLDR_DATA_TABLE_ENTRY LdrEntry = NULL;

    GetDriverObjectByName(&Object, L"\\Driver\\tdx");
    if (Object == NULL)
    {
        return FALSE;
    }

    // 将获取到的驱动对象节点赋值给自身LDR
    LdrEntry = (PLDR_DATA_TABLE_ENTRY)DriverObject->DriverSection;
    LdrEntry->DllBase = Object->DriverStart;
    ObDereferenceObject(Object);
    return TRUE;
}

VOID InitInLoadOrderLinks(PLDR_DATA_TABLE_ENTRY LdrEntry)
{
    InitializeListHead(&LdrEntry->InLoadOrderLinks);
    InitializeListHead(&LdrEntry->InMemoryOrderLinks);
}

VOID Reinitialize(PDRIVER_OBJECT DriverObject, PVOID Context, ULONG Count)
{
    MiProcessLoaderEntry m_MiProcessLoaderEntry = NULL;
    ULONG *p = NULL;

    m_MiProcessLoaderEntry =
GetMiProcessLoaderEntry(MiUnloadSystemImageAddress);
    if (m_MiProcessLoaderEntry == NULL)
    {
        return;
    }

    SupportSEH(DriverObject);

    m_MiProcessLoaderEntry(DriverObject->DriverSection, 0);
    InitInLoadOrderLinks((PLDR_DATA_TABLE_ENTRY)DriverObject->DriverSection);

    // 破坏驱动对象特征
    DriverObject->DriverSection = NULL;

```

```

DriverObject->DriverStart = NULL;
DriverObject->DriverSize = 0;
DriverObject->DriverUnload = NULL;
DriverObject->DriverInit = NULL;
DriverObject->DeviceObject = NULL;

DbgPrint("驱动隐藏 \n");
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

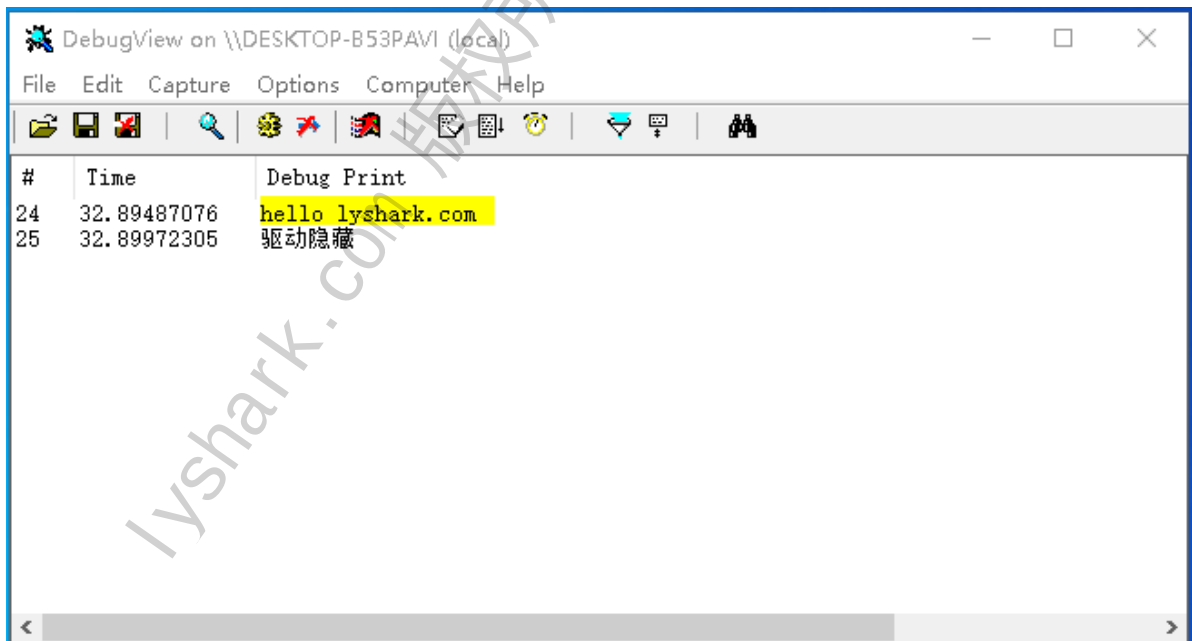
    MiUnloadSystemImageAddress = GetMiUnloadSystemImageAddress();
    MiProcessLoaderEntry MiProcessLoaderEntryAddress =
        GetMiProcessLoaderEntry(MiUnloadSystemImageAddress);

    // 无痕隐藏
    IoRegisterDriverReinitialization(Driver, Reinitialize, NULL);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行驱动程序，让后看到如下输出信息；



6.10 内核强制结束进程运行

通常使用 windows 系统自带的 任务管理器 可以正常地 结束 掉一般 进程，而某些 特殊的 进程在应用层很难被结束掉，例如某些 系统核心进程 其权限是在 0 环 内核态，但有时我们不得不想办法结束掉这些特殊的进程，当然某些正常进程在特殊状态下也会无法被正常结束，此时使用驱动前行在内核态将其结束掉就变得很有用了，驱动结束进程有多种方法。

- 1.标准方法就是使用 `ZwOpenProcess` 打开进程获得句柄, 然后使用 `ZwTerminateProcess` 这个内核API实现结束进程, 最后使用 `ZwClose` 关闭句柄。
- 2.第二种方法, 通过动态定位的方式找到 `PspTerminateThreadByPointer` 这个内核函数地址, 然后调用该函数结束掉进程中所有的线程, 当线程为空则进程也就消亡了。
- 3.第三种方法, 我将其称作是内存清零法, 其核心原理是通过打开进程, 得到进程的基址, 通过内存填充的方式将对端内存全部置0实现类似于结束的效果。

首先是第一种方法结束进程, 封装实现 `KillProcess` 函数, 用户传入 `lyshark.exe` 进程名, 进程内执行 `PsGetProcessImageFileName` 判断是否是我们要结束的如果是则, 调用 `ZwOpenProcess` 打开进程, 并发送 `ZwTerminateProcess` 终止信号从而正常结束, 其核心代码如下所示。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>

NTKERNELAPI UCHAR* PsGetProcessImageFileName(IN PEPROCESS Process);

// 根据进程ID返回进程EPROCESS结构体, 失败返回NULL
PEPROCESS GetProcessNameByProcessId(HANDLE pid)
{
    PEPROCESS ProcessObj = NULL;
    NTSTATUS Status = STATUS_UNSUCCESSFUL;
    Status = PsLookupProcessByProcessId(pid, &ProcessObj);
    if (NT_SUCCESS(Status))
        return ProcessObj;
    return NULL;
}

// 根据ProcessName获取到进程的PID号
HANDLE GetPidByProcessName(char *ProcessName)
{
    PEPROCESS pCurrentEprocess = NULL;
    HANDLE pid = 0;
    for (int i = 0; i < 1000000000; i += 4)
    {
        pCurrentEprocess = GetProcessNameByProcessId((HANDLE)i);
        if (pCurrentEprocess != NULL)
        {
            pid = PsGetProcessId(pCurrentEprocess);
            if (strstr(PsGetProcessImageFileName(pCurrentEprocess), ProcessName)
                != NULL)
            {
                ObDereferenceObject(pCurrentEprocess);
                return pid;
            }
            ObDereferenceObject(pCurrentEprocess);
        }
    }
    return (HANDLE)-1;
}

// 传入进程名称, 终止掉该进程
```

```

BOOLEAN KillProcess(PCHAR ProcessName)
{
    PEPROCESS pCurrentEprocess = NULL;
    HANDLE pid = 0;
    HANDLE Handle = NULL;
    OBJECT_ATTRIBUTES obj;
    CLIENT_ID cid = { 0 };
    NTSTATUS Status = STATUS_UNSUCCESSFUL;

    for (int i = 0; i < 10000000; i += 4)
    {
        pCurrentEprocess = GetProcessNameByProcessId((HANDLE)i);
        if (pCurrentEprocess != NULL)
        {
            pid = PsGetProcessId(pCurrentEprocess);

            // 判断当前镜像名称是否是需结束的进程
            if (strstr(PsGetProcessImageFileName(pCurrentEprocess), ProcessName)
                != NULL)
            {
                ObDereferenceObject(pCurrentEprocess);

                // 找到后开始结束
                InitializeObjectAttributes(&obj, NULL, OBJ_KERNEL_HANDLE |
                OBJ_CASE_INSENSITIVE, NULL, NULL);
                cid.UniqueProcess = (HANDLE)pid;
                cid.UniqueThread = 0;

                // 打开进程
                Status = ZwOpenProcess(&Handle, GENERIC_ALL, &obj, &cid);
                if (NT_SUCCESS(Status))
                {
                    // 发送终止信号
                    ZwTerminateProcess(Handle, 0);
                    ZwClose(Handle);
                }
                ZwClose(Handle);
                return TRUE;
            }
            ObDereferenceObject(pCurrentEprocess);
        }
    }
    return FALSE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    BOOLEAN Retn;

```

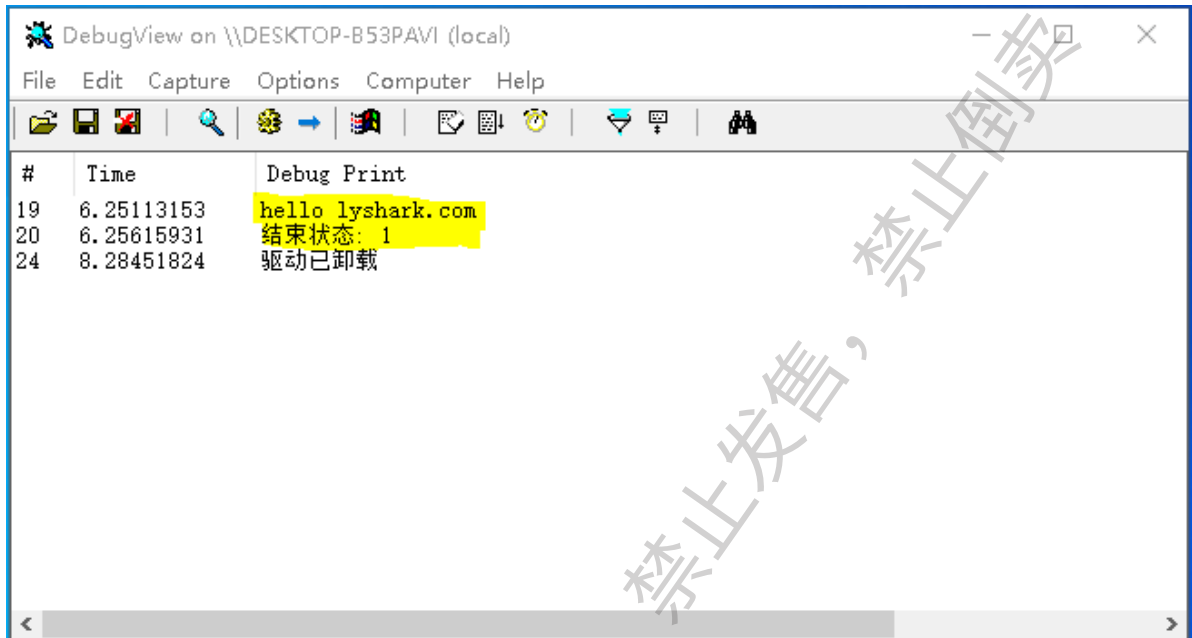
```

Retn = KillProcess("lyshark.exe");
DbgPrint("结束状态: %d \n", Retn);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

我们运行这个驱动，当进程 lyshark.exe 存在时则可以看到结束效果，当然这种方式只是在内核层面调用了结束进程函数，其本质上还是正常结束，只是这种方式权限要大一些仅此而已。



第二种方法，其原理就是将进程内的线程全部结束掉从而让进程自动结束，由于 PspTerminateThreadByPointer 没有被导出，所以我们需要动态的这个内存地址，然后动态调用即可，这个寻找方法可以总结为以下步骤。

1. 寻找 PsTerminateSystemThread 函数地址，这个地址可以直接通过 MmGetSystemRoutineAddress 函数得到。
2. 在 PsTerminateSystemThread 函数地址内向下扫描特征 e80cb6f6ff 得到 call nt!PspTerminateThreadByPointer 地址。

根据《驱动开发：内核枚举LoadImage映像回调》中使用的 SearchMemory 函数实现搜索 PspTerminateThreadByPointer 内存地址。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>

// 得到PspTerminateThreadByPointer内存地址
PVOID PspTerminateThreadByPointer()
{
    UNICODE_STRING ustrFuncName;
    PVOID pAddress = NULL;
    LONG lOffset = 0;
    PVOID pPsTerminateSystemThread = NULL;
    PVOID pPspTerminateThreadByPointer = NULL;
}

```

```

// 获取 PsTerminateSystemThread 函数地址
RtlInitUnicodeString(&ustrFuncName, L"PsTerminateSystemThread");
pPsTerminateSystemThread = MmGetSystemRoutineAddress(&ustrFuncName);

DbgPrint("pPsTerminateSystemThread = 0x%p \n", pPsTerminateSystemThread);
if (NULL == pPsTerminateSystemThread)
{
    return 0;
}

// 查找 PspTerminateThreadByPointer 函数地址

/*
1: kd> uf PsTerminateSystemThread
nt!PsTerminateSystemThread:
fffff802`254e6a90 4883ec28      sub     rsp,28h
fffff802`254e6a94 8bd1          mov     edx,ecx
fffff802`254e6a96 65488b0c2588010000 mov     rcx,qword ptr gs:[188h]
fffff802`254e6a9f f7417400040000 test     dword ptr [rcx+74h],400h
fffff802`254e6aa6 0f8444081100 je
nt!PsTerminateSystemThread+0x110860 (fffff802`255f72f0) Branch

nt!PsTerminateSystemThread+0x1c:
fffff802`254e6aac 41b001        mov     r8b,1
fffff802`254e6aaf e80cb6f6ff    call    nt!PspTerminateThreadByPointer (fffff802`254520c0)

nt!PsTerminateSystemThread+0x24:
fffff802`254e6ab4 4883c428      add     rsp,28h
fffff802`254e6ab8 c3            ret

nt!PsTerminateSystemThread+0x110860:
fffff802`255f72f0 b80d0000c0    mov     eax,0C000000Dh
fffff802`255f72f5 e9baf7eeff    jmp
nt!PsTerminateSystemThread+0x24 (fffff802`254e6ab4) Branch
*/

UCHAR pSpecialData[50] = { 0 };
ULONG ulSpecialDataSize = 0;

// fffff802`254e6aaf e80cb6f6ff    call    nt!PspTerminateThreadByPointer
// (fffff802`254520c0)
pSpecialData[0] = 0xE8;
ulSpecialDataSize = 1;

// 搜索地址 PsTerminateSystemThread --> PsTerminateSystemThread + 0xff 查找
// e80cb6f6ff
pAddress = SearchMemory(pPsTerminateSystemThread, (PVOID)
((PUCHAR)pPsTerminateSystemThread + 0xFF), pSpecialData, ulSpecialDataSize);
if (NULL == pAddress)
{
    return 0;
}

// 先获取偏移,再计算地址

```

```

    lOffset = *(PLONG)pAddress;
    pPspTerminateThreadByPointer = (PVOID)((PUCHAR)pAddress + sizeof(LONG) +
lOffset);
    if (NULL == pPspTerminateThreadByPointer)
    {
        return 0;
    }

    return pPspTerminateThreadByPointer;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

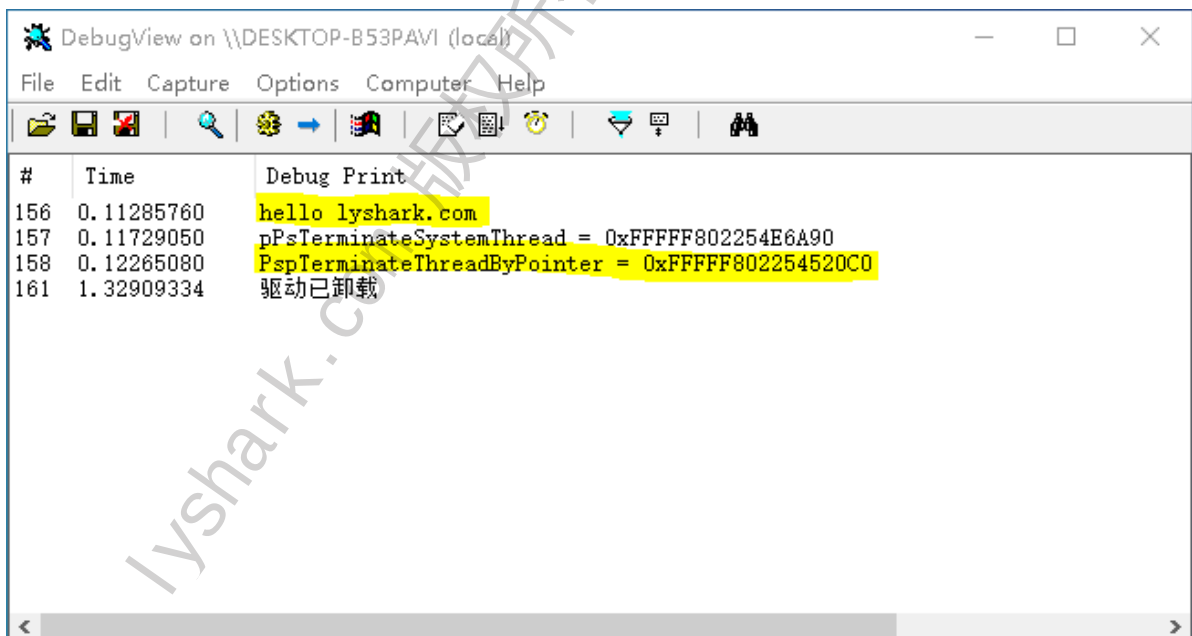
    PVOID address = PspTerminateThreadByPointer();

    DbgPrint("PspTerminateThreadByPointer = 0x%p \n", address);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行驱动程序，首先得到 PspTerminateThreadByPointer 的内存地址，效果如下。



得到内存地址以后直接将地址 typedef 转为指针函数，调用并批量结束进程内的线程即可。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>

```



```

typedef NTSTATUS(__fastcall *PSP_TERMINATE_THREAD_BY_POINTER) (PETHREAD pETHread,
NTSTATUS ntExitCode, BOOLEAN bDirectTerminate);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    PVOID pPspTerminateThreadByPointerAddress = 0xFFFFF802254520C0;
    HANDLE hProcessId = 6956;

    PEPROCESS pEProcess = NULL;
    PETHREAD pETHread = NULL;
    PEPROCESS pThreadEProcess = NULL;
    NTSTATUS status = STATUS_SUCCESS;
    ULONG i = 0;

    // 获取结束进程的进程结构对象EPROCESS
    status = PsLookupProcessByProcessId(hProcessId, &pEProcess);
    if (!NT_SUCCESS(status))
    {
        return status;
    }

    // 遍历所有线程，并结束所有指定进程的线程
    for (i = 4; i < 0x80000; i = i + 4)
    {
        status = PsLookupThreadByThreadId((HANDLE)i, &pETHread);
        if (NT_SUCCESS(status))
        {
            // 获取线程对应的进程结构对象
            pThreadEProcess = PsGetThreadProcess(pETHread);

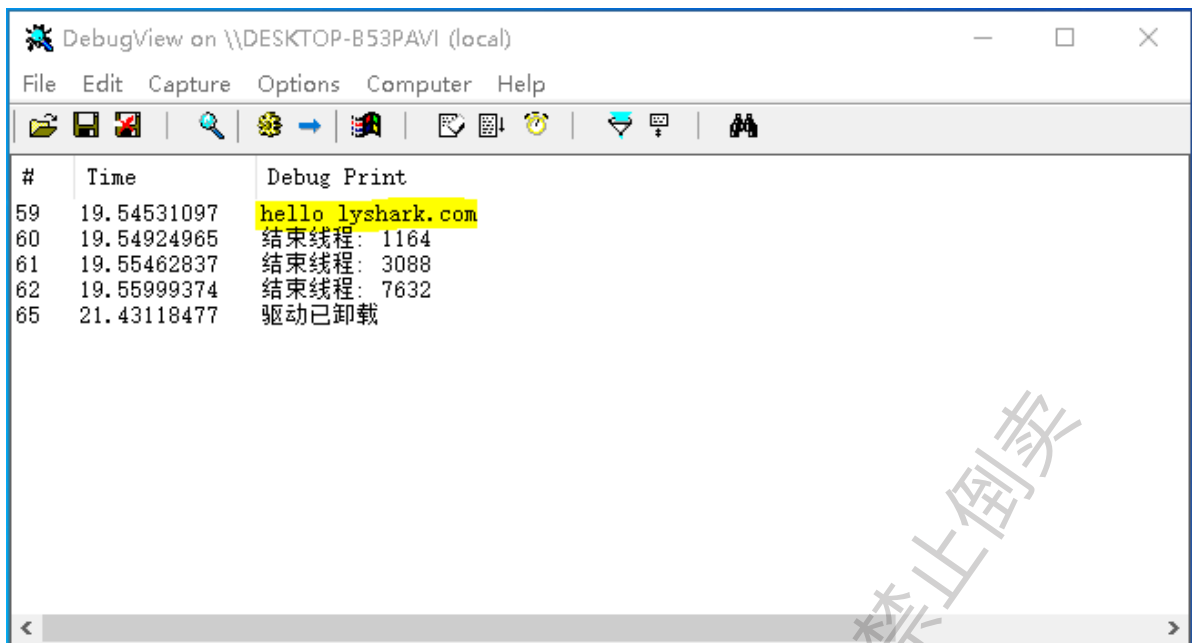
            // 结束进程中的线程
            if (pEProcess == pThreadEProcess)
            {
                ((PSP_TERMINATE_THREAD_BY_POINTER)pPspTerminateThreadByPointerAddress)(pETHread, 0,
                1);

                DbgPrint("结束线程: %d \n", i);
            }
            ObDereferenceObject(pETHread);
        }
    }
    ObDereferenceObject(pEProcess);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

循环结束进程 6956 内的所有线程信息，效果如下；



第七章：内核模块篇

7.1 内核判断驱动加载状态

在驱动开发中我们有时需要得到驱动自身是否被加载成功的状态，这个功能看似没啥用实际上在某些特殊场景中还是需要的，如下代码实现了判断当前驱动是否加载成功，如果加载成功，则输出该驱动的详细路径信息。

该功能实现的核心函数是 `NtQuerySystemInformation` 这是一个微软未公开的函数，也没有文档化，不过我们仍然可以通过动态指针的方式调用到它，该函数可以查询到很多系统信息状态，首先需要定义一个指针。

```
typedef NTSTATUS(*NTQUERYSYSTEMINFORMATION)(  
    IN ULONG SystemInformationClass,  
    OUT PVOID SystemInformation,  
    IN ULONG_PTR SystemInformationLength,  
    OUT PULONG_PTR ReturnLength OPTIONAL);
```

其次还需要一个 `SYSTEM_MODULE_INFORMATION` 该结构内可以得到模块入口信息模块名称等，调用 `NtQuerySystemInformation` 数据会被格式化为 `SYSTEM_MODULE_INFORMATION` 方便调用。

```
typedef struct _SYSTEM_MODULE_INFORMATION {  
    HANDLE Section;  
    PVOID MappedBase;  
    PVOID Base;  
    ULONG Size;  
    ULONG Flags;  
    USHORT LoadOrderIndex;  
    USHORT InitOrderIndex;  
    USHORT LoadCount;  
    USHORT PathLength;  
    CHAR ImageName[256];  
} SYSTEM_MODULE_INFORMATION, *PSYSTEM_MODULE_INFORMATION;
```

最后是 SYSTEM_INFORMATION_CLASS 该结构同样是一个未文档化的结构体，本此代码中需要用到的枚举类型是 SystemModuleInformation 其他类型也放这里后期做参考用。

```
typedef enum _SYSTEM_INFORMATION_CLASS
{
    SystemBasicInformation = 0x0,
    SystemProcessorInformation = 0x1,
    SystemPerformanceInformation = 0x2,
    SystemTimeOfDayInformation = 0x3,
    SystemPathInformation = 0x4,
    SystemProcessInformation = 0x5,
    SystemCallCountInformation = 0x6,
    SystemDeviceInformation = 0x7,
    SystemProcessorPerformanceInformation = 0x8,
    SystemFlagsInformation = 0x9,
    SystemCallTimeInformation = 0xa,
    SystemModuleInformation = 0xb,
    SystemLocksInformation = 0xc,
    SystemStackTraceInformation = 0xd,
    SystemPagedPoolInformation = 0xe,
    SystemNonPagedPoolInformation = 0xf,
    SystemHandleInformation = 0x10,
    SystemObjectInformation = 0x11,
    SystemPageFileInformation = 0x12,
    SystemVdmInstemulInformation = 0x13,
    SystemVdmBopInformation = 0x14,
    SystemFileCacheInformation = 0x15,
    SystemPoolTagInformation = 0x16,
    SystemInterruptInformation = 0x17,
    SystemDpcBehaviorInformation = 0x18,
    SystemFullMemoryInformation = 0x19,
    SystemLoadGdiDriverInformation = 0x1a,
    SystemUnloadGdiDriverInformation = 0x1b,
    SystemTimeAdjustmentInformation = 0x1c,
    SystemSummaryMemoryInformation = 0x1d,
    SystemMirrorMemoryInformation = 0x1e,
    SystemPerformanceTraceInformation = 0x1f,
    SystemObsolete0 = 0x20,
    SystemExceptionInformation = 0x21,
    SystemCrashDumpStateInformation = 0x22,
    SystemKernelDebuggerInformation = 0x23,
    SystemContextSwitchInformation = 0x24,
    SystemRegistryQuotaInformation = 0x25,
    SystemExtendServiceTableInformation = 0x26,
    SystemPrioritySeperation = 0x27,
    SystemVerifierAddDriverInformation = 0x28,
    SystemVerifierRemoveDriverInformation = 0x29,
    SystemProcessorIdleInformation = 0x2a,
    SystemLegacyDriverInformation = 0x2b,
    SystemCurrentTimeZoneInformation = 0x2c,
    SystemLookasideInformation = 0x2d,
    SystemTimeslipNotification = 0x2e,
    SystemSessionCreate = 0x2f,
    SystemSessionDetach = 0x30,
    SystemSessionInformation = 0x31,
```

SystemRangeStartInformation = 0x32,
SystemVerifierInformation = 0x33,
SystemVerifierThunkExtend = 0x34,
SystemSessionProcessInformation = 0x35,
SystemLoadGdiDriverInSystemSpace = 0x36,
SystemNumaProcessorMap = 0x37,
SystemPrefetcherInformation = 0x38,
SystemExtendedProcessInformation = 0x39,
SystemRecommendedSharedDataAlignment = 0x3a,
SystemComPlusPackage = 0x3b,
SystemNumaAvailableMemory = 0x3c,
SystemProcessorPowerInformation = 0x3d,
SystemEmulationBasicInformation = 0x3e,
SystemEmulationProcessorInformation = 0x3f,
SystemExtendedHandleInformation = 0x40,
SystemLostDelayedWriteInformation = 0x41,
SystemBigPoolInformation = 0x42,
SystemSessionPoolTagInformation = 0x43,
SystemSessionMappedViewInformation = 0x44,
SystemHotpatchInformation = 0x45,
SystemObjectSecurityMode = 0x46,
SystemWatchdogTimerHandler = 0x47,
SystemWatchdogTimerInformation = 0x48,
SystemLogicalProcessorInformation = 0x49,
SystemWow64SharedInformationObsolete = 0x4a,
SystemRegisterFirmwareTableInformationHandler = 0x4b,
SystemFirmwareTableInformation = 0x4c,
SystemModuleInformationEx = 0x4d,
SystemVerifierTriageInformation = 0x4e,
SystemSuperfetchInformation = 0x4f,
SystemMemoryListInformation = 0x50,
SystemFileCacheInformationEx = 0x51,
SystemThreadPriorityClientIdInformation = 0x52,
SystemProcessorIdleCycleTimeInformation = 0x53,
SystemVerifierCancellationInformation = 0x54,
SystemProcessorPowerInformationEx = 0x55,
SystemRefTraceInformation = 0x56,
SystemSpecialPoolInformation = 0x57,
SystemProcessIdInformation = 0x58,
SystemErrorPortInformation = 0x59,
SystemBootEnvironmentInformation = 0x5a,
SystemHypervisorInformation = 0x5b,
SystemVerifierInformationEx = 0x5c,
SystemTimeZoneInformation = 0x5d,
SystemImageFileExecutionOptionsInformation = 0x5e,
SystemCoverageInformation = 0x5f,
SystemPrefetchPatchInformation = 0x60,
SystemVerifierFaultsInformation = 0x61,
SystemSystemPartitionInformation = 0x62,
SystemSystemDiskInformation = 0x63,
SystemProcessorPerformanceDistribution = 0x64,
SystemNumaProximityNodeInformation = 0x65,
SystemDynamicTimeZoneInformation = 0x66,
SystemCodeIntegrityInformation = 0x67,
SystemProcessorMicrocodeUpdateInformation = 0x68,

```

SystemProcessorBrandString = 0x69,
SystemVirtualAddressInformation = 0x6a,
SystemLogicalProcessorAndGroupInformation = 0x6b,
SystemProcessorCycleTimeInformation = 0x6c,
SystemStoreInformation = 0x6d,
SystemRegistryAppendString = 0x6e,
SystemAitSamplingValue = 0x6f,
SystemVhdBootInformation = 0x70,
SystemCpuQuotaInformation = 0x71,
SystemNativeBasicInformation = 0x72,
SystemErrorPortTimeouts = 0x73,
SystemLowPriorityIoInformation = 0x74,
SystemBootEntropyInformation = 0x75,
SystemVerifierCountersInformation = 0x76,
SystemPagedPoolInformationEx = 0x77,
SystemSystemPtesInformationEx = 0x78,
SystemNodeDistanceInformation = 0x79,
SystemAcpiAuditInformation = 0x7a,
SystemBasicPerformanceInformation = 0x7b,
SystemQueryPerformanceCounterInformation = 0x7c,
SystemSessionBigPoolInformation = 0x7d,
SystemBootGraphicsInformation = 0x7e,
SystemScrubPhysicalMemoryInformation = 0x7f,
SystemBadPageInformation = 0x80,
SystemProcessorProfileControlArea = 0x81,
SystemCombinePhysicalMemoryInformation = 0x82,
SystemEntropyInterruptTimingInformation = 0x83,
SystemConsoleInformation = 0x84,
SystemPlatformBinaryInformation = 0x85,
SystemThrottleNotificationInformation = 0x86,
SystemHypervisorProcessorCountInformation = 0x87,
SystemDeviceDataInformation = 0x88,
SystemDeviceDataEnumerationInformation = 0x89,
SystemMemoryTopologyInformation = 0x8a,
SystemMemoryChannelInformation = 0x8b,
SystemBootLogoInformation = 0x8c,
SystemProcessorPerformanceInformationEx = 0x8d,
SystemSpare0 = 0x8e,
SystemSecureBootPolicyInformation = 0x8f,
SystemPageFileInformationEx = 0x90,
SystemSecureBootInformation = 0x91,
SystemEntropyInterruptTimingRawInformation = 0x92,
SystemPortableWorkspaceEfiLauncherInformation = 0x93,
SystemFullProcessInformation = 0x94,
SystemKernelDebuggerInformationEx = 0x95,
SystemBootMetadataInformation = 0x96,
SystemSoftRebootInformation = 0x97,
SystemElamCertificateInformation = 0x98,
SystemOfflineDumpConfigInformation = 0x99,
SystemProcessorFeaturesInformation = 0x9a,
SystemRegistryReconciliationInformation = 0x9b,
MaxSystemInfoClass = 0x9c,
} SYSTEM_INFORMATION_CLASS;

```

最后的 `JudgeLoadDriver()` 是核心函数，我们看下该函数具体是如何实现的，原理很简单。

- 1.通过MmGetSystemRoutineAddress得到动态的地址。
- 2.动态调用m_NtQuerySystemInformation得到参数。
- 3.判断自身是否被加载，如果是输出路径。

```
#include <ntifs.h>
#include <windef.h>
#include <stdlib.h>

typedef NTSTATUS(*NTQUERYSYSTEMINFORMATION)(
IN ULONG SystemInformationClass,
OUT PVOID SystemInformation,
IN ULONG_PTR SystemInformationLength,
OUT PULONG_PTR ReturnLength OPTIONAL);

typedef struct _SYSTEM_MODULE_INFORMATION {
HANDLE Section;
PVOID MappedBase;
PVOID Base;
ULONG Size;
ULONG Flags;
USHORT LoadOrderIndex;
USHORT InitOrderIndex;
USHORT LoadCount;
USHORT PathLength;
CHAR ImageName[256];
} SYSTEM_MODULE_INFORMATION, *PSYSTEM_MODULE_INFORMATION;

typedef enum _SYSTEM_INFORMATION_CLASS
{
SystemBasicInformation = 0x0,
SystemProcessorInformation = 0x1,
SystemPerformanceInformation = 0x2,
SystemTimeOfDayInformation = 0x3,
SystemPathInformation = 0x4,
SystemProcessInformation = 0x5,
SystemCallCountInformation = 0x6,
SystemDeviceInformation = 0x7,
SystemProcessorPerformanceInformation = 0x8,
SystemFlagsInformation = 0x9,
SystemCallTimeInformation = 0xa,
SystemModuleInformation = 0xb,
SystemLocksInformation = 0xc,
} SYSTEM_INFORMATION_CLASS;

// 判断当前Driver是否加载成功
// By: LyShark
ULONG JudgeLoadDriver()
{
NTQUERYSYSTEMINFORMATION m_NtQuerySystemInformation = NULL;
UNICODE_STRING NtQuerySystemInformation_Name;
PSYSTEM_MODULE_INFORMATION ModuleEntry;
ULONG_PTR RetLength, BaseAddr, EndAddr;
ULONG ModuleNumbers, Index;
NTSTATUS Status;
PVOID Buffer;
```

```

    RtlInitUnicodeString(&NtQuerySystemInformation_Name,
L"NtQuerySystemInformation");
    m_NtQuerySystemInformation =
(NTQUERYSYSTEMINFORMATION)MmGetSystemRoutineAddress(&NtQuerySystemInformation_Na
me);
    if (m_NtQuerySystemInformation == NULL)
    {
        DbgPrint("获取NtQuerySystemInformation函数失败! \n");
        return 1;
    }

    RetLength = 0;
    Status = m_NtQuerySystemInformation(SystemModuleInformation, NULL, 0,
&RetLength);
    if (Status < 0 && Status != STATUS_INFO_LENGTH_MISMATCH)
    {
        DbgPrint("NtQuerySystemInformation调用失败! 错误码是: %x\n", Status);
        return 1;
    }

    Buffer = ExAllocatePoolWithTag(NonPagedPool, RetLength, 'lysh');
    if (Buffer == NULL)
    {
        DbgPrint("分配内存失败! \n");
        return 1;
    }

    Status = m_NtQuerySystemInformation(SystemModuleInformation, Buffer,
RetLength, &RetLength);
    if (Status < 0)
    {
        DbgPrint("NtQuerySystemInformation调用失败 %x\n", Status);
        return 1;
    }

    ModuleNumbers = *(ULONG*)Buffer;
    ModuleEntry = (PSYSTEM_MODULE_INFORMATION)((ULONG_PTR)Buffer + 8);
    for (Index = 0; Index < ModuleNumbers; ++Index)
    {
        BaseAddr = (ULONG_PTR)ModuleEntry->Base;
        EndAddr = BaseAddr + ModuleEntry->Size;
        if (BaseAddr <= (ULONG_PTR)JudgeLoadDriver && (ULONG_PTR)JudgeLoadDriver
<= EndAddr)
        {
            DbgPrint("模块名称是: %s\n", ModuleEntry->ImageName);
            return 2;
        }
        ++ModuleEntry;
    }

    return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{

```



```

    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

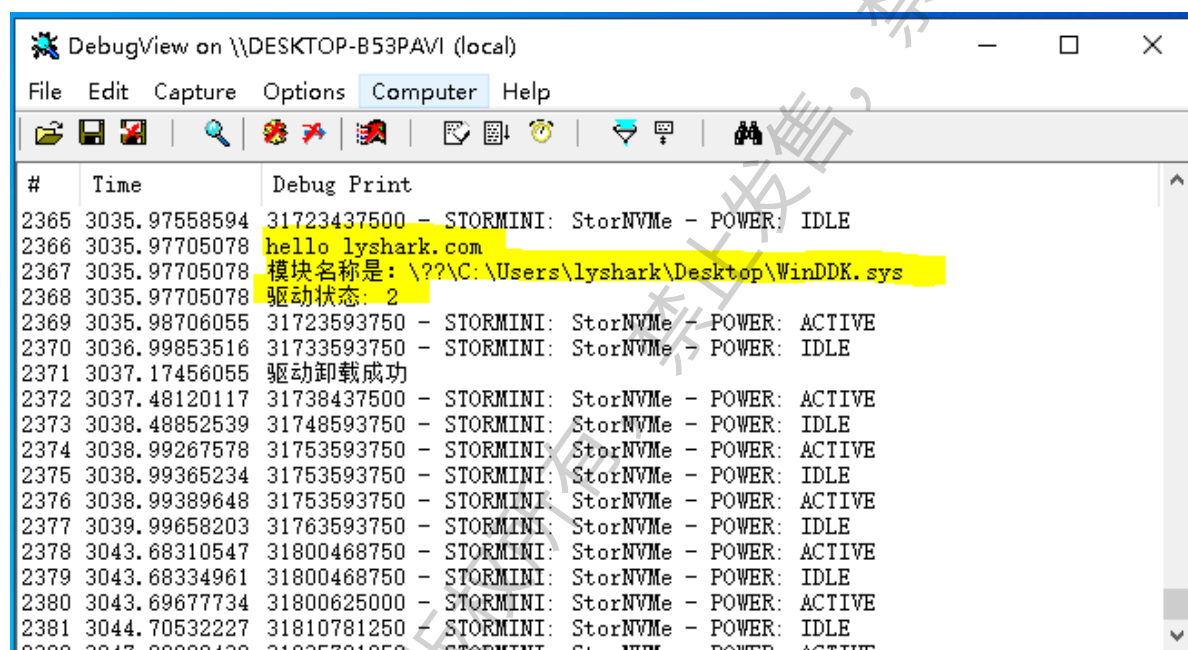
    ULONG ul = JudgeLoadDriver();

    DbgPrint("驱动状态: %d \n", ul);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

代码运行效果如下所示:



7.2 内核取ntoskrnl模块基地址

模块是程序加载时被动态装载的，模块在装载后其存在于内存中同样存在一个内存基址，当我们需要操作这个模块时，通常第一步就是要得到该模块的内存基址，模块分为用户模块和内核模块，这里的用户模块指的是应用层进程运行后加载的模块，内核模块指的是内核中特定模块地址，本篇文章将实现一个获取驱动 ntoskrnl.exe 的基地址以及长度，此功能是驱动开发中尤其是安全软件开发中必不可少的一个功能。

关于该程序的解释，官方的解析是这样的 ntoskrnl.exe 是 windows 操作系统的一个重要内核程序，里面存储了大量的二进制内核代码，用于调度系统时使用，也是操作系统启动后第一个被加载的程序，通常该进程在任务管理器中显示为 system。

使用ARK工具也可看出其代表的是第一个驱动模块。

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
驱动名	基地址	大小	驱动对象	驱动路径								
ntoskrnl.exe	0xFFFFF8051B000000	0x00AB6000	-	C:\Windows\system32\ntoskrnl.exe								
hal.dll	0xFFFFF8051AF5D000	0x000A3000	-	C:\Windows\system32\hal.dll								
kd.dll	0xFFFFF8051C000000	0x0000B000	-	C:\Windows\system32\kd.dll								
mcupdate_GenuineI...	0xFFFFF8051C010000	0x00201000	-	C:\Windows\system32\mcupdate_GenuineIntel.dll								
msrpc.sys	0xFFFFF8051C270000	0x00060000	-	C:\Windows\System32\drivers\msrpc.sys								
ksecdd.sys	0xFFFFF8051C240000	0x0002A000	0xFFFFCC0544...	C:\Windows\System32\drivers\ksecdd.sys								
werkernel.sys	0xFFFFF8051C220000	0x00011000	-	C:\Windows\System32\drivers\werkernel.sys								

那么如何使用代码得到如上图所展示的 **基地址** 以及 **大小** 呢，实现此功能我们需要调用

ZwQuerySystemInformation 这个API函数，这与上一篇文章《驱动开发：判断自身是否加载成功》所使用的 **NtQuerySystemInformation** 只是开头部分不同，但其本质上是不同的，如下是一些参考资料；

- 从内核模式调用 **Nt** 和 **Zw** 系列API，其最终都会连接到 **nooskrnl.lib** 导出库：
 - **Nt**系列API将直接调用对应的函数代码，而**Zw**系列API则通过调用 **KiSystemService** 最终跳转到对应的函数代码。
 - 重要的是两种不同的调用对内核中 **previous mode** 的改变，如果是从用户模式调用 **Native API** 则 **previous mode** 是用户态，如果从内核模式调用 **Native API** 则 **previous mode** 是内核态。
 - 如果 **previous** 为用户态时 **Native API** 将对传递的参数进行严格的检查，而为内核态时则不会检查。

调用 **Nt API** 时不会改变 **previous mode** 的状态，调用 **Zw API** 时会将 **previous mode** 改为内核态，因此在进行 **Kernel Mode Driver** 开发时可以使用 **Zw** 系列API可以避免额外的参数列表检查，提高效率。

Zw* 会设置 **KernelMode** 已避免检查，**Nt*** 不会自动设置，如果是 **KernelMode** 当然没问题，如果就 **UserMode** 就挂了。

回到代码上来，下方代码就是获取 **ntoskrnl.exe** 基地址以及长度的具体实现，核心代码就是调用 **ZwQuerySystemInformation** 得到 **SystemModuleInformation**，里面的对比部分是在比较当前获取的地址是否超出了 **ntoskrnl** 的最大和最小范围。

```
#include <ntifs.h>

static PVOID g_KernelBase = 0;
static ULONG g_KernelSize = 0;

#pragma pack(4)
typedef struct _PEB32
{
    UCHAR InheritedAddressSpace;
    UCHAR ReadImageFileExecOptions;
    UCHAR BeingDebugged;
    UCHAR BitField;
    ULONG Mutant;
    ULONG ImageBaseAddress;
    ULONG Ldr;
    ULONG ProcessParameters;
    ULONG SubSystemData;
    ULONG ProcessHeap;
    ULONG FastPebLock;
    ULONG AtlThunkSLISTPtr;
    ULONG IFEOKey;
    ULONG CrossProcessFlags;
    ULONG UserSharedInfoPtr;
```

```

        ULONG SystemReserved;
        ULONG AtlThunkSListPtr32;
        ULONG ApiSetMap;
    } PEB32, *PPEB32;

typedef struct _PEB_LDR_DATA32
{
    ULONG Length;
    UCHAR Initialized;
    ULONG SsHandle;
    LIST_ENTRY32 InLoadOrderModuleList;
    LIST_ENTRY32 InMemoryOrderModuleList;
    LIST_ENTRY32 InInitializationOrderModuleList;
} PEB_LDR_DATA32, *PPEB_LDR_DATA32;

typedef struct _LDR_DATA_TABLE_ENTRY32
{
    LIST_ENTRY32 InLoadOrderLinks;
    LIST_ENTRY32 InMemoryOrderLinks;
    LIST_ENTRY32 InInitializationOrderLinks;
    ULONG DllBase;
    ULONG EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING32 FullDllName;
    UNICODE_STRING32 BaseDllName;
    ULONG Flags;
    USHORT LoadCount;
    USHORT TlsIndex;
    LIST_ENTRY32 HashLinks;
    ULONG TimeDateStamp;
} LDR_DATA_TABLE_ENTRY32, *PLDR_DATA_TABLE_ENTRY32;
#pragma pack()

typedef struct _RTL_PROCESS_MODULE_INFORMATION
{
    HANDLE Section;
    PVOID MappedBase;
    PVOID ImageBase;
    ULONG ImageSize;
    ULONG Flags;
    USHORT LoadOrderIndex;
    USHORT InitOrderIndex;
    USHORT LoadCount;
    USHORT OffsetToFileName;
    UCHAR FullPathName[256];
} RTL_PROCESS_MODULE_INFORMATION, *PRTL_PROCESS_MODULE_INFORMATION;

typedef struct _RTL_PROCESS_MODULES
{
    ULONG NumberOfModules;
    RTL_PROCESS_MODULE_INFORMATION Modules[1];
} RTL_PROCESS_MODULES, *PRTL_PROCESS_MODULES;

typedef enum _SYSTEM_INFORMATION_CLASS
{

```

```

        SystemModuleInformation = 0xb,
    } SYSTEM_INFORMATION_CLASS;

// 取出kernelBase基地址
// By: lyshark.com
PVOID UtilKernelBase(OUT PULONG pSize)
{
    NTSTATUS status = STATUS_SUCCESS;
    ULONG bytes = 0;
    PRTL_PROCESS_MODULES pMods = 0;
    PVOID checkPtr = 0;
    UNICODE_STRING routineName;

    if (g_KernelBase != 0)
    {
        if (pSize)
            *pSize = g_KernelSize;
        return g_KernelBase;
    }

    RtlInitUnicodeString(&routineName, L"NtOpenFile");

    checkPtr = MmGetSystemRoutineAddress(&routineName);
    if (checkPtr == 0)
        return 0;

    __try
    {
        status = ZwQuerySystemInformation(SystemModuleInformation, 0, bytes,
&bytes);
        if (bytes == 0)
        {
            DbgPrint("Invalid SystemModuleInformation size\n");
            return 0;
        }

        pMods = (RTL_PROCESS_MODULES)ExAllocatePoolWithTag(NonPagedPoolNx,
bytes, "lyshark");
        RtlZeroMemory(pMods, bytes);

        status = ZwQuerySystemInformation(SystemModuleInformation, pMods, bytes,
&bytes);

        if (NT_SUCCESS(status))
        {
            PRTL_PROCESS_MODULE_INFORMATION pMod = pMods->Modules;

            for (ULONG i = 0; i < pMods->NumberOfModules; i++)
            {
                if (checkPtr >= pMod[i].ImageBase &&
                    checkPtr < (PVOID)((PUCHAR)pMod[i].ImageBase +
pMod[i].ImageSize))
                {
                    g_KernelBase = pMod[i].ImageBase;
                    g_KernelSize = pMod[i].ImageSize;
                }
            }
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        return 0;
    }
}

```

```

        if (pSize)
        {
            *pSize = g_KernelSize;
            break;
        }
    }
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
    return 0;
}

if (pMods)
    ExFreePoolWithTag(pMods, "lyshark");
return g_KernelBase;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    PULONG ulong = 0;
    UtilKernelBase(ulong);
    DbgPrint(("ntoskrnl.exe 模块基址: 0x%p \n", g_KernelBase);
    DbgPrint(("模块大小: 0x%p \n", g_KernelSize);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

我们编译并运行上方代码，效果如下：

#	Time	Debug Print
77	1864.27575684	21525937500 - STORMINI: StorNVMe - POWER: ACTIVE
78	1864.75048828	hello lyshark
79	1864.75073242	ntoskrnl.exe 模块基址: 0xFFFFF8051B000000
80	1864.75073242	模块大小: 0x0000000000AB6000
81	1865.28112793	21536093750 - STORMINI: StorNVMe - POWER: IDLE
82	1865.74230957	21540625000 - STORMINI: StorNVMe - POWER: ACTIVE
83	1866.74218750	21550625000 - STORMINI: StorNVMe - POWER: IDLE
84	1868.27038574	21565937500 - STORMINI: StorNVMe - POWER: ACTIVE
85	1868.27075195	21565937500 - STORMINI: StorNVMe - POWER: IDLE
86	1868.27099609	21565937500 - STORMINI: StorNVMe - POWER: ACTIVE
87	1869.27331543	21575937500 - STORMINI: StorNVMe - POWER: IDLE
88	1869.28637695	21576093750 - STORMINI: StorNVMe - POWER: ACTIVE
89	1870.29479980	21586250000 - STORMINI: StorNVMe - POWER: IDLE
90	1874.28869629	21626093750 - STORMINI: StorNVMe - POWER: ACTIVE
91	1874.29003906	21626093750 - STORMINI: StorNVMe - POWER: IDLE

7.3 内核取应用层模块基地址

在上一篇文章《驱动开发：内核取ntoskrnl模块基地址》中我们通过调用内核API函数获取到了内核进程ntoskrnl.exe的基址，当在某些场景中，我们不仅需要得到内核的基地址，也需要得到特定进程内某个模块的基地址，显然上篇文章中的方法是做不到的，本篇文章将实现内核层读取32位应用层中特定进程模块基址功能。

上一篇文章中的PPEB32, PLIST_ENTRY32等结构体定义依然需要保留，此处只保留核心代码，定义部分请看前一篇文章，自定义读取模块基址核心代码如下，调用GetModuleBaseWow64()用户需传入进程的PROCESS结构该结构可通过内核函数PsLookupProcessByProcessId获取到。

对于函数内部执行过程如下：

- 1.根据传入的EPROCESS结构调用KeStackAttachProcess附加到该进程内。
- 2.调用内核函数PsGetProcessWow64Process此函数可得到该进程空间内PEB结构数据。
- 3.通过for循环遍历整个pPeb->Ldr链表，并在遍历过程中通过RtlEqualUnicodeString判断是否是我们需要的模块。
- 4.如果判断是我们需要取出的模块名，则将LdrEntry->DllBase取出，此处取出的基地址也即是我们所需要的。
- 5.比较结束后，通过调用KeUnstackDetachProcess这个内核模块脱离进程空间。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
ULONGLONG GetModuleBaseWow64(_In_ PEPROCESS pEProcess, _In_ UNICODE_STRING
usModuleName)
{
    ULONGLONG BaseAddr = 0;
    KAPC_STATE KAPC = { 0 };
    KeStackAttachProcess(pEProcess, &KAPC);
    PPEB32 pPeb = (PPEB32)PsGetProcessWow64Process(pEProcess);
    if (pPeb == NULL || pPeb->Ldr == 0)
    {
        KeUnstackDetachProcess(&KAPC);
        return 0;
    }

    for (PLIST_ENTRY32 pListEntry = (PLIST_ENTRY32)((PPEB_LDR_DATA32)pPeb->Ldr)-
>InLoadOrderModuleList.Flink;
        pListEntry != &((PPEB_LDR_DATA32)pPeb->Ldr)->InLoadOrderModuleList;
        pListEntry = (PLIST_ENTRY32)pListEntry->Flink)
    {
        PLDR_DATA_TABLE_ENTRY32 LdrEntry = CONTAINING_RECORD(pListEntry,
LDR_DATA_TABLE_ENTRY32, InLoadOrderLinks);

        if (LdrEntry->BaseDllName.Buffer == NULL)
        {
            continue;
        }

        // 当前模块名链表
        UNICODE_STRING usCurrentName = { 0 };
```

```

        RtlInitUnicodeString(&usCurrentName, (PWCHAR)LdrEntry->
BaseDllName.Buffer);

        // 比较模块名是否一致
        if (RtlEqualUnicodeString(&usModuleName, &usCurrentName, TRUE))
        {
            BaseAddr = (ULONGLONG)LdrEntry->DllBase;
            KeUnstackDetachProcess(&KAPC);
            return BaseAddr;
        }
    }
    KeUnstackDetachProcess(&KAPC);
    return 0;
}

```

如上就是如何得到特定模块基址的方法，如下是入口函数的调用方法，首先通过传入 6164 这个PID号，得到进程 EProcess 结构，其次使用 RtlInitUnicodeString(&unicode, wchar_string) 初始化得到 kernel32.dll 字符串，最终调用 GetModuleBaseWow64 函数获取到进程 6164 中 kernel32.dll 的模块基址信息。

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    PEPROCESS pEProcess;
    HANDLE PID = (HANDLE)6164;

    // 初始化字符串
    UNICODE_STRING unicode;
    wchar_t *wchar_string = L"kernel32.dll";
    RtlInitUnicodeString(&unicode, wchar_string);

    // 取模块句柄
    PsLookupProcessByProcessId((HANDLE)PID, &pEProcess);
    ULONGLONG base32 = GetModuleBaseWow64(pEProcess, unicode);

    DbgPrint("ModuleBaseAddress: 0x%X \n", base32);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

这段代码输出效果如下所示:

#	Time	Debug Print
118	85.74868774	hello lyshark
119	85.74869537	ModuleBaseAddress: 0x76AD0000
120	86.74810028	4154375000 - STORMINI: StorNVMe - POWER: IDLE
121	86.92612457	驱动卸载成功
122	87.08043671	4157656250 - STORMINI: StorNVMe - POWER: ACTIVE
123	88.08354187	4167812500 - STORMINI: StorNVMe - POWER: IDLE
124	91.38561249	4200781250 - STORMINI: StorNVMe - POWER: ACTIVE
125	91.38615417	4200781250 - STORMINI: StorNVMe - POWER: IDLE
126	92.41188049	4211093750 - STORMINI: StorNVMe - POWER: ACTIVE
127	92.41244507	4211093750 - STORMINI: StorNVMe - POWER: IDLE
128	92.41275024	4211093750 - STORMINI: StorNVMe - POWER: ACTIVE
129	93.41336823	4221093750 - STORMINI: StorNVMe - POWER: IDLE
130	96.39055634	4250781250 - STORMINI: StorNVMe - POWER: ACTIVE
131	96.39131165	4250781250 - STORMINI: StorNVMe - POWER: IDLE
132	96.39143372	4250781250 - STORMINI: StorNVMe - POWER: ACTIVE

7.4 内核通过PEB得到进程参数

PEB结构 (Process Enviornment Block Structure) 其中文名是进程环境块信息，进程环境块内部包含了进程运行的详细参数信息，每一个进程在运行后都会存在一个特有的PEB结构，通过附加进程并遍历这段结构即可得到非常多的有用信息。

在应用层下，如果想要得到PEB的基地址只需要取 `fs:[0x30]` 即可，TEB线程环境块则是 `fs:[0x18]`，如果在内核层想要得到应用层进程的PEB信息我们需要调用特定的内核函数来获取，如下案例将教大家如何在内核层取到应用层进程的PEB结构。

首先在开始写代码之前需要先定义好 PEB 进程环境块结构体，用于对内存指针解析，新建 `peb.h` 文件并保存如下代码，这些是微软的结构体定义分为32位与64位，官方定义规范而已不需要费工夫。

```
#pragma once
#include <ntifs.h>

typedef struct _CURDIR // 2 elements, 0x18 bytes (sizeof)
{
    /*0x000*/ struct _UNICODE_STRING DosPath; // 3 elements, 0x10 bytes
    (sizeof)
    /*0x010*/ VOID* Handle;
}CURDIR, *PCURDIR;

typedef struct _RTL_DRIVE_LETTER_CURDIR // 4 elements, 0x18 bytes (sizeof)
{
    /*0x000*/ UINT16 Flags;
    /*0x002*/ UINT16 Length;
    /*0x004*/ ULONG32 TimeStamp;
    /*0x008*/ struct _STRING DosPath; // 3 elements, 0x10 bytes
    (sizeof)
}RTL_DRIVE_LETTER_CURDIR, *PRTL_DRIVE_LETTER_CURDIR;

typedef enum _SYSTEM_DLL_TYPE // 7 elements, 0x4 bytes
{
    PsNativeSystemDll = 0 /*0x0*/,
    PsWowX86SystemDll = 1 /*0x1*/,
    PsWowArm32SystemDll = 2 /*0x2*/,
    PsWowAmd64SystemDll = 3 /*0x3*/,
    PsWowChpx86SystemDll = 4 /*0x4*/,
    PsVsmEnclaveRuntimeDll = 5 /*0x5/>,
```



```

    PsSystemDllTotalTypes = 6 /*0x6*/
}SYSTEM_DLL_TYPE, *PSYSTEM_DLL_TYPE;

typedef struct _EWOW64PROCESS // 3 elements, 0x10 bytes (sizeof)
{
    /*0x000*/    VOID*        Peb;
    /*0x008*/    UINT16       Machine;
    /*0x00A*/    UINT8        _PADDING0_[0x2];
    /*0x00C*/    enum _SYSTEM_DLL_TYPE NtdllType;
}EWOW64PROCESS, *PEWOW64PROCESS;

typedef struct _RTL_USER_PROCESS_PARAMETERS // 37 elements, 0x440
bytes (sizeof)
{
    /*0x000*/    ULONG32      MaximumLength;
    /*0x004*/    ULONG32      Length;
    /*0x008*/    ULONG32      Flags;
    /*0x00C*/    ULONG32      DebugFlags;
    /*0x010*/    VOID*        ConsoleHandle;
    /*0x018*/    ULONG32      ConsoleFlags;
    /*0x01C*/    UINT8        _PADDING0_[0x4];
    /*0x020*/    VOID*        StandardInput;
    /*0x028*/    VOID*        StandardOutput;
    /*0x030*/    VOID*        StandardError;
    /*0x038*/    struct _CURDIR CurrentDirectory; // 2
elements, 0x18 bytes (sizeof)
    /*0x050*/    struct _UNICODE_STRING DllPath; // 3
elements, 0x10 bytes (sizeof)
    /*0x060*/    struct _UNICODE_STRING ImagePathName; // 3
elements, 0x10 bytes (sizeof)
    /*0x070*/    struct _UNICODE_STRING CommandLine; // 3
elements, 0x10 bytes (sizeof)
    /*0x080*/    VOID*        Environment;
    /*0x088*/    ULONG32      StartingX;
    /*0x08C*/    ULONG32      StartingY;
    /*0x090*/    ULONG32      CountX;
    /*0x094*/    ULONG32      CountY;
    /*0x098*/    ULONG32      CountCharsX;
    /*0x09C*/    ULONG32      CountCharsY;
    /*0x0A0*/    ULONG32      FillAttribute;
    /*0x0A4*/    ULONG32      WindowFlags;
    /*0x0A8*/    ULONG32      ShowWindowFlags;
    /*0x0AC*/    UINT8        _PADDING1_[0x4];
    /*0x0B0*/    struct _UNICODE_STRING WindowTitle; // 3
elements, 0x10 bytes (sizeof)
    /*0x0C0*/    struct _UNICODE_STRING DesktopInfo; // 3
elements, 0x10 bytes (sizeof)
    /*0x0D0*/    struct _UNICODE_STRING ShellInfo; // 3
elements, 0x10 bytes (sizeof)
    /*0x0E0*/    struct _UNICODE_STRING RuntimeData; // 3
elements, 0x10 bytes (sizeof)
    /*0x0F0*/    struct _RTL_DRIVE_LETTER_CURDIR CurrentDirectores[32];
    /*0x3F0*/    UINT64       EnvironmentSize;
    /*0x3F8*/    UINT64       EnvironmentVersion;
    /*0x400*/    VOID*        PackageDependencyData;

```



```

/*0x408*/    ULONG32    ProcessGroupId;
/*0x40C*/    ULONG32    LoaderThreads;
/*0x410*/    struct _UNICODE_STRING RedirectionDllName;           // 3
elements, 0x10 bytes (sizeof)
/*0x420*/    struct _UNICODE_STRING HeapPartitionName;         // 3
elements, 0x10 bytes (sizeof)
/*0x430*/    UINT64*    DefaultThreadPoolCpuSetMasks;
/*0x438*/    ULONG32    DefaultThreadPoolCpuSetMaskCount;
/*0x43C*/    UINT8      _PADDING2_[0x4];
}RTL_USER_PROCESS_PARAMETERS, *PRTL_USER_PROCESS_PARAMETERS;

typedef struct _PEB_LDR_DATA                                // 9 elements, 0x58
bytes (sizeof)
{
    /*0x000*/    ULONG32    Length;
    /*0x004*/    UINT8      Initialized;
    /*0x005*/    UINT8      _PADDING0_[0x3];
    /*0x008*/    VOID*      SsHandle;
    /*0x010*/    struct _LIST_ENTRY InLoadOrderModuleList;       // 2
elements, 0x10 bytes (sizeof)
    /*0x020*/    struct _LIST_ENTRY InMemoryOrderModuleList;    // 2
elements, 0x10 bytes (sizeof)
    /*0x030*/    struct _LIST_ENTRY InInitializationOrderModuleList; // 2
elements, 0x10 bytes (sizeof)
    /*0x040*/    VOID*      EntryInProgress;
    /*0x048*/    UINT8      ShutdownInProgress;
    /*0x049*/    UINT8      _PADDING1_[0x7];
    /*0x050*/    VOID*      ShutdownThreadId;
}PEB_LDR_DATA, *PPEB_LDR_DATA;

typedef struct _PEB64
{
    UCHAR InheritedAddressSpace;
    UCHAR ReadImageFileExecOptions;
    UCHAR BeingDebugged;
    UCHAR BitField;
    ULONG64 Mutant;
    ULONG64 ImageBaseAddress;
    PPEB_LDR_DATA Ldr;
    PRTL_USER_PROCESS_PARAMETERS ProcessParameters;
    ULONG64 SubSystemData;
    ULONG64 ProcessHeap;
    ULONG64 FastPebLock;
    ULONG64 AtlThunkSListPtr;
    ULONG64 IFEOKey;
    ULONG64 CrossProcessFlags;
    ULONG64 UserSharedInfoPtr;
    ULONG SystemReserved;
    ULONG AtlThunkSListPtr32;
    ULONG64 ApiSetMap;
} PEB64, *PPEB64;

#pragma pack(4)
typedef struct _PEB32
{

```

```

    UCHAR InheritedAddressSpace;
    UCHAR ReadImageFileExecOptions;
    UCHAR BeingDebugged;
    UCHAR BitField;
    ULONG Mutant;
    ULONG ImageBaseAddress;
    ULONG Ldr;
    ULONG ProcessParameters;
    ULONG SubSystemData;
    ULONG ProcessHeap;
    ULONG FastPebLock;
    ULONG AtlThunkSListPtr;
    ULONG IFEOKey;
    ULONG CrossProcessFlags;
    ULONG UsersSharedInfoPtr;
    ULONG SystemReserved;
    ULONG AtlThunkSListPtr32;
    ULONG ApiSetMap;
} PEB32, *PPEB32;

```

```

typedef struct _PEB_LDR_DATA32
{
    ULONG Length;
    BOOLEAN Initialized;
    ULONG SsHandle;
    LIST_ENTRY32 InLoadOrderModuleList;
    LIST_ENTRY32 InMemoryOrderModuleList;
    LIST_ENTRY32 InInitializationOrderModuleList;
    ULONG EntryInProgress;
} PEB_LDR_DATA32, *PPEB_LDR_DATA32;

```

```

typedef struct _LDR_DATA_TABLE_ENTRY32
{
    LIST_ENTRY32 InLoadOrderLinks;
    LIST_ENTRY32 InMemoryOrderModuleList;
    LIST_ENTRY32 InInitializationOrderModuleList;
    ULONG DllBase;
    ULONG EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING32 FullDllName;
    UNICODE_STRING32 BaseDllName;
    ULONG Flags;
    USHORT LoadCount;
    USHORT TlsIndex;
    union
    {
        LIST_ENTRY32 HashLinks;
        ULONG SectionPointer;
    }u1;
    ULONG Checksum;
    union
    {
        ULONG TimeDateStamp;
        ULONG LoadedImports;
    }u2;
}

```

```

        ULONG EntryPointActivationContext;
        ULONG PatchInformation;
    } LDR_DATA_TABLE_ENTRY32, *PLDR_DATA_TABLE_ENTRY32;

#pragma pack()

```

接着就来实现对PEB的获取操作，以 64位 为例，我们需要调用 `PsGetProcessPeb()` 这个内核函数，因为该内核函数没有被公开所以调用之前需要头部导出，该函数需要传入用户进程的 `EProcess` 结构，该结构可用 `PsLookupProcessByProcessId` 函数动态获取到，获取到以后直接 `KeStackAttachProcess()` 附加到应用层进程上，即可直接输出进程的PEB结构信息，如下代码。

```

#include "peb.h"
#include <ntifs.h>

// 定义导出
NTKERNELAPI PVOID NTAPI PsGetProcessPeb(_In_ PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

// LyShark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    NTSTATUS status = STATUS_UNSUCCESSFUL;
    PEPROCESS eproc = NULL;
    KAPC_STATE kpc = { 0 };

    PPEB64 pPeb64 = NULL;

    __try
    {
        // HANDLE)4656 进程PID
        status = PsLookupProcessByProcessId((HANDLE)4656, &eproc);

        // 得到64位PEB
        pPeb64 = (PPEB64)PsGetProcessPeb(eproc);

        DbgPrint("PEB64 = %p \n", pPeb64);

        if (pPeb64 != 0)
        {
            // 验证可读性
            ProbeForRead(pPeb64, sizeof(PEB32), 1);

            // 附加进程
            KeStackAttachProcess(eproc, &kpc);

            DbgPrint("进程基地址: 0x%p \n", pPeb64->ImageBaseAddress);
            DbgPrint("ProcessHeap = 0x%p \n", pPeb64->ProcessHeap);
            DbgPrint("BeingDebugged = %d \n", pPeb64->BeingDebugged);
        }
    }
    __except (EXCEPTION_EXECUTE_HANDLER)
    {
        status = STATUS_UNSUCCESSFUL;
    }
}

```

```

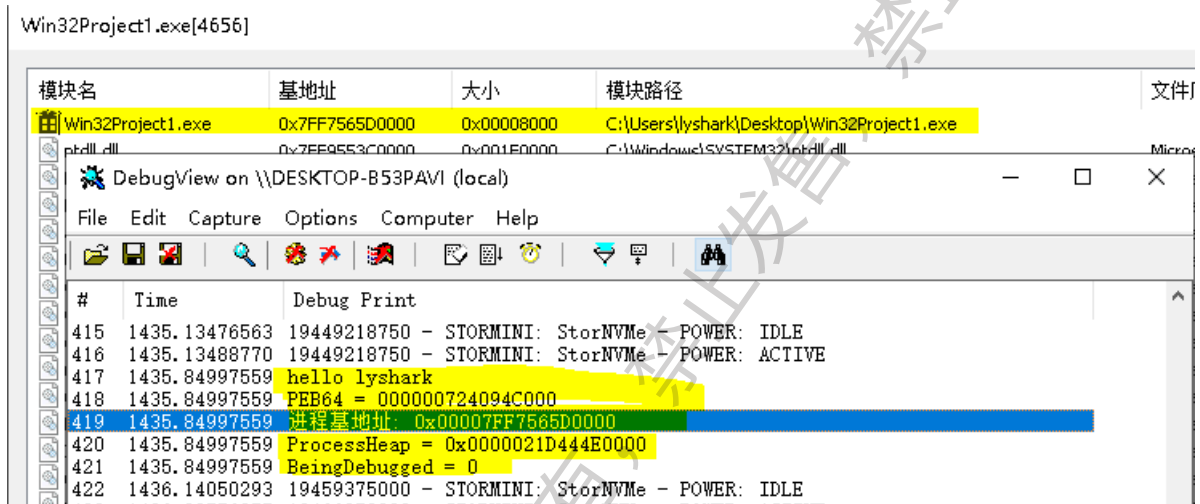
        // 脱离进程
        KeUnstackDetachProcess(&kpc);
    }
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

PEB64代码运行后，我们加载驱动即可看到如下结果：



而相对于64位进程来说，获取 32 位 进程的PEB信息可以直接调用 `PsGetProcessWow64Process()` 函数得到，该函数已被导出可以任意使用，获取PEB代码如下。

```

#include "peb.h"
#include <ntifs.h>

// 定义导出
NTKERNELAPI PVOID NTAPI PsGetProcessPeb(_In_ PEPROCESS Process);

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("Uninstall Driver Is OK \n");
}

// Lyshark
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark \n");

    NTSTATUS status = STATUS_UNSUCCESSFUL;
    PEPROCESS eproc = NULL;
    KAPC_STATE kpc = { 0 };

    PPEB32 pPeb32 = NULL;

```

```

__try
{
    // HANDLE)4656 进程PID
    status = PsLookupProcessByProcessId((HANDLE)6164, &eproc);

    // 得到32位PEB
    pPeb32 = (PPEB32)PsGetProcessWow64Process(eproc);

    DbgPrint("PEB32 = %p \n", pPeb32);

    if (pPeb32 != 0)
    {
        // 验证可读性
        ProbeForRead(pPeb32, sizeof(PEB32), 1);

        // 附加进程
        KeStackAttachProcess(eproc, &kpc);

        DbgPrint("进程基地址: 0x%p \n", pPeb32->ImageBaseAddress);
        DbgPrint("ProcessHeap = 0x%p \n", pPeb32->ProcessHeap);
        DbgPrint("BeingDebugged = %d \n", pPeb32->BeingDebugged);

        // 脱离进程
        KeUnstackDetachProcess(&kpc);
    }
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

PEB32代码运行后，我们加载驱动即可看到如下结果:

模块名	基地址	大小	模块路径	文件
windows.storage.dll	0x74FE0000	0x005C5000	C:\Windows\SysWOW64\windows.storage.dll	Micros
Windows 64Signer V1.2.exe	0x006F0000	0x0018B000	C:\Users\lyshark\Desktop\Windows 64Signer V1.2.exe	
DebugView on \\DESKTOP-B53PAVI (local)				
File Edit Capture Options Computer Help				
#	Time	Debug Print		
470	1608.01733398	hello lyshark		
471	1608.01733398	PEB32 = 00000000011B2000		
472	1608.01733398	进程基地址: 0x00000000006F0000		
473	1608.01733398	ProcessHeap = 0x00000000013C0000		
474	1608.01733398	BeingDebugged = 0		
475	1608.67468262	Uninstall Driver Is OK		

7.5 断链隐藏驱动程序自身

与断链隐藏进程功能类似，关于断链进程隐藏可参考《驱动开发：DKOM 实现进程隐藏》这一篇文章，断链隐藏驱动自身则用于隐藏自身SYS驱动文件，当驱动加载后那么使用ARK工具扫描将看不到自身驱动模块，此方法可能会触发PG会蓝屏，在某些驱动辅助中也会使用这种方法隐藏自己。

驱动实现代码如下所示：

```
#include <ntifs.h>

HANDLE hThread;

VOID ThreadRun(PVOID StartContext)
{
    LARGE_INTEGER times;
    PDRIVER_OBJECT pDriverObject;

    // 等待3秒 单位是纳秒
    times.QuadPart = -30 * 1000 * 1000;

    KeDelayExecutionThread(KernelMode, FALSE, &times);
    pDriverObject = (PDRIVER_OBJECT)StartContext;

    // 修改模块信息
    pDriverObject->DriverSize = 0;
    pDriverObject->DriverSection = NULL;
    pDriverObject->DriverExtension = NULL;
    pDriverObject->DriverStart = NULL;
    pDriverObject->DriverInit = NULL;
    pDriverObject->FastIoDispatch = NULL;
    pDriverObject->DriverStartIo = NULL;

    ZwClose(hThread);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    PLIST_ENTRY pModuleList;
    pModuleList = Driver->DriverSection;

    // 前一个模块的Flink=本模块的Flink
    pModuleList->Blink->Flink = pModuleList->Flink;

    // 前一个模块的Blink=本模块的Blink
    pModuleList->Flink->Blink = pModuleList->Blink;

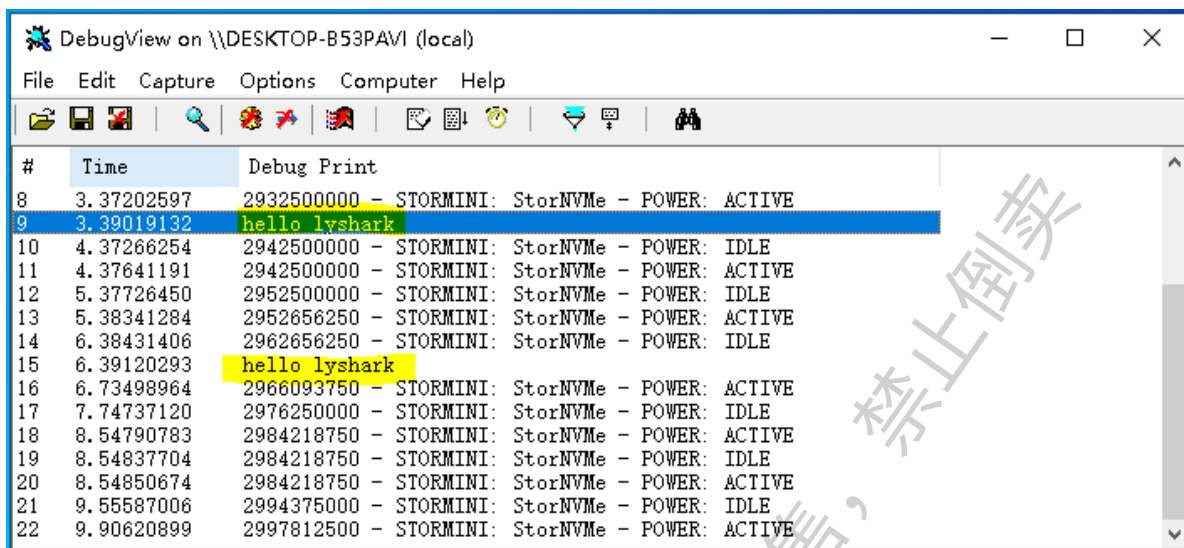
    PsCreateSystemThread(&hThread, GENERIC_ALL, NULL, NULL, NULL, ThreadRun,
Driver);
```

```

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

输出效果如下，驱动每隔3秒执行一次模块修改：



7.6 内核特征码搜索函数封装

在前面的系列教程如《驱动开发：内核枚举DpcTimer定时器》或者《驱动开发：内核枚举IoTimer定时器》里面 Lyshark 大量使用了 特征码定位 这一方法来寻找符合条件的 汇编指令 集，总体来说这种方式只能定位特征较小的指令如果特征值扩展到5位以上那么就需要写很多无用的代码，本章内容中将重点分析，并实现一个 通用 特征定位函数。

如下是一段特征码搜索片段，可以看到其实仅仅只是将上章中的搜索方式变成了一个 SearchSpecialCode 函数，如下函数，用户传入一个 扫描起始地址 以及搜索特征码的字节数组，即可完成搜索工作，具体的参数定义如下。

- pSearchBeginAddr 扫描的内存(内核)起始地址
- ulSearchLength 需要扫描的长度
- pSpecialCode 扫描特征码,传入一个UCHAR类型的字节数组
- ulSpecialCodeLength 特征码长度,传入字节数组长度

```

// By: LyShark.com
PVOID SearchSpecialCode(PVOID pSearchBeginAddr, ULONG ulSearchLength, PCHAR
pSpecialCode, ULONG ulSpecialCodeLength)
{
    PVOID pDestAddr = NULL;
    PCHAR pBeginAddr = (PCHAR)pSearchBeginAddr;
    PCHAR pEndAddr = pBeginAddr + ulSearchLength;
    PCHAR i = NULL;
    ULONG j = 0;

    for (i = pBeginAddr; i <= pEndAddr; i++)
    {
        // 遍历特征码
        for (j = 0; j < ulSpecialCodeLength; j++)
        {
            // 判断地址是否有效
            if (FALSE == MmIsValidAddress((PVOID)(i + j)))

```

```

    {
        break;
    }
    // 匹配特征码
    if (*(PUCHAR)(i + j) != pSpecialCode[j])
    {
        break;
    }
}

// 匹配成功
if (j >= ulSpecialCodeLength)
{
    pDestAddr = (PVOID)i;
    break;
}
}
return pDestAddr;
}

```

那么这个简单的特征码扫描函数该如何使用，这里我们就用《驱动开发：内核枚举IoTimer定时器》中枚举 IopTimerQueueHead 链表头部地址为案例进行讲解，如果你忘记了如何寻找链表头部可以去前面的文章中学习，这里只给出实现流程。

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```

nt!IoInitializeTimer+0x5d:
fffff805`74b85bed 488d5008      lea     rdx,[rax+8]
fffff805`74b85bf1 48897018      mov     qword ptr [rax+18h],rsi
fffff805`74b85bf5 4c8d054475e0ff lea     r8,[nt!IopTimerLock (fffff805`7498d140)]
fffff805`74b85bfc 48897820      mov     qword ptr [rax+20h],rdi
fffff805`74b85c00 488d0dd9ddcdf lea     rcx,[nt!IopTimerQueueHead (fffff805`748639e0)]
fffff805`74b85c07 e8141e98ff   call    nt!ExInterlockedInsertTailList (fffff805`74507a20)
fffff805`74b85c0c 33c0         xor     eax,eax

nt!IoInitializeTimer+0x7e:
fffff805`74b85c0e 488b5c2430    mov     rbx,qword ptr [rsp+30h]
fffff805`74b85c13 488b6c2438    mov     rbp,qword ptr [rsp+38h]
fffff805`74b85c18 488b742440    mov     rsi,qword ptr [rsp+40h]
fffff805`74b85c1d 4883c420      add     rsp,20h
fffff805`74b85c21 5f           pop     rdi
fffff805`74b85c22 c3           ret

```

我们首先通过 MmGetSystemRoutineAddress 得到 IoInitializeTimer 首地址，然后在偏移长度为 0x7e 范围内搜索特征码 48 8d 0d 特征，其代码可以总结为如下样子。

```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com \n"));

    // 得到基址
    PCHAR IoInitializeTimer = GetIoInitializeTimerAddress();
    DbgPrint("IoInitializeTimer Address = %p \n", IoInitializeTimer);

    // -----
    // Lyshark 开始定位特征

    // 设置起始位置
    PCHAR StartSearchAddress = (PCHAR)IoInitializeTimer;

```



```

// 设置结束位置
PUCHAR EndSearchAddress = StartSearchAddress + 0x7e;
DbgPrint("[LyShark 搜索区间] 起始地址: 0x%X --> 结束地址: 0x%X \n",
StartSearchAddress, EndSearchAddress);

// 设置搜索长度
LONGLONG size = EndSearchAddress - StartSearchAddress;
DbgPrint("[LyShark 搜索长度] 长度: %d \n", size);

PVOID ptr;

// 指定特征码
UCHAR pSpecialCode[256] = { 0 };

// 指定特征码长度
ULONG ulSpecialCodeLength = 3;

pSpecialCode[0] = 0x48;
pSpecialCode[1] = 0x8d;
pSpecialCode[2] = 0x0d;

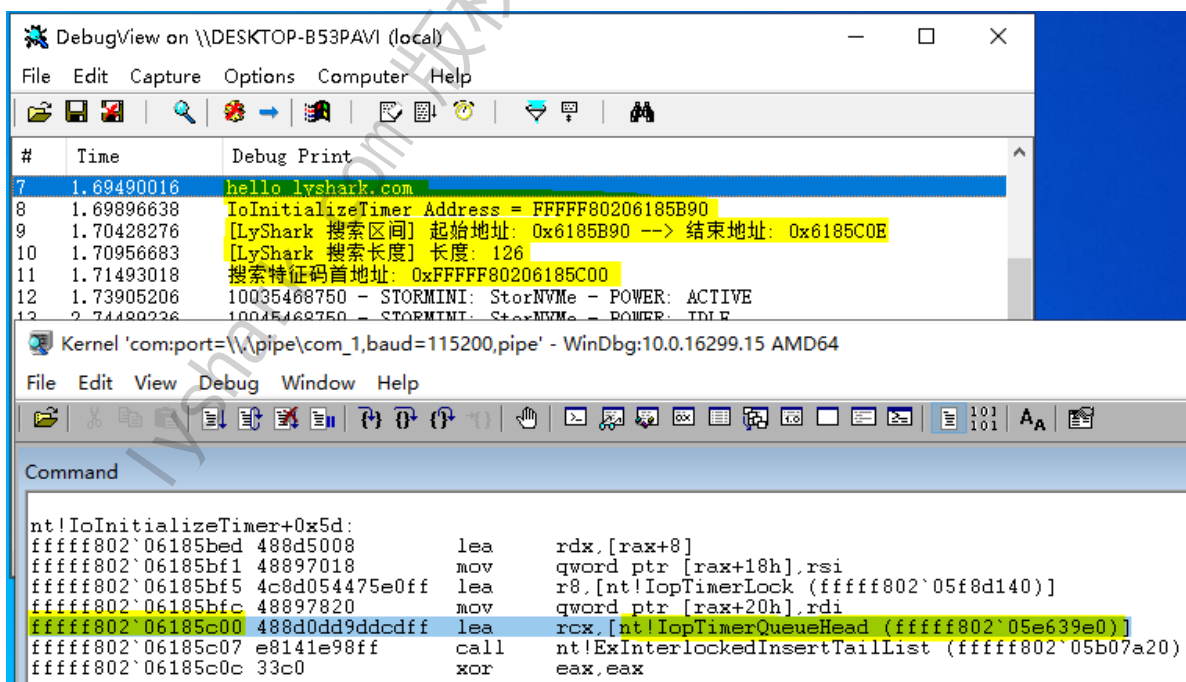
// 开始搜索,找到后返回首地址
ptr = SearchSpecialCode(StartSearchAddress, size, pSpecialCode,
ulSpecialCodeLength);

DbgPrint("搜索特征码首地址: 0x%p \n", ptr);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

代码运行后你会发现可以直接定位到我们所需要的位置上, 如下图所示:



如上图可以看到, 这个特征码定位函数返回的是内存地址, 而我们需要得到地址内的数据, 此时就需要提取以下。

例如当指令是:

```
fffff80206185c00 488d0dd9ddcdff lea rcx,[nt!IopTimerQueueHead
(fffff80205e639e0)]
```

那么就需要 `RtlCopyMemory` 跳过前三个字节，并在第四个字节开始取数据，并将读入的数据放入到 `IopTimerQueueHead_LyShark_Code` 变量内。

```
// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

// 开始搜索,找到后返回首地址
ptr = SearchSpecialCode(StartSearchAddress, size, pSpecialCode,
ulSpecialCodeLength);

DbgPrint("搜索特征码首地址: 0x%p \n", ptr);

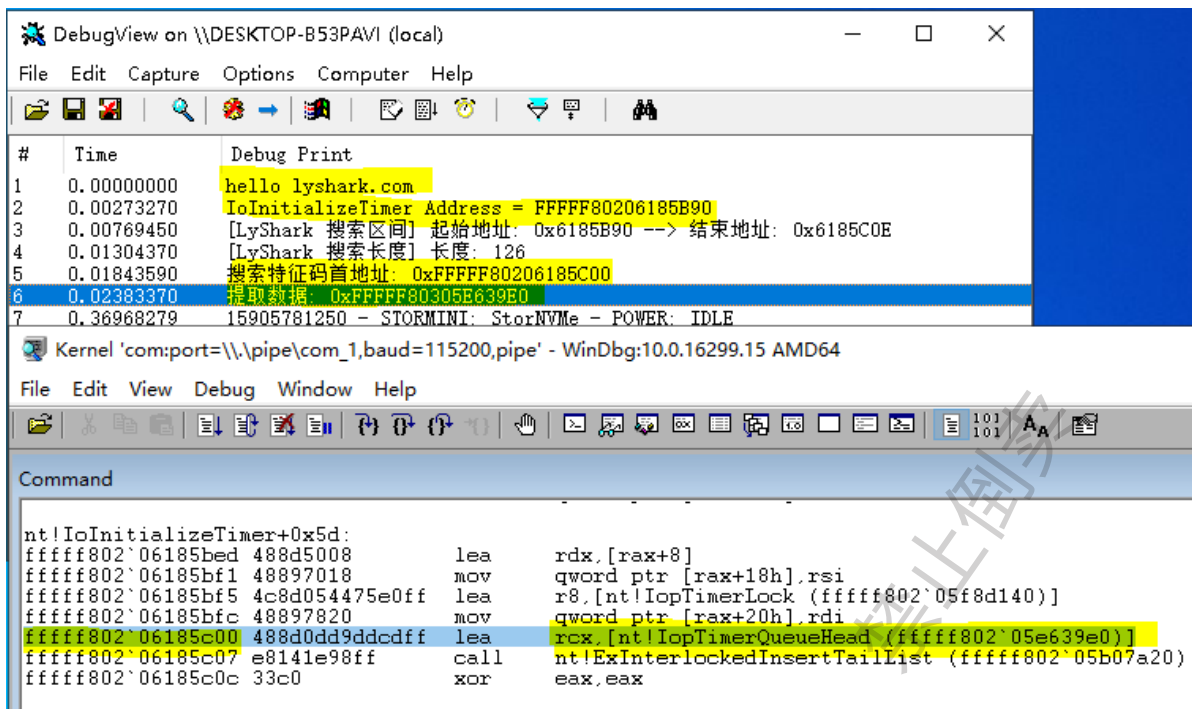
// 提取特征
// fffff802`06185c00 488d0dd9ddcdff lea rcx,[nt!IopTimerQueueHead
(fffff802`05e639e0)]
ULONG64 ioffset = 0;
ULONG64 IopTimerQueueHead_LyShark_Code = 0;

__try
{
    // 拷贝内存跳过lea,向后四字节
    RtlCopyMemory(&ioffset, (ULONG64)ptr + 3, 4);

    // 取出后面的IopTimerQueueHead内存地址 LyShark.com
    IopTimerQueueHead_LyShark_Code = ioffset + (ULONG64)ptr + 7;

    DbgPrint("提取数据: 0x%p \n", IopTimerQueueHead_LyShark_Code);
}
__except (1)
{
    DbgPrint("[LyShark] 拷贝内存异常 \n");
}
```

这样即可得到我们所需要的地址，如下结果所示：



7.7 内核LDE64引擎计算汇编长度

本章开始 Lyshark 将介绍如何在内核中实现 InlineHook 挂钩这门技术，内核挂钩的第一步需要实现一个动态计算汇编指令长度的功能，该功能可以使用 LDE64 这个反汇编引擎，该引擎小巧简单可以直接在驱动中使用，LDE引擎是 BeaEngine 引擎的一部分，后来让 Beatrix 打包成了一个 ShellCode 代码，并可以通过 typedef 动态指针的方式直接调用功能，本章内容作为后期 Hook 挂钩的铺垫部分，独立出来也是因为代码太多太占空间一篇文章写下来或很长影响阅读。

- LDE反汇编引擎源代码: <https://github.com/BeaEngine/lde64>

首先定义一个 lyshark_lde64.h 头文件，并写入如下 ShellCode 代码片段，当然这不是最新的，如果你需要最新的可以自己下载源代码编译后提取出来替换即可，不过该引擎很多年没有更新了替换的意义也不大毕竟功能就那么几行而已。

```
// 署名权
// right to sign one's name on a piece of work
// thanks to Av0id , cyberbob and lena151 for their remarks and advices
// PowerBy: BeaEngine | Beatrix | LyShark
// Email: me@lyshark.com

// 反汇编引擎
unsigned char szShellCode[12800] =
{
    // 参考博客园
    // https://www.cnblogs.com/LyShark/p/16832307.html
};
```

那么该如何调用呢？调用其实很容易，首先调用 lde_init() 函数将功能载入到内存，然后通过 lde_disasm() 直接调用功能，在调用时第一个参数传入需要计算的内存地址，第二个参数是位数，如果传入0则表示计算32位汇编汇编，如果传入64则计算64位汇编长度。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
```

```

// Email: me@lyshark.com
#include "lyshark_lde64.h"
#include <ntifs.h>

// 计算地址处指令有多少字节
// address = 地址
// bits 32位驱动传入0 64传入64
typedef INT(*LDE_DISASM)(PVOID address, INT bits);

LDE_DISASM lde_disasm;

// 初始化引擎
VOID lde_init()
{
    lde_disasm = ExAllocatePool(NonPagedPool, 12800);
    memcpy(lde_disasm, szShellCode, 12800);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    // 初始化反汇编引擎
    lde_init();

    UNICODE_STRING unstr;
    PVOID addr;

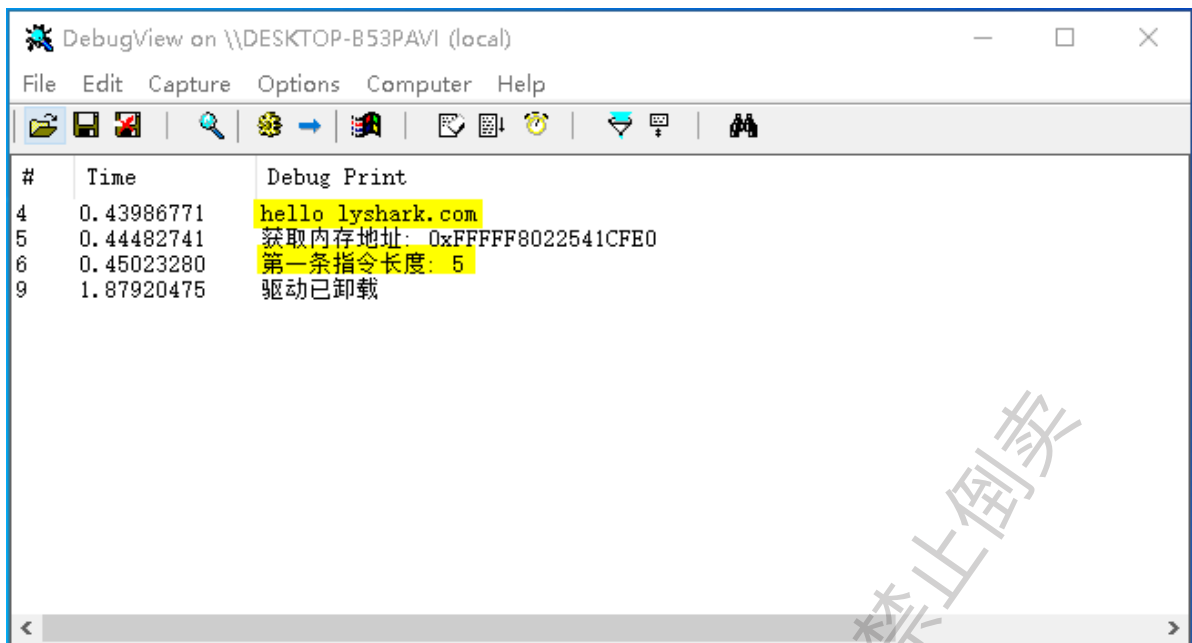
    RtlInitUnicodeString(&unstr, L"PsLookupProcessByProcessId");
    addr = MmGetSystemRoutineAddress(&unstr);
    DbgPrint("获取内存地址: 0x%p \n", addr);

    // 计算第一条汇编指令长度
    INT asm_len = lde_disasm(addr, 64);
    DbgPrint("第一条指令长度: %d \n", asm_len);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行上方的驱动程序，即可得到 `PsLookupProcessByProcessId` 函数第一条指令的实际长度，输出效果如下；



如果我们需要 Hook 挂钩 则最常用的就是填充 JMP 跨4G跳转, 该指令占用 14个字节 的内存长度, 但我们无法保证 14个字节 就是一个完整的指令长度, 有可能指令会被截断从而导致执行异常, 此时必须得到完整指令的长度, 指令长度就需要大于等于14, 所以代码中的计算应该这样来实现。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "lyshark_lde64.h"
#include <ntifs.h>

// 计算地址处指令有多少字节
// address = 地址
// bits 32位驱动传入0 64传入64
typedef INT(*LDE_DISASM)(PVOID address, INT bits);

LDE_DISASM lde_disasm;

// 初始化引擎
VOID lde_init()
{
    lde_disasm = ExAllocatePool(NonPagedPool, 12800);
    memcpy(lde_disasm, szShellCode, 12800);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");
}

// 得到完整指令长度, 避免截断
ULONG GetFullPatchSize(PUCHAR Address)
{
    ULONG LenCount = 0, Len = 0;

    // 至少需要14字节
    while (LenCount <= 14)
```

```

{
    Len = lde_disasm(Address, 64);
    Address = Address + Len;
    LenCount = LenCount + Len;
}
return LenCount;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    // 初始化反汇编引擎
    lde_init();

    UNICODE_STRING unstr;
    PVOID addr;

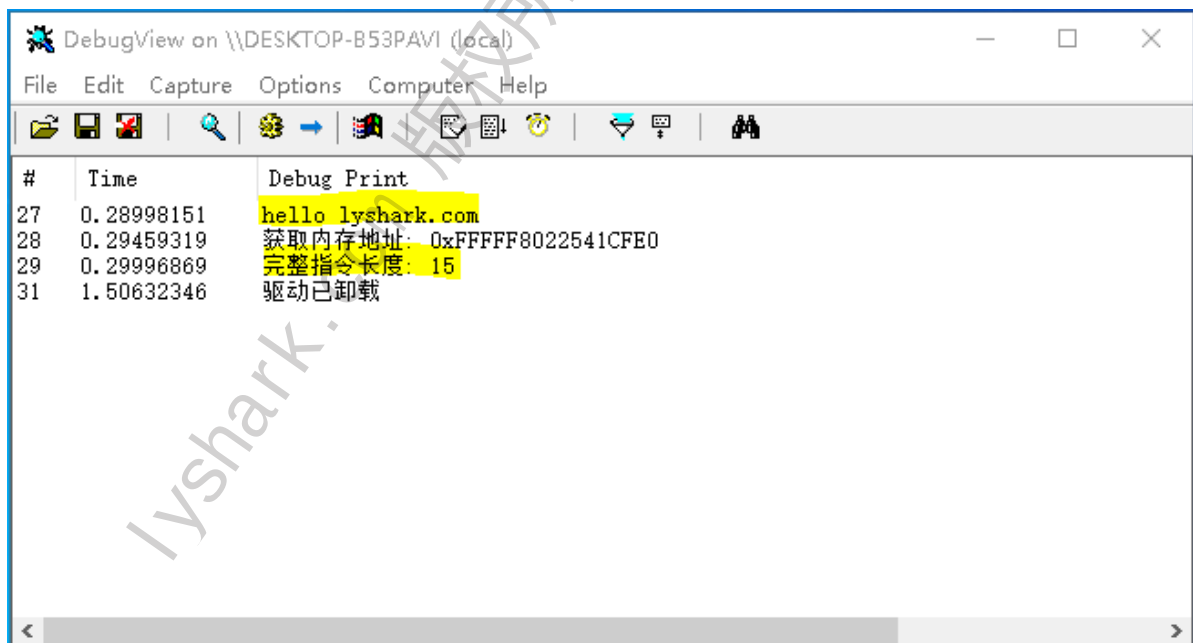
    RtlInitUnicodeString(&unstr, L"PsLookupProcessByProcessId");
    addr = MmGetSystemRoutineAddress(&unstr);
    DbgPrint("获取内存地址: 0x%p \n", addr);

    ULONG count = GetFullPatchSize(addr);
    DbgPrint("完整指令长度: %d \n", count);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行这个驱动程序，计算得到的结果与上图作比较，此处得到的才是一个完整的指令长度；



7.8 内核层InlineHook挂钩函数

在上一章《驱动开发：内核LDE64引擎计算汇编长度》中，LyShark教大家如何通过LDE64引擎实现计算反汇编指令长度，本章将在此基础之上实现内联函数挂钩，内核中的InlineHook函数挂钩其实与应用层一致，都是使用劫持执行流并跳转到我们自己的函数上来做处理，唯一的不同的是内核Hook只针对内核API函数，但由于其身处在最底层所以一旦被挂钩其整个应用层都将会受到影响，这就直接决定了

在内核层挂钩的效果是应用层无法比拟的，对于安全从业者来说学会使用内核挂钩也是很重要。

挂钩的原理可以总结为，通过 `MmGetSystemRoutineAddress` 得到原函数地址，然后保存该函数的 前15 个字节的指令，将自己的 `MyPsLookupProcessByProcessId` 代理函数地址写到原始函数上，此时如果有API被调用则默认会转向到我们自己的函数上面执行，恢复原理则是将提前保存好的前15个原始字节写回则恢复原函数的调用。

原理很简单，基本上 `InlineHook` 类的代码都是一个样子，如下是一段完整的挂钩

`PsLookupProcessByProcessId` 的驱动程序，当程序被加载时则默认会保护 `lyshark.exe` 进程，使其无法被用户使用任务管理器结束掉。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "lyshark_lde64.h"
#include <ntifs.h>
#include <windef.h>
#include <intrin.h>

#pragma intrinsic(_disable)
#pragma intrinsic(_enable)

// -----
// 汇编计算方法
// -----
// 计算地址处指令有多少字节
// address = 地址
// bits 32位驱动传入0 64传入64
typedef INT(*LDE_DISASM)(PVOID address, INT bits);

LDE_DISASM lde_disasm;

// 初始化引擎
VOID lde_init()
{
    lde_disasm = ExAllocatePool(NonPagedPool, 12800);
    memcpy(lde_disasm, szShellCode, 12800);
}

// 得到完整指令长度,避免截断
ULONG GetFullPatchSize(PUCHAR Address)
{
    ULONG LenCount = 0, Len = 0;

    // 至少需要14字节
    while (LenCount <= 14)
    {
        Len = lde_disasm(Address, 64);
        Address = Address + Len;
        LenCount = LenCount + Len;
    }
    return LenCount;
}
```

```

// -----
// Hook函数封装
// -----

// 定义指针方便调用
typedef NTSTATUS(__fastcall *PSLOOKUPPROCESSBYPROCESSID)(HANDLE ProcessId,
PEPROCESS *Process);

ULONG64 protect_eprocess = 0;           // 需要保护进程的eprocess
ULONG patch_size = 0;                  // 被修改了几个字节
PUCHAR head_n_byte = NULL;             // 前几个字节数组
PVOID original_address = NULL;         // 原函数地址

KIRQL WPOFFx64()
{
    KIRQL irq1 = KeRaiseIrqlToDpcLevel();
    UINT64 cr0 = __readcr0();
    cr0 &= 0xffffffffffffffffff;
    __writecr0(cr0);
    _disable();
    return irq1;
}

VOID WPONx64(KIRQL irq1)
{
    UINT64 cr0 = __readcr0();
    cr0 |= 0x10000;
    _enable();
    __writecr0(cr0);
    KeLowerIrql(irq1);
}

// 动态获取内存地址
PVOID GetProcessAddress(PCWSTR FunctionName)
{
    UNICODE_STRING UniCodeFunctionName;
    RtlInitUnicodeString(&UniCodeFunctionName, FunctionName);
    return MmGetSystemRoutineAddress(&UniCodeFunctionName);
}

/*
InlineHookAPI 挂钩地址

参数1: 待HOOK函数地址
参数2: 代理函数地址
参数3: 接收原始函数地址的指针
参数4: 接收补丁长度的指针
返回: 原来头N字节的数据
*/
PVOID KernelHook(IN PVOID ApiAddress, IN PVOID Proxy_ApiAddress, OUT PVOID
*Original_ApiAddress, OUT ULONG *PatchSize)
{
    KIRQL irq1;
    UINT64 tmpv;
    PVOID head_n_byte, ori_func;

```



```

// 保存跳转指令 JMP QWORD PTR [本条指令结束后的地址]
UCHAR jmp_code[] =
"\xFF\x25\x00\x00\x00\x00\xFF\xFF\xFF\xFF\xFF\xFF\xFF";

// 保存原始指令
UCHAR jmp_code_orifunc[] =
"\xFF\x25\x00\x00\x00\x00\xFF\xFF\xFF\xFF\xFF\xFF\xFF";

// 获取函数地址处指令长度
*PatchSize = GetFullPatchSize((PUCHAR)ApiAddress);

// 分配空间
head_n_byte = ExAllocatePoolWithTag(NonPagedPool, *PatchSize, "LyShark");

irq1 = WPOFFx64();

// 跳转地址拷贝到原函数上
RtlCopyMemory(head_n_byte, ApiAddress, *PatchSize);
WPONx64(irq1);

// 构建跳转

// 1.原始机器码+跳转机器码
ori_func = ExAllocatePoolWithTag(NonPagedPool, *PatchSize + 14, "LyShark");
RtlFillMemory(ori_func, *PatchSize + 14, 0x90);

// 2.跳转到没被打补丁的那个字节
tmpv = (ULONG64)ApiAddress + *PatchSize;
RtlCopyMemory(jmp_code_orifunc + 6, &tmpv, 8);
RtlCopyMemory((PUCHAR)ori_func, head_n_byte, *PatchSize);
RtlCopyMemory((PUCHAR)ori_func + *PatchSize, jmp_code_orifunc, 14);
*Original_ApiAddress = ori_func;

// 3.得到代理地址
tmpv = (UINT64)Proxy_ApiAddress;
RtlCopyMemory(jmp_code + 6, &tmpv, 8);

//4.打补丁
irq1 = WPOFFx64();
RtlFillMemory(ApiAddress, *PatchSize, 0x90);
RtlCopyMemory(ApiAddress, jmp_code, 14);
WPONx64(irq1);

return head_n_byte;
}

/*
InlineHookAPI 恢复挂钩地址

参数1: 被HOOK函数地址
参数2: 原始数据
参数3: 补丁长度
*/
VOID KernelUnHook(IN PVOID ApiAddress, IN PVOID OriCode, IN ULONG PatchSize)

```

```

{
    KIRQL irq1;
    irq1 = WPOFFx64();
    RtlCopyMemory(ApiAddress, OriCode, PatchSize);
    WPONx64(irq1);
}

// 实现我们自己的代理函数
NTSTATUS MyPsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS *Process)
{
    NTSTATUS st;
    st = ((PSLOOKUPPROCESSBYPROCESSID)original_address)(ProcessId, Process);
    if (NT_SUCCESS(st))
    {
        // 判断是否需要保护的进程
        if (*Process == (PEPROCESS)protect_eprocess)
        {
            *Process = 0;
            DbgPrint("[lyshark] 拦截结束进程 \n");
            st = STATUS_ACCESS_DENIED;
        }
    }
    return st;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动已卸载 \n");

    // 恢复Hook
    KernelUnHook(GetProcessAddress(L"PsLookupProcessByProcessId"), head_n_byte,
    patch_size);
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    // 初始化反汇编引擎
    lde_init();

    // 设置需要保护进程EProcess
    /*
    lyshark.com: kd> !process 0 0 lyshark.exe
    PROCESS fffff9a0a44ec4080
        SessionId: 1 Cid: 05b8 Peb: 0034d000 ParentCid: 13f0
        DirBase: 12a7d2002 ObjectTable: fffffd60bc036f080 HandleCount: 159.
        Image: lyshark.exe
    */
    protect_eprocess = 0xfffff9a0a44ec4080;

    // Hook挂钩函数
    head_n_byte = KernelHook(GetProcessAddress(L"PsLookupProcessByProcessId"),
    (PVOID)MyPsLookupProcessByProcessId, &original_address, &patch_size);
}

```

```

    DbgPrint("[lyshark] 挂钩保护完成 --> 修改字节: %d | 原函数地址: 0x%p \n",
    patch_size, original_address);

    for (size_t i = 0; i < patch_size; i++)
    {
        DbgPrint("[byte] = %x", head_n_byte[i]);
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行这段驱动程序，会输出挂钩保护的具体地址信息；

#	Time	Debug Print
252	6.54570532	hello lyshark.com
253	6.55054235	[lyshark] 挂钩保护完成 --> 修改字节: 15 原函数地址: 0xFFFF9A0A49B0A480
254	6.55582571	[byte] = 48
255	6.56115294	[byte] = 89
256	6.56647730	[byte] = 5c
257	6.57183456	[byte] = 24
258	6.57716417	[byte] = 18
259	6.58248663	[byte] = 56
260	6.58781099	[byte] = 48
261	6.59314060	[byte] = 83
262	6.59849405	[byte] = ec
263	6.60380697	[byte] = 20
264	6.60911179	[byte] = 48
265	6.61448288	[byte] = 89
266	6.61981201	[byte] = 7c
267	6.62514687	[byte] = 24

使用 winDBG 观察，会发现挂钩后原函数已经被替换掉了，而被替换的地址就是我们自己的 MyPsLookupProcessByProcessId 函数。

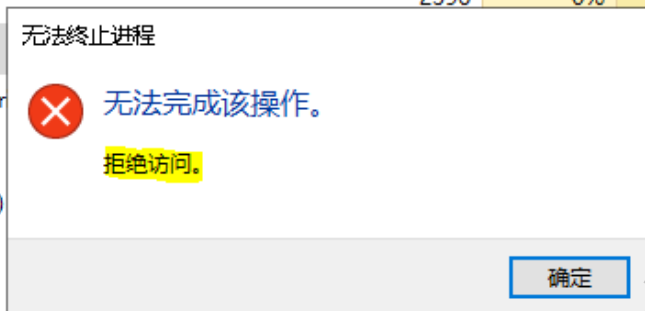
```

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
0: kd> uf PsLookupProcessByProcessId
Flow analysis was incomplete, some code may be missing
nt!PsLookupProcessByProcessId:
fffff802`2541cfe0 ff2500000000 jmp qword ptr [nt!PsLookupProcessByProcessId+0x6]
nt!PsLookupProcessByProcessId+0x6:
fffff802`2541cfe6 1018 adc byte ptr [rax],bl
fffff802`2541cfe8 452c02 sub al,2
fffff802`2541cfeb f8 cld

```

当你尝试使用任务管理器结束掉 lyshark.exe 进程时，则会提示拒绝访问。

进程	性能	应用历史记录	启动	用户	详细信息	服务
名称	状态	PID	1% CPU	31% 内存	0% 磁盘	
应用 (5)						
> 任务管理器		2396	0%	17.0 MB	0 MB/秒	
> lyshark				0.9 MB	0 MB/秒	
> Kernel Mode Dr				4.5 MB	0 MB/秒	
> DebugView				3.8 MB	0 MB/秒	
> 64Signer (32 位)				3.9 MB	0 MB/秒	
后台进程 (34)						
> 开始			0%	12.5 MB	0 MB/秒	
> 后台处理程序子系统应用		2240	0%	3.2 MB	0 MB/秒	



第八章：内核枚举篇

8.1 内核枚举IoTimer定时器

今天继续分享内核枚举系列知识，这次我们来学习如何通过代码的方式枚举内核 IoTimer 定时器，内核定时器其实就是在内核中实现的时钟，该定时器的枚举非常简单，因为在 IoInitializeTimer 初始化部分就可以找到 IopTimerQueueHead 地址，该变量内存储的就是定时器的链表头部。枚举IO定时器的案例并不多见，即便有也是无法使用过时的，此教程学到肯定就是赚到了。

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
系统回调	过滤驱动	DPC定时器	IO定时器	系统线程	卸载的驱动							
定时器对象	设备对象	状态	函数入口	模块路径								

枚举Io定时器过程是这样的：

- 1.找到 IoInitializeTimer 函数，该函数可以通过 MmGetSystemRoutineAddress 得到。
- 2.找到地址以后，我们向下增加 0xFF 偏移量，并搜索特征定位到 IopTimerQueueHead 链表头。
- 3.将链表头转换为 IO_TIMER 结构体，并循环链表头输出。

这里解释一下为什么要找 IoInitializeTimer 这个函数他是一个初始化函数，既然是初始化里面一定会涉及到链表的存储问题，找到他就能找到定时器链表基址，该函数的定义如下。

```

NTSTATUS
IoInitializeTimer(
    IN PDEVICE_OBJECT DeviceObject,    // 设备对象指针
    IN PIO_TIMER_ROUTINE TimerRoutine, // 定时器例程
    IN PVOID Context                   // 传给定时器例程的函数
);

```

接着我们需要得到IO定时器的结构定义，在 DEVICE_OBJECT 设备对象指针中存在一个 Timer 属性。

```

lyshark.com: kd> dt _DEVICE_OBJECT
ntdll!_DEVICE_OBJECT
+0x000 Type           : Int2B
+0x002 Size           : Uint2B
+0x004 ReferenceCount : Int4B
+0x008 DriverObject   : Ptr64 _DRIVER_OBJECT
+0x010 NextDevice     : Ptr64 _DEVICE_OBJECT
+0x018 AttachedDevice : Ptr64 _DEVICE_OBJECT
+0x020 CurrentIrp     : Ptr64 _IRP
+0x028 Timer          : Ptr64 _IO_TIMER
+0x030 Flags          : Uint4B
+0x034 Characteristics : Uint4B
+0x038 Vpb           : Ptr64 _VPB
+0x040 DeviceExtension : Ptr64 Void
+0x048 DeviceType     : Uint4B
+0x04c StackSize      : Char
+0x050 Queue          : <anonymous-tag>
+0x098 AlignmentRequirement : Uint4B
+0x0a0 DeviceQueue    : _KDEVICE_QUEUE
+0x0c8 Dpc            : _KDPC
+0x108 ActiveThreadCount : Uint4B
+0x110 SecurityDescriptor : Ptr64 Void
+0x118 DeviceLock     : _KEVENT
+0x130 SectorSize     : Uint2B
+0x132 Spare1         : Uint2B
+0x138 DeviceObjectExtension : Ptr64 _DEVOBJ_EXTENSION
+0x140 Reserved       : Ptr64 Void

```

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help



Command

```

0: kd> dt _DEVICE_OBJECT
ntdll!_DEVICE_OBJECT
+0x000 Type           : Int2B
+0x002 Size           : Uint2B
+0x004 ReferenceCount : Int4B
+0x008 DriverObject   : Ptr64 _DRIVER_OBJECT
+0x010 NextDevice     : Ptr64 _DEVICE_OBJECT
+0x018 AttachedDevice : Ptr64 _DEVICE_OBJECT
+0x020 CurrentIrp     : Ptr64 _IRP
+0x028 Timer          : Ptr64 _IO_TIMER
+0x030 Flags          : Uint4B
+0x034 Characteristics : Uint4B
+0x038 Vpb           : Ptr64 _VPB
+0x040 DeviceExtension : Ptr64 Void
+0x048 DeviceType     : Uint4B
+0x04c StackSize      : Char
+0x050 Queue          : <anonymous-tag>
+0x098 AlignmentRequirement : Uint4B
+0x0a0 DeviceQueue    : _KDEVICE_QUEUE
+0x0c8 Dpc            : _KDPC
+0x108 ActiveThreadCount : Uint4B
+0x110 SecurityDescriptor : Ptr64 Void
+0x118 DeviceLock     : _KEVENT
+0x130 SectorSize     : Uint2B
+0x132 Spare1         : Uint2B
+0x138 DeviceObjectExtension : Ptr64 _DEVOBJ_EXTENSION
+0x140 Reserved       : Ptr64 Void

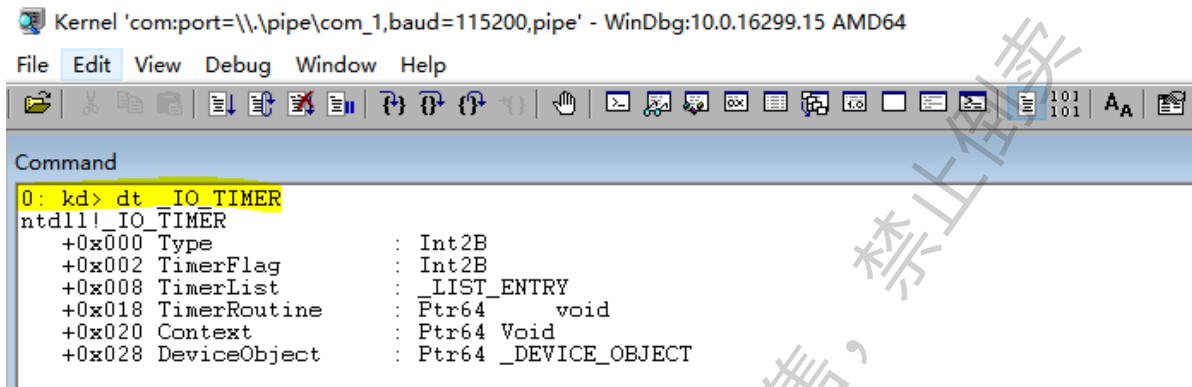
```

这里的这个 +0x028 Timer 定时器是一个结构体 _IO_TIMER 其就是IO定时器的所需结构体。

```

lyshark.com: kd> dt _IO_TIMER
ntdll!_IO_TIMER
+0x000 Type           : Int2B
+0x002 TimerFlag      : Int2B
+0x008 TimerList      : _LIST_ENTRY
+0x018 TimerRoutine   : Ptr64 void
+0x020 Context        : Ptr64 Void
+0x028 DeviceObject   : Ptr64 _DEVICE_OBJECT

```



Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```

0: kd> dt _IO_TIMER
ntdll!_IO_TIMER
+0x000 Type           : Int2B
+0x002 TimerFlag      : Int2B
+0x008 TimerList      : _LIST_ENTRY
+0x018 TimerRoutine   : Ptr64 void
+0x020 Context        : Ptr64 Void
+0x028 DeviceObject   : Ptr64 _DEVICE_OBJECT

```

如上方的基础知识有了也就够了，接着就是实际开发部分，首先我们需要编写一个 `GetIoInitializeTimerAddress()` 函数，让该函数可以定位到 `IoInitializeTimer` 所在内核中的基地址上面，具体实现调用代码如下所示。

```

#include <ntifs.h>

// 得到IoInitializeTimer基址
// By: LyShark 内核开发系列教程
PVOID GetIoInitializeTimerAddress()
{
    PVOID VariableAddress = 0;
    UNICODE_STRING uioiTime = { 0 };

    RtlInitUnicodeString(&uioiTime, L"IoInitializeTimer");
    VariableAddress = (PVOID)MmGetSystemRoutineAddress(&uioiTime);
    if (VariableAddress != 0)
    {
        return VariableAddress;
    }
    return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com \n"));

    // 得到基址
    PCHAR IoInitializeTimer = GetIoInitializeTimerAddress();
    DbgPrint(("IoInitializeTimer Address = %p \n", IoInitializeTimer);

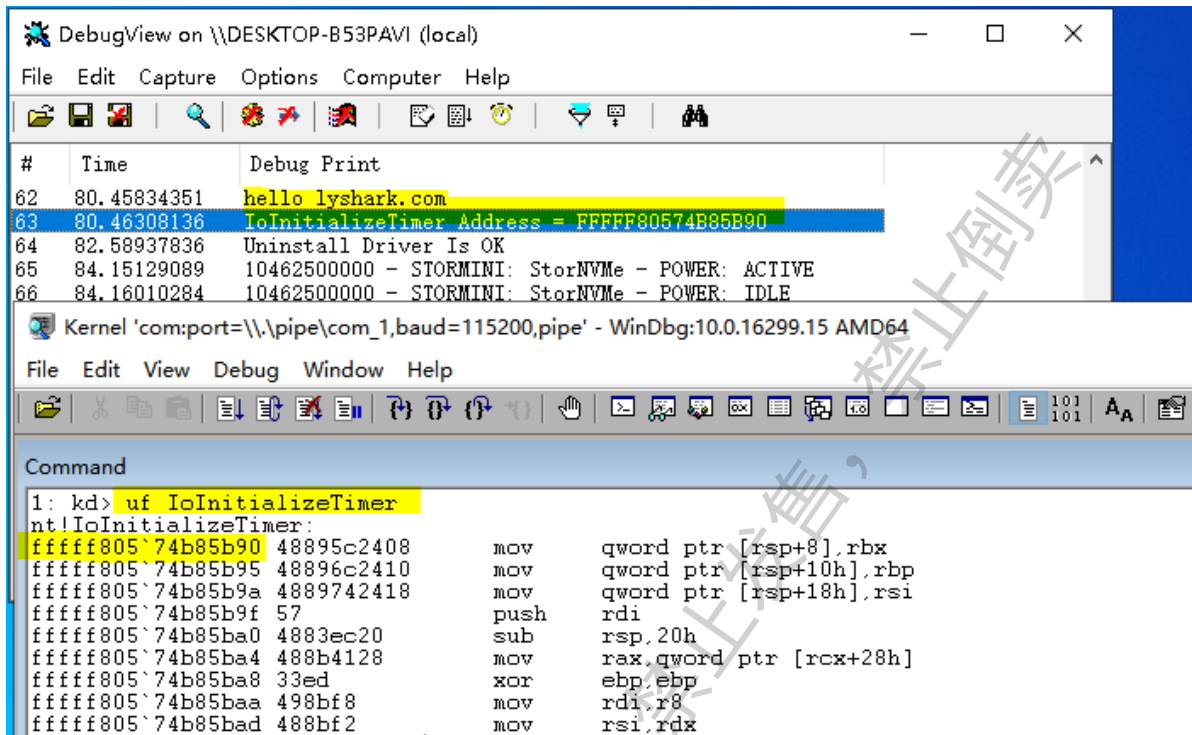
```

```

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

运行这个驱动程序，然后对比下是否一致：



接着我们在反汇编代码中寻找 IoTimerQueueHead，此处 LyShark 系统内这个偏移位置是 `nt!IoInitializeTimer+0x5d` 具体输出位置如下。



在 WinDBG 中标注出颜色 `lea rcx,[nt!IopTimerQueueHead (fffff805748639e0)]` 更容易看到。

```

Command
nt!IoInitializeTimer+0x5d:
ffff805`74b85bed 488d5008      lea     rdx,[rax+8]
ffff805`74b85bf1 48897018      mov     qword ptr [rax+18h],rsi
ffff805`74b85bf5 4c8d054475e0ff lea     r8,[nt!IopTimerLock (ffff805`7498d140)]
ffff805`74b85bfc 48897820      mov     qword ptr [rax+20h],rdi
ffff805`74b85c00 488d0dd9ddcdf lea     rcx,[nt!IopTimerQueueHead (ffff805`748639e0)]
ffff805`74b85c07 e8141e98ff   call    nt!ExInterlockedInsertTailList (ffff805`74507a20)
ffff805`74b85c0c 33c0         xor     eax,eax

nt!IoInitializeTimer+0x7e:
ffff805`74b85c0e 488b5c2430    mov     rbx,qword ptr [rsp+30h]
ffff805`74b85c13 488b6c2438    mov     rbp,qword ptr [rsp+38h]
ffff805`74b85c18 488b742440    mov     rsi,qword ptr [rsp+40h]
ffff805`74b85c1d 4883c420      add     rsp,20h
ffff805`74b85c21 5f           pop     rdi
ffff805`74b85c22 c3           ret

```

接着就是通过代码实现对此处的定位，定位我们就采用特征码搜索的方式，如下代码是特征搜索部分。

- StartSearchAddress 代表开始位置
- EndSearchAddress 代表结束位置，粗略计算0xff就可以定位到了。

```
#include <ntifs.h>
```

```
// 得到IoInitializeTimer基址
```

```
// By: LyShark 内核开发系列教程
```

```
PVOID GetIoInitializeTimerAddress()
```

```
{
```

```
    PVOID VariableAddress = 0;
```

```
    UNICODE_STRING uiioiTime = { 0 };
```

```
    RtlInitUnicodeString(&uiioiTime, L"IoInitializeTimer");
```

```
    VariableAddress = (PVOID)MmGetSystemRoutineAddress(&uiioiTime);
```

```
    if (VariableAddress != 0)
```

```
    {
```

```
        return VariableAddress;
```

```
    }
```

```
    return 0;
```

```
}
```

```
VOID UnDriver(PDRIVER_OBJECT driver)
```

```
{
```

```
    DbgPrint(("Uninstall Driver Is OK \n"));
```

```
}
```

```
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
```

```
{
```

```
    DbgPrint(("hello lyshark.com \n"));
```

```
    // 得到基址
```

```
    PCHAR IoInitializeTimer = GetIoInitializeTimerAddress();
```

```
    DbgPrint("IoInitializeTimer Address = %p \n", IoInitializeTimer);
```

```
    INT32 ioffset = 0;
```

```
    PLIST_ENTRY IoTimerQueueHead = NULL;
```

```
    PCHAR StartSearchAddress = IoInitializeTimer;
```

```
    PCHAR EndSearchAddress = IoInitializeTimer + 0xFF;
```



```

    UCHAR v1 = 0, v2 = 0, v3 = 0;

    for (PUCHAR i = StartSearchAddress; i < EndSearchAddress; i++)
    {
        if (MmIsAddressValid(i) && MmIsAddressValid(i + 1) && MmIsAddressValid(i
+ 2))
        {
            v1 = *i;
            v2 = *(i + 1);
            v3 = *(i + 2);

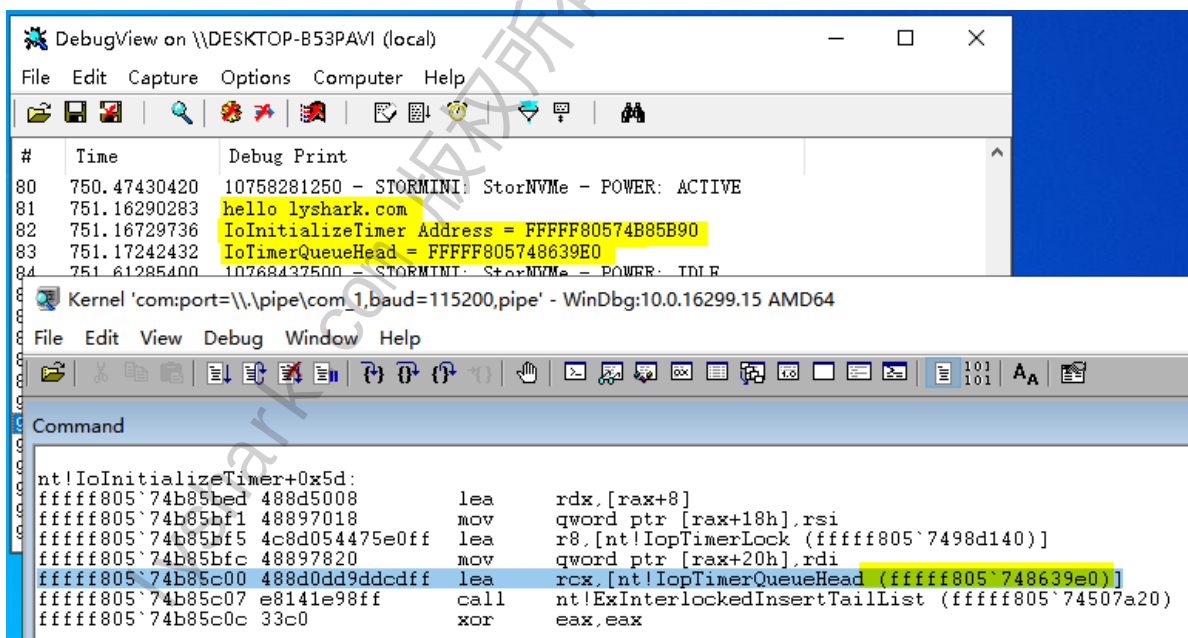
            // 三个特征码
            if (v1 == 0x48 && v2 == 0x8d && v3 == 0x0d)
            {
                memcpy(&ioffset, i + 3, 4);
                IoTimerQueueHead = (PLIST_ENTRY)(ioffset + (ULONG64)i + 7);
                DbgPrint("IoTimerQueueHead = %p \n", IoTimerQueueHead);
                break;
            }
        }
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

搜索三个特征码 `v1 == 0x48 && v2 == 0x8d && v3 == 0x0d` 从而得到内存位置，运行驱动对比下。

- 运行代码会取出 `lea` 指令后面的操作数，而不是取出 `lea` 指令的内存地址。



The screenshot shows the WinDbg interface with the following content:

DebugView on \\DESKTOP-B53PAVI (local)

File Edit Capture Options Computer Help

Time Debug Print

80 750.47430420 10758281250 - STORMINI: StorNVMe - POWER: ACTIVE

81 751.16290283 hello lyshark.com

82 751.16729736 IoInitializeTimer Address = FFFFF80574B85B90

83 751.17242432 IoTimerQueueHead = FFFFF805748639E0

84 751.61285400 10768437500 - STORMINI: StorNVMe - POWER: IDLE

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

File Edit View Debug Window Help

Command

```

nt!IoInitializeTimer+0x5d:
fffff805`74b85bed 488d5008 lea     rdx,[rax+8]
fffff805`74b85bf1 48897018 mov     qword ptr [rax+18h],rsi
fffff805`74b85bf5 4c8d054475e0ff lea     r8,[nt!IopTimerLock (fffff805`7498d140)]
fffff805`74b85bfc 48897820 mov     qword ptr [rax+20h],rdi
fffff805`74b85c00 488d0dd9ddcdff lea     rcx,[nt!IoTimerQueueHead (fffff805`748639e0)]
fffff805`74b85c07 e8141e98ff call    nt!ExInterlockedInsertTailList (fffff805`74507a20)
fffff805`74b85c0c 33c0 xor     eax,eax

```

最后一步就是枚举部分，我们需要前面提到的 `IO_TIMER` 结构体定义。

- `PIO_TIMER Timer = CONTAINING_RECORD(NextEntry, IO_TIMER, TimerList)` 得到结构体，循环输出即可。

```

// By: LyShark 内核开发系列教程
// https://www.cnblogs.com/LyShark/articles/16784393.html
#include <ntddk.h>

```

```
#include <ntstrsafe.h>
```

```
typedef struct _IO_TIMER
```

```
{  
    INT16      Type;  
    INT16      TimerFlag;  
    LONG32     Unknown;  
    LIST_ENTRY TimerList;  
    PVOID      TimerRoutine;  
    PVOID      Context;  
    PVOID      DeviceObject;  
}IO_TIMER, *PIO_TIMER;
```

```
// 得到IoInitializeTimer基址
```

```
PVOID GetIoInitializeTimerAddress()
```

```
{  
    PVOID VariableAddress = 0;  
    UNICODE_STRING uiioiTime = { 0 };  
  
    RtlInitUnicodeString(&uiioiTime, L"IoInitializeTimer");  
    VariableAddress = (PVOID)MmGetSystemRoutineAddress(&uiioiTime);  
    if (VariableAddress != 0)  
    {  
        return VariableAddress;  
    }  
    return 0;  
}
```

```
VOID UnDriver(PDRIVER_OBJECT driver)
```

```
{  
    DbgPrint("卸载完成... \n");  
}
```

```
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
```

```
{  
    DbgPrint(("hello lyshark.com \n"));
```

```
// 得到基址
```

```
PCHAR IoInitializeTimer = GetIoInitializeTimerAddress();  
DbgPrint("IoInitializeTimer Address = %p \n", IoInitializeTimer);
```

```
// 搜索IoTimerQueueHead地址
```

```
/*
```

```
    nt!IoInitializeTimer+0x5d:  
    fffff806`349963cd 488d5008      lea     rdx,[rax+8]  
    fffff806`349963d1 48897018      mov     qword ptr [rax+18h],rsi  
    fffff806`349963d5 4c8d05648de0ff lea     r8,[nt!IopTimerLock  
(fffff806`3479f140)]  
    fffff806`349963dc 48897820      mov     qword ptr [rax+20h],rdi  
    fffff806`349963e0 488d0d99f6cdfd lea     rcx,[nt!IopTimerQueueHead  
(fffff806`34675a80)]  
    fffff806`349963e7 e8c43598ff    call    nt!ExInterlockedInsertTailList  
(fffff806`343199b0)  
    fffff806`349963ec 33c0          xor     eax,eax
```

```

*/
INT32 ioffset = 0;
PLIST_ENTRY IoTimerQueueHead = NULL;

PUCHAR StartSearchAddress = IoInitializeTimer;
PUCHAR EndSearchAddress = IoInitializeTimer + 0xFF;
UCHAR v1 = 0, v2 = 0, v3 = 0;

for (PUCHAR i = StartSearchAddress; i < EndSearchAddress; i++)
{
    if (MmIsAddressValid(i) && MmIsAddressValid(i + 1) && MmIsAddressValid(i + 2))
    {
        v1 = *i;
        v2 = *(i + 1);
        v3 = *(i + 2);

        // fffff806`349963e0 48 8d 0d 99 f6 cd ff lea rcx,[nt!IoTimerQueueHead
        (fffff806`34675a80)]
        if (v1 == 0x48 && v2 == 0x8d && v3 == 0x0d)
        {
            memcpy(&ioffset, i + 3, 4);
            IoTimerQueueHead = (PLIST_ENTRY)(ioffset + (ULONG64)i + 7);
            DbgPrint("IoTimerQueueHead = %p \n", IoTimerQueueHead);
            break;
        }
    }
}

// 枚举列表
KIRQL OldIrql;

// 获得特权级
OldIrql = KeRaiseIrqlToDpcLevel();

if (IoTimerQueueHead && MmIsAddressValid((PVOID)IoTimerQueueHead))
{
    PLIST_ENTRY NextEntry = IoTimerQueueHead->Flink;
    while (MmIsAddressValid(NextEntry) && NextEntry !=
(PLIST_ENTRY)IoTimerQueueHead)
    {
        PIO_TIMER Timer = CONTAINING_RECORD(NextEntry, IO_TIMER, TimerList);

        if (Timer && MmIsAddressValid(Timer))
        {
            DbgPrint("IO对象地址: %p \n", Timer);
        }
        NextEntry = NextEntry->Flink;
    }
}

// 恢复特权级
KeLowerIrql(OldIrql);

Driver->DriverUnload = UnDriver;

```

```
return STATUS_SUCCESS;
}
```

运行这段源代码，并可得到以下输出，由于没有IO定时器所以输出结果是空的：

进程 驱动模块 内核层 内核钩子 应用层钩子 设置 监控 启动信息 注册表 服务 文件 网络 调试引擎

系统回调 过滤驱动 DPC定时器 IO定时器 系统线程 卸载的驱动

定时器对象	设备对象	状态	函数入口	模块路径
-------	------	----	------	------

DebugView on \\DESKTOP-B53PAVI (local)
File Edit Capture Options Computer Help

#	Time	Debug Print
96	762.45129395	10875937500 - STORMINI: StorNVMe - POWER: IDLE
97	1180.34191895	hello lyshark.com
98	1180.34606934	IoInitializeTimer Address = FFFFF80574B85B90
99	1180.35156250	IoTimerQueueHead = FFFFF805748639E0
100	1180.43054199	11106250000 - STORMINI: StorNVMe - POWER: IDLE

至此IO定时器的枚举就介绍完了，在教程中你已经学会了使用特征码定位这门技术，相信你完全可以输出内核中想要得到的任何结构体。

8.2 内核枚举DpcTimer定时器

在笔者上一篇文章《驱动开发：内核枚举IoTimer定时器》中我们通过 IoInitializeTimer 这个API函数为跳板，向下扫描特征码获取到了 IopTimerQueueHead 也就是IO定时器的队列头，本章学习的枚举 DPC定时器依然使用特征码扫描，唯一不同的是在新版系统中DPC是被异或加密的，想要找到正确的地址，只是需要在找到DPC表头时进行解密操作即可。

进程 驱动模块 内核层 内核钩子 应用层钩子 设置 监控 启动信息 注册表 服务 文件 网络 调试引擎

系统回调 过滤驱动 DPC定时器 IO定时器 系统线程 卸载的驱动

定时器对象	DPC	触发周期(s)	函数入口	模块路径
0xFFFFF803161EB1E0	0xFFFFF803161EB1A0	0	0xFFFFF803161E1170	C:\Windows\System32\Drivers\dfsc.sys
0xFFFFC88CCF6C26D0	0xFFFFC88CCF6C2690	0	0xFFFFF80315F9AA20	C:\Windows\system32\drivers\afd.sys
0xFFFFF803147323E0	0xFFFFF80314732420	0	0xFFFFF80314724410	C:\Windows\system32\drivers\pdc.sys
0xFFFFF803100263A0	0xFFFFF80310026300	0	0xFFFFF8030FCFC250	C:\Windows\system32\ntoskrnl.exe
0xFFFFF803152E9160	0xFFFFF803152E9100	0	0xFFFFF803150E6840	C:\Windows\System32\drivers\tcpip.sys
0xFFFFC88CCF318178	0xFFFFC88CCF3181B8	0	0xFFFFF8030FD7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFC88CCF319178	0xFFFFC88CCF3191B8	0	0xFFFFF8030FD7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFC88CCF31B178	0xFFFFC88CCF31B1B8	0	0xFFFFF8030FD7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFC88CCF31C178	0xFFFFC88CCF31C1B8	0	0xFFFFF8030FD7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFC88CCF31E178	0xFFFFC88CCF31E1B8	0	0xFFFFF8030FD7EB80	C:\Windows\system32\ntoskrnl.exe

DPC定时器的作用是什么？

- 在内核中可以使用DPC定时器设置任意定时任务，当到达某个节点时自动触发定时回调，定时器的内部使用 KTIMER 对象，当设置任务时会自动插入到 DPC 队列，由操作系统循环读取 DPC 队列并执行任务，枚举 DPC 定时器可得知系统中存在的 DPC 任务。

要想在新版系统中得到DPC定时器则需要执行的步骤有哪些？

- 1.找到 KiProcessorBlock 地址并解析成 _KPRCB 结构
- 2.在 _KPRCB 结构中得到 _KTIMER_TABLE 偏移
- 3.解析 _KTIMER_TABLE_ENTRY 得到加密后的双向链表

首先 _KPRCB 这个结构体与CPU内核对对应，获取方式可通过一个未导出的变量 nt!KiProcessorBlock 来得到，如下双核电脑，结构体存在两个与之对应的结构地址。

```

lyshark.com 0: kd> dq nt!KiProcessorBlock
fffff807`70a32cc0 fffff807`6f77c180 fffffbe81`3cee0180
fffff807`70a32cd0 00000000`00000000 00000000`00000000
fffff807`70a32ce0 00000000`00000000 00000000`00000000

```

此 `KiProcessorBlock` 是一个数组，其第一个结构体 `TimerTable` 则是结构体的偏移。

```

lyshark.com 0: kd> dt _KPRCB fffff807`6f77c180
ntdll!_KPRCB
+0x000 MxCsr           : 0x1f80
+0x3680 TimerTable     : _KTIMER_TABLE (此处)
+0x5880 DpcGate        : _KGATE

```

接下来是把所有的 `KTIMER` 都枚举出来，`KTIMER` 在 `TimerTable` 中的存储方式是数组+双向链表。

```

lyshark.com 0: kd> dt _KTIMER_TABLE
ntdll!_KTIMER_TABLE
+0x000 TimerExpiry     : [64] Ptr64 _KTIMER
+0x200 TimerEntries     : [256] _KTIMER_TABLE_ENTRY (此处)

```

到了 `_KTIMER_TABLE_ENTRY` 这里，`Entry` 开始的双向链表，每一个元素都对应一个 `Timer` 也就是说我们已经可以遍历所有未解密的 `Time` 变量了。

```

lyshark.com 0: kd> dt _KTIMER_TABLE_ENTRY 0xfffff807`6f77c180 + 0x3680
ntdll!_KTIMER_TABLE_ENTRY
+0x000 Lock           : 0
+0x008 Entry          : _LIST_ENTRY [ 0x00000000`00000000 -
0x00000000`00000000 ]
+0x018 Time           : _ULARGE_INTEGER 0x0

lyshark.com 0: kd> dt _KTIMER_TABLE_ENTRY 0xfffff807`6f77c180 + 0x3680 + 0x200
ntdll!_KTIMER_TABLE_ENTRY
+0x000 Lock           : 0
+0x008 Entry          : _LIST_ENTRY [ 0xfffffa707`a0d3e1a0 -
0xfffffa707`a0d3e1a0 ]
+0x018 Time           : _ULARGE_INTEGER 0x00000001`a8030353

```

至于如何解密，我们需要得到加密位置，如下通过 `KeSetTimer` 找到 `KeSetTimerEx` 从中得到 DCP 加密流程。

```

lyshark.com 0: kd> u nt!KeSetTimer
nt!KeSetTimer:
fffff803`0fc63a40 4883ec38      sub     rsp,38h
fffff803`0fc63a44 4c89442420     mov     qword ptr [rsp+20h],r8
fffff803`0fc63a49 4533c9        xor     r9d,r9d
fffff803`0fc63a4c 4533c0        xor     r8d,r8d
fffff803`0fc63a4f e80c000000     call    nt!KiSetTimerEx (fffff803`0fc63a60)
fffff803`0fc63a54 4883c438      add     rsp,38h
fffff803`0fc63a58 c3            ret
fffff803`0fc63a59 cc            int     3

0: kd> u nt!KiSetTimerEx 150

```

```

nt!KiSetTimerEx:
fffff803`0fc63a60 48895c2408      mov     qword ptr [rsp+8],rbx
fffff803`0fc63a65 48896c2410      mov     qword ptr [rsp+10h],rbp
fffff803`0fc63a6a 4889742418      mov     qword ptr [rsp+18h],rsi
fffff803`0fc63a6f 57             push    rdi
fffff803`0fc63a70 4154           push    r12
fffff803`0fc63a72 4155           push    r13
fffff803`0fc63a74 4156           push    r14
fffff803`0fc63a76 4157           push    r15
fffff803`0fc63a78 4883ec50       sub     rsp,50h
fffff803`0fc63a7c 488b057d0c5100 mov     rax,qword ptr [nt!KiWaitNever
(fffff803`10174700)]
fffff803`0fc63a83 488bf9         mov     rdi,rcx
fffff803`0fc63a86 488b35630e5100 mov     rsi,qword ptr [nt!KiWaitAlways
(fffff803`101748f0)]
fffff803`0fc63a8d 410fb6e9       movzx   ebp,r9b
fffff803`0fc63a91 4c8bac24a0000000 mov     r13,qword ptr [rsp+0A0h]
fffff803`0fc63a99 458bf8         mov     r15d,r8d
fffff803`0fc63a9c 4933f5         xor     rsi,r13
fffff803`0fc63a9f 488bda         mov     rbx,rdx
fffff803`0fc63aa2 480fce         bswap   rsi
fffff803`0fc63aa5 4833f1         xor     rsi,rcx
fffff803`0fc63aa8 8bc8           mov     ecx,eax
fffff803`0fc63aaa 48d3ce         ror     rsi,cl
fffff803`0fc63aad 4833f0         xor     rsi,rax
fffff803`0fc63ab0 440f20c1       mov     rcx,cr8
fffff803`0fc63ab4 48898c24a0000000 mov     qword ptr [rsp+0A0h],rcx
fffff803`0fc63abc b802000000     mov     eax,2
fffff803`0fc63ac1 440f22c0       mov     cr8,rax
fffff803`0fc63ac5 8b05dd0a5100   mov     eax,dword ptr [nt!KiIrqlFlags
(fffff803`101745a8)]
fffff803`0fc63acb 85c0           test    eax,eax
fffff803`0fc63acd 0f85b72d1a00   jne     nt!KiSetTimerEx+0x1a2e2a
(fffff803`0fe0688a)
fffff803`0fc63ad3 654c8b342520000000 mov     r14,qword ptr gs:[20h]
fffff803`0fc63adc 33d2           xor     edx,edx
fffff803`0fc63ade 488bcf         mov     rcx,rdi
fffff803`0fc63ae1 e86aa2fdff     call    nt!KiCancelTimer (fffff803`0fc3dd50)
fffff803`0fc63ae6 440fb6e0       movzx   r12d,a1
fffff803`0fc63aea 48897730       mov     qword ptr [rdi+30h],rsi
fffff803`0fc63aee 33c0           xor     eax,eax
fffff803`0fc63af0 44897f3c       mov     dword ptr [rdi+3Ch],r15d
fffff803`0fc63af4 8b0f           mov     ecx,dword ptr [rdi]
fffff803`0fc63af6 4889442430     mov     qword ptr [rsp+30h],rax
fffff803`0fc63afb 894c2430       mov     dword ptr [rsp+30h],ecx
fffff803`0fc63aff 488bcb         mov     rcx,rbx
fffff803`0fc63b02 48c1e920       shr     rcx,20h
fffff803`0fc63b06 4889442438     mov     qword ptr [rsp+38h],rax
fffff803`0fc63b0b 4889442440     mov     qword ptr [rsp+40h],rax
fffff803`0fc63b10 40886c2431     mov     byte ptr [rsp+31h],bp1
fffff803`0fc63b15 85c9           test    ecx,ecx
fffff803`0fc63b17 0f89c0000000   jns     nt!KiSetTimerEx+0x17d
(fffff803`0fc63bdd)
fffff803`0fc63b1d 33c9           xor     ecx,ecx
fffff803`0fc63b1f 8bd1           mov     edx,ecx

```

```

fffff803`0fc63b21 40f6c5fc      test    bp1,0FCh
fffff803`0fc63b25 0f85a3000000  jne     nt!KiSetTimerEx+0x16e
(fffff803`0fc63bce)
fffff803`0fc63b2b 48894c2420     mov     qword ptr [rsp+20h],rcx
fffff803`0fc63b30 48b80800000080f7ffff mov rax,0FFFFFF78000000008h
fffff803`0fc63b3a 4d8bc5        mov     r8,r13
fffff803`0fc63b3d 488b00        mov     rax,qword ptr [rax]
fffff803`0fc63b40 804c243340     or      byte ptr [rsp+33h],40h
fffff803`0fc63b45 482bc3        sub     rax,rbx
fffff803`0fc63b48 48894718      mov     qword ptr [rdi+18h],rax
fffff803`0fc63b4c 4803c2        add     rax,rdx
fffff803`0fc63b4f 48c1e812      shr     rax,12h
fffff803`0fc63b53 488bd7        mov     rdx,rdi
fffff803`0fc63b56 440fb6c8      movzx   r9d,a1
fffff803`0fc63b5a 44884c2432     mov     byte ptr [rsp+32h],r9b
fffff803`0fc63b5f 8b442430      mov     eax,dword ptr [rsp+30h]
fffff803`0fc63b63 8907          mov     dword ptr [rdi],eax
fffff803`0fc63b65 894f04        mov     dword ptr [rdi+4],ecx
fffff803`0fc63b68 498bce        mov     rcx,r14
fffff803`0fc63b6b e8209ffdf     call    nt!KiInsertTimerTable
(fffff803`0fc3da90)
fffff803`0fc63b70 84c0          test    al,al
fffff803`0fc63b72 0f8495000000  je      nt!KiSetTimerEx+0x1ad
(fffff803`0fc63c0d)
fffff803`0fc63b78 f7058608510000000200 test dword ptr
[nt!PerfGlobalGroupMask+0x8 (fffff803`10174408)],20000h
fffff803`0fc63b82 0f852f2d1a00  jne     nt!KiSetTimerEx+0x1a2e57
(fffff803`0fe068b7)
fffff803`0fc63b88 f081277fffffff lock and dword ptr [rdi],0FFFFFF7Fh
fffff803`0fc63b8f 488b8424a0000000 mov rax,qword ptr [rsp+0A0h]
fffff803`0fc63b97 4533c9        xor     r9d,r9d
fffff803`0fc63b9a 33d2          xor     edx,edx
fffff803`0fc63b9c 88442420      mov     byte ptr [rsp+20h],a1
fffff803`0fc63ba0 498bce        mov     rcx,r14
fffff803`0fc63ba3 458d4101      lea     r8d,[r9+1]
fffff803`0fc63ba7 e8044efeff    call    nt!KiExitDispatcher
(fffff803`0fc489b0)

```

如上汇编代码 `KiSetTimerEx` 中就是DPC加密细节，如果需要解密只需要逆操作即可，此处我就具体分析下加密细节，分析这个东西我建议你使用记事本带着色的。

分析思路是这样的，首先这里要传入待加密的DPC数据，然后经过 `KiwaitNever` 和 `KiwaitAlways` 对数据进行 `xor`, `ror`, `bswap` 等操作。


```

1 0: kd> u nt!KiSetTimerEx 150
2 nt!KiSetTimerEx:
3 fffff803`0fc63a60 48895c2408 mov qword ptr [rsp+8],rbx
4 fffff803`0fc63a65 48896c2410 mov qword ptr [rsp+10h],rbp
5 fffff803`0fc63a6a 4889742418 mov qword ptr [rsp+18h],rsi
6 fffff803`0fc63a6f 57 push rdi
7 fffff803`0fc63a70 4154 push r12
8 fffff803`0fc63a72 4155 push r13
9 fffff803`0fc63a74 4156 push r14
10 fffff803`0fc63a76 4157 push r15
11 fffff803`0fc63a78 4883ec50 sub rsp,50h
12 fffff803`0fc63a7c 488b057d0c5100 mov rax,qword ptr [nt!KiWaitNever (fffff803`10174700)]
13 fffff803`0fc63a83 488bf9 mov rdi,rcx
14 fffff803`0fc63a86 488b35630e5100 mov rsi,qword ptr [nt!KiWaitAlways (fffff803`101748f0)]
15 fffff803`0fc63a8d 410fb6e9 movzx ebp,r9b
16 fffff803`0fc63a91 4c8bac24a0000000 mov r13,qword ptr [rsp+0A0h]
17 fffff803`0fc63a99 458bf8 mov r15d,r8d
18 fffff803`0fc63a9c 4933f5 xor rsi,r13
19 fffff803`0fc63a9f 488bda mov rbx,rdx
20 fffff803`0fc63aa2 480fce bswap rsi
21 fffff803`0fc63aa5 4833f1 xor rsi,rcx
22 fffff803`0fc63aa8 8bc8 mov ecx,eax
23 fffff803`0fc63aaa 48d3ce ror rsi,cl
24 fffff803`0fc63aad 4833f0 xor rsi,rax
25 fffff803`0fc63ab0 440f20c1 mov rcx,cr8

```

将解密流程通过代码的方式实现。

```

#include <ntddk.h>
#include <ntstrsafe.h>

// 解密DPC
void DPC_Print(PKTIMER ptrTimer)
{
    ULONG_PTR ptrDpc = (ULONG_PTR)ptrTimer->Dpc;
    KDPC* DecDpc = NULL;
    DWORD nshift = (p2dq(ptrKiWaitNever) & 0xFF);

    // _RSI->Dpc = (_KDPC *)v19;
    // _RSI = Timer;
    ptrDpc ^= p2dq(ptrKiWaitNever); // v19 = KiWaitNever ^ v18;
    ptrDpc = _rotl64(ptrDpc, nShift); // v18 = __ROR8__((unsigned
__int64)Timer ^ _RBX, KiWaitNever);
    ptrDpc ^= (ULONG_PTR)ptrTimer;
    ptrDpc = _byteswap_uint64(ptrDpc); // __asm { bswap rbx }
    ptrDpc ^= p2dq(ptrKiWaitAlways); // _RBX = (unsigned __int64)DPC ^
KiWaitAlways;

    // real DPC
    if (MmIsAddressValid((PVOID)ptrDpc))
    {
        DecDpc = (KDPC*)ptrDpc;
        DbgPrint("DPC = %p | routine = %p \n", DecDpc, DecDpc->DeferredRoutine);
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{

```



```

DbgPrint("hello lyshark.com");

PKTIMER ptrTimer = NULL;

DPC_Print(ptrTimer);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

接着将这些功能通过代码实现，首先得到我们需要的函数地址，这些地址包括。

```

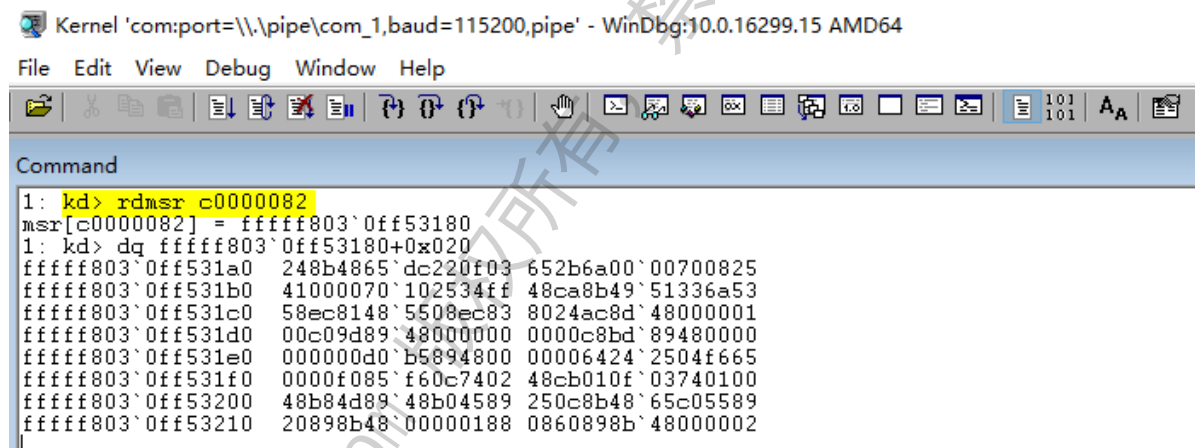
ULONG_PTR ptrKiProcessorBlock = 0xffffffff80770a32cc0;
ULONG_PTR ptrOffsetKTimerTable = 0x3680;
ULONG_PTR ptrKiWaitNever = 0xffffffff80770a316f8;
ULONG_PTR ptrKiWaitAlways = 0xffffffff80770a318e8;

```

此处我把它分为三步走，第一步找到 `KiProcessorBlock` 函数地址，第二步找到 `KeSetTimer` 并从里面寻找 `KeSetTimerEx`，第三步根据 `KiSetTimerEx` 地址，搜索到 `KiWaitNever()`, `KiWaitAlways()` 这两个函数内存地址，最终循环链表并解密DPC队列。

第一步：找到 `KiProcessorBlock` 函数地址，该地址可通过 `__readmsr()` 寄存器相加偏移得到。

在WinDBG中可以输入 `rdmsr c0000082` 得到MSR地址。



```

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg!10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
1: kd> rdmsr c0000082
msr[c0000082] = fffff803`0ff53180
1: kd> dq fffff803`0ff53180+0x020
fffff803`0ff531a0 248b4865`dc220f03 652b6a00`00700825
fffff803`0ff531b0 41000070`102534ff 48ca8b49`51336a53
fffff803`0ff531c0 58ec8148`5508ec83 8024ac8d`48000001
fffff803`0ff531d0 00c09d89`48000000 0000c8bd`89480000
fffff803`0ff531e0 000000d0`b5894800 00006424`2504f665
fffff803`0ff531f0 0000f085`f60c7402 48cb010f`03740100
fffff803`0ff53200 48b84d89`48b04589 250c8b48`65c05589
fffff803`0ff53210 20898b48`00000188 0860898b`48000002

```

MSR寄存器 使用 代码获取 也是很容易，只要找到MSR地址在加上 `0x20` 即可得到 `KiProcessorBlock` 的地址了。

```

/*
lyshark.com 0: kd> dp !KiProcessorBlock
fffff807`70a32cc0 fffff807`6f77c180 fffffbe81`3cee0180
fffff807`70a32cd0 00000000`00000000 00000000`00000000
fffff807`70a32ce0 00000000`00000000 00000000`00000000
fffff807`70a32cf0 00000000`00000000 00000000`00000000
fffff807`70a32d00 00000000`00000000 00000000`00000000
fffff807`70a32d10 00000000`00000000 00000000`00000000
fffff807`70a32d20 00000000`00000000 00000000`00000000
fffff807`70a32d30 00000000`00000000 00000000`00000000
*/

#include <ntddk.h>
#include <ntstrsafe.h>

```

```

// 得到KiProcessorBlock地址
ULONG64 GetKiProcessorBlock()
{
    ULONG64 PrcbAddress = 0;
    PrcbAddress = (ULONG64)__readmsr(0xc0000101) + 0x20;

    if (PrcbAddress != 0)
    {
        // PrcbAddress 是一个地址 这个地址存放了某个 CPU 的 _KPRCB 的地址
        return *(ULONG_PTR*)PrcbAddress;
    }
    return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONG64 address = GetKiProcessorBlock();
    if (address != 0)
    {
        DbgPrint("KiProcessorBlock = %p \n", address);
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行后即可得到输出效果如下：

#	Time	Debug Print
51	28.22346497	hello lyshark.com
52	28.22807693	KiProcessorBlock = FFFFB301976A5180
53	28.42008972	2779375000 - STORMINI: StorNVMe - POWER: IDLE
54	28.99872971	卸载完成...

第二步：找到 KeSetTimer 从里面搜索特征得到 call KeSetTimerEx 函数地址，还记得《驱动开发：内核枚举IoTimer定时器》中我们采用的特征码定位方式吗，没错本次还要使用这个方法，我们此处需要搜索到 e80c000000 这段特征。

```

/*
lyshark.com 0: kd> uf KeSetTimer
nt!KeSetTimer:
fffff807`70520a30 4883ec38      sub     rsp,38h
fffff807`70520a34 4c89442420    mov     qword ptr [rsp+20h],r8
fffff807`70520a39 4533c9       xor     r9d,r9d

```

```

fffff807`70520a3c 4533c0      xor     r8d,r8d
fffff807`70520a3f e80c000000    call    nt!KiSetTimerEx (fffff807`70520a50)
fffff807`70520a44 4883c438      add     rsp,38h
fffff807`70520a48 c3            ret

```

```
*/
```

```
#include <ntddk.h>
```

```
#include <ntstrsafe.h>
```

```
// 得到KiProcessorBlock地址
```

```
ULONG64 GetKeSetTimerEx()
```

```
{
```

```
    // 获取 KeSetTimer 地址
```

```
    ULONG64 ul_KeSetTimer = 0;
```

```
    UNICODE_STRING uc_KeSetTimer = { 0 };
```

```
    RtlInitUnicodeString(&uc_KeSetTimer, L"KeSetTimer");
```

```
    ul_KeSetTimer = (ULONG64)MmGetSystemRoutineAddress(&uc_KeSetTimer);
```

```
    if (ul_KeSetTimer == 0)
```

```
    {
```

```
        return 0;
```

```
    }
```

```
    // 前 30 字节找 call 指令
```

```
    BOOLEAN b_e8 = FALSE;
```

```
    ULONG64 ul_e8Addr = 0;
```

```
    for (INT i = 0; i < 30; i++)
```

```
    {
```

```
        // 验证地址是否可读写
```

```
        if (!MmIsAddressValid((PVOID64)ul_KeSetTimer))
```

```
        {
```

```
            continue;
```

```
        }
```

```
        // e8 0c 00 00 00 call nt!KiSetTimerEx (fffff807`70520a50)
```

```
        if (*(PUCHAR)(ul_KeSetTimer + i) == 0xe8)
```

```
        {
```

```
            b_e8 = TRUE;
```

```
            ul_e8Addr = ul_KeSetTimer + i;
```

```
            break;
```

```
        }
```

```
    }
```

```
// 找到 call 则解析目的地址
```

```
if (b_e8 == TRUE)
```

```
{
```

```
    if (!MmIsAddressValid((PVOID64)ul_e8Addr))
```

```
    {
```

```
        return 0;
```

```
    }
```

```
    INT ul_callCode = *(INT*)(ul_e8Addr + 1);
```

```
    ULONG64 ul_KiSetTimerEx = ul_e8Addr + ul_callCode + 5;
```

```
    return ul_KiSetTimerEx;
```

```
}
```

```

    return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

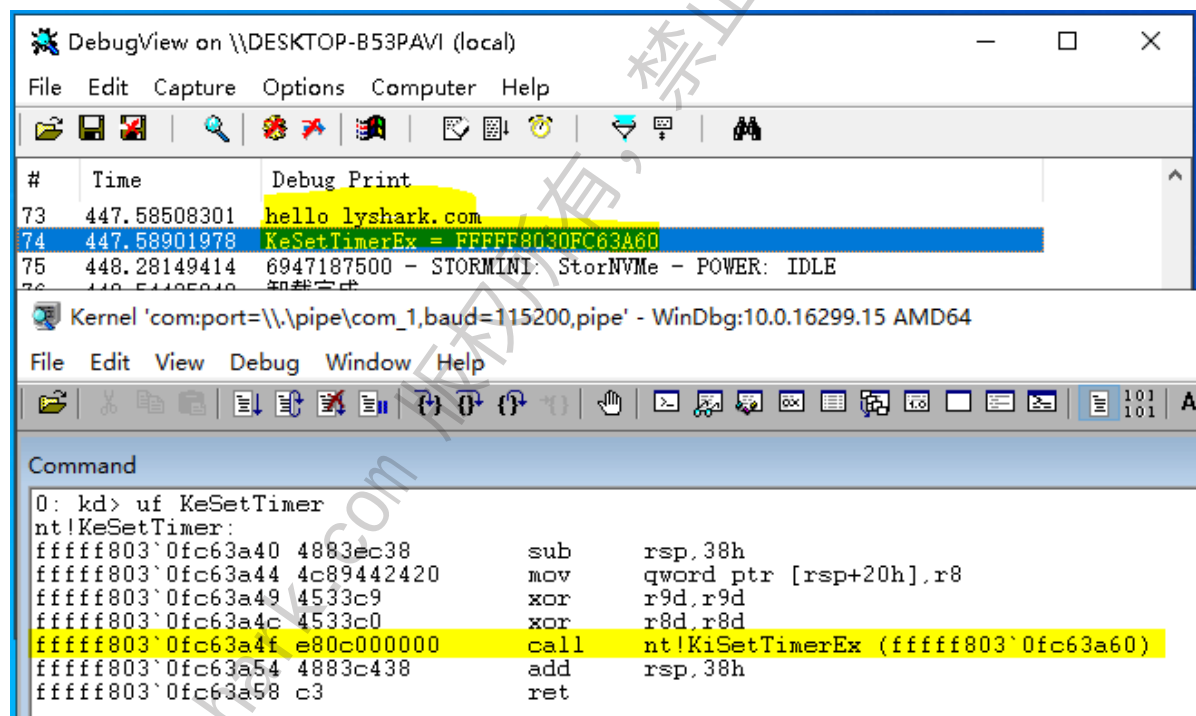
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONG64 address = GetKeSetTimerEx();
    if (address != 0)
    {
        DbgPrint("KeSetTimerEx = %p \n", address);
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

输出寻找CALL地址效果图如下：



第三步：也是最重要的一步，在KiSetTimerEx 里面，搜索特征，拿到里面的KiWaitNever(),KiWaitAlways() 这两个函数地址。

- 488b05850c5100 KiWaitNever
- 488b356b0e5100 KiWaitAlways

这个过程需要重复搜索，所以要把第一步和第二部过程归纳起来，具体代码如下所示。

```

/*
0: kd> uf KiSetTimerEx
nt!KiSetTimerEx:
ffffff807`70520a50 48895c2408    mov     qword ptr [rsp+8],rbx

```

```

fffff807`70520a55 48896c2410    mov     qword ptr [rsp+10h],rbp
fffff807`70520a5a 4889742418    mov     qword ptr [rsp+18h],rsi
fffff807`70520a5f 57            push    rdi
fffff807`70520a60 4154          push    r12
fffff807`70520a62 4155          push    r13
fffff807`70520a64 4156          push    r14
fffff807`70520a66 4157          push    r15
fffff807`70520a68 4883ec50     sub     rsp,50h
fffff807`70520a6c 488b05850c5100 mov     rax,qword ptr [nt!KiWaitNever
(fffff807`70a316f8)]
fffff807`70520a73 488bf9       mov     rdi,rcx
fffff807`70520a76 488b356b0e5100 mov     rsi,qword ptr [nt!KiWaitAlways
(fffff807`70a318e8)]
fffff807`70520a7d 410fb6e9     movzx   ebp,r9b
*/

#include <ntddk.h>
#include <ntstrsafe.h>

// 得到KiProcessorBlock地址
ULONG64 GetKeSetTimerEx()
{
    // 获取 KeSetTimer 地址
    ULONG64 ul_KeSetTimer = 0;
    UNICODE_STRING uc_KeSetTimer = { 0 };
    RtlInitUnicodeString(&uc_KeSetTimer, L"KeSetTimer");
    ul_KeSetTimer = (ULONG64)MmGetSystemRoutineAddress(&uc_KeSetTimer);
    if (ul_KeSetTimer == 0)
    {
        return 0;
    }

    // 前 30 字节找 call 指令
    BOOLEAN b_e8 = FALSE;
    ULONG64 ul_e8Addr = 0;

    for (INT i = 0; i < 30; i++)
    {
        // 验证地址是否可读写
        if (!MmIsAddressValid((PVOID64)ul_KeSetTimer))
        {
            continue;
        }

        // e8 0c 00 00 00 call nt!KiSetTimerEx (fffff807`70520a50)
        if (*(PUCHAR)(ul_KeSetTimer + i) == 0xe8)
        {
            b_e8 = TRUE;
            ul_e8Addr = ul_KeSetTimer + i;
            break;
        }
    }

    // 找到 call 则解析目的地址
    if (b_e8 == TRUE)

```

```

{
    if (!MmIsValidAddress((PVOID64)u1_e8Addr))
    {
        return 0;
    }

    INT u1_callCode = *(INT*)(u1_e8Addr + 1);
    ULONG64 u1_KiSetTimerEx = u1_e8Addr + u1_callCode + 5;
    return u1_KiSetTimerEx;
}
return 0;
}

// 得到KiWaitNever地址
ULONG64 GetKiWaitNever(ULONG64 address)
{
    // 验证地址是否可读写
    if (!MmIsValidAddress((PVOID64)address))
    {
        return 0;
    }

    // 前 100 字节找 找 KiWaitNever
    for (INT i = 0; i < 100; i++)
    {
        // 48 8b 05 85 0c 51 00 | mov rax, qword
        ptr[nt!KiWaitNever(fffff807`70a316f8)]
        if (*(PCHAR)(address + i) == 0x48 && *(PCHAR)(address + i + 1) == 0x8b &&
            *(PCHAR)(address + i + 2) == 0x05)
        {
            ULONG64 u1_movCode = *(UINT32*)(address + i + 3);
            ULONG64 u1_movAddr = address + i + u1_movCode + 7;
            // DbgPrint("找到KiWaitNever地址: %p \n", u1_movAddr);
            return u1_movAddr;
        }
    }
    return 0;
}

// 得到KiWaitAlways地址
ULONG64 GetKiWaitAlways(ULONG64 address)
{
    // 验证地址是否可读写
    if (!MmIsValidAddress((PVOID64)address))
    {
        return 0;
    }

    // 前 100 字节找 找 KiWaitNever
    for (INT i = 0; i < 100; i++)
    {
        // 48 8b 35 6b 0e 51 00 | mov rsi, qword ptr [nt!KiWaitAlways
        (fffff807`70a318e8)]
        if (*(PCHAR)(address + i) == 0x48 && *(PCHAR)(address + i + 1) == 0x8b &&
            *(PCHAR)(address + i + 2) == 0x35)
    }

```

```

    {
        ULONG64 u1_movCode = *(UINT32*)(address + i + 3);
        ULONG64 u1_movAddr = address + i + u1_movCode + 7;
        return u1_movAddr;
    }
}
return 0;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    ULONG64 address = GetKeSetTimerEx();
    if (address != 0)
    {
        ULONG64 KiWaitNeverAddress = GetKiWaitNever(address);
        DbgPrint("KiWaitNeverAddress = %p \n", KiWaitNeverAddress);

        ULONG64 KiWaitAlwaysAddress = GetKiWaitAlways(address);
        DbgPrint("KiWaitAlwaysAddress = %p \n", KiWaitAlwaysAddress);
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行这个程序，我们看下寻找到的地址是否与WinDBG中找到的地址一致。

The screenshot shows two windows. The top window is DebugView, displaying a log of kernel messages. The bottom window is WinDBG, showing the assembly code for the function nt!KiSetTimerEx.

DebugView Log:

#	Time	Debug Print
138	828.93640137	7674062500 - STORMINI: StorNVMe - POWER: ACTIVE
139	829.67199707	hello lyshark.com
140	829.67639160	KiWaitNeverAddress = FFFFF80310174700
141	829.68182373	KiWaitAlwaysAddress = FFFFF803101748F0
142	830.06951904	7684062500 - STORMINI: StorNVMe - POWER: IDLE

WinDBG Assembly:

```

1: kd> uf KiSetTimerEx
nt!KiSetTimerEx:
fffff803`0fc63a60 48895c2408      mov     qword ptr [rsp+8],rbx
fffff803`0fc63a65 48896c2410      mov     qword ptr [rsp+10h],rbp
fffff803`0fc63a6a 4889742418      mov     qword ptr [rsp+18h],rsi
fffff803`0fc63a6f 57             push    rdi
fffff803`0fc63a70 4154           push    r12
fffff803`0fc63a72 4155           push    r13
fffff803`0fc63a74 4156           push    r14
fffff803`0fc63a76 4157           push    r15
fffff803`0fc63a78 4883ec50       sub     rsp,50h
fffff803`0fc63a7c 488b057d0c5100 mov     rax,qword ptr [nt!KiWaitNever (fffff803`10174700)]
fffff803`0fc63a83 488bf9         mov     rdi,rcx
fffff803`0fc63a86 488b35630e5100 mov     rsi,qword ptr [nt!KiWaitAlways (fffff803`101748f0)]
fffff803`0fc63a8d 410fb6e9       movzx   ebp,r9b
fffff803`0fc63a91 4c8bac24a0000000 mov     r13,qword ptr [rsp+0A0h]
fffff803`0fc63a99 458bf8         mov     r15d,r8d

```

功能实现部分：最后将这些功能整合在一起，循环输出链表元素，并解密元素即可实现枚举当前系统DPC定时器。

代码核心API分析：

- KeNumberProcessors 得到CPU数量(内核常量)
- KeSetSystemAffinityThread 线程绑定到特定CPU上
- GetKiProcessorBlock 获得KPRCB的地址
- KeRevertToUserAffinityThread 取消绑定CPU

解密部分提取出 KiWaitNever 和 KiWaitAlways 用于解密计算，转换 PKDPC 对象结构，并输出即可。

```
// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

#include <Fltkernel.h>
#include <ntddk.h>
#include <intrin.h>

#define IRP_TEST CTL_CODE(FILE_DEVICE_UNKNOWN, 0x800, METHOD_BUFFERED,
FILE_ANY_ACCESS)

UNICODE_STRING name_device;           // 设备名
UNICODE_STRING name_symbol;          // 符号链接
PDEVICE_OBJECT deviceObj;            // 设备对象

typedef struct _KTIMER_TABLE_ENTRY
{
    ULONG_PTR    Lock;
    LIST_ENTRY   Entry;
    ULONG_PTR    Time;
}KTIMER_TABLE_ENTRY, *PKTIMER_TABLE_ENTRY;

typedef struct _KTIMER_TABLE
{
    ULONG_PTR    TimerExpiry[64];
    KTIMER_TABLE_ENTRY TimerEntries[256];
}KTIMER_TABLE, *PKTIMER_TABLE;

BOOLEAN get_KiWait(PULONG64 never, PULONG64 always)
{
    // 获取 KeSetTimer 地址
    ULONG64 ul_KeSetTimer = 0;
    UNICODE_STRING uc_KeSetTimer = { 0 };
    RtlInitUnicodeString(&uc_KeSetTimer, L"KeSetTimer");
    ul_KeSetTimer = (ULONG64)MmGetSystemRoutineAddress(&uc_KeSetTimer);
    if (ul_KeSetTimer == NULL)
    {
        return FALSE;
    }

    // 前 30 字节找 call 指令
    BOOLEAN b_e8 = FALSE;
    ULONG64 ul_e8Addr = 0;
    for (INT i = 0; i < 30; i++)
```



```

{
    if (!MmIsAddressValid((PVOID64)ul_KeSetTimer))
    {
        continue;
    }

    /*
    0: kd> u nt!KeSetTimer
    nt!KeSetTimer:
    ffffff803`0fc63a40 4883ec38      sub     rsp,38h
    ffffff803`0fc63a44 4c89442420      mov     qword ptr [rsp+20h],r8
    ffffff803`0fc63a49 4533c9         xor     r9d,r9d
    ffffff803`0fc63a4c 4533c0         xor     r8d,r8d
    ffffff803`0fc63a4f e80c000000     call    nt!KiSetTimerEx
(ffffff803`0fc63a60)
    ffffff803`0fc63a54 4883c438      add     rsp,38h
    ffffff803`0fc63a58 c3             ret
    ffffff803`0fc63a59 cc             int     3
    */

    // ffffff803`0fc63a4f e8 0c 00 00 00      call    nt!KiSetTimerEx
(ffffff803`0fc63a60)
    if (*(PUCHAR)(ul_KeSetTimer + i) == 0xe8)
    {
        b_e8 = TRUE;
        ul_e8Addr = ul_KeSetTimer + i;
        break;
    }
}

// 找到 call 则解析目的地址
/*
0: kd> u nt!KiSetTimerEx 120
nt!KiSetTimerEx:
fffff803`0fc63a60 48895c2408      mov     qword ptr [rsp+8],rbx
fffff803`0fc63a65 48896c2410      mov     qword ptr [rsp+10h],rbp
fffff803`0fc63a6a 4889742418      mov     qword ptr [rsp+18h],rsi
fffff803`0fc63a6f 57             push    rdi
fffff803`0fc63a70 4154           push    r12
fffff803`0fc63a72 4155           push    r13
fffff803`0fc63a74 4156           push    r14
fffff803`0fc63a76 4157           push    r15
fffff803`0fc63a78 4883ec50      sub     rsp,50h
fffff803`0fc63a7c 488b057d0c5100 mov     rax,qword ptr [nt!KiwaitNever
(ffffff803`10174700)]
fffff803`0fc63a83 488bf9         mov     rdi,rcx
    */

ULONG64 ul_KiSetTimerEx = 0;
if (b_e8 == TRUE)
{
    if (!MmIsAddressValid((PVOID64)ul_e8Addr))
    {
        return FALSE;
    }
}

```

```

    INT u1_callCode = *(INT*)(u1_e8Addr + 1);
    ULONG64 u1_callAddr = u1_e8Addr + u1_callCode + 5;
    u1_kiSetTimerEx = u1_callAddr;
}

// 没有 call 则直接在当前函数找
else
{
    u1_kiSetTimerEx = u1_keSetTimer;
}

// 前 50 字节找 找 kiwaitNever 和 kiwaitAlways
/*
0: kd> u nt!KiSetTimerEx l20
nt!KiSetTimerEx:
fffff803`0fc63a60 48895c2408      mov     qword ptr [rsp+8],rbx
fffff803`0fc63a65 48896c2410      mov     qword ptr [rsp+10h],rbp
fffff803`0fc63a6a 4889742418      mov     qword ptr [rsp+18h],rsi
fffff803`0fc63a6f 57             push    rdi
fffff803`0fc63a70 4154           push    r12
fffff803`0fc63a72 4155           push    r13
fffff803`0fc63a74 4156           push    r14
fffff803`0fc63a76 4157           push    r15
fffff803`0fc63a78 4883ec50       sub     rsp,50h
fffff803`0fc63a7c 488b057d0c5100 mov     rax,qword ptr [nt!KiwaitNever
(fffff803`10174700)]
fffff803`0fc63a83 488bf9         mov     rdi,rcx
fffff803`0fc63a86 488b35630e5100 mov     rsi,qword ptr [nt!KiwaitAlways
(fffff803`101748f0)]
*/
if (!MmIsAddressValid((PVOID64)u1_kiSetTimerEx))
{
    return FALSE;
}

ULONG64 u1_arr_ret[2];          // 存放 kiwaitNever 和 kiwaitAlways 的地址
INT i_sub = 0;                  // 对应 u1_arr_ret 的下标
for (INT i = 0; i < 50; i++)
{
    // // fffff803`0fc63a7c 488b057d0c5100 mov     rax,qword ptr
[nt!KiwaitNever (fffff803`10174700)]
    if (*(PUCHAR)(u1_kiSetTimerEx + i) == 0x48 && *(PUCHAR)(u1_kiSetTimerEx
+ i + 1) == 0x8b && *(PUCHAR)(u1_kiSetTimerEx + i + 6) == 0x00)
    {
        ULONG64 u1_movCode = *(UINT32*)(u1_kiSetTimerEx + i + 3);
        ULONG64 u1_movAddr = u1_kiSetTimerEx + i + u1_movCode + 7;

        // 只拿符合条件的前两个值
        if (i_sub < 2)
        {
            u1_arr_ret[i_sub++] = u1_movAddr;
        }
    }
}

*never = u1_arr_ret[0];

```

```

    *always = ul_arr_ret[1];

    return TRUE;
}

BOOLEAN EnumDpc()
{
    DbgPrint("hello lyshark.com \n");

    // 获取 CPU 核心数
    INT i_cpuNum = KeNumberProcessors;
    DbgPrint("CPU核心数: %d \n", i_cpuNum);

    for (KAFFINITY i = 0; i < i_cpuNum; i++)
    {
        // 线程绑定特定 CPU
        KeSetSystemAffinityThread(i + 1);

        // 获得 KPRCB 的地址
        ULONG64 p_PRCB = (ULONG64)__readmsr(0xc0000101) + 0x20;
        if (!MmIsValidAddress((PVOID64)p_PRCB))
        {
            return FALSE;
        }

        // 取消绑定 CPU
        KeRevertToUserAffinityThread();

        // 计算 TimerTable 在 _KPRCB 结构中的偏移
        PKTIMER_TABLE p_TimeTable = NULL;

        // windows 10 得到_KPRCB + 0x3680
        p_TimeTable = (PKTIMER_TABLE)((PULONG64)p_PRCB + 0x3680);

        // 遍历 TimerEntries[] 数组 (大小 256)
        for (INT j = 0; j < 256; j++)
        {
            // 获取 Entry 双向链表地址
            if (!MmIsValidAddress((PVOID64)p_TimeTable))
            {
                continue;
            }

            PLIST_ENTRY p_ListEntryHead = &(p_TimeTable->TimerEntries[j].Entry);

            // 遍历 Entry 双向链表
            for (PLIST_ENTRY p_ListEntry = p_ListEntryHead->Flink; p_ListEntry
            != p_ListEntryHead; p_ListEntry = p_ListEntry->Flink)
            {
                // 根据 Entry 取 _KTIMER 对象地址
                if (!MmIsValidAddress((PVOID64)p_ListEntry))
                {
                    continue;
                }
            }
        }
    }
}

```

```

        PKTIMER p_Timer = CONTAINING_RECORD(p_ListEntry, KTIMER,
TimerListEntry);

        // 硬编码取 KiwaitNever 和 KiwaitAlways
        ULONG64 never = 0, always = 0;
        if (get_Kiwait(&never, &always) == FALSE)
        {
            return FALSE;
        }

        // 获取解密前的 Dpc 对象
        if (!MmIsValidAddress((PVOID64)p_Timer))
        {
            continue;
        }

        ULONG64 ul_Dpc = (ULONG64)p_Timer->Dpc;
        INT i_Shift = (*((PULONG64)never) & 0xFF);

        // 解密 Dpc 对象
        ul_Dpc ^= *((ULONG_PTR*)never); // 异或
        ul_Dpc = _rotl64(ul_Dpc, i_Shift); // 循环左移
        ul_Dpc ^= (ULONG_PTR)p_Timer; // 异或
        ul_Dpc = _byteswap_uint64(ul_Dpc); // 颠倒顺序
        ul_Dpc ^= *((ULONG_PTR*)always); // 异或

        // 对象类型转换
        PKDPC p_Dpc = (PKDPC)ul_Dpc;

        // 打印验证
        if (!MmIsValidAddress((PVOID64)p_Dpc))
        {
            continue;
        }

        DbgPrint("定时器对象: 0x%p | 函数入口: 0x%p | 触发周期: %d \n ",
p_Timer, p_Dpc->DeferredRoutine, p_Timer->Period);
    }
}

return TRUE;
}

// 对应 IRP_MJ_DEVICE_CONTROL
NTSTATUS myIrpControl(IN PDEVICE_OBJECT pDevObj, IN PIRP pIRP)
{
    // 获取 IRP 对应的 I/O 堆栈指针
    PIO_STACK_LOCATION stack = IoGetCurrentIrpStackLocation(pIRP);

    // 得到输入缓冲区大小
    ULONG cbin = stack->Parameters.DeviceIoControl.InputBufferLength;

    // 得到输出缓冲区大小
    ULONG cbout = stack->Parameters.DeviceIoControl.OutputBufferLength;

```

```

// 得到 IOCTL 码
ULONG code = stack->Parameters.DeviceIoControl.IoControlCode;

// 捕获 I/O 操作类型 (MajorFunction)
switch (code)
{
case IRP_TEST:
{
break;
}
default:
break;
}

// 完成 IO 请求
IoCompleteRequest(pIRP, IO_NO_INCREMENT);

return STATUS_SUCCESS;
}

// 对应 IRP_MJ_CREATE 、 IRP_MJ_CLOSE
NTSTATUS dpc_CAC(IN PDEVICE_OBJECT pDevObj, IN PIRP pIRP)
{
// 将 IRP 返回给 I/O 管理器
IoCompleteRequest(
    pIRP, // IRP 指针
    IO_NO_INCREMENT // 线程优先级, IO_NO_INCREMENT : 不增加优先级
);

// 设置 I/O 请求状态
pIRP->IoStatus.Status = STATUS_SUCCESS;

// 设置 I/O 请求传输的字节数
pIRP->IoStatus.Information = 0;

return STATUS_SUCCESS;
}

NTSTATUS CreateDevice(IN PDRIVER_OBJECT DriverObject)
{
// 定义返回值
NTSTATUS status;

// 初始化设备名
RtlInitUnicodeString(&name_device, L"\\Device\\LySharkDriver");

// 创建设备
status = IoCreateDevice(
    DriverObject, // 指向驱动对象的指针
    0, // 设备扩展分配的字节数
    &name_device, // 设备名
    FILE_DEVICE_UNKNOWN, // 设备类型
    0, // 驱动设备附加信息
    TRUE, // 设备对象是否独占设备

```

```

        &deviceObj                                // 设备对象指针
    );

    if (!NT_SUCCESS(status))
    {
        return status;
    }

    // 初始化符号链接名
    RtlInitUnicodeString(&name_symbol, L"\\??\\LySharkDriver");

    // 创建符号链接
    status = IoCreateSymbolicLink(&name_symbol, &name_device);
    if (!NT_SUCCESS(status))
    {
        return status;
    }

    return STATUS_SUCCESS;
}

NTSTATUS DriverUnload(IN PDRIVER_OBJECT DriverObject)
{
    // 定义返回值
    NTSTATUS status;

    // 删除符号链接
    status = IoDeleteSymbolicLink(&name_symbol);
    if (!NT_SUCCESS(status))
    {
        return status;
    }

    // 删除设备
    IoDeleteDevice(deviceObj);

    return STATUS_SUCCESS;
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT DriverObject, IN PUNICODE_STRING
RegistryPath)
{
    // 定义返回值
    NTSTATUS status;

    // 指定驱动卸载函数
    DriverObject->DriverUnload = (PDRIVER_UNLOAD)DriverUnload;

    // 指定派遣函数
    DriverObject->MajorFunction[IRP_MJ_CREATE] = dpc_CAC;
    DriverObject->MajorFunction[IRP_MJ_CLOSE] = dpc_CAC;
    DriverObject->MajorFunction[IRP_MJ_DEVICE_CONTROL] = myIrpControl;

    // 创建设备
    status = CreateDevice(DriverObject);

```

```

if (!NT_SUCCESS(status))
{
    return status;
}

// 执行枚举
EnumDpc();

return STATUS_SUCCESS;
}

```

最终运行枚举程序，你将会看到系统中所有的定时器，与ARK工具对比是一致的。

The screenshot shows the 'DPC定时器' (DPC Timer) tool interface. It displays a table of DPC timers with columns for '定时器对象' (Timer Object), 'DPC', '触发周期(s)' (Trigger Period), '函数入口' (Function Entry), and '模块路径' (Module Path). Below the table, there is a 'DebugView on \\DESKTOP-B53PAVI (local)' window showing a list of debug prints. The debug prints include the timer object address, DPC function address, and trigger period for several timers.

定时器对象	DPC	触发周期(s)	函数入口	模块路径
0xFFFFB38370F396C8	0xFFFFB38370F39708	0	0xFFFFF80621F7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFB38370F3C178	0xFFFFB38370F3C1B8	0	0xFFFFF80621F7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFB38370F39B98	0xFFFFB38370F39BD8	0	0xFFFFF80621F7EB80	C:\Windows\system32\ntoskrnl.exe
0xFFFFF8062230B4C0	0xFFFFF8062230B500	0	0xFFFFF80622757630	C:\Windows\system32\ntoskrnl.exe

#	Time	Debug Print
113	29.17917442	hello lyshark.com
114	29.18279076	CPU核心数: 2
115	29.18798065	定时器对象: 0xFFFFB383721AF048 函数入口: 0xFFFFF80625CB13E0 触发周期: 2000
116	29.18798065	定时器对象: 0xFFFFB383704E0EC8 函数入口: 0xFFFFF80621F332C0 触发周期: 0
117	29.19336319	
118	29.19336319	
119	29.19865990	定时器对象: 0xFFFFF806268FB1E0 函数入口: 0xFFFFF806268F1170 触发周期: 0
120	29.19865990	
121	29.20407677	定时器对象: 0xFFFFF80624D323E0 函数入口: 0xFFFFF80624D24410 触发周期: 0
122	29.20407677	
123	29.21896362	定时器对象: 0xFFFFF806251806E8 函数入口: 0xFFFFF806251777F0 触发周期: 0

8.3 内核枚举PspCidTable句柄表

在上一篇文章《驱动开发：内核枚举DpcTimer定时器》中我们通过枚举特征码的方式找到了 DPC 定时器基址并输出了内核中存在的定时器列表，本章将学习如何通过特征码定位的方式寻找 windows 10 系统下面的 PspCidTable 内核句柄表地址。

首先引入一段基础概念；

- 1.在 windows 下所有的资源都是用对象的方式进行管理的（文件、进程、设备等都是对象），当要访问一个对象时，如打开一个文件，系统就会创建一个对象句柄，通过这个句柄可以对这个文件进行各种操作。
- 2.句柄和对象的联系是通过句柄表来进行的，准确来说一个句柄就是它所对应的对象在句柄表中的索引。
- 3.通过句柄可以在句柄表中找到对象的指针，通过指针就可以对，对象进行操作。

PspCidTable 就是这样的一种表（内核句柄表），表的内部存放的是 进程EPROCESS 和 线程ETHREAD 的内核对象，并通过 进程PID 和 线程TID 进行索引，ID号以4递增，内核句柄表不属于任何进程，也不连接在系统的句柄表上，通过它可以返回系统的任何对象。

内核句柄表与普通句柄表完全一样，但它与每个进程私有的句柄表有以下不同；

- 1.PspCidTable 中存放的对象是系统中所有的进程线程对象，其索引就是 PID 和 TID。
- 2.PspCidTable 中存放的直接是对象体 EPROCESS和ETHREAD，而每个进程私有的句柄表则存放的是对象头 OBJECT_HEADER。

- 3.PspCidTable 是一个独立的句柄表，而每个进程私有的句柄表以一个双链连接起来。
- 4.PspCidTable 访问对象时要掩掉低三位，每个进程私有的句柄表是双链连接起来的。

那么在 windows10 系统中该如何枚举句柄表；

- 1.首先找到 PsLookupProcessByProcessId 函数地址，该函数是被导出的可以动态拿到。
- 2.其次在 PsLookupProcessByProcessId 地址中搜索 PspReferenceCidTableEntry 函数。
- 3.最后在 PspReferenceCidTableEntry 地址中找到 PspCidTable 函数。

首先第一步先要得到 PspCidTable 函数内存地址，输入 `dp PspCidTable` 即可得到，如果在程序中则是调用 `MmGetSystemRoutineAddress` 取到。

```
Command
0: kd> dp PspCidTable
fffff802`08374530 fffffdc88`79605dc0 fffffb281`de2b07a0
fffff802`08374540 00000000`00000000 00010000`00000000
fffff802`08374550 0001017c`00001000 00000000`00009e0a
fffff802`08374560 00000000`00000000 00000000`00000000
fffff802`08374570 00000000`00000000 00000000`00000000
fffff802`08374580 0000002e`00000000 00000000`00000000
fffff802`08374590 fffffb281`de2b1c40 00000000`00000000
fffff802`083745a0 00000000`00000000 00000000`00000000
0: kd> uf PspCidTable
Flow analysis was incomplete, some code may be missing
nt!PspCidTable:
fffff802`08374530 c05d6079      rcr      byte ptr [rbp+60h],79h
fffff802`08374534 88dc      mov      ah,bl
```

PspCidTable是一个 `HANDLE_TABLE` 结构，当新建一个进程时，对应的会在 PspCidTable 存在一个该进程和线程对应的 `HANDLE_TABLE_ENTRY` 项。在 windows10 中依然采用 动态扩展 的方法，当句柄数少的时候就采用下层表，多的时候才启用中层表或上层表。

接着我们解析 `ffffdc88-79605dc0` 这个内存地址，执行 `dt _HANDLE_TABLE 0xffffdc8879605dc0` 得到规范化结构体。

```
Command
0: kd> dt _HANDLE_TABLE 0xffffdc8879605dc0
ntdll!_HANDLE_TABLE
+0x000 NextHandleNeedingPool : 0x1c00
+0x004 ExtraInfoPages : 0n0
+0x008 TableCode : 0xffffdc88`7d09b001
+0x010 QuotaProcess : (null)
+0x018 HandleTableList : LIST_ENTRY [ 0xffffdc88`79605dd8 - 0xffffdc88`79605dd8 ]
+0x028 UniqueProcessId : 0
+0x02c Flags : 1
+0x02c StrictFIFO : 0y1
+0x02c EnableHandleExceptions : 0y0
+0x02c Rundown : 0y0
+0x02c Duplicated : 0y0
+0x02c RaiseUMExceptionOnInvalidHandleClose : 0y0
+0x030 HandleContentionEvent : _EX_PUSH_LOCK
+0x038 HandleTableLock : _EX_PUSH_LOCK
+0x040 FreeLists : [1] _HANDLE_TABLE_FREE_LIST
+0x040 ActualEntry : [32] ""
+0x060 DebugInfo : (null)
```

内核句柄表分为三层如下；

- 下层表：是一个 `HANDLE_TABLE_ENTRY` 项的索引，整个表共有 256 个元素，每个元素是一个 8 个字节长的 `HANDLE_TABLE_ENTRY` 项及索引，`HANDLE_TABLE_ENTRY` 项中保存着指向对象的指针，下层表可以看成是进程和线程的稠密索引。
- 中层表：共有 256 个元素，每个元素是 4 个字节长的指向下层表的入口指针及索引，中层表可以看成是进程和线程的稀疏索引。
- 上层表：共有 256 个元素，每个元素是 4 个字节长的指向中层表的入口指针及索引，上层表可以看成是中层表的稀疏索引。

总结起来一个句柄表有一个上层表，一个上层表最多可以有 256 个中层表的入口指针，每个中层表最多可以有 256 个下层表的入口指针，每个下层表最多可以有 256 个进程和线程对象的指针。PspCidTable 表可以看成是 HANDLE_TBALE_ENTRY 项的多级索引。

如上图所示 TableCode 是指向句柄表的指针，低二位（二进制）记录句柄表的等级：0（00）表示一级表，1（01）表示二级表，2（10）表示三级表。这里的 0xffffdc88-7d09b001 就命名它是一个二级表。

```
Command
0: kd> dp PspCidTable
fffff802`08374530 fffffdc88`79605dc0 fffffb281`de2b07a0
fffff802`08374540 00000000`00000000 00010000`00000000
fffff802`08374550 0001017c`00001000 00000000`00009e0a
fffff802`08374560 00000000`00000000 00000000`00000000
fffff802`08374570 00000000`00000000 00000000`00000000
fffff802`08374580 00000002e`00000000 00000000`00000000
fffff802`08374590 fffffb281`de2b1c40 00000000`00000000
fffff802`083745a0 00000000`00000000 00000000`00000000
0: kd> dt _HANDLE_TABLE fffffdc88`79605dc0
ntdll!_HANDLE_TABLE
+0x000 NextHandleNeedingPool : 0x1c00
+0x004 ExtraInfoPages : 0n0
+0x008 TableCode : 0xffffdc88`7d09b001
+0x010 QuotaProcess : (null)
+0x018 HandleTableList : LIST_ENTRY [ 0xffffdc88`79605dd8 - 0xffffdc88`79605dd8 ]
+0x028 UniqueProcessId : 0
+0x02c Flags : 1
+0x02c StrictFIFO : 0y1
+0x02c EnableHandleExceptions : 0y0
+0x02c Rundown : 0y0
+0x02c Duplicated : 0y0
+0x02c RaiseUMExceptionOnInvalidHandleClose : 0y0
+0x030 HandleContentionEvent : _EX_PUSH_LOCK
+0x038 FreeTableLock : _EX_PUSH_LOCK
+0x040 FreeLists : [1] _HANDLE_TABLE_FREE_LIST
+0x040 ActualEntry : [32] ""
+0x060 DebugInfo : (null)
```

一级表里存放的就是进程和线程对象（加密过的，需要一些计算来解密），二级表里存放的是指向某个一级表的指针，同理三级表存放的是指向二级表的指针。

x64 系统中，每张表的大小是 0x1000（4096），一级表中存放的是 _handle_table_entry 结构（大小 = 16），二级表和三级表存放的是指针（大小 = 8）。

我们对 0xffffdc88-7d09b001 抹去低二位，输入 dp 0xffffdc887d09b000 输出的结果就是一张二级表，里面存储的就是一级表指针。

```
Command
0: kd> dp 0xffffdc887d09b000
ffffdc88`7d09b000 fffffdc88`7962a000 fffffdc88`7d09c000
ffffdc88`7d09b010 fffffdc88`7d4ff000 fffffdc88`7dcff000
ffffdc88`7d09b020 fffffdc88`7e34f000 fffffdc88`7ebf7000
ffffdc88`7d09b030 fffffdc88`7f2f5000 00000000`00000000
ffffdc88`7d09b040 00000000`00000000 00000000`00000000
ffffdc88`7d09b050 00000000`00000000 00000000`00000000
ffffdc88`7d09b060 00000000`00000000 00000000`00000000
ffffdc88`7d09b070 00000000`00000000 00000000`00000000
```

继续查看第一张一级表，输入 dp 0xffffdc887962a000 命令，我们知道一级句柄表是根据进程或线程 ID 来索引的，且以 4 累加，所以第一行对应 id = 0，第二行对应 id = 4。根据尝试，PID = 4 的进程是 System。

```
Command
0: kd> dp 0xffffdc887d09b000
ffffdc88`7d09b000 fffffdc88`7962a000 fffffdc88`7d09c000
ffffdc88`7d09b010 fffffdc88`7d4ff000 fffffdc88`7dcff000
ffffdc88`7d09b020 fffffdc88`7e34f000 fffffdc88`7ebf7000
ffffdc88`7d09b030 fffffdc88`7f2f5000 00000000`00000000
ffffdc88`7d09b040 00000000`00000000 00000000`00000000
ffffdc88`7d09b050 00000000`00000000 00000000`00000000
ffffdc88`7d09b060 00000000`00000000 00000000`00000000
ffffdc88`7d09b070 00000000`00000000 00000000`00000000
0: kd> dp fffffdc88`7962a000
ffffdc88`7962a000 00000000`00000000 00000000`00000000
ffffdc88`7962a010 b281de28`3300ffa7 00000000`00000000
ffffdc88`7962a020 00000000`00000000 fffffdc88`7e34f800
ffffdc88`7962a030 b281de29`50800001 00000000`00000000
ffffdc88`7962a040 b281de27`d0400001 00000000`00000000
ffffdc88`7962a050 b281de2b`e0800001 00000000`00000000
ffffdc88`7962a060 b281de35`70800001 00000000`00000000
ffffdc88`7962a070 b281de3c`11400001 00000000`00000000
```

所以此处的第二行 0xb281de28-3300ffa7 就是加密后的 System 进程的 EPROCESS 结构，对于Win10系统来说解密算法 (value >> 0x10) & 0xfffffffffffff0 是这样的，我们通过代码计算出来。

```
#include <windows.h>
#include <iostream>

int _tmain(int argc, _TCHAR* argv[])
{
    std::cout << "hello lyshark.com" << std::endl;

    ULONG64 u1_recode = 0xb281de283300ffa7;

    ULONG64 u1_decode = (ULONG64)u1_recode >> 0x10;
    u1_decode &= 0xfffffffffffff0;

    std::cout << "解密后地址: " << std::hex << u1_decode << std::endl;
    getchar();

    return 0;
}
```

运行程序得到如下输出，即可知道 System 系统进程解密后的 EPROCESS 结构地址是

0xfffffb281de283300

```
c:\users\admin\documents\visual studio 2013\Projects\ConsoleApplication1\x64\Release\ConsoleApplication1.exe
hello lyshark.com
解密后地址: ffffffb281de283300
```

回到WinDBG调试器，输入命令 dt _EPROCESS 0xfffffb281de283300 解析以下这个结构，输出结果是 System 进程。

Command

```
+0x420 DebugPort      : (null)
+0x428 WoW64Process    : (null)
+0x430 DeviceMap       : 0xffffdc88`79613540 Void
+0x438 EtwDataSource    : (null)
+0x440 PageDirectoryPte : 0
+0x448 ImageFilePointer : (null)
+0x450 ImageFileName    : [15] "System"
+0x45f PriorityClass    : 0x2 ''
+0x460 SecurityPort     : (null)
+0x468 SeAuditProcessCreationInfo : _SE_AUDIT_PROCESS_CREATION_INFO
+0x470 JobLinks         : _LIST_ENTRY [ 0x00000000`00000000 - 0x00000000`00000000 ]
+0x480 HighestUserAddress : 0x00007fff`ffff0000 Void
+0x488 ThreadListHead   : _LIST_ENTRY [ 0xfffffb281`de295738 - 0xfffffb281`e169e738 ]
+0x498 ActiveThreads    : 0x8b
+0x49c ImagePathHash    : 0
+0x4a0 DefaultHardErrorProcessing : 5
+0x4a4 LastThreadExitStatus : 0n0
+0x4a8 PrefetchTrace     : _EX_FAST_REF
+0x4b0 LockedPagesList   : (null)
+0x4b8 ReadOperationCount : _LARGE_INTEGER 0x14
+0x4c0 WriteOperationCount : _LARGE_INTEGER 0x17
+0x4c8 OtherOperationCount : _LARGE_INTEGER 0x294
+0x4d0 ReadTransferCount : _LARGE_INTEGER 0x13a0
+0x4d8 WriteTransferCount : _LARGE_INTEGER 0xd304cb
+0x4e0 OtherTransferCount : _LARGE_INTEGER 0x37b2
```

理论知识总结已经结束了，接下来就是如何实现枚举进程线程了，枚举流程如下：

- 1. 首先找到 PspCidTable 的地址。
- 2. 然后找到 HANDLE_TBALE 的地址。
- 3. 根据 TableCode 来判断层次结构。
- 4. 遍历层次结构来获取对象地址。
- 5. 判断对象类型是否为进程对象。
- 6. 判断进程是否有效。

这里先来实现获取 PspCidTable 函数的动态地址，代码如下。

```
// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#include <windef.h>

// 获取 PspCidTable
// By: LyShark.com
BOOLEAN get_PspCidTable(ULONG64* tableAddr)
{
    // 获取 PsLookupProcessByProcessId 地址
    UNICODE_STRING uc_funcName;
    RtlInitUnicodeString(&uc_funcName, L"PsLookupProcessByProcessId");
    ULONG64 ul_funcAddr = MmGetSystemRoutineAddress(&uc_funcName);
    if (ul_funcAddr == NULL)
    {
        return FALSE;
    }
    DbgPrint("PsLookupProcessByProcessId addr = %p \n", ul_funcAddr);

    // 前 40 字节有 call (PspReferenceCidTableEntry)
    /*
0: kd> uf PsLookupProcessByProcessId
nt!PsLookupProcessByProcessId:
fffff802`0841cfe0 48895c2418      mov     qword ptr [rsp+18h],rbx
fffff802`0841cfe5 56             push    rsi
```

```

fffff802`0841cfe6 4883ec20      sub     rsp,20h
fffff802`0841cfea 48897c2438      mov     qword ptr [rsp+38h],rdi
fffff802`0841cfef 488bf2         mov     rsi,rdx
fffff802`0841cff2 65488b3c2588010000 mov     rdi,qword ptr gs:[188h]
fffff802`0841cffb 66ff8fe6010000 dec     word ptr [rdi+1E6h]
fffff802`0841d002 b203          mov     dl,3
fffff802`0841d004 e887000000     call    nt!PspReferenceCidTableEntry
(fffff802`0841d090)
fffff802`0841d009 488bd8         mov     rbx,rax
fffff802`0841d00c 4885c0         test    rax,rax
fffff802`0841d00f 7435          je
nt!PsLookupProcessByProcessId+0x66 (fffff802`0841d046) Branch
*/
ULONG64 u1_entry = 0;
for (INT i = 0; i < 100; i++)
{
    // fffff802`0841d004 e8 87 00 00 00      call
nt!PspReferenceCidTableEntry (fffff802`0841d090)
    if (*(PUCHAR)(u1_funcAddr + i) == 0xe8)
    {
        u1_entry = u1_funcAddr + i;
        break;
    }
}

if (u1_entry != 0)
{
    // 解析 call 地址
    INT i_callCode = *(INT*)(u1_entry + 1);
    DbgPrint("i_callCode = %p \n", i_callCode);
    ULONG64 u1_callJump = u1_entry + i_callCode + 5;
    DbgPrint("u1_callJump = %p \n", u1_callJump);

    // 来到 call (PspReferenceCidTableEntry) 内找 PspCidTable
    /*
0: kd> uf PspReferenceCidTableEntry
nt!PspReferenceCidTableEntry+0x115:
fffff802`0841d1a5 488b0d8473f5ff mov     rcx,qword ptr
[nt!PspCidTable (fffff802`08374530)]
fffff802`0841d1ac b801000000     mov     eax,1
fffff802`0841d1b1 f0480fc107     lock xadd qword ptr [rdi],rax
fffff802`0841d1b6 4883c130       add     rcx,30h
fffff802`0841d1ba f0830c2400     lock or dword ptr [rsp],0
fffff802`0841d1bf 48833900       cmp     qword ptr [rcx],0
fffff802`0841d1c3 0f843fffffff  je
nt!PspReferenceCidTableEntry+0x78 (fffff802`0841d108) Branch
*/
for (INT i = 0; i < 0x120; i++)
{
    // fffff802`0841d1a5 48 8b 0d 84 73 f5 ff mov     rcx,qword ptr
[nt!PspCidTable (fffff802`08374530)]
    if (*(PUCHAR)(u1_callJump + i) == 0x48 && *(PUCHAR)(u1_callJump + i +
1) == 0x8b && *(PUCHAR)(u1_callJump + i + 2) == 0x0d)
    {
        // 解析 mov 地址

```

```

        INT i_movCode = *(INT*)(ul_callJump + i + 3);
        DbgPrint("i_movCode = %p \n", i_movCode);
        ULONG64 ul_movJump = ul_callJump + i + i_movCode + 7;
        DbgPrint("ul_movJump = %p \n", ul_movJump);

        // 得到 PspCidTable
        *tableAddr = ul_movJump;
        return TRUE;
    }
}

return FALSE;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark \n"));

    ULONG64 tableAddr = 0;

    get_PspCidTable(&tableAddr);

    DbgPrint("PspCidTable Address = %p \n", tableAddr);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行后即可得到动态地址，我们可以验证一下是否一致：

The screenshot shows two windows. The top window is DebugView on \\DESKTOP-B53PAVI (local), displaying a list of debug prints. The bottom window is WinDbg, showing the kernel command prompt with the output of the 'uf PspCidTable' command.

#	Time	Debug Print
1	0.00000000	4071250000 - STORMINI: StorNVMe - POWER: ACTIVE
2	0.00753130	4071250000 - STORMINI: StorNVMe - POWER: IDLE
3	0.01759370	4071406250 - STORMINI: StorNVMe - POWER: ACTIVE
4	1.05251265	hello lyshark
5	1.05732489	PsLookupProcessByProcessId addr = FFFFF8020841CFE0
6	1.06266582	i_callCode = 0000000000000087
7	1.06803918	ul_callJump = FFFFF8020841D090
8	1.07343614	i_movCode = 00000000FFF57454
9	1.07873988	ul_movJump = FFFFF80208374530
10	1.08401048	PspCidTable Address = FFFFF80208374530
11	1.17620623	4081406250 - STORMINI: StorNVMe - POWER: IDLE

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64

```

0: kd> uf PspCidTable
Flow analysis was incomplete, some code may be missing
nt!PspCidTable:
fffff802`08374530 c05d6079      rcr      byte ptr [rbp+60h].79h
fffff802`08374534 88dc      mov      ah,bl

```

继续增加对与三级表的动态解析代码，最终代码如下所示：

```

// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#include <windef.h>

// 获取 PspCidTable
// By: LyShark.com
BOOLEAN get_PspCidTable(ULONG64* tableAddr)
{
    // 获取 PsLookupProcessByProcessId 地址
    UNICODE_STRING uc_funcName;
    RtlInitUnicodeString(&uc_funcName, L"PsLookupProcessByProcessId");
    ULONG64 ul_funcAddr = MmGetSystemRoutineAddress(&uc_funcName);
    if (ul_funcAddr == NULL)
    {
        return FALSE;
    }
    DbgPrint("PsLookupProcessByProcessId addr = %p \n", ul_funcAddr);

    // 前 40 字节有 call (PspReferenceCidTableEntry)
    /*
0: kd> uf PsLookupProcessByProcessId
nt!PsLookupProcessByProcessId:
fffff802`0841cfe0 48895c2418      mov     qword ptr [rsp+18h],rbx
fffff802`0841cfe5 56             push    rsi
fffff802`0841cfe6 4883ec20       sub     rsp,20h
fffff802`0841cfea 48897c2438      mov     qword ptr [rsp+38h],rdi
fffff802`0841cfef 488bf2         mov     rsi,rdx
fffff802`0841cff2 65488b3c2588010000 mov     rdi,qword ptr gs:[188h]
fffff802`0841cffb 66ff8fe6010000 dec     word ptr [rdi+1E6h]
fffff802`0841d002 b203          mov     dl,3
fffff802`0841d004 e887000000     call    nt!PspReferenceCidTableEntry
fffff802`0841d090
fffff802`0841d009 488bd8         mov     rbx,rax
fffff802`0841d00c 4885c0         test    rax,rax
fffff802`0841d00f 7435          je
nt!PsLookupProcessByProcessId+0x66 (fffff802`0841d046) Branch
*/
    ULONG64 ul_entry = 0;
    for (INT i = 0; i < 100; i++)
    {
        // fffff802`0841d004 e8 87 00 00 00      call
nt!PspReferenceCidTableEntry (fffff802`0841d090)
        if (*(PUCHAR)(ul_funcAddr + i) == 0xe8)
        {
            ul_entry = ul_funcAddr + i;
            break;
        }
    }

    if (ul_entry != 0)
    {
        // 解析 call 地址
    }
}

```

```

INT i_callCode = *(INT*)(u1_entry + 1);
DbgPrint("i_callCode = %p \n", i_callCode);
ULONG64 u1_callJump = u1_entry + i_callCode + 5;
DbgPrint("u1_callJump = %p \n", u1_callJump);

// 来到 call (PspReferenceCidTableEntry) 内找 PspCidTable
/*
0: kd> uf PspReferenceCidTableEntry
nt!PspReferenceCidTableEntry+0x115:
fffff802`0841d1a5 488b0d8473f5ff mov     rcx,qword ptr
[nt!PspCidTable (fffff802`08374530)]
fffff802`0841d1ac b801000000     mov     eax,1
fffff802`0841d1b1 f0480fc107     lock xadd qword ptr [rdi],rax
fffff802`0841d1b6 4883c130       add     rcx,30h
fffff802`0841d1ba f0830c2400     lock or  dword ptr [rsp],0
fffff802`0841d1bf 48833900       cmp     qword ptr [rcx],0
fffff802`0841d1c3 0f843fffffff  je
nt!PspReferenceCidTableEntry+0x78 (fffff802`0841d108) Branch
*/
for (INT i = 0; i < 0x120; i++)
{
    // fffff802`0841d1a5 48 8b 0d 84 73 f5 ff mov     rcx,qword ptr
[nt!PspCidTable (fffff802`08374530)]
    if (*(PUCHAR)(u1_callJump + i) == 0x48 && *(PUCHAR)(u1_callJump + i +
1) == 0x8b && *(PUCHAR)(u1_callJump + i + 2) == 0x0d)
    {
        // 解析 mov 地址
        INT i_movCode = *(INT*)(u1_callJump + i + 3);
        DbgPrint("i_movCode = %p \n", i_movCode);
        ULONG64 u1_movJump = u1_callJump + i + i_movCode + 7;
        DbgPrint("u1_movJump = %p \n", u1_movJump);

        // 得到 PspCidTable
        *tableAddr = u1_movJump;
        return TRUE;
    }
}
return FALSE;
}

/* 解析一级表
// By: LyShark.com
BaseAddr: 一级表的基地址
index1: 第几个一级表
index2: 第几个二级表
*/
VOID parse_table_1(ULONG64 BaseAddr, INT index1, INT index2)
{
    // 遍历一级表 (每个表项大小 16 ), 表大小 4k, 所以遍历 4096/16 = 526 次
    PEPROCESS p_eprocess = NULL;
    PETHREAD p_ethread = NULL;
    INT i_id = 0;
    for (INT i = 0; i < 256; i++)
    {

```

```

if (!MmIsValidAddress((PVOID64)(BaseAddr + i * 16)))
{
    DbgPrint("非法地址= %p \n", BaseAddr + i * 16);
    continue;
}

ULONG64 u1_recode = *(PULONG64)(BaseAddr + i * 16);

// 解密
ULONG64 u1_decode = (LONG64)u1_recode >> 0x10;
u1_decode &= 0xffffffffffffffff;

// 判断是进程还是线程
i_id = i * 4 + 1024 * index1 + 512 * index2 * 1024;
if (PsLookupProcessByProcessId(i_id, &p_eprocess) == STATUS_SUCCESS)
{
    DbgPrint("进程PID: %d | ID: %d | 内存地址: %p | 对象: %p \n", i_id, i,
BaseAddr + i * 0x10, u1_decode);
}
else if (PsLookupThreadByThreadId(i_id, &p_ethread) == STATUS_SUCCESS)
{
    DbgPrint("线程TID: %d | ID: %d | 内存地址: %p | 对象: %p \n", i_id, i,
BaseAddr + i * 0x10, u1_decode);
}
}
}

/* 解析二级表
// By: LyShark.com
BaseAddr: 二级表基地址
index2: 第几个二级表
*/
VOID parse_table_2(ULONG64 BaseAddr, INT index2)
{
    // 遍历二级表 (每个表项大小 8), 表大小 4k, 所以遍历 4096/8 = 512 次
    ULONG64 u1_baseAddr_1 = 0;
    for (INT i = 0; i < 512; i++)
    {
        if (!MmIsValidAddress((PVOID64)(BaseAddr + i * 8)))
        {
            DbgPrint("非法二级表指针 (1): %p \n", BaseAddr + i * 8);
            continue;
        }
        if (!MmIsValidAddress((PVOID64)*(PULONG64)(BaseAddr + i * 8)))
        {
            DbgPrint("非法二级表指针 (2): %p \n", BaseAddr + i * 8);
            continue;
        }
        u1_baseAddr_1 = *(PULONG64)(BaseAddr + i * 8);
        parse_table_1(u1_baseAddr_1, i, index2);
    }
}

/* 解析三级表
// By: LyShark.com

```


BaseAddr: 三级表基地址

```
*/
VOID parse_table_3(ULONG64 BaseAddr)
{
    // 遍历三级表 (每个表项大小 8), 表大小 4k, 所以遍历 4096/8 = 512 次
    ULONG64 ul_baseAddr_2 = 0;
    for (INT i = 0; i < 512; i++)
    {
        if (!MmIsValidAddress((PVOID64)(BaseAddr + i * 8)))
        {
            continue;
        }
        if (!MmIsValidAddress((PVOID64)* (PULONG64)(BaseAddr + i * 8)))
        {
            continue;
        }
        ul_baseAddr_2 = *(PULONG64)(BaseAddr + i * 8);
        parse_table_2(ul_baseAddr_2, i);
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com \n"));

    ULONG64 tableAddr = 0;

    get_PspCidTable(&tableAddr);

    DbgPrint(("PspCidTable Address = %p \n", tableAddr);

    // 获取 _HANDLE_TABLE 的 TableCode
    ULONG64 ul_tableCode = *(PULONG64)((ULONG64)*(PULONG64)tableAddr + 8);
    DbgPrint(("ul_tableCode = %p \n", ul_tableCode);

    // 取低 2 位 (二进制 11 = 3)
    INT i_low2 = ul_tableCode & 3;
    DbgPrint(("i_low2 = %X \n", i_low2);

    // 一级表
    if (i_low2 == 0)
    {
        // TableCode 低 2 位抹零 (二进制 11 = 3)
        parse_table_1(ul_tableCode & (~3), 0, 0);
    }
    // 二级表
    else if (i_low2 == 1)
    {
        // TableCode 低 2 位抹零 (二进制 11 = 3)
        parse_table_2(ul_tableCode & (~3), 0);
    }
}
```

```

    }
    // 三级表
    else if (i_low2 == 2)
    {
        // TableCode 低 2位抹零 (二进制11 = 3)
        parse_table_3(ul_tableCode & (~3));
    }
    else
    {
        DbgPrint("LyShark提示: 错误,非法!");
        return FALSE;
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行如上完整代码，我们可以在WinDBG中捕捉到枚举到的进程信息：

#	Time	Debug Print
6	1.40835607	hello lyshark.com
7	1.41310668	PsLookupProcessByProcessId addr = FFFFF8020841CFE0
8	1.41846049	i_callCode = 0000000000000087
9	1.42382085	ul_callJmp = FFFFF8020841D090
10	1.42928135	i_movCode = 00000000FF57454
11	1.43486655	ul_movJmp = FFFFF80208374530
12	1.44027793	PspCidTable Address = FFFFF80208374530
13	1.44565725	ul_tableCode = FFFFF80208374530
14	1.45099461	i_low2 = 1
15	1.45639431	进程PID: 4 ID: 1 内存地址: FFFFF80208374530 对象: FFFFF80208374530
16	1.46178901	线程TID: 8 ID: 2 内存地址: FFFFF80208374530 对象: FFFFF80208374530
17	1.46716416	线程TID: 12 ID: 3 内存地址: FFFFF80208374530 对象: FFFFF80208374530
18	1.47233284	线程TID: 16 ID: 4 内存地址: FFFFF80208374530 对象: FFFFF80208374530
19	1.47776735	线程TID: 20 ID: 5 内存地址: FFFFF80208374530 对象: FFFFF80208374530
20	1.48314285	线程TID: 24 ID: 6 内存地址: FFFFF80208374530 对象: FFFFF80208374530
21	1.48850119	线程TID: 28 ID: 7 内存地址: FFFFF80208374530 对象: FFFFF80208374530
22	1.49370909	线程TID: 32 ID: 8 内存地址: FFFFF80208374530 对象: FFFFF80208374530
23	1.49914503	线程TID: 36 ID: 9 内存地址: FFFFF80208374530 对象: FFFFF80208374530

线程信息在进程信息的下面，枚举效果如下：

#	Time	Debug Print
894	6.19741106	线程TID: 5152 ID: 8 内存地址: FFFFF80208374530 对象: FFFFF80208374530
895	6.20275927	线程TID: 5160 ID: 10 内存地址: FFFFF80208374530 对象: FFFFF80208374530
896	6.20745420	线程TID: 5168 ID: 12 内存地址: FFFFF80208374530 对象: FFFFF80208374530
897	6.21280813	线程TID: 5176 ID: 14 内存地址: FFFFF80208374530 对象: FFFFF80208374530
898	6.21819448	线程TID: 5180 ID: 15 内存地址: FFFFF80208374530 对象: FFFFF80208374530
899	6.22355032	线程TID: 5184 ID: 16 内存地址: FFFFF80208374530 对象: FFFFF80208374530
900	6.22892714	线程TID: 5188 ID: 17 内存地址: FFFFF80208374530 对象: FFFFF80208374530
901	6.23432016	线程TID: 5192 ID: 18 内存地址: FFFFF80208374530 对象: FFFFF80208374530
902	6.23972321	线程TID: 5200 ID: 20 内存地址: FFFFF80208374530 对象: FFFFF80208374530
903	6.24488497	线程TID: 5204 ID: 21 内存地址: FFFFF80208374530 对象: FFFFF80208374530
904	6.25032473	线程TID: 5216 ID: 24 内存地址: FFFFF80208374530 对象: FFFFF80208374530
905	6.25564957	线程TID: 5220 ID: 25 内存地址: FFFFF80208374530 对象: FFFFF80208374530
906	6.26105118	线程TID: 5224 ID: 26 内存地址: FFFFF80208374530 对象: FFFFF80208374530
907	6.26644087	线程TID: 5228 ID: 27 内存地址: FFFFF80208374530 对象: FFFFF80208374530
908	6.27165461	线程TID: 5232 ID: 28 内存地址: FFFFF80208374530 对象: FFFFF80208374530
909	6.28022480	线程TID: 5236 ID: 29 内存地址: FFFFF80208374530 对象: FFFFF80208374530
910	6.28579903	线程TID: 5240 ID: 30 内存地址: FFFFF80208374530 对象: FFFFF80208374530
911	6.29111000	线程TID: 5244 ID: 31 内存地址: FFFFF80208374530 对象: FFFFF80208374530

至此文章就结束了，这里多说一句，实际上 ZwQuerySystemInformation 枚举系统句柄时就是走的这条双链，枚举系统进程如果使用的是这个API函数，那么不出意外它也是在这些内核表中做的解析。

8.4 内核枚举Minifilter微过滤驱动

Minifilter 是一种文件过滤驱动，该驱动简称为微过滤驱动，相对于传统的 `sfilter` 文件过滤驱动来说，微过滤驱动编写时更简单，其不需要考虑底层RIP如何派发且无需考虑兼容性问题，微过滤驱动使用过滤管理器 `FilterManager` 提供接口，由于提供了管理结构以及一系列管理API函数，所以枚举过滤驱动将变得十分容易。

通常文件驱动过滤是ARK重要功能之一，如下是一款闭源ARK工具的输出效果图。

进程	驱动模块	内核	内核钩子	应用层钩子	网络	注册表	文件	启动信息	监控	系统杂项	关于
系统回调	过滤驱动	DPC定时器	工作队列线程	Hal	Wdf	文件系统	系统调试	对象劫持	直接IO	GDT	
文件系统项目	微端口过滤器										
文件系统											
5Filter回调											
ClassInitData回调											
Npfs派发函数											
Msfs派发函数											
Usbport派发函数											
函数名称	函数地址	函数所在模块	文件厂商	过滤器地址	高度						
FilterUnload	0xFFFFF80209197640	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						
Instance...	0xFFFFF80209197940	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						
Instance...	0xFFFFF80209197B50	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						
Instance...	0xFFFFF80209197B80	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						
KtmNotifi...	0xFFFFF8020919C620	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						
IRP_MJ_...	0xFFFFF8020918D920	C:\Windows\s...	Microsof...	0xFFFFF838E93D7E010	328010						

由于 `MiniFilter` 提供了 `FltEnumerateFilters` 函数，所以只需要调用这些函数即可获取到所有的过滤器地址，我们看下微软公开的信息。

```
NTSTATUS FLTAPI FltEnumerateFilters(  
    [out] PFLT_FILTER *FilterList,  
    [in]  ULONG        FilterListSize,  
    [out] PULONG       NumberFiltersReturned  
);
```

此函数需要注意，如果用户将 `FilterList` 设置为 `NULL` 则默认是输出当前系统中存在的过滤器数量，而如果传入的是一个内存地址，则将会枚举系统中所有的过滤器信息。

使用 `FltEnumerateFilters` 这个API，它会返回过滤器对象 `FLT_FILTER` 的地址，然后根据过滤器对象的地址，加上一个偏移，获得记录过滤器 `PreCall`、`PostCall`、`IRP` 等信息的 `PFLT_OPERATION_REGISTRATION` 结构体指针。

上文之所以说要加上偏移，是因为 `FLT_FILTER` 的定义在每个系统都不同，比如 `WIN10 x64` 中的定义以下样子，这里我们需要记下 `+0x1a8 Operations` 因为他指向的就是 `_FLT_OPERATION_REGISTRATION` 结构的偏移地址。

```
lyshark.com: kd> dt fltmgr!_FLT_FILTER  
+0x000 Base : _FLT_OBJECT  
+0x030 Frame : Ptr64 _FLTP_FRAME  
+0x038 Name : _UNICODE_STRING  
+0x048 DefaultAltitude : _UNICODE_STRING  
+0x058 Flags : _FLT_FILTER_FLAGS  
+0x060 DriverObject : Ptr64 _DRIVER_OBJECT  
+0x068 InstanceList : _FLT_RESOURCE_LIST_HEAD  
+0x0e8 VerifierExtension : Ptr64 _FLT_VERIFIER_EXTENSION  
+0x0f0 VerifiedFiltersLink : _LIST_ENTRY  
+0x100 FilterUnload : Ptr64 long  
+0x108 InstanceSetup : Ptr64 long  
+0x110 InstanceQueryTeardown : Ptr64 long  
+0x118 InstanceTeardownStart : Ptr64 void  
+0x120 InstanceTeardownComplete : Ptr64 void  
+0x128 SupportedContextsListHead : Ptr64 _ALLOCATE_CONTEXT_HEADER  
+0x130 SupportedContexts : [7] Ptr64 _ALLOCATE_CONTEXT_HEADER
```

```

+0x168 PreVolumeMount      : Ptr64      _FLT_PREOP_CALLBACK_STATUS
+0x170 PostVolumeMount     : Ptr64      _FLT_POSTOP_CALLBACK_STATUS
+0x178 GenerateFileName    : Ptr64      long
+0x180 NormalizeNameComponent : Ptr64      long
+0x188 NormalizeNameComponentEx : Ptr64      long
+0x190 NormalizeContextCleanup : Ptr64      void
+0x198 KtmNotification     : Ptr64      long
+0x1a0 SectionNotification : Ptr64      long
+0x1a8 Operations          : Ptr64      _FLT_OPERATION_REGISTRATION
+0x1b0 OldDriverUnload     : Ptr64      void
+0x1b8 ActiveOpens         : _FLT_Mutex_LIST_HEAD
+0x208 ConnectionList      : _FLT_Mutex_LIST_HEAD
+0x258 PortList            : _FLT_Mutex_LIST_HEAD
+0x2a8 PortLock            : _EX_PUSH_LOCK

```

解析 FLT_OPERATION_REGISTRATION 结构体，可以看到这就是我们需要枚举的过滤器，只要拿到它输出即可：

```

lyshark.com: kd> dt fltmgr!_FLT_OPERATION_REGISTRATION
+0x000 MajorFunction      : UChar
+0x004 Flags              : Uint4B
+0x008 PreOperation       : Ptr64      _FLT_PREOP_CALLBACK_STATUS
+0x010 PostOperation      : Ptr64      _FLT_POSTOP_CALLBACK_STATUS
+0x018 Reserved1         : Ptr64      Void

```

枚举过滤器代码如下所示，需要配置连接器增加 fltmgr.lib 头文件。

- 配置属性 > 连接器 > 输入 > 附加依赖 -> fltmgr.lib
- 配置属性 > C/C++ > 常规 > 设置 警告等级2级 (警告视为错误关闭)

```

// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

#include <fltKernel.h>
#include <dontuse.h>
#include <suppress.h>

// 设置默认回调
NTSTATUS DriverDefaultHandle(PDEVICE_OBJECT pDevObj, PIRP pIrp)
{
    NTSTATUS status = STATUS_SUCCESS;
    pIrp->IoStatus.Status = status;
    pIrp->IoStatus.Information = 0;
    IoCompleteRequest(pIrp, IO_NO_INCREMENT);

    return status;
}

VOID DriverUnload(PDRIVER_OBJECT pDriverObject)
{
}

NTSTATUS DriverEntry(PDRIVER_OBJECT pDriverObject, PUNICODE_STRING pRegPath)

```

```

{
    DbgPrint("hello lyshark.com \n");

    NTSTATUS status = STATUS_SUCCESS;
    pDriverObject->DriverUnload = DriverUnload;
    for (ULONG i = 0; i < IRP_MJ_MAXIMUM_FUNCTION; i++)
    {
        pDriverObject->MajorFunction[i] = DriverDefaultHandle;
    }

    ULONG ulFilterListSize = 0;
    PFLT_FILTER *ppFilterList = NULL;
    ULONG i = 0;
    LONG lOperationsOffset = 0;
    PFLT_OPERATION_REGISTRATION pFltOperationRegistration = NULL;

    // 获取 Minifilter 过滤器Filter 的数量
    FltEnumerateFilters(NULL, 0, &ulFilterListSize);

    // 申请内存
    ppFilterList = (PFLT_FILTER *)ExAllocatePool(NonPagedPool, ulFilterListSize
*sizeof(PFLT_FILTER));
    if (NULL == ppFilterList)
    {
        return FALSE;
    }

    // 获取 Minifilter 中所有过滤器Filter 的信息
    status = FltEnumerateFilters(ppFilterList, ulFilterListSize,
&ulFilterListSize);
    if (!NT_SUCCESS(status))
    {
        return FALSE;
    }

    DbgPrint("过滤器数量: %d \n", ulFilterListSize);

    // 获取 PFLT_FILTER 中 Operations 偏移
    lOperationsOffset = 0x1A8;

    // 开始遍历 Minifilter
    __try
    {
        for (i = 0; i < ulFilterListSize; i++)
        {
            // 获取 PFLT_FILTER 中 Operations 成员地址
            pFltOperationRegistration = (PFLT_OPERATION_REGISTRATION)(*(PVOID *)
((PUCHAR)ppFilterList[i] + lOperationsOffset));

            __try
            {
                // 同一过滤器下的回调信息
                while (IRP_MJ_OPERATION_END != pFltOperationRegistration-
>MajorFunction)
                {

```

```

        if (IRP_MJ_MAXIMUM_FUNCTION > pFltOperationRegistration->MajorFunction)
        {
            // 显示
            DbgPrint("Filter: %p | IRP: %d | PreFunc: 0x%p | PostFunc=0x%p \n", ppFilterList[i], pFltOperationRegistration->MajorFunction, pFltOperationRegistration->PreOperation, pFltOperationRegistration->PostOperation);
        }

        // 获取下一个消息回调信息
        pFltOperationRegistration = (PFLT_OPERATION_REGISTRATION)((PUCHAR)pFltOperationRegistration + sizeof(FLT_OPERATION_REGISTRATION));
    }
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
}

// 释放内存
ExFreePool(ppFilterList);
ppFilterList = NULL;

return status;
}

```

运行代码输出枚举效果如下：

#	Time	Debug Print
3	0.3...	hello lyshark.com
4	0.3...	过滤器数量: 9
5	0.3...	Filter: FFFF838E93D7E010 IRP: 0 PreFunc: 0xFFFFF8020918D920 PostFunc=0xFFFFF8020918FC90
6	0.3...	Filter: FFFF838E93D7E010 IRP: 18 PreFunc: 0xFFFFF802091872B0 PostFunc=0x0000000000000000
7	0.3...	Filter: FFFF838E93D7E010 IRP: 6 PreFunc: 0xFFFFF80209193610 PostFunc=0xFFFFF80209193C90
8	0.3...	Filter: FFFF838E93D7E010 IRP: 4 PreFunc: 0xFFFFF802091752D0 PostFunc=0xFFFFF802091759D0
9	0.3...	Filter: FFFF838E93D7E010 IRP: 13 PreFunc: 0xFFFFF802091A70C0 PostFunc=0xFFFFF802091A7410
10	0.3...	Filter: FFFF838E93D7E010 IRP: 12 PreFunc: 0x0000000000000000 PostFunc=0xFFFFF80209178070
11	0.3...	Filter: FFFF838E93D7E010 IRP: 3 PreFunc: 0x0000000000000000 PostFunc=0xFFFFF80209175C20
12	0.3...	Filter: FFFF838E93D7E010 IRP: 7 PreFunc: 0xFFFFF802091A8160 PostFunc=0x0000000000000000
13	0.3...	Filter: FFFF838E968896E0 IRP: 0 PreFunc: 0xFFFFF8020B810F10 PostFunc=0xFFFFF8020B810FA0
14	0.3...	Filter: FFFF838E968896E0 IRP: 2 PreFunc: 0xFFFFF8020B811130 PostFunc=0x0000000000000000
15	0.3...	Filter: FFFF838E968896E0 IRP: 3 PreFunc: 0xFFFFF8020B810670 PostFunc=0xFFFFF8020B802F50
16	0.3...	Filter: FFFF838E968896E0 IRP: 4 PreFunc: 0xFFFFF8020B810670 PostFunc=0xFFFFF8020B802F50
17	0.4...	Filter: FFFF838E968896E0 IRP: 13 PreFunc: 0xFFFFF8020B810010 PostFunc=0xFFFFF8020B803C40
18	0.4...	Filter: FFFF838E968896E0 IRP: 0 PreFunc: 0xFFFFF8020BB441D0 PostFunc=0xFFFFF8020BB44910

8.5 内核特征码扫描PE代码段

在笔者上一篇文章《驱动开发：内核特征码搜索函数封装》中为了定位特征的方便我们封装实现了一个可以传入数组实现的 searchSpecialCode 定位函数，该定位函数其实还不能算的上简单，本章 Lyshark 将对特征码定位进行简化，让定位变得更简单，并运用定位代码实现扫描内核PE的 .text 代码段，并从代码段中得到某个特征所在内存位置。

老样子为了后续教程能够继续，先来定义一个 `lyshark.h` 头文件，该头文件中包含了我们本篇文章所必须要使用到的结构体定义，这些定义的函数如果不懂请去看 `Lyshark` 以前的文章，这里就不罗嗦了。

```
#include <ntifs.h>
#include <ntimage.h>

typedef struct _KLDL_DATA_TABLE_ENTRY
{
    LIST_ENTRY64 InLoadOrderLinks;
    ULONG64 __Undefined1;
    ULONG64 __Undefined2;
    ULONG64 __Undefined3;
    ULONG64 NonPagedDebugInfo;
    ULONG64 DllBase;
    ULONG64 EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING FullDllName;
    UNICODE_STRING BaseDllName;
    ULONG Flags;
    USHORT LoadCount;
    USHORT __Undefined5;
    ULONG64 __Undefined6;
    ULONG CheckSum;
    ULONG __padding1;
    ULONG TimeDateStamp;
    ULONG __padding2;
} KLDL_DATA_TABLE_ENTRY, *PKLDL_DATA_TABLE_ENTRY;

typedef struct _RTL_PROCESS_MODULE_INFORMATION
{
    HANDLE Section;
    PVOID MappedBase;
    PVOID ImageBase;
    ULONG ImageSize;
    ULONG Flags;
    USHORT LoadOrderIndex;
    USHORT InitOrderIndex;
    USHORT LoadCount;
    USHORT OffsetToFileName;
    UCHAR FullPathName[256];
} RTL_PROCESS_MODULE_INFORMATION, *PRTL_PROCESS_MODULE_INFORMATION;

typedef struct _RTL_PROCESS_MODULES
{
    ULONG NumberOfModules;
    RTL_PROCESS_MODULE_INFORMATION Modules[1];
} RTL_PROCESS_MODULES, *PRTL_PROCESS_MODULES;

typedef enum _SYSTEM_INFORMATION_CLASS
{
    SystemBasicInformation = 0x0,
    SystemProcessorInformation = 0x1,
    SystemPerformanceInformation = 0x2,
    SystemTimeOfDayInformation = 0x3,
    SystemPathInformation = 0x4,
```


SystemProcessInformation = 0x5,
SystemCallCountInformation = 0x6,
SystemDeviceInformation = 0x7,
SystemProcessorPerformanceInformation = 0x8,
SystemFlagsInformation = 0x9,
SystemCallTimeInformation = 0xa,
SystemModuleInformation = 0xb,
SystemLocksInformation = 0xc,
SystemStackTraceInformation = 0xd,
SystemPagedPoolInformation = 0xe,
SystemNonPagedPoolInformation = 0xf,
SystemHandleInformation = 0x10,
SystemObjectInformation = 0x11,
SystemPageFileInformation = 0x12,
SystemVdmInstemulInformation = 0x13,
SystemVdmBopInformation = 0x14,
SystemFileCacheInformation = 0x15,
SystemPoolTagInformation = 0x16,
SystemInterruptInformation = 0x17,
SystemDpcBehaviorInformation = 0x18,
SystemFullMemoryInformation = 0x19,
SystemLoadGdiDriverInformation = 0x1a,
SystemUnloadGdiDriverInformation = 0x1b,
SystemTimeAdjustmentInformation = 0x1c,
SystemSummaryMemoryInformation = 0x1d,
SystemMirrorMemoryInformation = 0x1e,
SystemPerformanceTraceInformation = 0x1f,
SystemObsolete0 = 0x20,
SystemExceptionInformation = 0x21,
SystemCrashDumpStateInformation = 0x22,
SystemKernelDebuggerInformation = 0x23,
SystemContextSwitchInformation = 0x24,
SystemRegistryQuotaInformation = 0x25,
SystemExtendServiceTableInformation = 0x26,
SystemPrioritySeparation = 0x27,
SystemVerifierAddDriverInformation = 0x28,
SystemVerifierRemoveDriverInformation = 0x29,
SystemProcessorIdleInformation = 0x2a,
SystemLegacyDriverInformation = 0x2b,
SystemCurrentTimeZoneInformation = 0x2c,
SystemLookasideInformation = 0x2d,
SystemTimeSlipNotification = 0x2e,
SystemSessionCreate = 0x2f,
SystemSessionDetach = 0x30,
SystemSessionInformation = 0x31,
SystemRangeStartInformation = 0x32,
SystemVerifierInformation = 0x33,
SystemVerifierThunkExtend = 0x34,
SystemSessionProcessInformation = 0x35,
SystemLoadGdiDriverInSystemSpace = 0x36,
SystemNumaProcessorMap = 0x37,
SystemPrefetcherInformation = 0x38,
SystemExtendedProcessInformation = 0x39,
SystemRecommendedSharedDataAlignment = 0x3a,
SystemComPlusPackage = 0x3b,

SystemNumaAvailableMemory = 0x3c,
SystemProcessorPowerInformation = 0x3d,
SystemEmulationBasicInformation = 0x3e,
SystemEmulationProcessorInformation = 0x3f,
SystemExtendedHandleInformation = 0x40,
SystemLostDelayedWriteInformation = 0x41,
SystemBigPoolInformation = 0x42,
SystemSessionPoolTagInformation = 0x43,
SystemSessionMappedViewInformation = 0x44,
SystemHotpatchInformation = 0x45,
SystemObjectSecurityMode = 0x46,
SystemWatchdogTimerHandler = 0x47,
SystemWatchdogTimerInformation = 0x48,
SystemLogicalProcessorInformation = 0x49,
SystemWow64SharedInformationObsolete = 0x4a,
SystemRegisterFirmwareTableInformationHandler = 0x4b,
SystemFirmwareTableInformation = 0x4c,
SystemModuleInformationEx = 0x4d,
SystemVerifierTriageInformation = 0x4e,
SystemSuperfetchInformation = 0x4f,
SystemMemoryListInformation = 0x50,
SystemFileCacheInformationEx = 0x51,
SystemThreadPriorityClientIdInformation = 0x52,
SystemProcessorIdleCycleTimeInformation = 0x53,
SystemVerifierCancellationInformation = 0x54,
SystemProcessorPowerInformationEx = 0x55,
SystemRefTraceInformation = 0x56,
SystemSpecialPoolInformation = 0x57,
SystemProcessIdInformation = 0x58,
SystemErrorPortInformation = 0x59,
SystemBootEnvironmentInformation = 0x5a,
SystemHypervisorInformation = 0x5b,
SystemVerifierInformationEx = 0x5c,
SystemTimeZoneInformation = 0x5d,
SystemImageFileExecutionOptionsInformation = 0x5e,
SystemCoverageInformation = 0x5f,
SystemPrefetchPatchInformation = 0x60,
SystemVerifierFaultsInformation = 0x61,
SystemSystemPartitionInformation = 0x62,
SystemSystemDiskInformation = 0x63,
SystemProcessorPerformanceDistribution = 0x64,
SystemNumaProximityNodeInformation = 0x65,
SystemDynamicTimeZoneInformation = 0x66,
SystemCodeIntegrityInformation = 0x67,
SystemProcessorMicrocodeUpdateInformation = 0x68,
SystemProcessorBrandString = 0x69,
SystemVirtualAddressInformation = 0x6a,
SystemLogicalProcessorAndGroupInformation = 0x6b,
SystemProcessorCycleTimeInformation = 0x6c,
SystemStoreInformation = 0x6d,
SystemRegistryAppendString = 0x6e,
SystemAitSamplingValue = 0x6f,
SystemVhdBootInformation = 0x70,
SystemCpuQuotaInformation = 0x71,
SystemNativeBasicInformation = 0x72,

```

SystemErrorPortTimeouts = 0x73,
SystemLowPriorityIoInformation = 0x74,
SystemBootEntropyInformation = 0x75,
SystemVerifierCountersInformation = 0x76,
SystemPagedPoolInformationEx = 0x77,
SystemSystemPtesInformationEx = 0x78,
SystemNodeDistanceInformation = 0x79,
SystemAcpiAuditInformation = 0x7a,
SystemBasicPerformanceInformation = 0x7b,
SystemQueryPerformanceCounterInformation = 0x7c,
SystemSessionBigPoolInformation = 0x7d,
SystemBootGraphicsInformation = 0x7e,
SystemScrubPhysicalMemoryInformation = 0x7f,
SystemBadPageInformation = 0x80,
SystemProcessorProfileControlArea = 0x81,
SystemCombinePhysicalMemoryInformation = 0x82,
SystemEntropyInterruptTimingInformation = 0x83,
SystemConsoleInformation = 0x84,
SystemPlatformBinaryInformation = 0x85,
SystemThrottleNotificationInformation = 0x86,
SystemHypervisorProcessorCountInformation = 0x87,
SystemDeviceDataInformation = 0x88,
SystemDeviceDataEnumerationInformation = 0x89,
SystemMemoryTopologyInformation = 0x8a,
SystemMemoryChannelInformation = 0x8b,
SystemBootLogoInformation = 0x8c,
SystemProcessorPerformanceInformationEx = 0x8d,
SystemSpare0 = 0x8e,
SystemSecureBootPolicyInformation = 0x8f,
SystemPageFileInformationEx = 0x90,
SystemSecureBootInformation = 0x91,
SystemEntropyInterruptTimingRawInformation = 0x92,
SystemPortableWorkspaceEfiLauncherInformation = 0x93,
SystemFullProcessInformation = 0x94,
SystemKernelDebuggerInformationEx = 0x95,
SystemBootMetadataInformation = 0x96,
SystemSoftRebootInformation = 0x97,
SystemElamCertificateInformation = 0x98,
SystemOfflineDumpConfigInformation = 0x99,
SystemProcessorFeaturesInformation = 0x9a,
SystemRegistryReconciliationInformation = 0x9b,
MaxSystemInfoClass = 0x9c,
} SYSTEM_INFORMATION_CLASS;

// 声明函数
// By: Lyshark.com
NTSYSAPI PIMAGE_NT_HEADERS NTAPI RtlImageNtHeader(_In_ PVOID Base);
NTSTATUS NTAPI ZwQuerySystemInformation(SYSTEM_INFORMATION_CLASS
SystemInformationClass, PVOID SystemInformation, ULONG SystemInformationLength,
PULONG ReturnLength);

typedef VOID(__cdecl *PMiProcessLoaderEntry)(PKLDR_DATA_TABLE_ENTRY section, IN
LOGICAL Insert);

```

```
typedef NTSTATUS(*NTQUERYSYSTEMINFORMATION)(IN ULONG SystemInformationClass, OUT
PVOID SystemInformation, IN ULONG_PTR SystemInformationLength, OUT PULONG_PTR
ReturnLength OPTIONAL);
```

我们继续，首先实现特征码字符串的解析与扫描实现此处 `UtilLySharkSearchPattern` 函数就是 `LyShark` 封装过的，这里依次介绍一下参数传递的含义。

- `pattern` 用于传入一段字符串特征值（以 `\x` 开头）
- `len` 代表输入特征码长度（除去 `\x` 后的长度）
- `base` 代表扫描内存的基地址
- `size` 代表需要向下扫描的长度
- `ppFound` 代表扫描到首地址以后返回的内存地址

这段代码该如何使用，如下我们以定位 `IoInitializeTimer` 为例，演示 `UtilLySharkSearchPattern` 如何定位特征的，如下代码 `pattern` 变量中就是我们需要定位的特征值，`pattern_size` 则是需要定位的特征码长度，在 `address` 地址位置向下扫描 128 字节，找到则返回到 `find_address` 变量内。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "lyshark.h"

PVOID GetIoInitializeTimerAddress()
{
    PVOID VariableAddress = 0;
    UNICODE_STRING uiioTime = { 0 };

    RtlInitUnicodeString(&uiioTime, L"IoInitializeTimer");
    VariableAddress = (PVOID)MmGetSystemRoutineAddress(&uiioTime);
    if (VariableAddress != 0)
    {
        return VariableAddress;
    }
    return 0;
}

// 对指定内存执行特征码扫描
NTSTATUS UtilLySharkSearchPattern(IN PCHAR pattern, IN ULONG_PTR len, IN const
VOID* base, IN ULONG_PTR size, OUT PVOID* ppFound)
{
    // 计算匹配长度
    // LyShark.com 特征码扫描
    NT_ASSERT(ppFound != 0 && pattern != 0 && base != 0);
    if (ppFound == 0 || pattern == 0 || base == 0)
    {
        return STATUS_INVALID_PARAMETER;
    }

    __try
    {
        for (ULONG_PTR i = 0; i < size - len; i++)
        {
            BOOLEAN found = TRUE;
```

```

        for (ULONG_PTR j = 0; j < len; j++)
        {
            if (pattern[j] != ((PUCHAR)base)[i + j])
            {
                found = FALSE;
                break;
            }
        }

        if (found != FALSE)
        {
            *ppFound = (PUCHAR)base + i;
            DbgPrint("[LyShark.com] 特征码匹配地址: %p \n", (PUCHAR)base + i);
            return STATUS_SUCCESS;
        }
    }
}

__except (EXCEPTION_EXECUTE_HANDLER)
{
    return STATUS_UNHANDLED_EXCEPTION;
}

return STATUS_NOT_FOUND;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com \n"));

    // 返回匹配长度5
    CHAR pattern[] = "\x48\x89\x6c\x24\x10";
    PVOID *find_address = NULL;

    int pattern_size = sizeof(pattern) - 1;
    DbgPrint("匹配长度: %d \n", pattern_size);

    // 得到基地址
    PVOID address = GetIoInitializeTimerAddress();

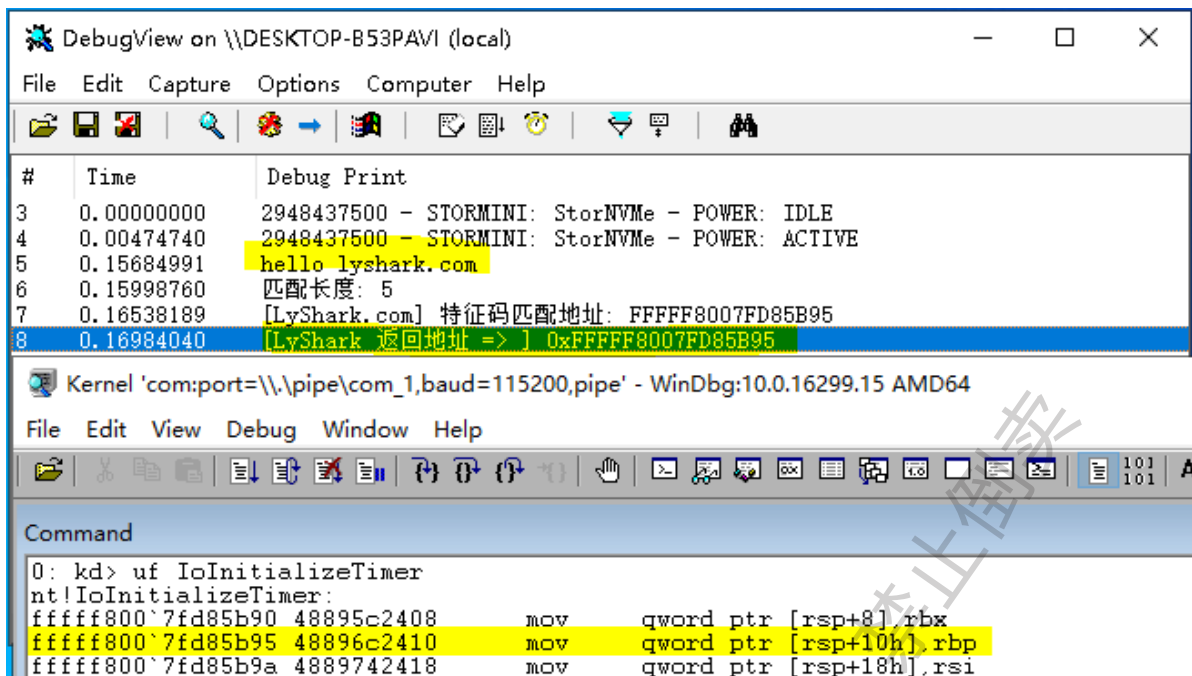
    // 扫描特征
    NTSTATUS nt = UtilLySharkSearchPattern((PUCHAR)pattern, pattern_size,
    address, 128, &find_address);

    DbgPrint("[LyShark 返回地址 => ] 0x%p \n", (ULONG64)find_address);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行驱动程序完成特征定位，并对比定位效果。



如上述所示定位函数我们已经封装好了，相信你能感受到这种方式要比使用数组更方便，为了能定位到内核PE结构我们需要使用 `RtlImageNtHeader` 来解析，这个内核函数专门用来得到内核程序的PE头部结构的，在下方案例中首先我们使用封装过的 `LySharkToolsutilKernelBase` 函数拿到内核基址，如果你不懂函数实现细节请阅读《驱动开发：内核取ntoskrnl模块基址》这篇文章，拿到基址以后可以直接使用 `RtlImageNtHeader` 对其PE头部进行解析，如下所示。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "lyshark.h"

// 定义全局变量
static PVOID g_KernelBase = 0;
static ULONG g_KernelSize = 0;

// 得到KernelBase基地址
// lyshark.com
PVOID LySharkToolsutilKernelBase(OUT PULONG pSize)
{
    NTSTATUS status = STATUS_SUCCESS;
    ULONG bytes = 0;
    PRTL_PROCESS_MODULES pMods = 0;
    PVOID checkPtr = 0;
    UNICODE_STRING routineName;

    if (g_KernelBase != 0)
    {
        if (pSize)
        {
            *pSize = g_KernelSize;
        }
        return g_KernelBase;
    }

    RtlInitUnicodeString(&routineName, L"NtOpenFile");
  
```

```

checkPtr = MmGetSystemRoutineAddress(&routineName);
if (checkPtr == 0)
    return 0;

__try
{
    status = ZwQuerySystemInformation(SystemModuleInformation, 0, bytes,
&bytes);
    if (bytes == 0)
    {
        return 0;
    }

    pMods = (PRTL_PROCESS_MODULES)ExAllocatePoolWithTag(NonPagedPoolNx,
bytes, L"LyShark");
    RtlZeroMemory(pMods, bytes);

    status = ZwQuerySystemInformation(SystemModuleInformation, pMods, bytes,
&bytes);

    if (NT_SUCCESS(status))
    {
        PRTL_PROCESS_MODULE_INFORMATION pMod = pMods->Modules;

        for (ULONG i = 0; i < pMods->NumberOfModules; i++)
        {
            if (checkPtr >= pMod[i].ImageBase && checkPtr < (PVOID)
((PUCHAR)pMod[i].ImageBase + pMod[i].ImageSize))
            {
                g_KernelBase = pMod[i].ImageBase;
                g_KernelSize = pMod[i].ImageSize;
                if (pSize)
                {
                    *pSize = g_KernelSize;
                }
                break;
            }
        }
    }
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
    return 0;
}

if (pMods)
{
    ExFreePoolWithTag(pMods, L"LyShark");
}

DbgPrint("KernelBase = > %p \n", g_KernelBase);
return g_KernelBase;
}

```

```

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com \n"));

    // 获取内核第一个模块的基地址
    PVOID base = LySharkToolsUtilKernelBase(0);
    if (!base)
        return STATUS_NOT_FOUND;

    // 得到NT头部PE32+结构
    // lyshark.com
    PIMAGE_NT_HEADERS64 pHdr = RtlImageNtHeader(base);
    if (!pHdr)
        return STATUS_INVALID_IMAGE_FORMAT;

    // 首先寻找代码段
    PIMAGE_SECTION_HEADER pFirstSection = (PIMAGE_SECTION_HEADER)(pHdr + 1);
    for (PIMAGE_SECTION_HEADER pSection = pFirstSection; pSection <
pFirstSection + pHdr->FileHeader.NumberOfSections; pSection++)
    {
        ANSI_STRING LySharkSection, LySharkName;
        RtlInitAnsistring(&LySharkSection, ".text");
        RtlInitAnsistring(&LySharkName, (PCCHAR)pSection->Name);

        DbgPrint("[LyShark.PE] 名字: %Z | 地址: %p | 长度: %d \n", LySharkName,
(PUCHAR)base + pSection->VirtualAddress, pSection->Misc.VirtualSize);
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行这段驱动程序，你会得到当前内核的所有PE节信息，枚举效果如下所示。

```
# T.. Debug Print
1 0... 4696250000 - STORMINI: StorNVMe - POWER: ACTIVE
2 0... 4696250000 - STORMINI: StorNVMe - POWER: IDLE
3 0... 4696406250 - STORMINI: StorNVMe - POWER: ACTIVE
4 0... hello lyshark.com
5 0... KernelBase = > FFFFFFFF8007F600000
6 0... [LyShark.PE] 名字: .text | 地址: FFFFFFFF8007F601000 | 长度: 3469640
7 0... [LyShark.PE] 名字: KVASCODE? | 地址: FFFFFFFF8007F951000 | 长度: 9184
8 0... [LyShark.PE] 名字: RETPOL | 地址: FFFFFFFF8007F954000 | 长度: 1856
9 0... [LyShark.PE] 名字: INITKDBGs? | 地址: FFFFFFFF8007F955000 | 长度: 103795
10 0... [LyShark.PE] 名字: POOLCODE_ | 地址: FFFFFFFF8007F96F000 | 长度: 351
11 0... [LyShark.PE] 名字: .rdata | 地址: FFFFFFFF8007F970000 | 长度: 742976
12 0... [LyShark.PE] 名字: .data | 地址: FFFFFFFF8007FA26000 | 长度: 962264
13 0... [LyShark.PE] 名字: .pdata | 地址: FFFFFFFF8007FB11000 | 长度: 386376
14 0... [LyShark.PE] 名字: .idata | 地址: FFFFFFFF8007FB70000 | 长度: 10480
15 0... [LyShark.PE] 名字: ALMOSTRO | 地址: FFFFFFFF8007FB73000 | 长度: 96480
16 0... [LyShark.PE] 名字: CFGRO | 地址: FFFFFFFF8007FB8B000 | 长度: 7224
```

既然能够得到PE头部数据了，那么我们只需要扫描这段空间并得到匹配到的数据即可，其实很容易实现，如下代码所示。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "lyshark.h"

// 定义全局变量
static PVOID g_KernelBase = 0;
static ULONG g_KernelSize = 0;

// 得到KernelBase基地址
// lyshark.com
PVOID LySharkToolsUtilKernelBase(OUT PULONG pSize)
{
    NTSTATUS status = STATUS_SUCCESS;
    ULONG bytes = 0;
    PRTL_PROCESS_MODULES pMods = 0;
    PVOID checkPtr = 0;
    UNICODE_STRING routineName;

    if (g_KernelBase != 0)
    {
        if (pSize)
        {
            *pSize = g_KernelSize;
        }
        return g_KernelBase;
    }

    RtlInitUnicodeString(&routineName, L"NtOpenFile");

    checkPtr = MmGetSystemRoutineAddress(&routineName);
    if (checkPtr == 0)
        return 0;
}
```



```

__try
{
    status = ZwQuerySystemInformation(SystemModuleInformation, 0, bytes,
&bytes);
    if (bytes == 0)
    {
        return 0;
    }

    pMods = (PRTL_PROCESS_MODULES)ExAllocatePoolWithTag(NonPagedPoolNx,
bytes, L"LyShark");
    RtlZeroMemory(pMods, bytes);

    status = ZwQuerySystemInformation(SystemModuleInformation, pMods, bytes,
&bytes);

    if (NT_SUCCESS(status))
    {
        PRTL_PROCESS_MODULE_INFORMATION pMod = pMods->Modules;

        for (ULONG i = 0; i < pMods->NumberOfModules; i++)
        {
            if (checkPtr >= pMod[i].ImageBase && checkPtr < (PVOID)
((PUCHAR)pMod[i].ImageBase + pMod[i].ImageSize))
            {
                g_KernelBase = pMod[i].ImageBase;
                g_KernelSize = pMod[i].ImageSize;
                if (pSize)
                {
                    *pSize = g_KernelSize;
                }
                break;
            }
        }
    }
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
    return 0;
}

if (pMods)
{
    ExFreePoolWithTag(pMods, L"LyShark");
}

DbgPrint("kernelBase = > %p \n", g_KernelBase);
return g_KernelBase;
}

```

// 对指定内存执行特征码扫描

```

NTSTATUS UtilLySharkSearchPattern(IN PCHAR pattern, IN UCHAR wildcard, IN
ULONG_PTR len, IN const VOID* base, IN ULONG_PTR size, OUT PVOID* ppFound)
{

```

```

NT_ASSERT(ppFound != 0 && pattern != 0 && base != 0);
if (ppFound == 0 || pattern == 0 || base == 0)
{
    return STATUS_INVALID_PARAMETER;
}

__try
{
    for (ULONG_PTR i = 0; i < size - len; i++)
    {
        BOOLEAN found = TRUE;
        for (ULONG_PTR j = 0; j < len; j++)
        {
            if (pattern[j] != wilddcard && pattern[j] != ((PUCHAR)base)[i +
j])
            {
                found = FALSE;
                break;
            }
        }

        if (found != FALSE)
        {
            *ppFound = (PUCHAR)base + i;
            DbgPrint("[LyShark] 特征码匹配地址: %p \n", (PUCHAR)base + i);
            return STATUS_SUCCESS;
        }
    }
}
__except (EXCEPTION_EXECUTE_HANDLER)
{
    return STATUS_UNHANDLED_EXCEPTION;
}

return STATUS_NOT_FOUND;
}

```

// 扫描代码段中的指令片段

NTSTATUS ByLySharkComUtilScanSection(IN PCCHAR section, IN PUCHAR pattern, IN UCHAR wilddcard, IN ULONG_PTR len, OUT PVOID* ppFound)

```

{
    NT_ASSERT(ppFound != 0);
    if (ppFound == 0)
        return STATUS_INVALID_PARAMETER;

    // 获取内核第一个模块的基地址
    PVOID base = LySharkToolsUtilKernelBase(0);
    if (!base)
        return STATUS_NOT_FOUND;

    // 得到NT头部PE32+结构
    PIMAGE_NT_HEADERS64 pHdr = RtlImageNtHeader(base);
    if (!pHdr)
        return STATUS_INVALID_IMAGE_FORMAT;
}

```

```

// 首先寻找代码段
PIMAGE_SECTION_HEADER pFirstSection = (PIMAGE_SECTION_HEADER)(pHdr + 1);
for (PIMAGE_SECTION_HEADER pSection = pFirstSection; pSection <
pFirstSection + pHdr->FileHeader.NumberOfSections; pSection++)
{
    ANSI_STRING LySharkSection, LySharkText;
    RtlInitAnsistring(&LySharkSection, section);
    RtlInitAnsistring(&LySharkText, (PCCHAR)pSection->Name);

    // 判断是不是我们要找的.text节
    if (RtlCompareString(&LySharkSection, &LySharkText, TRUE) == 0)
    {
        // 如果是则开始匹配特征码
        return UtilLySharkSearchPattern(pattern, wildcard, len, (PUCHAR)base
+ pSection->VirtualAddress, pSection->Misc.VirtualSize, ppFound);
    }
}

return STATUS_NOT_FOUND;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK \n"));
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    PMiProcessLoaderEntry m_MiProcessLoaderEntry = NULL;
    RTL_OSVERSIONINFOW Version = { 0 };

    Version.dwOSVersionInfoSize = sizeof(Version);
    RtlGetVersion(&Version);

    //获取内核版本号
    DbgPrint("主版本: %d -->次版本: %d --> 编译版本: %d", Version.dwMajorVersion,
Version.dwMinorVersion, Version.dwBuildNumber);

    if (Version.dwMajorVersion == 10)
    {
        // 如果是 win10 18363 则匹配特征
        if (Version.dwBuildNumber == 18363)
        {
            CHAR pattern[] = "\x48\x89\x5c\x24\x08";
            int pattern_size = sizeof(pattern) - 1;

            ByLySharkComUtilScanSection(".text", (PUCHAR)pattern, 0xcc,
pattern_size, (PVOID *)&m_MiProcessLoaderEntry);
            DbgPrint("[LyShark] 输出首地址: %p", m_MiProcessLoaderEntry);
        }
    }

    Driver->DriverUnload = UnDriver;
}

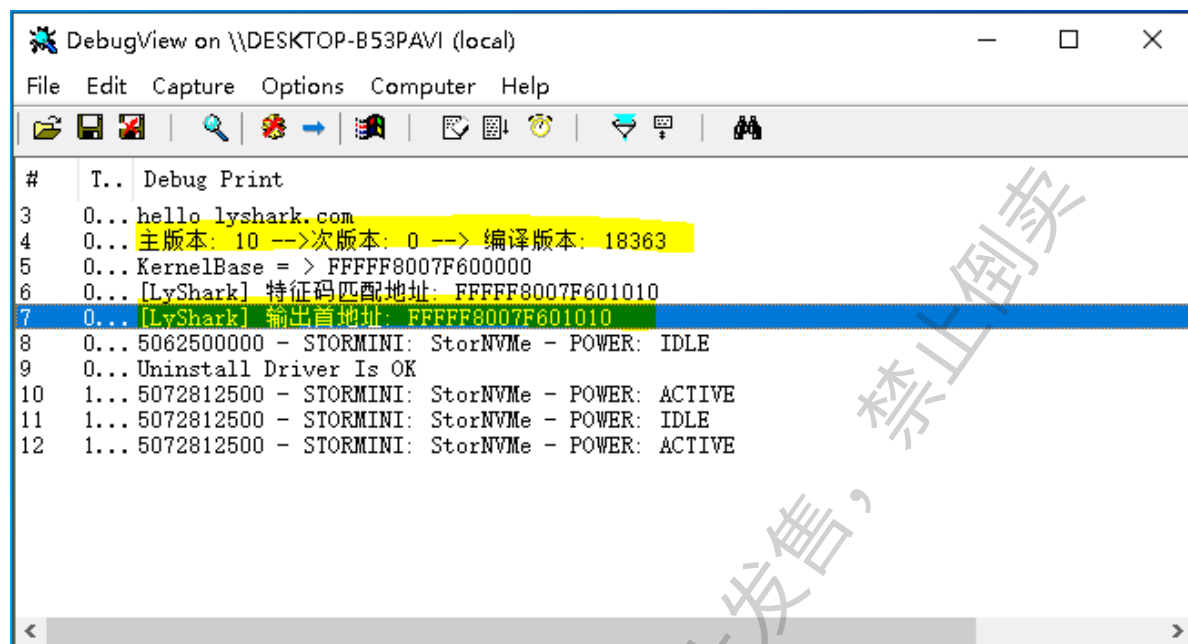
```

```

return STATUS_SUCCESS;
}

```

代码中首先判断系统主版本 windows 10 18363 如果是则执行匹配，只匹配 .text 也就是代码段中的数据，当遇到 0xcc 时则取消继续，否则继续执行枚举，程序输出效果如下所示。

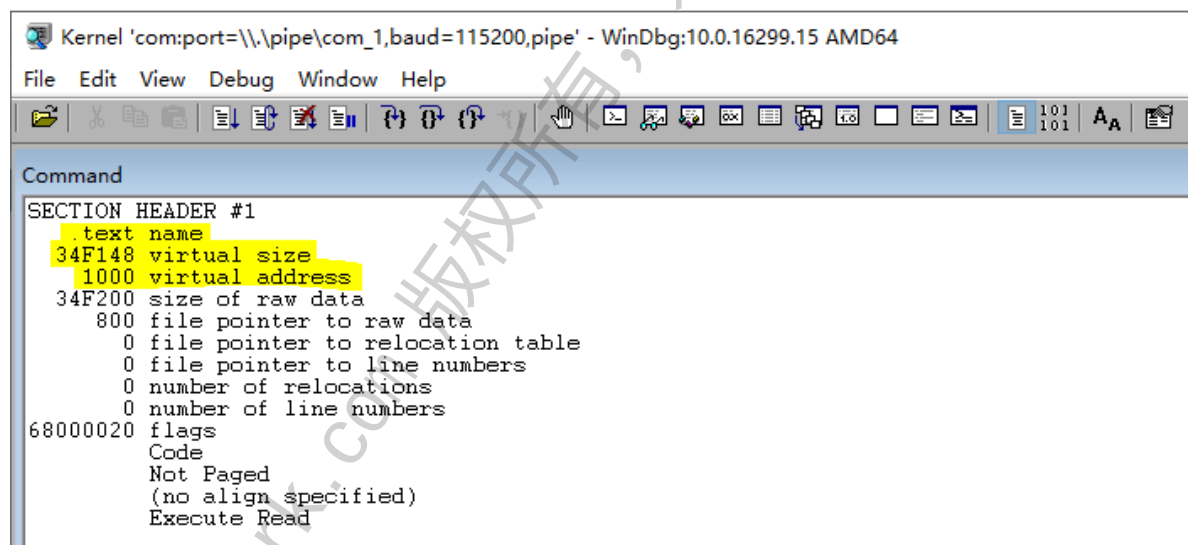


```

# | T.. | Debug Print
3 | 0... | hello lyshark.com
4 | 0... | 主版本: 10 -->次版本: 0 --> 编译版本: 18363
5 | 0... | KernelBase = > FFFFFFFF8007F600000
6 | 0... | [LyShark] 特征码匹配地址: FFFFFFFF8007F601010
7 | 0... | [LyShark] 输出首地址: FFFFFFFF8007F601010
8 | 0... | 506250000 - STORMINI: StorNVMe - POWER: IDLE
9 | 0... | Uninstall Driver Is OK
10 | 1... | 5072812500 - STORMINI: StorNVMe - POWER: ACTIVE
11 | 1... | 5072812500 - STORMINI: StorNVMe - POWER: IDLE
12 | 1... | 5072812500 - STORMINI: StorNVMe - POWER: ACTIVE

```

在WinDBG中输入命令 `!dh 0xffffffff8007f600000` 解析出内核PE头数据，可以看到如下所示，对比无误。



```

Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
SECTION HEADER #1
.text name
34F148 virtual size
1000 virtual address
34F200 size of raw data
800 file pointer to raw data
0 file pointer to relocation table
0 file pointer to line numbers
0 number of relocations
0 number of line numbers
68000020 flags
Code
Not Paged
(no align specified)
Execute Read

```

8.6 内核枚举LoadImage映像回调

在笔者之前的文章《驱动开发：内核特征码搜索函数封装》中我们封装实现了特征码定位功能，本章将继续使用该功能，本次我们需要枚举内核 LoadImage 映像回调，在Win64环境下我们可以设置一个 LoadImage 映像加载通告回调，当有新驱动或者DLL被加载时，回调函数就会被调用从而执行我们自己的回调例程，映像回调也存储在数组里，枚举时从数组中读取取值之后，需要进行位运算解密得到地址。

我们来看一款闭源ARK工具是如何实现的：

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
系统回调	过滤驱动	DPC定时器	IO定时器	系统线程	卸载的驱动							
回调入口	通知类型	模块路径										
0xFFFFF80180D1D760	ThreadObCall	C:\Windows\System32\drivers\PYArkSafe.sys										
0xFFFFF8018519D890	Registry	C:\Windows\system32\drivers\WdFilter.sys										
0xFFFFF80180465BE0	Registry	C:\Windows\system32\ntoskrnl.exe										
0xFFFFF801851A8410	ProcessObCall	C:\Windows\system32\drivers\WdFilter.sys										
0xFFFFF80180D1D420	ProcessObCall	C:\Windows\System32\drivers\PYArkSafe.sys										
0xFFFFF80180D1C550	LoadImage	C:\Windows\System32\drivers\PYArkSafe.sys										
0xFFFFF801869FB210	LoadImage	C:\Windows\system32\DRIVERS\ahcache.sys										
0xFFFFF801851AABA0	LoadImage	C:\Windows\system32\drivers\WdFilter.sys										

如上所述，如果我们需要拿到回调数组那么首先要得到该数组，数组的符号名是

`PspLoadImageNotifyRoutine` 我们可以在 `PsSetLoadImageNotifyRoutineEx` 中找到。

第一步使用WinDBG输入 `uf PsSetLoadImageNotifyRoutineEx` 首先定位到，能够找到

`PsSetLoadImageNotifyRoutineEx` 这里的两个位置都可以被引用，当然了，这个函数可以直接通过

`PsSetLoadImageNotifyRoutineEx` 函数动态拿到此处不需要我们动态定位。

```
Command
nt!PsSetLoadImageNotifyRoutineEx+0x41:
fffff801`80748a81 488d0dd8d3dbff lea rcx,[nt!PspLoadImageNotifyRoutine (fffff801`80505e60)]
fffff801`80748a88 4533c0 xor r8d,r8d
fffff801`80748a8b 488d0cd9 lea rcx,[rcx+rbx*8]
fffff801`80748a8f 488bd7 mov rdx,rdi
fffff801`80748a92 e80584a3ff call nt!ExCompareExchangeCallback (fffff801`80180e9c)
fffff801`80748a97 84c0 test al,al
fffff801`80748a99 0f849f000000 je nt!PsSetLoadImageNotifyRoutineEx+0xfe (fffff801`80748b3e) Branch

nt!PsSetLoadImageNotifyRoutineEx+0x5f:
fffff801`80748a9f f0ff05966e2600 lock inc dword ptr [nt!PspLoadImageNotifyRoutineCount (fffff801`809af93c)]
fffff801`80748aa6 8b05446b2600 mov eax,dword ptr [nt!PspNotifyEnableMask (fffff801`809af5f0)]
fffff801`80748aac a801 test al,1
fffff801`80748aae 7509 jne nt!PsSetLoadImageNotifyRoutineEx+0x79 (fffff801`80748ab9) Branch
```

我们通过获取到 `PsSetLoadImageNotifyRoutineEx` 函数的内存首地址，然后向下匹配特征码搜索找到 `488d0dd8e8dbff` 并取出 `PspLoadImageNotifyRoutine` 内存地址，该内存地址就是 `LoadImage` 映像模块的基址。

```
Command
nt!PsSetLoadImageNotifyRoutineEx+0x41:
fffff801`80748a81 488d0dd8d3dbff lea rcx,[nt!PspLoadImageNotifyRoutine (fffff801`80505e60)]
fffff801`80748a88 4533c0 xor r8d,r8d
fffff801`80748a8b 488d0cd9 lea rcx,[rcx+rbx*8]
fffff801`80748a8f 488bd7 mov rdx,rdi
fffff801`80748a92 e80584a3ff call nt!ExCompareExchangeCallback (fffff801`80180e9c)
fffff801`80748a97 84c0 test al,al
fffff801`80748a99 0f849f000000 je nt!PsSetLoadImageNotifyRoutineEx+0xfe (fffff801`80748b3e) Branch
```

如果使用代码去定位这段空间，则你可以这样写，这样即可得到具体特征地址。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntddk.h>
#include <windef.h>

// 指定内存区域的特征码扫描
PVOID SearchMemory(PVOID pStartAddress, PVOID pEndAddress, PCHAR pMemoryData,
ULONG ulMemoryDataSize)
{
    PVOID pAddress = NULL;
    PCHAR i = NULL;
    ULONG m = 0;
```

```

// 扫描内存
for (i = (PUCHAR)pStartAddress; i < (PUCHAR)pEndAddress; i++)
{
    // 判断特征码
    for (m = 0; m < ulMemoryDataSize; m++)
    {
        if (*(PUCHAR)(i + m) != pMemoryData[m])
        {
            break;
        }
    }
    // 判断是否找到符合特征码的地址
    if (m >= ulMemoryDataSize)
    {
        // 找到特征码位置，获取紧接着特征码的下一地址
        pAddress = (PVOID)(i + ulMemoryDataSize);
        break;
    }
}

return pAddress;
}

// 根据特征码获取 PspLoadImageNotifyRoutine 数组地址
PVOID SearchPspLoadImageNotifyRoutine(PUCHAR pSpecialData, ULONG
ulSpecialDataSize)
{
    UNICODE_STRING ustrFuncName;
    PVOID pAddress = NULL;
    LONG lOffset = 0;
    PVOID pPsSetLoadImageNotifyRoutine = NULL;
    PVOID pPspLoadImageNotifyRoutine = NULL;

    // 先获取 PsSetLoadImageNotifyRoutineEx 函数地址
    RtlInitUnicodeString(&ustrFuncName, L"PsSetLoadImageNotifyRoutineEx");
    pPsSetLoadImageNotifyRoutine = MmGetSystemRoutineAddress(&ustrFuncName);
    if (NULL == pPsSetLoadImageNotifyRoutine)
    {
        return pPspLoadImageNotifyRoutine;
    }

    // 查找 PspLoadImageNotifyRoutine 函数地址
    pAddress = SearchMemory(pPsSetLoadImageNotifyRoutine, (PVOID)
((PUCHAR)pPsSetLoadImageNotifyRoutine + 0xFF), pSpecialData, ulSpecialDataSize);
    if (NULL == pAddress)
    {
        return pPspLoadImageNotifyRoutine;
    }

    // 先获取偏移，再计算地址
    lOffset = *(PLONG)pAddress;
    pPspLoadImageNotifyRoutine = (PVOID)((PUCHAR)pAddress + sizeof(LONG) +
lOffset);

    return pPspLoadImageNotifyRoutine;
}

```

```

}

VOID UnDriver(PDRIVER_OBJECT Driver)
{
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    PVOID pPspLoadImageNotifyRoutineAddress = NULL;
    RTL_OSVERSIONINFOW osInfo = { 0 };
    UCHAR pSpecialData[50] = { 0 };
    ULONG ulSpecialDataSize = 0;

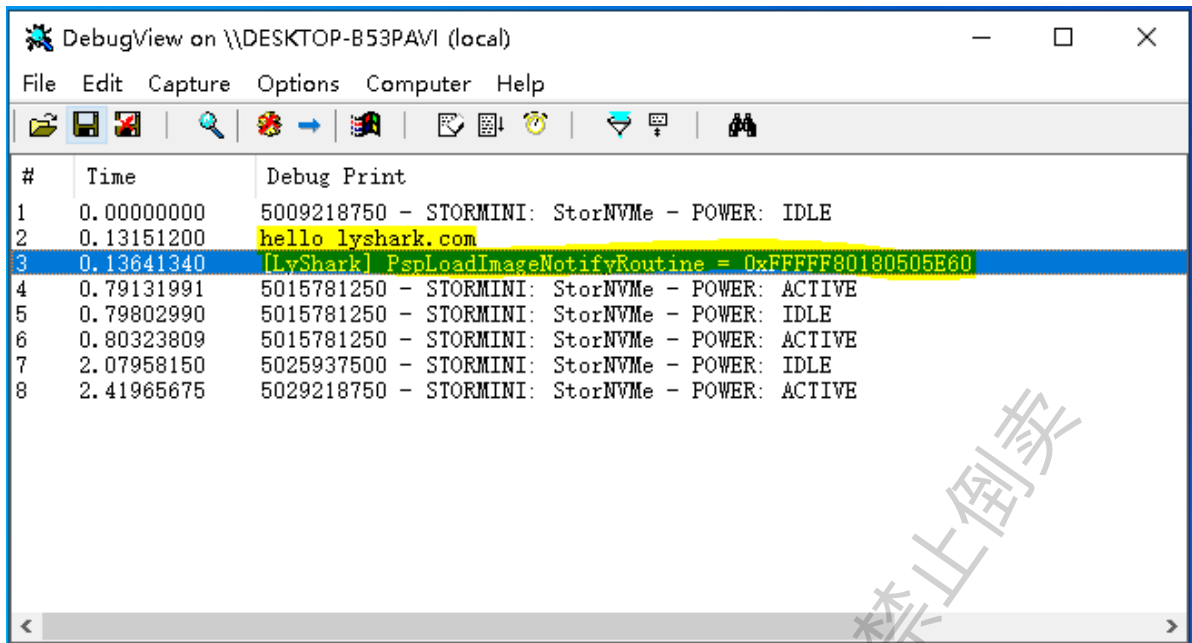
    // 获取系统版本信息，判断系统版本
    RtlGetVersion(&osInfo);
    if (10 == osInfo.dwMajorVersion)
    {
        // 48 8d 0d 88 e8 db ff
        // 查找指令 lea rcx,[nt!PspLoadImageNotifyRoutine (fffff804`44313ce0)]
        /*
        nt!PsSetLoadImageNotifyRoutineEx+0x41:
        ffffff801`80748a81 488d0dd8d3dbff lea rcx,
[nt!PspLoadImageNotifyRoutine (fffff801`80505e60)]
        ffffff801`80748a88 4533c0 xor r8d,r8d
        ffffff801`80748a8b 488d0cd9 lea rcx,[rcx+rbx*8]
        ffffff801`80748a8f 488bd7 mov rdx,rdi
        ffffff801`80748a92 e80584a3ff call nt!ExCompareExchangeCallback
(fffff801`80180e9c)
        ffffff801`80748a97 84c0 test al,al
        ffffff801`80748a99 0f849f000000 je
nt!PsSetLoadImageNotifyRoutineEx+0xfe (fffff801`80748b3e) Branch
        */
        pSpecialData[0] = 0x48;
        pSpecialData[1] = 0x8D;
        pSpecialData[2] = 0x0D;
        ulSpecialDataSize = 3;
    }

    // 根据特征码获取地址 获取 PspLoadImageNotifyRoutine 数组地址
    pPspLoadImageNotifyRoutineAddress =
SearchPspLoadImageNotifyRoutine(pSpecialData, ulSpecialDataSize);
    DbgPrint("[LyShark] PspLoadImageNotifyRoutine = 0x%p \n",
pPspLoadImageNotifyRoutineAddress);

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

将这个驱动拖入到虚拟机中并运行，输出结果如下：



有了数组地址接下来就是要对数组进行解密，如何解密？

- 1.首先拿到数组指针 `pPspLoadImageNotifyRoutineAddress + sizeof(PVOID) * i` 此处的i也就是下标。
- 2.得到的新地址在与 `pNotifyRoutineAddress & 0xfffffffffffff8` 进行与运算。
- 3.最后 `*(PVOID *)pNotifyRoutineAddress` 取出里面的参数。

增加解密代码以后，这段程序的完整代码也就可以被写出来了，如下所示。

```
// 署名
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntddk.h>
#include <windef.h>

// 指定内存区域的特征码扫描
PVOID SearchMemory(PVOID pStartAddress, PVOID pEndAddress, PCHAR pMemoryData,
ULONG ulMemoryDataSize)
{
    PVOID pAddress = NULL;
    PCHAR i = NULL;
    ULONG m = 0;

    // 扫描内存
    for (i = (PCHAR)pStartAddress; i < (PCHAR)pEndAddress; i++)
    {
        // 判断特征码
        for (m = 0; m < ulMemoryDataSize; m++)
        {
            if (*(PCHAR)(i + m) != pMemoryData[m])
            {
                break;
            }
        }
        // 判断是否找到符合特征码的地址
        if (m >= ulMemoryDataSize)
        {
            return i;
        }
    }
    return NULL;
}
```



```

        // 找到特征码位置，获取紧接着特征码的下一地址
        pAddress = (PVOID)(i + ulMemoryDataSize);
        break;
    }
}

return pAddress;
}

// 根据特征码获取 PspLoadImageNotifyRoutine 数组地址
PVOID SearchPspLoadImageNotifyRoutine(PUCHAR pSpecialData, ULONG
ulSpecialDataSize)
{
    UNICODE_STRING ustrFuncName;
    PVOID pAddress = NULL;
    LONG lOffset = 0;
    PVOID pPsSetLoadImageNotifyRoutine = NULL;
    PVOID pPspLoadImageNotifyRoutine = NULL;

    // 先获取 PsSetLoadImageNotifyRoutineEx 函数地址
    RtlInitUnicodeString(&ustrFuncName, L"PsSetLoadImageNotifyRoutineEx");
    pPsSetLoadImageNotifyRoutine = MmGetSystemRoutineAddress(&ustrFuncName);
    if (NULL == pPsSetLoadImageNotifyRoutine)
    {
        return pPspLoadImageNotifyRoutine;
    }

    // 查找 PspLoadImageNotifyRoutine 函数地址
    pAddress = SearchMemory(pPsSetLoadImageNotifyRoutine, (PVOID)
((PUCHAR)pPsSetLoadImageNotifyRoutine + 0xFF), pSpecialData, ulSpecialDataSize);
    if (NULL == pAddress)
    {
        return pPspLoadImageNotifyRoutine;
    }

    // 先获取偏移，再计算地址
    lOffset = *(PLONG)pAddress;
    pPspLoadImageNotifyRoutine = (PVOID)((PUCHAR)pAddress + sizeof(LONG) +
lOffset);

    return pPspLoadImageNotifyRoutine;
}

// 移除回调
NTSTATUS RemoveNotifyRoutine(PVOID pNotifyRoutineAddress)
{
    NTSTATUS status =
PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)pNotifyRoutineAddress
);
    return status;
}

VOID UnDriver(PDRIVER_OBJECT Driver)
{
}

```

```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    PVOID pPspLoadImageNotifyRoutineAddress = NULL;
    RTL_OSVERSIONINFOW osInfo = { 0 };
    UCHAR pSpecialData[50] = { 0 };
    ULONG ulSpecialDataSize = 0;

    // 获取系统版本信息，判断系统版本
    RtlGetVersion(&osInfo);
    if (10 == osInfo.dwMajorVersion)
    {
        // 48 8d 0d 88 e8 db ff
        // 查找指令 lea rcx,[nt!PspLoadImageNotifyRoutine (fffff804`44313ce0)]
        /*
        nt!PsSetLoadImageNotifyRoutineEx+0x41:
        fffff801`80748a81 488d0dd8d3dbff lea rcx,
[nt!PspLoadImageNotifyRoutine (fffff801`80505e60)]
        fffff801`80748a88 4533c0 xor r8d,r8d
        fffff801`80748a8b 488d0cd9 lea rcx,[rcx+rbx*8]
        fffff801`80748a8f 488bd7 mov rdx,rdi
        fffff801`80748a92 e80584a3ff call nt!ExCompareExchangeCallback
(fffff801`80180e9c)
        fffff801`80748a97 84c0 test al,al
        fffff801`80748a99 0f849f000000 je
nt!PsSetLoadImageNotifyRoutineEx+0xfe (fffff801`80748b3e) Branch
        */
        pSpecialData[0] = 0x48;
        pSpecialData[1] = 0x8D;
        pSpecialData[2] = 0x0D;
        ulSpecialDataSize = 3;
    }

    // 根据特征码获取地址 获取 PspLoadImageNotifyRoutine 数组地址
    pPspLoadImageNotifyRoutineAddress =
SearchPspLoadImageNotifyRoutine(pSpecialData, ulSpecialDataSize);
    DbgPrint("[LyShark] PspLoadImageNotifyRoutine = 0x%p \n",
pPspLoadImageNotifyRoutineAddress);

    // 遍历回调
    ULONG i = 0;
    PVOID pNotifyRoutineAddress = NULL;

    // 获取 PspLoadImageNotifyRoutine 数组地址
    if (NULL == pPspLoadImageNotifyRoutineAddress)
    {
        return FALSE;
    }

    // 获取回调地址并解密
    for (i = 0; i < 64; i++)
    {

```


8.7 内核枚举Registry注册表回调

在笔者上一篇文章《驱动开发：内核枚举LoadImage映像回调》中 LyShark 教大家实现了枚举系统回调中的 LoadImage 通知消息，本章将实现对 Registry 注册表通知消息的枚举，与 LoadImage 消息不同 Registry 消息不需要解密只要找到 CallbackListHead 消息回调链表头并解析为 _CM_NOTIFY_ENTRY 结构即可实现枚举。

我们来看一款闭源ARK工具是如何实现的：

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
系统回调	过滤驱动	DPC定时器	IO定时器	系统线程	卸载的驱动							
回调入口		通知类型										
0xFFFFF80636A2D760		ThreadObCall										
0xFFFFF8063C39D890		Registry										
0xFFFFF8063A065BE0		Registry										
0xFFFFF8063C3A8410		ProcessObCall										
0xFFFFF80636A2D420		ProcessObCall										
0xFFFFF80636A2C550		LoadImage										

注册表系统回调的枚举需要通过特征码搜索来实现，首先我们可以定位到 uf CmUnRegisterCallback 内核函数上，在该内核函数下方存在一个 CallbackListHead 链表节点，取出这个链表地址。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
Command
nt!CmUnRegisterCallback+0x6b:
fffff806`3a4271ab 4533c0          xor     r8d,r8d
fffff806`3a4271ae 488d542438      lea     rdx,[rsp+38h]
fffff806`3a4271b3 488d0d06eac3ff lea     rcx,[nt!CallbackListHead (fffff806`3a065bc0)]
fffff806`3a4271ba e855e2e2ff      call    nt!CmListGetNextElement (fffff806`3a255414)
fffff806`3a4271bf 488bf8          mov     rdi,rax
fffff806`3a4271c2 4889442440      mov     qword ptr [rsp+40h],rax
fffff806`3a4271c7 4885c0          test    rax,rax
fffff806`3a4271ca 0f84c7000000    je      nt!CmUnRegisterCallback+0x157 (fffff806`3a427297)

nt!CmUnRegisterCallback+0x90:
fffff806`3a4271d0 48395818        cmp     qword ptr [rax+18h],rbx
fffff806`3a4271d4 75d5            jne     nt!CmUnRegisterCallback+0x6b (fffff806`3a4271ab)
```

当得到注册表链表入口 0xfffff8063a065bc0 直接将其解析为 _CM_NOTIFY_ENTRY 即可得到数据，如果要遍历下一个链表则只需要 ListEntryHead.Flink 向下移动指针即可。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
```

```
// 注册表回调函数结构体定义
```

```
typedef struct _CM_NOTIFY_ENTRY
{
    LIST_ENTRY ListEntryHead;
    ULONG Unknown1;
    ULONG Unknown2;
    LARGE_INTEGER Cookie;
    PVOID Context;
    PVOID Function;
}CM_NOTIFY_ENTRY, *PCM_NOTIFY_ENTRY;
```

要想得到此处的链表地址，需要先通过 `MmGetSystemRoutineAddress()` 获取到

`CmUnRegisterCallback` 函数基址，然后在该函数起始位置向下搜索，找到这个链表节点，并将其后面的基址取出来，在上一篇《驱动开发：内核枚举 `LoadImage` 映像回调》文章中已经介绍了定位方式此处跳过介绍，具体实现代码如下。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#include <windef.h>

// 指定内存区域的特征码扫描
// PowerBy: LyShark.com
PVOID SearchMemory(PVOID pStartAddress, PVOID pEndAddress, PCHAR pMemoryData,
    ULONG ulMemoryDataSize)
{
    PVOID pAddress = NULL;
    PCHAR i = NULL;
    ULONG m = 0;

    // 扫描内存
    for (i = (PCHAR)pStartAddress; i < (PCHAR)pEndAddress; i++)
    {
        // 判断特征码
        for (m = 0; m < ulMemoryDataSize; m++)
        {
            if (*(PCHAR)(i + m) != pMemoryData[m])
            {
                break;
            }
        }
        // 判断是否找到符合特征码的地址
        if (m >= ulMemoryDataSize)
        {
            // 找到特征码位置，获取紧接着特征码的下一地址
            pAddress = (PVOID)(i + ulMemoryDataSize);
            break;
        }
    }

    return pAddress;
}

// 根据特征码获取 CallbackListHead 链表地址
// PowerBy: LyShark.com
PVOID SearchCallbackListHead(PCHAR pSpecialData, ULONG ulSpecialDataSize, LONG lSpecialOffset)
{
    UNICODE_STRING ustrFuncName;
    PVOID pAddress = NULL;
    LONG loffset = 0;
    PVOID pCmUnRegisterCallback = NULL;
    PVOID pCallbackListHead = NULL;
```

```

// 先获取 CmUnRegisterCallback 函数地址
RtlInitUnicodeString(&ustrFuncName, L"CmUnRegisterCallback");
pCmUnRegisterCallback = MmGetSystemRoutineAddress(&ustrFuncName);
if (NULL == pCmUnRegisterCallback)
{
    return pCallbackListHead;
}

// 查找 fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffff806`3a065bc0)]
/*
lyshark.com>
nt!CmUnRegisterCallback+0x6b:
fffff806`3a4271ab 4533c0 xor r8d,r8d
fffff806`3a4271ae 488d542438 lea rdx,[rsp+38h]
fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffff806`3a065bc0)]
fffff806`3a4271ba e855e2e2ff call nt!CmListGetNextElement
(fffff806`3a255414)
fffff806`3a4271bf 488bf8 mov rdi,rax
fffff806`3a4271c2 4889442440 mov qword ptr [rsp+40h],rax
fffff806`3a4271c7 4885c0 test rax,rax
fffff806`3a4271ca 0f84c7000000 je nt!CmUnRegisterCallback+0x157
(fffff806`3a427297) Branch
*/
pAddress = SearchMemory(pCmUnRegisterCallback, (PVOID)
((PUCHAR)pCmUnRegisterCallback + 0xFF), pSpecialData, ulSpecialDataSize);
if (NULL == pAddress)
{
    return pCallbackListHead;
}

// 先获取偏移再计算地址
loffset = *(PLONG)((PUCHAR)pAddress + 1*SpecialOffset);
pCallbackListHead = (PVOID)((PUCHAR)pAddress + 1*SpecialOffset + sizeof(LONG)
+ loffset);

return pCallbackListHead;
}

VOID UnDriver(PDRIVER_OBJECT Driver)
{
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    PVOID pCallbackListHeadAddress = NULL;
    RTL_OSVERSIONINFOW osInfo = { 0 };
    UCHAR pSpecialData[50] = { 0 };
    ULONG ulSpecialDataSize = 0;
    LONG lSpecialOffset = 0;

    DbgPrint("hello lyshark.com \n");
}

```

```

// 查找 fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffff806`3a065bc0)]
/*
lyshark.com>
nt!CmUnRegisterCallback+0x6b:
fffff806`3a4271ab 4533c0 xor r8d,r8d
fffff806`3a4271ae 488d542438 lea rdx,[rsp+38h]
fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffff806`3a065bc0)]
fffff806`3a4271ba e855e2e2ff call nt!CmListGetNextElement
(fffff806`3a255414)
fffff806`3a4271bf 488bf8 mov rdi,rax
fffff806`3a4271c2 4889442440 mov qword ptr [rsp+40h],rax
fffff806`3a4271c7 4885c0 test rax,rax
fffff806`3a4271ca 0f84c7000000 je nt!CmUnRegisterCallback+0x157
(fffff806`3a427297) Branch
*/
pSpecialData[0] = 0x48;
pSpecialData[1] = 0x80;
pSpecialData[2] = 0x00;
ulSpecialDataSize = 3;

// 根据特征码获取地址
pCallbackListHeadAddress = SearchCallbackListHead(pSpecialData,
ulSpecialDataSize, lSpecialOffset);

DbgPrint("[LyShark.com] CallbackListHead => %p \n",
pCallbackListHeadAddress);

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

运行这段代码，并可得到注册表回调入口地址，输出效果如下所示：

#	Time	Debug Print
1	0.00000000	4836406250 - STORMINI: StorNVMe - POWER: IDLE
2	0.34673479	4839687500 - STORMINI: StorNVMe - POWER: ACTIVE
3	1.06057155	hello lyshark.com
4	1.06527448	[LyShark.com] CallbackListHead => FFFFF8063A065BC0
5	1.48369396	4849843750 - STORMINI: StorNVMe - POWER: IDLE
6	1.62090075	4851093750 - STORMINI: StorNVMe - POWER: ACTIVE
7	2.89499855	4861250000 - STORMINI: StorNVMe - POWER: IDLE
8	3.17070532	4863906250 - STORMINI: StorNVMe - POWER: ACTIVE

得到了注册表回调入口地址，接着直接循环遍历输出这个链表即可得到所有的注册表回调。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#include <windef.h>

// 指定内存区域的特征码扫描
// PowerBy: LyShark.com
PVOID SearchMemory(PVOID pStartAddress, PVOID pEndAddress, PCHAR pMemoryData,
ULONG ulMemoryDataSize)
{
    PVOID pAddress = NULL;
    PCHAR i = NULL;
    ULONG m = 0;

    // 扫描内存
    for (i = (PCHAR)pStartAddress; i < (PCHAR)pEndAddress; i++)
    {
        // 判断特征码
        for (m = 0; m < ulMemoryDataSize; m++)
        {
            if (*(PCHAR)(i + m) != pMemoryData[m])
            {
                break;
            }
        }
        // 判断是否找到符合特征码的地址
        if (m >= ulMemoryDataSize)
        {
            // 找到特征码位置，获取紧接着特征码的下一地址
            pAddress = (PVOID)(i + ulMemoryDataSize);
            break;
        }
    }

    return pAddress;
}

// 根据特征码获取 CallbackListHead 链表地址
// PowerBy: LyShark.com
PVOID SearchCallbackListHead(PCHAR pSpecialData, ULONG ulSpecialDataSize, LONG
lSpecialOffset)
{
    UNICODE_STRING ustrFuncName;
    PVOID pAddress = NULL;
    LONG lOffset = 0;
    PVOID pCmUnRegisterCallback = NULL;
    PVOID pCallbackListHead = NULL;

    // 先获取 CmUnRegisterCallback 函数地址
    RtlInitUnicodeString(&ustrFuncName, L"CmUnRegisterCallback");
    pCmUnRegisterCallback = MmGetSystemRoutineAddress(&ustrFuncName);
    if (NULL == pCmUnRegisterCallback)

```



```

{
    return pCallbackListHead;
}

// 查找 ffffffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffffffff806`3a065bc0)]
/*
lyshark.com>
nt!CmUnRegisterCallback+0x6b:
fffff806`3a4271ab 4533c0 xor r8d,r8d
fffff806`3a4271ae 488d542438 lea rdx,[rsp+38h]
fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
(fffffffff806`3a065bc0)]
fffff806`3a4271ba e855e2e2ff call nt!CmListGetNextElement
(fffffffff806`3a255414)
fffff806`3a4271bf 488bf8 mov rdi,rax
fffff806`3a4271c2 4889442440 mov qword ptr [rsp+40h],rax
fffff806`3a4271c7 4885c0 test rax,rax
fffff806`3a4271ca 0f84c7000000 je nt!CmUnRegisterCallback+0x157
(fffffffff806`3a427297) Branch
*/
pAddress = SearchMemory(pCmUnRegisterCallback, (PVOID)
((PUCHAR)pCmUnRegisterCallback + 0xFF), pSpecialData, ulSpecialDataSize);
if (NULL == pAddress)
{
    return pCallbackListHead;
}

// 先获取偏移再计算地址
lOffset = *(PLONG)((PUCHAR)pAddress + lSpecialOffset);
pCallbackListHead = (PVOID)((PUCHAR)pAddress + lSpecialOffset + sizeof(LONG)
+ lOffset);

return pCallbackListHead;
}

// 注册表回调函数结构体定义
typedef struct _CM_NOTIFY_ENTRY
{
    LIST_ENTRY ListEntryHead;
    ULONG Unknown1;
    ULONG Unknown2;
    LARGE_INTEGER Cookie;
    PVOID Context;
    PVOID Function;
}CM_NOTIFY_ENTRY, *PCM_NOTIFY_ENTRY;

VOID UnDriver(PDRIVER_OBJECT Driver)
{
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    PVOID pCallbackListHeadAddress = NULL;
    RTL_OSVERSIONINFOW osInfo = { 0 };

```

```

    UCHAR pSpecialData[50] = { 0 };
    ULONG ulSpecialDataSize = 0;
    LONG lSpecialOffset = 0;

    DbgPrint("hello lyshark.com \n");

    // 查找 fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
    (fffff806`3a065bc0)]
    /*
    lyshark.com>
    nt!CmUnRegisterCallback+0x6b:
    fffff806`3a4271ab 4533c0 xor r8d,r8d
    fffff806`3a4271ae 488d542438 lea rdx,[rsp+38h]
    fffff806`3a4271b3 488d0d06eac3ff lea rcx,[nt!CallbackListHead
    (fffff806`3a065bc0)]
    fffff806`3a4271ba e855e2e2ff call nt!CmListGetNextElement
    (fffff806`3a255414)
    fffff806`3a4271bf 488bf8 mov rdi,rax
    fffff806`3a4271c2 4889442440 mov qword ptr [rsp+40h],rax
    fffff806`3a4271c7 4885c0 test rax,rax
    fffff806`3a4271ca 0f84c7000000 je nt!CmUnRegisterCallback+0x157
    (fffff806`3a427297) Branch
    */
    pSpecialData[0] = 0x48;
    pSpecialData[1] = 0x8D;
    pSpecialData[2] = 0x0D;
    ulSpecialDataSize = 3;

    // 根据特征码获取地址
    pCallbackListHeadAddress = SearchCallbackListHead(pSpecialData,
    ulSpecialDataSize, lSpecialOffset);

    DbgPrint("[LyShark.com] CallbackListHead => %p \n",
    pCallbackListHeadAddress);

    // 遍历链表结构
    ULONG i = 0;
    PCM_NOTIFY_ENTRY pNotifyEntry = NULL;

    if (NULL == pCallbackListHeadAddress)
    {
        return FALSE;
    }

    // 开始遍历双向链表
    pNotifyEntry = (PCM_NOTIFY_ENTRY)pCallbackListHeadAddress;
    do
    {
        // 判断pNotifyEntry地址是否有效
        if (FALSE == MmIsValidAddress(pNotifyEntry))
        {
            break;
        }

        // 判断回调函数地址是否有效
        if (MmIsValidAddress(pNotifyEntry->Function))

```



```

typedef struct _OBJECT_TYPE_INITIALIZER
{
    USHORT Length;                // Uint2B
    UCHAR ObjectTypeFlags;        // UChar
    ULONG ObjectTypeCode;         // Uint4B
    ULONG InvalidAttributes;      // Uint4B
    GENERIC_MAPPING GenericMapping; // _GENERIC_MAPPING
    ULONG ValidAccessMask;        // Uint4B
    ULONG RetainAccess;           // Uint4B
    POOL_TYPE PoolType;           // _POOL_TYPE
    ULONG DefaultPagedPoolCharge; // Uint4B
    ULONG DefaultNonPagedPoolCharge; // Uint4B
    PVOID DumpProcedure;          // Ptr64    void
    PVOID OpenProcedure;          // Ptr64    long
    PVOID CloseProcedure;         // Ptr64    void
    PVOID DeleteProcedure;        // Ptr64    void
    PVOID ParseProcedure;         // Ptr64    long
    PVOID SecurityProcedure;       // Ptr64    long
    PVOID QueryNameProcedure;      // Ptr64    long
    PVOID OkayToCloseProcedure;    // Ptr64    unsigned char
    ULONG WaitObjectFlagMask;     // Uint4B
    USHORT WaitObjectFlagOffset;  // Uint2B
    USHORT WaitObjectPointerOffset; // Uint2B
}OBJECT_TYPE_INITIALIZER, *POBJECT_TYPE_INITIALIZER;

typedef struct _OBJECT_TYPE
{
    LIST_ENTRY TypeList;          // _LIST_ENTRY
    UNICODE_STRING Name;          // _UNICODE_STRING
    PVOID DefaultObject;          // Ptr64 Void
    UCHAR Index;                  // UChar
    ULONG TotalNumberOfObjects;    // Uint4B
    ULONG TotalNumberOfHandles;    // Uint4B
    ULONG HighWaterNumberOfObjects; // Uint4B
    ULONG HighWaterNumberOfHandles; // Uint4B
    OBJECT_TYPE_INITIALIZER TypeInfo; // _OBJECT_TYPE_INITIALIZER
    EX_PUSH_LOCK TypeLock;        // _EX_PUSH_LOCK
    ULONG Key;                    // Uint4B
    LIST_ENTRY CallbackList;      // _LIST_ENTRY
}OBJECT_TYPE, *POBJECT_TYPE;

#pragma pack(1)
typedef struct _OB_CALLBACK
{
    LIST_ENTRY ListEntry;
    ULONGLONG Unknown;
    HANDLE ObHandle;
    PVOID ObTypeAddr;
    PVOID PreCall;
    PVOID PostCall;
}OB_CALLBACK, *POB_CALLBACK;

#pragma pack()

```

代码部分的实现很容易，由于进程与线程句柄的枚举很容易，直接通过 (POBJECT_TYPE) (*PsProcessType)->CallbackList 就可以拿到链表头结构，得到后将其解析为 POB_CALLBACK 并循环输出即可。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>
#include <wdm.h>
#include <ntddk.h>

typedef struct _OBJECT_TYPE_INITIALIZER
{
    USHORT Length; // Uint2B
    UCHAR ObjectTypeFlags; // Uchar
    ULONG ObjectTypeCode; // Uint4B
    ULONG InvalidAttributes; // Uint4B
    GENERIC_MAPPING GenericMapping; // _GENERIC_MAPPING
    ULONG ValidAccessMask; // Uint4B
    ULONG RetainAccess; // Uint4B
    POOL_TYPE PoolType; // _POOL_TYPE
    ULONG DefaultPagedPoolCharge; // Uint4B
    ULONG DefaultNonPagedPoolCharge; // Uint4B
    PVOID DumpProcedure; // Ptr64 void
    PVOID OpenProcedure; // Ptr64 long
    PVOID CloseProcedure; // Ptr64 void
    PVOID DeleteProcedure; // Ptr64 void
    PVOID ParseProcedure; // Ptr64 long
    PVOID SecurityProcedure; // Ptr64 long
    PVOID QueryNameProcedure; // Ptr64 long
    PVOID OkayToCloseProcedure; // Ptr64 unsigned char
    ULONG WaitObjectFlagMask; // Uint4B
    USHORT WaitObjectFlagOffset; // Uint2B
    USHORT WaitObjectPointerOffset; // Uint2B
} OBJECT_TYPE_INITIALIZER, *POBJECT_TYPE_INITIALIZER;

typedef struct _OBJECT_TYPE
{
    LIST_ENTRY TypeList; // _LIST_ENTRY
    UNICODE_STRING Name; // _UNICODE_STRING
    PVOID DefaultObject; // Ptr64 Void
    UCHAR Index; // Uchar
    ULONG TotalNumberOfObjects; // Uint4B
    ULONG TotalNumberOfHandles; // Uint4B
    ULONG HighWaterNumberOfObjects; // Uint4B
    ULONG HighWaterNumberOfHandles; // Uint4B
    OBJECT_TYPE_INITIALIZER TypeInfo; // _OBJECT_TYPE_INITIALIZER
    EX_PUSH_LOCK TypeLock; // _EX_PUSH_LOCK
    ULONG Key; // Uint4B
    LIST_ENTRY CallbackList; // _LIST_ENTRY
} OBJECT_TYPE, *POBJECT_TYPE;

#pragma pack(1)
typedef struct _OB_CALLBACK
```

```

{
    LIST_ENTRY ListEntry;
    ULONGLONG Unknown;
    HANDLE ObHandle;
    PVOID ObTypeAddr;
    PVOID PreCall;
    PVOID PostCall;
}OB_CALLBACK, *POB_CALLBACK;
#pragma pack()

VOID DriverUnload(PDRIVER_OBJECT pDriverObject)
{
}

NTSTATUS DriverEntry(PDRIVER_OBJECT pDriverObject, PUNICODE_STRING pRegPath)
{
    NTSTATUS status = STATUS_SUCCESS;

    DbgPrint("hello lyshark.com \n");

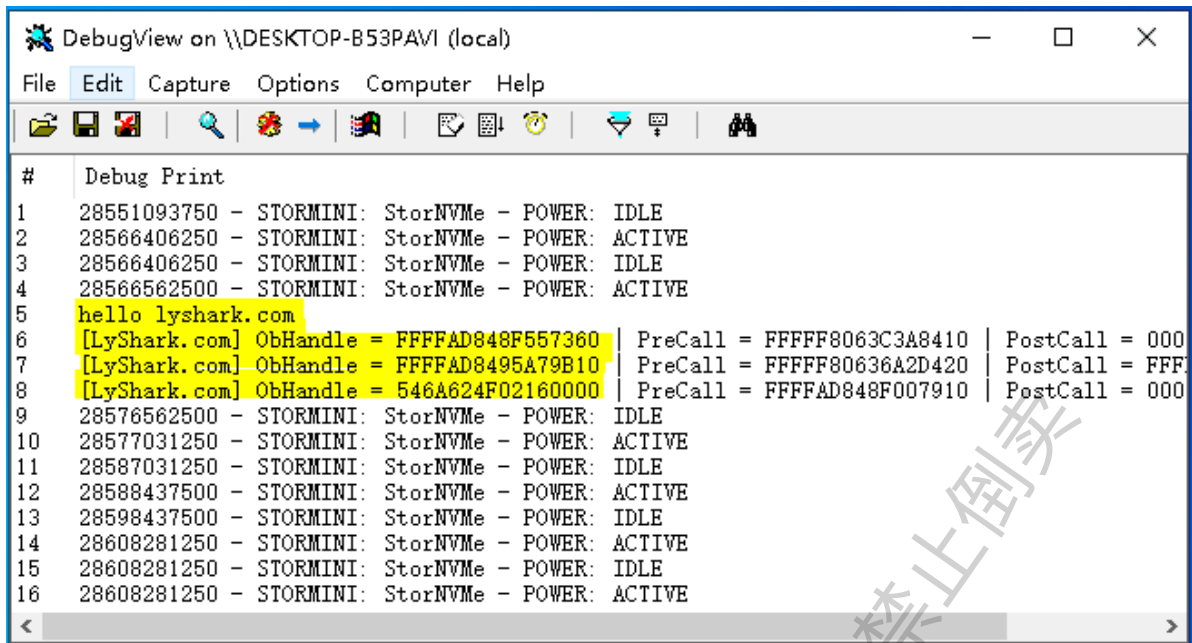
    POB_CALLBACK pObCallback = NULL;

    // 直接获取 CallbackList 链表
    LIST_ENTRY CallbackList = ((POBJECT_TYPE)(*PsProcessType))->CallbackList;

    // 开始遍历
    pObCallback = (POB_CALLBACK)CallbackList.Flink;
    do
    {
        if (FALSE == MmIsValidAddress(pObCallback))
        {
            break;
        }
        if (NULL != pObCallback->ObHandle)
        {
            // 显示
            DbgPrint("[LyShark.com] ObHandle = %p | PreCall = %p | PostCall = %p\n", pObCallback->ObHandle, pObCallback->PreCall, pObCallback->PostCall);
        }
        // 获取下一链表信息
        pObCallback = (POB_CALLBACK)pObCallback->ListEntry.Flink;
    } while (CallbackList.Flink != (PLIST_ENTRY)pObCallback);
    return status;
}

```

运行这段驱动程序，即可得到 `进程句柄` 回调：



```
# Debug Print
1 28551093750 - STORMINI: StorNVMe - POWER: IDLE
2 28566406250 - STORMINI: StorNVMe - POWER: ACTIVE
3 28566406250 - STORMINI: StorNVMe - POWER: IDLE
4 28566562500 - STORMINI: StorNVMe - POWER: ACTIVE
5 hello lyshark.com
6 [LyShark.com] ObHandle = FFFFAD848F557360 | PreCall = FFFFF8063C3A8410 | PostCall = 000
7 [LyShark.com] ObHandle = FFFFAD8495A79B10 | PreCall = FFFFF80636A2D420 | PostCall = FFF
8 [LyShark.com] ObHandle = 546A624F02160000 | PreCall = FFFFAD848F007910 | PostCall = 000
9 28576562500 - STORMINI: StorNVMe - POWER: IDLE
10 28577031250 - STORMINI: StorNVMe - POWER: ACTIVE
11 28587031250 - STORMINI: StorNVMe - POWER: IDLE
12 28588437500 - STORMINI: StorNVMe - POWER: ACTIVE
13 28598437500 - STORMINI: StorNVMe - POWER: IDLE
14 28608281250 - STORMINI: StorNVMe - POWER: ACTIVE
15 28608281250 - STORMINI: StorNVMe - POWER: IDLE
16 28608281250 - STORMINI: StorNVMe - POWER: ACTIVE
```

当然了如是 进程句柄 的枚举，如果是想要输出线程句柄，则只需要替换代码中的 `PsProcessType` 为 `((POBJECT_TYPE)(*PsThreadType))->CallbackList` 即可，修改后的代码如下。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>
#include <wdm.h>
#include <ntddk.h>

typedef struct _OBJECT_TYPE_INITIALIZER
{
    USHORT Length; // Uint2B
    UCHAR ObjectTypeFlags; // Uchar
    ULONG ObjectTypeCode; // Uint4B
    ULONG InvalidAttributes; // Uint4B
    GENERIC_MAPPING GenericMapping; // _GENERIC_MAPPING
    ULONG ValidAccessMask; // Uint4B
    ULONG RetainAccess; // Uint4B
    POOL_TYPE PoolType; // _POOL_TYPE
    ULONG DefaultPagedPoolCharge; // Uint4B
    ULONG DefaultNonPagedPoolCharge; // Uint4B
    PVOID DumpProcedure; // Ptr64 void
    PVOID OpenProcedure; // Ptr64 long
    PVOID CloseProcedure; // Ptr64 void
    PVOID DeleteProcedure; // Ptr64 void
    PVOID ParseProcedure; // Ptr64 long
    PVOID SecurityProcedure; // Ptr64 long
    PVOID QueryNameProcedure; // Ptr64 long
    PVOID OkayToCloseProcedure; // Ptr64 unsigned char
    ULONG waitObjectFlagMask; // Uint4B
    USHORT waitObjectFlagOffset; // Uint2B
    USHORT waitObjectPointerOffset; // Uint2B
}OBJECT_TYPE_INITIALIZER, *POBJECT_TYPE_INITIALIZER;

typedef struct _OBJECT_TYPE
```

```

{
    LIST_ENTRY TypeList;           // _LIST_ENTRY
    UNICODE_STRING Name;          // _UNICODE_STRING
    PVOID DefaultObject;          // Ptr64 Void
    UCHAR Index;                  // UChar
    ULONG TotalNumberOfObjects;    // Uint4B
    ULONG TotalNumberOfHandles;    // Uint4B
    ULONG HighwaterNumberOfObjects; // Uint4B
    ULONG HighwaterNumberOfHandles; // Uint4B
    OBJECT_TYPE_INITIALIZER TypeInfo; // _OBJECT_TYPE_INITIALIZER
    EX_PUSH_LOCK TypeLock;        // _EX_PUSH_LOCK
    ULONG Key;                    // Uint4B
    LIST_ENTRY CallbackList;      // _LIST_ENTRY
}OBJECT_TYPE, *POBJECT_TYPE;

#pragma pack(1)
typedef struct _OB_CALLBACK
{
    LIST_ENTRY ListEntry;
    ULONGLONG Unknown;
    HANDLE ObHandle;
    PVOID ObTypeAddr;
    PVOID PreCall;
    PVOID PostCall;
}OB_CALLBACK, *POB_CALLBACK;
#pragma pack()

VOID DriverUnload(PDRIVER_OBJECT pDriverObject)
{
}

NTSTATUS DriverEntry(PDRIVER_OBJECT pDriverObject, PUNICODE_STRING pRegPath)
{
    NTSTATUS status = STATUS_SUCCESS;

    DbgPrint("hello lyshark.com \n");

    POB_CALLBACK pObCallback = NULL;

    // 直接获取 CallbackList 链表
    LIST_ENTRY CallbackList = ((POBJECT_TYPE)(*PsThreadType))->CallbackList;

    // 开始遍历
    pObCallback = (POB_CALLBACK)CallbackList.Flink;
    do
    {
        if (FALSE == MmIsAddressValid(pObCallback))
        {
            break;
        }
        if (NULL != pObCallback->ObHandle)
        {
            // 显示
            DbgPrint("[LyShark] ObHandle = %p | PreCall = %p | PostCall = %p\n", pObCallback->ObHandle, pObCallback->PreCall, pObCallback->PostCall);
        }
    } while (pObCallback->Flink != &CallbackList);
}

```



```

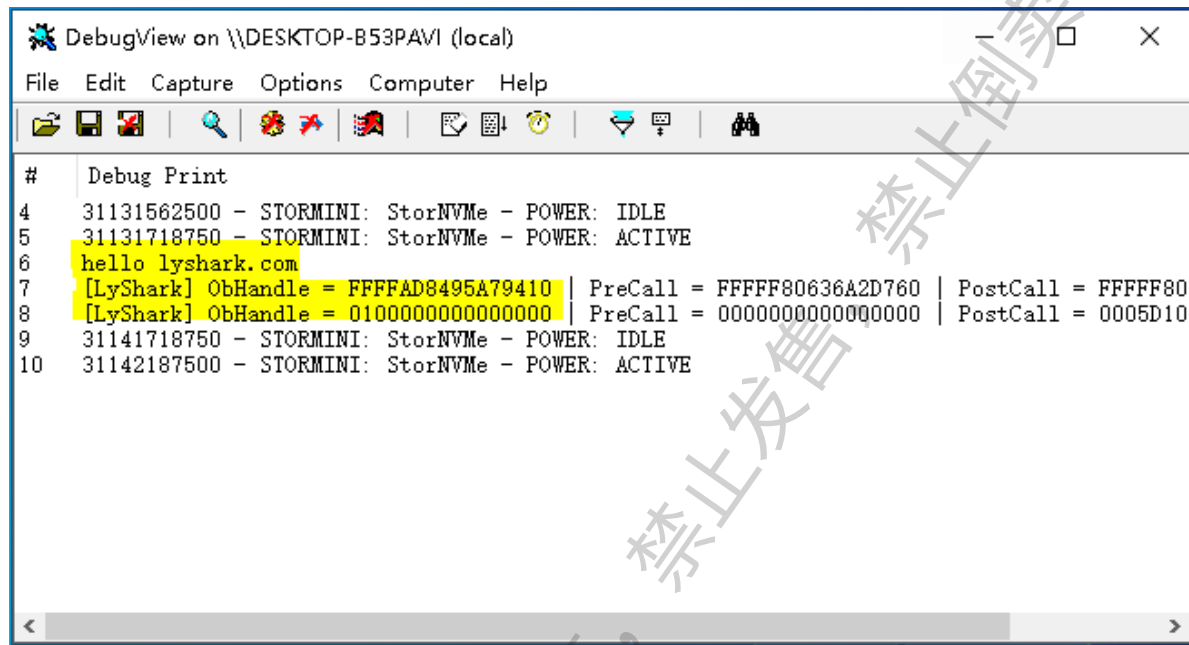
    }
    // 获取下一链表信息
    pobCallback = (POB_CALLBACK)pobCallback->ListEntry.Flink;

} while (CallbackList.Flink != (PLIST_ENTRY)pobCallback);

return status;
}

```

运行这段驱动程序，即可得到 线程句柄 回调：



第九章：内核监控篇

9.1 内核监控进程与线程回调

在前面的文章中 Lyshark 一直在重复的实现了对系统底层模块的枚举，今天我们将展开一个新的话题，内核监控，我们以 监控进程线程 创建为例，在 win10 系统中监控进程与线程可以使用微软提供给我们的两个新函数来实现，此类函数的原理是创建一个回调事件，当有进程或线程被创建或者注销时，系统会通过回调机制将该进程相关信息优先返回给我们自己的函数待处理结束后再转向系统层。

进程回调默认会设置 CreateProcess 通知，而线程回调则会设置 CreateThread 通知，我们来看ARK工具中的枚举效果。

进程	驱动模块	内核层	内核钩子	应用层钩子	设置	监控	启动信息	注册表	服务	文件	网络	调试引擎
系统回调	过滤驱动	DPC定时器	IO定时器	系统线程	卸载的驱动							
回调入口	通知类型	模块路径										
0xFFFFF803471CB830	CreateProcess	C:\Windows\System32\drivers\PyArkSafe.sys										
0xFFFFF803471CB570	CreateProcess	C:\Windows\System32\drivers\PyArkSafe.sys										
0xFFFFF803471810B0	CreateProcess	C:\Users\lyshark\Desktop\WinDDK.sys										
0xFFFFF80347FC3CF0	CreateProcess	C:\Windows\system32\drivers\peauth.sys										
0xFFFFF8034E9EFC00	CreateProcess	C:\Windows\system32\DRIVERS\vm3dmp.sys										
0xFFFFF8034DEF8C60	CreateProcess	C:\Windows\System32\drivers\dxgkrnl.sys										

- 通常情况下：
 - PsSetCreateProcessNotifyRoutineEx 用于监控进程
 - PsSetCreateThreadNotifyRoutine 用于监控线程

监控进程的启动与退出可以使用 `PsSetCreateProcessNotifyRoutineEx` 来创建回调，当新进程创建时会优先执行回调，我们看下微软是如何定义的结构。

```
// 参数1: 新进程回调函数
// 参数2: 是否注销
NTSTATUS PsSetCreateProcessNotifyRoutineEx(
    [in] PCREATE_PROCESS_NOTIFY_ROUTINE_EX NotifyRoutine,
    [in] BOOLEAN Remove
);
```

如上，该函数只有两个参数，第一个参数是回调函数，第二个参数是是否注销，通常在驱动退出时可以传入 `TRUE` 对该回调进行注销，通常情况下如果驱动关闭，则必须要注销回调，而对于 `MyLySharkCreateProcessNotifyEx` 自定义回调来说，则需要指定三个必须要有的参数传递。

```
// 参数1: 新进程的EProcess
// 参数2: 新进程PID
// 参数3: 新进程详细信息（仅在创建进程时有效）

VOID MyLySharkCreateProcessNotifyEx(PEPROCESS Process, HANDLE ProcessId,
PPS_CREATE_NOTIFY_INFO CreateInfo)
```

根据如上函数定义，就可以实现监控功能了，例如我们监控如果进程名是 `lyshark.exe` 则直接 `CreateInfo->CreationStatus = STATUS_UNSUCCESSFUL` 禁止该进程打开。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>

// 两个未公开函数导出
NTKERNELAPI PCHAR PsGetProcessImageFileName(PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);

// 通过PID获得进程名
PCHAR GetProcessNameByProcessId(HANDLE ProcessId)
{
    NTSTATUS st = STATUS_UNSUCCESSFUL;
    PEPROCESS ProcessObj = NULL;
    PCHAR string = NULL;
    st = PsLookupProcessByProcessId(ProcessId, &ProcessObj);
    if (NT_SUCCESS(st))
    {
        string = PsGetProcessImageFileName(ProcessObj);
        ObfDereferenceObject(ProcessObj);
    }
    return string;
}

// 绕过签名检查
BOOLEAN BypassCheckSign(PDRIVER_OBJECT pDriverObject)
{

```

```

#ifdef _WIN64
    typedef struct _KLDR_DATA_TABLE_ENTRY
    {
        LIST_ENTRY listEntry;
        ULONG64 __Undefined1;
        ULONG64 __Undefined2;
        ULONG64 __Undefined3;
        ULONG64 NonPagedDebugInfo;
        ULONG64 DllBase;
        ULONG64 EntryPoint;
        ULONG SizeOfImage;
        UNICODE_STRING path;
        UNICODE_STRING name;
        ULONG Flags;
        USHORT LoadCount;
        USHORT __Undefined5;
        ULONG64 __Undefined6;
        ULONG CheckSum;
        ULONG __padding1;
        ULONG TimeDateStamp;
        ULONG __padding2;
    } KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#else
    typedef struct _KLDR_DATA_TABLE_ENTRY
    {
        LIST_ENTRY listEntry;
        ULONG unknown1;
        ULONG unknown2;
        ULONG unknown3;
        ULONG unknown4;
        ULONG unknown5;
        ULONG unknown6;
        ULONG unknown7;
        UNICODE_STRING path;
        UNICODE_STRING name;
        ULONG Flags;
    } KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#endif

    PKLDR_DATA_TABLE_ENTRY pLdrData = (PKLDR_DATA_TABLE_ENTRY)pDriverObject->
    DriverSection;
    pLdrData->Flags = pLdrData->Flags | 0x20;

    return TRUE;
}

// 进程回调函数
VOID My_LyShark_Com_CreateProcessNotifyEx(PEPROCESS Process, HANDLE ProcessId,
PPS_CREATE_NOTIFY_INFO CreateInfo)
{
    char ProcName[16] = { 0 };
    if (CreateInfo != NULL)
    {
        strcpy_s(ProcName, 16, PsGetProcessImageFileName(Process));
    }
}

```

```

        DbgPrint("[LyShark] 父进程ID: %ld | 父进程名: %s | 进程名: %s | 进程路径: %wZ\n", CreateInfo->ParentProcessId, GetProcessNameByProcessId(CreateInfo->ParentProcessId), PsGetProcessImageFileName(Process), CreateInfo->ImageFileName);

        // 判断是否为指定进程
        if (0 == _stricmp(ProcName, "lyshark.exe"))
        {
            // 禁止打开
            CreateInfo->CreationStatus = STATUS_UNSUCCESSFUL;
        }
    }
else
{
    strcpy_s(ProcName, 16, PsGetProcessImageFileName(Process));
    DbgPrint("[LyShark] 进程[ %s ] 退出了，程序被关闭", ProcName);
}
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DWORD32 ref = 0;

    // 注销进程回调
    ref =
    PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)My_LyShark_Com_CreateProcessNotifyEx, TRUE);
    DbgPrint("[lyshark.com] 注销进程回调: %d\n", ref);
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;

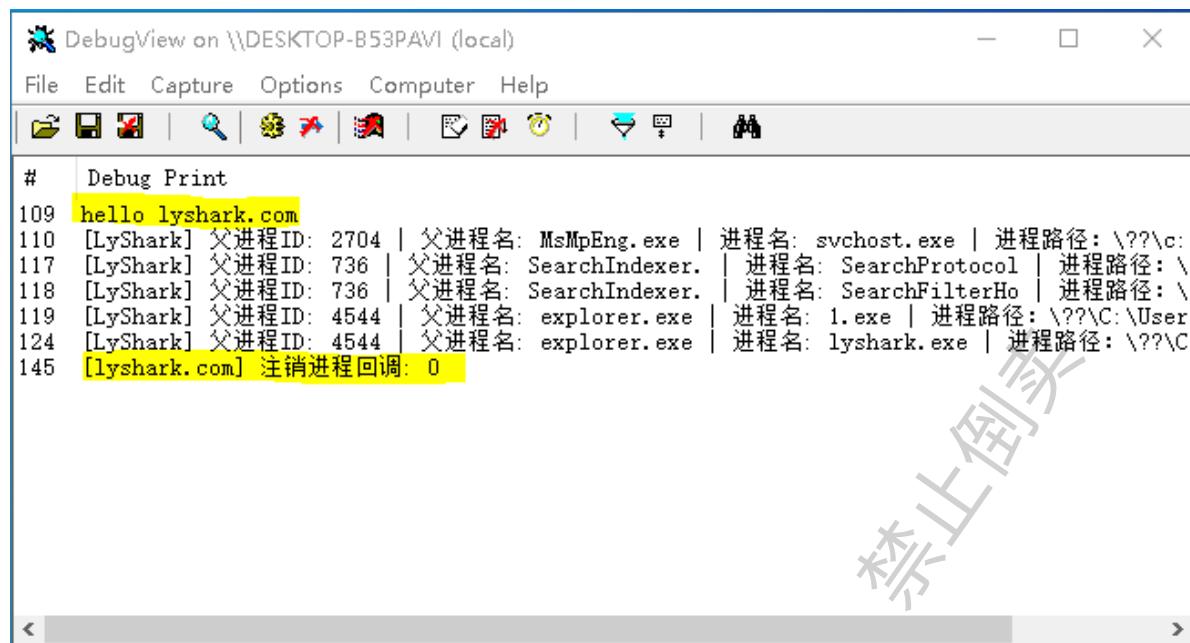
    // 绕过签名检查
    // LINKER_FLAGS=/INTEGRITYCHECK
    BypassCheckSign(Driver);

    DbgPrint("hello Lyshark.com\n");

    // 创建进程回调
    // 参数1: 新进程的EProcess
    // 参数2: 新进程PID
    // 参数3: 新进程详细信息 (仅在创建进程时有效)
    status =
    PsSetCreateProcessNotifyRoutineEx((PCREATE_PROCESS_NOTIFY_ROUTINE_EX)My_LyShark_Com_CreateProcessNotifyEx, FALSE);
    if (!NT_SUCCESS(status))
    {
        DbgPrint("[lyshark.com] 创建进程回调错误");
    }
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

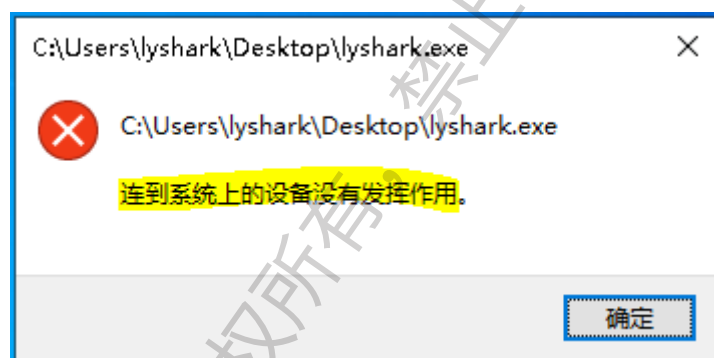
```

编译并运行这个驱动程序，我们可以在 ARK 工具中看到这个驱动所加载的 CreateProcess 的回调事件。



```
# Debug Print
109 hello lyshark.com
110 [LyShark] 父进程ID: 2704 | 父进程名: MsMpEng.exe | 进程名: svchost.exe | 进程路径: \\??\c:
117 [LyShark] 父进程ID: 736 | 父进程名: SearchIndexer. | 进程名: SearchProtocol | 进程路径: \
118 [LyShark] 父进程ID: 736 | 父进程名: SearchIndexer. | 进程名: SearchFilterHo | 进程路径: \
119 [LyShark] 父进程ID: 4544 | 父进程名: explorer.exe | 进程名: 1.exe | 进程路径: \\??\C:\User
124 [LyShark] 父进程ID: 4544 | 父进程名: explorer.exe | 进程名: lyshark.exe | 进程路径: \\??\C
145 [lyshark.com] 注销进程回调: 0
```

当驱动加载后，如果你尝试打开 lyshark.exe 那么会提示连接的设备没有发挥作用，我们则成功拦截了这次打开，当然如果在打开进程之前扫描其特征并根据特征拒绝进程打开，那么就可以实现一个简单的防恶意程序，进程监控在防恶意程序中也是用的最多的。



说完了 PsSetCreateProcessNotifyRoutineEx 回调的使用方式，LyShark将继续带大家看看 线程监控 如何实现，监控线程创建与监控进程差不多，检测线程需要调用 PsSetCreateThreadNotifyRoutine 创建回调函数，之后就可监控系统所有线程的创建，具体实现代码如下。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntifs.h>

// 两个未公开函数导出
NTKERNELAPI PCHAR PsGetProcessImageFileName(PEPROCESS Process);
NTKERNELAPI NTSTATUS PsLookupProcessByProcessId(HANDLE ProcessId, PEPROCESS
*Process);
NTKERNELAPI NTSTATUS PsLookupThreadByThreadId(HANDLE ThreadId, PETHREAD
*Thread);

// 绕过签名检查
BOOLEAN BypassCheckSign(PDRIVER_OBJECT pDriverObject)
{
#ifdef _WIN64
```

```

typedef struct _KLDR_DATA_TABLE_ENTRY
{
    LIST_ENTRY listEntry;
    ULONG64 __Undefined1;
    ULONG64 __Undefined2;
    ULONG64 __Undefined3;
    ULONG64 NonPagedDebugInfo;
    ULONG64 DllBase;
    ULONG64 EntryPoint;
    ULONG SizeOfImage;
    UNICODE_STRING path;
    UNICODE_STRING name;
    ULONG Flags;
    USHORT LoadCount;
    USHORT __Undefined5;
    ULONG64 __Undefined6;
    ULONG CheckSum;
    ULONG __padding1;
    ULONG TimeDateStamp;
    ULONG __padding2;
} KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#else
typedef struct _KLDR_DATA_TABLE_ENTRY
{
    LIST_ENTRY listEntry;
    ULONG unknown1;
    ULONG unknown2;
    ULONG unknown3;
    ULONG unknown4;
    ULONG unknown5;
    ULONG unknown6;
    ULONG unknown7;
    UNICODE_STRING path;
    UNICODE_STRING name;
    ULONG Flags;
} KLDR_DATA_TABLE_ENTRY, *PKLDR_DATA_TABLE_ENTRY;
#endif

PKLDR_DATA_TABLE_ENTRY pLdrData = (PKLDR_DATA_TABLE_ENTRY)pDriverObject->DriverSection;
pLdrData->Flags = pLdrData->Flags | 0x20;

return TRUE;
}

// 线程回调函数
VOID MyCreateThreadNotify(HANDLE ProcessId, HANDLE ThreadId, BOOLEAN CreateInfo)
{
    PEPROCESS eprocess = NULL;
    PETHREAD ethread = NULL;
    UCHAR *pwin32Address = NULL;

    // 通过此函数拿到程序的EPROCESS结构
    PsLookupProcessByProcessId(ProcessId, &eprocess);
    PsLookupThreadByThreadId(ThreadId, &ethread);
}

```

```

    if (CreateInfo)
    {
        DbgPrint("[lyshark.com] 线程TID: %ld | 所属进程名: %s | 进程PID: %ld \n",
ThreadId, PsGetProcessImageFileName(eprocess), PsGetProcessId(eprocess));
        /*
        if (0 == _stricmp(PsGetProcessImageFileName(eprocess), "lyshark.exe"))
        {
            DbgPrint("线程TID: %ld | 所属进程名: %s | 进程PID: %ld \n", ThreadId,
PsGetProcessImageFileName(eprocess), PsGetProcessId(eprocess));

            // dt _kthread
            // 寻找里面的 win32StartAddress 并写入ret
            pwin32Address = *(UCHAR**)((UCHAR*)ethread + 0x1c8);
            if (MmIsAddressValid(pwin32Address))
            {
                *pwin32Address = 0xc3;
            }
        }
        */
    }
    else
    {
        DbgPrint("[LyShark] %s 线程已退出...", ThreadId);
    }

    if (eprocess)
        ObDereferenceObject(eprocess);
    if (ethread)
        ObDereferenceObject(ethread);
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    NTSTATUS status;

    // 注销进程回调
    status = PsRemoveCreateThreadNotifyRoutine(MyCreateThreadNotify);
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status;

    DbgPrint("hello lyshark.com \n");

    // 绕过签名检查
    // LINKER_FLAGS=/INTEGRITYCHECK
    BypassCheckSign(Driver);

    // 创建线程回调
    // 参数1: 新线程ProcessID
    // 参数2: 新线程ThreadId
    // 参数3: 线程创建/退出标志
    status = PsSetCreateThreadNotifyRoutine(MyCreateThreadNotify);
}

```

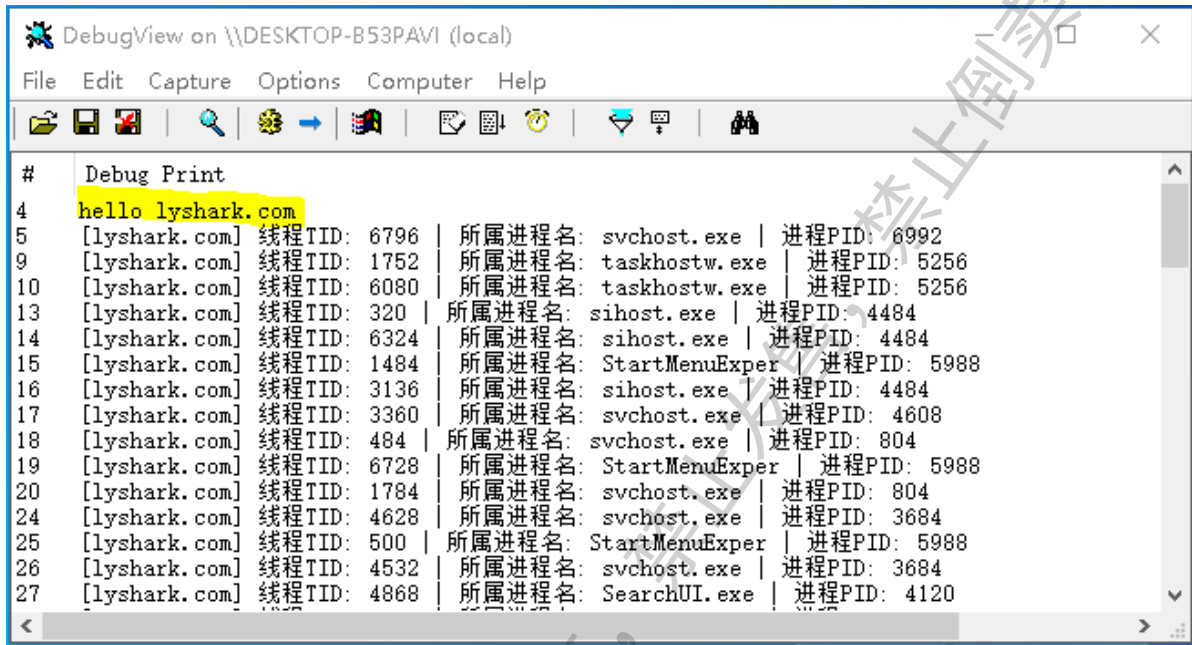
```

if (!NT_SUCCESS(status))
{
    DbgPrint("创建线程回调错误");
}

Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

运行后则可监控到系统总所有线程的创建与退出，效果如下所示：



9.2 内核注册并监控对象回调

在笔者上一篇文章《驱动开发：内核枚举进程与线程ObCall回调》简单介绍了如何枚举系统中已经存在的进程与线程回调，本章 Lyshark 将通过对象回调实现对进程线程的句柄监控，在内核中提供了 ObRegisterCallbacks 回调，使用这个内核回调函数，可注册一个对象回调，不过目前该函数只能监控进程与线程句柄操作，通过监控进程或线程句柄，可实现保护指定进程线程不被终止的目的。

由于目前对象回调只能监控进程与线程，而这个监控是通过 ObjectType 这么一个成员控制的，如果成员是 PsProcessType 则代表监控进程，反之 PsThreadType 则是监控线程，无论监控进程还是线程都调用 ObRegisterCallbacks 这个函数来完成注册。

函数 ObRegisterCallbacks 其微软对他的定义是这样的，用户传入 OB_OPERATION_REGISTRATION 结构，以及 OB_CALLBACK_REGISTRATION 回调结构，其中 PreOperation 则是传入的回调函数，也是最重要的，其次是 ObjectType 指定成进程回调。

```

NTSTATUS ObRegisterCallbacks(
    [in] POB_CALLBACK_REGISTRATION CallbackRegistration,
    [out] PVOID *RegistrationHandle
);

```

首先来实现一个检测的案例，注册一个进程回调对象 MyLySharkComObjectCallback，通过 ObRegisterCallbacks 注册的回调只需要传入一个填充好的 OB_CALLBACK_REGISTRATION 回调结构体，以及一个全局句柄即可，这个全局句柄的作用仅仅只是在程序结束时，调用 ObUnRegisterCallbacks 卸载监控而已，实现代码如下所示。


```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <ntstrsafe.h>

PVOID Globle_Object_Handle;

// 绕过签名检测
void BypassCheckSign(PDRIVER_OBJECT pDriverObj)
{
    typedef struct _LDR_DATA
    {
        struct _LIST_ENTRY InLoadOrderLinks;
        struct _LIST_ENTRY InMemoryOrderLinks;
        struct _LIST_ENTRY InInitializationOrderLinks;
        VOID* DllBase;
        VOID* EntryPoint;
        ULONG32 SizeOfImage;
        UINT8 _PADDING0_[0x4];
        struct _UNICODE_STRING FullDllName;
        struct _UNICODE_STRING BaseDllName;
        ULONG32 Flags;
    } LDR_DATA, *PLDR_DATA;

    PLDR_DATA ldr;
    ldr = (PLDR_DATA)(pDriverObj->DriverSection);
    ldr->Flags |= 0x20;
}

// 自定义回调
OB_PREOP_CALLBACK_STATUS MyLySharkComObjectCallBack(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION OperationInformation)
{
    DbgPrint("[lyshark] 执行回调函数... \n");
    return STATUS_SUCCESS;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    ObUnRegisterCallbacks(Globle_Object_Handle);
    DbgPrint("回调卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    BypassCheckSign(Driver);

    OB_OPERATION_REGISTRATION Base;
    OB_CALLBACK_REGISTRATION CallbackReg;
}

```

// 回调函数结构体

// 回调函数

```

CallbackReg.RegistrationContext = NULL; // 注册上下文(你回调函数返回参数)
CallbackReg.Version = OB_FLT_REGISTRATION_VERSION; // 注册回调版本
CallbackReg.OperationRegistration = &Base; // 回调结构体
CallbackReg.OperationRegistrationCount = 1; // 操作计数(下钩数量)

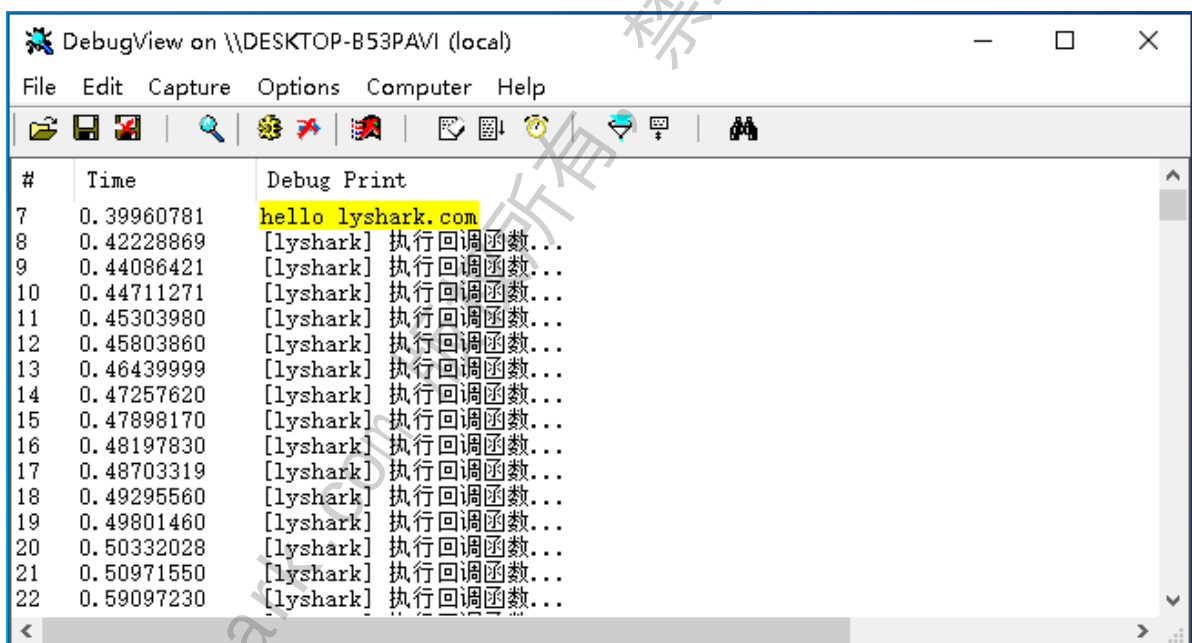
RtlUnicodeStringInit(&CallbackReg.Altitude, L"600000"); // 长度
Base.ObjectType = PsProcessType; // 进程操作类型.此处为进程操作

Base.Operations = OB_OPERATION_HANDLE_CREATE; // 操作句柄创建
Base.PreOperation = MyLySharkComObjectCallback; // 你自己的回调函数
Base.PostOperation = NULL;

// 注册回调
if (ObRegisterCallbacks(&CallbackReg, &Globe_Object_Handle))
{
    DbgPrint("[lyshark message] 回调注册成功...");
}
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

当驱动程序被加载以后，一旦有进程运行则会执行我们自己的 MyLySharkComObjectCallback 回调，而在回调函数内则可以执行任意功能，运行如下所示。



如上所示只是演示基本的回调申请流程，回调函数通常需要包含两个值，其一 RegistrationContext 用于标注上下文，其二 POB_PRE_OPERATION_INFORMATION 则用于标注进程或者线程创建的信息结构体。

```

OB_PREOP_CALLBACK_STATUS MyLySharkComObjectCallback(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION OperationInformation)

```

那么如何实现 拦截进程启动 这个功能呢，我们可以在回调函数中写入以下代码进行拦截。

- CreateHandleInformation.DesiredAccess 将打开句柄的权限清零
- CreateHandleInformation.OriginalDesiredAccess 判断是否终止

```

if (pOperationInformation->Operation == OB_OPERATION_HANDLE_CREATE)
{
    DbgPrint("lyshark.exe 进程打开 \n");
    pOperationInformation->Parameters->CreateHandleInformation.DesiredAccess=0;
    if ((pOperationInformation->Parameters->CreateHandleInformation.OriginalDesiredAccess & PROCESS_TERMINATE) == PROCESS_TERMINATE)
    {
        pOperationInformation->Parameters->CreateHandleInformation.DesiredAccess
        &= ~PROCESS_TERMINATE;
    }
}

```

拦截进程创建核心代码如下所示。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <ntstrsafe.h>

#define PROCESS_TERMINATE 0x1

// 导出两个API
NTKERNELAPI PEPROCESS IoThreadToProcess(PETHREAD Thread);
NTKERNELAPI char* PsGetProcessImageFileName(PEPROCESS Process);

// 全局句柄
PVOID Globle_Object_Handle = NULL;

// 绕过签名检测
void BypassCheckSign(PDRIVER_OBJECT pDriverObj)
{
    typedef struct _LDR_DATA
    {
        struct _LIST_ENTRY InLoadOrderLinks;
        struct _LIST_ENTRY InMemoryOrderLinks;
        struct _LIST_ENTRY InInitializationOrderLinks;
        VOID* DllBase;
        VOID* EntryPoint;
        ULONG32 SizeOfImage;
        UINT8 _PADDING0_[0x4];
        struct _UNICODE_STRING FullDllName;
        struct _UNICODE_STRING BaseDllName;
        ULONG32 Flags;
    } LDR_DATA, *PLDR_DATA;

    PLDR_DATA ldr;
    ldr = (PLDR_DATA)(pDriverObj->DriverSection);
    ldr->Flags |= 0x20;
}

// 判断是否是是需要保护的进程

```

```

BOOLEAN CheckProcess(PEPROCESS eprocess)
{
    char *Name = PsGetProcessImageFileName(eprocess);
    if (!_stricmp("lyshark.exe", Name))
        return TRUE;
    else
        return FALSE;
}

// 进程回调
OB_PREOP_CALLBACK_STATUS MyLySharkProcessObjectCallback(PVOID
RegistrationContext, POB_PRE_OPERATION_INFORMATION pOperationInformation)
{
    HANDLE pid;

    // 只取出进程回调
    if (pOperationInformation->ObjectType != *PsProcessType)
    {
        return OB_PREOP_SUCCESS;
    }

    // 得到所有进程的ID
    pid = PsGetProcessId((PEPROCESS)pOperationInformation->Object);
    // DbgPrint("进程PID= %ld \n", pid);

    UNREFERENCED_PARAMETER(RegistrationContext);

    // 验证是否是需要的进程
    if (CheckProcess((PEPROCESS)pOperationInformation->Object))
    {
        // 创建句柄
        if (pOperationInformation->Operation == OB_OPERATION_HANDLE_CREATE)
        {
            DbgPrint("lyshark.exe 进程打开事件 \n");
            pOperationInformation->Parameters->CreateHandleInformation.DesiredAccess=0;
            if ((pOperationInformation->Parameters->CreateHandleInformation.OriginalDesiredAccess & PROCESS_TERMINATE) == PROCESS_TERMINATE)
            {
                DbgPrint("[LyShark Message] 拦截进程打开 \n");
                pOperationInformation->Parameters->CreateHandleInformation.DesiredAccess &= ~PROCESS_TERMINATE;
            }
        }
        // 复制句柄
        if (pOperationInformation->Operation == OB_OPERATION_HANDLE_DUPLICATE)
        {
            DbgPrint("lyshark.exe 进程被关闭 \n");
            pOperationInformation->Parameters->DuplicateHandleInformation.DesiredAccess=0;
            if ((pOperationInformation->Parameters->DuplicateHandleInformation.OriginalDesiredAccess & PROCESS_TERMINATE) == PROCESS_TERMINATE)
            {

```

```

        pOperationInformation->Parameters-
>DuplicateHandleInformation.DesiredAccess &= ~PROCESS_TERMINATE;
    }
}
}
return OB_PREOP_SUCCESS;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    ObUnRegisterCallbacks(Globle_Object_Handle);
    DbgPrint("回调卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    BypassCheckSign(Driver);

    OB_OPERATION_REGISTRATION ob_process_callback;
    OB_CALLBACK_REGISTRATION op_process_operation;

    memset(&ob_process_callback, 0, sizeof(ob_process_callback));
    ob_process_callback.ObjectType = PsProcessType;
    ob_process_callback.Operations = OB_OPERATION_HANDLE_CREATE |
OB_OPERATION_HANDLE_DUPLICATE;
    ob_process_callback.PreOperation = MyLySharkProcessObjectCallback;
    ob_process_callback.PostOperation = NULL;

    RtlUnicodeStringInit(&op_process_operation.Altitude, L"600000");
    op_process_operation.RegistrationContext = NULL;
    op_process_operation.Version = OB FLT_REGISTRATION_VERSION;
    op_process_operation.OperationRegistration = &ob_process_callback;
    op_process_operation.OperationRegistrationCount = 1;

    // 注册进程回调
    if (ObRegisterCallbacks(&op_process_operation, &Globle_Object_Handle))
    {
        DbgPrint("进程回调注册成功...");
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

加载这个驱动，当有进程被创建时，则首先判断是否是 lyshark.exe 如果是则直接禁止打开，也就是终止掉。

DebugView on \\DESKTOP-B53PAVI (local)		
File Edit Capture Options Computer Help		
#	Time	Debug Print
50	0.16326611	hello lyshark.com
53	1.25948226	lyshark.exe 进程打开事件
54	1.26688552	lyshark.exe 进程打开事件
55	1.27287555	lyshark.exe 进程打开事件
56	1.27910721	lyshark.exe 进程打开事件
57	1.28388846	lyshark.exe 进程打开事件
58	1.28981233	[LyShark Message] 拦截进程打开
59	1.29596007	lyshark.exe 进程打开事件
60	1.30176306	lyshark.exe 进程打开事件
61	1.30786431	[LyShark Message] 拦截进程打开
62	1.31399238	lyshark.exe 进程打开事件
63	1.31966400	[LyShark Message] 拦截进程打开
65	2.27936673	lyshark.exe 进程打开事件
66	2.79969954	lyshark.exe 进程打开事件
70	3.10070729	lyshark.exe 进程打开事件
71	3.10858822	lyshark.exe 进程打开事件

同理进程可以被拦截，那么如果增加更多的过滤条件，则线程同样可以被拦截，拦截线程代码如下所示。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <ntstrsafe.h>

#define THREAD_TERMINATE2 0x1

// 导出两个API
NTKERNELAPI PEPROCESS IoThreadToProcess(PETHREAD Thread);
NTKERNELAPI char* PsGetProcessImageFileName(PEPROCESS Process);

// 全局句柄
PVOID Gbgle_Object_Handle = NULL;

// 绕过签名检测
void BypassCheckSign(PDRIVER_OBJECT pDriverObj)
{
    typedef struct _LDR_DATA
    {
        struct _LIST_ENTRY InLoadOrderLinks;
        struct _LIST_ENTRY InMemoryOrderLinks;
        struct _LIST_ENTRY InInitializationOrderLinks;
        VOID* DllBase;
        VOID* EntryPoint;
        ULONG32 SizeOfImage;
        UINT8 _PADDING0_[0x4];
        struct _UNICODE_STRING FullDllName;
        struct _UNICODE_STRING BaseDllName;
        ULONG32 Flags;
    } LDR_DATA, *PLDR_DATA;

    PLDR_DATA ldr;
```

```

    ldr = (PLDR_DATA)(pDriverObj->DriverSection);
    ldr->Flags |= 0x20;
}

// 判断是否是受保护的进程
BOOLEAN CheckProcess(PEPROCESS eprocess)
{
    char *Name = PsGetProcessImageFileName(eprocess);
    if (!_stricmp("lyshark.exe", Name))
        return TRUE;
    else
        return FALSE;
}

// 线程回调
OB_PREOP_CALLBACK_STATUS MyThreadObjectCallback(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION poperationInformation)
{
    PEPROCESS ep;
    PETHREAD et;
    HANDLE pid;

    // 线程过滤
    if (poperationInformation->ObjectType != *PsThreadType)
    {
        return OB_PREOP_SUCCESS;
    }

    et = (PETHREAD)poperationInformation->Object;
    ep = IoThreadToProcess(et);
    pid = PsGetProcessId(ep);

    // DbgPrint("线程PID= %ld | TID= %ld \n", pid, PsGetThreadId(et));
    UNREFERENCED_PARAMETER(RegistrationContext);

    if (CheckProcess(ep))
    {
        if (poperationInformation->Operation == OB_OPERATION_HANDLE_CREATE)
        {
            poperationInformation->Parameters->CreateHandleInformation.DesiredAccess=0;
            if ((poperationInformation->Parameters->CreateHandleInformation.OriginalDesiredAccess & THREAD_TERMINATE2) ==
THREAD_TERMINATE2)
            {
                DbgPrint("[LyShark] 拦截lyshark.exe进程内 %d 线程创建 \n",
PsGetThreadId(et));
                poperationInformation->Parameters->CreateHandleInformation.DesiredAccess &= ~THREAD_TERMINATE2;
            }
        }
        if (poperationInformation->Operation == OB_OPERATION_HANDLE_DUPLICATE)
        {
            poperationInformation->Parameters->DuplicateHandleInformation.DesiredAccess=0;

```

```

        if ((pOperationInformation->Parameters-
>DuplicateHandleInformation.OriginalDesiredAccess & THREAD_TERMINATE2) ==
THREAD_TERMINATE2)
        {
            pOperationInformation->Parameters-
>DuplicateHandleInformation.DesiredAccess &= ~THREAD_TERMINATE2;
        }
    }
    return OB_PREOP_SUCCESS;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    ObUnRegisterCallbacks(Globle_Object_Handle);
    DbgPrint("回调卸载完成... \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    BypassCheckSign(Driver);

    OB_OPERATION_REGISTRATION ob_thread_callback;
    OB_CALLBACK_REGISTRATION op_thread_operation;

    memset(&ob_thread_callback, 0, sizeof(ob_thread_callback));
    ob_thread_callback.ObjectType = PsThreadType;
    ob_thread_callback.Operations = OB_OPERATION_HANDLE_CREATE |
OB_OPERATION_HANDLE_DUPLICATE;
    ob_thread_callback.PreOperation = MyThreadObjectCallBack;
    ob_thread_callback.PostOperation = NULL;

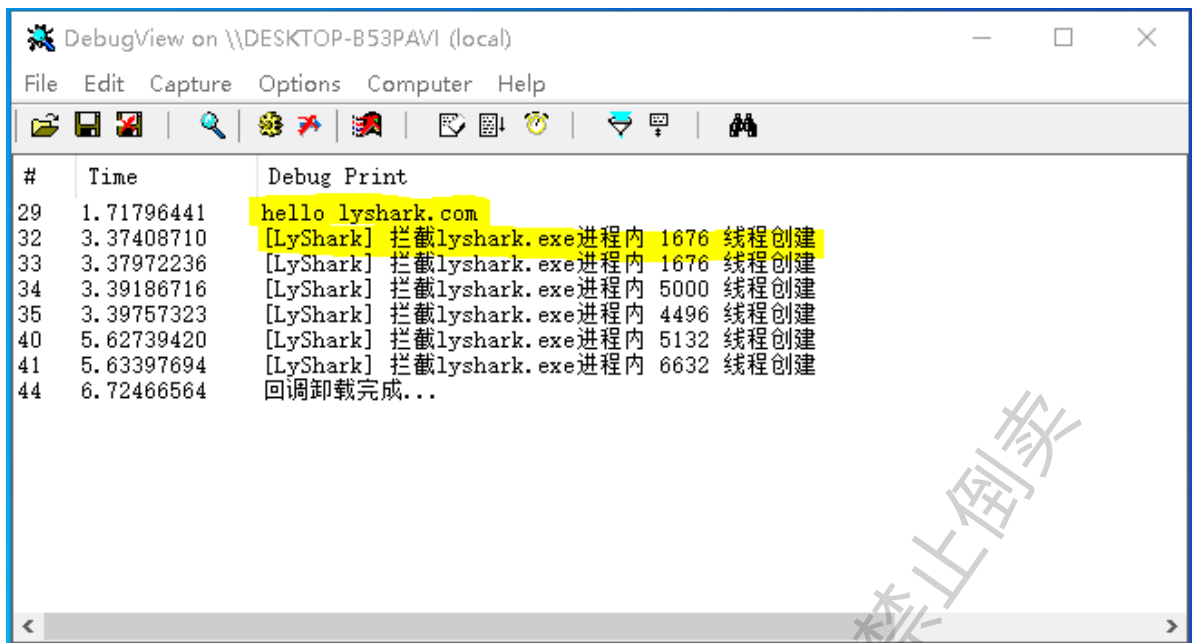
    RtlUnicodeStringInit(&op_thread_operation.Altitude, L"600001");
    op_thread_operation.RegistrationContext = NULL;
    op_thread_operation.Version = OB_FLT_REGISTRATION_VERSION;
    op_thread_operation.OperationRegistration = &ob_thread_callback;
    op_thread_operation.OperationRegistrationCount = 1;

    // 注册进程回调
    if (ObRegisterCallbacks(&op_thread_operation, &Globle_Object_Handle))
    {
        DbgPrint("进程回调注册成功...");
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

这段驱动加载后，如果有新线程被创建，则会被拦截并打印输出，效果图如下。



9.3 内核监视LoadImage映像回调

在笔者上一篇文章《驱动开发：内核注册并监控对象回调》介绍了如何运用 ObRegisterCallbacks 注册进程与线程回调，并通过该回调实现了拦截指定进行运行的效果，本章 Lyshark 将带大家继续探索一个新的回调注册函数，PsSetLoadImageNotifyRoutine 常用于注册 LoadImage 映像监视，当有模块被系统加载时则可以第一时间获取到加载模块信息，需要注意的是该回调函数内无法进行拦截，如需要拦截则需写入返回指令这部分内容将在下一章进行讲解，本章将主要实现对模块的监视功能。

监视模块加载与卸载需要分别使用两个函数，这两个函数的参数传递都是自己的回调地址。

- PsSetLoadImageNotifyRoutine 设置回调
- PsRemoveLoadImageNotifyRoutine 移除回调

此处 MyLySharkLoadImageNotifyRoutine 回调地址必须有三个参数传递组成，其中 FullImageName 代表完整路径，ModuleStyle 代表模块类型，一般来说 ModuleStyle=0 表示加载 SYS 驱动，如果 ModuleStyle=1 则表示加载的是 DLL，最后一个参数 ImageInfo 则是映像的详细参数结构体。

```
VOID MyLySharkLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE  
ModuleStyle, PIMAGE_INFO ImageInfo)
```

那么如何实现监视映像加载呢，来看如下完整代码片段，首先 PsSetLoadImageNotifyRoutine 注册回调，当有模块被加载则自动执行 MyLySharkLoadImageNotifyRoutine 回调函数，其内部首先判断 ModuleStyle 得出是什么类型的模块，然后再通过 GetDriverEntryByImageBase 拿到当前进程详细参数并打印输出。

```
// 署名权  
// right to sign one's name on a piece of work  
// PowerBy: LyShark  
// Email: me@lyshark.com  
#include <ntddk.h>  
#include <ntimage.h>  
  
// 未导出函数声明  
PCHAR PsGetProcessImageFileName(PEPROCESS pEProcess);  
  
// 获取到镜像装载基地址
```

```

PVOID GetDriverEntryByImageBase(PVOID ImageBase)
{
    PIMAGE_DOS_HEADER pDOSHeader;
    PIMAGE_NT_HEADERS64 pNTHHeader;
    PVOID pEntryPoint;
    pDOSHeader = (PIMAGE_DOS_HEADER)ImageBase;
    pNTHHeader = (PIMAGE_NT_HEADERS64)((ULONG64)ImageBase + pDOSHeader->e_lfanew);
    pEntryPoint = (PVOID)((ULONG64)ImageBase + pNTHHeader->OptionalHeader.AddressOfEntryPoint);
    return pEntryPoint;
}

// 获取当前进程名
UCHAR* GetCurrentProcessName()
{
    PEPROCESS pEProcess = PsGetCurrentProcess();
    if (NULL != pEProcess)
    {
        UCHAR *lpszProcessName = PsGetProcessImageFileName(pEProcess);
        if (NULL != lpszProcessName)
        {
            return lpszProcessName;
        }
    }
    return NULL;
}

// 设置自己的回调函数
VOID MyLySharkLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ModuleStyle, PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;

    // MmIsAddress 验证地址可用性
    if (FullImageName != NULL && MmIsAddressValid(FullImageName))
    {
        // ModuleStyle为零表示加载sys
        if (ModuleStyle == 0)
        {
            // 得到装载主进程名
            UCHAR *load_name = GetCurrentProcessName();
            pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
            DbgPrint("[LyShark SYS加载] 模块名称:%wZ --> 装载基址:%p --> 镜像长度: %d --> 装载主进程: %s \n", FullImageName, pDrvEntry, ImageInfo->ImageSize, load_name);
        }
        // ModuleStyle非零表示加载DLL
        else
        {
            // 得到装载主进程名
            UCHAR *load_name = GetCurrentProcessName();
            pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
        }
    }
}

```

```

        DbgPrint("[LyShark DLL加载] 模块名称:%wZ --> 装载基址:%p --> 镜像长度: %d
--> 装载主进程: %s \n", FullImageName, pDrvEntry, ImageInfo->ImageSize,
load_name);
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLySharkLoadImageNot
ifyRoutine);
    DbgPrint("[LyShark.com] 驱动卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    PsSetLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLySharkLoadImageNotify
Routine);
    DbgPrint("[LyShark.com] 驱动加载完成...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行这个驱动程序，则会输出被加载的驱动详细参数。

```

# Debug Print
28 hello lyshark.com
29 [LyShark.com] 驱动加载完成...
30 [LyShark SYS加载] 模块名称: \Device\HarddiskVolume4\ProgramData\Microsoft\Windows Defende
31 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\setupapi.dll --> 装载
32 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\devobj.dll --> 装载基
39 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\svchost.exe --> 装载
40 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\ntdll.dll --> 装载基
50 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\clbcatq.dll --> 装载
51 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\taskschd.dll --> 装载
52 [LyShark DLL加载] 模块名称: \Device\HarddiskVolume4\Windows\System32\sspicli.dll --> 装载
58 [LyShark.com] 驱动卸载完成...

```

9.4 内核运用LoadImage屏蔽驱动

在笔者上一篇文章《驱动开发：内核监视LoadImage映像回调》中 LyShark 简单介绍了如何通过 PsSetLoadImageNotifyRoutine 函数注册回调来监视驱动模块的加载，注意我这里用的是监视而不是监控之所以是监视而不是监控那是因为 PsSetLoadImageNotifyRoutine 无法实现参数控制，而如果我们想要控制特定驱动的加载则需要自己做一些事情来实现，如下 LyShark 将解密如何实现屏蔽特定驱动的加载。

要想实现 驱动屏蔽 其原理很简单，通过 ImageInfo->ImageBase 得到镜像基地址，然后调用 GetDriverEntryByImageBase 函数来得到程序的入口地址，找NT头的 OptionalHeader 节点，该节点里面就是被加载驱动入口，通过汇编在驱动头部写入 ret 返回指令，即可实现屏蔽加载特定驱动文件。

原理其实很容易理解，如果我们需要实现则只需要在《驱动开发：内核监视LoadImage映像回调》这篇文章的代码上稍加改进即可，当检测到 lyshark.sys 驱动加载时，直接跳转到入口处快速写入一个 Ret 让驱动返回即可，至于如何写出指令的问题如果不懂建议回头看看《驱动开发：内核CR3切换读写内存》文章中是如何读写内存的，这段代码实现如下所示。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <intrin.h>
#include <ntimage.h>

PVOID GetDriverEntryByImageBase(PVOID ImageBase)
{
    PIMAGE_DOS_HEADER pDOSHeader;
    PIMAGE_NT_HEADERS64 pNTHHeader;
    PVOID pEntryPoint;
    pDOSHeader = (PIMAGE_DOS_HEADER)ImageBase;
    pNTHHeader = (PIMAGE_NT_HEADERS64)((ULONG64)ImageBase + pDOSHeader->e_lfanew);
    pEntryPoint = (PVOID)((ULONG64)ImageBase + pNTHHeader->OptionalHeader.AddressOfEntryPoint);
    return pEntryPoint;
}

VOID UnicodeToChar(PUNICODE_STRING dst, char *src)
{
    ANSI_STRING string;
    RtlUnicodeStringToAnsiString(&string, dst, TRUE);
    strcpy(src, string.Buffer);
    RtlFreeAnsiString(&string);
}

// 使用开关写保护需要在[C/C++] -> [优化] -> 启用内部函数
// 关闭写保护
KIRQL WPOFFx64()
{
    KIRQL irq1 = KeRaiseIrqlToDpcLevel();
    UINT64 cr0 = __readcr0();
    cr0 &= 0xfffffffffffff;
    _disable();
    __writecr0(cr0);
    return irq1;
}

// 开启写保护
void WPONx64(KIRQL irq1)
{
    UINT64 cr0 = __readcr0();
    cr0 |= 0x10000;
```

```

_enable();
__writecr0(cr0);
KeLowerIrql(irql);
}

BOOLEAN DenyLoadDriver(PVOID DriverEntry)
{
    UCHAR fuck[] = "\xB8\x22\x00\x00\xC0\xC3";
    KIRQL kirql;
    /* 在模块开头写入以下汇编指令
    Mov eax,c0000022h
    ret
    */
    if (DriverEntry == NULL) return FALSE;
    kirql = WPOFFx64();
    memcpy(DriverEntry, fuck, sizeof(fuck) / sizeof(fuck[0]));
    WPONx64(kirql);
    return TRUE;
}

VOID MyLySharkComLoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE
ModuleStyle, PIMAGE_INFO ImageInfo)
{
    PVOID pDrvEntry;
    char szFullImageName[256] = { 0 };

    // MmIsAddress 验证地址可用性
    if (FullImageName != NULL && MmIsAddressValid(FullImageName))
    {
        // ModuleStyle为零表示加载sys
        if (ModuleStyle == 0)
        {
            pDrvEntry = GetDriverEntryByImageBase(ImageInfo->ImageBase);
            UnicodeToChar(FullImageName, szFullImageName);
            if (strstr(_strlwr(szFullImageName), "lyshark.sys"))
            {
                DbgPrint("[LyShark] 拦截SYS内核模块: %s", szFullImageName);
                DenyLoadDriver(pDrvEntry);
            }
        }
    }
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLySharkComLoadImage
NotifyRoutine);
    DbgPrint("驱动卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");
}

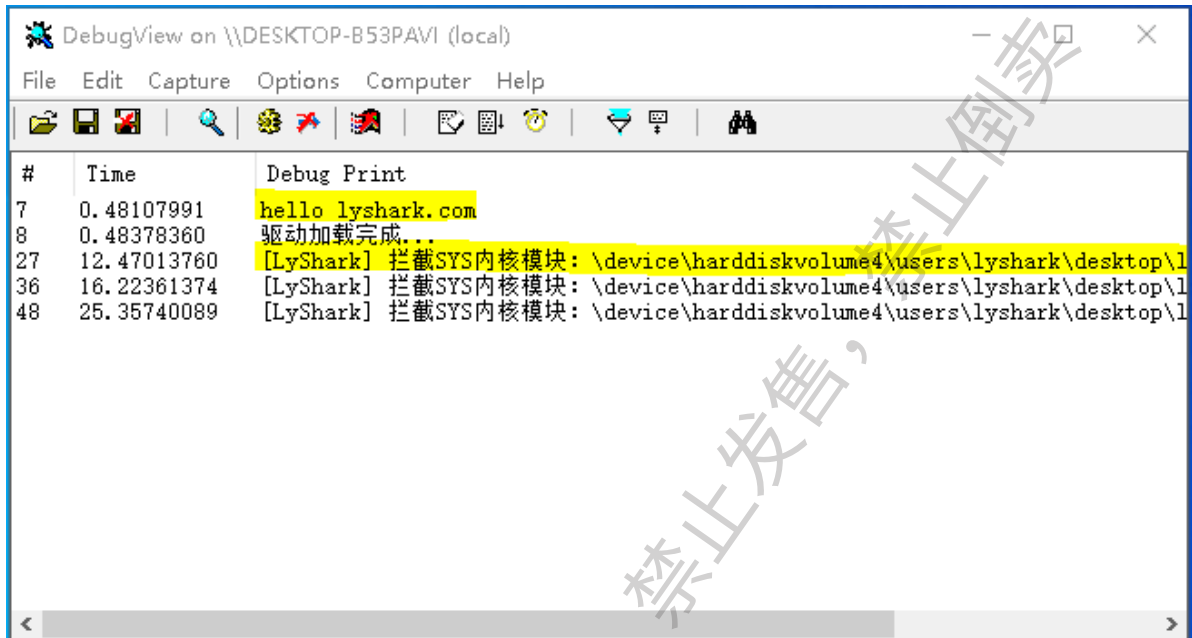
```

```

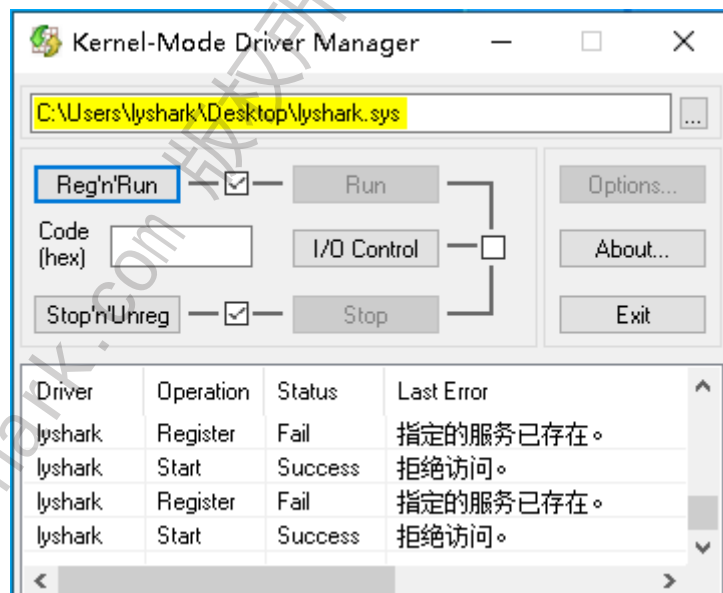
PsSetLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)MyLySharkComLoadImageNotifyRoutine);
    DbgPrint("驱动加载完成...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

首先运行我们的驱动，然后我们接着加载 lyshark.sys 则你会发现驱动被拦截了。



我们看下驱动加载器，提示的信息是拒绝访问，因为这个驱动其实是加载了的，只是入口处被填充了返回而已。



除了使用 Ret 强制返回的方法意外，屏蔽驱动加载还可以使用另一种方式实现禁用模块加载，例如当驱动被加载首先回调函数内可以接收到，当接收到以后直接调用 MmUnmapViewOfSection 函数强制卸载掉即可，如果使用这种方法实现则这段代码需要改进成如下样子。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

```

```

#include <ntifs.h>
#include <ntimage.h>
#include <intrin.h>

NTSTATUS MmUnmapViewOfSection(PEPROCESS Process, PVOID BaseAddress);
NTSTATUS SetNotifyRoutine();
NTSTATUS RemoveNotifyRoutine();

VOID LoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ProcessId,
PIMAGE_INFO ImageInfo);
NTSTATUS U2C(PUNICODE_STRING pustrSrc, PCHAR pszDest, ULONG ulDestLength);
VOID ThreadProc(_In_ PVOID StartContext);

// 拒绝加载驱动
NTSTATUS DenyLoadDriver(PVOID pImageBase);

// 拒绝加载DLL模块
NTSTATUS DenyLoadDll(HANDLE ProcessId, PVOID pImageBase);

typedef struct _MY_DATA
{
    HANDLE ProcessId;
    PVOID pImageBase;
}MY_DATA, *PMY_DATA;

// 设置消息回调
NTSTATUS SetNotifyRoutine()
{
    NTSTATUS status = STATUS_SUCCESS;
    status = PsSetLoadImageNotifyRoutine(LoadImageNotifyRoutine);
    return status;
}

// 关闭消息回调
NTSTATUS RemoveNotifyRoutine()
{
    NTSTATUS status = STATUS_SUCCESS;
    status = PsRemoveLoadImageNotifyRoutine(LoadImageNotifyRoutine);
    return status;
}

VOID LoadImageNotifyRoutine(PUNICODE_STRING FullImageName, HANDLE ProcessId,
PIMAGE_INFO ImageInfo)
{
    DbgPrint("PID: %d --> 完整路径: %wZ --> 大小: %d --> 基址: 0x%p \n",
ProcessId, FullImageName, ImageInfo->ImageSize, ImageInfo->ImageBase);

    HANDLE hThread = NULL;
    CHAR szTemp[1024] = { 0 };
    U2C(FullImageName, szTemp, 1024);
    if (NULL != strstr(szTemp, "lyshark.sys"))
    {
        // EXE或者DLL
        if (0 != ProcessId)
        {

```

```

        // 创建多线程 延时1秒钟后再卸载模块
        PMY_DATA pMyData = ExAllocatePool(NonPagedPool, sizeof(MY_DATA));
        pMyData->ProcessId = ProcessId;
        pMyData->pImageBase = ImageInfo->ImageBase;
        PsCreateSystemThread(&hThread, 0, NULL, NtCurrentProcess(), NULL,
ThreadProc, pMyData);
        DbgPrint("[LyShark] 禁止加载DLL文件 \n");
    }
    // 驱动
    else
    {
        DenyLoadDriver(ImageInfo->ImageBase);
        DbgPrint("[LyShark] 禁止加载SYS驱动文件 \n");
    }
}

// 拒绝加载驱动
NTSTATUS DenyLoadDriver(PVOID pImageBase)
{
    NTSTATUS status = STATUS_SUCCESS;
    PMDL pMdl = NULL;
    PVOID pVoid = NULL;
    ULONG ulShellcodeLength = 16;
    UCHAR pShellcode[16] = { 0xB8, 0x22, 0x00, 0x00, 0xC0, 0xC3, 0x90, 0x90,
0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90, 0x90 };
    PIMAGE_DOS_HEADER pDosHeader = pImageBase;
    PIMAGE_NT_HEADERS pNtHeaders = (PIMAGE_NT_HEADERS)((PUCHAR)pDosHeader +
pDosHeader->e_lfanew);
    PVOID pDriverEntry = (PVOID)((PUCHAR)pDosHeader + pNtHeaders->
OptionalHeader.AddressOfEntryPoint);

    pMdl = MmCreateMdl(NULL, pDriverEntry, ulShellcodeLength);
    MmBuildMdlForNonPagedPool(pMdl);
    pVoid = MmMapLockedPages(pMdl, KernelMode);
    RtlCopyMemory(pVoid, pShellcode, ulShellcodeLength);
    MmUnmapLockedPages(pVoid, pMdl);
    IoFreeMdl(pMdl);

    return status;
}

// 调用 MmUnmapViewOfSection 函数来卸载已经加载的 DLL 模块
NTSTATUS DenyLoadDll(HANDLE ProcessId, PVOID pImageBase)
{
    NTSTATUS status = STATUS_SUCCESS;
    PEPROCESS pEProcess = NULL;

    status = PsLookupProcessByProcessId(ProcessId, &pEProcess);
    if (!NT_SUCCESS(status))
    {
        return status;
    }

    // 卸载模块

```



```

        status = MmUnmapViewOfSection(pEProcess, pImageBase);
        if (!NT_SUCCESS(status))
        {
            return status;
        }
        return status;
    }

VOID ThreadProc(_In_ PVOID StartContext)
{
    PMY_DATA pMyData = (PMY_DATA)StartContext;
    LARGE_INTEGER liTime = { 0 };

    // 延时 1 秒 负值表示相对时间
    liTime.QuadPart = -10 * 1000 * 1000;
    KeDelayExecutionThread(KernelMode, FALSE, &liTime);

    // 卸载
    DenyLoadDll(pMyData->ProcessId, pMyData->pImageBase);

    ExFreePool(pMyData);
}

NTSTATUS U2C(PUNICODE_STRING pustrSrc, PCHAR pszDest, ULONG ulDestLength)
{
    NTSTATUS status = STATUS_SUCCESS;
    ANSI_STRING strTemp;

    RtlZeroMemory(pszDest, ulDestLength);
    RtlUnicodeStringToAnsiString(&strTemp, pustrSrc, TRUE);
    if (ulDestLength > strTemp.Length)
    {
        RtlCopyMemory(pszDest, strTemp.Buffer, strTemp.Length);
    }
    RtlFreeAnsiString(&strTemp);

    return status;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    PsRemoveLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)RemoveNotifyRoutine);
    DbgPrint("驱动卸载完成...");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.ocm \n");

    PsSetLoadImageNotifyRoutine((PLOAD_IMAGE_NOTIFY_ROUTINE)SetNotifyRoutine);
    DbgPrint("驱动加载完成...");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

加载这段驱动程序，当有DLL文件被加载后，则会强制弹出，从而实现屏蔽模块加载的作用。

当然用 LoadImage 回调做监控并不靠谱，因为它很容易被绕过，其实系统里存在一个开关，叫做 PspNotifyEnableMask 如果它的值被设置为 0，那么所有的相关操作都不会经过回调，所有回调都会失效。

```
Kernel 'com:port=\\.\pipe\com_1,baud=115200,pipe' - WinDbg:10.0.16299.15 AMD64
File Edit View Debug Window Help
[Icons]
Command
1: kd> uf PspNotifyEnableMask
Flow analysis was incomplete, some code may be missing
nt!PspNotifyEnableMask:
fffff807`819af5f0 0f0000          sldt     word ptr [rax]
fffff807`819af5f3 0000          add     byte ptr [rax],al
fffff807`819af5f5 0000          add     byte ptr [rax],al
fffff807`819af5f7 0000          add     byte ptr [rax],al
fffff807`819af5f9 0023          add     byte ptr [rbx],ah
fffff807`819af5fb 7c07          jl      nt!CmpMasterHive+0x4 (fffff807`819af604) Branch
nt!PiLoggedErrorEventsMask+0x9:
fffff807`819af5fd f8          cld
nt!CmpMasterHive+0x4:
fffff807`819af604 86d7          xchg    dl,bh
```

9.5 内核监控Register注册表回调

在笔者前一篇文章《驱动开发：内核枚举Registry注册表回调》中实现了对注册表的枚举，本章将实现对注册表的监控，不同于32位系统在64位系统中，微软为我们提供了两个针对注册表的专用内核监控函数，通过这两个函数可以在不劫持内核API的前提下实现对注册表增加，删除，创建等事件的有效监控，注册表监视通常会通过 CmRegisterCallback 创建监控事件并传入自己的回调函数，与该创建对应的是 CmUnRegisterCallback 当注册表监控结束后可用于注销回调。

- CmRegisterCallback 设置注册表回调
- CmUnRegisterCallback 注销注册表回调

默认情况下 CmRegisterCallback 需传入三个参数，参数一回调函数地址，参数二空余，参数三回调句柄，微软定义如下。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

// 参数1: 回调函数地址
// 参数2: 无作用
// 参数3: 回调句柄
NTSTATUS CmRegisterCallback(
    [in]          PEX_CALLBACK_FUNCTION Function,
    [in, optional] PVOID Context,
    [out]         PLARGE_INTEGER Cookie
);
```

自定义注册表回调函数 MyLySharkCallback 需要保留三个参数， CallbackContext 回调上下文， Argument1 是操作类型， Argument2 定义详细结构体指针。

```
NTSTATUS MyLySharkCallback(_In_ PVOID CallbackContext, _In_opt_ PVOID Argument1,
    _In_opt_ PVOID Argument2)
```

在自定义回调函数内 Argument1 则可获取到操作类型，类型是一个 REG_NOTIFY_CLASS 枚举结构，微软对其的具体定义如下所示。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

typedef enum _REG_NOTIFY_CLASS {
    RegNtDeleteKey,
    RegNtPreDeleteKey = RegNtDeleteKey,
    RegNtSetValueKey,
    RegNtPreSetValueKey = RegNtSetValueKey,
    RegNtDeleteValueKey,
    RegNtPreDeleteValueKey = RegNtDeleteValueKey,
    RegNtSetInformationKey,
    RegNtPreSetInformationKey = RegNtSetInformationKey,
    RegNtRenameKey,
    RegNtPreRenameKey = RegNtRenameKey,
    RegNtEnumerateKey,
    RegNtPreEnumerateKey = RegNtEnumerateKey,
    RegNtEnumerateValueKey,
    RegNtPreEnumerateValueKey = RegNtEnumerateValueKey,
    RegNtQueryKey,
    RegNtPreQueryKey = RegNtQueryKey,
    RegNtQueryValueKey,
    RegNtPreQueryValueKey = RegNtQueryValueKey,
    RegNtQueryMultipleValueKey,
    RegNtPreQueryMultipleValueKey = RegNtQueryMultipleValueKey,
    RegNtPreCreateKey,
    RegNtPostCreateKey,
    RegNtPreOpenKey,
    RegNtPostOpenKey,
    RegNtKeyHandleClose,
    RegNtPreKeyHandleClose = RegNtKeyHandleClose,
    //
    // .Net only
    //
    RegNtPostDeleteKey,
    RegNtPostSetValueKey,
    RegNtPostDeleteValueKey,
    RegNtPostSetInformationKey,
    RegNtPostRenameKey,
    RegNtPostEnumerateKey,
    RegNtPostEnumerateValueKey,
    RegNtPostQueryKey,
    RegNtPostQueryValueKey,
    RegNtPostQueryMultipleValueKey,
    RegNtPostKeyHandleClose,
    RegNtPreCreateKeyEx,
    RegNtPostCreateKeyEx,
    RegNtPreOpenKeyEx,
    RegNtPostOpenKeyEx,
    //
    // new to windows Vista
```

```

//
RegNtPreFlushKey,
RegNtPostFlushKey,
RegNtPreLoadKey,
RegNtPostLoadKey,
RegNtPreUnLoadKey,
RegNtPostUnLoadKey,
RegNtPreQueryKeySecurity,
RegNtPostQueryKeySecurity,
RegNtPreSetKeySecurity,
RegNtPostSetKeySecurity,
//
// per-object context cleanup
//
RegNtCallbackObjectContextCleanup,
//
// new in Vista SP2
//
RegNtPreRestoreKey,
RegNtPostRestoreKey,
RegNtPreSaveKey,
RegNtPostSaveKey,
RegNtPreReplaceKey,
RegNtPostReplaceKey,

MaxRegNtNotifyClass //should always be the last enum
} REG_NOTIFY_CLASS;

```

其中对于注册表最常用的监控项为以下几种类型，当然为了实现监控则我们必须使用之前，如果使用之后则只能起到监视而无法做到监控的目的。

- RegNtPreCreateKey 创建注册表之前
- RegNtPreOpenKey 打开注册表之前
- RegNtPreDeleteKey 删除注册表之前
- RegNtPreDeleteValueKey 删除键值之前
- RegNtPreSetValueKey 修改注册表之前

如果需要使用监视，首先 `CmRegisterCallback` 注册一个自定义回调，当有消息时则触发

`MyLySharkCallback` 其内部获取到 `IooperateType` 操作类型，并通过 `switch` 选择不同的处理例程，每个处理例程都通过 `GetFullPath` 得到注册表完整路径，并打印出来，这段代码实现如下。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include <ntifs.h>
#include <windef.h>

// 未导出函数声明 pEProcess -> PID
PUCHAR PsGetProcessImageFileName(PEPROCESS pEProcess);

NTSTATUS ObQueryNameString(
    _In_ PVOID Object,
    _Out_writes_bytes_opt_(Length) POBJECT_NAME_INFORMATION ObjectNameInfo,

```

```

    _In_ ULONG Length,
    _Out_ PULONG ReturnLength
);

// 注册表回调Cookie
LARGE_INTEGER g_lRegCookie;

// 获取注册表完整路径
BOOLEAN GetFullPath(PUNICODE_STRING pRegistryPath, PVOID pRegistryObject)
{
    // 判断数据地址是否有效
    if ((FALSE == MmIsAddressValid(pRegistryObject)) ||
        (NULL == pRegistryObject))
    {
        return FALSE;
    }
    // 申请内存
    ULONG ulSize = 512;
    PVOID lpObjectNameInfo = ExAllocatePool(NonPagedPool, ulSize);
    if (NULL == lpObjectNameInfo)
    {
        return FALSE;
    }
    // 获取注册表路径
    ULONG ulRetLen = 0;
    NTSTATUS status = ObQueryNameString(pRegistryObject,
    (POBJECT_NAME_INFORMATION)lpObjectNameInfo, ulSize, &ulRetLen);
    if (!NT_SUCCESS(status))
    {
        ExFreePool(lpObjectNameInfo);
        return FALSE;
    }
    // 复制
    RtlCopyUnicodeString(pRegistryPath, (PUNICODE_STRING)lpObjectNameInfo);
    // 释放内存
    ExFreePool(lpObjectNameInfo);
    return TRUE;
}

// 注册表回调函数
NTSTATUS MyLysharkCallback(_In_ PVOID CallbackContext, _In_opt_ PVOID Argument1,
_In_opt_ PVOID Argument2)
{
    NTSTATUS status = STATUS_SUCCESS;
    UNICODE_STRING ustrRegPath;

    // 获取操作类型
    LONG loperateType = (REG_NOTIFY_CLASS)Argument1;

    // 申请内存
    ustrRegPath.Length = 0;
    ustrRegPath.MaximumLength = 1024 * sizeof(WCHAR);
    ustrRegPath.Buffer = ExAllocatePool(NonPagedPool,
    ustrRegPath.MaximumLength);
    if (NULL == ustrRegPath.Buffer)

```

```

{
    return status;
}

RtlZeroMemory(ustrRegPath.Buffer, ustrRegPath.MaximumLength);

// 判断操作
switch (lOperateType)
{
    // 创建注册表之前
    case RegNtPreCreateKey:
    {
        // 获取注册表路径
        GetFullPath(&ustrRegPath, ((PREG_CREATE_KEY_INFORMATION)Argument2)->RootObject);
        DbgPrint("[创建注册表] [%wZ] [%wZ]\n", &ustrRegPath,
            ((PREG_CREATE_KEY_INFORMATION)Argument2)->CompleteName);
        break;
    }
    // 打开注册表之前
    case RegNtPreOpenKey:
    {
        // 获取注册表路径
        GetFullPath(&ustrRegPath, ((PREG_CREATE_KEY_INFORMATION)Argument2)->RootObject);
        DbgPrint("[打开注册表] [%wZ] [%wZ]\n", &ustrRegPath,
            ((PREG_CREATE_KEY_INFORMATION)Argument2)->CompleteName);
        break;
    }
    // 删除键之前
    case RegNtPreDeleteKey:
    {
        // 获取注册表路径
        GetFullPath(&ustrRegPath, ((PREG_DELETE_KEY_INFORMATION)Argument2)->Object);
        DbgPrint("[删除键] [%wZ] \n", &ustrRegPath);
        break;
    }
    // 删除键值之前
    case RegNtPreDeleteValueKey:
    {
        // 获取注册表路径
        GetFullPath(&ustrRegPath,
            ((PREG_DELETE_VALUE_KEY_INFORMATION)Argument2)->Object);
        DbgPrint("[删除键值] [%wZ] [%wZ] \n", &ustrRegPath,
            ((PREG_DELETE_VALUE_KEY_INFORMATION)Argument2)->ValueName);

        // 获取当前进程，即操作注册表的进程
        PEPROCESS pEProcess = PsGetCurrentProcess();
        if (NULL != pEProcess)
        {
            UCHAR *lpszProcessName = PsGetProcessImageFileName(pEProcess);
            if (NULL != lpszProcessName)
            {
                DbgPrint("进程 [%s] 删除了键值对 \n", lpszProcessName);
            }
        }
    }
}

```

```

    }
    break;
}
// 修改键值之前
case RegNtPreSetValueKey:
{
    // 获取注册表路径
    GetFullPath(&ustrRegPath, ((PREG_SET_VALUE_KEY_INFORMATION)Argument2)-
>Object);
    DbgPrint("[修改键值][%wZ][%wZ] \n", &ustrRegPath,
((PREG_SET_VALUE_KEY_INFORMATION)Argument2)->ValueName);
    break;
}
default:
    break;
}

// 释放内存
if (NULL != ustrRegPath.Buffer)
{
    ExFreePool(ustrRegPath.Buffer);
    ustrRegPath.Buffer = NULL;
}

return status;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint(("Uninstall Driver Is OK\n"));

    // 注销当前注册表回调
    if (0 < g_liRegCookie.QuadPart)
    {
        CmUnRegisterCallback(g_liRegCookie);
    }
}

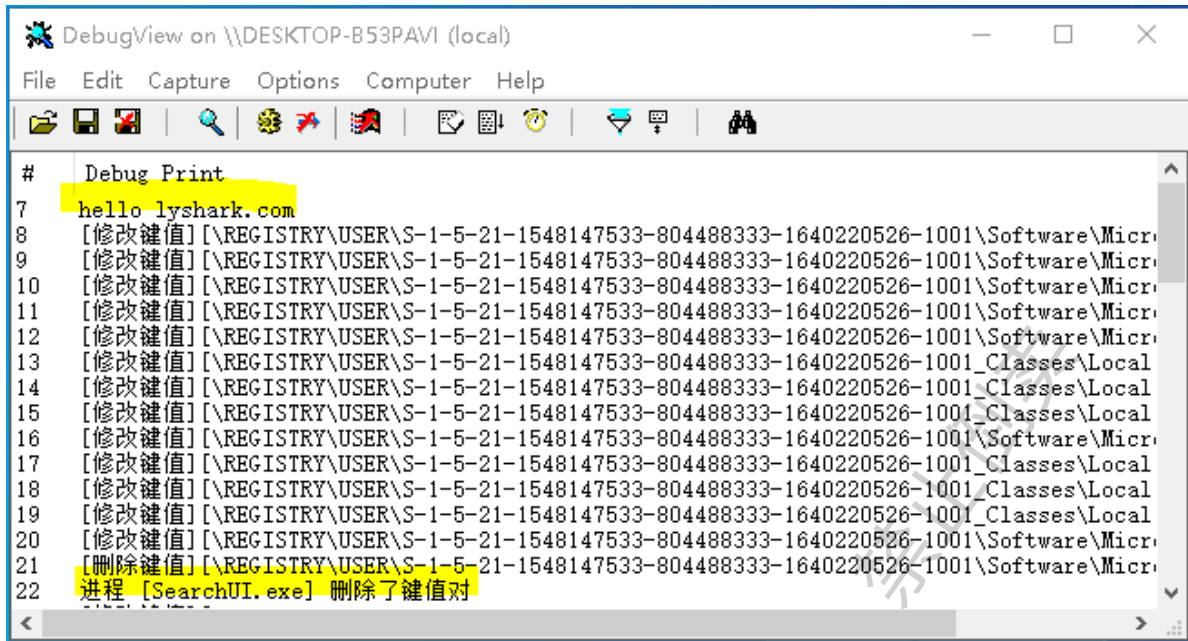
NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint(("hello lyshark.com\n"));

    // 设置注册表回调
    NTSTATUS status = CmRegisterCallback(MyLySharkCallback, NULL,
&g_liRegCookie);
    if (!NT_SUCCESS(status))
    {
        g_liRegCookie.QuadPart = 0;
        return status;
    }

    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行驱动程序，则会输出当前系统中所有针对注册表的操作，如下图所示。



```
# Debug Print
7 hello lyshark.com
8 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
9 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
10 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
11 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
12 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
13 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
14 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
15 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
16 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
17 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
18 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
19 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Classes\Local
20 [修改键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
21 [删除键值] [\REGISTRY\USER\S-1-5-21-1548147533-804488333-1640220526-1001\Software\Micro
22 进程 [SearchUI.exe] 删除了键值对
```

如上的代码只能实现注册表项的监视，而如果需要监控则需要在回调函数 `MyLySharkCallback` 判断，如果指定注册表项是需要保护的则直接返回 `status = STATUS_ACCESS_DENIED`；从而达到保护注册表的目的，核心代码如下所示。

```
// 反注册表删除回调
NTSTATUS MyLySharkCallback(_In_ PVOID CallbackContext, _In_opt_ PVOID Argument1,
_In_opt_ PVOID Argument2)
{
    NTSTATUS status = STATUS_SUCCESS;
    UNICODE_STRING ustrRegPath;

    // 获取操作类型
    LONG loperateType = (REG_NOTIFY_CLASS)Argument1;
    ustrRegPath.Length = 0;
    ustrRegPath.MaximumLength = 1024 * sizeof(WCHAR);
    ustrRegPath.Buffer = ExAllocatePool(NonPagedPool,
    ustrRegPath.MaximumLength);
    if (NULL == ustrRegPath.Buffer)
    {
        return status;
    }

    RtlZeroMemory(ustrRegPath.Buffer, ustrRegPath.MaximumLength);
    // 判断操作
    switch (loperateType)
    {
        // 删除键值之前
        case RegNtPreDeleteValueKey:
        {
            // 获取注册表路径
            GetFullPath(&ustrRegPath,
            ((PREG_DELETE_VALUE_KEY_INFORMATION)Argument2)->Object);
            DbgPrint("[删除键值] [%wZ] [%wZ] \n", &ustrRegPath,
            ((PREG_DELETE_VALUE_KEY_INFORMATION)Argument2)->ValueName);
        }
    }
}
```



```

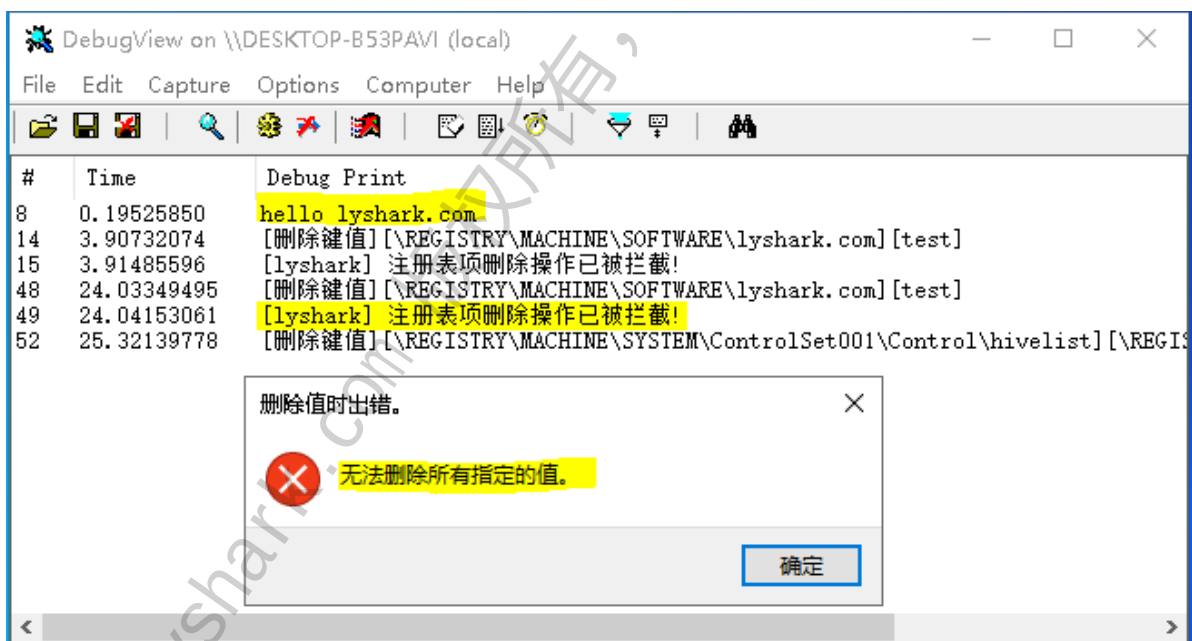
// 如果要删除指定注册表项则拒绝
Pwch pszRegister = L"\\REGISTRY\\MACHINE\\SOFTWARE\\lyshark.com";
if (wcscmp(ustrRegPath.Buffer, pszRegister) == 0)
{
    DbgPrint("[lyshark] 注册表项删除操作已被拦截! \n");
    // 拒绝操作
    status = STATUS_ACCESS_DENIED;
}
break;
}
default:
    break;
}

// 释放内存
if (NULL != ustrRegPath.Buffer)
{
    ExFreePool(ustrRegPath.Buffer);
    ustrRegPath.Buffer = NULL;
}

return status;
}

```

运行驱动程序，然后我们尝试删除 \\LyShark\\HKEY_LOCAL_MACHINE\\SOFTWARE\\lyshark.com 里面的子项，则会提示如下信息。



当然这里的 `RegNtPreDeleteValueKey` 是指的删除操作，如果将其替换成 `RegNtPreSetValueKey`，那么只有当注册表被创建才会拦截，此时就会变成拦截创建。

```

// 拦截创建操作
NTSTATUS MyLysharkCallback(_In_ PVOID CallbackContext, _In_opt_ PVOID Argument1,
_In_opt_ PVOID Argument2)
{
    NTSTATUS status = STATUS_SUCCESS;
    UNICODE_STRING ustrRegPath;

```

```

// 获取操作类型
LONG lOperateType = (REG_NOTIFY_CLASS)Argument1;

// 申请内存
ustrRegPath.Length = 0;
ustrRegPath.MaximumLength = 1024 * sizeof(WCHAR);
ustrRegPath.Buffer = ExAllocatePool(NonPagedPool,
ustrRegPath.MaximumLength);
if (NULL == ustrRegPath.Buffer)
{
    return status;
}
RtlZeroMemory(ustrRegPath.Buffer, ustrRegPath.MaximumLength);

// 判断操作
switch (lOperateType)
{
// 修改键值之前
case RegNtPreSetValueKey:
{
    // 获取注册表路径
    GetFullPath(&ustrRegPath,
((PREG_DELETE_VALUE_KEY_INFORMATION)Argument2)->Object);

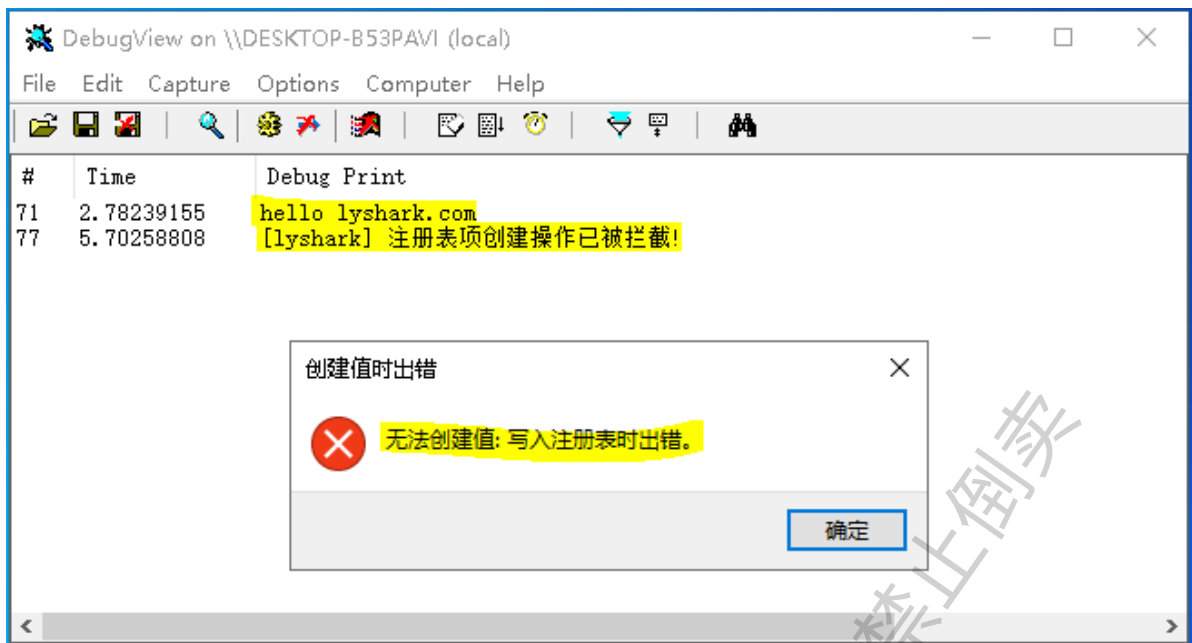
    // 拦截创建
    PWCH pszRegister = L"\\REGISTRY\\MACHINE\\SOFTWARE\\lyshark.com";
    if (wcscmp(ustrRegPath.Buffer, pszRegister) == 0)
    {
        DbgPrint("[lyshark] 注册表项创建操作已被拦截! \n");
        status = STATUS_ACCESS_DENIED;
    }
    break;
}
default:
    break;
}

// 释放内存
if (NULL != ustrRegPath.Buffer)
{
    ExFreePool(ustrRegPath.Buffer);
    ustrRegPath.Buffer = NULL;
}

return status;
}

```

加载驱动保护，然后我们尝试在 \\LyShark\\HKEY_LOCAL_MACHINE\\SOFTWARE\\lyshark.com 里面创建一个子项，则会提示创建失败。



9.6 内核监控FileObject文件回调

本篇文章与上一篇文章《驱动开发：内核注册并监控对象回调》所使用的方式是一样的都是使用 `ObRegisterCallbacks` 注册回调事件，只不过上一篇博文中 LyShark 将回调结构体 `OB_OPERATION_REGISTRATION` 中的 `ObjectType` 填充为了 `PsProcessType` 和 `PsThreadType` 格式从而实现监控进程与线程，本章我们需要将该结构填充为 `IoFileObjectType` 以此来实现对文件的监控，文件过滤驱动不仅仅可以用来监控文件的打开，还可以用它实现对文件的保护，一旦驱动加载则文件是不可被删除和改动的。

与进程线程回调有少许的不同，文件回调需要开启驱动的 `TypeInfo.SupportsObjectCallbacks` 开关，并定义一些微软结构，如下是我们所需要的公开结构体，可在微软官方或WinDBG中获取到最新的，将其保存为 `lyshark.h` 方便后期引用。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include <ntddk.h>
#include <ntstrsafe.h>

typedef struct _CALLBACK_ENTRY
{
    LIST_ENTRY CallbackList;
    OB_OPERATION Operations;
    ULONG Active;
    PVOID Handle;
    POBJECT_TYPE ObjectType;
    POB_PRE_OPERATION_CALLBACK PreOperation;
    POB_POST_OPERATION_CALLBACK PostOperation;
    ULONG unknown;
} CALLBACK_ENTRY, *PCALLBACK_ENTRY;

typedef struct _LDR_DATA // 24 elements, 0xE0 bytes
(sizeof)
{
```

```

/*0x000*/ struct _LIST_ENTRY InLoadOrderLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x010*/ struct _LIST_ENTRY InMemoryOrderLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x020*/ struct _LIST_ENTRY InInitializationOrderLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x030*/ VOID* DllBase;
/*0x038*/ VOID* EntryPoint;
/*0x040*/ ULONG32 SizeOfImage;
/*0x044*/ UINT8 _PADDING0_[0x4];
/*0x048*/ struct _UNICODE_STRING FullDllName; // 3
elements, 0x10 bytes (sizeof)
/*0x058*/ struct _UNICODE_STRING BasedDllName; // 3
elements, 0x10 bytes (sizeof)
/*0x068*/ ULONG32 Flags;
/*0x06c*/ UINT16 LoadCount;
/*0x06e*/ UINT16 TlsIndex;
union // 2 elements, 0x10
bytes (sizeof)
{
/*0x070*/ struct _LIST_ENTRY HashLinks;
// 2 elements, 0x10 bytes (sizeof)
struct // 2 elements, 0x10
bytes (sizeof)
{
/*0x070*/ VOID* SectionPointer;
/*0x078*/ ULONG32 CheckSum;
/*0x07c*/ UINT8 _PADDING1_[0x4];
};
};
union // 2 elements, 0x8
bytes (sizeof)
{
/*0x080*/ ULONG32 TimeDateStamp;
/*0x080*/ VOID* LoadedImports;
};
/*0x088*/ struct _ACTIVATION_CONTEXT* EntryPointActivationContext;
/*0x090*/ VOID* PatchInformation;
/*0x098*/ struct _LIST_ENTRY ForwarderLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x0A8*/ struct _LIST_ENTRY ServiceTagLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x0B8*/ struct _LIST_ENTRY StaticLinks; // 2
elements, 0x10 bytes (sizeof)
/*0x0c8*/ VOID* ContextInformation;
/*0x0d0*/ UINT64 OriginalBase;
/*0x0d8*/ union _LARGE_INTEGER LoadTime; // 4
elements, 0x8 bytes (sizeof)
}LDR_DATA, *PLDR_DATA;

typedef struct _OBJECT_TYPE_INITIALIZER

// 25 elements, 0x70 bytes (sizeof)
{
/*0x000*/ UINT16 Length;

```

```

union

    // 2 elements, 0x1 bytes (sizeof)
{
    /*0x002*/      UINT8      ObjectTypeFlags;
    struct

        // 7 elements, 0x1 bytes (sizeof)
        {
            /*0x002*/      UINT8      CaseInsensitive : 1;

            // 0 BitPosition
            /*0x002*/      UINT8      UnnamedObjectsOnly : 1;

            // 1 BitPosition
            /*0x002*/      UINT8      UseDefaultObject : 1;

            // 2 BitPosition
            /*0x002*/      UINT8      SecurityRequired : 1;

            // 3 BitPosition
            /*0x002*/      UINT8      MaintainHandleCount : 1;

            // 4 BitPosition
            /*0x002*/      UINT8      MaintainTypeList : 1;

            // 5 BitPosition
            /*0x002*/      UINT8      SupportsObjectCallbacks : 1;

            // 6 BitPosition
        };
};
/*0x004*/      ULONG32      ObjectTypeCode;
/*0x008*/      ULONG32      InvalidAttributes;
/*0x00C*/      struct _GENERIC_MAPPING GenericMapping;

    // 4 elements, 0x10 bytes (sizeof)
/*0x01C*/      ULONG32      ValidAccessMask;
/*0x020*/      ULONG32      RetainAccess;
/*0x024*/      enum _POOL_TYPE PoolType;
/*0x028*/      ULONG32      DefaultPagedPoolCharge;
/*0x02C*/      ULONG32      DefaultNonPagedPoolCharge;
/*0x030*/      PVOID DumpProcedure;
/*0x038*/      PVOID OpenProcedure;
/*0x040*/      PVOID CloseProcedure;
/*0x048*/      PVOID DeleteProcedure;
/*0x050*/      PVOID ParseProcedure;
/*0x058*/      PVOID SecurityProcedure;
/*0x060*/      PVOID QueryNameProcedure;
/*0x068*/      PVOID OkayToCloseProcedure;
}OBJECT_TYPE_INITIALIZER, *POBJECT_TYPE_INITIALIZER;

typedef struct _EX_PUSH_LOCK // 7 elements, 0x8 bytes (sizeof)
{
    union // 3 elements, 0x8 bytes (sizeof)

```

```

{
    struct // 5 elements, 0x8 bytes (sizeof)
    {
        /*0x000*/          UINT64      Locked : 1;          // 0
    BitPosition
        /*0x000*/          UINT64      waiting : 1;          // 1
    BitPosition
        /*0x000*/          UINT64      waking : 1;           // 2
    BitPosition
        /*0x000*/          UINT64      Multipleshared : 1;    // 3
    BitPosition
        /*0x000*/          UINT64      shared : 60;           // 4
    BitPosition
    };
    /*0x000*/          UINT64      value;
    /*0x000*/          VOID*      Ptr;
};
}EX_PUSH_LOCK, *PEX_PUSH_LOCK;

typedef struct _MY_OBJECT_TYPE // 12 elements, 0xD0 bytes
(sizeof)
{
    /*0x000*/          struct _LIST_ENTRY TypeList;          // 2 elements, 0x10
bytes (sizeof)
    /*0x010*/          struct _UNICODE_STRING Name;          // 3 elements, 0x10
bytes (sizeof)
    /*0x020*/          VOID*      DefaultObject;
    /*0x028*/          UINT8      Index;
    /*0x029*/          UINT8      _PADDING0_[0x3];
    /*0x02C*/          ULONG32    TotalNumberOfObjects;
    /*0x030*/          ULONG32    TotalNumberOfHandles;
    /*0x034*/          ULONG32    HighWaterNumberOfObjects;
    /*0x038*/          ULONG32    HighWaterNumberOfHandles;
    /*0x03C*/          UINT8      _PADDING1_[0x4];
    /*0x040*/          struct _OBJECT_TYPE_INITIALIZER TypeInfo; // 25 elements, 0x70
bytes (sizeof)
    /*0x0B0*/          struct _EX_PUSH_LOCK TypeLock;        // 7 elements, 0x8
bytes (sizeof)
    /*0x0B8*/          ULONG32    Key;
    /*0x0BC*/          UINT8      _PADDING2_[0x4];
    /*0x0C0*/          struct _LIST_ENTRY CallbackList;       // 2 elements, 0x10
bytes (sizeof)
}MY_OBJECT_TYPE, *PMY_OBJECT_TYPE;

```

对于开启了 `TypeInfo.SupportsObjectCallbacks` 属性的驱动来说自然就支持文件路径转换，当系统中有文件被加载则自动执行 `LySharkFileObjectpreCall` 回调事件，过滤掉无效路径后即可直接输出，完整代码如下所示；

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include "lyshark.h"

```

```

PVOID obHandle;
DRIVER_INITIALIZE DriverEntry;

// 文件回调
OB_PREOP_CALLBACK_STATUS LySharkFileObjectpreCall(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION OperationInformation)
{
    UNICODE_STRING DosName;
    PFILE_OBJECT fileo = OperationInformation->Object;
    HANDLE CurrentProcessId = PsGetCurrentProcessId();
    UNREFERENCED_PARAMETER(RegistrationContext);

    if (OperationInformation->ObjectType != *IoFileObjectType)
    {
        return OB_PREOP_SUCCESS;
    }

    // 过滤无效指针
    if (fileo->FileName.Buffer == NULL ||
        !MmIsValidAddress(fileo->FileName.Buffer) ||
        fileo->DeviceObject == NULL ||
        !MmIsValidAddress(fileo->DeviceObject))
    {
        return OB_PREOP_SUCCESS;
    }

    // 过滤无效路径
    if (!_wcsicmp(fileo->FileName.Buffer, L"\\Endpoint") ||
        !_wcsicmp(fileo->FileName.Buffer, L"?") ||
        !_wcsicmp(fileo->FileName.Buffer, L"\\.\\.\\.") ||
        !_wcsicmp(fileo->FileName.Buffer, L"\\\"))
    {
        return OB_PREOP_SUCCESS;
    }

    // 将对象转为DOS路径
    RtlVolumeDeviceToDosName(fileo->DeviceObject, &DosName);
    DbgPrint("[LyShark] 进程PID = %ld | 文件路径 = %wZ%wZ \n",
        (ULONG64)CurrentProcessId, &DosName, &fileo->FileName);

    return OB_PREOP_SUCCESS;
}

VOID EnableObType(POBJECT_TYPE ObjectType)
{
    PMY_OBJECT_TYPE myobtype = (PMY_OBJECT_TYPE)ObjectType;
    myobtype->TypeInfo.SupportsObjectCallbacks = 1;
}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    UNREFERENCED_PARAMETER(driver);
    ObUnregisterCallbacks(obHandle);
}

```

```

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    NTSTATUS status = STATUS_SUCCESS;
    PLDR_DATA ldr;

    DbgPrint("hello lyshark.com \n");

    OB_CALLBACK_REGISTRATION obRegFileCallback;
    OB_OPERATION_REGISTRATION opRegFileCallback;

    // enable IoFileObjectType
    EnableObType(*IoFileObjectType);

    // bypass MmVerifyCallbackFunction
    ldr = (PLDR_DATA)Driver->DriverSection;
    ldr->Flags |= 0x20;

    // 初始化回调
    memset(&obRegFileCallback, 0, sizeof(obRegFileCallback));
    obRegFileCallback.Version = ObGetFilterVersion();
    obRegFileCallback.OperationRegistrationCount = 1;
    obRegFileCallback.RegistrationContext = NULL;
    RtlInitUnicodeString(&obRegFileCallback.Altitude, L"321000");
    obRegFileCallback.OperationRegistration = &opRegFileCallback;

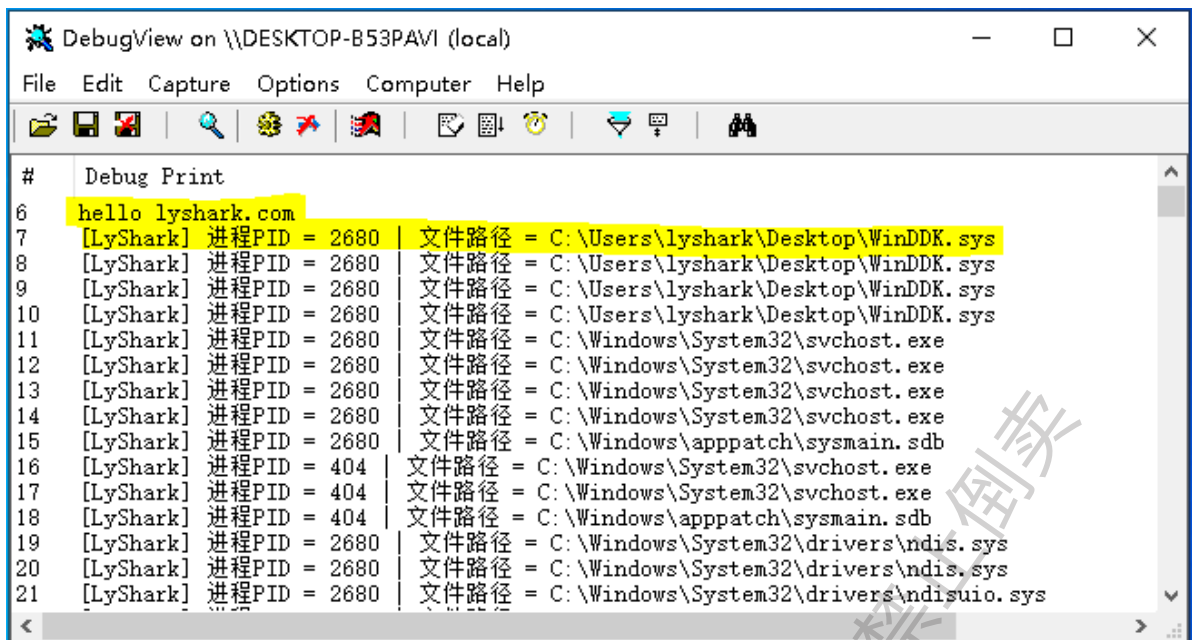
    memset(&opRegFileCallback, 0, sizeof(opRegFileCallback));
    opRegFileCallback.ObjectType = IoFileObjectType;
    opRegFileCallback.Operations = OB_OPERATION_HANDLE_CREATE |
    OB_OPERATION_HANDLE_DUPLICATE;
    opRegFileCallback.PreOperation =
    (POB_PRE_OPERATION_CALLBACK)&LySharkFileObjectpreCall;

    status = ObRegisterCallbacks(&obRegFileCallback, &obHandle);
    if (!NT_SUCCESS(status))
    {
        DbgPrint("注册回调错误 \n");
        status = STATUS_UNSUCCESSFUL;
    }

    UNREFERENCED_PARAMETER(RegistryPath);
    Driver->DriverUnload = &UnDriver;
    return status;
}

```

运行这个驱动程序，当系统中有新文件被加载时则自动输出该文件所属进程PID以及该文件的详细路径。



```
# Debug Print
6 hello lyshark.com
7 [LyShark] 进程PID = 2680 | 文件路径 = C:\Users\lyshark\Desktop\WinDDK.sys
8 [LyShark] 进程PID = 2680 | 文件路径 = C:\Users\lyshark\Desktop\WinDDK.sys
9 [LyShark] 进程PID = 2680 | 文件路径 = C:\Users\lyshark\Desktop\WinDDK.sys
10 [LyShark] 进程PID = 2680 | 文件路径 = C:\Users\lyshark\Desktop\WinDDK.sys
11 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\svchost.exe
12 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\svchost.exe
13 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\svchost.exe
14 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\svchost.exe
15 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\apppatch\sysmain.sdb
16 [LyShark] 进程PID = 404 | 文件路径 = C:\Windows\System32\svchost.exe
17 [LyShark] 进程PID = 404 | 文件路径 = C:\Windows\System32\svchost.exe
18 [LyShark] 进程PID = 404 | 文件路径 = C:\Windows\apppatch\sysmain.sdb
19 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\drivers\ndis.sys
20 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\drivers\ndis.sys
21 [LyShark] 进程PID = 2680 | 文件路径 = C:\Windows\System32\drivers\ndisuio.sys
```

至于如何阻止打开一个文件其实与《驱动开发：内核注册并监控对象回调》文章中使用的方法是一致的，首先判断 `OperationInformation->Operation` 是不是 `OB_OPERATION_HANDLE_CREATE` 或

`OB_OPERATION_HANDLE_DUPLICATE` 如果是，则直接设置 `Parameters-`

`>CreateHandleInformation.DesiredAccess` 为0直接拒绝加载。

```
// 文件回调
OB_PREOP_CALLBACK_STATUS LySharkFileObjectpreCall(PVOID RegistrationContext,
POB_PRE_OPERATION_INFORMATION OperationInformation)
{
    UNICODE_STRING DosName;
    PFILE_OBJECT fileo = OperationInformation->Object;
    HANDLE CurrentProcessId = PsGetCurrentProcessId();
    UNREFERENCED_PARAMETER(RegistrationContext);

    if (OperationInformation->ObjectType != *IoFileObjectType)
    {
        return OB_PREOP_SUCCESS;
    }

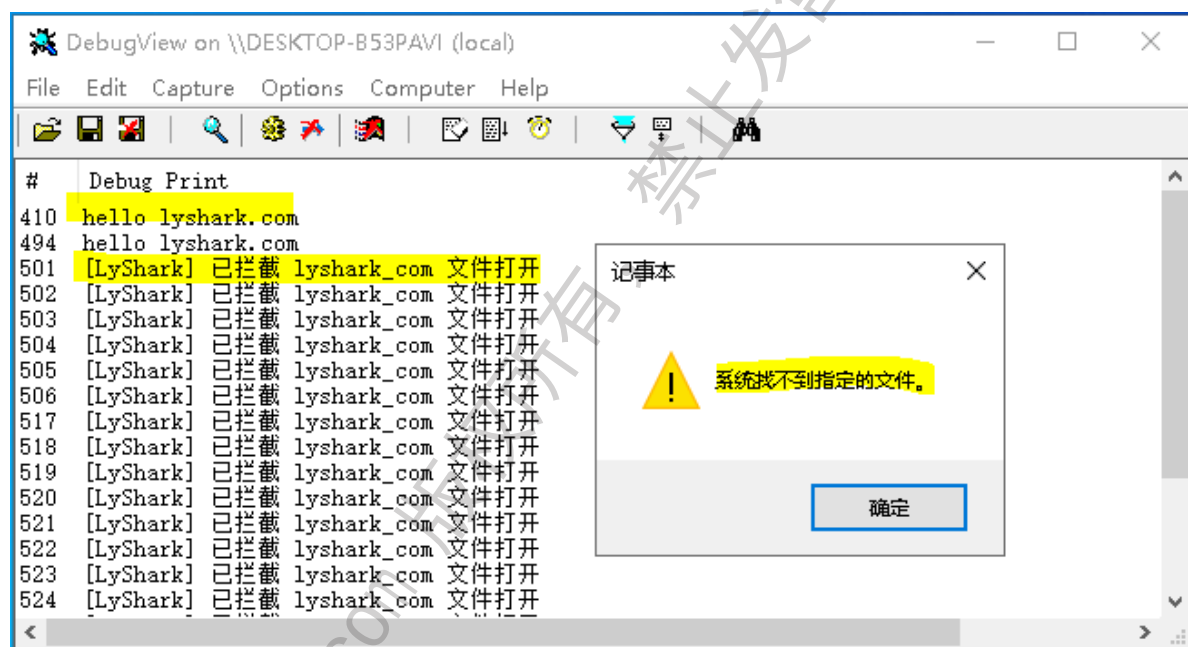
    // 过滤无效指针
    if (fileo->FileName.Buffer == NULL ||
        !MmIsValidAddress(fileo->FileName.Buffer) ||
        fileo->DeviceObject == NULL ||
        !MmIsValidAddress(fileo->DeviceObject))
    {
        return OB_PREOP_SUCCESS;
    }

    // 过滤无效路径
    if (!_wcsicmp(fileo->FileName.Buffer, L"\\Endpoint") ||
        !_wcsicmp(fileo->FileName.Buffer, L"?") ||
        !_wcsicmp(fileo->FileName.Buffer, L"\\.\\.\\.") ||
        !_wcsicmp(fileo->FileName.Buffer, L"\\\"))
    {
        return OB_PREOP_SUCCESS;
    }
}
```

```
// 阻止打开lyshark_com.txt文本
if (wcsstr(_wcslwr(fileo->FileName.Buffer), L"lyshark_com.txt"))
{
    if (OperationInformation->Operation == OB_OPERATION_HANDLE_CREATE)
    {
        OperationInformation->Parameters->CreateHandleInformation.DesiredAccess = 0;
    }
    if (OperationInformation->Operation == OB_OPERATION_HANDLE_DUPLICATE)
    {
        OperationInformation->Parameters->DuplicateHandleInformation.DesiredAccess = 0;
    }
    DbgPrint("[LyShark] 已拦截 lyshark_com 文件打开 \n");
}

return OB_PREOP_SUCCESS;
}
```

运行修改后的驱动程序，然后尝试打开 lyshark_com.txt 则会提示系统找不到指定文件。



第十章：其他篇

10.1 内核封装WSK网络通信接口

本章 LyShark 将带大家学习如何在内核中使用标准的 socket 套接字通信接口，我们都知道 windows 应用层下可直接调用 winSocket 来实现网络通信，但在内核模式下应用层API接口无法使用，内核模式下有一套专有的 WSK 通信接口，我们对 WSK 进行封装，让其与应用层调用规范保持一致，并实现内核与内核直接通过 socket 通信的案例。

当然在早期如果实现网络通信一般都会采用 TDI 框架，但在新版本 windows10 系统上虽然依然可以使用 TDI 接口，但是 LyShark 并不推荐使用，因为微软已经对接口搁置了，为了使 WSK 通信更加易用，我们需要封装内核层中的通信API，新建 LySocket.hpp 头文件，该文件中封装了 WSK 通信API接口，其封装格式与应用层接口保持了高度一致，当需要在内核中使用Socket通信时可直接引入本文件。

我们需要使用 WDM 驱动程序，并配置以下参数。

- 配置属性 -> 连接器 -> 输入 -> 附加依赖 -> \$(DDK_LIB_PATH)\Netio.lib
- 配置属性 -> C/C++ -> 常规 -> 设置 警告等级2级 (警告视为错误关闭)

配置好以后，我们就开始吧，先来看看服务端如何实现！

对于 **服务端** 来说，驱动通信必须保证服务端开启多线程来处理异步请求，不然驱动加载后系统会处于等待状态，而一旦等待则系统将会卡死，那么对于服务端 **DriverEntry** 入口说我们不能让其等待，必须使用 **PsCreateSystemThread** 来启用系统线程，该函数属于WDM的一部分，官方定义如下：

```
NTSTATUS PsCreateSystemThread(
    [out]          PHANDLE          ThreadHandle,
    [in]           ULONG            DesiredAccess,
    [in, optional] POBJECT_ATTRIBUTES ObjectAttributes,
    [in, optional] HANDLE          ProcessHandle,
    [out, optional] PCLIENT_ID      ClientId,
    [in]           PKSTART_ROUTINE StartRoutine,
    [in, optional] PVOID           StartContext
);
```

我们使用 **PsCreateSystemThread** 函数开辟线程 **TcpListenWorker** 在线程内部执行如下流程启动驱动服务端，由于我们自己封装实现了标准接口组，所以使用起来几乎与应用层无任何差异了。

- CreateSocket 创建套接字
- Bind 绑定套接字
- Accept 等待接收请求
- Receive 用于接收返回值
- Send 用于发送返回值

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include "LySocket.hpp"

PETHREAD m_EThread = NULL;

// 线程函数
// PowerBy: LyShark
VOID TcpListenWorker(PVOID Context)
{
    WSK_SOCKET* paccept_socket = NULL;
    SOCKADDR_IN LocalAddress = { 0 };
    SOCKADDR_IN RemoteAddress = { 0 };
    NTSTATUS status = STATUS_UNSUCCESSFUL;

    // 创建套接字
    PWSK_SOCKET TcpSocket = CreateSocket(AF_INET, SOCK_STREAM, IPPROTO_TCP,
    WSK_FLAG_LISTEN_SOCKET);
    if (TcpSocket == NULL)
    {
        return;
    }
}
```

```

// 设置绑定地址
LocalAddress.sin_family = AF_INET;
LocalAddress.sin_addr.s_addr = INADDR_ANY;
LocalAddress.sin_port = HTON_SHORT(8888);

status = Bind(TcpSocket, (PSOCKADDR)&LocalAddress);
if (!NT_SUCCESS(status))
{
    return;
}

// 循环接收
while (1)
{
    CHAR* read_buffer = (CHAR*)ExAllocatePoolWithTag(NonPagedPool, 2048,
"read");
    paccept_socket = Accept(TcpSocket, (PSOCKADDR)&LocalAddress,
(PSOCKADDR)&RemoteAddress);
    if (paccept_socket == NULL)
    {
        continue;
    }

    // 接收数据
    memset(read_buffer, 0, 2048);
    int read_len = Receive(paccept_socket, read_buffer, 2048, 0);
    if (read_len != 0)
    {
        DbgPrint("[内核A] => %s \n", read_buffer);

        // 发送数据
        char send_buffer[2048] = "Hi, lyshark.com B";
        Send(paccept_socket, send_buffer, strlen(send_buffer), 0);

        // 接收确认包
        memset(read_buffer, 0, 2048);
        Receive(paccept_socket, read_buffer, 2, 0);
    }

    // 清理堆
    if (read_buffer != NULL)
    {
        ExFreePool(read_buffer);
    }

    // 关闭当前套接字
    if (paccept_socket)
    {
        CloseSocket(paccept_socket);
    }
}

if (TcpSocket)
{
    CloseSocket(TcpSocket);
}

```

```

    }
    PsTerminateSystemThread(STATUS_SUCCESS);
    return;
}

// 关闭套接字
VOID UnDriver(PDRIVER_OBJECT driver)
{
    WSKCleanup();
    KeWaitForSingleObject(m_EThread, Executive, KernelMode, FALSE, NULL);
    if (m_EThread != NULL)
    {
        ObDereferenceObject(m_EThread);
    }
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    // 初始化
    WSKStartup();

    HANDLE hThread = NULL;
    NTSTATUS status = STATUS_UNSUCCESSFUL;

    // 创建系统线程
    status = PsCreateSystemThread(&hThread, THREAD_ALL_ACCESS, NULL, NULL, NULL,
    TcpListenWorker, NULL);
    if (!NT_SUCCESS(status))
    {
        return status;
    }

    // 获取线程EProcess结构
    status = ObReferenceObjectByHandle(hThread, THREAD_ALL_ACCESS, NULL,
    KernelMode, (PVOID*)&m_EThread, NULL);
    if (NT_SUCCESS(status) == FALSE)
    {
        return status;
    }

    ZwClose(hThread);
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

对于客户端来说，只需要创建套接字并连接到指定地址即可，这个过程大体上可以总结为如下；

- CreateSocket 创建套接字
- Bind 绑定套接字
- Connect 链接服务端驱动
- Send 发送数据到服务端
- Receive 接收数据到服务端

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include "LySocket.hpp"

VOID UnDriver(PDRIVER_OBJECT driver)
{
    // 卸载并关闭Socket库
    WSKCleanup();
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");
    // 初始化
    WSKStartup();

    NTSTATUS status = STATUS_SUCCESS;
    SOCKADDR_IN LocalAddress = { 0, };
    SOCKADDR_IN RemoteAddress = { 0, };

    // 创建套接字
    PWSK_SOCKET TcpSocket = CreateSocket(AF_INET, SOCK_STREAM, IPPROTO_TCP,
    WSK_FLAG_CONNECTION_SOCKET);
    if (TcpSocket == NULL)
    {
        Driver->DriverUnload = UnDriver;
        return STATUS_SUCCESS;
    }

    LocalAddress.sin_family = AF_INET;
    LocalAddress.sin_addr.s_addr = INADDR_ANY;
    status = Bind(TcpSocket, (PSOCKADDR)&LocalAddress);

    // 绑定失败则关闭驱动
    if (!NT_SUCCESS(status))
    {
        CloseSocket(TcpSocket);

        Driver->DriverUnload = UnDriver;
        return STATUS_SUCCESS;
    }

    // 初始化服务端地址与端口信息
    ULONG address[4] = { 127, 0, 0, 1 };

    RemoteAddress.sin_family = AF_INET;
    RemoteAddress.sin_addr.s_addr = change_uint(address[0], address[1],
    address[2], address[3]);
    RemoteAddress.sin_port = HTON_SHORT(8888);

    status = Connect(TcpSocket, (PSOCKADDR)&RemoteAddress);

```

```

// 连接服务端,如果失败则关闭驱动
if (!NT_SUCCESS(status))
{
    CloseSocket(TcpSocket);
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

// 发送数据
char send_buffer[2048] = "hello lyshark.com A";
Send(TcpSocket, send_buffer, strlen(send_buffer), 0);

// 接收数据
CHAR* read_buffer = (CHAR*)ExAllocatePoolWithTag(NonPagedPool, 2048,
"read");

memset(read_buffer, 0, 1024);
Receive(TcpSocket, read_buffer, 2048, 0);
DbgPrint("[内核B] => %s \n", read_buffer);

// 发送确认包
Send(TcpSocket, "ok", 2, 0);

// 释放内存
ExFreePool(read_buffer);
CloseSocket(TcpSocket);
Driver->DriverUnload = UnDriver;
return STATUS_SUCCESS;
}

```

编译两个驱动程序，首先运行 `server.sys` 驱动，运行后该驱动会在后台等待客户端连接，接着运行 `client.sys` 屏幕上可输出如下提示，说明通信已经建立了。

The screenshot shows the DebugView application window. The title bar reads 'DebugView on \\DESKTOP-B53PAV4 (local)'. The menu bar includes 'File', 'Edit', 'Capture', 'Options', 'Computer', and 'Help'. The toolbar contains various icons for file operations and debugging. The main pane displays a table of log entries:

#	Time	Debug Print
23	9.79544926	hello lyshark.com
46	24.02797127	hello lyshark.com
47	24.02863693	[内核A] => hello lyshark.com A
48	24.02865601	[内核B] => Hi, lyshark.com B

10.2 内核封装TDI网络通信接口

在上一篇文章《驱动开发：内核封装WSK网络通信接口》中，Lyshark 已经带大家看过了如何通过WSK接口实现套接字通信，但WSK实现的通信是内核与内核模块之间的，而如果需要内核与应用层之间通信则使用TDK会更好一些因为它更接近应用层，本章将使用TDK实现，TDI全称传输驱动接口，其主要负责连接 socket 和协议驱动，用于实现访问传输层的功能，该接口比 NDIS 更接近于应用层，在早期Win系统

中常用于实现过滤防火墙，同样经过封装后也可实现通信功能，本章将运用TDI接口实现驱动与应用层之间传输字符串，结构体，多线程收发等技术。

- TDI传输字符串
- TDI多线程收发
- TDI传数结构实现认证

TDI 传输字符串: 服务端在应用层侦听，客户端是驱动程序，驱动程序加载后自动连接应用层并发送消息。

首先来看 应用层(服务端) 代码，具体我就不说了，来看教程的都是有基础的。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")
#define PORT 8888

int main(int argc, char *argv[])
{
    printf("hello lyshark.com \n");
    WSADATA WSAData;
    SOCKET sock, msgsock;
    struct sockaddr_in ServerAddr;

    if (WSAStartup(MAKEWORD(2, 0), &WSAData) != SOCKET_ERROR)
    {
        ServerAddr.sin_family = AF_INET;
        ServerAddr.sin_port = htons(PORT);
        ServerAddr.sin_addr.s_addr = INADDR_ANY;

        sock = socket(AF_INET, SOCK_STREAM, 0);
        int BindRet = bind(sock, (LPSOCKADDR)&ServerAddr, sizeof(ServerAddr));
        int LinsRet = listen(sock, 10);
    }

    while (1)
    {
        char buf[1024] = { 0 };
        msgsock = accept(sock, (LPSOCKADDR)0, (int *)0);
        memset(buf, 0, sizeof(buf));

        recv(msgsock, buf, 1024, 0);
        printf("内核返回: %s \n", buf);

        char send_buffer[1024] = { 0 };
        memset(send_buffer, 0, 1024);
        strcpy(send_buffer, "Hi,RO !");
        send(msgsock, send_buffer, strlen(send_buffer), 0);
        closesocket(msgsock);
    }
}
```



```

}
closesocket(sock);
WSACleanup();
return 0;
}

```

再来是驱动层代码，如下所示；

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include "MyTDI.hpp"

// 发送接收数据
NTSTATUS SendOnRecv()
{
    NTSTATUS status = STATUS_SUCCESS;
    HANDLE hTdiAddress = NULL;
    HANDLE hTdiEndPoint = NULL;
    PDEVICE_OBJECT pTdiAddressDevObj = NULL;
    PFILE_OBJECT pTdiEndPointFileObject = NULL;
    LONG pServerIp[4] = { 127, 0, 0, 1 };
    LONG lServerPort = 8888;
    UCHAR szSendData[] = "hello lyshark";
    ULONG ulSendDataLength = 1 + strlen(szSendData);
    HANDLE hThread = NULL;

    // TDI初始化
    status = TdiOpen(&pTdiAddressDevObj, &pTdiEndPointFileObject, &hTdiAddress,
&hTdiEndPoint);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }

    // TDI TCP连接服务器
    status = TdiConnection(pTdiAddressDevObj, pTdiEndPointFileObject, pServerIp,
lServerPort);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }

    // TDI TCP发送信息
    status = TdiSend(pTdiAddressDevObj, pTdiEndPointFileObject, szSendData,
ulSendDataLength);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }
    DbgPrint("发送: %s\n", szSendData);

    // 创建接收信息多线程，循环接收信息

```

```

char szRecvData[1024] = { 0 };
ULONG ulRecvDataLenngth = 1024;
RtlZeroMemory(szRecvData, ulRecvDataLenngth);

// TDI TCP接收信息
do
{
    ulRecvDataLenngth = TdiRecv(pTdiAddressDevObj, pTdiEndPointFileObject,
szRecvData, ulRecvDataLenngth);
    if (0 < ulRecvDataLenngth)
    {
        DbgPrint("接收数据: %s\n", szRecvData);
        break;;
    }

} while (TRUE);

// 释放
TdiClose(pTdiEndPointFileObject, hTdiAddress, hTdiEndPoint);
return STATUS_SUCCESS;
}

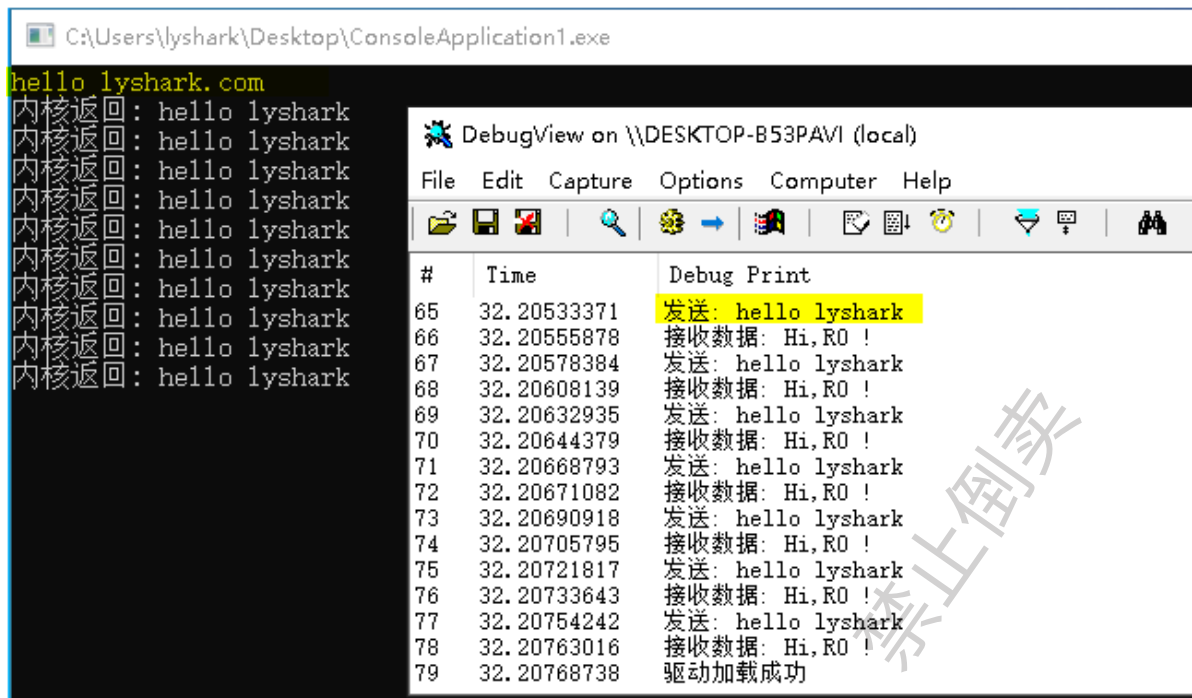
VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    for (int x = 0; x < 10; x++)
    {
        SendOnRecv();
    }

    DbgPrint("驱动加载成功 \n");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

首先运行应用层开启服务端侦听，然后运行驱动程序，会输出如下信息；



TDI 多线程收发包: 实现驱动内部发送数据包后开启一个线程用于等待应用层返回并输出结果，多线程收发在发送数据包后需要创建新的线程等待接收。

首先是服务端代码。

```
// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <winsock2.h>

#pragma comment(lib, "ws2_32.lib")
#define PORT 8888

int main(int argc, char *argv[])
{
    printf("hello lyshark.com \n");
    WSADATA WSAData;
    SOCKET sock, msgsock;
    struct sockaddr_in ServerAddr;

    if (WSAStartup(MAKEWORD(2, 0), &WSAData) != SOCKET_ERROR)
    {
        ServerAddr.sin_family = AF_INET;
        ServerAddr.sin_port = htons(PORT);
        ServerAddr.sin_addr.s_addr = INADDR_ANY;

        sock = socket(AF_INET, SOCK_STREAM, 0);
        int BindRet = bind(sock, (LPSOCKADDR)&ServerAddr, sizeof(ServerAddr));
        int LinsRet = listen(sock, 10);
    }

    while (1)
```

```

{
    char buf[1024] = { 0 };
    msgsock = accept(sock, (LPSOCKADDR)0, (int *)0);
    memset(buf, 0, sizeof(buf));

    recv(msgsock, buf, 1024, 0);
    printf("内核返回: %s \n", buf);

    char send_buffer[1024] = { 0 };
    memset(send_buffer, 0, 1024);
    strcpy(send_buffer, "Hi,R0 !");
    send(msgsock, send_buffer, strlen(send_buffer), 0);
    closesocket(msgsock);
}
closesocket(sock);
WSACleanup();
return 0;
}

```

驱动程序代码如下，RecvThreadProc 主要负责数据接收，SendThreadData 负责数据发送。

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#include "LySocket.hpp"

typedef struct _MY_DATA
{
    PDEVICE_OBJECT pTdiAddressDevObj;
    PFILE_OBJECT pTdiEndPointFileObject;
    HANDLE hTdiAddress;
    HANDLE hTdiEndPoint;
}MY_DATA, *PMY_DATA;

// 接收信息多线程
VOID RecvThreadProc(_In_ PVOID StartContext)
{
    PMY_DATA pMyData = (PMY_DATA)StartContext;
    NTSTATUS status = STATUS_SUCCESS;
    char szRecvData[1024] = { 0 };
    ULONG ulRecvDataLenngth = 1024;
    RtlZeroMemory(szRecvData, ulRecvDataLenngth);

    // TDI TCP接收信息
    do
    {
        ulRecvDataLenngth = TdiRecv(pMyData->pTdiAddressDevObj, pMyData->pTdiEndPointFileObject, szRecvData, ulRecvDataLenngth);
        if (0 < ulRecvDataLenngth)
        {
            DbgPrint("线程句柄:%x --> 接收数据包: %s\n", pMyData->hTdiEndPoint, szRecvData);
            break;;
        }
    }
}

```

```

    }

    } while (TRUE);

    // 释放
    TdiClose(pMyData->pTdiEndPointFileObject, pMyData->hTdiAddress, pMyData->hTdiEndPoint);
    ExFreePool(pMyData);
}

// 多线程发送
NTSTATUS SendThreadData()
{
    NTSTATUS status = STATUS_SUCCESS;
    HANDLE hTdiAddress = NULL;
    HANDLE hTdiEndPoint = NULL;
    PDEVICE_OBJECT pTdiAddressDevObj = NULL;
    PFILE_OBJECT pTdiEndPointFileObject = NULL;
    LONG pServerIp[4] = { 127, 0, 0, 1 };
    LONG lServerPort = 8888;
    UCHAR szSendData[] = "hello lyshark";
    ULONG ulSendDataLength = 1 + strlen(szSendData);
    HANDLE hThread = NULL;

    // TDI初始化
    status = TdiOpen(&pTdiAddressDevObj, &pTdiEndPointFileObject, &hTdiAddress, &hTdiEndPoint);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }

    // TDI TCP连接服务器
    status = TdiConnection(pTdiAddressDevObj, pTdiEndPointFileObject, pServerIp, lServerPort);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }

    // TDI TCP发送信息
    status = TdiSend(pTdiAddressDevObj, pTdiEndPointFileObject, szSendData, ulSendDataLength);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }
    DbgPrint("发送 %s\n", szSendData);

    // 创建接收信息多线程，循环接收信息
    PMY_DATA pMyData = ExAllocatePool(NonPagedPool, sizeof(MY_DATA));
    pMyData->pTdiAddressDevObj = pTdiAddressDevObj;
    pMyData->pTdiEndPointFileObject = pTdiEndPointFileObject;
    pMyData->hTdiAddress = hTdiAddress;
    pMyData->hTdiEndPoint = hTdiEndPoint;
}

```

```

        PsCreateSystemThread(&hThread, 0, NULL, NtCurrentProcess(), NULL,
        RecvThreadProc, pMyData);
    }

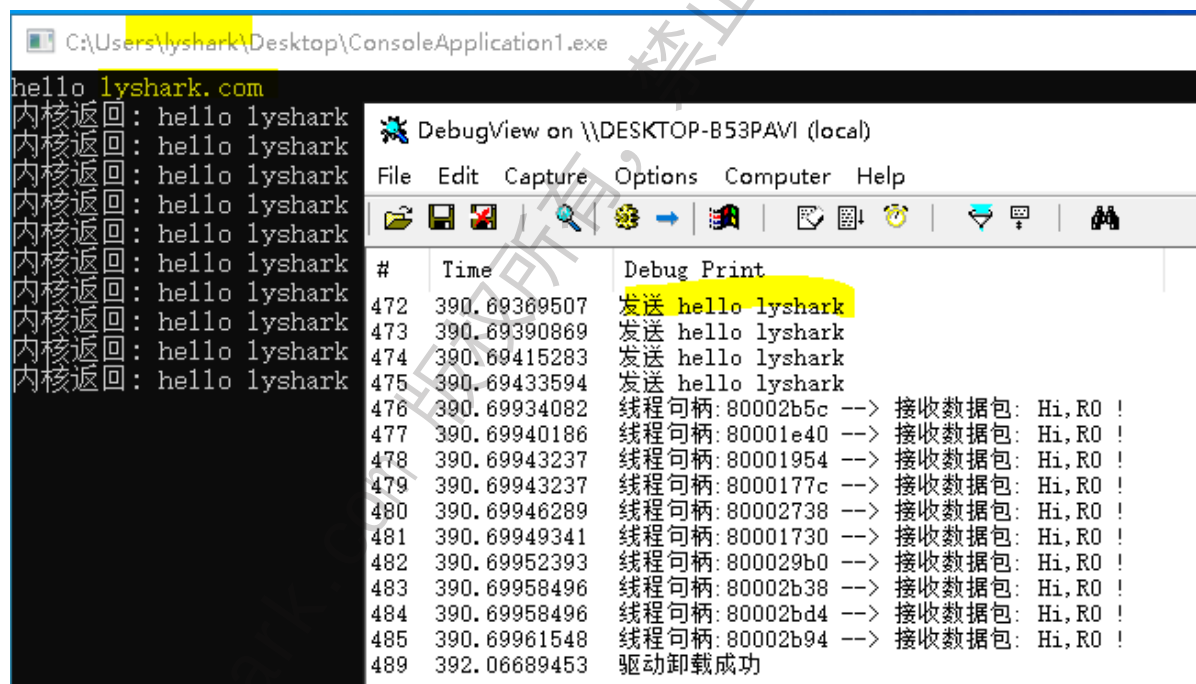
    VOID UnDriver(PDRIVER_OBJECT driver)
    {
        DbgPrint("驱动卸载成功 \n");
    }

    NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
    {
        DbgPrint("hello lyshark.com \n");

        for (int x = 0; x < 10; x++)
        {
            SendThreadData();
        }
        Driver->DriverUnload = UnDriver;
        return STATUS_SUCCESS;
    }

```

运行应用层服务端等待侦听，运行驱动程序输出如下效果；



The screenshot shows a Windows console window with the title "C:\Users\lyshark\Desktop\ConsoleApplication1.exe". The console output is as follows:

```

hello lyshark.com
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark
内核返回: hello lyshark

```

Overlaid on the console is a DebugView window titled "DebugView on \\DESKTOP-B53PAVI (local)". The window shows a list of debug print messages:

#	Time	Debug Print
472	390.69369507	发送 hello lyshark
473	390.69390869	发送 hello lyshark
474	390.69415283	发送 hello lyshark
475	390.69433594	发送 hello lyshark
476	390.69934082	线程句柄: 80002b5c --> 接收数据包: Hi, R0 !
477	390.69940186	线程句柄: 80001e40 --> 接收数据包: Hi, R0 !
478	390.69943237	线程句柄: 80001954 --> 接收数据包: Hi, R0 !
479	390.69943237	线程句柄: 8000177c --> 接收数据包: Hi, R0 !
480	390.69946289	线程句柄: 80002738 --> 接收数据包: Hi, R0 !
481	390.69949341	线程句柄: 80001730 --> 接收数据包: Hi, R0 !
482	390.69952393	线程句柄: 800029b0 --> 接收数据包: Hi, R0 !
483	390.69958496	线程句柄: 80002b38 --> 接收数据包: Hi, R0 !
484	390.69958496	线程句柄: 80002bd4 --> 接收数据包: Hi, R0 !
485	390.69961548	线程句柄: 80002b94 --> 接收数据包: Hi, R0 !
489	392.06689453	驱动卸载成功

TDI 传数结构实现认证: 驱动内部发送结构体给应用层，应用层验证结构体成员，此功能可实现对驱动程序的控制机制，例如是否允许驱动加载卸载等，通常用于驱动辅助认证。

应用层代码

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com

#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
#include <winsock2.h>

```

```

#pragma comment(lib, "ws2_32.lib")
#define PORT 8888

// 传输结构体
typedef struct
{
    int uuid;
    char username[256];
    char password[256];
}SocketData;

int main(int argc, char *argv[])
{
    printf("hello lyshark.com \n");

    WSADATA WSAData;
    SOCKET sock, msgsock;
    struct sockaddr_in ServerAddr;

    if (WSAStartup(MAKEWORD(2, 0), &WSAData) != SOCKET_ERROR)
    {
        ServerAddr.sin_family = AF_INET;
        ServerAddr.sin_port = htons(PORT);
        ServerAddr.sin_addr.s_addr = INADDR_ANY;

        sock = socket(AF_INET, SOCK_STREAM, 0);
        int BindRet = bind(sock, (LPSOCKADDR)&ServerAddr, sizeof(ServerAddr));
        int LinsRet = listen(sock, 10);
    }

    while (1)
    {
        char buf[8192] = { 0 };
        msgsock = accept(sock, (LPSOCKADDR)0, (int *)0);
        memset(buf, 0, sizeof(buf));

        // 接收返回数据
        recv(msgsock, buf, sizeof(SocketData), 0);

        // 强转结构体
        SocketData* msg = (SocketData*)buf;

        printf("UUID = %d \n", msg->uuid);
        printf("名字 = %s \n", msg->username);
        printf("密码 = %s \n", msg->password);

        // 验证通过则继续使用
        if ((strcmp(msg->username, "lyshark") == 0) && (strcmp(msg->password,
"123") == 0))
        {
            char send_buffer[8192] = { 0 };
            memset(send_buffer, 0, 8192);
            strcpy(send_buffer, "success");
            send(msgsock, send_buffer, strlen(send_buffer), 0);
        }
    }
}

```

```

        closesocket(msgsock);
    }
    // 不通过则禁止驱动加载
    else
    {
        char send_buffer[8192] = { 0 };
        memset(send_buffer, 0, 8192);
        strcpy(send_buffer, "error");
        send(msgsock, send_buffer, strlen(send_buffer), 0);
        closesocket(msgsock);
    }
}
closesocket(sock);
WSACleanup();
return 0;
}

```

驱动层代码

```

// 署名权
// right to sign one's name on a piece of work
// PowerBy: LyShark
// Email: me@lyshark.com
#include "LySocket.hpp"

// 传输结构体
typedef struct
{
    int uuid;
    char username[256];
    char password[256];
}SocketData;

// 验证账号密码是否正确
BOOLEAN CheckDriver()
{
    NTSTATUS status = STATUS_SUCCESS;
    HANDLE hTdiAddress = NULL;
    HANDLE hTdiEndPoint = NULL;
    PDEVICE_OBJECT pTdiAddressDevObj = NULL;
    PFILE_OBJECT pTdiEndPointFileObject = NULL;
    LONG pServerIp[4] = { 127, 0, 0, 1 };
    LONG lServerPort = 8888;

    // TDI初始化
    status = TdiOpen(&pTdiAddressDevObj, &pTdiEndPointFileObject, &hTdiAddress,
    &hTdiEndPoint);
    if (!NT_SUCCESS(status))
    {
        return STATUS_SUCCESS;
    }

    // TDI TCP连接服务器
    status = TdiConnection(pTdiAddressDevObj, pTdiEndPointFileObject, pServerIp,
    lServerPort);
}

```



```

if (!NT_SUCCESS(status))
{
    return STATUS_SUCCESS;
}

SocketData ptr;

RtlZeroMemory(&ptr, sizeof(SocketData));

// 填充结构
ptr.uuid = 1001;
RtlCopyMemory(ptr.username, "lyshark", strlen("xxxxxxx"));
RtlCopyMemory(ptr.password, "123123", strlen("xxxxxx"));

// TDI TCP发送信息
status = TdiSend(pTdiAddressDevObj, pTdiEndPointFileObject, &ptr,
sizeof(SocketData));
if (!NT_SUCCESS(status))
{
    return STATUS_SUCCESS;
}

// 创建接收信息多线程，循环接收信息
char szRecvData[8192] = { 0 };
ULONG ulRecvDataLenngth = 8192;
RtlZeroMemory(szRecvData, ulRecvDataLenngth);

// TDI TCP接收信息
do
{
    ulRecvDataLenngth = TdiRecv(pTdiAddressDevObj, pTdiEndPointFileObject,
szRecvData, ulRecvDataLenngth);
    if (0 < ulRecvDataLenngth)
    {
        DbgPrint("接收数据: %s\n", szRecvData);

        if (strncmp(szRecvData, "success", 7) == 0)
        {
            // 释放
            TdiClose(pTdiEndPointFileObject, hTdiAddress, hTdiEndPoint);
            return TRUE;
        }
        else if (strncmp(szRecvData, "error", 5) == 0)
        {
            // 释放
            TdiClose(pTdiEndPointFileObject, hTdiAddress, hTdiEndPoint);
            return FALSE;
        }
        break;;
    }
} while (TRUE);

// 释放
TdiClose(pTdiEndPointFileObject, hTdiAddress, hTdiEndPoint);
return STATUS_SUCCESS;

```

```

}

VOID UnDriver(PDRIVER_OBJECT driver)
{
    DbgPrint("驱动卸载成功 \n");
}

NTSTATUS DriverEntry(IN PDRIVER_OBJECT Driver, PUNICODE_STRING RegistryPath)
{
    DbgPrint("hello lyshark.com \n");

    BOOLEAN ref = CheckDriver();

    if (ref == FALSE)
    {
        DbgPrint("[LyShark.com] 驱动已过期,无法加载 \n");
        Driver->DriverUnload = UnDriver;
        return STATUS_SUCCESS;
    }

    DbgPrint("[*] 驱动正常使用 \n");
    Driver->DriverUnload = UnDriver;
    return STATUS_SUCCESS;
}

```

运行应用层服务端，并运行驱动程序，则会验证该驱动是否合法，如果合法则加载不合法则拒绝；

The screenshot shows a Windows console window with the following output:

```

C:\Users\lyshark\Desktop\ConsoleApplication1.exe
hello lyshark.com
UUID = 1001
名字 = lyshark
密码 = 123123

```

Overlaid on the console is the DebugView utility window, titled "DebugView on \\DESKTOP-B53PAV1 (local)". It displays a list of debug prints:

#	Time	Debug Print
126	82.96752167	hello lyshark.com
127	82.96826172	接收数据: error
128	82.96833038	[LyShark.com] 驱动已过期,无法加载
131	84.05626678	驱动卸载成功

参考文献

<http://www.blogfshare.com/details-in-pspcidtbale.html>

<https://blog.csdn.net/whatday/article/details/17189093>

<https://www.cnblogs.com/kuangke/p/5761615.html>

<https://www.cnblogs.com/kuangke/p/9397511.html>

<https://www.cnblogs.com/ciyze0101/p/5468175.html>

<https://blog.csdn.net/zhuhuibeishadiao/article/details/75658816>

<https://www.docin.com/p-1508418694.html>

lyshark.com 版权所有，禁止发售，禁止倒卖